

Joshua Bloch

Updated
for
Java 9



Effective Java

Third Edition

Best practices for



...the Java Platform



Java Efetiva

Terceira Edição

Esta página intencionalmente deixada em branco

Java Efetiva

Terceira Edição

Joshua Bloch

 Addison-Wesley

Boston • Columbus • Indianapolis • Nova York • São Francisco • Amsterdã • Cidade do
Cabo Dubai • Londres • Madri • Milão • Munique • Paris • Montreal • Toronto • Delhi •
Cidade do México São Paulo • Sydney • Hong Kong • Seul • Singapura • Taipei • Tóquio

Muitas das designações usadas por fabricantes e vendedores para distinguir seus produtos são reivindicadas como marcas registradas. Quando essas designações aparecem neste livro, e o editor estava ciente de uma reivindicação de marca registrada, as designações foram impressas com letras maiúsculas iniciais ou em letras maiúsculas todas as maiúsculas.

O autor e o editor tiveram cuidado na preparação deste livro, mas não oferecem nenhuma garantia expressa ou implícita de qualquer tipo e não assumem responsabilidade por erros ou omissões. Não se assume responsabilidade por danos incidentais ou consequentes relacionados ou decorrentes do uso das informações ou programas aqui contidos.

Para informações sobre a compra deste título em grandes quantidades, ou para oportunidades especiais de vendas (que podem incluir versões eletrônicas; designs personalizados de capa; e conteúdo específico para seu negócio, metas de treinamento, foco de marketing ou interesses de marca), por favor, entre em contato com nosso departamento de vendas corporativas pelo atcorpsales@pearsoned.com ou pelo telefone (800) 382-3419.

Para dúvidas sobre vendas do governo, entre em contato com governmentsales@pearsoned.com.

Para dúvidas sobre vendas fora dos EUA, entre em contato com intlcs@pearson.com.

Visite-nos na Web: informit.com/aw

Número de Controle da Biblioteca do Congresso: 2017956176

Direitos autorais © 2018 Pearson Education Inc. Partes
copyright © 2001-2008 Oracle e/ou suas afiliadas. Todos
os direitos reservados.

Todos os direitos reservados. Impresso nos Estados Unidos da América. Esta publicação é protegida por direitos autorais, e a permissão deve ser obtida do editor antes de qualquer reprodução, armazenamento em um sistema de recuperação ou transmissão proibida, de qualquer forma ou por qualquer meio, eletrônico, mecânico, fotocópia, gravação, ou igualmente. Para informações sobre permissões, formulários de solicitação e os contatos apropriados dentro do Departamento de Direitos e Permissões Globais da Pearson Education, por favor, visite www.pearsoned.com/permissions/.

ISBN-13: 978-0-13-468599-
ISBN-10: 0-13-468599-7

Para minha família: Cindy, Tim e Matt

Esta página intencionalmente deixada em branco

Conteúdo

Prefácio xi

Prefácio xiii

Agradecimentos xvii

1 Introdução 1

2 Criação e Destruição de Objetos 5

Item 1: Considere métodos de fábrica estática em vez de construtores... 5Item 2:
Considere um construtor diante de muitos construtores
parâmetros 10Item 3: Fazer cumprir a propriedade singleton com um
construtor privado
ou um tipo enum 17Item 4: Impor a não instanciabilidade com
um construtor privado 19Item 5: Preferência injeção de
dependência a recursos de cabeamento direto 20Item 6: Evite
criar objetos desnecessários 22Item 7: Eliminar referências a
objetos obsoletos 26Item 8: Evite finalizadores e limpadores . . .
. . . 29Item 9: Prefira tentar com recursos a tentar finalmente 34

3 Métodos Comuns a Todos os Objetos 37

Item 10: Obedecer ao contrato geral quando prevalecer equivale a 37Item
11: Sempre sobrescreva hashCode quando você sobrescreve é igual a . . 50Item
12: Sempre sobrepor para String 55Item 13: Anular o clone com
discernimento 58Item 14: Considerar implementar Comparáveis
66

4 Aulas e Interfaces 73

Item 15: Minimizar a acessibilidade de aulas e membros 73Item 16: Em
aulas públicas, use métodos de acessório, não campos públicos 78Item 17:
Minimize a mutabilidade 80Item 18: Composição favorável à herança . . .
87

Item 19: Projetar e documentar para herança ou, caso contrário, proibi-la	93
Item 20: Preferência interfaces a classes abstratas	99
Item 21: Projetar interfaces para a posteridade	104
Item 22: Usar interfaces apenas para definir tipos	109
Item 23: Prefira hierarquias de classes a classes marcadas	109
Item 24: Favoreça classes de membros estáticas em vez de não estáticas	112
Item 25: Limitar os arquivos-fonte a uma única classe de nível superior	115

5 Genéricos 117

Item 26: Não usar tipos brutos	117
Item 27: Eliminar avisos não verificados	123
Item 28: Prefira listas a arranjos	126
Item 29: Favoreça tipos genéricos	130
Item 30: Favor de métodos genéricos	135
Item 31: Use curingas limitados para aumentar a flexibilidade da API	139
Item 32: Combinar genéricos e variados com discernimento	146
Item 33: Considerar recipientes heterogêneos tipo seguro	151

6 Enums e Anotações 157

Item 34: Use enums em vez de constantes de inteligência	157
Item 35: Usar campos de instância em vez de ordinais	168
Item 36: Use EnumSet em vez de campos de bits	169
Item 37: Use o EnumMap em vez da indexação ordinal	171
Item 38: Emular enums extensíveis com interfaces	176
Item 39: Prefira anotações a padrões de nomeação	180
Item 40: Use consistentemente a anotação de Sobreposição	188
Item 41: Usar interfaces de marcação para definir tipos	191

7 Lambdas e Riachos 193

Item 42: Preferir lambdas a classes anônimas	193
Item 43: Preferência de referências de método a lambdas	197
Item 44: Favorecer o uso de interfaces funcionais padrão	199
Item 45: Use os fluxos de água com critério	203
Item 46: Preferência de funções sem efeitos colaterais em streams	210
Item 47: Preferência Coleta ao Fluxo como tipo de devolução	216
Item 48: Tenha cautela ao fazer fluxos paralelos	222

8 Métodos 227

Item 49: Verificar parâmetros quanto à validade 227Item 50:
Faça cópias defensivas quando necessário 231Item 51:
Assinaturas de métodos de design cuidadosas 236Item 52: Use
a sobrecarga com discernimento 238Item 53: Use varargs com
prudência 245Item 54: Devolver coleções ou arrays vazios, não
nulos 247Item 55: Devolver opcionais com critério
249Item 56: Escrever comentários de documentação para todos os
elementos da API expostos 254

9 Programação Geral 261

Item 57: Minimizar o escopo das variáveis locais 261Item 58:
Prefere loops para cada para os tradicionais para laços 264Item
59: Conhecer e usar as bibliotecas 267Item 60: Evite flutuar
e dobrar se respostas exatas forem necessárias. 270Item 61:
Preferência tipos primitivos a primitivos em caixas 273Item
62: Evite strings onde outros tipos são mais apropriados. . 276Item
63: Cuidado com o desempenho da concatenação de strings
279Item 64: Referir-se a objetos por suas interfaces 280Item
65: Prefira interfaces à reflexão 282Item 66: Use métodos
nativos com discernimento. 285Item 67: Otimizar com critério .
. 286Item 68: Seguir as convenções de nomenclatura geralmente
aceitas 289

10 Exceções 293

Item 69: Usar exceções apenas para condições excepcionais... . 293Item 70:
Usar exceções marcadas para condições recuperáveis e
exceções em tempo de execução para erros de programação
296Item 71: Evitar o uso desnecessário de exceções verificadas
. 298Item 72: Favoreço o uso de exceções padrão. 300Item 73:
Incluir exceções apropriadas à abstração. 302Item 74:
Documente todas as exceções lançadas por cada método.
304Item 75: Inclua informações de captura de falhas em mensagens
detalhadas. . 306Item 76: Buscar a atomicidade da falha
308Item 77: Não ignore exceções 310

11 Concorrência 311

Item 78: Sincronizar o acesso a dados mutáveis compartilhados 311
 Item 79: Evitar sincronização excessiva 317
 Item 80: Prefira executores, tarefas e fluxos a threads 323
 Item 81: Prefira que as utilidades de concorrência aguardem e notifiquem 325
 Item 82: Segurança do fio do documento 330
 Item 83: Use uma inicialização preguiçosa com critério 333
 Item 84: Não dependa do agendador de threads 336

12 Serialização 339

Item 85: Preferências alternativas à serialização em Java 339
 Item 86: Implementar o serializável com grande cautela... . 343
 Item 87: Considere usar um formulário serializado personalizado 346
 Item 88: Escrever métodos readObject defensivamente 353
 Item 89: Por exemplo, controle, prefira tipos enum para lerResolva 359
 Item 90: Considere proxies de serialização em vez de serializados

Instâncias 363

Itens correspondentes à Segunda Edição 367

Referências. 371

Índice 377

Prefácio

SE um colega dissesse para você: "Meu cônjuge esta noite hoje fabrica o

Refeição incomum em uma casa. Você vai participar?" três coisas provavelmente passariam pela sua mente: terceiro, que você foi convidado para jantar; segundo, que o inglês não era a primeira língua do seu colega; E primeiro, bastante confusão.

Se você já estudou uma segunda língua e depois tentou usá-la fora da sala de aula, sabe que há três coisas que você deve dominar: como a língua é estruturada (gramática), como nomear as coisas sobre as quais você quer falar (vocabulário) e as formas habituais e eficazes de dizer coisas do dia a dia (uso). Muitas vezes, só os dois primeiros são abordados na sala de aula, e você encontra falantes nativos constantemente reprimindo o riso enquanto tenta se fazer entender.

É muito parecido com uma linguagem de programação. Você precisa entender a linguagem principal: ela é algorítmica, funcional, orientada a objetos? Você precisa conhecer o vocabulário: quais estruturas de dados, operações e recursos são oferecidos pelas bibliotecas padrão? E você precisa estar familiarizado com as formas usuais e eficazes de estruturar seu código. Livros sobre linguagens de programação geralmente cobrem apenas as duas primeiras ou discutem o uso de forma esporádica. Talvez seja porque os dois primeiros são, de certa forma, mais fáceis de escrever sobre eles. Gramática e vocabulário são laços próprios da língua, mas o uso é característico de uma comunidade que a utiliza.

A linguagem de programação Java, por exemplo, é orientada a objetos com herança simples e suporta um estilo de codificação imperativo (orientado a sentenças) dentro de cada método. As bibliotecas abordam suporte a exibição gráfica, rede, computação distribuída e segurança. Mas como a linguagem é melhor utilizada na prática?

Há outro ponto. Programas, ao contrário das frases faladas e da maioria dos livros e revistas, provavelmente serão alterados ao longo do tempo. Normalmente não basta produzir um código que funcione de forma eficaz e seja facilmente compreendido por outros filhos; Também é necessário organizar o código para que seja fácil de modificar. Pode haver várias maneiras de escrever código para alguma tarefa T. Dessas dez formas, sete serão desajeitadas, ineficientes ou confusas. Dos outros três, qual provavelmente será mais parecido com o código necessário para a tarefa T' no lançamento do software do próximo ano?

Existem inúmeros livros dos quais você pode aprender a gramática da linguagem de programação Java, incluindo *The™ Java Programming Language* de Arnold, Gosling e Holmes, ou *The™ Java Language Specification* de Gosling, Joy, yourself e Bracha. Da mesma forma, existem dezenas de livros sobre bibliotecas e APIs associadas à linguagem de programação Java.

Este livro atende à sua terceira necessidade: uso tradicional e eficaz. Joshua Bloch passou anos expandindo, implementando e utilizando a linguagem de programação Java na Sun Microsystems; Ele também leu muitos códigos de outras pessoas, inclusive o meu. Aqui ele oferece bons conselhos, organizados de forma sistemática, sobre como estruturar seu código para que funcione bem, para que outras pessoas possam entendê-lo, para que futuras modificações e melhorias sejam menos propensas a causar dores de cabeça — talvez, até, para que seus programas sejam agradáveis, elegantes e graciosos.

Guy L. Steele Jr.
Burlington,
Massachusetts Abril de 2001

Prefácio

Prefácio da Terceira Edição

Em 1997, quando Java era nova, James Gosling (o pai de Java) a descreveu como um

"linguagem de colarinho azul" que era "bem simples" [Gosling97]. Mais ou menos na mesma época, Bjarne Stroustrup (o pai do C++) descreveu o C++ como uma "linguagem multiparadigma" que "difere deliberadamente de linguagens projetadas para suportar uma única forma de escrever programas" [Stroustrup95]. Stroustrup alertou:

Grande parte da relativa simplicidade do Java é—como na maioria das linguagens novas—em parte uma ilusão e em parte uma função de sua incompletude. Com o passar do tempo, o Java crescerá significativamente em tamanho e complexidade. Ele dobrará ou triplicará o tamanho e aumentará extensões ou bibliotecas dependentes da implementação. [Stroustrup] Agora, vinte anos depois, é justo dizer que Gosling e Stroustrup estavam certos. O Java agora é grande e complexo, com múltiplas abstrações para muitas coisas, desde execução paralela, iteração, até a representação de datas e horários.

Ainda gosto de Java, embora meu entusiasmo tenha diminuído um pouco conforme a plataforma cresceu. Dado seu tamanho e complexidade crescentes, a necessidade de um guia de melhores práticas atualizado é ainda mais crítica. Com esta terceira edição do *Effective Java*, fiz o meu melhor para te oferecer uma. Espero que esta edição continue a satisfazer essa necessidade, mantendo-se fiel ao espírito das duas primeiras edições.

Pequeno é bonito, mas simples não é fácil.

*San Jose,
Califórnia* Novembro de 2017

P.S. Seria negligente não mencionar uma boa prática do setor que tem ocupado boa parte do meu tempo ultimamente. Desde o nascimento do nosso campo nos anos 1950, reimplementamos livremente as APIs uns dos outros. Essa prática foi fundamental para o sucesso meteórico da tecnologia de computadores. Sou ativo no esforço para preservar essa liberdade [CompSci17], e incentivo você a se juntar a mim. É crucial para a saúde contínua da nossa profissão que mantemos o direito de reimplementar as APIs uns dos outros.

Prefácio da Segunda Edição

Muita coisa aconteceu com a plataforma Java desde que escrevi a primeira edição deste livro em 2001, e já passou da hora de uma segunda edição. O conjunto mais significativo de mudanças foi a adição de genéricos, tipos de enum, anotações, autoboxing e o for-eachloop no Java 5. Um segundo lugar próximo foi a adição da nova biblioteca de concorrência, `java.util.concurrent`, também lançada em Java 5. Com Gilad Bracha, tive a sorte de liderar as equipes que projetaram os novos recursos de linguagem. Também tive a sorte de fazer parte da equipe que projetou e desenvolveu a biblioteca de concorrência, liderada por Doug Lea.

A outra grande mudança na plataforma é a ampla adoção de modernos Ambientes Integrados de Desenvolvimento (IDEs), como Eclipse, IntelliJ IDEA e NetBeans, e de ferramentas de análise estática, como FindBugs. Embora eu não tenha participado desses esforços, me beneficieei imensamente deles e aprendi como eles afetam a experiência de desenvolvimento em Java.

Em 2004, mudei do Sun para o Google, mas continuei envolvido no desenvolvimento da plataforma Java nos últimos quatro anos, contribuindo para as APIs de concorrência e coleções por meio do Google e do JavaCommunity Process. Também tive o prazer de usar a plataforma Java para desenvolver bibliotecas para uso dentro do Google. Agora eu sei como é ser usuário.

Como foi o caso em 2001, quando escrevi a primeira edição, meu objetivo principal é compartilhar minha experiência com vocês para que possam imitar meus sucessos enquanto evitam meus fracassos. O novo material continua a fazer uso liberal de exames reais das bibliotecas da plataforma Java.

A primeira edição foi um sucesso além das minhas maiores expectativas, e fiz o meu melhor para me manter fiel ao seu espírito enquanto cobria todo o material novo necessário para atualizar o livro. Era inevitável que o livro crescesse, e cresceu, de cinquenta e sete itens para setenta e oito. Não só adicionei vinte e três itens, como revisei completamente todo o material original e aposentei alguns itens cujos dias melhores já haviam passado. No Apêndice, você pode ver como o material desta edição se relaciona com o material da primeira edição.

No Prefácio da Primeira Edição, escrevi que a linguagem de programação Java e suas bibliotecas eram extremamente propícias à qualidade e produtividade, e um prazer de trabalhar. As mudanças nas versões 5 e 6 fizeram algo bom e melhoraram. A plataforma é muito maior agora do que era em 2001 e mais complexa, mas uma vez que você aprende os padrões e expressões para usar os novos recursos, eles tornam seus programas melhores e sua vida mais fácil. Espero que esta edição capture meu continu-

O entusiasmo pela plataforma ajuda a tornar seu uso e seus novos recursos mais eficaz e agradável.

*San Jose,
Califórnia abril de 2008*

Prefácio da Primeira Edição

Em 1996, me arrisquei e fui para o oeste trabalhar na JavaSoft, como era chamada na época, porque estava claro que era ali que a ação estava. Nos cinco anos que se seguiram, atuei como arquiteto de bibliotecas de plataforma Java. Já projetei, implementei e mantive muitas bibliotecas e atuei como consultor para muitas outras. Presidir essas bibliotecas à medida que a plataforma Java amadurecia era uma oportunidade única na vida. Não é exagero dizer que tive o privilégio de trabalhar com alguns dos grandes engenheiros de software da nossa geração. Nesse processo, aprendi muito sobre a linguagem de programação Java — o que funciona, o que não funciona, e como usar a linguagem e suas bibliotecas da melhor forma possível.

Este livro é minha tentativa de compartilhar minha experiência com você para que você possa imitar meus sucessos enquanto evita meus fracassos. Peguei emprestado o formato do *Effective C++* da Scott Meyers, que consiste em cinquenta itens, cada um transmitindo uma regra específica para aprimorar seus programas e designs. Achei o formato extremamente eficaz, e espero que você também ache.

Em muitos casos, tomei a liberdade de ilustrar os itens com exemplos reais das bibliotecas da plataforma Java. Ao descrever algo que poderia ter sido feito melhor, tentei escolher código que escrevi eu mesmo, mas ocasionalmente pego em algo escrito por um colega. Peço sinceras desculpas se, apesar dos meus melhores esforços, ofendi alguém. Exemplos negativos são citados não para culpar, mas sim no espírito de cooperação, para que todos possamos nos beneficiar da experiência daqueles que já passaram antes.

Embora este livro não seja direcionado apenas a desenvolvedores de componentes reutilizáveis, ele inevitavelmente é influenciado pela minha experiência escrevendo esses componentes nas últimas duas décadas. Naturalmente, penso em termos de APIs exportadas (Interfaces de Programação de Aplicações), e incentivo você a fazer o mesmo. Mesmo que você não esteja desenvolvendo componentes reutilizáveis, pensar nesses termos tende a melhorar a qualidade do software que você escreve. Além disso, não é incomum escrever um código reutilizável-

Nem sem saber: Você escreve algo útil, compartilha com seu amigo do outro lado do corredor, e logo você tem meia dúzia de usuários. Neste ponto, você não tem mais flexibilidade para mudar a API à vontade e agradece por todo o esforço que colocou no design da API quando escreveu o software.

Meu foco em design de APIs pode parecer um pouco artificial para os fãs das novas metodologias leves de desenvolvimento de software, como a Programação Extrema. Essas metodologias enfatizam a escrita do programa mais simples que poderia funcionar. Se você está usando uma dessas metodologias, verá que o foco em API design é útil no processo de refatoração. Os objetivos fundamentais da refatoração são a melhoria da estrutura do sistema e a evitação de duplicação de código. Esses objetivos são impossíveis de alcançar na ausência de APIs bem projetadas para os componentes do sistema.

Nenhuma língua é perfeita, mas algumas são excelentes. Achei a linguagem de programação Java e suas bibliotecas extremamente favoráveis à qualidade e produtividade, além de um prazer trabalhar com ela. Espero que este livro capture meu entusiasmo e ajude a tornar seu uso da língua mais eficaz e agradável.

*Cupertino, Califórnia,
abril de 2001*

Agradecimentos

Agradecimentos pela Terceira Edição

Agradeço aos leitores das duas primeiras edições deste livro por lhe darem uma forma tão gentil e

Recebimento entusiasmado, por levar suas ideias a sério e por me mostrar a influência positiva que isso teve sobre eles e seu trabalho. Agradeço aos muitos professores que usaram o livro em seus cursos e às muitas equipes de engenharia que o adotaram.

Agradeço a toda a equipe da Addison-Wesley and Pearson pela gentileza, profissionalismo, paciência e elegância sob extrema pressão. Durante tudo isso, meu editor Greg Doench permaneceu imperturbável: um excelente editor e um perfeito cavalheiro. Receio que o cabelo dele tenha ficado um pouco grisalho por causa desse projeto, e peço humildes desculpas. Minha gerente de projeto, Julie Nahil, e minha editora de projeto, Dana Wilson, eram tudo o que eu poderia esperar: diligente, pontual, organizada e amigável. Minha revisora, Kim Wimpsett, era meticulosa e de bom gosto.

Mais uma vez fui abençoado com a melhor equipe de críticos imaginável, e agradeço sinceramente a cada um deles. A equipe principal, que revisou quase todos os capítulos, era composta por Cindy Bloch, Brian Kernighan, Kevin Bourrillion, Joe Bowbeer, William Chargin, Joe Darcy, Brian Goetz, Tim Halloran, Stuart Marks, Tim Peierls e Yoshiki Shibata. Outros críticos incluíram Marcus Biel, DanBloch, Beth Bottos, Martin Buchholz, Michael Diamond, Charlie Garrod, TomHawtin, Doug Lea, Aleksey Shipilëv, Lou Wasserman e Peter Weinberger. Esses críticos fizeram inúmeras sugestões que levaram a grandes melhorias neste livro e me salvou de muitos constrangimentos.

Agradeço especialmente a William Chargin, Doug Lea e Tim Peierls, que serviram como caixas de ressonância para muitas das ideias deste livro. William, Doug e Tim foram sempre generosos com seu tempo e conhecimento.

Por fim, agradeço à minha esposa, Cindy Bloch, por me incentivar a escrever, por ler cada item em forma bruta, por escrever o índice, por me ajudar com todas as coisas que invariavelmente surgem quando você assume um grande projeto, e por me suportar enquanto eu escrevia.

Agradecimentos pela Segunda Edição

Agradeço aos leitores da primeira edição deste livro por lhe darem uma recepção tão gentil e entusiasmada, por levarem suas ideias a sério e por me mostrarem a influência positiva que ele teve sobre eles e seu trabalho. Agradeço aos muitos professores que usaram o livro em seus cursos e às muitas equipes de engenharia que o adotaram.

Agradeço a toda a equipe da Addison-Wesley pela gentileza, profissionalismo, paciência e elegância sob pressão. Durante tudo isso, meu editor Greg Doench permaneceu imperturbável: um excelente editor e um perfeito cavalheiro. Minha gerente de produção, Julie Nahil, era tudo o que um gerente de produção deve ser: diligente, pontual, organizada e amigável. Minha revisora, Barbara Wood, era meticulosa e de bom gosto.

Mais uma vez fui abençoado com a melhor equipe de críticos imaginável, e agradeço sinceramente a cada um deles. A equipe principal, que revisava cada capítulo, era composta por Lexi Baugher, Cindy Bloch, Beth Bottos, Joe Bowbeer, Brian Goetz, Tim Halloran, Brian Kernighan, Rob Konigsberg, Tim Peierls, Bill Pugh, Yoshiki Shibata, Peter Stout, Peter Weinberger e Frank Yellin. Outros críticos incluíram Pablo Bellver, Dan Bloch, Dan Bornstein, Kevin Bourrillion, Martin Buchholz, Joe Darcy, Neal Gafter, Laurence Gonsalves, Aaron Green-house, Barry Hayes, Peter Jones, Angelika Langer, Doug Lea, Bob Lee, Jeremy Manson, Tom May, Mike McCloskey, Andriy Tereshchenko e Paul Tyma. Mais uma vez, esses críticos fizeram inúmeras sugestões que levaram a grandes melhorias neste livro e me pouparam de muitos constrangimentos. E, novamente, qualquer constrangimento restante é minha responsabilidade.

Agradeço especialmente a Doug Lea e Tim Peierls, que serviram como caixas de ressonância para muitas das ideias deste livro. Doug e Tim foram sempre generosos com seu tempo e conhecimento.

Agradeço à minha gerente do Google, Prabha Krishna, pelo apoio e incentivo contínuos.

Por fim, agradeço à minha esposa, Cindy Bloch, por me incentivar a escrever, por ler cada item em forma bruta, por me ajudar com o Framemaker, por escrever o índice e por me suportar enquanto eu escrevia.

Agradecimentos pela Primeira Edição

Agradeço a Patrick Chan por sugerir que eu escrevesse este livro e por apresentar a ideia para Lisa Friendly, editora-gerente da série; Tim Lindholm, editor técnico da série; e Mike Hendrickson, editor executivo da Addison-Wesley. Agradeço à Lisa, ao Tim e ao Mike por me incentivarem a seguir com o projeto e por sua paciência sobre-humana e fê inabalável de que um dia eu escreveria este livro.

Agradeço a James Gosling e sua equipe original por me darem algo ótimo para escrever, e agradeço aos muitos engenheiros de plataforma Java que seguiram os passos de James. Em especial, agradeço aos meus colegas do Java PlatformTools and Libraries Group da Sun por seus insights, incentivo e apoio. A equipe é composta por Andrew Bennett, Joe Darcy, Neal Gafter, Iris Garcia, Konstantin Kladko, Ian Little, Mike McCloskey e Mark Reinhold. Ex-membros incluem Zhenghua Li, Bill Maddox e Naveen Sanjeeva.

Agradeço ao meu gerente, Andrew Bennett, e ao meu diretor, Larry Abrahams, por terem dado total e entusiasmado apoio a este projeto. Agradeço a Rich Green, vice-presidente de Engenharia da Java Software, por proporcionar um ambiente onde engenheiros têm liberdade para pensar criativamente e publicar seus trabalhos.

Fui abençoado com a melhor equipe de críticos imaginável, e agradeço sinceramente a cada um deles: Andrew Bennett, Cindy Bloch, Dan Bloch, BethBottos, Joe Bowbeer, Gilad Bracha, Mary Campione, Joe Darcy, David Eckhardt, Joe Fialli, Lisa Friendly, James Gosling, Peter Hagggar, David Holmes, Brian Kernighan, Konstantin Kladko, Doug Lea, Zhenghua Li, Tim Lindholm, Mike McCloskey, Tim Peierls, Mark Reinhold, Ken Russell, Bill Shannon, Peter Stout, Phil Wadler e dois críticos anônimos. Eles fizeram inúmeras sugestões que levaram a grandes melhorias neste livro e me pouparam de muitos constrangimentos. Qualquer constrangimento restante é minha responsabilidade.

Numerosos colegas, dentro e fora da Sun, participaram de discussões técnicas que melhoraram a qualidade deste livro. Entre outros, Ben Gomes, Steffen Grarup, Peter Kessler, Richard Roda, John Rose e David Stoutami contribuíram com insights úteis. Um agradecimento especial é devido a Doug Lea, que serviu como um centro de ressonância para muitas das ideias deste livro. Doug tem sido infalível com seu tempo e conhecimento.

Agradeço a Julie Dinicola, Jacqui Doucette, Mike Hendrickson, Heather Olszyk, Tracy Russ e toda a equipe da Addison-Wesley pelo apoio e profissionalismo. Mesmo sob uma agenda impossivelmente apertada, eles sempre foram amigáveis e prestáveis.

Agradeço a Guy Steele por escrever o Prefácio. Sinto-me honrado por ele ter escolhido participar deste projeto.

Por fim, agradeço à minha esposa, Cindy Bloch, por me incentivar e, ocasionalmente, ameaçar a escrever este livro, por ler cada item em sua forma bruta, por me ajudar com Framemaker, por escrever o índice e por me suportar enquanto eu escrevia.

CHAPTER 1

Introdução

ESTE livro foi criado para ajudar você a fazer uso eficaz da programação em Java

Linguagem e suas bibliotecas fundamentais: `java.lang`, `java.util` e `java.io`, e subpacotes como `java.util.concurrent` e `java.util.function`. Outras bibliotecas são discutidas de tempos em tempos.

Este livro consiste em noventa itens, cada um transmitindo uma regra. As práticas de captura de regras geralmente consideradas benéficas pelos melhores e mais experientes programadores. Os itens são agrupados de forma geral em onze capítulos, cada um cobrindo um aspecto amplo do design de software. O livro não foi feito para ser lido de capa a capa: cada item se sustenta por si só, mais ou menos. Os itens são fortemente referenciados para que você possa traçar seu próprio caminho facilmente ao longo do livro.

Muitos novos recursos foram adicionados à plataforma desde a última edição deste livro. A maioria dos itens deste livro usa esses recursos de alguma forma. Esta tabela mostra onde ir para a cobertura principal dos principais recursos:

Característica	Itens	Lançamento
Lambdas	Itens 42–44	Java 8
Riachos	Itens 45–48	Java 8
Opcionais	Item 55	Java 8
Métodos padrão em interfaces	Item 21	Java 8
Tente com recursos	Item 9	Java 7
@SafeVarargs	Item 32	Java 7
Módulos	Item 15	Java 9

A maioria dos itens é ilustrada com exemplos de programas. Uma característica chave deste livro é que ele contém exemplos de código ilustrando muitos padrões e expressões de design. Quando apropriado, são cruzadas com o trabalho de referência padrão nesta área [Gamma95].

Muitos itens contêm um ou mais exemplos de programas ilustrando algumas práticas a serem evitadas. Exemplos assim, às vezes conhecidos como antipadrões, são claramente rotulados com um comentário como // Nunca faça isso!. Em cada caso, o item explica por que o exemplo é ruim e sugere uma abordagem alternativa.

Este livro não é para iniciantes: ele parte do pressuposto de que você já está confortável com Java. Se não estiver, considere um dos muitos excelentes textos introdutórios, como Java Precisely [Sestoft16], de Peter Sestoft. Embora o Java Eficaz seja projetado para ser acessível a qualquer pessoa com conhecimento prático da linguagem, deve fornecer reflexão até mesmo para programadores avançados.

A maioria das regras deste livro deriva de alguns princípios fundamentais. Clareza e simplicidade são de extrema importância. O usuário de um componente nunca deve se surpreender com seu comportamento. Os componentes devem ser o menor possível, mas não menores. (Como usado neste livro, o termo componente refere-se a qualquer elemento de software reutilizável, desde um método individual até um framework complexo composto por múltiplos pacotes.) O código deve ser reutilizado em vez de copiado. As dependências entre componentes devem ser mantidas ao mínimo. Os erros devem ser detectados o mais rápido possível após serem cometidos, idealmente em tempo de compilação.

Embora as regras deste livro não se apliquem 100% do tempo, elas caracterizam as melhores práticas de programação na grande maioria dos casos. Você não deve seguir essas regras de forma escrava, mas violá-las apenas ocasionalmente e com razão. Aprender a arte da programação, como a maioria das outras disciplinas, consiste primeiro em aprender as regras e depois saber quando quebrá-las.

Na maior parte, este livro não é sobre performance. Trata-se de escrever programas que sejam claros, corretos, utilizáveis, robustos, flexíveis e fáceis de manter. Se você conseguir fazer isso, geralmente é relativamente simples conseguir o desempenho que precisa (Item 67). Alguns itens discutem preocupações de desempenho, e alguns desses itens fornecem números de desempenho. Esses números, que são apresentados com a frase "Na minha máquina", devem ser considerados aproximados, no máximo.

Para o que vale, minha máquina é uma Intel Core i7-4770K quad-core de 3,5GHz caseira e envelhecida, com 16 gigabytes de RAM DDR3-1866 CL9, rodando a versão Zulu 9.0.0.15 do OpenJDK da Azul, sobre Microsoft Windows 7 Professional SP1 (64 bits).

Ao discutir recursos da linguagem de programação Java e suas bibliotecas, às vezes é necessário consultar versões específicas. Para conveniência, este livro usa apelidos em vez de nomes oficiais de lançamento. Esta tabela mostra o mapa entre nomes de lançamento e apelidos:

Nome oficial do lançamento	Apelido
JDK 1.0.x	Java 1.0
JDK 1.1.x	Java 1.1
Plataforma Java 2, Edição Padrão, v1.2	Java 2
Java 2 Platform, Edição Padrão, v1.3	Java 3
Plataforma Java 2, Edição Padrão, v1.4	Java 4
Plataforma Java 2, Edição Padrão, v5.0	Java 5
Plataforma Java, Edição Padrão 6	Java 6
Java Platform, Edição Padrão 7	Java 7
Plataforma Java, Edição Padrão 8	Java 8
Plataforma Java, Edição Padrão 9	Java 9

Os exemplos são razoavelmente completos, mas preferem a legibilidade em vez da completude. Eles usam livremente classes dos pacotes `java.util` e `java.io`. Para compilar exemplos, você pode ter que adicionar uma ou mais declarações de importação, ou outros formulários padrão. O site do livro <http://joshbloch.com/effectivejava>, contains uma versão expandida de cada exemplo, que você pode compilar e rodar.

Na maior parte, este livro utiliza termos técnicos conforme definidos na Especificação da Linguagem Java, Java SE 8 Edition [JLS]. Alguns termos merecem destaque especial. A linguagem suporta quatro tipos de tipos: interfaces (incluindo anotações), classes (incluindo enums), arrays e primitivas. Os três primeiros são conhecidos como tipos de referência. Instâncias de classe e arrays são objetos; valores primitivos não são. Os membros de uma classe consistem em seus campos, métodos, classes membros e interfaces membro. A assinatura de um método consiste em seu nome e nos tipos de seus parâmetros formais; a assinatura não inclui o tipo de retorno do método.

Este livro utiliza alguns termos de forma diferente da Especificação da Linguagem Java. Diferente da Especificação da Linguagem Java, este livro usa herança como sinônimo de subclasse. Em vez de usar o termo herança para interfaces, este livro

simplesmente afirma que uma classe implementa uma interface ou que uma interface estende a outra. Para descrever o nível de acesso que se aplica quando nenhum é especificado, este livro usa o tradicional `package-private` em vez do `packageaccess` tecnicamente correto [JLS, 6.6.1].

Este livro utiliza alguns termos técnicos que não estão definidos na Especificação Java Language. O termo API exportada, ou simplesmente API, refere-se às classes, interfaces, construtores, membros e formas serializadas pelas quais um programador acessa uma classe, interface ou pacote. (O termo API, que é abreviação de interface de programação de aplicação, é usado em preferência ao termo `interface` que seria preferível para evitar confusão com a construção de linguagem com esse nome.)

Um programador que escreve um programa que usa uma API é chamado de usuário da API. Uma classe cuja implementação utiliza uma API é cliente da API.

Classes, interfaces, construtores, membros e formulários serializados são conhecidos coletivamente como elementos API. Uma API exportada consiste nos elementos da API acessíveis fora do pacote que define a API. Esses são os elementos da API que qualquer cliente pode usar e que o autor da API faz commit para suportar. Curiosamente, eles também são os elementos para os quais a utilidade Javadoc gera documentação em seu modo padrão de operação. De forma geral, a API exportada de um pacote consiste nos membros públicos e protegidos e construtores de cada classe pública ou interface do pacote.

No Java 9, um sistema de módulos foi adicionado à plataforma. Se uma biblioteca utiliza o sistema de módulos, sua API exportada é a união das APIs exportadas de todos os pacotes exportados pela declaração de módulo da biblioteca.

CHAPTER 2

Criando e Destruindo Objetos

ESTE capítulo trata da criação e destruição de objetos: quando e como criar

eles, quando e como evitar criá-los, como garantir que sejam destruídos de forma oportuna e como gerenciar quaisquer ações de limpeza que devam preceder sua destruição.

Item 1: Considere métodos estáticos de fábrica em vez de construtores

A maneira tradicional de uma classe permitir que um cliente obtenha uma instância é fornecer um construtor público. Existe outra técnica que deveria fazer parte do kit de ferramentas de todo programador. Uma classe pode fornecer um método público de fábrica estática, que é simplesmente um método estático que retorna uma instância da classe. Aqui está um exemplo simples do Booleano (a classe primitiva em caixa para booleano). Este método traduz um valor primitivo booleano em uma referência de objeto booleano:

```
publicamente estático booleano valueOf(boolean b) {  
    retorne b ? Booleano.TRUE : Booleano.FALSE;}
```

Note que um método de fábrica estático não é o mesmo que o padrão do Método de Fábrica do Design Patterns [Gamma95]. O método de fábrica estática descrito neste item não tem equivalente direto em Design Patterns.

Uma classe pode fornecer aos seus clientes métodos de fábrica estática em vez de, ou além de, construtores públicos. Fornecer um método de fábrica estática em vez de um construtor público traz tanto vantagens quanto desvantagens.

Uma vantagem dos métodos de fábrica estática é que, ao contrário dos construtores, eles possuem nomes. Se os parâmetros de um construtor não descrevem, por si só, o objeto devolvido, uma fábrica estática com um nome bem escolhido é mais fácil de usar e o código cliente resultante mais fácil de ler. Por exemplo, o

construtor `BigInteger(int, int, Random)`, que retorna um `BigInteger` que provavelmente é primo, teria sido melhor expresso como um método estático de fábrica chamado `BigInteger.probablePrime`. (Esse método foi adicionado no Java 4.)

Uma classe pode ter apenas um único construtor com uma determinada assinatura. Sabe-se que program-mers conseguem contornar essa restrição fornecendo dois construtores cujas listas de parâmetros diferem apenas na ordem de seus tipos de parâmetros. Isso é uma péssima ideia. O usuário de tal API nunca conseguirá lembrar qual construtor é qual e acabará chamando o errado por engano. Pessoas que leem código que usa esses construtores não saberão o que o código faz sem consultar a documentação da classe.

Por terem nomes, métodos de fábrica estática não compartilham a restrição discutida no parágrafo anterior. Nos casos em que uma classe parece exigir múltiplos construtores com a mesma assinatura, substitua os construtores por métodos `staticfactory` e nomes cuidadosamente escolhidos para destacar suas diferenças.

Uma segunda vantagem dos métodos de fábrica estática é que, ao contrário dos construtores, eles não são obrigados a criar um novo objeto toda vez que são invocados. Isso permite que classes imutáveis (Item 17) usem instâncias pré-construídas, ou armazenem em cache instâncias conforme são construídas, e as dispensem repetidamente para evitar criar objetos duplicados desnecessários. O método booleano `valueOf(boolean)` *illus*-trata essa técnica: ele nunca cria um objeto. Essa técnica é semelhante ao padrão *Flyweight* [Gamma95]. Pode melhorar muito o desempenho se objetos equivalentes forem solicitados com frequência, especialmente se forem caros para criar.

A capacidade dos métodos estáticos de fábrica de retornar o mesmo objeto a partir de invocações repetidas permite que as classes mantenham controle rigoroso sobre quais instâncias existem a qualquer momento. Classes que fazem isso são chamadas de controle de instância. Existem vários motivos para escrever classes controladas por instâncias. O controle de instância permite que uma classe garanta que ela é singleton (Item 3) ou não instanciável (Item 4). Além disso, permite que a classe de valor imutável (Item 17) garanta que não existem duas instâncias iguais: $a.igual(a)$ (b) se e somente se $a == b$. Essa é a base do padrão *Flyweight* [Gamma95]. Tipos `enum` (Item 34) oferecem essa garantia.

Uma terceira vantagem dos métodos de fábrica estática é que, ao contrário dos construtores, eles podem retornar um objeto de qualquer subtipo do seu tipo de retorno. Isso te dá grande flexibilidade na escolha da classe do objeto devolvido.

Uma aplicação dessa flexibilidade é que uma API pode retornar objetos sem tornar suas classes públicas. Esconder classes de implementação dessa forma leva a uma API muito compacta. Essa técnica se presta a frameworks baseados em interfaces (Item 20), onde interfaces fornecem tipos naturais de retorno para métodos de fábrica estática.

Antes do Java 8, interfaces não podiam ter métodos estáticos. Por convenção, os métodos `staticfactory` para uma interface chamada `Type` eram colocados em uma classe `companion` não instanciável (Item 4) chamada `Types`. Por exemplo, o `Java Collections Framework` possui quarenta e cinco implementações utilitárias de suas interfaces, fornecendo coleções não modificáveis, coleções sincronizadas e similares. Quase todas essas implementações são exportadas por métodos de fábrica estáticos em uma classe não instanciável (`java.util.Collections`). As classes dos objetos retornados são todas não públicas.

A API do `Framework de Cobranças` é muito menor do que seria se não tivesse exportado quarenta e cinco classes públicas separadas, uma para cada implementação de conveniência. Não é apenas a maior parte da API que é reduzida, mas o peso conceitual: o número e a dificuldade dos conceitos que os programadores precisam dominar para usar a API. O programador sabe que o objeto retornado possui exatamente a API especificada por sua interface, então não há necessidade de ler documentação adicional de classe para a classe de implementação. Além disso, usar tal método de fábrica estática exige que o cliente se refira ao objeto retornado por interface em vez da classe de implementação, o que geralmente é uma boa prática (Item 64).

A partir do Java 8, a restrição de que interfaces não podem conter métodos estáticos foi eliminada, então normalmente há pouca razão para fornecer uma classe `companion` não instanciável para uma interface. Muitos membros públicos estáticos que se sentiriam à vontade em uma classe assim deveriam ser colocados na própria interface. Note, no entanto, que ainda pode ser necessário colocar a maior parte do código de implementação por trás desses métodos estáticos em uma classe separada de pacotes privados. Isso porque o Java 8 exige que todos os membros estáticos de uma interface sejam públicos. O Java 9 permite métodos `static` privados, mas campos `static` e classes membros `static` ainda são obrigados a ser públicos.

Uma quarta vantagem das fábricas estáticas é que a classe do objeto retornado pode variar de chamada para chamada em função dos parâmetros de entrada. Qualquer subtipo do tipo de retorno declarado é permitido. A classe do objeto retornado também pode variar de lançamento para lançamento.

A classe `EnumSet` (Item 36) não possui construtores públicos, apenas fábricas estáticas. Na implementação `OpenJDK`, eles retornam uma instância de uma de duas subclasses, dependendo do tamanho do tipo de `enum` subjacente: se tiver sessenta e quatro ou menos elementos, como a maioria dos tipos de `enum`, as fábricas estáticas retornam uma `RegularEnumSet`, que é respaldada por um único longo; se o tipo `enum` tiver sessenta e cinco ou mais elementos, as fábricas retornam uma instância `JumboEnumSet`, respaldada por um array longo.

A existência dessas duas classes de implementação é invisível para os clientes. Se o `RegularEnumSet` deixasse de oferecer vantagens de desempenho para pequenos tipos de `enum`, poderia ser eliminado de uma versão futura sem efeitos colaterais. Da mesma forma, uma futura versão poderia adicionar uma terceira ou quarta implementação do `EnumSet` se isso se mostrar benéfico.

pela performance. Os clientes não sabem nem se importam com a classe do objeto que recebem de volta da fábrica; eles se importam apenas que seja alguma subclasse do EnumSet.

Uma quinta vantagem das fábricas estáticas é que a classe do objeto retornado não precisa existir quando a classe contendo o método é escrita. Esses métodos flexíveis e estáticos formam a base dos frameworks de provedores de serviço, como a API de Conectividade JavaDatabase (JDBC). Um framework de provedor de serviços é um sistema no qual provedores implementam um serviço, e o sistema disponibiliza as implementações aos clientes, desacoplando os clientes das implementações.

Existem três componentes essenciais em um framework de provedor de serviços: uma interface de serviço, que representa uma implementação; uma API de registro de provedores, que os provedores usam para registrar implementações; e uma API de acesso ao serviço, que os clientes usam para obter instâncias do serviço. A API de acesso ao serviço pode permitir que os clientes especifiquem critérios para escolher uma implementação. Na ausência desses critérios, a API retorna uma instância de uma implementação padrão, ou permite que o cliente percorra todas as implementações disponíveis. A API de acesso a serviços é a fábrica estática flexível que forma a base do framework do provedor de serviços.

Um quarto componente opcional de um framework de provedor de serviço é a interface de fornecedor de serviços, que descreve um objeto de fábrica que produz instâncias da interface do serviço. Na ausência de uma interface com o provedor de serviço, as implementações devem ser instanciadas de forma reflexiva (Item 65). No caso do JDBC, Connection atua como interface de serviço, DriverManager.registerDriver é a API de registro do provedor, DriverManager.getConnection é a API de acesso ao serviço, e Driver é a interface do provedor de serviço.

Existem muitas variantes do padrão de framework para provedores de serviço. Por exemplo, a API de acesso ao serviço pode devolver uma interface de serviço mais rica aos clientes do que a fornecida pelos provedores. Este é o padrão Bridge [Gamma95]. Frameworks de injeção de dependências (Item 5) podem ser vistos como provedores de serviços poderosos. Desde o Java 6, a plataforma inclui um framework de provedor de serviços de uso geral, java.util.ServiceLoader, então você não precisa, e geralmente não deveria, escrever seu próprio (Item 59). A JDBC não usa ServiceLoader, pois o primeiro é anterior ao segundo.

A principal limitação de fornecer apenas métodos de fábrica estática é que classes sem construtores públicos ou protegidos não podem ser subclassificadas. Por exemplo, é impossível subclassar qualquer uma das classes de implementação de conveniência no Collections Framework. Pode-se argumentar que isso pode ser uma bênção disfarçada porque incentiva os programadores a usarem composição em vez de herança (Item 18), e é necessário para tipos imutáveis (Item 17).

Uma segunda limitação dos métodos de fábrica estática é que eles são difíceis de encontrar para programadores. Eles não se destacam na documentação da API

que os construtores fazem, então pode ser difícil descobrir como instanciar uma classe que fornece métodos de fábrica estáticos em vez de construtores. A ferramenta Javadoc talvez chame atenção para métodos estáticos de fábrica. Enquanto isso, você pode reduzir esse problema chamando atenção para fábricas estáticas na documentação de classe ou interface e seguindo convenções comuns de nomenclatura. Aqui estão alguns nomes comuns para métodos de fábrica estática. Esta lista está longe de ser exaustiva:

- **from**—Um método de conversão de tipo que pega um único parâmetro e retorna uma instância correspondente desse tipo, por exemplo:

```
Data d = Data.de(instante);
```

- **de** — Um método de agregação que pega múltiplos parâmetros e retorna uma postura desse tipo que os incorpora, por exemplo:

```
Definir<Rank> faceCards = EnumSet.of(VALETE, DAMA, REI);
```

- **valueOf**—Uma alternativa mais detalhada a **from** e **of**, por exemplo:

```
BigInteger primo = BigInteger.valueOf(Integer.MAX_VALUE);
```

- **instance** ou **getInstance** — Retorna uma instância descrita por seus parâmetros (se houver), mas que não pode ser dita ter o mesmo valor, por exemplo:

```
StackWalker luke = StackWalker.getInstance(options);
```

- **create** ou **newInstance** — Instância semelhante ou **getInstance**, exceto que **newInstance** garante que cada chamada retorne uma nova instância, por exemplo:

```
Object newArray = Array.newInstance(classObject, arrayLen);
```

- **getType**—Como **getInstance**, mas usado se o método de fábrica estiver em uma classe diferente. Tipo é o tipo de objeto retornado pelo método de fábrica, por exemplo:

```
FileStore fs = Files.getFileStore(path);
```

- **newType**—Como **newInstance**, mas usado se o método de fábrica estiver em uma classe diferente. Tipo é o tipo de objeto retornado pelo método de fábrica, por exemplo:

```
BufferedReader br = Files.newBufferedReader(path);
```

- **type**—Uma alternativa concisa ao **getType** e **newType**, por exemplo:

```
<Complaint> lista de Lista = Coleções.lista(legacyLitany);
```

Em resumo, métodos de fábrica estática e construtores públicos têm seus próprios usos, e vale a pena entender seus méritos relativos. Frequentemente, fábricas estáticas são preferíveis, então evite o reflexo de fornecer construtores públicos sem antes considerar fábricas estáticas.

Item 2: Considere um construtor diante de muitos parâmetros construtores

Fábricas e construtores estáticos compartilham uma limitação: eles não escalam bem para grandes quantidades de parâmetros opcionais. Considere o caso de uma classe representando o rótulo de Informações Nutricionais que aparece nos alimentos embalados. Esses rótulos têm alguns campos obrigatórios — tamanho da porção, porções por recipiente e calorias por porção — e mais de vinte campos opcionais — gordura total, gordura saturada, gordura trans, colesterol, sódio e assim por diante. A maioria dos produtos possui valores diferentes de zero para apenas alguns desses campos opcionais.

Que tipo de construtores ou fábricas estáticas você deveria escrever para uma classe assim? Tradicionalmente, os programadores usavam o padrão de construtor telescópico, no qual você fornece um construtor apenas com os parâmetros necessários, outro com um único parâmetro opcional, um terceiro com dois parâmetros opcionais, e assim por diante, colocando um construtor com todos os parâmetros opcionais. Veja como isso fica na prática. Para ser breve, apenas quatro campos opcionais são mostrados:

```

Padrão de construtor telescópico - não escala bem!
classe
pública NutritionFacts {
    private final int porção Tamanho; (mL)                                Obrigatório
    Porções finais privadas de int; // (por recipiente)
    requerido int int davo final calorias; // (por porção) opcional final
    privado int gordura; // (g/porção) opcional privado final int
    sódio; // (mg/porção) opcional privado final privado int
    carboidrato; (g/porção) opcional

    publica NutritionFacts(int porçãoTamanho, int porções) {
        this(porçãoTamanho, porções, 0);
    }

    NutriçãoFactos públicos (int porçãoTamanho, int porções,
        int calorias) {this(porçãoTamanho, porções,
        calorias, 0);}

    NutriçãoFactos públicos (int porçãoTamanho, int porções,
        int calorias, int gordura) {isto(porçãoTamanho,
        porções, calorias, gordura, 0);}

    NutriçãoFactos públicos (int porçãoTamanho, int porções,
        int calorias, int gordura, int sódio) {this(porçãoTamanho,
        porções, calorias, gordura, sódio, 0);}

```

```
NutriçãoFactos públicos (int porçãoTamanho, int porções,
int calorias, int gordura, int sódio, int carboidrato)
{this.poringSize= porçãoTamanho; Isso.porções= porções; isso.
calorias= calorias; this.fat= fat; this.sodium= sodium;
isto.carboidrato = carboidrato;}}
```

Quando você quer criar uma instância, você usa o construtor com a lista de parâmetros mais curtos contendo todos os parâmetros que deseja definir:

```
NutriçãoFactos, cocaCola =
    novos NutritionFacts (240, 8, 100, 0, 35, 27);
```

Normalmente, essa invocação de construtor exigirá muitos parâmetros que você não quer definir, mas você é obrigado a passar um valor para eles mesmo assim. Nesse caso, passamos um valor de 0 para gordura. Com "apenas" seis parâmetros isso pode não parecer tão ruim, mas rapidamente sai do controle à medida que o número de parâmetros aumenta.

Em resumo, o padrão de construtores telescopados funciona, mas é difícil escrever código cliente quando há muitos parâmetros, e ainda mais difícil de ler. Thereader fica se perguntando o que todos esses valores significam e precisa contar cuidadosamente parâmetros para descobrir. Longas sequências de parâmetros idênticos podem causar bugs sutis. Se o cliente acidentalmente reverter dois desses parâmetros, o compilador não reclamará, mas o programa terá um comportamento inadequado em tempo de execução (Item 51).

Uma segunda alternativa quando você enfrenta muitos parâmetros opcionais no aconstrutor é o padrão JavaBeans, no qual você chama um construtor sem parâmetros para criar o objeto e depois chama os métodos setter para definir cada parâmetro requerido e cada parâmetro opcional de interesse:

```
JavaBeans Pattern - permite inconsistência, exige mutabilidade,
classe pública NutritionFacts {
    Parâmetros inicializados para valores padrão (se houver)private
    int servingSize= -1; Obrigatório; sem valor padrãoPrivate int
    servings= -1; Obrigatório; sem valor padrãoprivate int calorias=
    0; Private Int FAT= 0; Private Int Sódio= 0; private int
    carboidrato = 0;

    NutriçãoFactos() { }
```



```

    Setters public void setServingSize(int
    val)                                { porçãoTamanho = val; }
    setServings(int val) de vazio-público { porções = val; }
    public void setCalories(int val)      { calorias = val; }
    empty público setFat(int val)         { gordo = val; }
    vazio-público setSodium(int val)      { sódio = val; }
    empty public setCarbohydrate(int val) { carbohydrate = val; }}

```

Esse padrão não apresenta nenhuma das desvantagens do padrão de construtores telescópicos. É fácil, embora um pouco extenso, criar instâncias, e fácil de ler o código resultante:

```

NutriçãoFactos, cocaCola = novo NutritionFacts();
cocaCola.setPorçãoTamanho(240);
cocaCola.setPorções(8); cocaCola.setCalories(100);
cocaCola.setSodium(35);
cocaCola.setCarbohydrate(27);

```

Infelizmente, o padrão JavaBeans tem desvantagens sérias próprias. Como a construção é dividida em múltiplas chamadas, um JavaBean pode estar em estado inconsistente no meio da construção. A classe não tem a opção de impor consistência apenas verificando a validade dos parâmetros do construtor. Tentar usar um objeto quando ele está em um estado inconsistente pode causar falhas muito distantes do código que contém o bug e, portanto, difícil de depurar. Uma desvantagem relacionada é que o padrão JavaBeans impede a possibilidade de tornar uma classe imutável (Item 17) e exige um esforço adicional por parte do programador para garantir a segurança das threads.

É possível reduzir essas desvantagens "congelando" manualmente o objeto quando sua construção está completa e não permitindo que ele seja usado até congelado, mas essa variante é difícil de manejar e raramente usada na prática. Além disso, pode causar erros em tempo de execução porque o compilador não pode garantir que o programador chame o método de congelamento em um objeto antes de usá-lo.

Felizmente, existe uma terceira alternativa que combina a segurança do padrão telescopingconstructor com a legibilidade do padrão JavaBeans. É uma forma do padrão theBuilder [Gamma95]. Em vez de criar o objeto desejado diretamente, o cliente chama um construtor (ou fábrica estática) com todos os parâmetros necessários e obtém um objeto construtor. Então o cliente chama métodos semelhantes ao setter no objeto construtor para definir cada parâmetro opcional de interesse. Por fim, o cliente chama um método parameterlessbuild para gerar o objeto, que normalmente é imutável. O construtor é tipicamente uma classe membro estática (Item 24) da classe que ele constrói. Veja como fica na prática:

```

Builder Patternpublic classe
NutritionFacts {
    private final int porção Tamanho;
    Serviço de Inteligência Final Privado;
    calorias finais privadas de
    inteligência; Privado Final, Int, Fat;
    Soldado Individual Final Int Sódium;
    Carboidrato INT final privado;

    classe estática pública Builder {
        Parâmetros necessários, private
        final int porçãoTamanho; Serviço
        de Inteligência Final Privado;

        Parâmetros opcionais - inicializados para valores padrão,
        calorias privadas int=0; Private Int FAT= 0; Private Int
        Sódio= 0; private int carboidrato= 0;

        public Builder(int servingSize, int servings) {
            this.poringSize = porçãoSize;
            this.porções= porções;}

        Calorias públicas do Builder (int val)
            { calorias = val;         devolva isto; }
        Construtor público fat(int val)
            { fat = val;              devolva isto; }
        Construtor público de sódio (int val)
            { sódio = val;            devolva isto; }
        Carboidrato construtor público (int val)
            { carboidrato = val;      devolva isto; }

        NutritionFacts público build() {
            retornar novos NutricionFacts(this);}}

    privado: NutritionFacts(Builder builder) {
        porçãoTamanho = construtor.tamanhoPorção;
        porções       = construtor.serviços;
        calorias       = construtor.calorias;
        Gordo          = builder.fat;
        Sódio          = construtor.sódio;
        carboidrato = construtor.carboidrat;}}

```

A classe `NutricionFacts` é imutável, e todos os valores padrão dos parâmetros estão em um só lugar. Os métodos `setter` do construtor retornam o próprio construtor para que as invocações possam ser encadeadas, resultando em uma API fluente. Veja como o código do cliente se apresenta:

```
NutriçãoFactos, cocaCola = novo NutritionFacts.Builder(240, 8)
    .calorias(100).sódio(35).carboidratos(27).construção();
```

Esse código cliente é fácil de escrever e, mais importante, fácil de ler. O `Builderpattern` simula parâmetros opcionais nomeados, como encontrados em `Python` e `Scala`.

Verificações de validade foram omitidas por brevidade. Para detectar parâmetros inválidos o mais rápido possível, verifique a validade dos parâmetros no construtor e nos métodos do construtor. Verifique invariantes envolvendo múltiplos parâmetros no construtor invocados pelo método `thebuild`. Para garantir esses invariantes contra ataques, faça as verificações nos campos objeto após copiar os parâmetros do construtor (Item 50). Se uma verificação falhar, lance um `ArgumentoExceção Illegal` (Item 72) cuja mensagem detalhada indica quais parâmetros são inválidos (Item 75).

O padrão `Builder` é muito adequado para hierarquias de classes. Use uma hierarquia paralela de construtores, cada um aninhado na classe correspondente. Classes abstratas têm construtores abstratos; As aulas de concreto têm construtores de concreto. Por exemplo, considere uma classe abstrata na raiz de uma hierarquia que representa vários tipos de pizza:

```
Padrão construtor para hierarchias de classes classe abstrato público Pizza {

cobertura pública enum { PRESUNTO, COGUMELOS, CEBOLA, PIMENTA,
LINGUIÇA }final<Topping> Coberturas do conjunto; classe estática
abstrata Builder<T estende Builder<T>> {
    <Topping> EnumSet toppings = EnumSet.noneOf(Topping.class); public T addTopping(Topping
topping) {
toppings.add(Objects.requireNonNull(topping)); retorne a si
mesmo();}abstrato Pizza build();// Subclasses devem sobrescrever
esse método para retornar "este" abstrato protegido T
self();}Pizza(Builder<?> builder) {

toppings = builder.toppings.clone(); Veja o Item 50}}
```

Note que `Pizza.Builder` é um tipo genérico com um parâmetro de tipo recursivo (Item 30). Isso, junto com o método do `eu abstrato`, permite que a cadeia de métodos funcione corretamente em subclasses, sem a necessidade de casts. Essa solução para o fato de Java não possuir um tipo de `si` é conhecida como o `idiom do tipo self simulado`.

Aqui estão duas subclasses concretas de pizza, uma das quais representa uma pizza padrão no estilo Nova York, a outra um calzone. O primeiro tem um tamanho de parâmetro exigido, enquanto o segundo permite especificar se o molho deve ser dentro ou fora:

```

classe pública NyPizza estende Pizza {
    public enum Tamanho { PEQUENO, MÉDIO, GRANDE
    } privado final Tamanho tamanho;

    classe pública estática Builder estende Pizza.Builder<Builder> {
        privado final Tamanho;

        public Builder(Tamanho, tamanho) {
            this.size = Objects.requireNonNull(size);
        }

        @Override build público do NyPizza() {
            retorne novo NyPizza(this);
        }

        @Override protegido Builder self() { return this; }}

    privada NyPizza(Builder builder) {
        super(construtor); tamanho =
        construtor.tamanho;}}

classe pública Calzone estende Pizza {
    molho booleano final privadoDentro;

    classe pública estática Builder estende Pizza.Builder<Builder> {
        molho booleano privado Inside = falso; Padrão

        Builder SauceInside() {
            sauceInside = verdadeiro;
            devolva isto;}

        @Override build público Calzone() {
            retorne ao novo Calzone(este);
        }

        @Override protegido Builder self() { return this; }}

    privado Calzone (Construtor construtor) {
        super(construtor); sauceInside =
        construtor.sauceInside;}}

```

Note que o método `build` em cada builder de subclasse é declarado para retornar a subclasse correta: o método `build` de `NyPizza.Builder` retorna `NyPizza`, enquanto o de `Calzone.Builder` retorna `Calzone`. Essa técnica, na qual um submétodo é declarado para retornar um subtipo do tipo de retorno declarado na superclasse, é conhecida como tipagem de retorno covariante. Isso permite que os clientes utilizem esses construtores sem a necessidade de fundição.

O código cliente desses "construtores hierárquicos" é essencialmente idêntico ao código do construtor simples `NutritionFacts`. O exemplo de código cliente mostrado `Nextassumes` importações estáticas em constantes de `enum` para maior brevidade:

```
NyPizza = nova NyPizza.Builder(PEQUENO)
    .adicionarTopping(SASICHA).adicionarTopping(CEBOLA).build();
Calzone calzone = novo
    Calzone.Builder().addTopping(HAM).sauceInside().build();
```

Uma vantagem menor dos construtores em relação aos construtores é que os construtores podem ter parâmetros variáveis multi-típle porque cada parâmetro é especificado em seu próprio método. Alternativamente, os construtores podem agregar os parâmetros passados em múltiplas chamadas para um método em um único campo, como demonstrado anteriormente no método `addTopping`.

O padrão `Builder` é bem flexível. Um único construtor pode ser usado repetidamente para construir múltiplos objetos. Os parâmetros do construtor podem ser ajustados entre invocações do método de construção para variar os objetos criados. Um construtor pode preencher alguns campos automaticamente na criação do objeto, como um número de série que aumenta a cada vez que um objeto é criado.

O padrão `Builder` também tem desvantagens. Para criar um objeto, você deve primeiro criar seu construtor. Embora o custo de criar esse construtor provavelmente não seja perceptível na prática, ele pode ser um problema em situações críticas de desempenho. Além disso, o padrão `Builder` é mais prolixo que o padrão de construtor telescópico, então deve ser usado apenas se houver parâmetros suficientes para valer a pena, digamos, quatro ou mais. Mas lembre-se de que talvez você queira adicionar mais parâmetros no futuro. Mas se você começar com construtores ou fábricas estáticas e mudar para um construtor quando a classe evoluir a ponto de o número de parâmetros sair do controle, os construtores obsoletos ou fábricas estáticas vão se destacar muito. Portanto, muitas vezes é melhor começar com um construtor desde o início.

Em resumo, o padrão `Builder` é uma boa escolha ao projetar classes cujos construtores ou fábricas estáticas teriam mais do que alguns parâmetros, especialmente se muitos dos parâmetros forem opcionais ou do mesmo tipo. O código do cliente é muito mais fácil de ler e escrever com builders do que com telescopingconstructors, e builders são muito mais seguros que o `JavaBeans`.

Item 3: Aplicar a propriedade singleton com um construtor privado ou um tipo enum

Um singleton é simplesmente uma classe que é instanciada exatamente uma vez [Gamma95]. Toneladas individuais normalmente representam um objeto sem estado, como uma função (Item 24), ou um componente do sistema que é intrinsecamente único. Tornar uma classe um singleton pode dificultar o teste dos clientes, pois é impossível substituir uma mockimplementation por um singleton a menos que implemente uma interface que sirva como seu tipo.

Existem duas maneiras comuns de implementar singletons. Ambos se baseiam em manter o construtor privado e exportar um membro público estático para fornecer acesso à única instância. Em uma abordagem, o membro é um campo final:

```
Singleton com campo público final
classe pública Elvis {
    instância pública estática final Elvis INSTANCE =
        novo Elvis(); soldado Elvis() { ... }

    vazio público deixaOPrédio() { ... }}
```

O construtor privado é chamado apenas uma vez, para inicializar o campo final público estático `Elvis.INSTANCE`. A ausência de um construtor público ou protegido garante um universo "monoélvico": exatamente uma instância de `Elvis` existirá assim que a `Elvis` class for inicializada — nem mais, nem menos. Nada do que um cliente faz pode mudar isso, com uma ressalva: um cliente privilegiado pode invocar o construtor privado de forma reflexiva (Item 65) com a ajuda do método `AccessibleObject.setAccessible`. Se você precisar se defender desse ataque, modifique o construtor para que ele jogue uma exceção caso ele seja solicitado a criar uma segunda instância.

Na segunda abordagem para implementar singletons, o membro público é o método da fábrica estática:

```
Singleton com classe pública
estática de fábrica Elvis {
    instância estática privada final Elvis INSTANCE = novo
    Elvis(); soldado Elvis() { ... } publicamente estático
    Elvis getInstance() { return INSTANCE; }

    vazio público deixaOPrédio() { ... }}
```

Todas as chamadas para `Elvis.getInstance` retornam a mesma referência de objeto, e nenhuma outra instância `Elvis` será criada (com a mesma ressalva mencionada anteriormente).

A principal vantagem da abordagem de campo público é que a API deixa claro que a classe é singleton: o campo público estático é final, então sempre conterá a mesma referência de objeto. A segunda vantagem é que é mais simples.

Uma vantagem da abordagem de fábrica estática é que ela te dá flexibilidade para mudar de ideia sobre se a classe é singleton sem mudar sua API. O método de fábrica retorna a única instância, mas ele pode ser modificado para retornar, por exemplo, uma instância separada para cada thread que a invoca. Uma segunda vantagem é que você pode escrever uma fábrica genérica singleton se sua aplicação exigir isso (Item 30). Uma vantagem final de usar uma fábrica estática é que uma referência de método pode ser usada como fornecedor, por exemplo, `Elvis::instance` é um `<Elvis>Fornecedor`. A menos que uma dessas vantagens seja relevante, a abordagem de campo público é preferível.

Para tornar uma classe singleton que usa qualquer uma dessas abordagens serializável (Capítulo 12), não basta simplesmente adicionar `implements Serializable` à sua declaração. Para manter a garantia de singleton, declare todos os campos transientes da instância e forneça um método `readResolve` (Item 89). Caso contrário, cada vez que uma instância serializada for desserializada, uma nova instância será criada, levando, no caso do nosso exemplo, a avistamentos espúrios de Elvis. Para evitar que isso aconteça, adicione este método `readResolve` à classe `Elvis`:

```
método readResolve para preservar singleton
propertyprivate Object readResolve() {
    Devolva o único Elvis verdadeiro e deixe o coletor de lixo// cuidar
    do imitador do Elvis.return INSTANCE;}
```

Uma terceira forma de implementar um singleton é declarar um enum de elemento único:

```
Enum singleton - a abordagem preferida public
enum Elvis {
    EXEMPLO;

    vazio público deixaOPrédio() { ... }}
```

Essa abordagem é semelhante à abordagem de campo público, mas é mais concisa, fornece a maquinaria de serialização gratuitamente e oferece uma garantia inabalável contra múltiplas instanciações, mesmo diante de sofisticados ataques de serialização ou reflexão. Essa abordagem pode parecer um pouco artificial, mas um tipo de elemento único geralmente é a melhor forma de implementar um singleton. Note que você não pode usar essa abordagem se seu singleton precisar estender uma superclasse diferente de `Enum` (embora você possa declarar um enum para implementar interfaces).

Item 4: Importar a não instanciabilidade com um construtor privado

Ocasionalmente, você vai querer escrever uma classe que seja apenas um agrupamento de métodos estáticos e campos estáticos. Essas classes adquiriram má reputação porque algumas pessoas as abusam para evitar pensar em termos de objetos, mas elas têm usos válidos. Eles podem ser usados para agrupar métodos relacionados em valores primitivos ou arrays, da maneira `java.lang.Math` ou `java.util.Arrays`. Eles também podem ser usados para agrupar métodos estáticos, incluindo fábricas (Item 1), para objetos que implementam alguma interface, no estilo `java.util.Collections`. (A partir do Java 8, você também pode colocar esses métodos na interface, assumindo que seja seu para modificar.) Por fim, tais classes podem ser usadas para agrupar métodos em uma classe final, já que você não pode colocá-los em uma subclasse.

Tais classes utilitárias não foram projetadas para serem instanciadas: uma instância seria benonsensical. Na ausência de construtores explícitos, entretanto, o compilador produz um construtor padrão público, sem parâmetros. Para um usuário, esse construtor é indistinguível de qualquer outro. Não é incomum ver classes instintivamente instantiáveis em APIs publicadas.

Tentar impor a não instanciabilidade tornando uma classe abstrata não funciona. A classe pode ser subclassificada e a subclasse instanciada. Além disso, isso leva o usuário a pensar que a classe foi projetada para herança (Item 19). Há, no entanto, um ditado simples para garantir a não instanciabilidade. Um construtor padrão é gerado somente se uma classe não contiver construtores explícitos, então uma classe pode ser tornada não instanciável ao incluir um construtor privado:

```
Utilityclass não instanciabil,
classpublic UtilityClass {
    Suprimir o construtor padrão para
    noninstantiabilityprivate UtilityClass() {
        lançar um novo AssertionError();}...//
    Restante omitido}
```

Como o construtor explícito é privado, ele é inacessível fora da classe. O `AssertionError` não é estritamente obrigatório, mas oferece seguro caso o construtor seja acidentalmente invocado dentro da classe. Garante que a aula nunca será iniciada sob nenhuma circunstância. Esse idioma é levemente contraintuitivo porque o construtor é fornecido expressamente para que não possa ser invocado. Portanto, é sensato incluir um comentário, como mostrado anteriormente.

Como efeito colateral, esse idioma também impede que a classe seja subclassificada. Todos os construtores devem invocar um construtor de superclasse, explícita ou implicitamente, e uma subclasse não teria nenhum construtor de superclasse acessível para invocar.

Item 5: Prefira injeção de dependência a recursos de cabeamento direto

Muitas classes dependem de um ou mais recursos subjacentes. Por exemplo, um corretor ortográfico depende de um dicionário. Não é incomum ver tais classes implementadas como classes de utilidade estática (Item 4):

```

    Uso inadequado de utilidade estática - inflexível e
    indetetável!classe pública SpellChecker {
        dicionário final estático privado de Léxico = ...;

        SpellChecker() {} // Noninstantiable

        boolean estático público isValid(String word) { ... }sugestões de
        listas públicas<String> estáticas (erro de digitação na sequência) {
        ... }}

```

Da mesma forma, não é incomum vê-los implementados como singletons (Item 3):

```

    Uso inadequado de singleton - inflexível e não testável!classe
    pública SpellChecker {
        dicionário final privado de Léxico = ...;

        Corretor ortográfico privado(...) {}INSTANCE
        estática pública = novo Corretor Ortográfico(...);

        boolean public isValid(String word) { ... }sugestões de
        lista<String> pública (erro de digitação na cadeia) { ... }}

```

Nenhuma dessas abordagens é satisfatória, pois assumem que existe apenas um dicionário que vale a pena usar. Na prática, cada idioma possui seu próprio dicionário, e dicionários especiais são usados para vocabulários específicos. Além disso, pode ser desejável usar um dicionário especial para testes. É um desejo ilusório assumir que um único dicionário será suficiente para sempre.

Você poderia tentar fazer o SpellChecker suportar múltiplos dicionários tornando o campo do dicionário não final e adicionando um método para alterar o dicionário em um corretor ortográfico existente, mas isso seria estranho, propenso a erros e inviável em um contexto simultâneo. Classes de utilidade estática e singletons são inadequadas para classes cujo comportamento é parametrizado por um recurso subjacente.

O que é necessário é a capacidade de suportar múltiplas instâncias da classe (no nosso exemplo, SpellChecker), cada uma das quais usa o recurso desejado pelo cliente (no nosso exemplo, o dicionário). Um padrão simples que atende a esse requisito é passar o recurso para o construtor ao criar uma nova instância. Esta é uma forma de injeção de dependência: o dicionário é uma dependência do corretor ortográfico e é injetado no corretor ortográfico quando ele é criado.

```

A injeção de dependência oferece flexibilidade e teabilidade
classe pública SpellChecker {
    dicionário final privado de Léxico;

    Corretor ortográfico público (dicionário de léxico) {
        this.dictionary = Objects.requireNonNull(dictionary));

    boolean public isValid(String word) { ... }sugestões de
    lista<String> pública (erro de digitação na cadeia) { ... }}

```

O padrão de injeção de dependências é tão simples que muitos programadores o usam por anos sem saber que tem um nome. Enquanto nosso exemplo do corretor ortográfico tinha apenas um recurso (o dicionário), a injeção de dependências funciona com um número arbitrário de recursos e grafos de dependência arbitrários. Ela preserva a imutabilidade (Item 17), então múltiplos clientes podem compartilhar objetos dependentes (assumindo que os clientes desejem os mesmos recursos subjacentes). A injeção de dependências é igualmente aplicável a construtores, fábricas estáticas (Item 1) e builders (Item 2).

Uma variante útil do padrão é passar uma fábrica de recursos para o construtor. Uma fábrica é um objeto que pode ser chamado repetidamente para criar instâncias de um tipo. Tais fábricas incorporam o padrão Método de Fábrica [Gamma95]. <T> A interface do Supplier, introduzida no Java 8, é perfeita para representar fábricas. Métodos que recebem um fornecedor<T> na entrada devem normalmente restringir o parâmetro de tipo da fábrica usando um tipo de coringa limitado (Item 31) para permitir que o cliente passe uma fábrica que crie qualquer subtipo de um tipo especificado. Por exemplo, aqui está um método que cria um mosaico usando uma fábrica fornecida pelo cliente para produzir cada azulejo:

```

Mosaic criar (Fornecedor<? estende Tile> tileFactory) { ... }

```

Embora a injeção de dependências melhore muito a flexibilidade e a teabilidade, pode sobrecarregar grandes projetos, que normalmente contêm milhares de dependências. Essa desordem pode ser praticamente eliminada usando um framework de injeção de dependências, como Dagger [Dagger], Guice [Guice] ou Spring [Spring]. O uso desses frameworks está além do escopo deste livro, mas observe que APIs projetadas para injeção manual de dependências são trivialmente adaptadas para uso por esses frameworks.

Em resumo, não use uma classe singleton ou utilidade estática para implementar uma classe que dependa de um ou mais recursos subjacentes cujo comportamento afeta o da classe, e não faça com que a classe crie esses recursos diretamente. Em vez disso, passe os recursos, ou fábricas para criá-los, para o construtor (ou fábrica estática ou construtor). Essa prática, conhecida como injeção de dependência, aumenta muito a flexibilidade, reutilização e testabilidade de uma classe.

Item 6: Evite criar objetos desnecessários

Frequentemente é apropriado reutilizar um único objeto em vez de criar um novo objeto equivalente a cada vez que ele é necessário. Reutilizar pode ser mais rápido e mais estiloso. Um objeto pode sempre ser reutilizado se for imutável (Item 17).

Como um exemplo extremo do que não fazer, considere esta afirmação:

```
String s = novo String("bikini");    NÃO FAÇA ISSO!
```

A instrução cria uma nova instância de `String` a cada execução, e nenhuma dessas criações de objetos é necessária. O argumento para o construtor `String("bikini")` é ele próprio uma instância de `String`, funcionalmente idêntica a todos os objetos criados pelo construtor. Se esse uso ocorrer em um loop ou em um método frequentemente invocado, milhões de instâncias de `String` podem ser criadas desnecessariamente.

A versão aprimorada é simplesmente a seguinte:

```
String s = "biquíni";
```

Esta versão usa uma única instância de `String`, em vez de criar uma nova cada vez que é executada. Além disso, é garantido que o objeto será descontrolado por qualquer outro código rodando na mesma máquina virtual que contenha a mesma string literal [JLS, 3.10.5].

Você pode frequentemente evitar criar objetos desnecessários usando `meth-ods` de fábrica estática (Item 1) em vez de construtores em classes imutáveis que oferecem ambos. Por exemplo, o método de fábrica `Boolean.valueOf(String)` é preferível ao construtor `Boolean(String)`, que foi descontinuado no Java 9. O construtor deve criar um novo objeto a cada vez que for chamado, enquanto o método de fábrica nunca é necessário e na prática não será necessário. Além de reutilizar objetos imutáveis, você também pode reutilizar objetos mutáveis se souber que eles não serão modificados.

Algumas criações de objetos são muito mais caras que outras. Se você vai precisar de um "objeto caro" assim repetidamente, pode ser aconselhável armazená-lo em cache para reutilizar. Infelizmente, nem sempre é óbvio quando você está criando um objeto assim. Suponha que você queira escrever um método para determinar se uma cadeia é um número romano válido. Aqui está a maneira mais fácil de fazer isso usando uma expressão regular:

```
O desempenho pode ser muito melhorado!static
boolean isRomanNumeral(String s) {
    retorne s.matches("(^(?=.)M*(C[MD]|D?C{0,3})"
        + "(X[CL]|L?X{0,3})(I[XV]|V?I{0,3})$)");
}
```

O problema dessa implementação é que ela depende do método `String.matches`. Embora o `String.matches` seja a maneira mais fácil de verificar se uma string corresponde a uma expressão regular, ele não é adequado para uso repetido em situações críticas de desempenho. O problema é que ele cria internamente uma instância de `Pattern` para a expressão regular e a usa apenas uma vez, após o que se torna elegível para a coleta de lixo. Criar uma instância de `Padrão` é caro porque requer compilar a expressão regular em uma máquina de estados finitos.

Para melhorar o desempenho, compile explicitamente a expressão regular em uma instância de `Pattern` (que é imutável) como parte da inicialização da classe, armazene-a em cache e use a mesma instância para cada invocação do método `isRomanNumeral`:

```
Reutilizando objetos caros para melhorar o desempenho,
classe RomanNumerals {
    private static final Pattern ROMAN = Pattern.compile(
        "(?=[MDCLXVI])M*(C[MD]|D?C{0,3})" + "(X[CL]|L?"
        + "X{0,3})(I[XV]|V?I{0,3})$");

    static boolean isRomanNumeral(String s) {
        return ROMAN.matcher(s).matches();}
}
```

A versão aprimorada do `isRomanNumeral` proporciona ganhos significativos de desempenho se invocada com frequência. Na minha máquina, a versão original leva uma string de entrada de 8 caracteres de 1,1 μ s, enquanto a versão melhorada leva 0,17 μ s, o que é 6,5 vezes mais rápido. Além de melhorar o desempenho, mas, provavelmente, também a clareza. Criar um campo final estático para a instância `Pattern`, que de outra forma seria invisível, nos permite dar um nome, que é muito mais legível do que a própria expressão regular.

Se a classe contendo a versão aprimorada do método `isRomanNumeral` for inicializada, mas o método nunca for invocado, o campo `ROMAN` será inicializado desnecessariamente. Seria possível eliminar a inicialização inicializando preguiçosamente o campo (Item 83) na primeira invocação do método `isRomanNumeral`, mas isso não é recomendado. Como costuma acontecer com inicializações preguiçosas, ela cumpriria a implementação sem nenhuma melhoria mensurável de desempenho (Item 67).

Quando um objeto é imutável, é óbvio que pode ser reutilizado com segurança, mas há outras situações em que é muito menos óbvio, até contraintuitivo. Considere o caso dos adaptadores [Gamma95], também conhecidos como views. Um adaptador é um objeto que delega a um objeto de suporte, fornecendo uma interface alternativa. Como um adaptador não tem estado além do de seu objeto de suporte, não há necessidade de criar mais de uma instância de um dado adaptador para um determinado objeto.

Por exemplo, o método `keySet` da interface `Map` retorna uma visualização `Set` do objeto `Mapa`, composta por todas as chaves do mapa. Ingenuamente, parece que cada chamada para `keySet` teria que criar uma nova instância de `Set`, mas toda chamada para `keySet` de um determinado objeto `Map` pode devolver a mesma instância de `Set`. Embora a instância de `Set` retornada seja tipicamente mutável, todos os objetos retornados são funcionalmente idênticos: quando um dos objetos retornados muda, todos os outros também mudam, porque todos são respaldados pela mesma instância de `Mapa`. Embora seja em grande parte inofensivo criar múltiplas instâncias do objeto `keySet view`, isso é desnecessário e não traz benefícios.

Outra forma de criar objetos desnecessários é o `autoboxing`, que permite ao programador misturar primitivos e tipos primitivos em caixa, boxando e unboxando automaticamente conforme necessário. O `autoboxing` borra, mas não apaga, a distinção entre tipos primitivos e primitivos em caixa. Existem distinções semânticas sutis e diferenças de desempenho nem tão sutis (Item 61). Considere o método a seguir, que calcula a soma de todos os valores positivos de `int`. Para isso, o programa precisa usar aritmética longa porque um `int` não é grande o suficiente para conter a soma de todos os valores positivos de `int`:

```
Horriblemente lento! Você consegue identificar a criação
do objeto? private static long sum() {
    Soma longa = 0L; para (i longo = 0; i <=
        Integer.MAX_VALUE; i++)
        soma += i;

    soma de
    retorno;}

```

Esse programa recebe a resposta certa, mas é muito mais lento do que deveria, devido a um erro tipográfico de um caractere. A soma das variáveis é declarada como um `Long` em vez de um `longo`, o que significa que o programa constrói cerca de 231 instâncias de `Long` (aproximadamente uma para cada vez que o `i` `longo` é somado à soma `Long`). Mudar a declaração de soma de `Long` para `longo` reduz o tempo de execução de 6,3 segundos para 0,59 segundos na minha máquina. A lição é clara: prefira primitivos em caixa primitivos e fique atento ao `autoboxing` não intencional.

Este item não deve ser interpretado como se a criação de objetos fosse expensativa e devesse ser evitada. Pelo contrário, a criação e recuperação de pequenos objetos cujos construtores fazem pouco trabalho explícito é barata, especialmente em implementações modernas de JVM. Criar objetos adicionais para aumentar a clareza, simplicidade ou poder de um programa geralmente é algo positivo.

Por outro lado, evitar a criação de objetos mantendo seu próprio pool de objetos é uma má ideia, a menos que os objetos no pool sejam extremamente pesados. O clássico

Um exemplo de objeto que justifica um pool de objetos é uma conexão com banco de dados. O custo de estabelecer a conexão é suficientemente alto para que faça sentido reutilizar esses objetos. De modo geral, porém, manter seu próprio pool de objetos atrapalha seu código, aumenta a pegada de memória e prejudica o desempenho. Implementações modernas de JVM possuem coletores de lixo altamente otimizados que superam facilmente esses pools de objetos em objetos leves.

O contraponto a esse item é o item 50 sobre cópia defensiva. O item apresentado diz: "Não crie um novo objeto quando você deveria reutilizar um existente", enquanto o Item 50 diz: "Não reutilize um objeto existente quando você deveria criar um novo." Note que a penalidade por reutilizar um objeto quando é exigida cópia defensiva é muito maior do que a penalidade por criar desnecessariamente um objeto duplicado. Não fazer cópias defensivas quando necessário pode levar a bugs insidiosos e falhas de segurança; criar objetos afeta desnecessariamente apenas o estilo e a performance.

Item 7: Eliminar referências obsoletas a objetos

Se você mudasse de uma linguagem com gerenciamento manual de memória, como C ou C++, para uma linguagem de coleta de lixo como Java, seu trabalho como programador ficava muito mais fácil pelo fato de que seus objetos eram automaticamente recuperados quando você terminava de usá-los. Parece quase mágica quando você experimenta pela primeira vez. Isso pode facilmente dar a impressão de que você não precisa pensar em gerenciamento de memória, mas isso não é totalmente verdade.

Considere a seguinte implementação simples de pilha:

```
Você consegue identificar o "vazamento
de memória"? classe pública Stack {
    elementos privados de Objeto[] private int size = 0; Int.
    final estática privada DEFAULT_INITIAL_CAPACITY = 16;

    public Stack() {
        elementos = novo Objeto[DEFAULT_INITIAL_CAPACITY];

        empty public push(Object e) {
            ensureCapacity();
            elementos[size++] = e;}

    Objeto público pop() {
        if (tamanho == 0)
            lançar um novo EmptyStackException();
        return elementos[--size];}

    /**
     * Garantir espaço para pelo menos mais um elemento,
     * aproximadamente* dobrando a capacidade cada vez que o array
     * precisar crescer.*/private void ensureCapacity() {

        if (elements.length == tamanho)
            elements = Arrays.copyOf(elements, 2 * tamanho + 1);
    }}
```

Não há nada obviamente errado com esse programa (mas veja o item 29 para a versão genérica). Você poderia testá-lo exaustivamente, e ele passaria em todos os testes com excelência, mas há um problema à espreita. De forma geral, o programa possui um "vazamento de memória", que pode se manifestar silenciosamente como desempenho reduzido devido a

aumento da atividade dos coletores de lixo ou maior consumo de memória. Em casos extremos, tais vazamentos de memória podem causar paginação de disco e até falhas de programa com um `OutOfMemoryError`, mas tais falhas são relativamente raras.

Então, onde está o vazamento de memória? Se uma pilha cresce e depois encolhe, os objetos que foram retirados da pilha não serão coletados pelo lixo, mesmo que o programa que usa a pilha não tenha mais referências a eles. Isso ocorre porque a pilha mantém referências obsoletas a esses objetos. Uma referência obsoleta é simplesmente uma referência que nunca mais será desreferenciada. Nesse caso, quaisquer referências fora da "porção ativa" do array de elementos são obsoletas. A porção ativa consiste nos elementos cujo índice é menor que o tamanho.

Vazamentos de memória em linguagens de coleta de lixo (mais propriamente conhecidos como retenções de objetos não intencionais) são insidiosos. Se uma referência a objeto for mantida involuntariamente, não apenas esse objeto é excluído da coleta de lixo, mas também quaisquer objetos referenciados por ele, e assim por diante. Mesmo que apenas algumas referências a objetos sejam retidas involuntariamente, muitos, muitos objetos podem ser impedidos de serem coletados pelo lixo, com efeitos potencialmente grandes no desempenho.

A solução para esse tipo de problema é simples: anular referências quando elas se tornam obsoletas. No caso da nossa classe `Stack`, a referência a um item se torna obsoleta assim que ele sai da pilha. A versão corrigida do `popmethod` é assim:

```
Objeto público pop() {
    if (tamanho == 0)
        lançar um novo EmptyStackException();
    Resultado do objeto =
        elementos[--tamanho]; elementos[tamanho] = nulo; Eliminar o
        resultado obsoleto de referência;}

```

Um benefício adicional de anular referências obsoletas é que, se elas forem subsequentemente desreferenciadas por engano, o programa falhará imediatamente com `aNullPointerException`, em vez de fazer a coisa errada silenciosamente. É sempre benéfico detectar erros de programação o mais rápido possível.

Quando os programadores são prejudicados pela primeira vez por esse problema, podem compensar demais anulando todas as referências de objeto assim que o programa termina de usá-las. Isso não é nem necessário nem desejável; Isso atrapalha o programa desnecessariamente. Anular referências a objetos deveria ser a exceção, não a norma. A melhor maneira de eliminar uma referência obsoleta é deixar que a variável que continha a referência saia do escopo. Isso ocorre naturalmente se você definir cada variável no escopo mais restrito possível (Item 57).

Então, quando você deve anular uma referência? Qual aspecto da classe `Stack` a torna suscetível a vazamentos de memória? Simplificando, ele gerencia sua própria memória. O pool de armazenamento consiste nos elementos do array de elementos (as células de referência de objetos, não os próprios objetos). Os elementos na porção ativa do `thearray` (conforme definido anteriormente) são alocados, e os do restante do array são livres. O coletor de lixo não tem como saber disso; Para o coletor de lixo, todas as referências de objetos no array de elementos são igualmente válidas. Só o programador sabe que a parte inativa do array não é importante. A programação comunica efetivamente esse fato ao coletor de lixo ao anular manualmente os elementos do array assim que eles se tornam parte da parte inativa.

De modo geral, sempre que uma classe gerencia sua própria memória, o programático deve ficar atento a vazamentos de memória. Sempre que um elemento é liberado, quaisquer referências a objeto contidas nele devem ser anuladas.

Outra fonte comum de vazamentos de memória são os caches. Depois que você coloca uma referência de objeto em um cache, é fácil esquecer que ela está lá e deixá-la no cache muito depois de se tornar irrelevante. Existem várias soluções para esse problema. Se você tiver a sorte de implementar um cache para o qual uma entrada é exatamente relevante, desde que haja referências à sua chave fora do cache, represente o cache como um `WeakHashMap`; As inscrições serão removidas automaticamente assim que se tornarem obsoletas. Lembre-se de que o `WeakHashMap` é útil apenas se o tempo de vida desejado das entradas do cache for determinado por referências externas à chave, e não pelo valor.

Mais comumente, a vida útil de uma entrada de cache é menos bem definida, as entradas de `withentries` se tornando menos valiosas com o tempo. Nessas circunstâncias, o cache deve ser ocasionalmente limpo de entradas que caíram em desuso. Isso pode ser feito por um thread em segundo plano (talvez um `ScheduledThreadPoolExecutor`) ou como efeito adicional de adicionar novas entradas ao cache. A classe `LinkedHashMap` facilita a segunda abordagem com seu método `removeEldestEntry`. Para caches mais sofisticados, talvez você precise usar `java.lang.ref` diretamente.

Uma terceira fonte comum de vazamentos de memória são ouvintes e outros retornos de chamada. Se você implementar uma API onde os clientes registram `callbacks`, mas não os desregistram explicitamente, eles vão se acumular a menos que você tome alguma atitude. Uma forma de garantir que os `callbacks` sejam coletados rapidamente é armazenar apenas referências fracas a eles, por exemplo, armazenando-as apenas como chaves em um `WeakHashMap`.

Como vazamentos de memória normalmente não se manifestam como falhas óbvias, eles podem permanecer presentes em um sistema por anos. Eles normalmente são descobertos apenas como resultado de uma inspeção cuidadosa do código ou com o auxílio de uma ferramenta de `debugging` conhecida como `heap profiler`. Portanto, é muito desejável aprender a antecipar problemas como esse antes que ocorram e evitar que aconteçam.

Item 8: Evite finalizadores e limpadores

Finalizadores são imprevisíveis, muitas vezes perigosos e geralmente desnecessários. Seu uso pode causar comportamentos erráticos, baixo desempenho e problemas de portabilidade. Finalizadores têm alguns usos válidos, que abordaremos mais adiante neste item, mas, como regra, você deve evitá-los. A partir do Java 9, finalizadores foram descontinuados, mas ainda são usados pelas bibliotecas do Java. O substituto do Java 9 para finalizers iscleaners. Limpadores são menos perigosos que finalizadores, mas ainda são imprevisíveis, lentos e geralmente desnecessários.

Programadores de C++ são alertados a não pensar em finalizadores ou limpadores como análogos de Java dos destruidores C++. Em C++, destrutores são a forma normal de recuperar os recursos associados a um objeto, um contraponto necessário aos construtores. Em Java, o coletor de lixo recupera o armazenamento associado a um objeto quando ele se torna inacessível, sem necessidade de esforço especial por parte do programador. Destrutores em C++ também são usados para recuperar outros recursos que não sejam memória. Em Java, `try-with-resources` ou `try-finally` block é usado para esse propósito (Item 9).

Uma falha dos finalizadores e limpadores é que não há garantia de que serão executados prontamente [JLS, 12.6]. Pode levar um tempo arbitrário entre o momento em que um objeto se torna inacessível e o momento em que seu finalizador ou limpador é executado. Isso significa que você nunca deve fazer nada que precise de tempo em um finalizador ou limpeza. Por exemplo, é um erro grave depender de um finalizador ou limpador para fechar arquivos, porque descritores de arquivos abertos são um recurso limitado. Se muitos arquivos forem deixados abertos devido à demora do sistema na execução de finalizadores ou limpadores, um programa pode falhar porque não consegue mais abrir arquivos.

A rapidez com que finalizadores e limpadores são executados depende principalmente do algoritmo de coleta de lixo, que varia bastante entre as implementações. O comportamento de um programa que depende da rapidez da execução do finalizador ou do limpador também pode variar. É totalmente possível que tal programa rode perfeitamente na JVM onde você o testa e depois fracasse miseravelmente naquela preferida pelo seu cliente mais importante.

A finalização tardia não é apenas um problema teórico. Fornecer um finalizador para uma classe pode atrasar arbitrariamente a recuperação de suas instâncias. Um colega depurou uma aplicação de interface gráfica em execução que estava misteriosamente morrendo com um erro de Memória Fora de Memória. A análise revelou que, no momento de sua morte, o aplicativo tinha milhares de objetos gráficos em sua fila de finalizador apenas esperando para serem finalizados e recuperados. Infelizmente, o thread finalizer estava rodando com prioridade menor do que outro thread de aplicação, então os objetos não eram finalizados na taxa em que eles se tornavam elegíveis para finalização. A especificação da linguagem não garante-

É claro qual thread executará finalizadores, então não há uma maneira portátil de evitar esse tipo de problema além de evitar usar finalizadores. Cleaners são um pouco melhores que finalizers nesse aspecto porque os autores de classe têm controle sobre seus próprios threads de limpeza, mas os cleaners ainda rodam em segundo plano, sob o controle do coletor de lixo, então não há garantia de limpeza rápida.

A especificação não só não garante que finalizadores ou limpadores funcionarão rapidamente; Não oferece nenhuma garantia de que eles vão funcionar. É totalmente possível, até provável, que um programa termine sem executá-los em alguns objetos que não são mais acessíveis. Como consequência, você nunca deve depender de um finalizador ou limpador para atualizar o estado persistente. Por exemplo, depender de um finalizador ou limpador liberar um travamento persistente em um recurso compartilhado, como um banco de dados, é uma boa forma de paralisar todo o seu sistema distribuído.

Não se deixe seduzir pelos métodos `System.gc` e `System.runFinalization`. Eles podem aumentar as chances de finalizadores ou limpadores serem executados, mas não garantem isso. Dois métodos já afirmaram garantir isso: `System.runFinalizersOnExit` e seu irmão maligno, `Runtime.runFinalizersOnExit`. Esses métodos são fatalmente falhos e estão obsoletos há décadas [ThreadStop].

Outro problema com finalizadores é que uma exceção não capturada lançada durante a finalização é ignorada, e a finalização desse objeto termina [JLS, 12.6]. Exceções não capturadas podem deixar outros objetos em estado corrompido. Se outro thread tentar usar tal objeto corrompido, pode resultar de um comportamento arbitrário não determinístico. Normalmente, uma exceção não capturada termina a thread e imprime um stacktrace, mas não se isso ocorrer em um finalizador — nem mesmo imprime um aviso. Os limpadores não têm esse problema porque uma biblioteca que usa um limpador tem controle sobre sua thread.

Há uma penalidade severa de desempenho ao usar finalizadores e limpadores. Na minha máquina, o tempo para criar um objeto `AutoFechable` simples, fechá-lo usando `try-with-resources`, e para o coletor de lixo recuperá-lo é cerca de 12 ns. Usar um finalizador aumenta o tempo para 550 ns. Em outras palavras, é cerca de 50 vezes mais lento criar e destruir objetos com finalizadores. Isso ocorre principalmente porque os finalizadores dificultam a coleta eficiente de lixo. Os limpadores são comparáveis em velocidade aos finalizadores se você usar para limpar todas as instâncias da classe (cerca de 500 ns por instância na minha máquina), mas os limpadores são muito mais rápidos se você os usar apenas como rede de segurança, como discutido abaixo. Nessas circunstâncias, criar, limpar e destruir um objeto consome cerca de 66 ns na minha máquina, o que significa que você paga um fator de cinco (não cinquenta) pelo seguro de uma rede de segurança se não usá-la.

Finalizadores têm um sério problema de segurança: eles expõem sua classe a ataques de finalizadores. A ideia por trás de um ataque finalizador é simples: se uma exceção for

lançado de um construtor ou de seus equivalentes de serialização—os métodos `readObject` e `readResolve` (Capítulo 12)—o finalizador de uma subclasse maliciosa pode anular o objeto parcialmente construído que deveria ter "morrido na trepadeira". Esse finalizador pode registrar uma referência ao objeto em um campo estático, evitando que ele seja coletado como lixo. Uma vez que o objeto malformado foi registrado, é simples invocar métodos arbitrários nesse objeto que nunca deveriam ter sido permitidos em primeiro lugar. Lançar uma exceção de um construtor deve ser suficiente para impedir que um objeto apareça; Na presença dos finalizadores, não é. Tais ataques podem ter consequências graves. Classes finais são imunes a ataques de finalizador porque ninguém pode escrever uma subclasse maliciosa de uma classe final. Para proteger classes não finais de ataques de finalizador, escreva um método final de finalização que não faça nada.

Então, o que você deveria fazer em vez de escrever um finalizador ou limpador para uma classe cujos objetos encapsulam recursos que precisam de terminação, como arquivos ou threads? Basta fazer sua classe implementar o `AutoCloseable` e exigir que seus clientes invoquem o método de fechamento em cada instância quando não for mais necessário, normalmente usando `try-with-resources` para garantir a terminação mesmo diante de exceções (Item 9). Um detalhe que vale mencionar é que a instância deve acompanhar se foi fechada: o método de fechamento deve registrar em um campo que o objeto não é mais válido, e outros métodos devem verificar esse campo e lançar um `UnlawfulStateException` se forem chamados após o fechamento do objeto.

Então, para que, se é que servem para algo, limpadores e finalizadores? Eles talvez tenham dois usos legítimos. Uma delas é atuar como uma rede de segurança caso o proprietário de um recurso não chame seu método de encerramento. Embora não haja garantia de que o limpador ou finalizador vá funcionar prontamente (ou sequer executar), é melhor liberar o recurso atrasado do que nunca, caso o cliente não o faça. Se você está pensando em escrever um finalizador de rede de segurança assim, pense bem se a proteção vale o custo. Algumas classes `Javalibrary`, como `FileInputStream`, `FileOutputStream`, `ThreadPoolExecutor` e `java.sql.Connection`, possuem finalizadores que servem como redes de segurança.

Um segundo uso legítimo de limpadores diz respeito a objetos com pares nativos. Um par nativo é um objeto nativo (não Java) ao qual um objeto normal delega via métodos nativos. Como um par nativo não é um objeto normal, o coletor de lixo não sabe sobre ele e não pode recuperá-lo quando seu par Java for recuperado. Um limpador ou finalizador podem ser um veículo adequado para essa tarefa, assumindo que o desempenho seja aceitável e que o par nativo não possua recursos críticos. Se o desempenho for inaceitável ou o colega nativo tiver recursos que devem reclamar prontamente, a turma deve ter um método próximo, como descrito anteriormente.

Limpadores são um pouco complicados de usar. Abaixo está uma aula simples de sala demonstrando a instalação. Vamos supor que os cômodos precisam ser limpos antes de serem recuperados. A classe `Room` implementa o `AutoCloseable`; O fato de sua rede de segurança de limpeza automática usar um limpador é apenas um detalhe de implementação. Ao contrário dos finalizadores, os limpadores não poluem a API pública de uma classe:

```

Uma classe autofechable usando um limpador como rede de
segurança (sala de classe pública implementa AutoCloseable {
    estático privado final Cleaner cleaner = Cleaner.create();

    Recurso que precisa de limpeza. Não deve se referir à classe
    estática Room!private State implementa Runnable {
        int numJunkPiles; Quantidade de pilhas de sucata neste quarto

        State(int numJunkPiles) {
            this.numJunkPiles = numJunkPiles;}

        Invocado por método fechado ou
        cleaner@Override run de void público() {
            system.out.println ("Sala de limpeza");
            numJunkPiles = 0;}}

    O estado deste quarto, compartilhado com nosso estado
    final privado para limpeza e estado;

    Nosso limpável. Limpa o quarto quando ele é elegível para o
    GCPRIVATE final Cleaner. Limpável limpável;

    Public Room(int numJunkPiles) {
        estado = novo estado(numJunkPiles); limpável =
        cleaner.register(isto, estado);}

    @Override público nulo fechar() {
        cleanable.clean();}}

```

A classe `State` estática e aninhada contém os recursos necessários para o faxineiro para limpar o ambiente. Nesse caso, é simplesmente o campo `numJunkPiles`, que representa a quantidade de bagunça na sala. Mais realisticamente, pode ser um `final long` que contém um ponteiro para um par nativo. O estado implementa o `Runnable`, e o método `itsrun` é chamado no máximo uma vez, pelo `Cleanable` que obtemos quando registramos nossa instância de Estado com nosso `cleaner` no construtor `Room`. A chamada para o `runMethod` será acionada por uma de duas coisas: Normalmente, ela é acionada por uma chamada para

O método de fechar do quarto chama o método limpo do Cleanable. Se o cliente falhar em chamar o método de fechamento até que uma instância de Sala seja elegível para coleta de lixo, o limpador (esperançosamente) chamará o método run do Estado.

É fundamental que uma instância de Estado não se refira à sua instância de Sala. Se acontecesse, criaria uma circularidade que impediria que a instância da Sala se tornasse elegível para coleta de lixo (e de ser limpa automaticamente). Portanto, o Estado deve ser uma classe aninhada estática porque classes aninhadas não estáticas contêm referências às suas instâncias anexas (Item 24). Também é desaconselhável usar um lambda, pois eles podem facilmente capturar referências a objetos que o cercam.

Como dissemos antes, a faxineira do Room é usada apenas como rede de segurança. Se os clientes cercarem todas as instâncias de Room em blocos de tentativa com recursos, a limpeza automática nunca será necessária. Esse cliente bem comportado demonstra que o comportamento:

```
classe pública Adulto {
    public static void main(String[] args) {
        try (Room myRoom = new Room(7)) {
            System.out.println("Adeus");
        }
    }
}
```

Como era de se esperar, ao rodar o programa para adultos, aparece Adeus, seguido de limpeza da sala. Mas e quanto a esse programa mal-comportado, que nunca limpa seu quarto?

```
classe pública Adolescente {
    public static void main(String[] args) {
        novo quarto (99);
        System.out.println("Paz");
    }
}
```

Você poderia esperar que imprimisse Paz, seguido de Sala de limpeza, mas no meu computador nunca imprime Sala de limpeza; simplesmente sai. Essa é a imprevisibilidade de que falamos antes. A especificação do Cleaner diz: "O comportamento dos cleaners durante o Sys-tem.exit é específico da implementação. Não há garantias sobre se ações de limpeza são invocadas ou não." Embora a especificação não diga isso, o mesmo vale para a saída normal do programa. Na minha máquina, adicionar a linha System.gc() ao método principal do Teenager é suficiente para fazer ele imprimir Limpeza da sala antes de sair, mas não há garantia de que você verá o mesmo comportamento na sua máquina.

Em resumo, não use cleaners, ou em versões anteriores ao Java 9, finalizadores, exceto como rede de segurança ou para terminar recursos nativos não críticos. Mesmo assim, cuidado com a indeterminação e as consequências de desempenho.

Item 9: Prefira tentar com recursos a tentar finalmente

As bibliotecas Java incluem muitos recursos que precisam ser fechados manualmente invocando um método de fechamento. Exemplos incluem `InputStream`, `OutputStream` e `java.sql.Connection`. Fechar recursos muitas vezes é negligenciado pelos clientes, com consequências previsíveis de desempenho graves. Embora muitos desses recursos usem finalizadores como rede de segurança, eles não funcionam muito bem (Item 8).

Historicamente, uma declaração `try-finally` era a melhor forma de garantir que um recurso fosse fechado corretamente, mesmo diante de uma exceção ou retorno:

```
try-finally - Não é mais a melhor forma de fechar resources!static
String firstLineOfFile(String path) lança IOException {
    BufferedReader br = novo BufferedReader(novo
        FileReader(caminho)); try {
        return br.readLine();}
    finalmente {
        br.close();}}
```

Isso pode não parecer ruim, mas piora quando você adiciona um segundo recurso:

```
try-finally é feio quando usado com mais de um recurso!static void
copy(String src, String dst) lança IOException {
    InputStream in = novo FileInputStream(src);
    try {
        OutputStream out = novo FileOutputStream(dst);
        try {
            byte[] buf = novo byte[BUFFER_SIZE];
            int n; enquanto ((n = in.read(buf)) >=
                0)
                out.write(buf, 0, n);
        } finalmente {
            out.close();}}
    finalmente {
        in.close();}}
```

Pode ser difícil de acreditar, mas até bons programadores erravam na maioria das vezes. Para começar, errei na página 88 do *Java Puzzlers* [Bloch05], e ninguém percebeu por anos. Na verdade, dois terços dos usos do método `close` nas bibliotecas Java estavam errados em 2007.

Até mesmo o código correto para fechar recursos com instruções try-finally, como ilustrado nos dois exemplos anteriores, apresenta uma deficiência sutil. O código tanto no try block quanto no finally block é capaz de gerar exceções. Por exemplo, no método `firstLineOfFile`, a chamada para `readLine` pode lançar uma exceção devido a uma falha no dispositivo físico subjacente, e a chamada para fechar pode então falhar pelo mesmo motivo. Nessas circunstâncias, a segunda exceção oblitera completamente a primeira. Não há registro da primeira exceção no rastreamento da pilha de exceções, o que pode complicar muito a depuração de sistemas. Inreal — geralmente é a primeira exceção que você quer ver para diagnosticar o problema. Embora seja possível escrever código para suprimir a segunda exceção em favor da primeira, praticamente ninguém fez isso porque é muito prolixo.

Todos esses problemas foram resolvidos de uma vez só quando o Java 7 introduziu a instrução try-with-resources [JLS, 14.20.3]. Para ser utilizável com essa construção, um recurso deve implementar a interface `AutoCloseable`, que consiste em um único método de fechamento com retorno de vazio (`void-return`). Muitas classes e interfaces nas bibliotecas Java e em bibliotecas de terceiros agora implementam ou estendem o `AutoCloseable`. Se você escrever uma classe que representa um recurso que precisa ser fechado, sua classe também deve implementar `AutoCloseable`.

Veja como nosso primeiro exemplo é usando try-with-resources:

```
try-with-resources - a melhor maneira de fechar resources!static
String firstLineOfFile(String path) throws IOException {
    try (BufferedReader br = new BufferedReader(
        novo FileReader(caminho))) {return
        br.readLine();}}
```

E aqui está como nosso segundo exemplo é usando try-with-resources:

```
try-with-resources em múltiplos recursos - short e sweetstatic
void copy(String src, String dst) throws IOException {
    try (InputStream in = novo FileInputStream(src);
        OutputStream out = novo FileOutputStream(dst))
        {byte[] buf = novo byte[BUFFER_SIZE]; int n;
        enquanto ((n = in.read(buf)) >= 0)

        out.write(buf, 0, n);
    }}
```

Além de serem mais curtas e legíveis que as versões de usar recursos são mais curtas e fáceis de ler do que as originais, como também oferecem diagnósticos muito melhores. Considere o `firstLineOfFile`

método. Se exceções forem lançadas tanto pela chamada `readLine` quanto pelo fechamento (invisível), a última exceção é suprimida em favor da primeira. Na verdade, múltiplas exceções podem ser suprimidas para preservar a exceção que você realmente quer ver. Essas exceções suprimidas não são simplesmente descartadas; Eles estão impressos no traço da pilha com uma anotação dizendo que foram suprimidos. Você também pode acessá-los programaticamente com o método `getSuppressed`, que foi adicionado ao `Throwable` no Java 7.

Você pode colocar cláusulas `catch` em instruções `try-with-resources`, assim como você canon nas instruções `try-finally` regulares. Isso permite que você lide com exceções sem manchar seu código com outra camada de aninhamento. Como um exemplo um pouco forçado, aqui está uma versão do nosso método `firstLineOfFile` que não lança exceções, mas recebe um valor padrão para retornar se não conseguir abrir o arquivo ou ler dele:

```
try-with-resources com uma cláusula catch
String static String
firstLineOfFile(String path, String defaultVal) {
    try (BufferedReader br = new BufferedReader(
        new FileReader(path))) {return
        br.readLine();} catch (IOException
        e) {
    return defaultVal;}}
```

A lição é clara: sempre use `try` com recursos em vez de tentar finalmente ao trabalhar com recursos que precisam ser fechados. O código resultante é mais curto e claro, e as exceções que ele gera são mais úteis. A instrução `try-with-resources` facilita escrever código correto usando recursos que precisam ser fechados, o que era praticamente impossível usando `try-finally`.

CHAPTER 3

Métodos Comuns a Todos os Objetos

EMBORA `Object` seja uma classe concreta, ela é projetada principalmente para extensão.

Todos os seus métodos não finais (`equals`, `hashCode`, `toString`, `clone` e `finalize`) possuem contratos gerais explícitos porque são projetados para serem sobrepostos. É responsabilidade de qualquer classe que sobreponha esses métodos obedecer aos seus contratos gerais; a falha em fazer isso impedirá que outras classes que dependem dos contratos (tais como `HashMap` e `HashSet`) funcionem corretamente em conjunto com a classe.

Este capítulo explica quando e como sobrescrever os métodos `Object` não finais. O método de finalização é omitido neste capítulo porque foi discutido no Item 8. Embora não seja um método `Object`, `Comparable.compareTo` é discutido neste capítulo porque tem um caráter semelhante.

Item 10: Obedecer ao contrato geral quando prevalecer equivale

Anular o método de `equals` parece simples, mas existem muitas maneiras de errar, e as consequências podem ser graves. A maneira mais fácil de evitar problemas é não sobrepor o método dos iguais, caso em que cada instância da classe é igual apenas a si mesma. Essa é a coisa certa a fazer se qualquer uma das seguintes condições se aplicar:

- Cada instância da classe é inerentemente única. Isso é verdade para classes como `Thread` que representam entidades ativas em vez de valores. A implementação de iguais fornecida pelo `Object` tem exatamente o comportamento correto para essas classes.
- Não há necessidade de a turma oferecer um teste de "igualdade lógica". Por exemplo, `java.util.regex.Pattern` poderia ter sobrescrito iguais para verificar se duas instâncias de `Pattern` representavam exatamente a mesma expressão regular, mas os projetistas não achavam que os clientes precisariam ou desejariam essa funcionalidade. Nessas circunstâncias, a implementação de iguais herdada do `Object` é ideal.

- Uma superclasse já sobrescreveu igual a uma das outras, e o comportamento da superclasse é apropriado para essa classe. Por exemplo, a maioria das implementações de Set herda sua implementação igual a AbstractSet, implementações List de AbstractList e implementações Map de AbstractMap.
- A classe é privada ou pacote privada, e você tem certeza de que seu método igualitário nunca será invocado. Se você é extremamente avesso ao risco, pode ignorar o método de equals para garantir que ele não seja invocado acidentalmente:

```
@Override booleano público é igual a (Object o) {
    lançar um novo AssertionError(); Método nunca é chamado}
```

Então, quando é apropriado sobrepor iguais? É quando uma classe tem uma noção de igualdade lógica que difere da mera identidade do objeto e uma superclasse ainda não sobrescreveu iguais a outros. Isso geralmente acontece com classes de valor. Uma classe de valor é simplesmente uma classe que representa um valor, como Inteiro ou String. Um programador que compara referências a objetos de valor usando o método de igualdade espera descobrir se eles são logicamente equivalentes, e não se se referem ao mesmo objeto. Não só é necessário sobrepor o método equals para satisfazer as expectativas do programador, como permite que instâncias sirvam como chaves de mapas ou setelements com comportamento previsível e desejável.

Um tipo de classe de valor que não exige que o método de equals seja sobrescrito é uma classe que usa controle de instância (Item 1) para garantir que, no máximo, exista um objeto com cada valor. Tipos de enum (Item 34) se enquadram nessa categoria. Para essas classes, igualdade lógica é igual à identidade do objeto, então Object's é igual a metodofunções como um método lógico é igual a método.

Quando você sobrepor o método dos iguais, deve seguir seu contrato geral. Aqui está o contrato, da especificação para o Objeto:

O método de iguais implementa uma relação de equivalência. Ele possui as seguintes propriedades:

- Reflexivo: Para qualquer valor de referência não nulo x, x.igual(x) deve retornar verdadeiro.
- Simétrico: Para quaisquer valores de referência não nulos x e y, x.igual(y) deve voltar a ser verdadeiro se e somente se y.igual a (x) retorna verdadeiro.
- Transitivo: Para quaisquer valores de referência não nulos x, y, z, se x.igual(y) retorna verdadeiro e y.igual (z) retorna verdadeiro, então x.igual(z) deve devolver verdadeiro.
- Consistente: Para quaisquer valores de referência não nulos x e y, múltiplas invocações de x.equals(y) devem devolver consistentemente true ou consistentemente false, desde que nenhuma informação usada nas comparações de equals seja modificada.
- Para qualquer valor de referência não nulo x, x.igual(nulo) deve devolver falso.

A menos que você tenha inclinação matemática, isso pode parecer um pouco assustador, mas não ignore! Se você violar essa informação, pode acabar se comportando de forma serrática ou travando, e pode ser muito difícil identificar a origem da falha.

Parafraseando John Donne, nenhuma classe é uma ilha. Instâncias de uma classe são frequentemente passadas para outra. Muitas classes, incluindo todas as classes de coleções, dependem de os objetos passados a elas obedecerem ao contrato de equals.

Agora que você está ciente dos perigos de violar o contrato de equals, vamos analisar o contrato em detalhes. A boa notícia é que, apesar das aparências, realmente não é muito complicado. Depois que você entende, não é difícil seguir o que ele tem.

Então, o que é uma relação de equivalência? De forma geral, é um operador que particiona um conjunto de elementos em subconjuntos cujos elementos são considerados iguais entre si. Esses subconjuntos são conhecidos como classes de equivalência. Para que um método de igual seja útil, todos os elementos de cada classe de equivalência devem ser intercambiáveis do ponto de vista do usuário. Agora vamos examinar os cinco requisitos por vez:

Reflexividade — O primeiro requisito diz apenas que um objeto deve ser igual a si mesmo. É difícil imaginar violar essa questão sem querer. Se você vio-late e depois adicionasse uma instância da sua classe a uma coleção, o método contains poderia muito bem dizer que a coleção não continha a instância que você acabou de adicionar.

Simetria — O segundo requisito diz que quaisquer dois objetos devem concordar sobre se são iguais. Diferente do primeiro requisito, não é difícil imaginar que este seja um vídeo sem querer. Por exemplo, considere a seguinte classe, que implementa uma cadeia insensível a maiúsculas e minúsculas. O caso da string é preservado por toString, mas ignorado em comparações iguais a isso:

```
Quebrado - viola simetria! classe final
pública CaseInsensitiveString {
    cordas finais privadas;

    public CaseInsensitiveString(String s) {
        this.s = Objects.requireNonNull(s);}

    Quebrado - viola simetria! @Override booleano
    público é igual a (Objeto o) {
        if (o instanceof CaseInsensitiveString)
            return s.equalsIgnoreCase(
                ((StringInsensível a Caso O).s);
        if (o instanceof String) Interoperabilidade unidirecional!
        return s.equalsIgnoreCase((String) o); return
        false;}...// Restante omitido}
```

O método bem-intencionado dos iguais nesta classe tenta ingenuamente interoperar com cordas comuns. Vamos supor que tenhamos uma corda insensível a maiúsculas e minúsculas e uma comum:

```
CaseInsensitiveString cis = nova CaseInsensitiveString("Polish");  
Strings s = "polish";
```

Como esperado, `cis.equals(s)` retorna verdadeiro. O problema é que, enquanto o método dos iguais em `String` é alheio a cadeias insensíveis a maiúsculas, o método dos iguais em `String` é alheio a cadeias insensíveis a maiúsculas. Portanto, `s.igual(cis)` retorna falso, uma clara violação da simetria. Suponha que você coloque uma string insensível a maiúsculas minúsculas em uma coleção:

```
<CaseInsensitiveString> lista = nova ArrayList<>  
( ); lista.add(cis);
```

O que o `list.contains(s)` retorna neste ponto? Vai saber? Na implementação atual do OpenJDK, ele retorna falso, mas isso é apenas um artefato de implementação. Em outra implementação, ele poderia facilmente retornar `true` ou lançar uma exceção em tempo de execução. Depois de violar o contrato de igualdade, simplesmente não sabe como outros objetos vão se comportar quando confrontados com o seu.

Para eliminar o problema, basta remover a tentativa mal concebida de interoperar com `String` do método de `equals`. Depois disso, você pode refatorar o método em uma única declaração de retorno:

```
@Override booleano público é igual a (Object o) {  
    return o instanceof CaseInsensitiveString &&  
        ((StringInsensívelCase) o).s.igualIgnorarCaso(s);  
}
```

Transitividade — O terceiro requisito do contrato de igual diz que, se um objeto é igual a um segundo e o segundo objeto é igual a um terceiro, então o primeiro objeto deve ser igual ao terceiro. Novamente, não é difícil imaginar violar esse requisito sem querer. Considere o caso de uma subclasse que adiciona um novo componente de valor à sua superclasse. Em outras palavras, a subclasse adiciona uma parte de

informação que afeta equivale a comparações. Vamos começar com uma classe simples de ponto inteiro bidimensional imutável:

```

classe pública Point
{private final int x;
    Private Final Int Y;

    public Point(int x, int y) {
        this.x =
        x;this.y = y;}

    @Override booleano público é igual a (Object o) {
        se (!( o instância de Point))
        retorne falso; Ponto p = (Ponto)o;
        return p.x == x && p.y == y;}

    ... Restante omitido
}

```

Suponha que você queira estender essa classe, adicionando a noção de cor a um ponto:

```

classe pública ColorPoint estende Point {
    cor final privada;

    public ColorPoint(int x, int y, Color color) {
        super(x, y); this.color =
        color;}

    ... Restante omitido
}

```

Como deve ser o método de igualdade? Se você deixar de fora completamente, a implementação é herdada do Point e a informação de cor é ignorada nas comparações iguais. Embora isso não viole o contrato de iguais, é claramente inaceitável. Suponha que você escreva um método de iguais que retorna verdadeiro apenas se seu argumento for outro ponto de cor com a mesma posição e cor:

```

Quebrado - viola simetria!@Override booleano
público é igual a (Objeto o) {
    se (!( o instância do ColorPoint))
    retorne falso; return super.equals(o) && ((ColorPoint) o).color ==
    color;}

```

O problema desse método é que você pode obter resultados diferentes ao comparar um ponto com um ponto de cor e vice-versa. A primeira comparação ignora a cor, enquanto a segunda sempre retorna falsa porque o tipo do argumento está incorreto. Para tornar isso concreto, vamos criar um ponto e um ponto de cor:

```
Ponto p = novo Ponto(1, 2); ColorPoint cp = novo
ColorPoint(1, 2, Color.RED);
```

Então `p.igual(cp)` retorna verdadeiro, enquanto `cp.igual(p)` retorna falso. Você pode tentar resolver o problema fazendo com que o `ColorPoint.equals` ignore a cor ao fazer "comparações mistas":

```
Quebrado - viola a transitividade!@Override
booleano público é igual a (Objeto o) {
    se (!( o instância de Point))
        retorne falso;

    Se o for um ponto normal, faça uma comparação para
    daltônico (!( o instância do ColorPoint))
        retorno igual a (isto);

    o é um ColorPoint; faça uma comparação completa: retorne
    super.equals(o) && ((ColorPoint) o).color == color;}
```

Essa abordagem fornece simetria, mas às custas da transitividade:

```
ColorPoint p1 = novo ColorPoint(1, 2, Color.RED);
Ponto p2 = novo Ponto(1, 2); ColorPoint p3 = novo
ColorPoint(1, 2, Color.BLUE);
```

Agora `p1.igual(p2)` e `p2.igual(p3)` retornam verdadeiro, enquanto `p1.igual(p3)` retorna falso, uma clara violação da transitividade. As duas primeiras comparações são "daltônicas", enquanto a terceira leva em conta a cor.

Além disso, essa abordagem pode causar recursão infinita: suponha que existam duas subclasses de `Point`, digamos `ColorPoint` e `SmellPoint`, cada uma com esse método meio que igual. Depois, uma chamada para `myColorPoint.equals(mySmellPoint)` gerará um `StackOverflowError`.

Então, qual é a solução? Acontece que esse é um problema fundamental das relações de equivalência em linguagens orientadas a objetos. Não há como estender uma classe instantiável e adicionar um componente de valor preservando o `equalsContract`, a menos que você esteja disposto a abrir mão dos benefícios da abstração orientada a objetos.

Você pode ouvir dizer que é possível estender uma classe instanciada e adicionar um `valuecomponent` preservando o contrato `equals` usando um teste `getClass` no lugar do teste `instanceof` no método `equals`:

```
Quebrado - viola o princípio de substituição de Liskov (página
43)@Override booleano público é igual a (Objeto o) {
    if (o == null || o.getClass() != getClass())
        retorne falso; Ponto p = (Ponto)
o; return p.x == x && p.y == y;}
```

Isso tem o efeito de igualar objetos apenas se eles possuem a mesma classe de implementação. Isso pode não parecer tão ruim, mas as consequências são inaceitáveis: uma instância de uma subclasse de `Ponto` ainda é um `Ponto`, e ainda precisa funcionar como tal, mas falha se você adotar essa abordagem! Vamos supor que queiramos escrever um método para saber se um ponto está no círculo unitário. Aqui está uma forma de fazermos isso:

```
Inicialize unidadeCirculo para conter todos os Pontos na unidade
circuloPrivado static final Set<Point> unitCircle = Set.of(
    novo Ponto(1, 0), novo Ponto( 0, 1),
    novo Ponto(-1, 0), novo Point( 0, -1));

boolean estático público onUnitCircle(Point p) {
    return unitCircle.contains(p);}
```

Embora essa possa não ser a forma mais rápida de implementar a funcionalidade, funciona bem. Suponha que você estenda `Point` de alguma forma trivial que não adiciona um componente de valor, por exemplo, fazendo com que seu construtor acompanhe quantas instâncias foram criadas:

```
classe pública CounterPoint estende Point {
    contador final AtomicInteger estático privado =
        novo AtomicInteger();

    public CounterPoint(int x, int y) {
        super(x, y); counter.incrementAndGet();}public static int
        numberCreated() { return counter.get(); }}
```

O princípio da substituição de Liskov diz que qualquer propriedade importante de um tipo também deve valer para todos os seus subtipos, de modo que qualquer método escrito para o tipo deve funcionar igualmente bem em seus subtipos [Liskov87]. Esta é a declaração formal do nosso

afirmação anterior de que uma subclasse de Ponto (como ContraPonto) ainda é um Ponto e deve agir como tal. Mas suponha que passemos um Contraponto para o método `onUnitCircle`. Se a classe `Point` usar um método `getClass` baseado em `getClass`, o método `onUnitCircle` retornará `false` independentemente das coordenadas `x` e `y` do `CounterPoint` instance. Isso ocorre porque a maioria das coleções, incluindo o `HashSet` usado pelo método `onUnitCircle`, usa o método `equals` para testar o `forcontainment`, e nenhuma instância do `CounterPoint` é igual a qualquer `Point`. Se, no entanto, você usar um método adequado baseado em `instanceof-equals` no `Point`, o método `sameonUnitCircle` funciona bem quando apresentado com uma instância `CounterPoint`.

Embora não haja uma maneira satisfatória de estender uma classe instanciada e adicionar um componente de valor, há uma boa solução: siga o conselho do Item 18, "Composição a favor em vez da herança." Em vez de ter o `ColorPoint` estendendo `Ponto`, dê ao `ColorPoint` um campo `Point` privado e um método de visualização pública (Item 6) que retorne o ponto na mesma posição desse ponto de cor:

```
Adiciona um componente de valor sem violar a classe ColorPoint de
contratoPublic igual a {
    ponto final privado de
    pontos; cor final privada;

    public ColorPoint(int x, int y, Color color) {
        ponto = novo Ponto(x, y); this.color =
        Objects.requireNonNull(color);}

    /**
     * Retorna a visão pontual desse ponto de
     cor.*/public Point asPoint() {

        ponto de
        retorno;}

    @Override booleano público é igual a (Object o) {
        se (!( o instância do ColorPoint))
        retorne falso; ColorPoint cp = (ColorPoint) o; return
        cp.point.equals(ponto) && cp.color.equals(color);}

    ... Restante omitido
}
```

Existem algumas classes nas bibliotecas da plataforma Java que estendem uma classe instância-tiável e adicionam um componente de valor. Por exemplo, `java.sql.Timestamp`

estende `java.util.Date` e adiciona um campo de nanossegundos. A implementação de `equals` para `Timestamp` viola simetria e pode causar comportamento errático se objetos `Timestamp` e `Date` forem usados na mesma coleção ou forem misturados. A classe `Timestamp` tem um aviso alertando programadores contra a mistura de datas e carimbos de tempo. Embora você não tenha problemas desde que os mantenha separados, nada impede que você misture, e os erros resultantes podem ser difíceis de depurar. Esse comportamento da classe `Timestamp` foi um erro e não deve ser imitado.

Note que você pode adicionar um componente de valor a uma subclasse de uma classe abstrata sem violar o contrato de `equals`. Isso é importante para o tipo de hierarquias de classes que você obtém seguindo o conselho do Item 23, "Prefira hierarquias de classes com classes totagadas." Por exemplo, você poderia ter uma classe abstrata `Shape` com componentes sem valor, uma subclasse `Círculo` que adiciona um campo de raio, e uma subclasse `Retângulo` que adiciona campos de comprimento e largura. Problemas do tipo mostrado anteriormente não ocorrerão enquanto for impossível criar uma instância de superclasse diretamente.

Consistência — O quarto requisito do contrato de igual diz que, se dois objetos forem iguais, eles devem permanecer iguais por todo o tempo, a menos que um (ou ambos) deles sejam modificados. Em outras palavras, objetos mutáveis podem ser iguais a diferentes objetos em diferentes momentos, enquanto objetos imutáveis não podem. Ao escrever uma aula, pense bem se ela deve ser imutável (Item 17). Se você concluir que deve, certifique-se de que seu método de `equals` impõe a restrição de que objetos iguais permanecem iguais e objetos desiguais permaneçam iguais por todo o tempo.

Independentemente de uma classe ser imutável ou não, não escreva um método de igual que dependa de recursos não confiáveis. É extremamente difícil satisfazer o requisito de conformidade se você violar essa proibição. Por exemplo, o método `java.net.URL.equals` baseia-se na comparação dos endereços IP dos hosts associados às URLs. Traduzir um nome de host para um endereço IP pode exigir acesso à rede, e não é garantido que isso produza os mesmos resultados ao longo do tempo. Isso pode fazer com que o método `URL.equals` viole o contrato de igual e tem causado problemas na prática. O comportamento do método de `equals` do `URL` foi um grande erro e não deve ser emulado. Infelizmente, não pode ser alterado devido a requisitos de compatibilidade. Para evitar esse tipo de problema, métodos iguais devem realizar apenas cálculos dissuasivos mínimos em objetos residentes na memória.

Não nulidade — O último requisito não tem nome oficial, então tomei a liberdade de chamá-lo de "não-nulidade". Diz que todos os objetos devem ser iguais a nulos. Embora seja difícil imaginar acidentalmente retornar `true` em resposta à invocação `o.equals(null)`, não é difícil imaginar lançar acidentalmente um

`NullPointerException`. O contrato geral proíbe isso. Muitas classes possuem métodos iguais que protegem contra isso com um teste explícito para nulo:

```
@Override booleano público é igual a (Object o) {  
    if (o == null)  
        return false;...}
```

Esse teste é desnecessário. Para testar seu argumento para igualdade, o método `equals` deve primeiro lançar seu argumento para um tipo apropriado para que seus acessores possam ser invocados ou seus campos acessados. Antes de fazer o cast, o método deve usar a instância de operador para verificar se seu argumento é do tipo correto:

```
@Override booleano público é igual a (Object o) {  
    se (!( ou instância de MyType))  
        retorne falso; MyType mt =  
        (MyType) o;...}
```

Se esse teste de tipo estivesse faltando e o método `equals` recebesse um argumento do tipo errado, o método `equals` geraria uma `ClassCastException`, que violaria o contrato `equals`. Mas a instância do operador é especificada para retornar `false` se seu primeiro operando for nulo, independentemente do tipo que aparece no segundo operando [JLS, 15.20.2]. Portanto, a verificação de tipo retornará `false` se o `null` for passado, então você não precisa de um check-up explícito de `null`.

Juntando tudo, aqui está uma receita para um método de alta qualidade igual:

1. Use o operador `==` para verificar se o argumento é uma referência a esse objeto. Se sim, retorne verdadeiro. Isso é apenas uma otimização de desempenho, mas vale a pena se a comparação for potencialmente cara.
2. Use o operador `instanceof` para verificar se o argumento tem o tipo correto. Se não, retorne falso. Normalmente, o tipo correto é a classe em que o método ocorre. Ocasionalmente, é alguma interface implementada por essa classe. Use uma interface se a classe implementar uma interface que refina o contrato igual para permitir comparações entre classes que implementam a interface. Interfaces de coleção como `Set`, `List`, `Map` e `Map.Entry` possuem essa característica.
3. Apresentar o argumento do tipo correto. Como esse elenco foi precedido por um teste de instância, é garantido que ele tenha sucesso.

4. Para cada campo "significativo" na classe, verifique se esse campo do argumento corresponde ao campo correspondente desse objeto. Se todos esses testes forem bem-sucedidos, volte a ser verdadeiro; caso contrário, retorne false. Se o tipo no Passo 2 for uma interface, você deve acessar os campos do argumento por métodos de interface; Se o tipo for uma classe, você pode conseguir acessar os campos diretamente, dependendo da acessibilidade deles. Para campos primitivos cujo tipo não é float ou double, use o operador `==` para comparações; para campos de referência de objetos, chame o método `equals` recursivamente; para campos float, use o método estático `Float.compare(float, float)`; e para campos duplos, use `Double.compare(double, double)`. O tratamento especial dos campos flutuantes e duplos é necessário pela existência de `Float.NaN`, `-0.0f` e os valores duplos análogos; veja JLS 15.21.1 ou a documentação do `Float.equals` para detalhes. Embora você possa comparar campos float and double com os métodos estáticos `Float.equals` e `Double.equals`, isso implicaria autoboxing em toda comparação, o que teria desempenho ruim. Para campos de array, aplique essas diretrizes a cada elemento. Se cada elemento em um campo de array for significativo, use um dos métodos `Arrays.equals`. Alguns campos de referência de objetos podem legitimamente conter nulo. Para evitar a possibilidade de uma `Exceção NullPointerException`, verifique tais campos para igualdade usando o método estático `Objects.equals(Object, Object)`. Para algumas classes, como `CaseInsensitiveString` acima, comparações de campos são mais complexas do que simples testes de igualdade. Se for esse o caso, talvez você queira armazenar uma forma canônica do campo para que o método dos iguais possa fazer uma comparação ex-ato barata em formas canônicas, em vez de uma com-parassisona não padrão mais cara. Esta técnica é mais apropriada para classes imutáveis (Item 17); Se o objeto puder mudar, você deve manter a forma canônica atualizada. O desempenho do método dos iguais pode ser afetado pela ordem em que os campos são comparados. Para melhor desempenho, você deve primeiro comparar áreas que têm mais probabilidade de diferir, menos caras de comparar ou, idealmente, ambas. Você não deve comparar campos que não fazem parte do estado lógico de um objeto, como campos de bloqueio usados para sincronizar operações. Você não precisa comparar campos derivados, que podem ser calculados a partir de "campos significativos", mas fazer isso pode melhorar o desempenho do método dos iguais. Se um campo derivado equivaler a uma descrição somatória de todo o objeto, comparar esse campo economizará o custo de comparar os dados reais caso a comparação falhe. Por exemplo, suponha que você tenha uma classe `Polygon` e armazene a área em cache. Se dois polígonos têm áreas desiguais, você não precisa se preocupar em comparar suas arestas e vértices.

Quando terminar de escrever seu método de equals, faça a si mesmo três perguntas: Ele é simétrico? É transitivo? É consistente? E não se pergunte só; escreva testes unitários para verificar, a menos que você tenha usado AutoValue (página 49) para gerar seu método de equals, caso em que pode omitir os testes com segurança. Se os laços próprios não se confirmarem, descubra o motivo e modifique o método dos iguais de acordo. Claro que seu método de igualdade também deve satisfazer as outras duas propriedades (reflexividade e não-nulidade), mas essas duas geralmente se resolvem sozinhas.

Um método de igual construção de acordo com a receita anterior é mostrado nesta classe simplista de Número de Telefone:

```
Turma com um típico equivale a metodopublic
última classe Número de Telefone {
    private final short areaCode, prefixo, lineNum;

    public PhoneNumber(int areaCode, int prefix, int lineNum) {
        this.areaCode = rangeCheck(areaCode, 999, "código de área");
        this.prefix= rangeCheck(prefix,999, "prefixo"); this.lineNum=
            rangeCheck(lineNum, 9999, "line num");}

    private static short rangeCheck(int val, int max, String arg) {
        se (val < 0 || val > max)
        lançar novo IllegalArgumentException(arg + ": " + val); return
        (curto) val;}

    @Override booleano público é igual a (Object o) {
        if (o == isto)
            retorno verdadeiro; se (!( o
            instância de NúmeroDeTelefone))
            retorne falso; Número de Telefone pn =
            (NúmeroDeTelefone)o; return pn.lineNum == lineNum &&
            pn.prefixo == prefixo
            && pn.areaCode == areaCode;
    }... // Restante omitido}
```

Aqui estão algumas ressalvas finais:

- Sempre sobrescrever o código hash quando você sobrescrever é igual a (Item 11).
- Não tente ser muito esperto. Se você simplesmente testar campos para igualdade, não é difícil seguir o contrato de igualdade. Se você for muito agressivo na busca por equivalência, é fácil se meter em encrenca. Geralmente é uma má ideia levar qualquer forma de aliasing em consideração. Por exemplo, a classe File não deve tentar equiparar links simbólicos que se referem ao mesmo arquivo. Felizmente, não funciona.

- Não substitua outro tipo por Objeto na declaração de igualdade. Não é incomum que um programador escreva um método de equals que seja assim e depois passe horas tentando entender por que ele não funciona direito:

```
Broken - o tipo de parâmetro deve ser
Object!public boolean igual a (MyClass o) {
...}
```

O problema é que esse método não sobrepõe `Object.equals`, cujo argumento é do tipo `Object`, mas o sobrecarrega (Item 52). É inaceitável fornecer um método "fortemente tipado" igual a esse mesmo além do normal, pois isso pode causar anotações de `Override` em subclasses que gerem falsos positivos e forneçam uma falsa sensação de segurança. O uso consistente da anotação de `Sobreposição`, como ilustrado ao longo deste item, evitará que você cometa esse erro (Item 40). Isso significa que o método não compila, e a mensagem de erro vai dizer exatamente o que está errado:

```
Ainda está quebrado, mas não compile!@Override
boolean público é igual a (MyClass o) {
...}
```

Escrever e testar métodos iguais (e `hashCode`) é tedioso, e o código resultante é mundano. Uma excelente alternativa a escrever e testar esses métodos manualmente é usar o framework open source `AutoValue` do Google, que gera automaticamente esses métodos para você, acionados por uma única anotação na classe. Na maioria dos casos, os métodos gerados pelo `AutoValue` são essencialmente idênticos aos que você mesmo escreveria.

IDEs, também, possuem recursos para gerar métodos iguais e `hashCode`, mas o código-fonte resultante é mais prolixo e menos legível do que o código que usa `AutoValue`, não rastreia mudanças na classe automaticamente e, portanto, requer testes. Dito isso, fazer com que IDEs gerem métodos iguais (e `hashCode`) geralmente é preferível a implementá-los manualmente, porque IDEs não cometem erros descuidados, e os humanos cometem.

Resumindo, não substitua o método `equals` a menos que seja necessário: em muitos casos, a implementação herdada do `Object` faz exatamente o que você quer. Se você sobrescrever `equals`, certifique-se de comparar todos os campos significativos da classe e compará-los de forma a preservar todas as cinco cláusulas do `equalscontract`.

Item 11: Sempre sobrescrever hashCode quando você sobrescreve é igual a

Você deve sobrescrever hashCode em toda classe que sobrescreve igual a . Se você falhar, sua classe violará o contrato geral para hashCode, o que impedirá que ele funcione corretamente em coleções como HashMap e HashSet. Aqui está o contrato, adaptado da especificação Object:

- Quando o método hashCode é invocado repetidamente em um objeto durante a execução de uma aplicação, ele deve retornar consistentemente o mesmo valor, desde que nenhuma informação usada em comparações iguais seja modificada. Esse valor não precisa permanecer consistente de uma execução de uma aplicação para outra.
- Se dois objetos forem iguais de acordo com o método equals(Object), então chamar hashCode nos dois objetos deve produzir o mesmo resultado inteiro.
- Se dois objetos forem desiguais de acordo com o método equals(Object), não é necessário que chamar hashCode em cada um dos objetos produza resultados distintos. No entanto, o programador deve estar ciente de que produzir resultados distintos para objetos desiguais pode melhorar o desempenho das tabelas hash. A principal disposição que é violada quando você não consegue sobrescrever o hashCode é a segunda: objetos iguais devem ter códigos hash iguais. Duas instâncias distintas podem ser logicamente iguais segundo o método de uma classe igual a elas, mas no método hashCode do ToObject, elas são apenas dois objetos sem muita coisa incomum. Portanto, o método hashCode do Object retorna dois números aparentemente aleatórios em vez de dois números iguais, conforme exigido pelo contrato. Por exemplo, suponha que você tente usar instâncias da classe PhoneNumber do Item 10 como chaves em um HashMap:

```
Mapa<NúmeroDeTelefone, String> m = novo HashMapa<>();  
m.put (novo Número de Telefone (707, 867, 5309), "Jenny");
```

Neste ponto, você poderia esperar que m.get(novo Número de Telefone(707, 867, 5309)) retornasse "Jenny", mas, em vez disso, ele retorna nulo. Note que duas instâncias de Número de Telefone estão envolvidas: uma é usada para inserção no HashMap, e uma segunda instância igual é usada para recuperação (tentada). A falha da classe PhoneNumber em sobrepor o hashCode faz com que as duas instâncias iguais tenham códigos hash desiguais, violando o contrato hashCode. Portanto, o método get provavelmente procurará o número de telefone em um bucket de hash diferente daquele em que foi armazenado pelo método put. Mesmo que as duas instâncias hajam para o mesmo bucket, o método get quase certamente retornará null, porque o HashMap tem uma optimi-zation que armazena em cache o código hash associado a cada entrada e não se preocupa em verificar a igualdade dos objetos se os códigos de hash não coincidirem.

Resolver esse problema é tão simples quanto escrever um método hashCode adequado para o PhoneNumber. Então, como deve ser um método hashCode? É trivial escrever um mau caso. Este, por exemplo, é sempre legal, mas nunca deve ser usado:

```
A pior implementação legal possível de hashCode - nunca use!  
int público hashCode() { return 42; }
```

É legal porque garante que objetos iguais tenham o mesmo código hash. É atroz porque garante que todo objeto tenha o mesmo código hash. Portanto, todo objeto faz hash para o mesmo bucket, e as tabelas de hash degeneram para listas enlaçadas.

Programas que deveriam rodar em tempo linear rodam em tempo quadrático. Para tabelas de grande hash, essa é a diferença entre trabalhar e não funcionar.

Uma boa função de hash tende a produzir códigos de hash desiguais para instâncias desiguais. É exatamente isso que se entende pela terceira parte do contrato hashCode. Idealmente, uma função de hash deve distribuir qualquer coleção razoável de instâncias desiguais de forma uniforme entre todos os valores de int. Alcançar esse ideal pode ser difícil. Felizmente, não é tão difícil conseguir uma aproximação justa. Aqui está uma receita simples:

1. Declare uma variável int nomeada result e inicie-a no código de hash c para o primeiro campo significativo do seu objeto, conforme calculado no passo 2.a. (Lembre-se do Item 10 que um campo significativo é um campo que afeta comparações iguais.)

2. Para cada campo significativo remanescente f no seu objeto, faça o seguinte:

- a. Calcule um código de hash int c para o campo: i. Se o campo for de tipo primitivo, calcule Type.hashCode(f), onde

O tipo é a classe primitiva em caixa correspondente ao type. ii de f. Se o campo for uma referência de objeto e o método de esta classe é igual

compara o corpo invocando recursivamente iguais, invocando recursivamente hashCode no campo. Se for necessária uma comparação mais complexa, calcule uma "representação canônica" para esse campo e invoque o código hash na representação canônica. Se o valor do campo for nulo, use 0 (ou alguma outra constante, mas 0 é tradicional). iii. Se o campo for um array, trate-o como se cada elemento significativo fosse um

Campo separado. Ou seja, calcular um código hash para cada elemento significativo aplicando essas regras recursivamente, e combinar os valores do passo 2.b. Se o array não tiver elementos significativos, use uma constante, preferencialmente não 0. Se todos os elementos forem significativos, use Arrays.hashCode. b. Combine o código de hash c calculado no passo 2.a no resultado da seguinte forma:

```
resultado = 31 * resultado + c;
```

3. Resultado do retorno.

Quando você terminar de escrever o método `hashCode`, pergunte a si mesmo se instâncias iguais têm códigos hash iguais. Escreva testes unitários para verificar sua intuição (a menos que você tenha usado `AutoValue` para gerar seus métodos `equals` e `hashCode`, caso em que pode omitir esses testes com segurança). Se instâncias iguais tiverem códigos hash desiguais, descubra o motivo e corrija o problema.

Você pode excluir campos derivados do cálculo do código hash. Em outras palavras, você pode ignorar qualquer campo cujo valor possa ser calculado a partir dos campos incluídos no cálculo. Você deve excluir quaisquer campos que não sejam usados em com-parisons iguais, ou corre o risco de violar a segunda disposição do contrato `HashCode`.

A multiplicação no passo 2.b faz com que o resultado dependa da ordem dos campos, resultando em uma função hash muito melhor se a classe tiver múltiplos campos semelhantes. Por exemplo, se a multiplicação fosse omitida de uma função de hash de string, os allanagrams teriam códigos de hash idênticos. O valor 31 foi escolhido porque é um primo ímpar. Se fosse par e a multiplicação transbordasse, a informação seria perdida, pois multiplicar por 2 é equivalente a deslocar. A vantagem de usar um primo é menos clara, mas é tradicional. Uma boa propriedade do 31 é que a multiplicação pode ser substituída por um shift e uma subtração para melhor desempenho em algumas arquiteturas: $31 * i == (i << 5) - i$. VMs modernas fazem esse tipo de otimização automaticamente.

Vamos aplicar a receita anterior à classe `Número de Telefone`:

```
HashCode típico method@Override
public int hashCode() {
    int result = Short.hashCode(areaCode); resultado = 31 *
    resultado + Short.hashCode(prefixo); resultado = 31 *
    resultado + Short.hashCode(lineNum); retorne
    resultado;}

```

Como esse método retorna o resultado de um cálculo determinístico simples cujas únicas entradas são os três campos significativos em uma instância de `PhoneNumber`, é claro que instâncias iguais de `PhoneNumber` têm códigos hash iguais. Esse método é, de fato, uma implementação perfeitamente boa de `hashCode` para `PhoneNumber`, no mesmo nível das bibliotecas da plataforma Java. É simples, razoavelmente rápido e faz um trabalho razoável ao distribuir números de telefone desiguais em diferentes buckets de hash.

Embora a receita deste item ofereça funções de hash razoavelmente boas, elas não são de última geração. Elas são comparáveis em qualidade às funções de hash encontradas nos tipos de valor das bibliotecas da plataforma Java e são adequadas para a maioria dos usos. Se você tem uma necessidade real de funções de hash menos propensas a produzir colisões, veja `Guava'scom.google.common.hash.Hashing` [Guava].

A classe `Objects` possui um método estático que pega um número arbitrário de objetos e retorna um código hash para eles. Esse método, chamado `hash`, permite que você escreva métodos `hashCode` de uma linha cuja qualidade é comparável àqueles escritos de acordo com a receita neste item. Infelizmente, eles rodam mais lentamente porque envolvem a criação de arrays para passar um número variável de argumentos, além de `boxing` e `unboxing` caso algum dos argumentos seja do tipo primitivo. Esse tipo de função de hash é recomendado apenas em situações onde o desempenho não é crítico. Aqui está uma função de hash para `PhoneNumber` escrita usando esta técnica:

```
Método hashCode de uma linha - medíocre
performance@Override int público hashCode() {
    return Objects.hash(lineNum, prefixo, areaCode);}
```

Se uma classe for imutável e o custo de calcular o código hash for significativo, você pode considerar armazenar o código hash no objeto em cache em vez de recalculá-lo toda vez que for solicitado. Se você acredita que a maioria dos objetos desse tipo será usada como chaves de hash, então deve calcular o código de hash quando a instância for criada. Caso contrário, você pode optar por inicializar o código hash de forma preguiçosa na primeira vez que o código hash for invocado. É necessário algum cuidado para garantir que a classe permaneça segura para threads na presença de um campo inicializado preguiçosamente (Item 83). Nossa aula de Número de Telefone não merece esse tratamento, mas só para mostrar como se faz, aqui está. Note que o valor inicial do campo `hashCode` (neste caso, 0) não deve ser o código hash de uma instância comumente criada:

```
método hashCode com hash codeprivate int hashCode inicializado
preguiçosamente; Inicializado automaticamente para 0

@Override public int hashCode() {
    int result = hashCode;
    se (resultado == 0) {
        resultado = Short.hashCode(areaCode); resultado = 31 *
        resultado + Short.hashCode(prefixo); resultado = 31 *
        resultado + Short.hashCode(lineNum); hashCode =
        resultado;}retorne resultado;}
```

Não se deixe levar por campos significativos da composição do código hash para melhorar o desempenho. Embora a função hash resultante possa rodar mais rápido, sua baixa qualidade pode degradar o desempenho das tabelas hash a ponto de se tornarem inúteis. Em particular, a função hash pode ser confrontada com um

grande coleção de instâncias que diferem principalmente em regiões que você escolheu ignorar. Se isso acontecer, a função hash mapeará todas essas instâncias para alguns códigos hash, e programas que deveriam rodar em tempo linear acabarão rodando em tempo quadrático.

Isso não é apenas um problema teórico. Antes do Java 2, a função de hash de string usava no máximo dezesseis caracteres espaçados uniformemente ao longo da string, começando pelo primeiro caractere. Para grandes coleções de nomes hierárquicos, como URLs, essa função exibia exatamente o comportamento patológico descrito anteriormente.

Não forneça uma especificação detalhada para o valor retornado pelo hashCode, então os clientes não podem razoavelmente depender disso; Isso te dá flexibilidade para mudar isso. Muitas classes nas bibliotecas Java, como String e Integer, especificam o valor exato retornado pelo método hashCode como função do valor da instância. Isso não é uma boa ideia, mas um erro com o qual somos obrigados a conviver: isso impede a capacidade de melhorar a função de hash em versões futuras. Se você deixar os detalhes não especificados e uma falha for encontrada na função de hash ou uma função hash melhor for descoberta, você pode mudá-la em uma versão posterior.

Em resumo, você deve sobrescrever hashCode toda vez que sobrescrever igual, ou seu programa não rodará corretamente. Seu método hashCode deve obedecer ao `generalContract` especificado no `Object` e deve fazer um trabalho razoável atribuindo hashcodes desiguais a instâncias desiguais. Isso é fácil de conseguir, embora um pouco tedioso, usando a receita da página 51. Como mencionado no Item 10, o framework `AutoValue` oferece uma alternativa fina para escrever manualmente métodos iguais e hashCode, e os IDEs também fornecem parte dessa funcionalidade.

Item 12: Sempre sobrescrever para String

Embora o Object forneça uma implementação do método toString, a string que ele retorna geralmente não é o que o usuário da sua classe deseja ver. Consiste no nome da classe seguido de um sinal "at" (@) e a representação hexadecimal sem sinal do código hash, por exemplo, `PhoneNumber@163b91`. O contrato geral fortoString diz que a sequência retornada deve ser "uma representação concisa, porém informativa, fácil de ler para uma pessoa." Embora se possa argumentar que `thatPhoneNumber@163b91` é conciso e fácil de ler, não é muito informativo quando comparado ao `707-867-5309`. O contrato toString continua: "É recomendado que todas as subclasses sobreponham esse método." Ótimo conselho, de fato!

Embora não seja tão crítico quanto obedecer aos contratos de equals e hashCode (Items 10 e 11), fornecer uma boa implementação toString torna sua classe muito mais agradável de usar e facilita a depuração dos sistemas que usam a classe. O método toString é automaticamente invocado quando um objeto é passado para `toprintln`, `printf`, o operador de concatenação de strings, ou `assert`, ou é impresso por `adebugger`. Mesmo que você nunca chame o toString em um objeto, outros podem. Por exemplo, um componente que tem referência ao seu objeto pode incluir a representação de string do objeto em uma mensagem de erro registrada. Se você não conseguir sobrescrever para String, a mensagem pode ser praticamente inútil. Se você forneceu um bom método toString para o `PhoneNumber`, gerar uma mensagem de diagnóstico útil é tão simples quanto isso:

```
System.out.println ("Falhou a conexão em " + telefoneNúmero);
```

Programadores geram mensagens de diagnóstico dessa forma, independentemente de você sobrescrever para String, mas as mensagens não serão úteis a menos que você faça isso. Os benefícios de fornecer um bom método toString vão além das instâncias da classe `toobjects` contendo referências a essas instâncias, especialmente coleções. Qual você preferiria ver ao imprimir um mapa, `{Jenny=PhoneNumber@163b91}` ou `{Jenny=707-867-5309}`?

Quando possível, o método toString deve retornar toda a informação interessante contida no objeto, como mostrado no exemplo do número de telefone. É impraticável se o objeto for grande ou se contiver um estado que não favorece a representação das cordas. Nessas circunstâncias, o ToString deve devolver um `sum-mary`, como o `Manhattan residential phone listing (1487536 listings)` ou `Thread[main,5,main]`. Idealmente, a corda deve ser autoexplicativa. (O exemplo do `TheThread` reprovou nesse teste.) Uma penalidade particularmente irritante por não conseguir

Inclua todas as informações interessantes de um objeto em sua representação de string em relatórios de testfail que se parecem com isto:

Falha de afirmação: esperado {abc, 123}, mas era {abc, 123}.

Uma decisão importante que você terá que tomar ao implementar um `toStringMethod` é especificar o formato do valor de retorno na documentação. Recomenda-se que você faça isso para classes de valor, como número de telefone ou matriz. A vantagem de especificar o formato é que ele serve como uma representação padrão, inequívoca e legível pelo ser humano do objeto. Essa representação pode ser usada para entrada e saída e em objetos de dados persistentes legíveis por humanos, como arquivos CSV. Se você especificar o formato, geralmente é uma boa ideia fornecer uma fábrica estática ou construtor correspondente para que os programadores possam facilmente traduzir entre o objeto e sua representação de strings. Essa abordagem é adotada por muitas classes de valor nas bibliotecas da plataforma Java, incluindo `BigInteger`, `BigDecimal` e a maioria das classes primitivas em caixa.

A desvantagem de especificar o formato do valor de retorno do `toString` é que, uma vez especificado, você fica preso com ele para sempre, assumindo que sua classe seja amplamente usada. Programadores escrevem código para analisar a representação, gerá-la e incorporá-la em dados persistentes. Se você mudar a representação em uma versão futura, vai quebrar o código e os dados deles, e eles vão miar. Ao optar por não especificar um formato, você preserva a flexibilidade de adicionar informações ou melhorar o formato em uma versão posterior.

Quer você decida especificar o formato, deve documentar claramente suas intenções. Se você especificar o formato, deve fazê-lo com precisão. Por exemplo, aqui está um método `toString` para acompanhar a classe `PhoneNumber` no Item 11:

```
/**
 * Retorna a representação da string desse número de telefone.* A
 * string consiste em doze caracteres cujo formato é* "XXX-YYY-ZZZZ",
 * onde XXX é o código de área, YYY é o* prefixo e ZZZZ é o número da
 * linha. Cada uma das letras maiúsculas* representa um único dígito
 * decimal.** Se qualquer uma das três partes desse número de telefone
 * for pequena demais* para preencher seu campo, o campo é preenchido
 * com zeros à frente.* Por exemplo, se o valor do número da linha for
 * 123, os últimos* quatro caracteres da representação da string serão
 * "0123".*/@Override public String toString() {

    return String.format("%03d-%03d-%04d",
        areaCode, prefixo, lineNum);
}
```

Se você decidir não especificar um formato, o comentário de documentação deve ser algo assim:

```
/**
 * Retorna uma breve descrição desta poção. Os detalhes exatos* da
 * representação não são especificados e sujeitos a alterações,* mas os
 * seguintes podem ser considerados típicos:** "[Poção #9: tipo=amor,
 * cheiro=terebintina, look=tinta índia]"*/@Override fio público
 toString() { ... }
```

Após ler este comentário, programadores que produzem código ou dados persistentes — que dependem dos detalhes do formato — não terão ninguém além de si mesmos para culpar quando o formato for alterado.

Independentemente de você especificar o formato, forneça acesso programático à informação contida no valor retornado por `toString`. Por exemplo, a classe `Número de Telefone` deve conter acessores para o código de área, prefixo e número de linha. Se você não fizer isso, força os programadores que precisam dessa informação a superarem a string. Além de reduzir o desempenho e gerar trabalho desnecessário para os programadores, esse processo é propenso a erros e resulta em sistemas frágeis que quebram se você mudar o formato. Ao não fornecer acessores, você transforma a string `for-mat` em uma API de fato, mesmo que tenha especificado que ela pode mudar.

Não faz sentido escrever um método `toString` em uma classe de utilidade estática (Item 4). Também não deve escrever um método `toString` na maioria dos tipos de `enum` (Item 34), porque o Java oferece um método perfeitamente bom para você. No entanto, você deve escrever o método `atoString` em qualquer classe abstrata cujas subclasses compartilhem uma representação de strings comum. Por exemplo, os métodos `toString` na maioria das implementações de coleções são herdados das classes de coleções abstratas.

A funcionalidade de código aberto `AutoValue` do Google, discutida no Item 10, gerará o método `atoString` para você, assim como a maioria dos IDEs. Esses métodos são ótimos para te mostrar o conteúdo de cada área, mas não são especializados no significado da disciplina. Então, por exemplo, seria inadequado usar um método gerado automaticamente para a nossa aula de `Número de Telefone` (já que números de telefone têm uma representação de strings padrão), mas seria perfeitamente aceitável para nossa aula de `Poções`. Dito isso, um método `toString` gerado automaticamente é muito preferível ao herdado do `Object`, que não diz nada sobre o valor de um objeto.

Para recapitular, substitua a implementação do `Object toString` em cada `instantiableclass` que você escrever, a menos que uma superclasse já tenha feito isso. Isso torna as classes muito mais agradáveis de usar e ajuda na depuração. O método `toString` deve devolver uma descrição concisa e útil do objeto, em um formato esteticamente agradável.

Item 13: Anular o clone com critério

A interface `Cloneable` foi pensada como uma interface mixin (Item 20) para classes que anunciassem que elas permitem clonagem. Infelizmente, ela não serve a esse propósito. Sua principal falha é que ele não possui um método de clonagem, e o método de clone do `Object` é protegido. Você não pode, sem recorrer ao reflexo (Item 65), invocar clone em um objeto apenas porque ele implementa o `Cloneable`. Até mesmo uma invocação reflexiva pode falhar, porque não há garantia de que o objeto tenha um método clone acessível. Apesar dessa falha e de muitas outras, a instalação é bastante utilizada, então vale a pena entendê-la. Este item mostra como implementar um método de clonagem bem comportado, discute quando é apropriado fazê-lo e apresenta alternativas.

Então, o que o `Cloneable` faz, considerando que não contém métodos? Ele determina o comportamento da implementação de clone protegido do `Object`: se uma classe implementa `Cloneable`, o método clone do `Object` retorna uma cópia campo a campo do objeto; caso contrário, ele lança `CloneNotSupportedException`. Este é um uso altamente atípico de interfaces e não deve ser emulado. Normalmente, implementar uma interface diz algo sobre o que uma classe pode fazer pelos seus clientes. Neste caso, ele modifica o comportamento de um método protegido em uma superclasse.

Embora a especificação não o diga, na prática, espera-se que uma classe implementando `Cloneable` forneça um método de clonagem pública que funcione corretamente. Para alcançar isso, a classe e todas as suas superclasses devem obedecer a um protocolo complexo, impossível de aplicar e pouco documentado. O mecanismo resultante é frágil, perigoso e extralinguístico: ele cria objetos sem chamar um construtor.

O contrato geral para o método clone é fraco. Aqui está, copiado da especificação do `Object`:

Cria e retorna uma cópia desse objeto. O significado exato de "copiar" pode depender da classe do objeto. A intenção geral é que, para qualquer objeto `x`, a expressão

```
x.clone() != x
```

será verdadeira, e a expressão

```
x.clone().getClass() == x.getClass()
```

Será verdade, mas esses não são requisitos absolutos. Embora normalmente seja o caso de

```
x.clone().equals(x)
```

Será verdade, isso não é um requisito absoluto.

Por convenção, o objeto retornado por esse método deve ser obtido chamando `super.clone`. Se uma classe e todas as suas superclasses (exceto Objeto) obedecem a essa convenção, será o caso de que

`x.clone().getClass() == x.getClass()`. Por convenção, o objeto retornado deve ser independente do objeto que está sendo clonado. Para alcançar essa independência, pode ser necessário modificar um ou mais campos do objeto retornado pelo `super.clone` antes de retorná-lo. Esse mecanismo é vagamente semelhante ao constructor chaining, exceto que não é `tenforced`: se o método `clone` de uma classe retorna uma instância que não é obtida chamando `super.clone`, mas chamando um construtor, o compilador não reclamará, mas se uma subclasse dessa classe chamar `super.clone`, o objeto resultante terá a classe errada, impedindo que o método da subclasse de `clone` funcione corretamente. Se uma classe que substitui o `clone` for final, essa convenção pode ser ignorada com segurança, pois não há subclasses com que se preocupar. Mas se uma classe final tem um método `clone` que não invoca `super.clone`, não há motivo para a classe implementar `Cloneable`, pois não depende do comportamento da implementação `clone` do `Object`.

Suponha que você queira implementar o `Cloneable` em uma classe cuja superclasse fornece um método de `clone` bem comportado. Primeiro ligue para o `super.clone`. O objeto que você receber de volta será uma réplica totalmente funcional do original. Qualquer campo declarado na sua classe terá valores idênticos aos do original. Se todo campo contiver um valor primitivo ou uma referência a um objeto imutável, o objeto retornado pode ser exatamente o que você precisa, caso em que não é necessário processamento adicional. Esse é o caso, por exemplo, para a classe Número de Telefone no Item 11, mas note que classes imutáveis nunca deveriam fornecer um método `clone` porque isso apenas incentivaria o desperdício de cópias. Com essa ressalva, veja como seria um método `clone` para o Número de Telefone:

```
Método clone para classe sem referências ao clone público
state@Override PhoneNumber clone() {
    try {
        return (TelefoneNúmero) super.clone();
    } catch (CloneNotSupportedException e) {
        lançar um novo                               Não pode acontecer
        AssertionError();
    }
}
```

Para que esse método funcione, a declaração de classe para `PhoneNumber` teria que ser modificada para indicar que ele implementa `Cloneable`. Embora o método `Object.clone` retorne `Object`, este método `clone` retorna `PhoneNumber`. É legal

e desejável fazer isso porque o Java suporta tipos de retorno covariantes. Em outras palavras, o tipo de retorno de um método sobrescrito pode ser uma subclasse do tipo de retorno do método sobrescrito. Isso elimina a necessidade de elenco no cliente. Devemos lançar o resultado do `super.clone` do Objeto para o Número de Telefone antes de retornar, mas o elenco tem garantia de sucesso.

A chamada para `super.clone` está contida em um bloco try-catch. Isso acontece porque o Object declara seu método clone para lançar `CloneNotSupportedException`, que é uma exceção verificada. Como o `PhoneNumber` implementa o `Cloneable`, sabemos que a ligação para `super.clone` terá sucesso. A necessidade desse boilerplate indica que `CloneNotSupportedException` deveria ter sido desmarcado (Item 71).

Se um objeto contém campos que se referem a objetos mutáveis, a simples `cloneImplementation` mostrada anteriormente pode ser desastrosa. Por exemplo, considere a classe de pilha no Item 7:

```

classe pública Stack {
    elementos privados de Objeto[] private int size = 0; Int.
    final estática privada DEFAULT_INITIAL_CAPACITY = 16;

    public Stack() {
        this.elementos = novo Objeto[DEFAULT_INITIAL_CAPACITY];}

    empty public push(Object e) {
        ensureCapacity();
        elementos[size++] = e;}

    Objeto público pop() {
        if (tamanho == 0)
            lançar um novo EmptyStackException(); Resultado do objeto =
            elementos[--tamanho]; elementos[tamanho] = nulo; Eliminar o
            resultado obsoleto de referência;}

    Garanta espaço para pelo menos mais um
    element.private void ensureCapacity() {
        if (elements.length == tamanho)
            elements = Arrays.copyOf(elements, 2 * tamanho + 1);
    }}

```

Suponha que você queira tornar essa classe clonável. Se o método `clone` merelyretorne `super.clone()`, a instância `Stack` resultante terá o valor correto em

seu campo de tamanho, mas seu campo `elements` se referirá ao mesmo array da instância original `Stack`. Modificar o original destruirá os invariantes no clone e vice-versa. Você rapidamente vai perceber que seu programa gera resultados sem sentido ou lança um `NullPointerException`.

Essa situação nunca poderia ocorrer como resultado de chamar o único construtor na classe `Stack`. Na prática, o método `clone` funciona como um construtor; Você deve garantir que não cause dano ao objeto original e que estabeleça corretamente invariantes no clone. Para que o método `clone` no `Stack` funcione corretamente, ele deve copiar os componentes internos da `stack`. A maneira mais fácil de fazer isso é chamar o clone recursivamente no array de elementos:

```
Método clone para classe com referências a mútuaveis
state@Override clone público da pilha() {
    try {
        Resultado da pilha = (Pilha) super.clone();
        resultado.elements = elementos.clone();
        retorne resultado;} catch
        (CloneNotSupportedException e) {
            lançar novo AssertionError();}}
```

Note que não precisamos lançar o resultado de `elements.clone` para `Object[]`. Chamar o `clone` em um array retorna um array cujos tipos de tempo de execução e tempo de compilação são idênticos aos do array que está sendo clonado. Esse é o idioma preferido para duplicar um array. Na verdade, os arrays são o único uso convincente da instalação `clone`.

Note também que a solução anterior não funcionaria se o campo de elementos fosse final porque o clone seria proibido de atribuir um novo valor ao campo. Este é um problema fundamental: assim como a serialização, a arquitetura Clonável é incompatível com o uso normal dos campos finais referentes a objetos mutáveis, exceto nos casos em que os objetos mutáveis podem ser compartilhados com segurança entre um objeto e seu clone. Para tornar uma classe clonável, pode ser necessário remover modificadores finais de alguns campos.

Nem sempre basta simplesmente chamar `clone` recursivamente. Por exemplo, suponha que você esteja escrevendo um método `clone` para uma tabela hash cujos internos consistem em um array de buckets, cada um dos quais faz referência à primeira entrada de uma lista enlazada de pares-chave-valor. Para desempenho, a classe implementa sua própria lista leve `singlylinked` em vez de usar `java.util.LinkedList` internamente:

```
classe pública HashTable implementa Cloneable {
    entrada privada[] baldes = ...;
```

```

    classe estática privada Entrada {
        chave final de Objeto;
        Valor do objeto;
        Entradapróxima;

        Entry(Chave de objeto, valor de objeto, próxima entrada) {
            this.key    = chave;
            this.value = value; this.next=
next;}}... // Restante omitido}

```

Suponha que você simplesmente clone o array de baldes recursivamente, como fizemos com o Stack:

```

Método do clone quebrado - resulta em estado mutável
compartilhado!@Override clone público do HashTable {
    try {
        Resultado HashTable = (HashTable) super.clone();
        result.buckets = buckets.clone(); retorne resultado;}
    catch (CloneNotSupportedException e) {

        lançar novo AssertionError();}}

```

Embora o clone tenha seu próprio array bucket, esse array faz referência às mesmas listas enlazadas do original, o que pode facilmente causar comportamentos não determinísticos tanto no clone quanto no original. Para resolver esse problema, você terá que copiar a lista linkedlist que compõe cada bucket. Aqui está uma abordagem comum:

```

Método de clonagem recursiva para classe com estadopublic mutável
complexo HashTable implementa Cloneable {
    entrada privada[] baldes = ...;

    classe estática privada Entrada {
        chave final de Objeto;
        Valor do objeto;
        Entradapróxima;

        Entry(Chave de objeto, valor de objeto, próxima entrada) {
            this.key    = chave;
            this.value = value;
            this.next= next;}
    }
}

```

```

Copie recursivamente a lista encadenada com este EntryEntry
deepCopy() {
    return new Entry(chave, valor,
        próximo == null ? null : next.deepCopy());
}

```

```

@Override clone público do HashTable {
    try {
        Resultado HashTable = (HashTable) super.clone();
        result.buckets = nova Entrada[buckets.length];
        para (int i = 0; i < buckets.length; i++)
            se (buckets[i] != nulo)
                result.buckets[i] = buckets[i].deepCopy();
        retorne resultado;} catch
        (CloneNotSupportedException e) {
            lançar novo AssertionError();}... //
            Restante omitido}

```

A classe privada `HashTable.Entry` foi ampliada para suportar um método "deepcopy". O método `clone` no `HashTable` aloca um novo array de baldes do tamanho correto e itera sobre o array original dos baldes, copiando profundamente cada balde vazio do outro. O método `deepCopy` no `Entry` se invoca recursivamente para copiar toda a lista encadenada encabeçada pela entrada. Embora essa técnica seja fofa e funcione bem se os buckets não forem muito longos, não é uma boa forma de clonar uma lista encadeada porque consome um frame de pilha para cada elemento da lista. Se a lista for longa, isso pode facilmente causar um excesso de pilha. Para evitar que isso aconteça, você pode substituir a recursão no `deepCopy` por iteração:

```

Copie iterativamente a lista encadenada com este EntryEntry
deepCopy() {
    Resultado da entrada = nova Entrada (chave, valor, próximo);
    for (Entrada p = resultado; p.next != null; p = p.next)
        p.next = nova Entrada(p.next.key, p.próximo.valor,
            p.próximo.próximo); retorne resultado;}

```

Uma abordagem final para clonar objetos mutáveis complexos é chamar `super.clone`, definir todos os campos do objeto resultante para seu estado inicial e então chamar métodos de nível superior para regenerar o estado do objeto original. No caso do exemplo do nosso `HashTable`, o campo `buckets` seria inicializado em um novo array de buckets, e o método `put(key, value)` (não mostrado) seria invocado para cada key-

mapeamento de valores na tabela hash que está sendo clonada. Essa abordagem normalmente gera um método de clone simples e razoavelmente elegante que não funciona tão rápido quanto um que manipula diretamente o interior do clone. Embora essa abordagem seja limpa, ela é antitética a toda a arquitetura Cloneable porque sobrescreve cegamente a cópia de objeto campo a campo que forma a base da arquitetura.

Como um construtor, um método clone nunca deve invocar um método sobrescrito no clone em construção (Item 19). Se clone invocar um método que foi sobreposto em uma subclasse, esse método será executado antes que a subclasse tenha tido a chance de corrigir seu estado no clone, possivelmente levando à corrupção no clone e no original. Portanto, o método put(chave, valor) discutido no parágrafo anterior deve ser final ou privado. (Se for privado, presumivelmente é o "método auxiliar" para um método público não final.)

O método clone do objeto é declarado para lançar CloneNotSupportedException, mas métodos de sobreposição não precisam. Métodos de clonagem pública devem omitir a throwsclause, pois métodos que não lançam exceções verificadas são mais fáceis de usar (Item 71).

Você tem duas opções ao projetar uma classe para herança (Item 19), mas qualquer que seja a que escolher, a classe não deve implementar o Clonável. Você pode escolher imitar o comportamento do Objeto implementando um método de clone protegido que funciona corretamente e que é declarado para lançar CloneNotSupportedException. Isso dá às subclasses a liberdade de implementar Cloneable ou não, como se elas estendessem o Objeto diretamente. Alternativamente, você pode optar por não implementar um método de clone funcional e impedir que subclasses implementem um, fornecendo a seguinte implementação de clone degenerado:

```
clone method for extensible class que não suporta  
Cloneable@Overrideprotected final Object clone() throws  
CloneNotSupportedException {  
    lançar novo CloneNotSupportedException();}
```

Há mais um detalhe que merece ser notado. Se você escreve uma classe thread-safe que implementa Cloneable, lembre-se de que seu método clone deve ser devidamente sincronizado, assim como qualquer outro método (Item 78). O método clone do Object não é sincronizado, então mesmo que sua implementação seja satisfatória, talvez você precise escrever um método clone sincronizado que retorne super.clone().

Para recapitular, todas as classes que implementam o Cloneable devem sobrescrever o clone com um método público cujo tipo de retorno seja a própria classe. Esse método deve primeiro chamar o .clone e depois corrigir quaisquer campos que precisem de correção. Normalmente, isso significa copiar quaisquer objetos mutáveis que compõem a "estrutura profunda" interna do objeto e substituir as referências do clone a esses objetos por referências às suas cópias.

Embora essas cópias internas geralmente possam ser feitas chamando o clone recursivamente, essa nem sempre é a melhor abordagem. Se a classe contém apenas campos primitivos ou referências a objetos imutáveis, então é provável que nenhum campo precise ser fixado. Existem exceções a essa regra. Por exemplo, um campo que representa um número de série ou outro ID único precisará ser corrigido mesmo que seja primitivo ou imutável.

Toda essa complexidade é realmente necessária? Raramente. Se você estende uma classe que já implementa Cloneable, você tem pouca escolha a não ser implementar um método de clone bem comportado. Caso contrário, geralmente é melhor oferecer um meio alternativo de cópia de objetos. Uma abordagem melhor para copiar objetos é fornecer um construtor de cópias ou uma fábrica de cópias. Um construtor de cópia é simplesmente um construtor que toma um único argumento cujo tipo é a classe que contém o construtor, por exemplo,

```
Copiar constructorpublic Yum(Yum yum) { ... };
```

Uma fábrica de cópias é o análogo estático (Item 1) de um construtor de cópia:

```
Copiar factorypublic static Yum newInstance(Yum yum) { ... };
```

A abordagem do construtor de cópias e sua variante de fábrica estática têm muitas vantagens em relação ao Cloneable/clone: não dependem de um mecanismo de criação de objetos extralinguísticos propenso a riscos; Eles não exigem adesão incorrigível a convenções pouco documentadas; eles não entram em conflito com o uso correto dos campos finais; Eles não lançam exceções desnecessárias e verificadas; E eles não exigem gesso.

Além disso, um construtor de cópias ou fábrica pode tomar um argumento cujo tipo é uma interface implementada pela classe. Por exemplo, por convenção, todas as implementações de coleções de uso geral fornecem um construtor cujo argumento é o tipo Collection ou Map. Construtores e fábricas de cópias baseados em interface, mais conhecidos como construtores de conversão e fábricas de conversão, permitem que o cliente escolha o tipo de implementação da cópia em vez de forçar o cliente a aceitar o tipo de implementação do original. Por exemplo, suponha que você tenha um HashSet, s, e queira copiá-lo como um TreeSet. O método clone não oferece essa funcionalidade, mas é fácil com um construtor de conversão: new TreeSet<>(s).

Diante de todos os problemas associados ao Cloneable, novas interfaces não deveriam estendê-lo, e novas classes extensíveis não deveriam implementá-lo. Embora seja menos prejudicial para as classes finais implementar o Cloneable, isso deve ser visto como uma otimização de desempenho, reservada para os raros casos em que é justificada (Item 67). Como regra, a funcionalidade de cópia é melhor fornecida por construtores ou fábricas. Uma exceção notável a essa regra são os arrays, que são melhor copiados com o método clone.

Item 14: Considere implementar o Comparável

Diferente dos outros métodos discutidos neste capítulo, o método `compareTo` não é declarado em `Object`. Na verdade, é o único método na interface `Comparável`. É semelhante em caráter ao método de `equals` do `Object`, exceto que permite comparações de ordem além de comparações simples de igualdade, e é genérico. Ao implementar `Comparável`, uma classe indica que suas instâncias têm uma ordem natural. Ordenar um array de objetos que implementam `Comparável` é tão simples quanto isto:

```
Arrays.sort(a);
```

É igualmente fácil pesquisar, calcular valores extremos e manter coleções ordenadas automaticamente de objetos comparáveis. Por exemplo, o seguinte pro-gram, que se baseia no fato de que o `String` implementa o `Comparable`, imprime uma lista analfabetizada de seus argumentos de linha de comando com duplicatas eliminadas:

```
classe pública WordList {  
    public static void main(String[] args) {  
        Set<String> s = novo TreeSet<>();  
        Collections.addAll(s, args);  
        System.out.println(s);  
    }  
}
```

Ao implementar o `Comparable`, você permite que sua classe interopere com todos os muitos algoritmos genéricos e implementações de coleção que dependem dessa interface. Você ganha uma quantidade enorme de poder com um pequeno esforço. Praticamente todas as classes de valor nas bibliotecas da plataforma Java, assim como todos os enumtipos (Item 34), implementam o `Comparable`. Se você está escrevendo uma classe de valor com uma ordem natural óbvia, como ordem alfabética, numérica ou ordem cronológica, você deve implementar a interface `Comparável`:

```
interface pública<T> Comparável {  
    int compareTo(T t);  
}
```

O contrato geral do método `compareTo` é semelhante ao dos iguais:

Compara esse objeto com o objeto especificado para a ordem. Retorna um inteiro negativo, zero ou inteiro positivo, pois esse objeto é menor, igual ou maior que o objeto especificado. Lança `ClassCastException` se o tipo do objeto especificado impedir que ele seja comparado a esse objeto.

Na descrição a seguir, a notação `sgn(expressão)` designa a função *signum* matemática-emática, que é definida para devolver -1, 0 ou 1, dependendo se o valor da expressão é negativo, zero ou positivo.

- O implementador deve garantir que `sgn(x.compareTo(y)) == -sgn(y.compareTo(x))` para todos `x` e `y`. (Isso implica que `x.compareTo(y)` deve lançar uma exceção se e somente se `y.compareTo(x)` lançar uma exceção.)
- O implementador também deve garantir que a relação seja transitiva: `(x.compareTo(y) > 0 & y.compareTo(z) > 0)` implica `x.compareTo(z) > 0`.
- Finalmente, o implementador deve garantir que `x.compareTo(y) == 0` implica que `sgn(x.compareTo(z)) == sgn(y.compareTo(z))`, para todo `z`.
- É fortemente recomendado, mas não obrigatório, que `(x.compareTo(y) == 0) == (x.equals(y))`. De modo geral, qualquer classe que implemente a interface `Comparable` e viole essa condição deve indicar claramente esse fato. A linguagem recomendada é "Nota: Esta classe tem uma ordenação natural que é inconsistente com iguais." Não se deixe desanimar pela natureza matemática deste contrato. Assim como o `equalsContract` (Item 10), esse contrato não é tão complicado quanto parece. Diferente do `equalsMethod`, que impõe uma relação global de equivalência a todos os objetos, `compareTo` não precisa funcionar entre objetos de diferentes tipos: quando confrontado com objetos de tipos diferentes, `compareTo` pode lançar `ClassCastException`. Normalmente, é exatamente isso que ele faz. O contrato permite *comparisons* intertipo, que normalmente são definidos em uma interface implementada pelos objetos que estão sendo comparados. Assim como uma classe que viola o contrato do `HashCode` pode quebrar outras classes que dependem do *hashing*, uma classe que viola o contrato `compareTo` pode quebrar outras classes que dependem da comparação. Classes que dependem da comparação incluem as coleções ordenadas `TreeSet` e `TreeMap` e as classes utilitárias `Collections` and `Arrays`, que contêm algoritmos de busca e ordenação. Vamos revisar as cláusulas do contrato `compareTo`. A primeira disposição diz que, se você inverter a direção de uma comparação entre duas referências de objetos, o esperado acontece: se o primeiro objeto for menor que o segundo, então o segundo deve ser maior que o primeiro; Se o primeiro objeto for igual ao segundo, então o segundo deve ser igual ao primeiro; e se o primeiro objeto for maior que o segundo, então o segundo deve ser menor que o primeiro. A segunda disposição diz que, se um objeto for maior que um segundo e o segundo maior que um terceiro, então o primeiro deve ser maior que o terceiro. A disposição final diz que todos os objetos que se comparam como iguais devem gerar os mesmos resultados quando comparados a qualquer outro objeto.

Uma consequência dessas três disposições é que o teste de igualdade imposto pelo método `compareTo` deve obedecer às mesmas restrições impostas pelo contrato de `Comparable`: reflexividade, simetria e transitividade. Portanto, a mesma ressalva se aplica: não há como estender uma classe instanciada com um novo componente de valor preservando o contrato `compareTo`, a menos que você esteja disposto a abrir mão dos benefícios da abstração orientada a objetos (Item 10). A mesma solução alternativa também se aplica. Se você quiser adicionar um componente de valor a uma classe que implementa `Comparable`, não o estenda; escreva uma classe não relacionada contendo uma instância da primeira classe. Depois, forneça um método de "visualização" que retorne a instância contida. Isso libera você para implementar qualquer método `compareTo` que desejar na classe `Containing`, permitindo que seu cliente visualize uma instância da classe `Contained` como uma instância da classe `Containing` quando necessário.

O parágrafo final do contrato `compareTo`, que é uma sugestão forte e não um requisito verdadeiro, simplesmente afirma que o teste de igualdade imposto pelo método `compareTo` geralmente deveria devolver os mesmos resultados do método `equals`. Se essa disposição for obedecida, a ordenação imposta pelo método `compareTo`, diz-se consistente com iguais. Se for violado, a ordem é considerada inconsistente com iguais. Uma classe cujo método `compareTo` impõe uma ordem que é inconsistente com iguais ainda funcionará, mas coleções ordenadas contendo elementos da classe podem não obedecer ao contrato geral das interfaces de coleção apropriadas (Coleção, Set ou Mapa). Isso ocorre porque os contratos gerais para essas interfaces são definidos em termos do método dos iguais, mas as coleções ordenadas usam o teste de igualdade imposto pelo `compareTo` no lugar dos iguais. Não é uma catástrofe se isso acontecer, mas é algo para se ter em conta.

Por exemplo, considere a classe `BigDecimal`, cujo método `compareTo` é inconsistente com iguais. Se você criar uma instância `HashSet` vazia e depois adicionar `BigDecimal("1.0")` e um novo `BigDecimal("1.00")`, o conjunto conterá dois elementos porque as duas instâncias `BigDecimal` adicionadas ao conjunto são desiguais quando comparadas usando o método `equals`. Se, no entanto, você realizar o mesmo procedimento usando um `TreeSet` em vez de um `HashSet`, o conjunto conterá apenas um elemento porque as duas instâncias `BigDecimal` são iguais quando comparadas usando o método `compareTo`. (Veja a documentação do `BigDecimal` para detalhes.)

Escrever um método `compareTo` é semelhante a escrever um método de `equals`, mas existem algumas diferenças importantes. Como a interface `Comparable` é parametrizada, o método `compareTo` é tipado estaticamente, então você não precisa fazer check-up ou formular seu argumento. Se o argumento for do tipo errado, a invocação nem sequer irá compilar. Se o argumento for nulo, a invocação deve lançar uma `NullPointerException`, e isso acontecerá assim que o método tentar acessar seus membros.

No método `compareTo`, os campos são comparados por ordem em vez de igualdade. Para comparar campos de referência de objetos, invoque o método `compareTo` recursivamente. Se o `afield` não implementar o `Comparable` ou você precisar de uma ordem não padrão, use o `Comparador` em vez disso. Você pode escrever seu próprio comparador ou usar um já existente, como neste método `compareTo` para `CaseInsensitiveString` no Item 10:

```
Campo único Comparável com campo de referência de objeto
classe final pública CaseInsensitiveString
    implementa<CaseInsensitiveString> Comparable {public
        int compareTo(CaseInsensitiveString cis) {
            return String.CASE_INSENSITIVE_ORDER.compare(s, cis.s);}... //
            Restante omitido}
```

Note que o `CaseInsensitiveString` implementa o `<CaseInsensitiveString>Comparable`. Isso significa que uma referência `CaseInsensitiveString` pode ser comparada apenas com outra referência `CaseInsensitiveString`. Este é o padrão normal a seguir ao declarar uma classe para implementar `Comparable`.

Edições anteriores deste livro recomendavam que os métodos `compareTo` comparassem campos primitivos integrais usando os operadores relacionais `<` e `>`, e campos primitivos de ponto flutuante usando os métodos estáticos `Double.compare` e `Float.compare`. No Java 7, métodos de comparação estática foram adicionados a todas as primitivclasses em caixa do Java. O uso dos operadores relacionais `<` e `>` nos métodos `compareTo` é verboso e propenso a erros, e não é mais recomendado.

Se uma classe tem múltiplos campos significativos, a ordem em que você os compara é fundamental. Comece pela área mais importante e vá descendo. Se uma comparação resultar em algo diferente de zero (que representa igualdade), você terminou; Apenas devolva o resultado. Se o corpo mais significativo for igual, compare o campo mais significativo em seguida, e assim por diante, até encontrar um corpo desigual ou comparar o campo menos significativo. Aqui está um método `compareTo` para a classe `Número de Telefone` no Item 11 demonstrando essa técnica:

```
Múltiplos campos Comparável com campos primitivospublic
int compareTo(PhoneNumber pn) {int result =
    Short.compare(areaCode, pn.areaCode);
    if (resultado == 0) {
        resultado = Short.compare(prefixo,
            pn.prefix); if (resultado == 0)
            resultado = Short.compare(lineNum, pn.lineNum);
    }return result;}
```

No Java 8, a interface do Comparador era equipada com um conjunto de métodos de construção de comparadores, que permitem a construção fluente de comparadores. Esses comparadores podem então ser usados para implementar um método `compareTo`, conforme exigido pela interface `Comparável`. Muitos programadores preferem a concisão dessa abordagem, embora ela tenha um custo de desempenho modesto: ordenar arrays de instâncias `PhoneNumber` é cerca de 10% mais lento na minha máquina. Ao usar essa abordagem, considere usar a funcionalidade de importação estática do Java para poder se referir aos métodos de construção de `staticcomparator` por seus nomes simples para maior clareza e brevidade. Veja como o método `compareTo` para o `PhoneNumber` se apresenta usando essa abordagem:

```
Comparável com métodos de construção de comparadores, estática
privada final<PhoneNumber> Comparador COMPARATOR =
    comparingInt((Número de Telefone pn) -> pn.Código de área)
        .thenComparingInt(pn ->
            pn.prefix).thenComparingInt(pn -> pn.lineNum);

public int compareTo(PhoneNumber pn) {
    return COMPARATOR.compare(this, pn);}
}
```

Essa implementação constrói um comparador no momento de inicialização da classe, usando dois métodos de construção de comparadores. A primeira é comparar a intenção. É um método estático que pega uma função extratora de chave que mapeia uma referência de objeto para uma chave do tipo `int` e retorna um comparador que ordena instâncias de acordo com essa chave. No exemplo anterior, `comparingInt` pega um `lambda ()` que extrai o código de área de um `Número de Telefone` e retorna um `Comparador<PhoneNumber>` que ordena os números de telefone de acordo com seus códigos de área. Note que o `lambda` especifica explicitamente o tipo de seu parâmetro de entrada (`Número de Telefone pn`). Acontece que, nessa situação, a inferência de tipos do Java não é poderosa o suficiente para descobrir o tipo por si só, então somos forçados a ajudá-lo para fazer o programa compilar.

Se dois números de telefone têm o mesmo código de área, precisamos refinar ainda mais a comparação, e é exatamente isso que o segundo método de construção do comparador, o `thenComparingInt`, faz. É um método de instância no `Comparator` que assume uma função extratora de chave e retorna um comparador que primeiro aplica o comparador original e depois usa a chave extraída para desempatar. Você pode empilhar quantas chamadas quiser para `thenComparingInt`, resultando em uma ordem lexicográfica. No exemplo acima, empilhamos duas chamadas para `thenComparingInt`, resultando em uma ordem cuja chave secundária é o prefixo e cuja chave terciária é o número da linha. Note que não precisávamos especificar o tipo de parâmetro da função extratora de chave passada para nenhuma das chamadas para `thenComparingInt`: a inferência de tipo do Java foi inteligente o suficiente para descobrir isso sozinha.

A classe `Comparador` possui um conjunto completo de métodos de construção. Existem análogos a `comparingInt` e `thenComparingInt` para os tipos primitivos longo e duplo. As versões `int` também podem ser usadas para tipos de integrais mais restritos, como `short`, como em nosso exemplo `PhoneNumber`. As versões duplas também podem ser usadas para flutuação. Isso oferece cobertura de todos os tipos numéricos primitivos do Java.

Também existem métodos de construção de comparadores para tipos de referência de objetos. O método estático, chamado `comparação`, possui duas sobrecargas. Usa-se um extrator de chaves e a ordem natural das chaves. A segunda utiliza tanto um extrator de chaves quanto um comparador nas chaves extraídas. Existem três sobrecargas do método de instância, que é chamado `thenComparing`. Uma sobrecarga usa apenas um comparador e o usa para fornecer uma ordem secundária. Uma segunda sobrecarga requer apenas um extrator de chave e usa a ordem natural da chave como ordem secundária. A sobrecarga final requer tanto um extrator de chave quanto um comparador para serem usados nas chaves extraídas.

Ocasionalmente, você pode ver `compareTo` ou métodos de comparação que dependem do fato de que a diferença entre dois valores é negativa se o primeiro valor for menor que o segundo, zero se os dois valores forem iguais, e positivo se o primeiro valor for maior. Aqui está um exemplo:

```
QUEBRADO comparador baseado em diferenças - viola
transitividade!static<Object> Comparator hashCodeOrder = new Comparator<>() {
    public int compare(Object o1, Object o2) {
        return o1.hashCode() - o2.hashCode();};}
```

Não use essa técnica. Ele é repleto de perigos devido ao transbordo de inteiros e artefatos aritméticos de ponto flutuante IEEE 754 [JLS 15.20.1, 15.21.1]. Além disso, os métodos resultantes dificilmente serão significativamente mais rápidos do que aqueles escritos usando as técnicas descritas neste item. Use um método de comparação estática:

```
Comparador baseado em comparação estática metodótica<Object>
Comparador hashCodeOrder = novo Comparador<>() {
    public int compare(Object o1, Object o2) {
        return Integer.compare(o1.hashCode(), o2.hashCode());};}
```

ou um método de construção de comparador:

```
Comparador baseado em Construção do comparador,
metodoestático<Object> Comparador hashCodeOrder =
    Comparator.comparingInt(o -> o.hashCode());
```

Em resumo, sempre que você implementar uma classe de valor com uma ordenação sensata, deve fazer com que a classe implemente a interface *Comparável* para que suas instâncias possam ser facilmente ordenadas, pesquisadas e usadas em coletões baseadas em comparação. Ao comparar valores de campo nas implementações dos *compareTo* methods, evite o uso dos operadores *<* e *>*. Em vez disso, use os métodos comparáveis estáticos nas classes primitivas em caixa ou os métodos de construção do comparador na interface do *Comparador*.

CHAPTER 4

Classes e Interfaces

CLASSES e interfaces estão no cerne da linguagem de programação Java.

Eles são suas unidades básicas de abstração. A linguagem oferece muitos elementos poderosos que você pode usar para projetar classes e interfaces. Este capítulo contém diretrizes para ajudar você a fazer o melhor uso desses elementos, de modo que suas classes e interfaces sejam utilizáveis, robustas e flexíveis.

Item 15: Minimizar a acessibilidade de aulas e membros

O fator mais importante que distingue um componente bem projetado de um mal projetado é o grau em que o componente esconde seus dados internos e outros detalhes de implementação dos demais componentes. Um componente bem projetado esconde todos os detalhes de sua implementação, separando claramente sua API de sua implementação. Os componentes então se comunicam apenas por meio de suas APIs e estão alheios ao funcionamento interno uns dos outros. Esse conceito, conhecido como ocultação de informação ou encapsulamento, é um princípio fundamental do design de software [Parnas72].

Ocultar informações é importante por muitos motivos, a maioria deles decorrente do fato de que ele desacopla os componentes que compõem um sistema, permitindo que eles sejam desenvolvidos, testados, otimizados, usados, compreendidos e modificados isoladamente. Isso acelera o desenvolvimento do sistema porque os componentes podem ser desenvolvidos em paralelo. Isso alivia o peso da manutenção porque os componentes podem ser compreendidos mais rapidamente e depurados ou substituídos, sem medo de danificar outros componentes. Embora ocultar informações não cause, por si só, bom desempenho, permite uma afinação eficaz de desempenho: uma vez que um sistema está completo e o perfil determinou quais componentes estão causando problemas de desempenho (Item 67), esses componentes podem ser otimizados sem afetar a correção dos outros. O ocultamento de informações aumenta o reuso de softwares porque componentes que não são fortemente copulados frequentemente se mostram úteis em outros contextos além daqueles para os quais foram usados.

desenvolvido. Por fim, ocultar informações diminui o risco na construção de sistemas grandes porque componentes individuais podem se mostrar bem-sucedidos mesmo que o sistema não seja bem-sucedido.

Java possui muitas funcionalidades para ajudar no ocultamento de informações. O mecanismo de controle de acesso [JLS, 6.6] especifica a acessibilidade de classes, interfaces e membros. A acessibilidade de uma entidade é determinada pela localização de sua declaração e por qual, se houver, dos modificadores de acesso (privado, protegido e público) estão presentes na declaração. O uso adequado desses modificadores é essencial para ocultar informações.

A regra prática é simples: torne cada classe ou membro o mais inacessível possível. Em outras palavras, use o menor nível de acesso possível, consistente com o funcionamento correto do software que você está escrevendo.

Para classes e interfaces de nível superior (não aninhadas), existem apenas dois níveis de acesso possíveis: pacote-privado e público. Se você declarar uma classe de nível superior ou interagir com o modificador público, ela será pública; caso contrário, será privado no pacote. Se uma classe ou interface de alto nível puder ser tornada privada como pacote, deveria ser. Ao torná-lo privado como pacote, você faz parte da implementação em vez da API exportada, e pode modificá-lo, substituí-lo ou eliminá-lo em uma versão subsequente sem medo de prejudicar clientes existentes. Se você tornar isso público, é obrigado a apoiar para sempre para manter a compatibilidade.

Se uma classe ou interface de nível superior privada por pacote for usada por apenas uma classe, considere tornar a classe de nível superior uma classe privada estática aninhada da única classe que a utiliza (Item 24). Isso reduz sua acessibilidade de todas as classes em seu pacote para a única classe que o utiliza. Mas é muito mais importante reduzir a acessibilidade de uma classe publicamente gratuita do que de uma classe de nível superior privada por pacotes: a classe pública faz parte da API do pacote, enquanto a classe de nível superior privada já faz parte de sua implementação.

Para membros (campos, métodos, classes aninhadas e interfaces aninhadas), existem quatro níveis de acesso possíveis, listados aqui em ordem de acessibilidade crescente:

- **privado** — O membro é acessível apenas pela classe de nível superior onde foi declarado.
- **Pacote privado** — O membro é acessível a partir de qualquer classe do pacote onde ele é declarado. Tecnicamente conhecido como acesso padrão, esse é o nível de acesso que você obtém se nenhum modificador de acesso for especificado (exceto para membros de interface, que são públicos por padrão).
- **protegido** — O membro é acessível a partir de subclasses da classe onde é declarado (sujeito a algumas restrições [JLS, 6.6.2]) e de qualquer classe no pacote onde é declarado.
- **público** — O membro é acessível de qualquer lugar.

Depois de projetar cuidadosamente a API pública da sua turma, seu reflexo deve ser tornar todos os outros membros privados. Só se outra classe do mesmo pacote realmente precisar acessar um membro você deve remover o modificador privado, tornando o pacote do membro privado. Se você se pegar fazendo isso com frequência, deve reexamina o design do seu sistema para ver se outra decomposição pode gerar classes que sejam melhor desacopladas entre si. Dito isso, tanto membros privados quanto membros privados fazem parte da implementação de uma classe e normalmente não impactam sua API exportada. Esses campos, no entanto, podem "vazar" para a API exportada se a classe implementar o `Serializable` (Itens 86 e 87).

Para os membros das turmas públicas, ocorre um grande aumento na acessibilidade quando o nível de acesso passa de pacote privado para protegido. Um membro protegido faz parte da API exportada da classe e deve ser suportado para sempre. Além disso, um membro protegido de uma classe exportada representa um compromisso público com um detalhe de implementação (Item 19). A necessidade de membros protegidos deveria ser relativamente rara.

Existe uma regra fundamental que restringe sua capacidade de reduzir o acesso a metanfetaminas. Se um método sobrescreve um método de superclasse, ele não pode ter um nível de acesso mais restritivo na subclasse do que na superclasse [JLS, 8.4.8.3]. Isso é necessário para garantir que uma instância da subclasse seja utilizável em qualquer lugar onde uma instância da superclasse seja utilizável (o princípio da substituição de Liskov, veja o Item 15). Se você violar essa regra, o compilador gerará uma mensagem de erro quando você tentar compilar a subclasse. Um caso especial dessa regra é que, se uma classe implementar uma interface, todos os métodos de classe que estão na interface devem ser declarados públicos na classe.

Para facilitar o teste do seu código, você pode se sentir tentado a tornar uma classe, interface ou membro mais acessível do que seria necessário. Isso está tranquilo até certo ponto. É aceitável tornar um membro privado de uma classe pública para testá-lo, mas não é aceitável aumentar ainda mais a acessibilidade. Em outras palavras, não é aceitável tornar uma classe, interface ou membro parte da API exportada de uma era de pacotes para facilitar os testes. Felizmente, também não é necessário, pois os testes podem ser executados como parte do pacote em teste, ganhando assim acesso aos seus elementos privados de pacotes.

Campos de instância das classes públicas raramente devem ser públicos (Item 16). Se um campo de instância não for final ou for uma referência a um objeto mutável, então ao torná-lo público, você perde a capacidade de limitar os valores que podem ser armazenados no campo. Isso significa que você abre mão da capacidade de impor invariantes envolvendo o campo. Além disso, você perde a capacidade de agir quando o campo é modificado, então classes com campos mutáveis públicos geralmente não são seguras para threads. Mesmo que um campo seja final e se refira a um objeto imutável, ao torná-lo público você perde a flexibilidade de mudar para uma nova representação interna de dados na qual o campo não existe.

O mesmo conselho se aplica a campos estáticos, com uma exceção. Você pode expor constantes por meio de campos finais estáticos públicos, assumindo que as constantes formam parte integral da abstração fornecida pela classe. Por convenção, tais campos possuem nomes compostos por letras maiúsculas, com palavras separadas por sublinhados (Item 68). É fundamental que esses campos contenham valores primitivos ou referências a objetos imutáveis (Item 17). um corpo contendo uma referência a um objeto mutável tem todas as desvantagens de um corpo não final. Embora a referência não possa ser modificada, o objeto referenciado pode ser modificado — com resultados desastrosos.

Note que um array de comprimento diferente de zero é sempre mutável, então é errado que uma classe tenha um campo público estático do array final, ou um acessor que retorne tal campo. Se uma classe possui tal campo ou acessor, os clientes poderão modificar as tendências do array. Essa é uma fonte frequente de falhas de segurança:

```
Potencial buraco de segurança!estática
pública final Coisa final[] VALORES =    { ... };
```

Cuidado com o fato de que alguns IDEs geram acessores que retornam referências a campos de matrizes privadas, resultando exatamente nesse problema. Existem duas maneiras de resolver o problema. Você pode tornar o array público privado e adicionar uma lista pública imutável:

```
private static final Thing[] PRIVATE_VALUES = { ... };p
ublic static final<Thing> List VALUES =
    Collections.unmodifiableList(Arrays.asList(PRIVATE_VALUES));
```

Alternativamente, você pode tornar o array privado e adicionar um método público que retorne uma cópia de um array privado:

```
private static final Thing[] PRIVATE_VALUES = { ... };p
ublic static final Thing[] values() {
    return PRIVATE_VALUES.clone();}
```

Para escolher entre essas alternativas, pense no que o cliente provavelmente fará com o resultado. Qual tipo de devolução será mais conveniente? Qual dará melhor desempenho?

A partir do Java 9, há dois níveis adicionais de acesso implícito introduzidos como parte do sistema de módulos. Um módulo é um agrupamento de pacotes, assim como um pacote é um agrupamento de classes. Um módulo pode exportar explicitamente alguns de seus pacotes via declarações de exportação em sua declaração de módulo (que, por convenção, está contida em um arquivo fonte chamado `module-info.java`). Membros públicos e protegidos que exportam pacotes em um módulo são inacessíveis fora do módulo; dentro do

módulo, acessibilidade não é afetada pelas declarações de exportação. Usar o sistema de módulos permite compartilhar classes entre pacotes dentro de um módulo sem que eles sejam visíveis para o mundo inteiro. Membros públicos e protegidos de classes públicas em pacotes não exportados dão origem aos dois níveis de acesso implícitos, que são análogos intramodulares dos níveis público normal e protegido. A necessidade desse tipo de compartilhamento é relativamente rara e muitas vezes pode ser eliminada reorganizando as classes dentro dos seus pacotes.

Diferente dos quatro níveis principais de acesso, os dois níveis baseados em módulos são em grande parte consultivos. Se você colocar o arquivo JAR de um módulo no caminho da classe da sua aplicação em vez do caminho do módulo, os pacotes no módulo reverterem ao seu comportamento não modular: todos os membros públicos e protegidos das classes públicas dos pacotes têm sua acessibilidade normal, independentemente de os pacotes serem exportados pelo módulo [Reinhold, 1.2]. O único lugar onde os níveis de acesso recém-introduzidos são rigorosamente aplicados é o próprio JDK: os pacotes não exportados nas bibliotecas Java são realmente inacessíveis fora de seus módulos.

Além de a proteção de acesso proporcionada pelos módulos é de utilidade limitada para o programador Java típico, e em grande parte de natureza consultiva; Para aproveitá-la, você deve agrupar seus pacotes em módulos, tornar todas as dependências explícitas nas declarações de módulos, reorganizar sua árvore de origem e tomar ações especiais para acomodar qualquer acesso a pacotes não modularizados dentro dos seus módulos [Reinhold, 3]. Ainda é cedo para dizer se os módulos alcançarão uso amplo fora do próprio JDK. Enquanto isso, parece melhor evitá-los, a menos que você tenha uma necessidade urgente.

Resumindo, você deve reduzir o acesso aos elementos do programa o máximo possível (dentro do razoável). Após projetar cuidadosamente uma API pública mínima, você deve evitar que classes isoladas, interfaces ou membros se tornem parte da API. Com exceção dos campos finais estáticos públicos, que servem como constantes, as classes públicas não devem ter campos públicos. Certifique-se de que os objetos referenciados pelos campos finais estáticos públicos sejam imutáveis.

Item 16: Em classes públicas, use métodos de acesso, não campos públicos

Ocasionalmente, você pode se sentir tentado a escrever classes degeneradas que não servem para nada além de agrupar campos de instância:

```
Classes degeneradas como essa não deveriam ser
públicas!class Point {
    duplo público x;duplo
    público y;}
```

Como os campos de dados dessas classes são acessados diretamente, essas classes não oferecem os benefícios da encapsulação (Item 15). Você não pode mudar a representação sem mudar a API, não pode impor invariantes e não pode tomar ações auxiliares quando um campo é acessado. Programadores orientados a objetos de linha dura consideram que tais classes são anátema e devem sempre ser substituídas por classes com campos privados e métodos públicos de acesso (getters) e, para classes mutáveis, mutadores (setters):

```
Encapsulamento de dados por métodos de acessor e mutadoresclass
Point {
    Soldado Duplo X;
    Soldado Duplo Y;

    Ponto público (duplo x, duplo y) {
        this.x =
        x;this.y = y;}

    public double getX() { return x;
    }public double getY() { return y; }

    empty público setX(duplo x) { this.x = x;
    }conjunto público void Y(duplo y) { this.y = y; }}
```

Certamente, os mais radicais estão certos quando se trata de aulas públicas: se uma aula for acessível fora do seu pacote, forneça métodos de acesso para preservar a flexibilidade de mudar a representação interna da classe. Se uma classe pública expõe seus campos de dados, toda esperança de alterar sua representação se perde porque o código cliente pode ser distribuído amplamente.

No entanto, se uma classe é privada em pacotes ou uma classe aninhada privada, não há nada inerentemente errado em expor seus campos de dados — assumindo que eles façam um

Trabalho adequado para descrever a abstração fornecida pela turma. Essa abordagem gera menos desordem visual do que a abordagem do método acessório, tanto na definição de classe quanto no código cliente que a utiliza. Embora o código cliente esteja vinculado à representação interna da classe, esse código fica confinado ao pacote que contém a classe. Se uma mudança na representação se tornar desejável, você pode fazer a alteração sem tocar em nenhum código fora do pacote. No caso de uma classe aninhada privada, o escopo da mudança é ainda mais restrito à classe envolvente.

Várias classes nas bibliotecas da plataforma Java violam o conselho de que as classes públicas não devem expor campos diretamente. Exemplos proeminentes incluem as classes `Point` e `Dimension` no pacote `java.awt`. Em vez de exemplos a serem imitados, essas classes devem ser vistas como histórias de advertência. Como descrito no Item 67, a decisão de expor os componentes internos da classe `Dimensão` resultou em um sério problema de desempenho que ainda existe hoje.

Embora nunca seja uma boa ideia para uma aula pública expor campos diretamente, isso não é tão prejudicial se os campos forem imutáveis. Você não pode mudar a representação dessa classe sem mudar sua API, e não pode tomar ações auxiliares quando um campo é lido, mas pode impor invariantes. Por exemplo, essa classe garante que cada instância representa um tempo válido:

```
Turma pública com campos imutáveis expostos - questionablepublic
final class Time {
    Privada Estática Final Int HOURS_PER_DAY    = 24;
    Privado Estático Final Int MINUTES_PER_HOUR = 60;

    Hora final de INT pública; minuto
    final de internacionalidade público;

    Public Time(int hour, int minute) {
        se (hora < 0 || hora >= HOURS_PER_DAY)
            lançar um novo IllegalArgumentException("Hora: " + hora); se
            (minuto < 0 || minuto >= MINUTES_PER_HOUR)
                jogar um novo IllegalArgumentException("Min: " + minuto); this.hour =
                hour; this.minute = minute; }... // Restante omitido}
```

Em resumo, as aulas públicas nunca deveriam expor campos mutáveis. É menos prejudicial, embora ainda questionável, que as aulas públicas expõem campos imutáveis. No entanto, às vezes é desejável que classes privadas ou privadas aninhadas exponham campos, sejam mutáveis ou imutáveis.

Item 17: Minimizar a mutabilidade

Uma classe imutável é simplesmente uma classe cujas instâncias não podem ser modificadas. Todas as informações contidas em cada instância são fixas durante a vida útil do objeto, mudanças não podem ser observadas. As bibliotecas da plataforma Java contêm muitas classes imutáveis, incluindo `String`, as classes primitivas em caixa, e `BigInteger` e `BigDecimal`. Existem muitos bons motivos para isso: classes imutáveis são mais fáceis de projetar, implementar e usar do que classes mutáveis. Eles são menos propensos a erros e mais seguros.

Para tornar uma classe imutável, siga estas cinco regras:

1. Não forneça métodos que modifiquem o estado do objeto (conhecidos como mutadores).
2. Garantir que a classe não possa ser estendida. Isso impede que subclasses descuidadas ou maliciosas comprometam o comportamento imutável da classe ao se comportarem como se o estado do objeto tivesse mudado. Prevenir subclasses geralmente é feito tornando a disciplina final, mas há uma alternativa que discutiremos depois.
3. Torne todos os campos finais. Isso expressa claramente sua intenção de uma forma imposta pelo sistema. Além disso, é necessário garantir o comportamento correto se uma referência a uma instância recém-criada for passada de um thread para outro sem sincronização, conforme detalhado no modelo de memória [JLS, 17.5; Goetz06, 16].
4. Tornar todos os campos privados. Isso impede que os clientes obtenham acesso a objetos mutáveis referenciados por campos e modifiquem esses objetos diretamente. Embora seja tecnicamente permitido que classes imutáveis tenham campos finais públicos contendo valores primitivos ou referências a objetos imutáveis, isso não é recomendado porque impede a alteração da representação interna em uma versão posterior (Itens 15 e 16).
5. Garantir acesso exclusivo a quaisquer componentes mutáveis. Se sua classe tiver qualquer campo que se refere a objetos mutáveis, certifique-se de que os clientes da classe não possam obter referências a esses objetos. Nunca inicialize tal campo em uma referência de objeto fornecido pelo cliente nem retorne o campo de um acessório. Faça cópias defensivas (Item 50) em construtores, acessórios e métodos `readObject` (Item 88).

Muitas das classes de exemplo em itens anteriores são imutáveis. Uma dessas classes é `Número de Telefone` no Item 11, que possui acessórios para cada atributo, mas não mutadores correspondentes. Aqui está um exemplo um pouco mais complexo:

```

Classe de número complexo imutável
classe final pública Complexo {
    Final Privada Dupla Re; Individual
    Individual Final Double Im;

    Complexo público (double re, double im) {
        this.re = direita;
        this.im = em;}

    público duplo realPart()          { retornar re; }
    public double imaginaryPart() { return im; }

    public Complex plus(Complex c) {
        return new Complex(re + c.re, im + c.im);}

    Complexo público menos(Complexo c) {
        return new Complex(re - c.re, im - c.im);}

    Tempos complexos públicos (Complexo c) {
        retorne novo complexo (re * c.re - im * c.im,
                                re * c.im + im * c.re);
    }

    Complexo público dividido(Complexo c) {
        TPM duplo = c.re * c.re + c.im * c.im; retorne novo
        Complex((re * c.re + im * c.im) / tmp,
                (im * c.re - re * c.im) / tmp);
    }

    @Override booleano público é igual a (Object o) {
        if (o == isto)
            retorno verdadeiro; se (!( o
            instância de Complexo))
            retorne falso; Complexo c
            = (Complexo) o;

        Veja a página 47 para descobrir por que usamos compare em vez
        de ==return Double.compare(c.re, re) == 0

        && Double.compare(c.im, im) == 0;
    }@Override public int hashCode() {

        return 31 * Double.hashCode(re) + Double.hashCode(im);}

    @Override String toString() {return "(" + re + "
    + " + im + "i");}}

```

Essa classe representa um número complexo (um número com partes reais e imaginárias). Além dos métodos padrão de Objetos, ele fornece acessores para as partes real e imaginária e fornece as quatro operações aritméticas básicas: adição, subtração, multiplicação e divisão. Note como as operações aritméticas criam e retornam uma nova instância Complexa em vez de modificar essa instância. Esse padrão é conhecido como abordagem funcional porque os métodos retornam o resultado da aplicação de uma função ao seu operando, sem modificá-la. Compare isso com a abordagem procedural ou imperativa, na qual métodos aplicam um procedimento ao seu operando, causando a mudança de seu estado. Note que os nomes dos métodos são preposições (como *mais*) e não verbos (como *soma*). Isso enfatiza o fato de que os métodos não alteram os valores dos objetos. As classes `BigInteger` e `BigDecimal` não obedeciam a essa convenção de nomenclatura, o que levou a muitos erros de uso.

A abordagem funcional pode parecer antinatural se você não estiver familiarizado com ela, mas ela permite a imutabilidade, o que traz muitas vantagens. Objetos imutáveis são simples. Um objeto imutável pode estar exatamente em um estado, o estado em que foi criado. Se você garantir que todos os construtores estabeleçam invariantes de classe, então é garantido que esses invariantes permanecerão verdadeiros para sempre, sem nenhum esforço adicional da sua parte ou do programador que usa a classe. Objetos mutáveis, por outro lado, podem ter espaços de estados arbitrariamente complexos. Se a documentação não fornecer uma descrição precisa das transições de estado realizadas pelos métodos *muta-tor*, pode ser difícil ou impossível usar uma classe mutável de forma confiável.

Objetos imutáveis são inerentemente seguros para fios; eles não requerem sincronização. Eles não podem ser corrompidos por múltiplas threads acessando-as simultaneamente. Essa é, de longe, a abordagem mais fácil para garantir a segurança da rosca. Como nenhum thread pode observar qualquer efeito de outro thread sobre um objeto imutável, objetos imutáveis podem ser compartilhados livremente. Classes imutáveis, portanto, devem incentivar os clientes a reutilizar instâncias existentes sempre que possível. Uma maneira fácil de fazer isso é fornecer constantes finais estáticas públicas para valores comumente usados. Por exemplo, a classe `Complexo` pode fornecer estas constantes:

```
Complexo final estático público ZERO = novo Complexo(0, 0);
estática pública final Complexo UM = novo Complexo(1, 0); Complexo
final estático público Complexo I = novo Complexo(0, 1);
```

Essa abordagem pode ser levada um passo adiante. Uma classe imutável pode fornecer fábricas estáticas (Item 1) que armazenam em cache instâncias frequentemente solicitadas para evitar criar novas instâncias quando as existentes servirem. Todas as classes primitivas em `box` e o `BigInteger` fazem isso. O uso dessas fábricas estáticas faz com que os clientes compartilhem instâncias em vez de criar novas, reduzindo o consumo de memória e a coleta de lixo

custos. Optar por fábricas estáticas em vez de construtores públicos ao projetar uma nova classe oferece flexibilidade para adicionar cache depois, sem modificar clientes.

Uma consequência do fato de que objetos imutáveis podem ser compartilhados livremente é que você nunca precisa fazer cópias defensivas deles (Item 50). Na verdade, você nunca precisa fazer cópias porque as cópias seriam para sempre equivalentes aos originais. Portanto, você não precisa e não deve fornecer um método clone ou construtor de cópias (Item 13) em uma classe imutável. Isso não era bem compreendido nos primeiros dias da plataforma Java, então a classe String tem um construtor de cópia, mas raramente, se é que alguma vez, deve ser usado (Item 6).

Você não só pode compartilhar objetos imutáveis, mas eles podem compartilhar seus internos. Por exemplo, a classe BigInteger usa uma representação de magnitude de sinal internamente. O sinal é representado por um int, e a magnitude é representada por um array int. O método da negação produz um novo Grandeinteiro de magnitude semelhante e sinal oposto. Não precisa copiar o array mesmo sendo mutável; o recém-criado BigInteger aponta para o mesmo array interno do original.

Objetos imutáveis são ótimos blocos de construção para outros objetos, sejam eles mutáveis ou imutáveis. É muito mais fácil manter os invariantes de um objeto complexo se você souber que seus objetos componentes não vão mudar por baixo dele. Um caso especial desse princípio é que objetos imutáveis formam ótimas chaves de mapa e elementos de conjunto: você não precisa se preocupar com seus valores mudando uma vez que estejam no mapa ou conjunto, o que destruiria os invariantes do mapa ou conjunto.

Objetos imutáveis fornecem atomicidade de falha gratuitamente (Item 76). O estado deles nunca muda, então não há possibilidade de uma inconsistência temporária.

A principal desvantagem das classes imutáveis é que elas requerem um objeto separado para cada valor distinto. Criar esses objetos pode ser caro, especialmente se forem grandes. Por exemplo, suponha que você tenha um BigInteger de um milhão de bits e queira mudar seu bit de ordem baixa:

```
Grandeinteiro moby = ...;
moby = moby.flipBit(0);
```

O método flipBit cria uma nova instância BigInteger, também com um milhão de bits de comprimento, que difere da original em apenas um bit. A operação requer tempo e espaço proporcionais ao tamanho do BigInteger. Compare isso tojava.util.BitSet. Assim como o BigInteger, o BitSet representa uma sequência arbitrariamente longa de bits, mas ao contrário do BigInteger, o BitSet é mutável. A classe BitSet fornece um método que permite alterar o estado de um único bit de uma instância de um milhão de bits em tempo constante:

```
BitSet moby = ...;
moby.flip(0);
```


O problema de desempenho é ampliado se você realizar uma operação em múltiplos passos que gera um novo objeto a cada etapa, eventualmente descartando todos os objetos, exceto o resultado final. Existem duas abordagens para lidar com esse problema. A primeira é adivinhar quais operações multipasso serão comumente necessárias e fornecê-las como primitivas. Se uma operação multistep for fornecida como primitiva, a classe `Immutable` não precisa criar um objeto separado a cada etapa. Internamente, a classe imutável pode ser arbitrariamente inteligente. Por exemplo, o `BigInteger` possui uma "classe companheira" mutável com a idade dos pacotes-privada que usa para acelerar operações multipasso, como a exponenciação modular. É muito mais difícil usar a classe companheira mutável do que usar `BigInteger`, por todas as razões mencionadas anteriormente. Felizmente, você não precisa usá-lo: os implementadores do `BigInteger` fizeram o trabalho difícil por você.

A abordagem de classe companheira mutável em pacotes e privada funciona bem se você conseguir prever com precisão quais operações complexas os clientes vão querer executar na classe `Immutable`. Se não, o melhor é oferecer uma aula pública de `MutableCompanion`. O principal exemplo dessa abordagem nas bibliotecas da plataforma Java é a classe `String`, cujo companheiro mutável é o `StringBuilder` (e seu predecessor obsoleto, o `StringBuffer`).

Agora que você sabe como criar uma classe imutável e entende os prós e contras da imutabilidade, vamos discutir algumas alternativas de design. Lembre-se de que, para garantir a imutabilidade, uma classe não deve permitir ser subclassificada. Isso pode ser feito tornando a disciplina final, mas existe outra alternativa, mais flexível. Em vez de tornar uma classe final imutável, você pode tornar todos os seus construtores privados ou pacotes privados e adicionar fábricas públicas estáticas no lugar dos construtores públicos (Item 1). Para concretizar isso, veja como o `Complex` ficaria se você adotasse essa abordagem:

```
Classe imutável com fábricas estáticas em vez de constructors public
class Complexo {
    final Privada Dupla re; Individual
    Individual final Double im;

    privado Complex(double re, double im) {
        this.re = direita;
        this.im = em;}

    Valor Complexo estático público de(double re, double im) {
        return new Complex(re, im);}

    ... // Restante inalterado}
```

Essa abordagem costuma ser a melhor alternativa. É o mais flexível porque permite o uso de múltiplas classes de implementação privadas de pacotes. Para seus clientes que residem fora do seu pacote, a classe imutável é efetivamente final porque é impossível estender uma classe que vem de outro pacote e que não possui um construtor público ou protegido. Além de permitir a flexibilidade de múltiplas classes de implementação, essa abordagem possibilita ajustar o desempenho da classe em versões subsequentes, melhorando as capacidades de cache de objetos das fábricas estáticas.

Não era amplamente entendido que classes imutáveis precisavam ser efetivamente finais quando `BigInteger` e `BigDecimal` eram escritos, então todos os seus métodos podem ser sobrepostos. Infelizmente, isso não pôde ser corrigido posteriormente, mantendo a compatibilidade retroativa. Se você escreve uma classe cuja segurança depende da imutabilidade de um argumento `BigInteger` ou `BigDecimal` de um cliente não confiável, você deve verificar se o argumento é um "`BigInteger` ou `BigDecimal` real", e não uma instância de uma subclasse não confiável. Se for o segundo, você deve copiá-lo definitivamente sob a suposição de que pode ser mutável (Item 50):

```
public static BigInteger safeInstance(BigInteger val) {
    return val.getClass() == BigInteger.class ?
        val : novo BigInteger(val.toByteArray());
}
```

A lista de regras para classes imutáveis no início deste item diz que nenhum método pode modificar o objeto e que todos os seus campos devem ser finais. Na verdade, essas regras são um pouco mais fortes do que o necessário e podem ser relaxadas para melhorar o desempenho. Na verdade, nenhum método pode produzir uma mudança externamente visível no estado do objeto. No entanto, algumas classes imutáveis possuem um ou mais campos não finais nos quais armazenam em cache os resultados de cálculos caros na primeira vez que são necessários. Se o mesmo valor for solicitado novamente, o valor em cache é retornado, economizando o custo do recálculo. Esse truque funciona justamente porque o objeto é imutável, o que garante que o cálculo produziria o mesmo resultado se fosse repetido.

Por exemplo, o método `hashCode` do `PhoneNumber` (Item 11, página 53) calcula o código hash na primeira vez que ele é invocado e o armazena em cache caso seja invocado novamente. Essa técnica, um exemplo de inicialização preguiçosa (Item 83), também é usada pelo `String`.

Uma ressalva deve ser acrescentada sobre serializabilidade. Se você escolher que sua classe imutável implemente `Serializable` e ela contiver um ou mais campos que se referem a objetos mutáveis, você deve fornecer um método explícito `readObject` ou `readResolve`, ou usar o `ObjectOutputStream.writeUnshared` e

`ObjectInputStream.readUnshare`, mesmo que a forma serializada padrão seja aceitável. Caso contrário, um atacante poderia criar uma instância mutável da sua classe. Este tema é abordado em detalhes no Item 88.

Resumindo, resista à vontade de escrever um levantador para cada colocador. As classes deveriam ser imutáveis, a menos que haja um motivo muito bom para torná-las mutáveis. Classes imutáveis oferecem muitas vantagens, e sua única desvantagem é o potencial de problemas de desempenho em certas circunstâncias. Você deve sempre tornar objetos de valor pequeno, como Número de Telefone e Complexo, imutáveis. (Existem várias classes nas bibliotecas da plataforma Java, como `java.util.Date` e `java.awt.Point`, que deveriam ser imutáveis, mas não são.) Você deveria considerar seriamente tornar objetos de valor maior, como `String` e `BigInteger`, imutáveis também. Você deve fornecer uma classe companheira móvel pública para sua classe imutável apenas depois de confirmar que ela é necessária para alcançar desempenho satisfatório (Item 67).

Existem algumas classes para as quais a imutabilidade é impraticável. Se uma classe não pode ser tornada imutável, limite sua mutabilidade o máximo possível. Reduzir o número de estados em que um objeto pode existir facilita a razão sobre o objeto e diminui a probabilidade de erros. Portanto, torne cada campo final, a menos que haja um motivo convincente para tornar não final. Combinando o conselho deste item com o do item 15, sua inclinação natural deve ser declarar todos os exames finais particulares, a menos que haja um bom motivo para fazer diferente.

Construtores devem criar objetos totalmente inicializados com todos os seus invariantes estabelecidos. Não forneça um método público de inicialização separado do `theconstructor` ou da `static factory`, a menos que haja um motivo convincente para isso. Da mesma forma, não forneça um método de "reinicialização" que permita reutilizar um objeto como se tivesse sido construído com um estado inicial diferente. Esses métodos geralmente proporcionam pouco ou nenhum benefício de desempenho, à custa de uma complexidade maior.

A classe `CountDownLatch` exemplifica esses princípios. É mutável, mas seu espaço de estado é mantido intencionalmente pequeno. Você cria uma instância, usa uma vez e pronto: quando a contagem do travador regressivo chegar a zero, você pode não reutilizá-la.

Uma nota final deve ser adicionada sobre a classe Complexa neste item. Este exemplo tinha a intenção de ilustrar apenas a imutabilidade. Não é uma implementação de números complexos de força industrial. Ele usa as fórmulas padrão para multiplicação complexa e divisão, que não são corretamente arredondadas e fornecem poorsemântica para NaNs complexos e infinitos [Kahan91, Smith62, Thomas94].

Item 18: Composição de favores em detrimento da herança

A herança é uma forma poderosa de realizar a reutilização de código, mas nem sempre é a melhor ferramenta para o trabalho. Se usado de forma inadequada, leva a softwares frágeis. É seguro usar a herança dentro de um pacote, onde as implementações da subclasse e da superclasse estão sob o controle dos mesmos programadores. Também é seguro usar a herança ao estender classes especificamente projetadas e documentadas para extensão (Item 19). Herdar classes comuns de concreto através dos limites de pacotes, no entanto, é perigoso. Como lembrete, este livro usa a palavra "herança" para significar herança de implementação (quando uma classe estende outra). Os problemas discutidos neste item não se aplicam à herança de interface (quando uma classe implementa uma interface ou quando uma interface estende outra).

Ao contrário da invocação por método, a herança viola o encapsulamento [Snyder86]. Em outras palavras, uma subclasse depende dos detalhes de implementação de sua superclasse para sua função adequada. A implementação da superclasse pode mudar de uma versão para outra, e se isso acontecer, a subclasse pode quebrar, mesmo que seu código não tenha sido tocado. Como consequência, uma subclasse deve evoluir em conjunto com sua superclasse, a menos que os autores da superclasse a tenham projetado e documentado especificamente para ser estendida.

Para tornar isso concreto, vamos supor que tenhamos um programa que usa um HashSet. To ajustar o desempenho do nosso programa, precisamos consultar o HashSet para saber quantos elementos foram adicionados desde que foi criado (não confundir com seu tamanho atual, que diminui quando um elemento é removido).

Para fornecer essa funcionalidade, escrevemos uma variante HashSet que mantém a contagem do número de tentativas de inserção de elementos e exporta um acessor para essa contagem. A HashSetclass contém dois métodos capazes de adicionar elementos, add e addAll, então weoverride ambos os métodos:

```
Broken - Uso inadequado da classe inheritance!public
InstrumentedHashSet<E> estende o HashSet<E> {
    O número de tentativas de inserção de
    elementosprivateprivate, int, addCount = 0;

    public InstrumentedHashSet() {}

    public InstrumentedHashSet(int initCap, float loadFactor) {
        super(initCap, loadFactor);}
```

```

@Override publicamente booleano add(E e) {
    addCount++; return super.add(e);} @Override booleano público
    addAll(Collection<? estende E> c) {

        addCount += c.size(); return
        super.addAll(c);} public int
        getAddCount() {

            return addCount;}}

```

Essa aula parece razoável, mas não funciona. Suponha que criemos uma instância e adicionemos três elementos usando o método `addAll`. A propósito, note que criamos uma lista usando o método de fábrica estático `List.of`, que foi adicionado no Java 9; se você está usando uma versão anterior, use `Arrays.asList` em vez disso:

```

InstrumentedHashSet<String> s = novo InstrumentedHashSet<>();
s.addAll(Lista.de("Estalo", "Estalar", "Estalar"));

```

Esperaríamos que o método `getAddCount` retornasse três neste ponto, mas ele retorna seis. O que deu errado? Internamente, o método `addAll` do `HashSet` é implêndido sobre seu método de adição, embora o `HashSet`, razoavelmente, não documente esse detalhe de implementação. O método `addAll` no `InstrumentedHashSet` adicionava três ao `addCount` e então invocava a implementação do `addAll` do `HashSet` usando `super.addAll`. Isso, por sua vez, invocava o método `add`, como sobrescrito em `InstrumentedHashSet`, uma vez para cada elemento. Cada uma dessas três invocações adicionou mais uma ao `addCount`, para um aumento total de seis: cada elemento adicionado com o método `addAll` é contado duas vezes.

Poderíamos "consertar" a subclasse eliminando sua substituição do método `addAll`. Embora a classe resultante funcione, ela dependeria para sua função correta do fato de que o método `addAll` do `HashSet` é implementado sobre seu método `add`. Esse "autouso" é um detalhe de implementação, não garantido para se manter em todas as implementações da plataforma Java e sujeito a alterações de lançamento para lançamento. Portanto, a classe `InstrumentedHashSet` resultante seria frágil.

Seria um pouco melhor sobrescrever o método `addAll` para iterar sobre a coleção especificada, chamando o método `add` uma vez para cada elemento. Isso garantiria o resultado correto, independentemente de o método `addAll` do `HashSet` ser implementado sobre o método `add`, pois a implementação do `addAll` do `HashSet` não seria mais invocada. Essa técnica, no entanto, não resolve todos os nossos problemas. Isso equivale a reimplementar métodos de superclasse que podem ou não

resultar em autouso, que é difícil, demorado, propenso a erros e pode reduzir o desempenho. Além disso, nem sempre é possível porque alguns métodos não podem ser implementados sem acesso a campos privados inacessíveis à subclasse.

Uma causa relacionada à fragilidade em subclasses é que sua superclasse pode adquirir novos métodos em versões subsequentes. Suponha que um programa dependa para sua segurança do fato de que todos os elementos inseridos em alguma coleção satisfazem algum predicado. Isso pode ser garantido subclassificando a coleção e sobrescrevendo cada método capaz de adicionar um elemento para garantir que o predicado seja satisfeito antes de adicionar o elemento. Isso funciona bem até que um novo método capaz de inserir um elemento seja adicionado à superclasse em uma versão subsequente. Quando isso acontece, torna-se possível adicionar um elemento "ilegal" apenas invocando o novo método, que não é sobreposto na subclasse. Isso não é um problema puramente teórico. Várias falhas de segurança dessa natureza precisaram ser corrigidas quando Hashtable e Vector foram readaptadas para participar do Structure de Coleções.

Ambos os problemas decorrem de métodos que sobrescrevem. Você pode pensar que é seguro estender uma aula se apenas adicionar novos métodos e evitar sobrescrever métodos existentes. Embora esse tipo de extensão seja muito mais seguro, não está isento de riscos. Se a superclasse adquirir um novo método em uma versão subsequente e você tiver o azar de ter dado à subclasse um método com a mesma assinatura e um tipo de retorno diferente, sua subclasse não compilará mais [JLS, 8.4.8.3]. Se você deu à subclasse um método com a mesma assinatura e tipo de retorno do novo método da superclasse, então você está sobrepondo isso, então está sujeito aos problemas descritos anteriormente. Além disso, é duvidoso que seu método cumpra o contrato do novo método de superclasse, pois esse contrato ainda não havia sido escrito quando você escreveu o método da subclasse.

Felizmente, existe uma forma de evitar todos os problemas descritos acima. Em vez de estender uma disciplina existente, dê à sua nova turma um campo privado que faça referência a uma instância da classe existente. Esse design é chamado de composição porque a classe existente se torna um componente da nova. Cada método de instância na nova classe invoca o método correspondente na instância contida da classe existente e retorna os resultados. Isso é conhecido como encaminhamento, e os métodos da nova classe são chamados de métodos de encaminhamento. A classe resultante será sólida, sem dependências dos detalhes de implementação da classe existente. Mesmo adicionar novos métodos à classe existente não terá impacto na nova classe. Para concretizar isso, aqui está um substituto para o InstrumentedHashSet que usa a abordagem de composição e encaminhamento. Note que a implementação está dividida em duas partes: a própria classe e uma classe de encaminhamento reutilizável, que contém todos os métodos de encaminhamento e nada mais:

Classe Wrapper - usa composição no lugar da classe inheritance public InstrumentedSet<E> estende<E> ForwardingSet {

private int addCount = 0;

public InstrumentedSet(Set<E> s) {
super(s);}

@Override publicamente booleano add(E e) {
addCount++; return super.add(e);} @Override booleano público
addAll(Collection<? estende E> c) {

addCount += c.size(); return
super.addAll(c);} public int
getAddCount() {

return addCount;}}

Reutilizável encaminhamento classe public ForwardingSet<E> implementa Set<E> {

finalis<E> privados Conjuntos s; publicamente ForwardingSet<E>Set s) { this.s =
s; }

Limpar o vazio público() { s.clear(); }
boolean public contains(Object o) { return s.contains(o); }
boolean public isEmpty() { return s.isEmpty(); } public int size()
{ return s.size(); } iterador iterador<E> publico() { return
s.iterator(); } publicamente booleano add(E e) { return
s.add(e); } boolean public remove(Object o) { return
s.remove(o); } booleano público contém Tudo(Collection<?> c)

{ return s.containsAll(c); }
publicamente booleano addAll(Collection<? estende E> c)
{ return s.addAll(c); }
boolean public removeAll(Collection<?> c)
{ return s.removeAll(c); }
retenção booleana pública Tudo(Coleção<?> c)
{ return s.retainAll(c); }

Public Object[] toArray()
{ return s.toArray(); }
público <T> T[] toArray(T[] a) { return s.toArray(a); }
@Override booleano público é igual a (Objeto o)
{ return s.equals(o); }

@Override int público hashCode() { return s.hashCode(); }
@Override public String toString() { return s.toString(); } }

O design da classe `InstrumentedSet` é possibilitado pela existência da interface `theSet`, que captura a funcionalidade da classe `HashSet`. Além de robusto, esse design é extremamente flexível. A classe `InstrumentedSet` impeleita a interface `Set` e possui um único construtor cujo argumento também é do tipo `Set`. Em essência, a classe transforma um `Set` em outro, adicionando a funcionalidade de instrução. Diferente da abordagem baseada em herança, que funciona apenas para uma única classe concreta e requer um construtor separado para cada construtor suportado na superclasse, a classe wrapper pode ser usada para instrumentar qualquer `Set` implementação e funcionará em conjunto com qualquer construtor pré-existente:

```
<Instant> Tempos de Set = novo InstrumentedSet<>(novo TreeSet<>(cmp)); Set<E> s
= novo InstrumentedSet<>(novo HashSet<>(INIT_CAPACITY));
```

A classe `InstrumentedSet` pode até ser usada para instrumentar temporariamente uma instância de `set` que já foi usada sem instrumentação:

```
Caminhada do vazio estático (Set<Dog> dogs) {
    InstrumentedSet<Dog> iDogs = novo InstrumentedSet<>(dogs);... //
    Dentro deste método, use iDogs em vez de dog}
```

A classe `InstrumentedSet` é conhecida como classe wrapper porque cada instância `InstrumentedSet` contém ("envolve") outra instância de `Set`. Isso também é conhecido como padrão Decorator [Gamma95] porque a classe `InstrumentedSet` "decora" um conjunto adicionando instrumentação. Às vezes, a combinação de composição e encaminhamento é chamada de delegação. Tecnicamente, não é delegação a menos que o objeto wrapper passe para o objeto wrapped [Lieberman86; Gamma95].

As desvantagens das classes de wrapper são poucas. Uma ressalva é que as classes de envolvimento não são adequadas para uso em frameworks de callback, onde objetos passam autorreferências para outros objetos para invocações subsequentes ("callbacks"). Como o objeto wrapped não conhece seu wrapper, ele passa uma referência para si mesmo (`this`) e os callbacks escapam do wrapper. Isso é conhecido como o problema SELF [Lieberman86]. Algumas pessoas se preocupam com o impacto no desempenho das invocações do método de encaminhamento ou com o impacto na pegada de memória dos objetos wrapper. Nenhum dos dois acaba tendo muito impacto na prática. É tedioso escrever métodos de encaminhamento, mas você precisa escrever a classe reutilizável de encaminhamento para cada interface apenas uma vez, e classes de encaminhamento podem ser fornecidas para você. Por exemplo, Guava fornece classes de encaminhamento para todas as interfaces de coleção [Guava].

A herança é apropriada apenas em circunstâncias em que a subclasse é realmente um subtipo da superclasse. Em outras palavras, uma classe B deve estender uma classe A somente se existir uma relação "é-a" entre as duas classes. Se você estiver tentado a fazer uma classe B estender uma classe A, pergunte a si mesmo: Todo B é realmente um A? Se você não consegue responder sim com sinceridade a essa pergunta, B não deve estender A. Se a resposta for não, geralmente ocorre que B deve conter uma instância privada de A e expor uma API diferente: A não é uma parte essencial de B, apenas um detalhe de sua implementação.

Existem várias violações óbvias desse princípio nas bibliotecas de plataforma Java. Por exemplo, uma pilha não é um vetor, então a pilha não deve estender o Vetor. Da mesma forma, uma lista de propriedades não é uma tabela hash, então Propriedades não deve se estender Hashtable. Em ambos os casos, a composição teria sido preferível.

Se você usa herança onde a composição é apropriada, expõe desnecessariamente detalhes da implementação. A API resultante te vincula à implementação original, limitando para sempre o desempenho da sua classe. Mais sério, ao expor os internos, você permite que os clientes acessem diretamente eles. No mínimo, isso pode levar a uma semântica confusa. Por exemplo, se `p` se refere a uma instância de `Properties`, então `p.getProperty(key)` pode gerar resultados diferentes de `p.get(key)`: o método former leva em conta os defaults, enquanto o último método, que é herado do `Hashtable`, não leva. Mais seriamente, o cliente pode corromper invariantes da subclasse modificando diretamente a superclasse. No caso de `Properties`, os projetistas pretendiam que apenas strings fossem permitidas como chaves e valores, mas o acesso direto à `Hashtable` subjacente permitia que esse invariante fosse violado. Uma vez violado, não é mais possível usar outras partes da `PropertiesAPI` (carregar e armazenar). Quando esse problema foi descoberto, já era tarde demais para corrigi-lo porque os clientes dependiam do uso de chaves e valores que não fossem de cadeia.

Há um último conjunto de perguntas que você deve se fazer antes de decidir usar a herança em vez da redação. A classe que você está pensando em expandir tem alguma falha na API? Se sim, você se sente confortável em propagar esses direitos para a API da sua turma? A herança propaga quaisquer falhas na API da superclasse, enquanto a composição permite que você projete uma nova API que esconda essas falhas.

Resumindo, a herança é poderosa, mas é problemática porque viola o encapsulamento. É apropriado apenas quando existe uma relação genuína de subtipo entre a subclasse e a superclasse. Mesmo assim, a herança pode levar à fragilidade se a subclasse estiver em um pacote diferente da superclasse e a superclasse não for projetada para herança. Para evitar essa fragilidade, use *composition and forwarding* em vez de *inheritance*, especialmente se existir uma interface apropriada para implementar uma classe wrapper. Além de serem classes de wrapper mais robustas que subclasses, elas também são mais poderosas.

Item 19: Projetar e documentar para herança ou, caso contrário, proibi-la

O item 18 alertou sobre os perigos de subclassificar uma classe "estrangeira" que não foi projetada e documentada para herança. Então, o que significa para uma classe ser projetada e documentada para herança?

Primeiro, a classe deve documentar precisamente os efeitos de sobrepor qualquer método. Em outras palavras, a classe deve documentar seu autouso de métodos sobrescritivos. Para cada método público ou protegido, a documentação deve indicar quais métodos sobrescritos o método invoca, em qual sequência e como os resultados de cada invocação afetam o processamento subsequente. (Por anulável, queremos dizer não final e seja público ou protegido.) De forma mais geral, uma classe deve documentar quaisquer circunstâncias em que possa invocar um método anulável. Por exemplo, invocações podem vir de threads em segundo plano ou inicializadores estáticos.

Um método que invoca métodos sobrescritos contém uma descrição dessas invocações ao final de seu comentário de documentação. A descrição está em uma seção especial da especificação, rotulada como "Requisitos de Implementação", que é gerada pela tag Javadoc `@implSpec`. Esta seção descreve o funcionamento interno do método. Aqui está um exemplo, copiado da especificação `forjava.util.AbstractCollection`:

```
remove booleano público (Objeto o)
```

Remove uma única instância do elemento especificado dessa coleção, se ele estiver presente (operação opcional). Mais formalmente, remove um elemento e tal que `Objects.equals(o, e)`, se essa coleção contiver um ou mais desses elementos. Retorna verdadeiro se essa coleção conteve o elemento especificado (ou equivalentemente, se essa coleção mudou como resultado da chamada). Requisitos de Implementação: Esta implementação itera sobre a coleção procurando o elemento especificado. Se encontrar o elemento, ele remove o elemento da coleção usando o método `remove` do iterador. Note que esta implementação gera uma `ExceçãoOperacionalUnsupportedOperation` se o iterador retornado pelo método `iterator` desta coleção não implementar o método `remove` e esta coleção contiver o objeto especificado.

Essa documentação não deixa dúvidas de que sobrescrever o método `iterator` afetará o comportamento do método `remove`. Também descreve exatamente como o comportamento do iterador retornado pelo método do iterador afetará o comportamento do método `remove`. Compare isso com a situação no Item 18, onde o programador que subclasse `HashSet` simplesmente não podia dizer se sobrescrever o método `add` afetaria o comportamento do método `addAll`.

Mas isso não viola o ditado de que uma boa documentação de API deve descrever o que um determinado método faz e não como ele faz? Sim, tem! Isso é uma consequência infeliz do fato de que a herança viola a encapsulação. Para documentar uma classe para que ela possa ser subclassificada com segurança, você deve descrever detalhes de implementação que, de outra forma, deveriam permanecer não especificados.

A tag `@implSpec` foi adicionada no Java 8 e muito usada no Java 9. Essa tag deveria estar ativada por padrão, mas desde o Java 9, a utilitária Javadoc ainda a ignora, a menos que você passe a tag de linha de comando `"apiNote:a:API Note:"`.

Projetar para herança envolve mais do que apenas documentar padrões de autouso. Para permitir que programadores escrevam subclasses eficientes sem dificuldades excessivas, a classe pode precisar fornecer ganchos para seu funcionamento interno na forma de métodos protegidos escolhidos de forma justamente ou, em casos raros, campos protegidos. Por exemplo, considere o método `removeRange` do `java.util.AbstractList`:

```
protected void removeRange(int fromIndex, int toIndex)
```

Remove desta lista todos os elementos cujo índice é entre `fromIndex`, inclusivo, e `toIndex`, exclusivo. Desloca quaisquer elementos sucessivos para a esquerda (reduz seu índice). Essa chamada encurta a lista por `toIndex - fromIndex` elementos. (Se `toIndex == fromIndex`, essa operação não tem efeito.) Esse método é chamado pela operação `clear` nesta lista e suas sublistas. Sobrepor esse método para aproveitar os componentes internos da implementação da lista pode melhorar substancialmente o desempenho da operação `clear` nesta lista e em suas sublistas. Requisitos de Implementação: Esta implementação recebe uma lista `Iterator` posicionada antes do `fromIndex` e chama repetidamente `ListIterator.next` seguido de `ListIterator.remove`, até que todo o intervalo seja removido. Nota: Se o `ListIterator.remove` requer tempo linear, esta implementação requer tempo quadrático. Parâmetros:

<code>fromIndex</code>	Índice do primeiro elemento a ser removido.
<code>toIndex</code>	índice após o último elemento a ser removido.

Esse método não interessa aos usuários finais de uma implementação de Lista. Ele é fornecido exclusivamente para facilitar que subclasses forneçam um método rápido e claro nas sublistas. Na ausência do método `removeRange`, as subclasses teriam que se contentar com desempenho quadrático quando o método `clear` fosse invocado em sublistas ou reescrever todo o mecanismo de `subList` do zero — não é uma tarefa fácil!

Então, como você decide quais membros protegidos expor quando projeta uma classe para herança? Infelizmente, não existe solução mágica. O melhor que você pode fazer é pensar bem, faça seu melhor palpite e teste escrevendo subclasses. Você deve expor o menor número possível de membros protegidos, pois cada um representa um compromisso com um detalhe de implementação. Por outro lado, você não deve expor poucos, pois um membro protegido ausente pode tornar uma classe praticamente inutilizável para herança.

A única maneira de testar uma classe projetada para herança é escrevendo subclasses. Se você omitir um membro protegido crucial, tentar escrever uma subclasse tornará a teomissão dolorosamente óbvia. Por outro lado, se várias subclasses forem escritas e não for um membro protegido, você provavelmente deveria torná-lo privado. A experiência mostra que três subclasses geralmente são suficientes para testar uma classe extensível. Uma ou mais dessas subclasses devem ser escritas por alguém que não seja o autor da superclasse.

Quando você projeta para herança uma classe que provavelmente terá amplo uso, perceba que está se comprometendo para sempre com os padrões de autouso que documenta e com as decisões de implementação implícitas em seus métodos e campos protegidos. Esses compromissos podem dificultar ou impossibilitar a melhoria da performance ou funcionalidade da classe em uma versão subsequente. Portanto, você deve testar sua classe escrevendo subclasses antes de lançá-la.

Além disso, note que a documentação especial necessária para herança atrapalha a documentação normal, que é projetada para programadores que criam instâncias da sua classe e invocam métodos nelas. Até o momento desta redação, há poucas ferramentas para separar a documentação comum da API de informações de interesse apenas para programadores que implementam subclasses.

Existem algumas restrições a mais que uma classe deve obedecer para permitir a herança. Construtores não devem invocar métodos sobrescrevíveis, direta ou indiretamente. Se você violar essa regra, ocorrerá falha do programa. O construtor de superclasse é executado antes do construtor da subclasse, então o método de sobreposição na subclasse será invocado antes que o construtor da subclasse seja executado. Se o método sobrescrito depender de qualquer inicialização realizada pelo construtor da subclasse, o método não se comportará como esperado. Para tornar isso concreto, aqui está uma aula que viola essa regra:

```
classe pública Super {  
    Broken - construtor invoca um methodpublic super() {  
  
    overrideMe();}empty public  
    overrideMe() {}}
```

Aqui está uma subclasse que sobrescreve o método `overrideMe`, que é erroneamente invocado pelo único construtor do `Super`:

```

classe final pública Sub estende Super {
    Final em branco, definido por constructorprivate final instantâneo instantâneo;

    Sub() {
        instantâneo = instante.agora();}

    Método de sobreposição invocado pela superclasse constructor@Override empty
    público overrideMe() {
        System.out.println(instant);}

    public static void main(String[] args) {
        Sub = novo Sub();
        sub.overrideMe();}}

```

Você poderia esperar que esse programa imprimisse o instante duas vezes, mas ele imprime `null` na primeira vez porque o `overrideMe` é invocado pelo `Super` construtor antes que o construtor `Sub` tenha a chance de inicializar o campo `instantâneo`. Note que este programa observa um campo final em dois estados diferentes! Note também que, se o `overrideMe` tivesse invocado qualquer método no instante, ele teria lançado um `NullPointerException` quando o construtor `Super` invocava o `overrideMe`. A única razão pela qual esse programa não lança uma `NullPointerException` como está é que o método `println` tolera parâmetros nulos.

Note que é seguro invocar métodos privados, métodos finais e `meth-ods` estáticos, nenhum dos quais pode ser revertido, de um construtor.

As interfaces `Clonável` e `Serializável` apresentam dificuldades especiais ao projetar para herança. Geralmente, não é uma boa ideia para uma classe projetada para herança implementar qualquer uma dessas interfaces, pois elas impõem um ônus substancial aos programadores que estendem a classe. Existem, no entanto, ações especiais que você pode adotar para permitir que subclasses implementem essas interfaces sem exigir que o façam. Essas ações são descritas no Item 13 e no Item 86.

Se você decidir implementar `Cloneable` ou `Serializable` em uma classe projetada para herança, deve estar ciente de que, como os métodos `clone` e `readObject` se comportam muito como construtores, uma restrição semelhante se aplica: nem `clone` nem `readObject` podem invocar um método `overridable`, direta ou indiretamente. No caso do `readObject`, o método de substituição será executado antes do

o estado da subclasse foi desserializado. No caso do clone, o método de substituição será executado antes que o método clone da subclasse tenha a chance de corrigir o estado do clone. Em ambos os casos, é provável que uma falha do programa ocorra. No caso do clone, a falha pode danificar tanto o objeto original quanto o clone. Isso pode acontecer, por exemplo, se o método de sobreposição assumir que está modificando a cópia do clone da estrutura profunda do objeto, mas a cópia ainda não foi feita.

Por fim, se você decidir implementar o `Serializable` em uma classe projetada para herança e a classe tiver um método `readResolve` ou `writeReplace`, você deve proteger o método `readResolve` ou `writeReplace` em vez de privado. Se esses métodos forem privados, serão silenciosamente ignorados pelas subclasses. Este é mais um caso em que um detalhe de implementação passa a fazer parte da API de uma classe para permitir a herança.

A essa altura, já deveria estar claro que projetar uma classe para herança exige grande esforço e impõe limitações substanciais à classe. Esta não é uma decisão a ser tomada levemente. Existem algumas situações em que isso é claramente o certo a se fazer, como classes abstratas, incluindo implementações esqueléticas de interfaces (Item 20). Existem outras situações em que claramente é a coisa errada a se fazer, como classes imutáveis (Item 17).

Mas e quanto às aulas comuns de concreto? Tradicionalmente, eles não são nem finais, nem projetados e documentados para subclasse, mas essa situação é perigosa. Cada vez que uma alteração é feita em uma classe assim, há a chance de que subclasses estendendo a classe quebre. Isso não é apenas um problema teórico. Não é incomum receber relatórios de bugs relacionados à subclasse após modificar o interior de uma classe concreta não final que não foi projetada e documentada para herança.

A melhor solução para esse problema é proibir a subclasse em classes que não sejam projetadas e documentadas para serem subclassificadas com segurança. Existem duas maneiras de proibir a subclasse. O mais fácil dos dois é declarar a turma como final. A alternativa é tornar todos os construtores privados ou pacotes privados e adicionar fábricas públicas estáticas no lugar dos construtores. Essa alternativa, que oferece a flexibilidade para usar subclasses internamente, é discutida no Item 17. Qualquer uma das abordagens é aceitável.

Esse conselho pode ser um tanto controverso porque muitos programadores já se acostumaram a subclassificar classes comuns de concreto para adicionar facilidades como instrumentação, notificação e sincronização, ou para limitar funcionalidades. Se a classe implementar alguma interface que capture sua essência, como `Set`, `List` ou `Map`, então você não deve sentir escrúpulos em proibir subclasses. O padrão de classe `Wrapper`, descrito no Item 18, oferece uma alternativa superior à herança para aumentar a funcionalidade.

Se uma classe concreta não implementa uma interface padrão, você pode incomodar alguns programadores ao proibir a herança. Se você sentir que deve permitir a herança de tal classe, uma abordagem razoável é garantir que a classe nunca invoque nenhum de seus métodos anuláveis e documentar esse fato. Em outras palavras, eliminar completamente o uso autônomo da classe de métodos sobrescritos.

Fazendo isso, você criará uma classe que seja razoavelmente segura para subclasse. Sobrepor um método nunca afetará o comportamento de nenhum outro método.

Você pode eliminar mecanicamente o uso próprio de métodos sobrescritos por parte de uma classe, sem mudar seu comportamento. Mova o corpo de cada método overridable para um "método helper" privado e faça com que cada método overridable invoque seu método privatehelper. Depois, substitua cada autouso de um método sobrescrito por uma invocação direta do método assistente privado do método sobrescrito.

Em resumo, criar uma classe para herança é um trabalho árduo. Você deve documentar todos os padrões de autouso deles e, uma vez documentado, deve se comprometer com eles durante toda a turma. Se você não fizer isso, as subclasses podem se tornar dependentes dos detalhes de implementação da superclasse e podem quebrar se a implementação da superclasse mudar. Para permitir que outros escrevam subclasses eficientes, você também pode precisar exportar um ou mais métodos protegidos. A menos que você saiba que há uma necessidade real de subclasses, provavelmente é melhor proibir a herança declarando sua classe como final ou garantindo que não haja construtores acessíveis.

Item 20: Prefira interfaces a classes abstratas

Java possui dois mecanismos para definir um tipo que permite múltiplas implementações: interfaces e classes abstratas. Desde a introdução dos métodos padrão para interfaces no Java 8 [JLS 9.4.3], ambos os mecanismos permitem fornecer implementações para alguns métodos de instância. Uma diferença importante é que, para implementar o tipo definido por uma classe abstrata, uma classe deve ser uma subclasse da classe abstrata. Como o Java permite apenas uma herança única, essa restrição sobre classes abstratas restringe severamente seu uso como definições de tipos. Qualquer classe que defina todos os métodos necessários e obedeça ao contrato geral pode implementar uma interface, independentemente de onde a classe resida na hierarquia de classes.

Classes existentes podem ser facilmente adaptadas para implementar uma nova interface. Tudo o que você precisa fazer é adicionar os métodos necessários, caso ainda não existam, e adicionar a cláusula `implements` à declaração de classe. Por exemplo, muitas classes existentes foram adaptadas para implementar as interfaces `Comparable`, `Iterable` e `Autocloseable` quando foram adicionadas à plataforma. Classes existentes não podem, em geral, ser adaptadas para expandir uma nova classe abstrata. Se você quer que duas classes estendam a mesma classe abstrata, você precisa colocá-la no topo da hierarquia de tipos, onde ela é ancestral de ambas as classes. Infelizmente, isso pode causar grandes danos colaterais à hierarquia de tipos, forçando todos os descendentes da nova classe abstrata a subclassá-la, seja apropriado ou não.

Interfaces são ideais para definir mixins. De forma geral, um mixin é um tipo que uma classe pode implementar além do seu "tipo primário", para declarar que ele promove algum comportamento opcional. Por exemplo, `Comparable` é uma interface mixin que permite a uma classe declarar que suas instâncias estão ordenadas em relação a outros objetos mutuamente comparáveis. Essa interface é chamada de mixin porque permite que a funcionalidade opcional seja "misturada" à funcionalidade principal do tipo. Classes abstratas não podem ser usadas para definir mixins pelo mesmo motivo que não podem ser adaptadas a classes existentes: uma classe não pode ter mais de um pai, e não há um lugar razoável na hierarquia de classes para inserir um mixin.

Interfaces permitem a construção de estruturas de tipos não hierárquicas. Hierarquias de tipos são ótimas para organizar algumas coisas, mas outras não caem de forma rígida em uma hierarquia. Por exemplo, suponha que tenhamos uma interface representando um cantor e outra representando um compositor:

```
interface pública Singer {
    AudioClip sing(Song s);}
```



```
interface pública Compositor {  
    Compor música(int chartPosition);}
```

Na vida real, alguns cantores também são compositores. Como usamos interfaces — em vez de classes abstratas — para definir esses tipos, é perfeitamente permissível que uma única classe implemente tanto Cantor quanto Compositor. Na verdade, podemos definir uma terceira interface que estende tanto o Cantor quanto o Compositor e adiciona novos métodos apropriados para a combinação:

```
interface pública SingerSongwriter estende Singer, Songwriter {  
    AudioClip strum(); void  
    actSensitive();}
```

Você nem sempre precisa desse nível de flexibilidade, mas quando precisa, as interfaces são uma salvação. A alternativa é uma hierarquia de classes inflada contendo uma classe separada para cada combinação suportada de atributos. Se houver n atributos no sistema de tipos, há 2^n combinações possíveis que você pode precisar suportar. Isso é o que se conhece como uma explosão combinatória. Hierarquias de classes inchadas podem levar a classes inchadas com muitos métodos que diferem apenas no tipo de seus argumentos, porque não existem tipos na hierarquia de classes capazes de capturar comportamentos comuns.

Interfaces permitem aprimoramentos de funcionalidade seguros e poderosos por meio do idioma wrap-per class (Item 18). Se você usa classes abstratas para definir tipos, deixa o programador que quer adicionar funcionalidades sem alternativa além da herança. As classes resultantes são menos poderosas e mais frágeis do que as classes wrapper.

Quando houver uma implementação óbvia de um método de interface em termos de outros métodos de interface, considere fornecer assistência de implementação a pro-grammers na forma de um método padrão. Para um exemplo dessa técnica, veja o método `removeIf` na página 104. Se você fornecer métodos padrão, certifique-se de documentá-los para herança usando a tag Javadoc `@implSpec` (Item 19).

Existem limites para a quantidade de assistência de implementação que você pode fornecer com métodos padrão. Embora muitas interfaces especifiquem o comportamento de `Object` methods, como `equals` e `hashCode`, você não tem permissão para fornecer métodos padrão para eles. Além disso, interfaces não são permitidas a conter campos de instância nem membros estáticos não públicos (com exceção de métodos estáticos privados). Por fim, você não pode adicionar métodos padrão a uma interface que não controla.

Você pode, no entanto, combinar as vantagens de interfaces e classes abstratas fornecendo uma classe de implementação esquelética abstrata para acompanhar uma interface. The interface define o tipo, talvez fornecendo alguns métodos padrão, enquanto a classe de implementação esquelética implementa a interface não primitiva restante

métodos sobre os métodos de interface primitiva. Estender uma implementação esquelética tira a maior parte do trabalho de implementar uma interface. Este é o padrão `TemplateMethod` [Gamma95].

Por convenção, classes de implementação esquelética são chamadas de `AbstractInterface`, onde `Interface` é o nome da interface que elas implementam. Por exemplo, o Framework das Coleções fornece uma implementação esquelética para acompanhar cada interface principal de coleções: `AbstractCollection`, `AbstractSet`, `AbstractList` e `AbstractMap`. Pode-se argumentar que faria sentido chamá-los de `SkeletalCollection`, `SkeletalSet`, `SkeletalList` e `SkeletalMap`, mas a convenção `Abstract` agora está firmemente estabelecida. Quando devidamente projetadas, implementações esqueléticas (seja uma classe abstrata separada ou consistindo apenas de métodos padrão em uma interface) podem facilitar muito para programadores fornecerem suas próprias implementações de uma interface. Por exemplo, aqui está um método de fábrica estática contendo uma implementação completa e totalmente funcional do `List` sobre o `AbstractList`:

```
Implementação concreta construída sobre implementação esquelética List<Integer>
intArrayAsList(int[] a) {
    Objects.requireNonNull(a);

    O operador diamond só é legal aqui em Java 9 e versões posteriores// Se você
    estiver usando uma versão anterior, especifique <Integer>return new AbstractList<
    () {
        @Override Integer público get(int i) {
            devolver a[i]; Autoboxing (Item 6)
        }

        @Override conjunto público inteiro(int i, Integer val) {
            int oldVal = a[i]; a[i] = val;// Auto-
            unboxingreturn oldVal;// Autoboxing}

        @Override public int size() {
            retorne a.comprimento;}}};}
```

Quando você considera tudo o que uma implementação de `Lista` faz por você, este exemplo é uma demonstração impressionante do poder das implementações esqueléticas. Inciden-tally, este exemplo é um Adaptador [Gamma95] que permite visualizar um array `int` como uma lista de instâncias inteiras. Devido a toda a translação entre valores `int` e instâncias inteiras (boxing e unboxing), sua performance não é muito boa. Note que a implementação assume a forma de classe anônima (Item 24).

A beleza das classes de implementação esquelética é que elas fornecem toda a assistência de implementação das classes abstratas sem impor as severas restrições que as classes abstratas impõem quando servem como definições de tipos. Para a maioria dos implementadores de uma interface com uma classe de implementação esquelética, estender essa classe é a escolha óbvia, mas é estritamente opcional. Se uma classe não puder ser feita para estender a implementação esquelética, a classe sempre pode implementar a interface diretamente. A classe ainda se beneficia de quaisquer métodos padrão presentes na própria interface. Além disso, a implementação esquelética ainda pode auxiliar a tarefa do implementador. A classe que implementa a interface pode encaminhar invocações de `interfaceMethods` para uma instância contida de uma classe interna privada que estende a implementação esquelética. Essa técnica, conhecida como herança múltipla simulada, está intimamente relacionada ao idioma da classe de envolvimento discutido no Item 18. Ela oferece muitos dos benefícios de múltiplas heranças, evitando armadilhas.

Escrever uma implementação esquelética é um processo relativamente simples, embora um pouco tedioso. Primeiro, estude a interface e decida quais métodos são os primitivos em termos dos quais os outros podem ser implementados. Essas primitivas serão os métodos abstratos na sua implementação esquelética. Em seguida, forneça métodos padrão na interface para todos os métodos que podem ser implementados diretamente sobre as primitivas, mas lembre-se de que você pode não fornecer métodos padrão para métodos `Object`, como `equals` e `hashCode`. Se as primitivas e métodos padrão cobrem a interface, você está acabado e não precisa de uma classe de implementação esquelética. Caso contrário, escreva uma classe declarada para implementar a interface, com implementações de todos os métodos de interface restantes. A classe pode conter quaisquer campos e métodos não públicos apropriados para a tarefa.

Como exemplo simples, considere a interface `Map.Entry`. Os primíveis óbvios são `getKey`, `getValue` e (opcionalmente) `setValue`. A interface especifica o comportamento de `equals` e `hashCode`, e há uma implementação óbvia de `toString` em termos das primitivas. Como você não pode fornecer implementações padrão para os métodos `Object`, todas as implementações são colocadas na classe de implementação esquelética:

```
Classe de implementação esquelética classe abstractpublic AbstractMapEntry<K,V>

    implementa Map.Entry<K,V> { // Entradas em um mapa modificável devem
    sobrescrever isso method@Override V_setValue(V valor) público {

        lançar um novo UnsupportedOperationException();}
```

```

Implementa o contrato geral de Map.Entry.equals@Override booleano público
é igual a (Object o) {
    if (o == isto)
        retorno verdadeiro; se (!( ou instância de Map.Entry))

    retorne falso; Map.Entry<?,?> e =
    (Map.Entry) o; return
    Objects.equals(e.getKey(),      getKey())
    && Objects.equals(e.getValue(), getValue());
}

Implementa o contrato geral de Map.Entry.hashCode@Override público int
hashCode() {
    return Objects.hashCode(getKey())
    ^ Objects.hashCode(getValue());
}

@Override público String toString() {
return getKey() + "=" + getValue();}

```

Note que essa implementação esquelética não pôde ser implementada na interface `Map.Entry` ou como uma subinterface porque métodos padrão não são permitidos para substituir métodos `Object` como `equals`, `hashCode` e `toString`.

Como implementações esqueléticas são projetadas para herança, você deve seguir todas as diretrizes de design e documentação do Item 19. Para ser breve, os comentários de documentação foram omitidos do exemplo anterior, mas boa documentação é absolutamente essencial em uma implementação esquelética, seja ela composta por métodos padrão em uma interface ou uma classe abstrata separada.

Uma variante menor da implementação esquelética é a implementação simples, exemplificada por `AbstractMap.SimpleEntry`. Uma implementação simples é como a implementação esquelética, pois implementa uma interface e é projetada para herança, mas difere por não ser abstrata: é a implementação funcional mais simples possível. Você pode usá-lo como está ou subclassá-lo conforme as circunstâncias exigirem.

Resumindo, uma interface geralmente é a melhor forma de definir um tipo que permite múltiplas implementações. Se você exportar uma interface não trivial, deve considerar fortemente fornecer uma implementação esquelética para acompanhá-la. Na medida do possível, você deve fornecer a implementação esquelética via métodos padrão na interface para que todos os implementadores da interface possam utilizá-la. Dito isso, restrições nas interfaces normalmente exigem que uma implementação esquelética assuma a forma de uma classe abstrata.

Item 21: Interfaces de design para a posteridade

Antes do Java 8, era impossível adicionar métodos às interfaces sem quebrar implementações existentes. Se você adicionasse um novo método a uma interface, implementações existentes, em geral, não teriam o método, resultando em um erro em tempo de compilação. No Java 8, a construção padrão de método foi adicionada [JLS 9.4], com a intenção de permitir a adição de métodos às interfaces existentes. Mas adicionar novos métodos às interfaces existentes é cheio de riscos.

A declaração para um método padrão inclui uma implementação padrão que é usada por todas as classes que implementam a interface, mas não implementam o método padrão. Embora a adição de métodos padrão ao Java torne possível adicionar métodos a uma interface existente, não há garantia de que esses métodos funcionarão em todas as implementações pré-existent. Métodos padrão são "injetados" em implementações existentes sem o conhecimento ou consentimento de seus implementadores. Antes do Java 8, essas implementações eram escritas com o entendimento tácito de que suas interfaces nunca adquiririam novos métodos.

Muitos novos métodos padrão foram adicionados às interfaces principais de coleta no Java 8, principalmente para facilitar o uso de lambdas (Capítulo 6). Os métodos padrão das bibliotecas Java são implementações de uso geral de alta qualidade e, na maioria dos casos, funcionam bem. Mas nem sempre é possível escrever um método padrão que mantenha todos os invariantes de toda implementação concebível.

Por exemplo, considere o método `removeIf` (`removeIf`), que foi adicionado à interface `Collection` no Java 8. Esse método remove todos os elementos para os quais uma função booleana dada (ou predicado) retorna verdadeiro. A implementação padrão é especificada para percorrer a coleção usando seu iterador, invocando o predicado em cada elemento e usando o método `remove` do iterador para remover os elementos para os quais o predicado retorna verdadeiro. Presumivelmente, a declaração é algo assim:

```
Método padrão adicionado à interface de Coleção no Java 8 padrão booleano
removeIf(Predicate<? super E> filter) {
    Objects.requireNonNull(filter); resultado booleano = falso;
    para (<E>Iterador it = iterador(); it.hasNext(); ) {
        if (filter.test(it.next())) {
            it.remove(); resultado =
            verdadeiro;}}retorne
            resultado;}
```

Esta é a melhor implementação de uso geral que alguém poderia escrever para o método `removeIf`, mas infelizmente, ela falha em algumas implementações reais de `Collection`. Por exemplo, considere `org.apache.commons.collections4.collection.SynchronizedCollection`. Esta classe, da Apache Commonslibrary, é semelhante à que retorna pela fábrica estática `Collections.synchronizedCollection` em `java.util`. A versão Apache também oferece a capacidade de usar um objeto fornecido pelo cliente para travamento, no lugar da coleção. Em outras palavras, é uma classe wrapper (Item 18), cujos métodos todos se sincronizam em um objeto de travamento antes de delegar para a coleção wrapped.

A classe `Apache SynchronizedCollection` ainda está sendo mantida ativamente, mas, até o momento desta redação, ela não substitui o método `removeIf`. Se essa classe for usada em conjunto com o Java 8, ela herdarà a implementação padrão `removeIf`, que não pode, de fato não pode, manter a promessa fundamentalmente da classe: sincronizar automaticamente em torno de cada invocação de método. A implementação padrão não sabe nada sobre sincronização e não tem acesso ao campo que contém o objeto de travamento. Se um cliente chama o método `removeIf` em uma instância de `SynchronizedCollection` na presença de modificação concorrente da coleção por outro thread, pode ocorrer uma Atualidade de Modificação Concorrente ou outro comportamento não especificado.

Para evitar que isso acontecesse em implementações semelhantes de bibliotecas de plataforma Java, como a classe pacote-privada retornada por `Collections.synchronizedCollection`, os mantenedores do JDK precisavam sobrescrever a implementação padrão `removeIf` e outros métodos semelhantes para realizar a sincronização necessária antes de invocar a implementação padrão. Implementações de `Collection` pré-existent que não faziam parte da plataforma Java não tiveram a oportunidade de fazer mudanças análogas em sincronia com a mudança da interface, e algumas ainda não o fizeram.

Na presença de métodos padrão, implementações existentes de um interface podem compilar sem erro ou aviso, mas falhar em tempo de execução. Embora não seja muito comum, esse problema também não é um caso isolado. Alguns dos métodos adicionados às interfaces de coleções no Java 8 são conhecidos por serem suscetíveis, e algumas implementações existentes são conhecidas por serem afetadas.

Usar métodos padrão para adicionar novos métodos a interfaces existentes deve ser evitado, a menos que a necessidade seja crítica, caso em que você deve pensar seriamente se uma implementação de interface existente pode ser quebrada pela implementação do seu método padrão. Métodos padrão são, no entanto, extremamente úteis para fornecer implementações padrão de métodos quando uma interface é criada, para facilitar a tarefa de implementar a interface (Item 20).

Também vale notar que os métodos padrão não foram projetados para suportar a remoção de métodos das interfaces ou a alteração das assinaturas de métodos existentes. Nenhuma dessas mudanças na interface é possível sem quebrar os clientes existentes.

A moral é clara. Embora os métodos padrão agora façam parte da plataforma Java, ainda é de extrema importância projetar interfaces com muito cuidado. Embora métodos padrão permitam adicionar métodos a interfaces existentes, há grande risco em fazer isso. Se uma interface contém uma falha menor, pode irritar seus usuários para sempre; se uma interface for severamente deficiente, ela pode condenar a API que a contém.

Portanto, é fundamental testar cada nova interface antes de lançá-la. Vários programadores devem implementar cada interface de maneiras diferentes. No mínimo, você deve buscar três implementações diferentes. Igualmente importante é escrever múltiplos programas clientes que usem instâncias de cada nova interface para realizar diversas tarefas. Isso ajudará muito a garantir que cada interface atenda a todos os seus usos pretendidos. Esses passos permitirão que você descubra falhas nas interfaces antes de serem lançadas, quando você ainda pode corrigi-las facilmente. Embora possa ser possível corrigir algumas falhas de interface após o lançamento de uma interface, não se pode contar com isso.

Item 22: Use interfaces apenas para definir tipos

Quando uma classe implementa uma interface, a interface serve como um tipo que pode ser usado para se referir a instâncias da classe. O fato de uma classe implementar uma interface deveria, portanto, dizer algo sobre o que um cliente pode fazer com instâncias da classe. É inadequado definir uma interface para qualquer outro propósito.

Um tipo de interface que falha nesse teste é a chamada interface constante. Tal interface não contém métodos; Consiste unicamente em campos finais estáticos, cada um exportando uma constante. Classes que usam essas constantes implementam a interface para evitar a necessidade de qualificar nomes de constantes com um nome de classe. Aqui está um exemplo:

```
Antipadrão de interface constante - não use! interface pública PhysicalConstants
{
    Número (1/mol)dupla final de Avogadro
    AVOGADROS_NUMBER
                                = 6.022_140_857e23;

    Constante de Boltzmann (J/K)dupla final estática BOLTZMANN_CONSTANT =
    1.380_648_52e-23;

    Massa do elétron (kg)estático duplo final
    ELECTRON_MASS
                                = 9.109_383_56e-31;
}
```

O padrão constante de interface é um uso ruim das interfaces. O fato de uma classe usar algumas constantes internamente é um detalhe de implementação. Implementar uma constantinterface faz com que esse detalhe de implementação vaze para a API exportada da classe. Não é importante para os usuários de uma classe que a classe implemente uma interface constante. Na verdade, pode até confundi-los. Pior ainda, representa um compromisso: se em uma versão futura a classe for modificada para que não precise mais usar as constantes, ainda deve implementar a interface para garantir compatibilidade binária. Se a classe anonfinal implementar uma interface constante, todas as suas subclasses terão seus namespaces poluídos pelas constantes na interface.

Existem várias interfaces constantes nas bibliotecas da plataforma Java, como o modo `asjava.io.ObjectStreamConstants`. Essas interfaces devem ser consideradas anomalias e não devem ser emuladas.

Se você quiser exportar constantes, há várias opções razoáveis. Se as constantes estiverem fortemente ligadas a uma classe ou interface existente, você deve adicioná-las à classe ou interface. Por exemplo, todas as classes primitivas numéricas em caixa, como `Integer` e `Double`, exportam `MIN_VALUE` e `MAX_VALUE` constantes. Se as constantes forem melhor vistas como membros de um tipo enumerado, você deve exportar

eles com um tipo enum (Item 34). Caso contrário, você deve exportar as constantes com uma classe utilitária não instanciada (Item 4). Aqui está uma versão da classe utilitária do exemplo `thePhysicalConstants` mostrado anteriormente:

Pacote de classes utilitário constante `com.effectivejava.science`;

```
classe pública PhysicalConstants {
    private PhysicalConstants() { }    Previne a instanciação

    AVOGADROS_NUMBER final estática pública = 6.022_140_857e23; público
    estático final duplo BOLTZMANN_CONST= 1.380_648_52e-23; Duplo final
    estático público ELECTRON_MASS= 9.109_383_56e-31;}
```

Aliás, observe o uso do caractere sublinhado (`_`) nos literais numéricos. Os sublinhados, que são legais desde o Java 7, não têm efeito sobre os valores dos literais numéricos, mas podem torná-los muito mais fáceis de ler se usados com discrição. Considere adicionar sublinhados a literais numéricos, sejam fixos de ponto flutuante, se contêm cinco ou mais dígitos consecutivos. Para literais base dez, sejam integrais ou ponto flutuante, você deve usar sublinhados para separar os literais em grupos de três dígitos indicando potências positivas e negativas de mil.

Normalmente, uma classe utilitária exige que os clientes qualifiquem nomes constantes com um nome de classe, por exemplo, `PhysicalConstants.AVOGADROS_NUMBER`. Se você fizer uso intensivo das constantes exportadas por uma classe utilitária, pode evitar a necessidade de qualificar as constantes com o nome da classe utilizando a facilidade de importação estática:

Uso de importação estática para evitar constantes qualificadoras `import static com.effectivejava.science.PhysicalConstants.*`;

```
classe pública Test {
    átomos duplos (mols duplos) {
        retorne AVOGADROS_NUMBER * mols;}...// Muitos outros usos de
    PhysicalConstants justificam a importação estática}
```

Em resumo, interfaces devem ser usadas apenas para definir tipos. Eles não devem ser usados apenas para exportar constantes.

Item 23: Preferência hierarquias de classes a classes marcadas

Ocasionalmente, você pode encontrar uma classe cujas instâncias vêm em duas ou mais versões e contêm um campo de tag indicando a versão da instância. Por exemplo, considere esta classe, que é capaz de representar um círculo ou um retângulo:

Classe marcada - muito inferior a uma hierarquia de classes! classe Figura {

```
enum Shape { RETÂNGULO, CÍRCULO };
```

```
Campo de etiqueta - a forma desta figura final Forma forma;
```

```
Esses campos são usados apenas se a forma for  
RETANGULARdobro de comprimento; largura duplo;
```

```
Esse campo é usado apenas se a forma for raio CIRCLEDobro;
```

```
Construtor para circleFigure(raio duplo) {
```

```
forma = Forma.CÍRCULO;  
this.radius = radius;}
```

```
Construtor para FiguraRetangular(comprimento duplo, largura dupla) {
```

```
forma = Forma.RETÂNGULO;  
this.length = comprimento;  
this.width = width;}
```

```
área dupla() {switch(shape) {
```

```
caso RETÂNGULO:  
retorno comprimento * largura; CÍRCULO do caso:
```

```
return Math.PI * (raio * raio); padrão:
```

```
throw new AssertionError(shape);}}}
```

Essas classes marcadas apresentam inúmeras limitações. Eles estão recheados de formulários padrão, incluindo declarações `enum`, campos de etiquetas e instruções `switch`. A capacidade de leitura é ainda mais prejudicada porque múltiplas implementações são misturadas em uma única classe. A pegada de memória é aumentada porque as instâncias ficam sobrecarregadas com campos irrelevantes pertencentes a outras versões. Campos não podem ser tornados finais a menos que construtores iniciem campos irrelevantes, resultando em mais padrão padrão. Construtores devem definir o campo de tag e inicializar os campos de dados corretos sem ajuda do `com-compiler`: se você inicializar os campos errados, o programa falhará em tempo de execução. Você não pode adicionar um sabor a uma classe marcada a menos que possa modificar seu arquivo fonte. Se você adicionar um a flavor, deve lembrar de adicionar um `case` a cada instrução `switch`, ou a classe falhará em tempo de execução. Por fim, o tipo de dado de uma instância não dá nenhuma pista sobre seu formato. Em resumo, as classes marcadas são prolixas, propensas a erros e ineficientes.

Felizmente, linguagens orientadas a objetos como o Java oferecem uma alternativa muito melhor para definir um único tipo de dado capaz de representar objetos de múltiplos tipos. Uma classe marcada é apenas uma imitação pálida de uma hierarquia de classes.

Para transformar uma classe marcada em uma hierarquia de classes, primeiro defina uma classe abstrata contendo um método abstrato para cada método na classe marcada cujo comportamento depende do valor da etiqueta. Na classe `Figura`, existe apenas um método desse tipo, que é `área`. Essa classe abstrata é a raiz da hierarquia de classes. Se houver métodos cujo comportamento não dependa do valor da tag, coloque-os em `thisclass`. Da mesma forma, se houver campos de dados usados por todas as variações, coloque-os em `thisclass`. Não existem métodos ou campos independentes de sabor na classe `Figura`.

Em seguida, defina uma subclasse concreta da classe raiz para cada variante da classe originalmente marcada. No nosso exemplo, há dois: `círculo` e `retângulo`. Inclua em cada subclasse os campos de dados específicos para seu estilo. No nosso exemplo, `raio` é particular ao `círculo`, e `comprimento` e `largura` são particulares ao `retângulo`. Também inclua em cada subclasse a implementação apropriada de cada método abstrato na classe raiz. Aqui está a hierarquia de classes correspondente à classe original de `Figura`:

```
Substituição da hierarquia de classes para uma classe abstrato de classe marcada
Figura {
    Área dupla abstrata();}
```

```
classe Círculo estende Figura {
    raio duplo final;

    Circle(raio duplo) { this.radius = radius; }

    @Override área dupla() { return Math.PI * (raio * raio); }}
```

```

classe Retângulo se estende Figura {
    comprimento duplo final; largura dupla final;

    Retângulo (comprimento duplo, largura dupla) {
        this.length = comprimento; this.width= width;}@Override
    área dupla() { retorno comprimento * largura; }}

```

Essa hierarquia de classes corrige todas as falhas das classes marcadas mencionadas anteriormente. O código é simples e claro, não contendo nenhum dos padrões encontrados no original. A implementação de cada versão recebe sua própria classe, e nenhuma dessas classes é limitada por campos de dados irrelevantes. Todos os campos são finais. O compilador garante que o construtor de cada classe inicialize seus campos de dados e que cada classe tenha uma implementação para cada método abstrato declarado na classe raiz. Isso elimina a possibilidade de falha em tempo de execução devido à falta de uma caixa de comutador. Múltiplos programadores podem estender a hierarquia de forma independente e interoperável, sem acesso à fonte para a classe raiz. Há um tipo de dado separado associado a cada variante, permitindo que programadores indiquem o sabor de uma variável e restrinjam variáveis e parâmetros de entrada a um determinado sabor.

Outra vantagem das hierarquias de classes é que elas podem refletir relações hierárquicas naturais entre tipos, permitindo maior flexibilidade e melhor verificação de tipos em tempo de compilação. Suponha que a classe marcada no exemplo original também permitisse quadrados. A hierarquia de classes poderia refletir o fato de que um quadrado é um tipo especial de retângulo (assumindo que ambos são imutáveis):

```

classe Quadrado estende Retângulo {
    Quadrado (lado duplo) {
        super(lado, lado);}}

```

Note que os campos na hierarquia acima são acessados diretamente, e não pelos métodos do acessor. Isso foi feito por brevidade e seria um projeto ruim se a hierarquia fosse pública (Item 16).

Em resumo, classes marcadas raramente são apropriadas. Se você estiver tentando escrever uma classe com um campo de tag explícito, pense se a tag poderia ser eliminada e a classe substituída por uma hierarquia. Quando você encontrar uma classe existente com campo atag, considere refatorá-la em uma hierarquia.

Item 24: Favoreça classes membros estáticas em vez de não estáticas

Uma classe aninhada é uma classe definida dentro de outra classe. Uma classe aninhada deve existir apenas para servir à sua classe envolvente. Se uma classe aninhada seria útil em algum outro contexto, então deveria ser uma classe de nível superior. Existem quatro tipos de classes aninhadas: classes membros estáticas, classes membros não estáticas, classes anônimas e classes locais. Todos, exceto o primeiro tipo, são conhecidos como classes internas. Esse item te diz quando usar qual tipo de classe aninhada e por quê.

Uma classe membro estática é o tipo mais simples de classe aninhada. É melhor pensá-lo como uma classe comum que está dentro de outra classe e tem acesso a todos os membros da classe adjacente, até mesmo aqueles declarados privados. A classe membro `Static` é um membro estático de sua classe envolvente e obedece às mesmas regras de acessibilidade que outros membros estáticos. Se for declarado privado, é acessível apenas dentro da classe adjacente, e assim por diante.

Um uso comum de uma classe membro estática é como classe auxiliar pública, útil apenas em conjunto com sua classe externa. Por exemplo, considere um `enum` descrevendo as operações suportadas por uma calculadora (Item 34). O `enum` da Operação deve ser uma classe pública estática da classe `Calculadora`. Clientes do `Calculator` podiam então se referir a operações usando nomes como `Calculator.Operation.PLUS` e `Calculator.Operation.MINUS`.

Sintaticamente, a única diferença entre classes membros estáticas e não estáticas é que classes estáticas têm o modificador `static` em suas declarações. Apesar da semelhança sintática, esses dois tipos de classes aninhadas são muito diferentes. Cada instância de uma classe membro não estática é implicitamente associada a uma instância que encerra sua classe contidora. Dentro dos métodos de instância de uma classe `nonstaticmember`, você pode invocar métodos na instância que encerra ou obter uma referência à instância envolvente usando a construção qualificada `esta` [JLS, 15.8.4]. Se uma instância de uma classe aninhada pode existir isoladamente de uma instância de sua classe `enclosing`, então a classe aninhada deve ser uma classe membro estática: é impossível criar uma instância de uma classe membro não estática sem uma instância envolvente.

A associação entre uma instância de classe membro não estática e sua instância envolvente é estabelecida quando a instância da classe membro é criada e não pode ser modificada posteriormente. Normalmente, a associação é estabelecida automaticamente invocando um construtor de classe membro não estático dentro de um método de instância da classe que envolve. É possível, embora raro, estabelecer a associação manualmente usando a expressão `enclosingInstance.new MemberClass(args)`. Como era de se esperar, a associação ocupa espaço na classe de classe membro não estática e adiciona tempo à sua construção.

Um uso comum de uma classe membro não estática é definir um Adapter[Gamma95] que permite que uma instância da classe externa seja vista como uma instância de alguma classe não relacionada. Por exemplo, implementações da interface Map tipicamente usam classes membros não estáticas para implementar suas visualizações de coleção, que são retornadas pelos métodos keySet, entrySet e values do Map. De forma semelhante, implementações das interfaces de coleção, como Set e List, normalmente usam classes não-staticmember para implementar seus iteradores:

```

Uso típico de uma classe social membro não estática<E> MySet estende
AbstractSet<E> {
    ... // A maior parte da turma foi omitida

    @Override iterator público<E> () {
        return new MyIterator();
    }

    classe privada MyIterator implementa Iterator<E> {
        ...}
}

```

Se você declarar uma classe membro que não exige acesso a uma instância de encerramento, sempre coloque o modificador estático em sua declaração, tornando-a uma classe estática em vez de uma classe membro não estática. Se você omitir esse modificador, cada instância terá uma referência oculta e supérflua à sua instância que encerra. Como mencionado anteriormente, armazenar essa referência leva tempo e espaço. Mais seriamente, pode resultar na retenção da instância anexante quando, de outra forma, seria elegível para coleta de lixo (Item 7). O vazamento de memória resultante pode ser catastrófico. Muitas vezes é difícil de detectar porque a referência é invisível.

Um uso comum de classes estáticas privadas é representar componentes do objeto representado pela classe que encerra. Por exemplo, considere um Mapinstance, que associa chaves a valores. Muitas implementações de Map possuem um objeto Entry interno para cada par-chave-valor no mapa. Embora cada entrada esteja associada a um mapa, os métodos em uma entrada (getKey, getValue e setValue) não precisam de acesso ao mapa. Portanto, seria desperdício usar uma classe nãostaticmember para representar entradas: uma classe privada static member é a melhor. Se você acidentalmente omitir o modificador estático na declaração de entrada, o mapa ainda funcionará, mas cada entrada conterá uma referência supérflua ao mapa, o que desperdiça espaço e tempo.

É duplamente importante escolher corretamente entre uma classe estática e uma não-staticmember se a classe em questão for um membro público ou protegido de um

classe exportada. Nesse caso, a classe membro é um elemento API exportado e não pode ser alterada de uma classe membro não estática para uma estática em uma versão subsequente sem violar compatibilidade retroativa.

Como era de se esperar, uma turma anônima não tem nome. Não faz parte da classe que o cerca. Em vez de ser declarado junto com outros membros, ele é simultaneamente declarado e instanciado no ponto de uso. Classes anônimas são permitidas em qualquer ponto do código onde uma expressão seja legal. Classes anônimas possuem instâncias envolventes se e somente se ocorrerem em um contexto não estático. Mas mesmo que ocorram em um contexto estático, não podem ter membros estáticos além de variáveis constantes, que são campos primitivos finais ou de string inicializados para expressões constantes [JLS, 4.12.4].

Existem muitas limitações quanto à aplicabilidade de aulas anônimas. Você não pode instanciar eles a não ser no momento em que são declarados. Você não pode realizar testes de instância nem fazer qualquer outra coisa que exija que você nomeie a turma. Você não pode declarar uma classe anônima para implementar múltiplas interfaces ou estender uma classe e implementar uma interface ao mesmo tempo. Clientes de uma classe anônima não podem invocar nenhum membro além daqueles herdados de seu supertipo. Como as aulas anônimas ocorrem no meio das expressões, elas devem ser curtas — cerca de dez linhas ou menos — ou a legibilidade será prejudicada.

Antes da adição dos lambdas ao Java (Capítulo 6), classes anônimas eram o meio preferido de criar pequenos objetos de função e processar objetos em tempo real, mas agora os lambdas são preferidos (Item 42). Outro uso comum de classes anônimas é na implementação de métodos de fábrica estática (veja `intArrayAsList` no Item 20).

As classes locais são as menos usadas dos quatro tipos de classes aninhadas. Uma classe local pode ser declarada praticamente em qualquer lugar onde uma variável local possa ser declarada e obedece às mesmas regras de escopo. Classes locais têm atributos em comum com cada um dos outros tipos de classes aninhadas. Assim como as classes membros, elas têm nomes e podem ser usadas repetidamente. Como as classes anônimas, elas têm instâncias envolventes somente se forem definidas em um contexto não estático, e não podem conter membros estáticos. E, assim como as aulas anônimas, devem ser mantidas curtas para não prejudicar a legibilidade.

Para recapitular, existem quatro tipos diferentes de classes aninhadas, e cada uma tem seu lugar. Se uma classe aninhada precisar ser visível fora de um único método ou for longa demais para caber confortavelmente dentro de um método, use uma classe membro. Se cada instância de uma classe membro precisar de uma referência à sua instância envolvente, torne-a não estática; Caso contrário, faça com que seja estático. Assumindo que a classe pertence a um método, se você precisar criar instâncias a partir de apenas um local e houver um tipo pré-existente que caracterize a classe, torne-a uma classe anônima; Caso contrário, faça uma aula local.

Item 25: Limitar os arquivos-fonte a uma única classe de nível superior

Embora o compilador Java permita definir múltiplas classes de topo em um único arquivo fonte, não há benefícios associados a isso, e há riscos significativos. Os riscos vêm do fato de que definir múltiplas classes de topo em um arquivo fonte torna possível fornecer múltiplas definições para uma classe. Qual definição será usada é afetada pela ordem em que os arquivos fonte são passados para o compilador.

Para tornar isso concreto, considere este arquivo fonte, que contém apenas uma Mainclass que se refere a membros de outras duas classes de nível superior (Utensílio e Sobremesa):

```
classe pública Main {
    public static void main(String[] args) {
        System.out.println(Utensil.NAME + Dessert.NAME);}}
```

Agora suponha que você defina tanto Utensílio quanto Sobremesa em um único arquivo fonte named Utensil.java:

```
Duas classes definidas em um único arquivo. Nunca faça isso! Talheres de classe
{
    final estático String NAME = "pan";}

classe Sobremesa {
    final estático String NAME = "bolo";}
```

Claro que o programa principal imprime panquecas.

Agora suponha que você acidentalmente crie outro arquivo fonte named Dessert.java que defina as mesmas duas classes:

```
Duas classes definidas em um único arquivo. Nunca faça isso! Talheres de classe
{
    final estático NOME da String = "pot";}

classe Sobremesa {
    final static STRING NAME = "pie";}
```

Se você tiver a sorte de compilar o programa com o Main.java Dessert.java commandjavac, a compilação vai falhar, e o compilador vai

dizer que você definiu múltiplas as classes `Utensílio` e `Sobremesa`. Isso porque o compilador primeiro compila `Main.java` e, quando vê a referência a `Utensílio` (que precede a referência a `Sobremesa`), ele procura em `Utensil.java` para essa classe e encontra tanto `Utensílio` quanto `Sobremesa`. Quando o `com-piler` encontra `Dessert.java` na linha de comando, ele puxa esse arquivo também, fazendo com que encontre as definições de `Utensílio` e `Sobremesa`.

Se você compilar o programa com o comando `javac Main.java orjavac Main.java Utensil.java`, ele vai se comportar como antes de você escrever `theDessert.java` arquivo, imprimindo `pancake`. Mas se você compilar o programa com o comando `javac Dessert.java Main.java`, ele vai imprimir `potpie`. O comportamento do programa é, portanto, afetado pela ordem em que os arquivos-fonte são passados para o compilador, o que é claramente inaceitável.

Resolver o problema é tão simples quanto dividir as classes de nível superior (`Utensil` e `Dessert`, no caso do nosso exemplo) em arquivos-fonte separados. Se você estiver tentado a colocar várias classes de topo em um único arquivo fonte, considere usar classes `staticmember` (Item 24) como alternativa a dividir as classes em arquivos de fonte separados. Se as classes são subservientes a outra classe, torná-las classes membros estáticos geralmente é a melhor alternativa porque melhora a legibilidade e possibilita reduzir a acessibilidade das classes declarando-as privadas (Item 15). Veja como nosso exemplo é com classes membros estáticas:

```
Classes membros estáticas em vez de múltiplas classes de nível superior
classe pública Test {
    public static void main(String[] args) {
        System.out.println(Utensil.NAME + Dessert.NAME);}

    classe estática privada Utensil {
        final estático String NAME = "pan";}

    classe privada estática Dessert {
        final estático String NAME = "cake";}}
```

A lição é clara: nunca coloque múltiplas classes ou interfaces de alto nível em um único arquivo fonte. Seguir essa regra garante que você não pode ter múltiplas definições para uma única classe em tempo de compilação. Isso, por sua vez, garante que os arquivos de classe gerados pela compilação e o comportamento do programa resultante sejam independentes da ordem em que os arquivos fonte são passados para o compilador.

CHAPTER 5

Genéricos

DESDE o Java 5, genéricos fazem parte da linguagem. Antes dos genéricos, você tinha para lançar todos os objetos que você lê de uma coleção. Se alguém acidentalmente inserisse um objeto do tipo errado, os casts poderiam falhar em tempo de execução. Com genéricos, você informa ao thecompiler quais tipos de objetos são permitidos em cada coleção. O compilador insere casts automaticamente para você e avisa na hora da compilação se você tentar inserir um objeto do tipo errado. Isso resulta em programas que são ao mesmo tempo mais seguros e claros, mas esses benefícios, que não se limitam às cobranças, têm um preço. Este capítulo explica como maximizar os benefícios e minimizar as complicações.

Item 26: Não use tipos brutos

Primeiro, alguns termos. Uma classe ou interface cuja declaração possui um ou mais parâmetros de tipo é uma classe ou interface genérica [JLS, 8.1.2, 9.1.2]. Por exemplo, a interface `List` possui um único parâmetro de tipo, `E`, que representa seu tipo de elemento. O nome completo da interface é `List<E>` (leia-se "lista de E"), mas as pessoas costumam chamá-la de `List`, para abreviar. Classes e interfaces genéricas são coletivamente conhecidas como tipos genéricos.

Cada tipo genérico define um conjunto de tipos parametrizados, que consistem no nome da classe ou interface seguido de uma lista com colchetes angulados dos parâmetros reais do tipo correspondentes aos parâmetros formais do tipo genérico [JLS, 4.4,

Por exemplo, `List<String>` (leia-se "lista de string") é um tipo parametrizado que representa uma lista cujos elementos são do tipo `String`. (`String` é o parâmetro de tipo real correspondente ao parâmetro formal de tipo `E`.) Finalmente, cada tipo genérico define um tipo bruto, que é o nome do genérico usado sem quaisquer parâmetros de tipo acompanhantes [JLS, 4.8]. Por exemplo, o tipo `theraw` correspondente a `List<E>` é `List`. Tipos brutos se comportam como se todas as informações genéricas de tipo tivessem sido apagadas da declaração de tipo. Eles existem principalmente para compatibilidade com códigos pré-genéricos.

Antes da adição dos genéricos ao Java, isso teria sido uma declaração exemplar de coleção. Até o Java 9, ainda é legal, mas está longe de ser exemplar:

```
Tipo de coleta bruta - não faça isso!
```

```
Minha coleção de selos. Contém apenas instâncias de selos.selos finais privados da
coleção final = ... ;
```

Se você usar essa declaração hoje e depois acidentalmente colocar uma moeda na sua coleção de selos, a inserção errada compila e roda sem erro (embora o compilador emita um aviso vago):

```
Inserção errada de moeda em selos-selos de coleção.add(nova moeda( ... )); Emite
aviso de "chamada descontrolada"
```

Você não recebe erro até tentar recuperar a moeda da coleção de selos:

```
Tipo bruto de iterador - não faça isso!for (Iterador i = stamps.iterator();
i.hasNext(); )
    Carimbo = (Carimbo) i.next(); Lança ClassCastException
    stamp.cancel();
```

Como mencionado ao longo deste livro, vale a pena identificar erros o mais rápido possível após serem cometidos, de preferência na época da compilação. Nesse caso, você só descobre o erro em tempo de execução, muito depois de ele ter ocorrido, e em um código que pode estar distante do código que contém o erro. Quando você vê o `ClassCastException`, precisa procurar no código a invocação do método que colocou o `thecoin` na coleção de selos. O compilador não pode te ajudar, porque não entende o comentário que diz: "Contém apenas instâncias de Stamp."

Com genéricos, a declaração de tipo contém a informação, não o comentário:

```
Tipo de coleção parametrizada - typesafeprivate final<Stamp> Selos da coleção = ...
;
```

A partir dessa declaração, o compilador sabe que os carimbos devem conter apenas `Stamp` instances e garante que isso seja verdadeiro, assumindo que toda a sua base de código compila sem emitir (ou suprimir; veja o Item 27) qualquer aviso. Quando o carimbo é declarado com uma declaração de tipo parametrizada, a inserção errada gera uma mensagem de erro em tempo de compilação que diz exatamente o que está errado:

```
Test.java:9: erro: tipos incompatíveis: Moeda não pode ser convertida em Selo
```

```
c.add (nova moeda());
      ^
```

O compilador insere casts invisíveis para você ao recuperar elementos das coleções e garante que eles não vão falhar (assumindo, novamente, que todo o seu código não gerou ou suprimiu nenhum aviso do compilador). Embora a possibilidade de inserir acidentalmente uma moeda em uma coleção de selos possa parecer improvável, o problema é real. Por exemplo, é fácil imaginar colocar um `BigInteger` em uma coleção que deveria conter apenas instâncias `BigDecimal`.

Como mencionado anteriormente, é legal usar tipos brutos (tipos genéricos sem seus parâmetros de tipo), mas você nunca deve fazer isso. Se você usar tipos brutos, perde todos os benefícios de segurança e expressividade dos genéricos. Considerando que você não deve usá-los, por que os designers de linguagem permitiram tipos brutos em primeiro lugar? Por compatibilidade. O Java estava prestes a entrar em sua segunda década quando os genéricos foram adicionados, e havia uma enorme quantidade de código existente que não usava genéricos. Foi considerado fundamental que todo esse código permanecesse legal e interoperasse com códigos mais recentes que usam genéricos. Tinha que ser legal passar instâncias de tipos parametrizados para métodos projetados para uso com tipos brutos, e vice-versa. Esse requisito, conhecido como compatibilidade de migração, levou as decisões a suportar tipos brutos e implementar genéricos usando apagamento (Item 28).

Embora você não deva usar tipos brutos como `List`, é aceitável usar tipos parametrizados para permitir a inserção de objetos arbitrários, como `List<Object>`. Qual é a diferença entre a Lista de tipos bruto e a Lista de tipos parametrizada `<Object>`? De forma geral, o primeiro optou por não aderir ao sistema de tipos genérico, enquanto o segundo informou explicitamente ao compilador que é capaz de armazenar objetos de qualquer tipo. Embora você possa passar `<String>` uma Lista para um parâmetro do tipo Lista, não pode passá-la para um parâmetro do tipo `List<Object>`. Existem regras de subtipagem para genéricos, e `List<String>` é um subtipo do tipo bruto `List`, mas não do tipo parametrizado `List<Object>` (Item 28). Como consequência, você perde a segurança de tipo se usar um tipo bruto como `List`, mas não se usar um tipo parametrizado como `List<Object>`.

Para concretizar isso, considere o seguinte programa:

```
Falhas em tempo de execução - o método unsafeAdd usa um tipo bruto
(List)!public static void main(String[] args) {
    <String> String de lista = novo ArrayList<>(); unsafeAdd(strings,
    Integer.valueOf(42)); String s = strings.get(0); Tem cast gerado
    pelo compilador}

private static void unsafeAdd(List list, Object o) {
    list.add(o);}
}
```

Este programa compila, mas como usa o tipo bruto `List`, você recebe um aviso:

```
Test.java:10: aviso: [desmarcado] chamada não marcada para
adicionar(E) como membro do tipo bruto Lista
    list.add(o);
           ^
```

E, de fato, se você rodar o programa, recebe uma `ClassCastException` quando o programa tenta lançar o resultado das strings de invocação `get(0)`, que é um `Inteiro`, para uma `String`. Este é um cast gerado pelo compilador, então normalmente é garantido que tenha sucesso, mas neste caso ignoramos um aviso do compilador e pagamos o preço.

Se você substituir o tipo bruto `List` pelo tipo parametrizado `List<Object>` na declaração `unsafeAdd` e tentar recompilar o programa, verá que ele não compila mais, mas emite a mensagem de erro:

```
Test.java:5: erro: tipos incompatíveis: Lista<String> não pode ser
convertida em Lista<Object>
    unsafeAdd(strings, Integer.valueOf(42));
           ^
```

Você pode ficar tentado a usar um tipo bruto para uma coleção cujo tipo de elemento é desconhecido e não importa. Por exemplo, suponha que você queira escrever um método que pega dois conjuntos e retorna o número de elementos que eles têm em comum. Veja como você escreveria esse método se fosse novo em genéricos:

```
Uso do tipo bruto para tipo de elemento desconhecido - não faça
isso! static int numElementsInCommon(Set s1, Set s2) {
    resultado de int = 0; para
    (Objeto o1 : s1) se
    (s2 contém(o1))
        result++;
    retorne resultado;}

```

Esse método funciona, mas usa tipos brutos, que são perigosos. O alternativo seguro é usar tipos de coringa ilimitados. Se você quiser usar um tipo genérico, mas não sabe ou não se importa com o parâmetro de tipo real, pode usar um ponto de interrogação em vez disso. Por exemplo, o tipo de coringa ilimitado para o `Type Genérico Set<E>` é `Set<?>` (leia-se "conjunto de algum tipo"). É o tipo de `Set` parametrizado mais geral, capaz de conter qualquer conjunto. Veja como o `numElementsInCommon` declaration se apresenta com tipos de wildcard ilimitados:

```
Usa tipo curinga ilimitado - typesafe e flexible static int
numElementsInCommon(Set<?> s1, Set<?> s2) { ... }
```

Qual é a diferença entre o tipo curinga ilimitado `Set<?>` e o tipo `throw Set`? A interrogação realmente te dá alguma coisa? Sem querer insistir, o tipo `throw Set` é seguro e o tipo bruto não é. Você pode colocar qualquer elemento em uma coleção com um tipo bruto, corrompendo facilmente o tipo invariante da coleção (como demonstrado pelo método `unsafeAdd` na página 119); você não pode colocar nenhum elemento (além de nulo) em uma `Coleção<?>`. Tentar fazer isso gerará uma mensagem de erro em tempo de compilação como esta:

```
Wildcard.java:13: erro: tipos incompatíveis: String não pode ser
convertida para CAP#1
    c.add("Proibido");
      ^
onde CAP#1 é uma variável de tipo nova:
    CAP#1 estende Objeto a partir da captura de ?
```

Admito que essa mensagem de erro deixa a desejar, mas o `com-piler` cumpriu seu papel, impedindo que você corrompa o tipo invariável da coleção, seja qual for o tipo de elemento. Você não só não pode colocar nenhum elemento (além de nulo) em uma `Coleção<?>`, como também não pode assumir nada sobre o tipo de objetos que você obtém. Se essas restrições forem inaceitáveis, você pode usar métodos genéricos (Item 30) ou tipos de curinga limitados (Item 31).

Existem algumas pequenas exceções à regra de que você não deve usar tipos brutos. Você deve usar tipos brutos nos literais da classe. A especificação não permite o uso de tipos parametrizados (embora permita tipos de `array` e `primitivetypes`) [JLS, 15.8.2]. Em outras palavras, `List.class`, `String[].class`, `andint.class` são todos legais, mas `<String>List.class` e `List<?>.class` não são.

Uma segunda exceção à regra diz respeito à instância do operador. Como informações genéricas de tipo são apagadas em tempo de execução, é ilegal usar o `instanceof` operador em tipos parametrizados que não sejam tipos de curinga ilimitados. O uso de tipos `throw` limitados em função dos tipos brutos não afeta o comportamento do operador `theinstanceof` de forma alguma. Nesse caso, os colchetes angulares e pontos de interrogação são apenas ruído. Esta é a forma preferida de usar o operador de instância com tipos genéricos:

```
Uso legítimo do tipo bruto - instanceof operator if (o
instance of Set) { // Tipo bruto
    Set<?> s = (Set<?>) o;      Tipo wildcard
    ...}
```

Note que, uma vez determinado que o é um Set, você deve conjurá-lo para o tipo coringa Set<?>, não para o tipo bruto Set. Este é um cast verificado, então não causará um aviso do compilador.

Resumindo, usar tipos brutos pode gerar exceções em tempo de execução, então não os use. Eles são fornecidos apenas para compatibilidade e interoperabilidade com o legacycode que antecede a introdução dos genéricos. Como uma rápida revisão, Set<Object> é um tipo parametrizado que representa um conjunto que pode conter objetos de qualquer tipo, Set<?> é um tipo coringa que representa um conjunto que pode conter apenas objetos de um tipo desconhecido, e Set é um tipo bruto, que opta por não usar o sistema genérico de tipos. Os dois primeiros são seguros, e o último não.

Para referência rápida, os termos introduzidos neste item (e alguns introduzidos posteriormente neste capítulo) são resumidos na tabela a seguir:

Mandato	Exemplo	Item
Tipo parametrizado	Lista<String>	Item 26
Parâmetro de tipo real	Corda	Item 26
Tipo genérico	Lista<E>	Itens 26, 29
Parâmetro formal de tipo	E	Item 26
Tipo de coringa ilimitado	Lista<?>	Item 26
Tipo bruto	Lista	Item 26
Parâmetro de tipo limitado	<E extends Number>	Item 29
Limitado por tipos recursivo	<T extends Comparable<T>>	Item 30
Tipo de coringa limitado	Lista<? estende Número>	Item 31
Método genérico	<E> Lista<E> estática comoLista(E[] a)	Item 30
Token de tipo	String.class	Item 33

Item 27: Eliminar avisos não verificados

Quando você programa com genéricos, verá muitos avisos de compilador: avisos de cast desmarcados, avisos de invocação de métodos desmarcados, avisos de tipo vararg parameterizados não marcados e avisos de conversão não verificados. Quanto mais experiência você adquirir com genéricos, menos avisos receberá, mas não espere que o código recém-escrito compile de forma limpa.

Muitos avisos não verificados são fáceis de eliminar. Por exemplo, suponha que você acidentalmente escreva esta declaração:

```
<Lark> Exaltação de conjunto = novo HashSet();
```

O compilador vai lembrar gentilmente o que você fez de errado:

```
Venery.java:4: aviso: [desmarcado] conversão desmarcada
  <Lark> Exaltação de conjunto = novo HashSet();
                        ^
necessário: Set<Lark>encontrado:HashSet
```

Você pode então fazer a correção indicada, fazendo o aviso desaparecer. Note que você não precisa especificar o parâmetro de tipo, apenas indicar que ele está presente com o operador diamante ($\diamond$), introduzido no Java 7. O com-compiler então inferirá o parâmetro de tipo correto (neste caso, Lark):

```
<Lark> Exaltação de conjunto = novo HashSet<>();
```

Alguns avisos serão muito mais difíceis de eliminar. Este capítulo está repleto de exemplos desses avisos. Quando receber avisos que exigem alguma reflexão, persevere! Elimine todos os avisos não controlados que puder. Se você eliminar todos os avisos, tem a certeza de que seu código é typesafe, o que é muito bom. Isso significa que você não receberá uma `ExceptionClassCast` em tempo de execução, e aumenta sua confiança de que seu programa vai se comportar como você planejou.

Se você não puder eliminar um aviso, mas puder provar que o código que provocou o aviso é typesafe, então (e só então) suprima o aviso com uma anotação `@SuppressWarnings("não marcada")`. Se você suprimir avisos sem antes provar que o código é typesafe, está dando a si mesmo uma falsa sensação de segurança. O código pode compilar sem emitir nenhum aviso, mas ainda pode lançar uma `ClassCastException` em tempo de execução. Se, no entanto, você ignorar avisos não verificados que sabe serem seguros (em vez de suprimi-los), não vai perceber quando um novo aviso aparecer que representa um problema real. O novo aviso vai se perder em meio a todos os falsos alarmes que você não silenciou.

A anotação `SuppressWarnings` pode ser usada em qualquer declaração, desde uma declaração local individual até uma classe inteira. Sempre use a anotação `SuppressWarnings` no menor escopo possível. Normalmente, isso será uma declaração de variável ou um método ou construtor muito curto. Nunca use `SuppressWarnings` em uma turma inteira. Fazer isso pode mascarar avisos críticos.

Se você se pegar usando a anotação `SuppressWarnings` em um método ou construtor com mais de uma linha, talvez consiga movê-la para uma declaração local variável. Talvez você precise declarar uma nova variável local, mas vale a pena. Por exemplo, considere este método `toArray`, que vem do `ArrayList`:

```
public <T> T[] toArray(T[] a) {
    se (a.length < size)
        return (T[]) Arrays.copyOf(elements, size, a.getClass());
    system.arraycopy(elementos, 0, a, 0, tamanho); se (a.length > size)

    a[tamanho] = nulo;
    retorne a;}
```

Se você compilar o `ArrayList`, o método gera este aviso:

```
ArrayList.java:305: aviso: [desmarcado] elenco desmarcado
    return (T[]) Arrays.copyOf(elements, size, a.getClass());
                               ^
requerido: T[]encontrado:Object[]
```

É ilegal colocar uma anotação `SuppressWarnings` na instrução `return`, porque não é uma declaração [JLS, 9.7]. Você pode ficar tentado a colocar a anotação em todo o método, mas não faça isso. Em vez disso, declare uma variável local para segurar o valor `thereturn` e anote sua declaração, assim:

```
Adicionando variável local para reduzir o escopo de @SuppressWarningspublic <T> T[]
toArray(T[] a) {
    se (a.comprimento < tamanho) {
        Esse cast está correto porque o array que estamos criando// é do mesmo tipo
        do que passou, que é T[].@SuppressWarnings("desmarcado") T[] resultado =

        (T[]) Arrays.copyOf(elements, size, a.getClass()); retorne
        resultado;}system.arraycopy(elementos, 0, a, 0, tamanho); se
        (a.length > size)

    a[tamanho] = nulo;
    retorne a;}
```

O método resultante compila de forma limpa e minimiza o escopo no qual avisos não controlados são suprimidos.

Toda vez que você usar uma anotação `@SuppressWarnings("não marcada")`, adicione um comentário explicando por que é seguro fazê-lo. Isso ajudará outras pessoas a entenderem o código e, mais importante, diminuirá as chances de alguém modificar o código para tornar o cálculo inseguro. Se você achar difícil escrever esse tipo de comentário, continue pensando. Você pode acabar percebendo que a operação sem controle não é segura, afinal.

Em resumo, avisos não verificados são importantes. Não os ignore. O aviso `Unchecked` representa o potencial para uma `ClassCastException` em tempo de execução. Faça o possível para eliminar esses avisos. Se você não conseguir eliminar um aviso não verificado e conseguir provar que o código que o provocou é `typesafe`, suprima o aviso com uma anotação `@SuppressWarnings("não verificada")` no escopo mais restrito possível. Registre a justificativa da sua decisão de suprimir o aviso em um comentário.

Item 28: Preferência listas a arrays

Arrays diferem dos tipos genéricos em dois aspectos importantes. Primeiro, os arrays são covariantes. Essa palavra assustadora significa simplesmente que, se `Sub` é um subtipo de `Super`, então o tipo de arranjo `Sub[]` é um subtipo do tipo de arranjo `Super[]`. Genéricos, por outro lado, são invariantes: para quaisquer dois tipos distintos `Tipo1` e `Tipo2`, `Lista<Tipo1>` não é nem subtipo nem supertipo de `Lista<Tipo2>` [JLS, 4.10; Naftalin07, 2.5]. Você pode pensar que isso significa que genéricos são deficientes, mas, pode-se argumentar, são os arrays que são deficientes. Este fragmento de código é legal:

```
Falha em tempo de execução! Object[] objectArray = novo Long[1];
objectArray[0] = "Eu não me encaixo"; // Throws ArrayStoreException
```

Mas este não é:

```
Não vai compilar! List<Object> ol = new <Long>ArrayList(); //
Incompatível typesol.add("Eu não me encaixo");
```

De qualquer forma, você não pode colocar uma `String` em um contêiner `Long`, mas com um array você descobre que cometeu um erro em tempo de execução; Com uma lista, você descobre no momento da compilação. Claro, você prefere descobrir na hora da compilação.

A segunda grande diferença entre arrays e genéricos é que os arrays são reificados [JLS, 4.7]. Isso significa que os arrays conhecem e aplicam seu tipo de elemento em tempo de execução. Como mencionado anteriormente, se você tentar colocar uma `String` em um array de `Long`, você receberá uma `ArrayStoreException`. Genéricos, por outro lado, são implementados por apagamento [JLS, 4.6]. Isso significa que eles aplicam suas restrições de tipo apenas em tempo de compilação e descartam (ou apagam) suas informações de tipo de elemento em tempo de execução. A apagamento foi o que permitiu que tipos genéricos interoperassem livremente com código legado que não usava genéricos (Item 26), garantindo uma transição suave para genéricos no Java 5.

Por causa dessas diferenças fundamentais, arrays e genéricos não combinam bem. Por exemplo, é ilegal criar um array de tipo genérico, tipo parametrizado ou parâmetro tipo. Portanto, nenhuma dessas expressões de criação de array é legal: `nova Lista<E>[]`, `nova Lista<String>[]`, `nova E[]`. Todos resultarão em erros de criação de arranjos genéricos em tempo de compilação.

Por que é ilegal criar um array genérico? Porque não é tipo seguro. Se fosse legal, casts gerados pelo compilador em um programa correto poderiam falhar em tempo de execução com uma `ClassCastException`. Isso violaria a garantia fundamental fornecida pelo sistema genérico de tipos.

Para tornar isso mais concreto, considere o seguinte fragmento de código:

```
Por que a criação de arrays genéricos é ilegal - não compila!  
List<String>[] stringLists = nova Lista<String>[1];//  
(1)Lista<Integer> intLista = Lista.de(42);// (2)Objecto[]  
objetos = stringLists;// (3)objetos[0] = intLista;// (4)String  
s = stringLists[0].get(0);// (5)
```

Vamos supor que a linha 1, que cria um array genérico, seja legal. A linha 2 cria e inicializa uma `Lista<Integer>` contendo um único elemento. A linha 3 armazena o `String` array `theList` em uma variável do array `Object`, o que é legal porque os arrays são covariantes. A linha 4 armazena a `Lista<Integer>` no único elemento do array `theObject`, que é bem-sucedido porque genéricos são implementados por apagamento: o tipo de tempo de tempo de uma `<Integer>` instância de `List` é simplesmente `List`, e o tipo de runtime de uma `<String>` instância `List[]` é `List[]`, então essa atribuição não gera uma `ArrayStoreException`. Agora estamos encenados. Armazenamos uma `<Integer>` instância de `List` em um array que é declarado para conter apenas `<String>` instâncias de `List`. Na linha 5, recuperamos o único elemento da lista única neste arranjo. O compilador automaticamente lança o elemento recuperado para `String`, mas é um `Inteiro`, então recebemos uma `ClassCastException` em tempo de execução. Para evitar que isso aconteça, a linha 1 (que cria um array genérico) deve gerar um erro em tempo de compilação.

Tipos como `E`, `Lista<E>` e `Lista<String>` são tecnicamente conhecidos como tipos não reficáveis [JLS, 4.7]. Intuitivamente falando, um tipo não reficável é aquele cuja representação em tempo de tempo contém menos informação do que sua representação em tempo de compilação. Por causa do apagamento, os únicos tipos parametrizados que são reficáveis são os tipos curinga ilimitados, como `Lista<?>` e `Mapa<?,?>` (Item 26). É legal, embora raramente útil, criar arrays de tipos de wildcard ilimitados.

A proibição de criar arrays genéricos pode ser irritante. Isso significa, para examina-ple, que geralmente não é possível que uma coleção genérica retorne um array do tipo seu elemento (mas veja o Item 33 para uma solução parcial). Também significa que você recebe avisos confusos ao usar métodos `varargs` (Item 53) em combinação com tipos genéricos. Isso porque toda vez que você invoca um método `varargs`, um array é criado para armazenar os parâmetros `varargs`. Se o tipo de elemento desse array não for reficável, você recebe um aviso. A anotação `SafeVarargs` pode ser usada para resolver essa questão (Item 32).

Quando você recebe um erro genérico de criação de array ou um aviso de cast desverificado em acast para um tipo de array, a melhor solução geralmente é usar a preferência do tipo de coleção `List<E>` ao tipo de array `E[]`. Você pode sacrificar um pouco de concisão ou performance, mas em troca obtém melhor segurança de tipos e interoperabilidade.

Por exemplo, suponha que você queira escrever uma classe `Chooser` com um construtor que recebe uma coleção, e um único método que retorna um elemento da coleção escolhido aleatoriamente. Dependendo da coleção que você passa para o construtor, você pode usar um `escolhedor` como dado de jogo, uma bola mágica 8 ou uma fonte de dados para uma simulação Monte Carlo. Aqui está uma implementação simplista, sem genéricos:

```
Chooser - uma classe que precisa urgentemente de genéricos!
classe pública Chooser {

    escolha privada de Objecto final[] Array;

    Public Chooser(Escolhas de coleção) {
        choiceArray = choices.toArray();}

    Objeto público choose() {
        Random rnd = ThreadLocalRandom.current(); return
        choiceArray[rnd.nextInt(choiceArray.length)];}}
```

Para usar essa classe, você precisa lançar o valor de retorno do método `choose` `fromObject` para o tipo desejado toda vez que usar o método `invoke`, e o cast falhará em tempo de execução se você errar o tipo. Seguindo o conselho do Item 29 a sério, tentamos modificar o `Chooser` para torná-lo genérico. As mudanças são mostradas em negrito:

```
Um primeiro passo para tornar o Chooser genérico - não compila a classe pública<T>
Chooser {
    escolha final privada T[] Array;

    Public Chooser(<T>Escolhas de coleção) {
        choiceArray = choices.toArray();}

    método escolha inalterado}
```

Se você tentar compilar essa classe, receberá esta mensagem de erro:

```
Chooser.java:9: erro: tipos incompatíveis: Object[] não pode ser convertido em T[]

    choiceArray = choices.toArray();
                        ^
onde T é uma variável de tipo:
    T estende Objeto declarado na classe Escolhedor
```

Sem problema, você diz, vou fundir o array de Object para um array T:

```
choiceArray = (T[]) escolhas.toArray();
```

Isso elimina o erro, mas em vez disso você recebe um aviso:

```
Chooser.java:9: aviso: [desmarcado] elenco não verificado
    choiceArray = (T[]) escolhas.toArray();
                        ^
required: T[], found: Object[] onde T é uma variável
de tipo: T estende Objeto declarado na classe Chooser
```

O compilador está dizendo que não pode garantir a segurança do elenco em tempo de execução porque o programa não saberá qual tipo T representa — lembre-se, informações de elementos são apagadas dos genéricos em tempo de execução. O programa vai funcionar? Sim, mas o compilador não pode provar. Você poderia provar para si mesmo, colocar a prova em um comentário e suprimir o aviso com uma anotação, mas é melhor eliminar a causa do aviso (Item 27).

Para eliminar o aviso de lançamento desmarcado, use uma lista em vez de um array. Aqui está uma versão da classe Chooser que compila sem erro ou aviso:

```
Seletor baseado em listas - typesafe public classe Chooser<T> {

    final privado<T> Lista escolhidaLista;

    Public Chooser(<T>Escolhas de coleção) {
        choiceList = nova ArrayList<>(escolhas);}

    public T choose() {
        Random rnd = ThreadLocalRandom.current(); return
        choiceList.get(rnd.nextInt(choiceList.size()));}
```

Esta versão é um pouco mais prolixa, e talvez um pouco mais lenta, mas vale a pena pela tranquilidade de que você não terá uma `ClassCastException` durante a execução.

Resumindo, arrays e genéricos têm regras de tipo muito diferentes. Arrays são covariantes e reificados; genéricos são invariantes e apagados. Como consequência, arrays fornecem segurança de tipos em tempo de execução, mas não segurança de tipo em tempo de compilação, e vice-versa para genéricos. Como regra, arrays e genéricos não combinam bem. Se você se pegar misturando e recebendo erros ou avisos em tempo de compilação, seu primeiro impulso deve ser substituir os arrays por listas.

Item 29: Favoreça tipos genéricos

Geralmente não é muito difícil parametrizar suas declarações e usar os tipos e métodos genéricos fornecidos pelo JDK. Escrever seus próprios generic types é um pouco mais difícil, mas vale a pena aprender como fazer.

Considere a implementação simples (de brinquedo) da pilha do Item 7:

```
Coleção baseada em objetos - um candidato principal para a classe genericspublic
Stack {
    elementos privados de Objeto[] private int size = 0; Int.
    final estática privada DEFAULT_INITIAL_CAPACITY = 16;

    public Stack() {
        elementos = novo Objeto[DEFAULT_INITIAL_CAPACITY];}

    empty public push(Object e) {
        ensureCapacity();
        elementos[size++] = e;}

    Objeto público pop() {
        if (tamanho == 0)
            lançar um novo EmptyStackException(); Resultado do objeto =
            elementos[--tamanho]; elementos[tamanho] = nulo; Eliminar o
            resultado obsoleto de referência;}

    boolean public isEmpty() {
        tamanho do retorno == 0;}

    private void ensureCapacity() {
        if (elements.length == tamanho)
            elements = Arrays.copyOf(elements, 2 * tamanho + 1);
    }}
}
```

Essa classe deveria ter sido parametrizada desde o início, mas como não foi, podemos generalizá-la depois. Em outras palavras, podemos parametrizá-lo sem prejudicar os clientes da versão original não parametrizada. Como está, o cliente tem objetos tocata que são retirados da pilha, e esses casts podem falhar em tempo de execução. O primeiro passo para gerar uma classe é adicionar um ou mais parâmetros de tipo a seu

declaração. Neste caso, existe um parâmetro de tipo, representando o tipo de elemento da pilha, e o nome convencional para esse parâmetro de tipo é E (Item 68).

O próximo passo é substituir todos os usos do tipo Objeto pelo parâmetro apropriado e então tentar compilar o programa resultante:

```
Tentativa inicial de generificar o Stack - não compila a classe pública<E>
Stack {
    elementos privados E[]; private int size = 0; Int. final
    estática privada DEFAULT_INITIAL_CAPACITY = 16;

    public Stack() {
        elementos = novo E[DEFAULT_INITIAL_CAPACITY];}

    empty public push(E e) {
        ensureCapacity();
        elementos[size++] = e;}

    public E pop() {
        if (tamanho == 0)
            lançar um novo EmptyStackException(); E resultado = elementos[--
            tamanho]; elementos[tamanho] = nulo; Eliminar o resultado obsoleto
            de referência; }... // nenhuma mudança no isVazio ou Capacidade de
            Garantia}
```

Geralmente você recebe pelo menos um erro ou aviso, e essa aula não é exceção. Felizmente, essa classe gera apenas um erro:

```
Stack.java:8: criação de arrays genéricos
    elementos = novo E[DEFAULT_INITIAL_CAPACITY];
                ^
```

Como explicado no Item 28, você não pode criar um array de tipo não reficável, suchas E. Esse problema surge toda vez que você escreve um tipo genérico que é suportado por anarray. Existem duas maneiras razoáveis de resolver isso. A primeira solução contorna diretamente a proibição da criação de arrays genéricos: criar um array de Object e castar

para o tipo de array genérico. Agora, no lugar de um erro, o compilador emitirá um aviso. Esse uso é legal, mas não é (em geral) typesafe:

```
Stack.java:8: aviso: [desmarcado] não marcado castfound: Object[], required: E[]
    elementos = (E[]) novo Objeto[DEFAULT_INITIAL_CAPACITY];
                  ^
```

O compilador pode não conseguir provar que seu programa é typesafe, mas você pode. Você precisa se convencer de que o elenco não controlado não comprometerá a segurança do tipo do programa. O array em questão (elementos) é armazenado em um campo privado e nunca retorna ao cliente nem é passado para qualquer outro método. Os únicos elementos armazenados no array são aqueles passados para o método push, que são do tipo E, então o cast não controlado não pode causar dano.

Depois de provar que um lançamento não controlado é seguro, suprima o aviso o mais restringido possível (Item 27). Nesse caso, o construtor contém apenas a criação de array sem controle, então é apropriado suprimir o aviso no inteiro do construtor. Com a adição de uma anotação para isso, o Stack compila de forma limpa, e você pode usá-lo sem casts explícitos ou medo de uma ClassCastException:

```
O array de elementos conterá apenas E instâncias de push(E).// Isso é
suficiente para garantir a segurança do tipo, mas o tipo de runtime//
do array não será E[]; sempre será
Object[]!@SuppressWarnings("unchecked")public Stack() {
    elementos = (E[]) novo Objeto[DEFAULT_INITIAL_CAPACITY];}
```

A segunda forma de eliminar o erro genérico de criação de array no Stack é trocar o tipo dos elementos do campo de E[] para Object[]. Se você fizer isso, vai receber um erro diferente:

```
Stack.java:19: tipos incompatíveis encontrados: Objeto, necessário: E
    E resultado = elementos[--tamanho];
                  ^
```

Você pode transformar esse erro em um aviso lançando o elemento recuperado do array para E, mas receberá um aviso:

```
Stack.java:19: aviso: [desmarcado] não marcado castfound: Objeto, necessário: E
    E resultado = (E) elementos[--tamanho];
                  ^
```

Como E é um tipo não reificável, não há como o compilador verificar o thecast em tempo de execução. Novamente, você pode facilmente provar para si mesmo que o conjuro não controlado é seguro, então é apropriado suprimir o aviso. De acordo com o conselho do Item 27, suprimimos o aviso apenas na missão que contém o elenco não controlado, não no método completo de pop:

```
Supressão apropriada de avisos públicos sem controle E pop() {
    if (tamanho == 0)
        lançar um novo EmptyStackException();
    push requer que elementos sejam do tipo E, então cast é
    correct@SuppressWarnings ("não marcado") E resultado =
        (E) elementos[--tamanho];
    elementos[tamanho] = nulo; Eliminar o resultado obsoleto de
    referência;}
```

Ambas as técnicas para eliminar a criação de arranjos genéricos têm seus aderentes. A primeira é mais legível: o array é declarado do tipo E[], indicando claramente que contém apenas E instâncias. Também é mais conciso: em uma classe genérica atípica, você lê do array em muitos pontos do código; A primeira técnica requer apenas um cast (onde o array é criado), enquanto a segunda requer um cast separado cada vez que um elemento do array é lido. Assim, a primeira técnica é preferível e mais comumente usada na prática. No entanto, isso causa poluição por heap (Item 32): o tipo de runtime do array não corresponde ao tipo de tempo de compilação (a menos que E seja Object). Isso deixa alguns programadores suficientemente enjoados a ponto de optarem pela segunda técnica, embora a poluição por heap seja inofensiva nessa situação.

O programa a seguir demonstra o uso da nossa classe genérica Stack. O programa imprime seus argumentos de linha de comando em ordem inversa e converte para maiúsculas. Não é necessário um cast explícito para invocar o método toUpperCase da String nos elementos que surgiram da pilha, e o cast gerado automaticamente tem garantia de sucesso:

```
Pequeno programa para exercitar nosso generico Stackpublic static void
main(String[] args) {
    <String> Pilha de pilha = nova pilha<>(); para (String arg : args)

        stack.push(arg); enquanto (!stack.isEmpty())

            System.out.println(stack.pop().toUpperCase());
}
```

O exemplo acima pode parecer contradizer o Item 28, que incentiva o uso de listas em vez de arrays. Nem sempre é possível ou desejável fazer listas de uso dentro dos seus tipos genéricos. Java não suporta listas nativamente, então alguns tipos genéricos, como `ArrayList`, precisam ser implementados sobre arrays. Outros tipos genéricos, como `HashMap`, são implementados sobre arrays para desempenho.

A grande maioria dos tipos genéricos é parecida com nosso exemplo de `Stack`, pois seus parâmetros de `type` não têm restrições: você pode criar uma `<Object>Stack`, `Stack<int[]>`, `Stack<List<String>>` ou `Stack` de qualquer outro tipo de referência de objeto. Note que você não pode criar uma Pilha de um tipo primitivo: tentar criar uma `Pilha<int>` ou uma `Pilha<double>` resultará em um erro em tempo de compilação. Essa é uma limitação fundamental do sistema genérico de tipos do Java. Você pode contornar essa restrição usando tipos primitivos em caixa (Item 61).

Existem alguns tipos genéricos que restringem os valores permitidos de seus parâmetros de tipo. Por exemplo, considere `java.util.concurrent.DelayQueue`, cuja declaração se parece com isto:

```
classe DelayQueue<E estende Delayed> implementa BlockingQueue<E>
```

A lista de parâmetros de tipo (`<E estende Delayed>`) exige que o parâmetro de tipo real `E` seja um subtipo de `java.util.concurrent.Delayed`. Isso permite que a implementação do `DelayQueue` e seus clientes aproveitem métodos `Delayed` nos elementos de uma `DelayQueue`, sem a necessidade de casting explícito ou do risco de uma `ClassCastException`. O parâmetro de tipo `E` é conhecido como parâmetro de tipo limitado. Note que a relação de subtipo é definida de modo que todo tipo é um subtipo de si mesmo [JLS, 4.10], então é legal criar uma `<Delayed>DelayQueue`.

Em resumo, tipos genéricos são mais seguros e fáceis de usar do que tipos que exigem casts no código do cliente. Ao projetar novos tipos, certifique-se de que eles possam ser usados sem esse tipo de fundição. Isso geralmente significa tornar os tipos genéricos. Se você tem tipos existentes que deveriam ser genéricos, mas não são, genericize-os. Isso facilitará a vida de novos usuários desses tipos sem quebrar clientes existentes (Item 26).

Item 30: Favoreça métodos genéricos

Assim como as classes podem ser genéricas, os métodos também podem ser. Métodos de utilidade estática que operam sobre tipos parametrizados geralmente são genéricos. Todos os métodos "algoritmos" nas Coleções (como `binarySearch` e ordenação) são genéricos.

Escrever métodos genéricos é semelhante a escrever tipos genéricos. Considere este método deficiente, que retorna a união de dois conjuntos:

```
Usa tipos brutos - inaceitável! (Item 26) união pública estática de
conjuntos(Set s1, Set s2) {
    Set result = novo HashSet(s1);
    result.addAll(s2); retorne
    resultado;}

```

Esse método compila, mas com dois avisos:

```
Union.java:5: aviso: [desmarcado] chamada desmarcada paraHashSet(Collection<?
estende E>) como membro do tipo bruto HashSet
    Set result = novo HashSet(s1);
                        ^
Union.java:6: aviso: [desmarcado] chamada desmarcada toaddAll(Collection<?
estende E>) como membro do tipo bruto Set
    result.addAll(s2);
                ^

```

Para corrigir esses avisos e tornar o método `typesafe`, modifique sua declaração para declarar um parâmetro de tipo representando o tipo de elemento para os três conjuntos (os dois argumentos e o valor de retorno) e use esse parâmetro de tipo em todo o método. A lista de parâmetros de tipo, que declara os parâmetros de tipo, alterna entre os modificadores de um método e seu tipo de retorno. Neste exemplo, a lista de parâmetros de tipo é `<E>`, e o tipo de retorno é `Set<E>`. As convenções de nomenclatura para parâmetros de tipo são as mesmas para métodos genéricos e tipos genéricos (Itens 29, 68):

```
Método genérico static <E> public Set<E> union(Set<E> s1, Set<E> s2) {
    Set<E> result = novo HashSet<>(s1);
    result.addAll(s2); retorne resultado;}

```

Pelo menos para métodos genéricos simples, é só isso. Esse método se acumula sem gerar avisos e oferece segurança de tipo, além de facilitar o

Use. Aqui está um programa simples para praticar o método. Este programa contém *nocasts* e compila sem erros ou avisos:

```
Programa simples para exercitar o método genérico static void main(String[] args) {

    <String> Conjunto = Conjunto de("Tom", "Dick", "Harry");
    Set<String> stooges = Set.de("Larry", "Moe", "Curly");
    Set<String> aflCio = sindicato(caras, patetas);
    System.out.println(aflCio);}
```

Quando você roda o programa, ele imprime [Moe, Tom, Harry, Larry, Curly, Dick]. (A ordem dos elementos na saída depende da implementação.)

Uma limitação do método de união é que os tipos dos três conjuntos (tanto os parâmetros de entrada quanto o valor de retorno) precisam ser exatamente os mesmos. Você pode tornar o método mais flexível usando tipos de coringa limitados (Item 31).

Ocasionalmente, você precisará criar um objeto que seja imutável, mas aplicável a muitos tipos diferentes. Como genéricos são implementados por apagamento (Item 28), você pode usar um único objeto para todas as parametrizações de tipo necessárias, mas é necessário escrever um método de fábrica estática para distribuir repetidamente o objeto para cada parametrização de tipo solicitado. Esse padrão, chamado de fábrica singleton genérica, é usado para objetos de função (Item 42) como `Collections.reverseOrder`, e ocasionalmente para coleções como `Collections.emptySet`.

Suponha que você queira escrever um dispensador de funções identidade. As bibliotecas fornecem o `Function.identity`, então não há motivo para escrever a sua própria (Item 59), mas é instrutivo. Seria um desperdício criar uma nova função identidade em tempo de objeto, porque ela é sem estado. Se os genéricos do Java fossem reificados, você precisaria de uma função identidade por tipo, mas como foram apagados, um `generic singleton` será suficiente. É assim que fica:

```
Padrão genérico singleton de fábrica, UnaryOperator estático<Object> privado
IDENTITY_FN = (t) -> t;

@SuppressWarnings("unchecked") publica <T> estática<T> UnaryOperator
identityFunction() {
    return (<T>UnaryOperator) IDENTITY_FN;}
```

O cast de `IDENTITY_FN` to `(<T>UnaryFunction)` gera um aviso `uncheckedcast`, já que `UnaryOperator<Object>` não é um `UnaryOperator<T>` para todo. Mas a função identidade é especial: ela retorna seu argumento sem modificação, então sabemos que é `typesafe` usá-la como uma `<T>UnaryFunction`, seja qual for o valor de `T`.

Portanto, podemos suprimir com confiança o aviso de elenco não controlado gerado por esse lançamento. Depois de fazer isso, o código se compila sem erro ou aviso.

Aqui está um programa de exemplo que usa nosso singleton genérico como `UnaryOperator<String>` e `UnaryOperator<Number>`. Como de costume, não contém casts nem compila sem erros ou avisos:

```
Programa de exemplo para exercitar singleton
public static void
main(String[] args) {String[] strings = { "jute", "hhab", "nylon" };

    UnaryOperator<String> sameString = identityFunction(); para (Strings s :
    strings)

        System.out.println(sameString.apply(s));

    Número[] números = { 1, 2, 0, 3L }; UnaryOperator<Number>
    sameNumber = identityFunction(); para (Número n : números)

        System.out.println(sameNumber.apply(n));
}
```

É permitido, embora relativamente raro, que um parâmetro de tipo seja limitado por alguma expressão envolvendo esse próprio parâmetro de tipo. Isso é o que se conhece como tipo ligado a caracteres recursivos. Um uso comum de limites de tipos recursivos é em conexão com a interface `Comparável`, que define a ordenação natural de um tipo (Item 14). Esta interface é mostrada aqui:

```
interface pública<T> Comparável {
    int compareTo(T o);}
```

O parâmetro de tipo `T` define o tipo com o qual elementos do tipo `<T>` implementando `Comparável` podem ser comparados. Na prática, quase todos os tipos podem ser comparados apenas com elementos de seu próprio tipo. Por exemplo, `String` implementa `Comparable<String>`, `Integer` implementa `<Integer>Comparable`, e assim por diante.

Muitos métodos usam uma coleção de elementos que implementam o `Similar` para ordená-lo, pesquisar nele, calcular seu mínimo ou máximo, e coisas do tipo. Para fazer essas coisas, é necessário que cada elemento da coleção seja comparável a todos os outros elementos nela, ou seja, que os elementos da lista sejam mutuamente comparáveis. Aqui está como expressar essa restrição:

```
Usando um tipo recursivo vinculado para expressar comparabilidade mútua
public
estática <E estende Comparável<E>> E max(Coleção<E> c);
```

O tipo limitado <E estende Comparável<E>> pode ser lido como "qualquer tipo E que possa ser comparado a si mesmo", o que corresponde mais ou menos precisamente à noção de comparabilidade mútua.

Aqui está um método para acompanhar a declaração anterior. Ele calcula o valor máximo em uma coleção de acordo com a ordem natural de seus elementos, e compila sem erros ou avisos:

```
Retorna o valor máximo em uma coleção - usa o tipo recursivo boundpublic static <E
estende Comparable<E>> E max(Collection<E> c) {
    if (c.isEmpty())
        lançar um novo IllegalArgumentException("Coleção vazia");

    E resultado = nulo; para (E e : c)

        if (resultado == null || e.compareTo(resultado) > 0)
            resultado = Objects.requireNonNull(e);

    retorne resultado;}
```

Note que esse método lança `IllegalArgumentException` se a lista estiver vazia. Uma alternativa melhor seria devolver um `Optional<E>` (Item 55).

Limites de tipos recursivos podem ficar muito mais complexos, mas felizmente raramente acontecem. Se você entender esse idioma, sua variante coringa (Item 31) e o idiom simulado-self-type (Item 2), você poderá lidar com a maioria dos limites de tipo recursivos que encontrar na prática.

Em resumo, métodos genéricos, assim como tipos genéricos, são mais seguros e fáceis de usar do que métodos que exigem que seus clientes coloquem casts explícitos nos parâmetros de entrada e valores de retorno. Assim como os tipos, você deve garantir que seus métodos possam ser usados sem conjurações, o que muitas vezes significa torná-los genéricos. E assim como os tipos, você deve gerar métodos existentes cujo uso exige lançamentos. Isso facilita a vida dos novos usuários sem quebrar clientes existentes (Item 26).

Item 31: Use curingas limitados para aumentar a flexibilidade da API

Como observado no Item 28, tipos parametrizados são invariantes. Em outras palavras, para quaisquer dois tipos distintos Tipo1 e Tipo2, Lista<Tipo1> não é nem subtipo nem supertipo de Lista<Tipo2>. Embora seja contraintuitivo que List<String> não seja um subtipo de List<Object>, ela realmente faz sentido. Você pode colocar qualquer objeto em uma Lista<Object>, mas só pode colocar strings em uma Lista<String>. Como aList<String> não pode fazer tudo o que uma List<Object> faz, ele não é um subtipo (pelo princípio da substituição de Liskov, Item 10).

Às vezes, você precisa de mais flexibilidade do que a tipagem invariante pode oferecer. Considere a classe Stack do Item 29. Para refrescar sua memória, aqui está a API pública dele:

```
classe pública Stack<E> {
    Stack(); público empurrar o
    vazio público (E e); E pop
    público(); boolean público
    isEmpty();}
```

Suponha que queiramos adicionar um método que pega uma sequência de elementos e os empurra todos para a pilha. Aqui está uma primeira tentativa:

```
método pushAll sem tipo de coringa - deficient!public void pushAll(<E>Iterable
src) {
    para (E e : src)
        empurrar(e);
}
```

Esse método compila de forma limpa, mas não é totalmente satisfatório. Se o tipo de elemento do src Iterable corresponder exatamente ao da pilha, funciona bem. Mas suponha que você tenha uma Stack<Number> e invoque push(intVal), onde intVal é do tipo Inteiro. Isso funciona porque Integer é um subtipo de Number. Então, logicamente, parece que isso também deveria funcionar:

```
<Number> Número da pilhaPilha = nova pilha<>(); Inteiros
iteráveis<Integer> = ... ; numberStack.pushAll(inteiros);
```

Se você tentar, porém, receberá esta mensagem de erro porque tipos parametrizados são invariantes:

```
StackTest.java:7: erro: tipos incompatíveis: Iterável<Integer>não pode ser
convertido em Iterável<Number>
    numberStack.pushAll(inteiros);
                        ^
```


Felizmente, há uma saída. A linguagem fornece um tipo especial de chamada de tipo parametrizado, um tipo curinga limitado, para lidar com situações como essa. O tipo do parâmetro de entrada para `pushAll` não deve ser "Iterável de E", mas sim "Iterável de algum subtipo de E", e existe um tipo curinga que significa exatamente isso: `Iterável<? estende E>`. (O uso da palavra-chave `estende` é um pouco enganoso: lembre-se do Item 29 que o subtipo é definido de modo que todo tipo é um subtipo de si mesmo, mesmo que não se estenda.) Vamos modificar o `pushAll` para usar este tipo:

```
Tipo curinga para um parâmetro que serve como um E
public void
pushAll(Iterável<? estende E> src) {
    para (E e : src)
        empurrar(e);
}
```

Com essa mudança, não só o Stack compila de forma limpa, como também o código do cliente que não compilaria com a declaração `pushAll` original. Como o Stack e seu cliente compilam de forma limpa, você sabe que tudo é `typesafe`.

Agora suponha que você queira escrever um método `popAll` para usar `pushAll`. O método `ThepopAll` retira cada elemento da pilha e adiciona os elementos à coleção dada. Veja como uma primeira tentativa de escrever o método `popAll` pode parecer:

```
método popAll sem tipo de curinga - deficient!
public void popAll(Collection<E> dst)
{
    while (!isEmpty())
        dst.add(pop());
}
```

Novamente, isso se compila de forma limpa e funciona bem se o tipo de elemento da `destination collection` corresponder exatamente ao da pilha. Mas, novamente, não é totalmente satisfatório. Suponha que você tenha uma pilha `<Number>` e variável do tipo `Object`. Se você tirar um elemento da pilha e armazenar na variável, ele compila e roda sem erro. Então, você não deveria conseguir fazer isso também?

```
<Number> Número da pilha
Pilha = nova <Number> pilha();
Objetos da coleção<Object> = ... ;
numberStack.popAll(objetos);
```

Se você tentar compilar esse código cliente contra a versão do `popAll` mostrada anteriormente, terá um erro muito parecido com o que tivemos com nossa primeira versão do `pushAll`: `Collection<Object>` não é um subtipo de `Collection<Number>`. Mais uma vez, os tipos curinga oferecem uma saída. O tipo do parâmetro de entrada para

popAll não deve ser "coleção de E", mas sim "coleção de algum supertipo de E" (onde o supertipo é definido de modo que E é um supertipo de si mesmo [JLS, 4.10]). Novamente, existe um tipo imprevisível que significa exatamente isso: Coleção<? super E>. Vamos modificar popAll para usá-lo:

```
Tipo de curinga para o parâmetro que serve como um E consumerpublic void
popAll(Collection<? super E> dst) {
    while (!isEmpty())
        dst.add(pop());
}
```

Com essa mudança, tanto o Stack quanto o código do cliente compilam de forma limpa. A lição é clara. Para máxima flexibilidade, use tipos de coringa nos parâmetros de entrada que representem produtores ou consumidores. Se um parâmetro de entrada for tanto produtor quanto consumidor, então tipos coringa não vão te ajudar: você precisa de uma correspondência exata de tipos, que é o que você obtém sem nenhum coringa.

Aqui está um mnemônico para ajudar você a lembrar qual tipo de coringa usar:

PECS significa produtor-estendidos, consumidor-super.

Em outras palavras, se um tipo parametrizado representa um produtor de T, use <? estende T>; se representar um consumidor T, use <? Super T>. No nosso exemplo da Pilha, o parâmetro pushAllsrc produz instâncias E para uso pela Pilha, então o tipo apropriado para src é Iterable<? estende E>; o parâmetro dst do popAll consome E instâncias da Pilha, então o tipo apropriado para dst é Collection<? super E>. O mnemônico ThePECS captura o princípio fundamental que guia o uso dos tipos coringa. Naftalin e Wadler chamam isso de Princípio de Comprar e Colocar [Naftalin07, 2.4].

Com esse mnemônico em mente, vamos dar uma olhada em alguns métodos e construtordeclarações dos itens anteriores deste capítulo. O construtor Chooser no Item 28 tem esta declaração:

```
Escolhidor público (<T>Escolhas de coleção)
```

Esse construtor usa as escolhas de coleção apenas para produzir valores do tipo T (e os armazena para uso posterior), então sua declaração deve usar um tipo curinga que estenda T. Aqui está a declaração resultante do construtor:

```
Tipo de curinga para o parâmetro que serve como um T producerpublic
Chooser(Collection<? estende T> escolhas)
```

E essa mudança faria alguma diferença na prática? Sim, seria. Suponha que você tenha uma Lista <Integer>e queira passá-la para o construtor

para um `<Number>` Escolhido. Isso não se compilaria com a declaração original, mas se compila assim que você adiciona o tipo de coringa limitado à declaração.

Agora vamos analisar o método de união do Item 30. Aqui está a declaração:

```
<E> União pública estática<E> de Set (Set<E> s1, Set<E> s2)
```

Ambos os parâmetros, `s1` e `s2`, são produtores de `E`, então o mnemônico do PECS nos diz que a declaração deve ser a seguinte:

```
<E> união estática pública<E> de set(Set<? estende E> s1,  
                                     Set<? estende E> s2)
```

Note que o tipo de retorno ainda está `Set<E>`. Não use tipos de coringa limitados como tipos de retorno. Em vez de oferecer flexibilidade adicional para seus usuários, isso os forçaria a usar tipos coringa no código do cliente. Com a declaração revisada, este código será compilado de forma limpa:

```
<Integer> Conjuntos inteiros = Conjunto.de(1, 3, 5); Pares<Double>de set=  
Set.de(2.0, 4.0, 6.0); <Number>Números de conjunto = união (inteiros, duplos);
```

Usados corretamente, os tipos coringa são quase invisíveis para os usuários de uma classe. Eles fazem com que métodos aceitem os parâmetros que deveriam aceitar e rejeitem aqueles que deveriam rejeitar. Se o usuário de uma classe precisa pensar em tipos coringa, provavelmente há algo errado com a API dela.

Antes do Java 8, as regras de inferência de tipos não eram inteligentes o suficiente para lidar com o fragmento de código anterior, que exige que o compilador use o tipo de retorno (ou tipo alvo) especificado contextualmente para inferir o tipo de `E`. O tipo alvo da invocação unional mostrada anteriormente é `Set<Number>`. Se você tentar compilar o fragmento em uma versão anterior do Java (com um substituto apropriado para a fábrica `Set.of`), você receberá uma mensagem de erro longa e complicada como esta:

```
Union.java:14: erro: tipos incompatíveis  
    <Number> Números de conjunto = união (inteiros, duplos);  
                                     ^  
required: Set<Number>found:Set<INT#1>onde  
INT#1,INT#2 são tipos de interseção:  
  
    INT#1 estende Número, Comparável<? estende INT#2>INT#2 estende Número,  
    Comparável<?>
```

Felizmente, existe uma forma de lidar com esse tipo de erro. Se o compilador não tiver o tipo correto, você sempre pode dizer qual tipo usar com um tipo explícito

argumento [JLS, 15.12]. Mesmo antes da introdução do tipo alvo no Java 8, isso não era algo que você precisasse fazer com frequência, o que é bom porque argumentos explícitos de tipo não são nada bonitos. Com a adição de um argumento explícito de tipo, como mostrado aqui, o fragmento de código compila limpamente em versões anteriores ao Java 8:

```
Parâmetro de tipo explícito - necessário antes do Java 8
Set<Number> numbers =
    Union.<Number>.uniao (inteiros, duplos);
```

Agora, vamos voltar nossa atenção para o método máximo no Item 30. Aqui está a declaração original:

```
estática pública <T estende Comparável<T>> T max(<T>Lista de listas)
```

Aqui está uma declaração revisada que usa tipos de curinga:

```
estática pública <T estende Comparável<? super T>> T max(
    Lista<? estende T> lista)
```

Para obter a declaração revisada do original, aplicamos o heuris-tic do PECS duas vezes. A aplicação direta é na lista de parâmetros. Ele produz `T instances`, então mudamos o tipo de `List<T>` para `List<? estende T>`. A aplicação complicada é para o parâmetro de tipo `T`. Esta é a primeira vez que vemos um coringa aplicado a um parâmetro de tipo. Originalmente, `T` foi especificado para estender `Comparável<T>`, mas um comparável de `T` consome `T` instâncias (e produz inteiros indicando relações de ordem). Portanto, o tipo parametrizado `<T> Comparável` é substituído pelo tipo curinga limitado `Comparável<? super T>`. Comparáveis são sempre consumidores, então você geralmente deve usar `Comparável<? super T>` em vez de `Comparável<T>`. O mesmo vale para comparadores; portanto, geralmente você deve usar `Comparator<? super T>` em vez de `Comparator<T>`.

A declaração máxima revisada é provavelmente a declaração de método mais complexa deste livro. A complexidade extra realmente te traz alguma coisa? De novo, sim. Aqui está um exemplo simples de uma lista que seria excluída pela declaração original, mas que é permitida pela versão revisada:

```
Lista<FuturoDeProgramado<?>>>FuturosAgendados = ... ;
```

A razão pela qual você não pode aplicar a declaração original do método a essa lista é que o `ScheduledFuture` não implementa o `Comparable<ScheduledFuture>`. Em vez disso, é uma subinterface do `Delayed`, que estende o `<Delayed>Comparable`. Em outras palavras, uma instância `ScheduledFuture` não é apenas comparável a outras

Instâncias `ScheduledFuture`; é comparável a qualquer instância `Delayed`, e isso é suficiente para fazer a declaração original rejeitá-la. De forma mais geral, o coringa é obrigado a suportar tipos que não implementam `Comparável` (ou `Comparador`) diretamente, mas estendem um tipo que implementa.

Há mais um tema relacionado à incógnita que merece ser discutido. Existe adualidade entre parâmetros de tipo e curingas, e muitos métodos podem ser declarados usando um ou outro. Por exemplo, aqui estão duas possíveis declarações para um método estático trocar dois itens indexados em uma lista. O primeiro usa o parâmetro de tipo ilimitado (Item 30) e o segundo um coringa ilimitado:

```
Duas possíveis declarações para o método swap estático <E> público (<E>List
list, int i, int j); swap estático público (List<?> list, int i, int j);
```

Qual dessas duas declarações é preferível e por quê? Em uma API pública, o segundo é melhor porque é mais simples. Você passa uma lista — qualquer lista — e o método troca os elementos indexados. Não há um parâmetro de tipo para se preocupar. Como regra, se um parâmetro de tipo aparecer apenas uma vez em uma declaração de método, substitua-o por um coringa. Se for um parâmetro de tipo ilimitado, substitua-o por um coringa ilimitado; Se for um parâmetro de tipo limitado, substitua-o por um coringa limitado.

Há um problema com a segunda declaração de troca. A implementação straightforward não compila:

```
public static void swap(List<?> list, int i, int j) {
    list.set(i, list.set(j, list.get(i)));}
```

Tentar compilar isso gera esta mensagem de erro pouco útil:

```
Swap.java:5: erro: tipos incompatíveis: Objeto não pode ser convertido para CAP#1
    list.set(i, list.set(j, list.get(i)));
                        ^
    onde CAP#1 é uma variável de tipo nova:
    CAP#1 estende Objeto a partir da captura de ?
```

Não parece certo não podermos colocar um elemento de volta na lista da qual acabamos de tirá-lo. O problema é que o tipo de lista é `Lista<?>`, e você não pode colocar nenhum valor além de nulo em uma `Lista<?>`. Felizmente, existe uma forma de implementar esse método sem recorrer a um lançamento inseguro ou um tipo bruto. A ideia é escrever um

método privado de ajuda para capturar o tipo coringa. O método helper deve ser um método genérico para capturar o tipo. É assim que fica:

```
public static void swap(List<?> list, int i, int j) {
    swapHelper(list, i, j);}

Private helper method for wildcard captureprivate <E> static void
swapHelper(List<E> list, int i, int j) {
    List.set(I, List.Set(J, List.Get(I)));}
```

O método swapHelper sabe que essa lista é uma Lista<E>. Portanto, ele sabe que qualquer valor que recebe dessa lista é do tipo E e que é seguro colocar qualquer valor do tipo E na lista. Essa implementação um pouco complicada do swap compila de forma limpa. Isso nos permite exportar a declaração baseada em wildcard, aproveitando internamente o método genérico mais complexo. Os clientes do método swap não precisam enfrentar a declaração mais complexa do swapHelper, mas eles se beneficiam dela. Vale notar que o método helper tem exatamente a assinatura que descartamos por ser complexa demais para o método público.

Em resumo, usar tipos curinga nas APIs, embora complicado, torna as APIs muito mais flexíveis. Se você escreve uma biblioteca que será amplamente utilizada, o uso correto dos tipos de wildcard deve ser considerado obrigatório. Lembre-se da regra básica: produtor-estende, consumidor-super (PECS). Também lembre-se de que todos os comparáveis e comparadores são consumidores.

Item 32: Combine genéricos e variações com critério

Os métodos do Vararg (Item 53) e genéricos foram adicionados à plataforma no Java 5, então você pode esperar que eles interajam de forma elegante; Infelizmente, não têm. O propósito dos varargs é permitir que os clientes passem um número variável de argumentos para um método, mas é uma abstração vazante: quando você invoca um método varargs, um array é criado para armazenar os parâmetros varargs; Esse array, que deveria ser um detalhe de implementação, é visível. Como consequência, você recebe avisos confusos do compilador quando varargsparameters têm tipos genéricos ou parametrizados.

Lembre-se do Item 28 que um tipo não reficável é aquele cuja representação em tempo de execução tem menos informações do que sua representação em tempo de compilação, e que quase todos os tipos genéricos e parametrizados são não reficáveis. Se um método declarar seu varargparâmetro como de tipo não reficável, o compilador gera um aviso na declaração. Se o método for invocado em parâmetros varargs cujo tipo inferido é não reficável, o compilador também gera um aviso sobre a invocação. Os avisos são mais ou menos assim:

```
aviso: [não verificado] Possível poluição por pilha de
      Lista <String>de vararg parametrizado
```

A poluição por heap ocorre quando uma variável de tipo parametrizado se refere a um objeto que não é desse tipo [JLS, 4.12.2]. Isso pode causar a falha dos casts gerados automaticamente do compilador, violando a garantia fundamental do sistema de tipos genérico.

Por exemplo, considere este método, que é uma variante disfarçada do fragmento do código na página 127:

```
Misturar genéricos e varargs pode violar a segurança do
tipo!static void dangerous (Lista<String>... stringLists) {
    List<Integer> intList = List.of(42); Object[] objetos =
    stringLists; objects[0] = intList;// Heap pollutionString s =
    stringLists[0].get(0); ClassCastException}
```

Esse método não possui casts visíveis, mas gera uma ClassCastException quando invocado com um ou mais argumentos. Sua última linha tem um cast invisível gerado pelo compilador. Esse cast falha, demonstrando que a segurança do tipo foi comprometida, e é inseguro armazenar um valor em um parâmetro genérico de varargs array.

Esse exemplo levanta uma questão interessante: Por que é legal declarar um método com um parâmetro varargs genérico, quando é ilegal criar explicitamente um genericarray? Em outras palavras, por que o método mostrado anteriormente gera apenas um aviso, enquanto o fragmento de código na página 127 gera um erro? O

A resposta é que métodos com parâmetros Varargs de tipos genéricos ou parametrizados podem ser muito úteis na prática, então os designers da linguagem optaram por conviver com essa inconsistência. Na verdade, as bibliotecas Java exportam vários desses métodos, incluindo `Arrays.asList(T... a)`, `Collections.addAll(Collection<? super T> c, T... elements)` e `EnumSet.of(E primeiro, E... resto)`. Ao contrário do método perigoso mostrado anteriormente, esses métodos de biblioteca são `typesafe`.

Antes do Java 7, não havia nada que o autor de um método com um parâmetro `genericvarargs` pudesse fazer sobre os avisos nos locais de chamada. Isso tornava essas APIs desagradáveis de usar. Os usuários precisavam aguentar os avisos ou, preferencialmente, eliminá-los com anotações `@SuppressWarnings` ("não marcadas") no site do Everycall (Item 27). Isso era tedioso, prejudicava a legibilidade e escondia avisos que denunciavam problemas reais.

No Java 7, a anotação `SafeVarargs` foi adicionada à plataforma, para permitir que o autor de um método com um parâmetro `varargs` genérico suprimisse automaticamente os avisos dos clientes. Em essência, a anotação `SafeVarargs` constitui uma promessa do autor de um método de que ela é `typesafe`. Em troca dessa promessa, o `com-piler` concorda em não avisar os usuários sobre o método de que chamadas podem ser inseguras.

É fundamental que você não anote um método com `@SafeVarargs` a menos que ele seja realmente seguro. Então, o que é preciso para garantir isso? Lembre-se de que um array genérico é criado quando o método é invocado, para manter os parâmetros `varargs`. Se o método não armazena nada no array (o que sobrescreveria os parâmetros) e não permite que uma referência ao array escape (o que permitiria que `untrustedcode` acesse o array), então está seguro. Em outras palavras, se o `varargs parameterarray` for usado apenas para transmitir um número variável de argumentos do chamador para o método — que é, afinal, o propósito dos `varargs` — então o método está seguro.

Vale notar que você pode violar a segurança de tipos sem nunca armazenar nada no array de parâmetros `varargs`. Considere o seguinte `varargsmethod` genérico, que retorna um array contendo seus parâmetros. À primeira vista, pode parecer uma utilidade útil:

```
UNSAFE - Expõe uma referência ao seu array de parâmetros genérico!  
<T>static T[] toArray(T... args) {  
    retorne args;}  
}
```

Esse método simplesmente retorna seu array de parâmetros `varargs`. O método pode não parecer perigoso, mas é! O tipo desse array é determinado pelos tipos em tempo de compilação dos argumentos passados para o método, e o compilador pode não ter informações suficientes para fazer uma determinação precisa. Como esse método retorna seu array de parâmetros `varargs`, ele pode propagar poluição do heap pela pilha de chamadas.

Para tornar isso concreto, considere o seguinte método genérico, que toma três argumentos do tipo `T` e retorna um array contendo dois dos argumentos, escolhidos aleatoriamente:

```
estático <T> T[] púlticaTwo(T a, T b, T c) {
    switch(ThreadLocalRandom.current().nextInt(3)) {
    caso 0: retorno ao Array(a, b); caso 1: retorno para
Array(a, c); caso 2: retornar paraArray(b, c);}lançar
um novo AssertionError(); Não consigo chegar aqui}
```

Esse método não é, por si só, perigoso e não geraria um aviso exceto que invoca o método `toArray`, que possui um parâmetro `varargs` genérico.

Ao compilar esse método, o compilador gera código para criar um array `varargs` parameter no qual passar duas instâncias `T` para o `toArray`. Esse código aloca array do tipo `Object[]`, que é o tipo mais específico que garante manter essas instâncias, independentemente dos tipos de objetos que sejam passados para o `pickTwo` no local da chamada. O método `toArray` simplesmente retorna esse array para `pickTwo`, que por sua vez o retorna ao seu chamador, então `pickTwo` sempre retornará um array do tipo `Object[]`.

Agora considere este método principal, que exerce o `pickTwo`:

```
public static void main(String[] args) {
    Atributos String[] = pickTwo("Bom", "Rápido", "Barato");}
```

Não há nada de errado com esse método, então ele compila sem gerar nenhum aviso. Mas quando você executa, ele gera uma `ClassCastException`, embora não contenha casts visíveis. O que você não vê é que o compilador gerou um cast oculto para `String[]` no valor retornado pelo `pickTwo` para que ele possa ser armazenado em atributos. O cast falha, porque `Object[]` não é um subtipo de `String[]`. Essa falha é bastante preocupante porque está dois níveis de distância do método que realmente causa a poluição do heap (`toArray`), e o array de parâmetros `varargs` não é modificado após os parâmetros reais serem armazenados nele.

Este exemplo pretende reforçar que não é seguro dar a outro método acesso a um array genérico de parâmetros `varargs`, com duas exceções: é seguro passar o array para outro método `varargs` que esteja corretamente anotado com `@SafeVarargs`, e é seguro passar o array para um método não-`varargs` que apenas calcula alguma função do conteúdo do array.

Aqui está um exemplo típico de uso seguro de um parâmetro varargs genérico. Esse método toma um número arbitrário de listas como argumentos e retorna uma única lista contendo os elementos de todas as listas de entrada em sequência. Como o método é anotado com `@SafeVarargs`, ele não gera nenhum aviso, nem na declaração nem em seus locais de chamada:

```
Método seguro com varargs genéricos
parameter@SafeVarargs static <T> List<T> flatten(List<?
estende T>... listas) {
    <T> Resultado da lista = novo ArrayList<>
    (); para (lista<? estende T> lista: listas)
    result.addAll(list); retorne
    resultado;}

```

A regra para decidir quando usar a anotação `SafeVarargs` é simples: Use `@SafeVarargs` em cada método com um parâmetro varargs de tipo genérico ou parametrizado, para que seus usuários não fiquem sobrecarregados com avisos desnecessários e confusos do compilador. Isso significa que você nunca deve escrever metanfetaminas perigosas ou 'toArray'. Toda vez que o compilador avisar sobre possível poluição de heap de um parâmetro genérico varargs em um método que você controla, verifique se o método é seguro. Como lembrete, um método varargs genérico é seguro se:

1. ele não armazena nada no array de parâmetros Varargs, e
2. Isso não torna o array (ou um clone) visível para código não confiável. Se alguma dessas proibições for violada, corrija. Note que a anotação `SafeVarargs` é legal apenas em métodos que não podem ser sobrepostos, pois é impossível garantir que todos os métodos de sobreposição possíveis serão seguros. No Java 8, a anotação era legal apenas em métodos estáticos e de instância final; no Java 9, tornou-se legal também em métodos de instância privada. Uma alternativa ao uso da anotação `SafeVarargs` é seguir o conselho do Item 28 e substituir o parâmetro varargs (que é um array disfarçado) pelo parâmetro `aList`. Veja como essa abordagem fica quando aplicada ao nosso método `flatten`. Note que apenas a declaração de parâmetros mudou:

```
List como alternativa typesafe a um parametrizado genérico <T>
varargs List<T> flatten(List<List<? estende T>> lists) {
    <T> Resultado da lista = novo ArrayList<>
    (); para (lista<? estende T> lista: listas)
    result.addAll(list); retorne
    resultado;}

```

Esse método pode então ser usado em conjunto com o método de fábrica estático `List.of` para permitir um número variável de argumentos. Note que essa abordagem se baseia no fato de que a declaração da `Lista` de está anotada com `@SafeVarargs`:

```
audiência = achatar (Lista.de(amigos, romanos, compatriotas));
```

A vantagem dessa abordagem é que o compilador pode provar que o método é `typesafe`. Você não precisa garantir a segurança dela com uma anotação do `SafeVarargs`, e não precisa se preocupar em ter errado ao determinar que era seguro. A principal desvantagem é que o código do cliente é um pouco mais prolixo e talvez um pouco mais lento.

Esse truque também pode ser usado em situações onde é impossível escrever um método `safevarargs`, como ocorre com o método `toArray` na página 147. Seu análogo `List` é o método `List.of`, então nem precisamos escrevê-lo; os autores das bibliotecas Java fizeram o trabalho por nós. O método `pickTwo` então se torna este:

```
<T> Lista<T> estática PickTwo(T a, T b, T c) {
    switch(rnd.nextInt(3)) {
        caso 0: devolver List.of(a, b); caso
        1: devolver List.of(a, c); caso 2:
        devolver List.of(b, c);}lançar um novo
        AssertionError();}
```

e o método principal passa a ser este:

```
public static void main(String[] args) {
    <String> Atributos da lista = pickTwo("Bom", "Rápido", "Barato");}
```

O código resultante é `typesafe` porque usa apenas genéricos, e não arrays.

Em resumo, `varargs` e genéricos não interagem bem porque o `varargsfacility` é uma abstração vazante construída sobre arrays, e arrays têm regras de tipo diferentes das genéricas. Embora parâmetros genéricos de `varargs` não sejam `typesafe`, eles são legais. Se você optar por escrever um método com um parâmetro `varargs` genérico (ou parametrizado), primeiro certifique-se de que o método seja `typesafe` e depois anote-o com `@SafeVarargs` para que não seja desagradável de usar.

Item 33: Considere recipientes heterogêneos typesafe

Usos comuns dos genéricos incluem coleções, como `Set<E>` e `Map<K,V>`, e contêineres de elemento único, como `ThreadLocal<T>` e `AtomicReference<T>`. Em todos esses usos, é o recipiente que é parametrizado. Isso limita você a um número fixo de parâmetros de tipo por contêiner. Normalmente é exatamente isso que você quer. Um conjunto possui um único parâmetro de tipo, representando seu tipo de elemento; um Mapa tem dois, representando seus tipos de chave e valor; e assim por diante.

Às vezes, porém, você precisa de mais flexibilidade. Por exemplo, uma linha de banco de dados pode ter um número arbitrário de colunas, e seria bom poder acessar todas elas de forma typesafe. Felizmente, existe uma maneira fácil de conseguir esse efeito. A ideia é parametrizar a chave em vez do container. Depois, apresente a chave parametrizada ao contêiner para inserir ou recuperar um valor. O sistema genérico de tipos é usado para garantir que o tipo do valor concorde com sua chave.

Como exemplo simples dessa abordagem, considere uma classe `Favoritos` que permite aos clientes armazenar e recuperar uma instância favorita com um número arbitrário de tipos. O objeto `Class` para o tipo desempenhará a parte da chave parametrizada. O motivo disso funcionar é que `Class` é genérica. O tipo de classe literal não é simplesmente `Classe`, mas `<T>Classe`. Por exemplo, `String.class` é do tipo `Class<String>`, e `Integer.class` é do tipo `Class<Integer>`. Quando um literal de classe é passado entre métodos para comunicar tanto informações de tipo em tempo de compilação quanto em tempo de execução, ele é chamado de token de tipo [Bracha04].

A API da classe `Favoritos` é simples. Parece apenas um mapa simples, exceto que a chave é parametrizada em vez do mapa. O cliente apresenta um objeto `Class` ao definir e obter favoritos. Aqui está a API:

```
Typesafe padrão de contêineres heterogêneos -
API public class Favoritos {
    empty público <T> putFavorito (<T>tipo de classe, instância T);
    public <T> T getFavorite (<T>tipo de classe);}
```

Aqui está um programa de exemplo que exercita a classe `Favoritos`, armazenando, recuperando e imprimindo uma instância favorita de `String`, `Integer` e `Class`:

```
Typesafe padrão de contêiner heterogêneo - cliente public
static void main(String[] args) {
    Favoritos f = novos Favoritos();
    f.putFavorite(String.class, "Java");
    f.putFavorite(Integer.class, 0xcafebabe);
    f.putFavorito(Class.class, Favoritos.class);}
```

```

String favoritoString = f.getFavorite(String.class); int
favoriteInteger = f.getFavorite(Integer.class); Class<?>
favoriteClass = f.getFavorite(Class.class);
System.out.printf("%s %x %s\n", favoritoString,
    FavoritoInteiro, favoritoClasse.NomeFavore());
}

```

Como era de se esperar, este programa publica Java *cafebabe* Favoritos. Note, incidentalmente, que o método `printf` do Java difere do do C no sentido de que você deve usar `%n` onde usaria `\n` em C. O `%n` gera o separador de linha específico da plataforma aplicável, que é `\n` em muitas, mas não em todas.

Uma instância de Favoritos é *typesafe*: ela nunca retornará um `Integer` quando você pedir uma `String`. Também é heterogênea: ao contrário de um mapa comum, todas as chaves são de tipos diferentes. Por isso, chamamos o Favoritos de um contêiner heterogêneo tipo seguro.

A implementação dos Favoritos é surpreendentemente pequena. Aqui está, em sua totalidade:

```

Typesafe padrão de contêiner heterogêneo - classe
implementationpublic Favorites {
    Map<Class<?>, Object> favoritos = novo HashMap<>();

    public <T> void putFavorite(<T>Tipo de classe, instância T) {
        favoritos.put(Objects.requireNonNull(type), instance);}

    public <T> T getFavorite(<T>Tipo de classe) {
        return type.cast(favoritos.get(type));}}

```

Há algumas coisas sutis acontecendo aqui. Cada instância de Favoritos é respaldada por um `Objecto` privado de `Mapa<Classe<?>>` chamado `favoritos`. Você pode pensar que não conseguiria colocar nada neste mapa por causa do tipo coringa ilimitado, mas a verdade é exatamente o oposto. O que se deve notar é que o tipo coringa é aninhado: não é o tipo do mapa que é do tipo coringa, mas sim o tipo de sua chave. Isso significa que cada chave pode ter um tipo parametrizado diferente: uma pode ser `Class<String>`, a próxima `Class<Integer>`, e assim por diante. É daí que vem a heterogeneidade.

A próxima coisa a notar é que o tipo de valor do Mapa dos favoritos é simplesmente `Objecto`. Em outras palavras, o Mapa não garante a relação de tipo entre chaves e valores, que é que todo valor é do tipo representado por sua chave. Em

de fato, o sistema de tipos do Java não é poderoso o suficiente para expressar isso. Mas sabemos que é verdade, e aproveitamos isso quando chega a hora de recuperar um favorito.

A implementação do `putFavorite` é trivial: ela simplesmente coloca os favoritos fazendo amapping do objeto `Class` para a instância favorita dada. Como mencionado, isso descarta a "ligação de tipos" entre a chave e o valor; ela perde o conhecimento de que o valor é uma instância da chave. Mas tudo bem, porque o método `getFavorites` pode e de fato restabelecer essa ligação.

A implementação do `getFavorite` é mais complicada do que a do `putFavorite`. Primeiro, ele obtém do mapa favoritos o valor correspondente ao determinado `Class` object. Esta é a referência correta ao objeto a ser retornada, mas tem o tipo de tempo de compilação errado: é `Object` (o tipo de valor do mapa de favoritos) e precisamos devolver um `T`. Assim, a implementação `getFavorite` projeta dinamicamente a referência do objeto para o tipo representado pelo objeto `Class`, usando o método `cast` do `Class`.

O método de `cast` é o análogo dinâmico do operador `cast` do Java. Ele simplesmente verifica que seu argumento seja uma instância do tipo representado pelo objeto `Classe`. Se sim, ele retorna o argumento; caso contrário, ele gera uma `ClassCastException`. Sabemos que a invocação `cast` no `getFavorite` não lança `ClassCastException`, assumindo que o código do cliente foi compilado limpo. Ou seja, sabemos que os valores no mapa de favoritos sempre correspondem aos tipos de suas chaves.

Então, o que o método de `elenco` faz por nós, dado que ele simplesmente retorna seu argumento? A assinatura do método `cast` aproveita ao máximo o fato de que `class` é genérico. Seu tipo de retorno é o parâmetro de tipo do objeto `Classe`:

```
classe publica<T> Classe {
    T cast(Objeto obj);}
```

É exatamente isso que o método `getFavorite` exige. É o que nos permite tornar os Favoritos tipo seguro sem recorrer a um `cast` não controlado para `T`.

Existem duas limitações para a classe `Favoritos` que valem a pena notar. Primeiro, um cliente malicioso pode corromper facilmente a segurança de tipos de uma instância de `Favoritos`, usando um objeto `Class` em sua forma bruta. Mas o código do cliente resultante gerava um aviso desverificado quando era compilado. Isso não é diferente de implementações de `normalcollection`, como `HashSet` e `HashMap`. Você pode facilmente colocar uma string em um `HashSet<Integer>` usando o tipo bruto `HashSet` (Item 26). Dito isso, você pode ter segurança tipo runtime se estiver disposto a pagar por isso. A forma de garantir que `Favoritos` nunca viole seu tipo invariante é ter o `putFavorite`

Verificação de método que a instância é na verdade uma instância do tipo representado pelo tipo, e já sabemos como fazer isso. Basta usar um elenco dinâmico:

```
Alcançando segurança do tipo runtime com um cast dinâmico <T>  
void público putFavorite (<T>Tipo de classe, instância T) {  
    favoritos.put(tipo, tipo.cast(instância));}
```

Existem wrappers de coleção em `java.util.Collections` que fazem o mesmo truque. Eles são chamados de `checkedSet`, `checkedList`, `checkedMap`, e assim por diante. Suas fábricas estáticas recebem um objeto de Classe (ou dois) além de uma coleção (ou mapa). As fábricas estáticas são métodos genéricos, garantindo que os tipos de compilação do objeto Classe e da coleção coincidam. As embalagens adicionam reificação às coleções que envolvem. Por exemplo, o invólucro lança uma `ClassCastException` em tempo de execução se alguém tentar colocar uma moeda na sua `<Stamp>Coleção`. Esses wrappers são úteis para rastrear código cliente que adiciona um elemento digitado incorretamente a uma coleção, em uma aplicação que mistura tipos genéricos e brutos.

A segunda limitação da classe `Favoritos` é que ela não pode ser usada em um tipo não reficível (Item 28). Em outras palavras, você pode armazenar sua `String` ou `String favorita[]`, mas não sua `Lista favorita<String>`. Se você tentar armazenar sua `Lista de <String>favoritos`, seu programa não vai compilar. O motivo é que você não consegue obter um objeto `Class` para `List<String>`. O literal da classe `<String>List.class` é um erro de sintaxe, e isso também é bom. `List<String>` e `List<Integer>` compartilham um único objeto `Class`, que é `List.class`. Isso causava um caos nas internas de um objeto `Favoritos` se os "literais de tipo" `<String>List.class` e `<Integer>List.class` fossem legais e retornassem a mesma referência ao objeto. Não existe uma solução totalmente satisfatória para essa limitação.

Os tokens de tipo usados pelos `Favoritos` são ilimitados: `getFavorite` e `put-Favorite` aceitam qualquer objeto Classe. Às vezes, você pode precisar limitar os tipos que podem ser passados para um método. Isso pode ser alcançado com um token de tipo limitado, que é simplesmente um token de tipo que impõe um limite sobre qual tipo pode ser representado, usando um parâmetro de tipo limitado (Item 30) ou um coringa limitado (Item 31).

A API de anotações (Item 39) faz uso extensivo de tokens de tipo limitado. Por exemplo, aqui está o método para ler uma anotação em tempo de execução. Esse método vem da interface `AnnotatedElement`, que é implementada pelos tipos reflexivos que representam classes, métodos, campos e outros elementos do programa:

```
public <T estende a Anotação>  
    T getAnnotation(<T>Tipo de Anotação de Classe);
```

O argumento, `annotationType`, é um token de tipo limitado que representa um tipo de anotação. O método retorna a anotação desse tipo do elemento, se tiver uma, ou nulo, se não tiver. Em essência, um elemento anotado é um contêiner heterogêneo `typesafe` cujas chaves são tipos de anotação.

Suponha que você tenha um objeto do tipo `Class<?>` e queira passá-lo para um método que exija um token de tipo limitado, como `getAnnotation`. Você poderia fazer o cast do objeto para `Class<?>` estende `Anotação`, mas esse cast está desmarcado, então geraria um aviso em tempo de compilação (Item 27). Felizmente, a classe `Class` fornece um método de instância que executa esse tipo de cast de forma segura (e dinâmica). O método é chamado de `asSubclass`, e ele projeta o objeto `Classe` sobre o qual é chamado para representar uma subclasse da classe representada por seu argumento. Se o lançamento for bem-sucedido, o método retorna seu argumento; se falhar, ele gera uma `ClassCastException`.

Veja como você usa o método `asSubclass` para ler uma anotação cujo tipo é desconhecido na hora da compilação. Este método compila sem erro ou aviso:

```

    Uso da asSubclass para castar com segurança para um tipo limitado
    tokenstatic Annotation getAnnotation(AnnotatedElement element,
                                         String annotationTypeName) {
        Class<?> annotationType = null; // Unbounded type tokenentry {

        annotationType = Classe.paraNome(NomeDeTipoAnotação);} catch
        (Exceção ex) {
            lançar um novo
            IllegalArgumentException(ex);}return
            element.getAnnotation(
                annotationType.asSubclass(Annotation.class));
    }

```

Em resumo, o uso normal de genéricos, exemplificado pelas APIs de coleções, restringe você a um número fixo de parâmetros de tipo por container. Você pode contornar essa restrição colocando o parâmetro `type` na chave em vez do container. Você pode usar objetos `Class` como chaves para esses containers heterogêneos `typesafe`. Um objeto de classe usado dessa forma é chamado de token tipo. Você também pode usar um tipo de chave personalizado. Por exemplo, você poderia ter um tipo `RowDatabase` representando uma linha de banco de dados (o container) e um tipo genérico `Coluna<T>` como sua chave.

Esta página intencionalmente deixada em branco

CHAPTER 6

Enums e Anotações

O JAVA suporta duas famílias de tipos de referência de propósito especial: um tipo de classe chamado de tipo enum, e de uma espécie de interface chamada tipo de anotação. Este capítulo discute as melhores práticas para o uso dessas famílias de tipos.

Item 34: Use enums em vez de constantes de inteligência

Um tipo enumerado é um tipo cujos valores legais consistem em um conjunto fixo de constantes, como as estações do ano, os planetas do sistema solar ou os naipes em um baralho de cartas. Antes da adição dos tipos enum à linguagem, um padrão comum para representar tipos enumerados era declarar um grupo de constantes int nomeadas, uma para cada membro do tipo:

```
O padrão int enum - severamente deficiente!
estática
pública final int APPLE_FUJI= 0; Int final estática
pública APPLE_PIPPIN= 1; Int final estática pública
APPLE_GRANNY_SMITH = 2;

Estática pública ORANGE_NAVEL final de INT    = 0;
Int final estático público ORANGE_TEMPLE = 1;
Int final estático público ORANGE_BLOOD= 2;
```

Essa técnica, conhecida como padrão int enum, apresenta muitas limitações. Não oferece nenhuma segurança de tipo e pouco poder expressivo. O compilador não reclama se você passar uma maçã para um método que espera laranja, comparar maçãs com laranjas com o operador ==, ou pior:

```
Saboroso purê de maçã com sabor cítrico! int i =
(APPLE_FUJI - ORANGE_TEMPLE) / APPLE_PIPPIN;
```

Note que o nome de cada constante de maçã é precedido por APPLE_ e o nome de cada constante laranja é prefixado por ORANGE_. Isso porque o Java

Não fornece namespaces para grupos ENUM INT. Prefixos evitam conflitos de nomes quando dois grupos int enum possuem constantes nomeadas idênticas, por exemplo, entre ELEMENT_MERCURY e PLANET_MERCURY.

Programas que usam enums de inteligência são frágeis. Como os enums int são constantes [JLS, 4.12.4], seus valores int são compilados nos clientes que os utilizam [JLS, 13.1]. Se o valor associado a um enum int for alterado, seus clientes devem ser recompilados. Se não, os clientes ainda vão fugir, mas o comportamento deles estará incorreto.

Não existe uma maneira fácil de traduzir constantes int enum em strings imprimíveis. Se você imprimir uma constante assim ou exibi-la de um depurador, tudo o que verá é um número, o que não ajuda muito. Não existe uma maneira confiável de iterar sobre todas as constantes int enums em um grupo, ou mesmo de obter o tamanho de um grupo int enum.

Você pode encontrar uma variante desse padrão em que constantes de string são usadas no lugar das constantes int. Essa variante, conhecida como padrão String enum, é ainda menos desejável. Embora forneça strings imprimíveis para suas constantes, pode levar usuários ingênuos a codificar constantes de string em código do cliente em vez de usar nomes de campos. Se tal constante de string codificada contiver um erro tipográfico, ela escapará da detecção em tempo de compilação e resultará em bugs em tempo de execução. Além disso, pode causar problemas de desempenho, porque depende de comparações de cordas.

Felizmente, o Java oferece uma alternativa que evita todas as limitações dos padrões de theint e string enum e oferece muitos benefícios adicionais. É o enumtipo [JLS, 8.9]. Veja como ele fica em sua forma mais simples:

```
public enum Apple { FUJI, PIPPIN, GRANNY_SMITH }  
O público é Laranja { Romance, Templo, Sangue }
```

À primeira vista, esses tipos de enum podem parecer semelhantes aos de outras linguagens, como C, C++ e C#, mas as aparências enganam. Os tipos de enum do Java são classes completas, muito mais poderosas do que suas contrapartes nesses outros LAN-guages, onde enums são essencialmente valores int.

A ideia básica por trás dos tipos de enum do Java é simples: são classes que exportam uma instância para cada constante de enumeração via um campo final público e estático. Enumtipos são efetivamente finais, por não terem construtores acessíveis.

Como os clientes não podem criar instâncias de um tipo enum nem estendê-lo, não podem existir instâncias além das constantes de enum declaradas. Em outras palavras, os tipos de enum são controlados por instâncias (página 6). Eles são uma generalização dos singletons (Item 3), que são essencialmente enums de elemento único.

Enums fornecem segurança de tipos em tempo de compilação. Se você declarar um parâmetro como do tipo Apple, você tem garantia de que qualquer referência de objeto não nulo passada para o parâmetro é um dos três valores válidos da Apple. Tentativas de passar valores do

Tipo errado resultará em erros em tempo de compilação, assim como tentativas de atribuir uma expressão de um tipo de enum a uma variável de outro, ou de usar o operador `==` para comparar valores de diferentes tipos de enum.

Tipos de enum com constantes nomeadas idênticas coexistem pacificamente porque cada tipo tem seu próprio namespace. Você pode adicionar ou reordenar constantes em um tipo enum sem recompilar seus clientes porque os campos que exportam as constantes fornecem uma camada de isolamento entre um tipo enum e seus clientes: valores constantes não são compilados nos clientes como estão nos padrões int enum. Por fim, você pode traduzir enums em strings imprimíveis chamando o método `toString` deles.

Além de corrigir as deficiências dos enums int, os tipos de enum permitem adicionar métodos e campos arbitrários e implementar interfaces arbitrárias. Eles fornecem implementações de alta qualidade de todos os métodos `Object` (Capítulo 3), implementam `Comparável` (Item 14) e `Serializável` (Capítulo 12), e sua forma serializada foi projetada para suportar a maioria das mudanças no tipo enum.

Então, por que você gostaria de adicionar métodos ou campos a um tipo de enum? Para começar, talvez você queira associar dados às suas constantes. Nossos tipos de maçã e laranja, por exemplo, podem se beneficiar de um método que devolve a cor da fruta, ou de um que devolve uma imagem dela. Você pode aumentar um tipo de enum com qualquer método que parecer apropriado. Um tipo de enum pode começar como uma simples coleção de `enum` constantes e evoluir ao longo do tempo para uma abstração completa.

Para um bom exemplo de um tipo de enum rico, considere os oito planetas do nosso sistema solar. Cada planeta tem uma massa e um raio, e a partir desses dois atributos você pode calcular sua gravidade superficial. Isso, por sua vez, permite calcular o peso de um objeto na superfície do planeta, dada a massa do objeto. Veja como esse enum fica. Os números entre parênteses após cada constante enum são parâmetros que são passados ao seu construtor. Neste caso, eles são a massa e o raio do planeta:

```
Enum type with data and behavior public
enum Planet {
    MERCÚRIO (3.302e+23, 2.439e6),
    VÊNUS (4.869e+24, 6.052e6),
    TERRA (5.975e+24, 6.378e6),
    MARTE (6.419e+23, 3.393e6),
    JÚPITER (1.899e+27, 7.149e7),
    SATURNO (5.685e+26, 6.027e7),
    URANO (8.683e+25, 2.556e7),
    NEPTUNO (1.024e+26, 2.477e7);

    massa final privada;                Em quilogramas
    raio duplo de final privado;          Em metros
    superfície dupla final privada Gravidade; Em m / s^2
}
```

```

    Constante gravitacional universal em m^3 / kg s^2 estática final privada G =
    6,67300E-11;

    ConstructorPlanet(dupla massa, dupla raio) {

        this.mass = massa; this.radius = raio;
        GravidadeSuperfície = G * massa / (raio * raio);}

    Missa dupla pública()          { devolver massa; }
    Raio duplo público()           { raio de retorno; }
    GravidadeDupla Dupla pública() { retorno de GravidadeSuperfície; }

    Peso da superfície dupla pública (massa dupla) {
        retorno massa * superfície          F = ma
    }}      Gravidade;

```

É fácil escrever um tipo enum rico como Planeta. Para associar dados às constantes, declare campos de instância e escreva um construtor que pegue os dados e os armazene nos campos. Enums são, por natureza, imutáveis, então todos os campos devem ser finais (Item 17). Campos podem ser públicos, mas é melhor torná-los privados e fornecer acessórios públicos (Item 16). No caso do Planeta, o construtor também calcula e armazena a gravidade da superfície, mas isso é apenas uma otimização. A gravidade poderia ser recalculada a partir da massa e do raio cada vez que fosse usada pelo método do peso superficial, que toma a massa de um objeto e retorna seu peso no planeta representado pela constante.

Embora o enum do Planeta seja simples, ele é surpreendentemente poderoso. Aqui está um programa curto que pega o peso terrestre de um objeto (em qualquer unidade) e imprime uma tabela do peso do objeto em todos os oito planetas (na mesma unidade):

```

classe pública WeightTable {
    public static void main(String[] args) {
        duplo peso terra = Duplo.parseDuplo(args[0]); massa dupla =
        peso da terra / Planeta.TERRA.GravidadeSuperfície(); para
        (Planeta p : Planeta.valores())
            System.out.printf("Peso em %s é %f%n",
                               p, p.PesoSuperfície(massa));
    }}

```

Note que o Planet, como todos os enums, possui um método de valores estáticos que retorna um array de seus valores na ordem em que foram declarados. Note também que o método toString retorna o nome declarado de cada valor enum, permitindo fácil impressão por println e printf. Se você não estiver satisfeito com essa representação de cordas, pode mudá-la

sobrescrevendo o método `toString`. Aqui está o resultado de rodar nosso `WeightTable` program (que não sobrescreve para `String`) com o argumento de linha de comando 185:

```
Peso em MERCÚRIO é 69,912739Peso
em VÊNUS é 167,434436Peso na
TERRA é 185,000000Peso em MARTE é
70,226739Peso em JÚPITER é
467,990696Peso em SATURN é
197,12011Peso em URANO é
167,398264Peso em NEPTUNO é
210,208751
```

Até 2006, dois anos após a adição dos enums a Java, Plutão era um planeta. Isso levanta a questão: "o que acontece quando você remove um elemento de um `enum` típico?" A resposta é que qualquer programa cliente que não faça referência ao elemento removido continuará funcionando normalmente. Por exemplo, nosso programa `WeightTable` simplesmente imprimiria uma tabela com uma linha a menos. E quanto a um programa cliente que se refere ao elemento removido (neste caso, `Planeta.Plutão`)? Se você recompilar o programa cliente, a compilação falhará com uma mensagem de erro útil na linha que se refere ao antigo planeta; Se você falhar em recompilar o cliente, ele lançará uma exceção útil dessa linha em tempo de execução. Esse é o melhor comportamento que você poderia esperar, muito melhor do que o que você teria com o padrão `int enum`.

Alguns comportamentos associados às constantes de `enum` podem precisar ser usados apenas dentro da classe ou pacote em que o `enum` está definido. Esses comportamentos são melhor implementados como métodos privados ou pacotes-privados. Cada constante então carrega consigo uma coleção oculta de comportamentos que permite que a classe ou pacote que contém o `enum` reaja adequadamente quando apresentada à constante. Assim como em outras classes, a menos que você tenha um motivo convincente para expor um `enum` método aos seus clientes, declare-o privado ou, se necessário, privado de pacote (Item 15).

Se um `enum` for geralmente útil, deve ser uma classe de nível superior; se seu uso estiver vinculado a uma classe de nível superior específica, deve ser uma classe membro dessa classe de nível superior (Item 24). Por exemplo, o `enum java.math.RoundingMode` representa um modo de arredondamento para frações decimais. Esses modos de arredondamento são usados pela classe `BigDecimal`, mas fornecem uma abstração útil que não está fundamentalmente ligada ao `BigDecimal`. Ao tornar o `RoundingMode` um `enum` de alto nível, os designers da biblioteca incentivam qualquer programador que precise de modos de arredondamento a reutilizar esse `enum`, levando a uma maior consistência entre APIs.

As técnicas demonstradas no exemplo do `Planeta` são suficientes para tipos `mostenum`, mas às vezes você precisa de mais. Existem dados diferentes associados a cada constante planetária, mas às vezes é necessário associar comportamentos fundamentalmente diferentes a cada constante. Por exemplo, suponha que você esteja escrevendo um `enum`

Digite para representar as operações em uma calculadora básica de quatro funções e você quer fornecer um método para realizar a operação aritmética representada por cada constante. Uma forma de conseguir isso é ativar o valor do enum:

```
Tipo de enum que ativa seu próprio valor - questionablepublic
enum Operação {
    MAIS, MENOS, VEZES, DIVIDE;

    A operação aritmética representada por este constantepublic
    double apply(double x, double y) {
        switch(this) {
            caso MAIS:         retorne x + y;
            caso MENOS:        retorne x - y;
            caso HORARIOS DO   retorne x * y;
            caso DIVIDE:        retorne x / y;
        }
        lançar um novo AssertionError("Op
        desconhecido: " + isto);
    }
}
```

Esse código funciona, mas não é nada bonito. Ele não compilará sem a `throwStatement` porque o fim do método é tecnicamente alcançável, mesmo que nunca seja alcançado [JLS, 14.21]. Pior ainda, o código é frágil. Se você adicionar uma constante de `newenum` mas esquecer de adicionar um case correspondente ao switch, o `enum` ainda compilará, mas falhará em tempo de execução quando você tentar aplicar a nova operação.

Felizmente, existe uma maneira melhor de associar um comportamento diferente a cada `enum` constante: declarar um método abstrato de aplicação no tipo `enum` e sobrepor-lo com um método concreto para cada constante em um corpo de classes específico para constante. Tais `methods` são conhecidos como implementações de métodos específicos para constantes:

```
Tipo enum com implementações específicas de método
constantepublic enum Operação {
    MAIS{public double apply(double x, double y){return x + y;}},
    MENOS{public double apply(double x, double y){return x - y;}},
    TIMES{public double apply(double x, double y){return x * y;}},
    DIVIDE{public double apply(double x, double y){return x / y;}};

    Resumo Público Double Apply(Double X, Double Y);
}
```

Se você adicionar uma nova constante à segunda versão do `Operation`, é improvável que esqueça de fornecer um método `apply`, pois o método segue imediatamente cada declaração de constante. No improvável caso de você esquecer, o `compiler` vai te lembrar porque métodos abstratos em um tipo `enum` precisam ser sobrepostos por métodos concretos em todas as suas constantes.

Implementações de métodos específicos para constantes podem ser combinadas com dados específicos para constantes. Por exemplo, aqui está uma versão do `Operation` que sobrescreve o método `thetoString` para retornar o símbolo comumente associado à operação:

```
Tipo enum com corpos de classe específicos constantes e
datapublic enum Operação {
    MAIS("+") {
        public double apply(double x, double y) { return x + y;
    }, MINUS("-") {

        public double apply(double x, double y) { return x - y;
    }, TIMES("*") {

        public double apply(double x, double y) { return x * y;
    }, DIVIDE("/") {

        Public double apply(double x, double y) { return x / y; }

    símbolo final privado de corda;

    Operação(Símbolo de string) { this.symbol = symbol; }

    @Override String toString() { símbolo de retorno; }

    Resumo Público Double Apply(Double X, Double Y);}
```

A implementação `toString` mostrada facilita a impressão de expressões aritméticas, como demonstrado por este pequeno programa:

```
public static void main(String[] args) {
    double x = Double.parseDouble(args[0]);
    double y = Double.parseDouble(args[1]);
    para (Operação op : Operation.values())
        System.out.printf("%f %s %f = %f\n",
                           x, op, y, op.apply(x, y));
}
```

Executar este programa com 2 e 4 como argumentos de linha de comando produz a seguinte saída:

```
2.000000 + 4.000000 = 6.000000
2.000000 - 4.000000 = -2.000000
2.000000 * 4.000000 = 8.000000
2.000000 / 4.000000 = 0.500000
```


Tipos de enum possuem um método `valueOf(String)` gerado automaticamente, que traduz o nome de uma constante para a própria constante. Se você sobrescreve o método `toString` em um tipo enum, considere escrever um método `fromString` para traduzir a representação personalizada da string de volta para o enum correspondente. O código a seguir (com o nome do tipo alterado adequadamente) fará o trabalho para qualquer enum, desde que cada constante tenha uma representação única de string:

```
Implementando um método fromString em um tipo enum, estático
privado final Map<String, Operation> stringToEnum =
    Stream.of(values()).collect(
        toMap(Object::toString, e -> e));

Retorna Operação para string, se anypublic static
Opcional<Operation> fromString(Símbolo String) {
    return Optional.ofNullable(stringToEnum.get(symbol));}
```

Note que as constantes de Operação são inseridas no mapa `stringToEnum` a partir da inicialização do campo estático que roda após a criação das constantes enum. O código anterior usa um fluxo (Capítulo 7) sobre o array retornado pelo método `values()`; antes do Java 8, teríamos criado um mapa de hash vazio e iterado sobre o array de valores inserindo os mapeamentos string-para-enum no mapa, e você ainda pode fazer isso se preferir. Mas note que tentar fazer com que cada constantput se coloque em um mapa a partir de seu próprio construtor não funciona. Isso causaria erro de compilação, o que é bom porque, se fosse legal, causaria uma `NullPointerException` em tempo de execução. Construtores de enum não têm permissão para acessar os campos estáticos do enum, com exceção de variáveis constantes (Item 34). Essa restrição é necessária porque os campos estáticos ainda não foram inicializados quando os construtores `whenenum` rodam. Um caso especial dessa restrição é que constantes de enum não podem acessar umas às outras a partir de seus construtores.

Também note que o método `fromString` retorna um `Opcional<String>`. Isso permite que o método indique que a string passada não representa uma operação válida, e força o cliente a confrontar essa possibilidade (Item 55).

Uma desvantagem das implementações específicas de métodos constantes é que elas dificultam o compartilhamento de código entre constantes enum. Por exemplo, considere anenum representando os dias da semana em um pacote de folha de pagamento. Esse enum tem um método que calcula o pagamento do trabalhador para aquele dia, considerando o salário base (por hora) e o número de minutos trabalhados naquele dia. Nos cinco dias úteis, qualquer tempo trabalhado acima do turno normal gera pagamento de horas extras; Nos dois dias de fim de semana, todo o trabalho gera pagamento de horas extras. Com a instrução `switch`, fica fácil

para fazer esse cálculo aplicando múltiplos rótulos de caso a cada um dos dois fragmentamentos de código:

```
Enum que ative seu valor para compartilhar código - questionableenum
PayrollDay {
    SEGUNDA, TERÇA-FEIRA, QUARTA, QUINTA, SEXTA,
    SÁBADO, DOMINGO;

    Privado Static Final Int MINS_PER_SHIFT = 8 * 60;

    int pay(int minutosTrabalhados, int payRate) {
        int basePagamento = minutosTrabalhados * TarifaPaga;

        int horas extrasPagamento;
        switch(this) {
            caso SÁBADO: caso DOMINGO: // Fim de semana
                pagamento extra = pagamento
                base / 2; intervalo; padrão:
                // Dia da semana
                horas extrasPagamento = minutosTrabalhado <= MINS_PER_SHIFT ?
                0 : (minutosTrabalhado - MINS_PER_SHIFT) * Remuneração / 2;

        return basePay + OvertimePay;}}
```

Esse código é inegavelmente conciso, mas é perigoso do ponto de vista da manutenção. Suponha que você adicione um elemento ao enum, talvez um valor especial para representar um dia de férias, mas esqueça de adicionar um caso correspondente à instrução switch. O programa ainda compila, mas o método de pagamento pagará silenciosamente ao trabalhador o mesmo valor por um dia de férias que para um dia comum da semana.

Para realizar o cálculo de pagamento com segurança e implementações específicas de métodos constantes, você teria que duplicar o cálculo de horas extras para cada constante, ou mover o cálculo para dois métodos auxiliares, um para dias úteis e outro para dias de fim de semana, e invocar o método auxiliar apropriado de cada constante. Qualquer abordagem resultaria em uma boa quantidade de código padrão, reduzindo substancialmente a legibilidade e aumentando a possibilidade de erro.

O padrão pode ser reduzido substituindo o método extraordinário abstrato no Payday por um método concreto que realize o cálculo de horas extras nos dias úteis. Assim, só os dias de fim de semana teriam que sobrescrever o método. Mas isso teria a mesma desvantagem da declaração switch: se você adicionasse mais um dia sem sobrescrever o método OvertimePay, herdaria silenciosamente o cálculo do dia da semana.

O que você realmente quer é ser forçado a escolher uma estratégia de pagamento de horas extras toda vez que adicionar uma constante de enum. Felizmente, existe uma maneira boa de conseguir isso. A ideia é mover o cálculo de pagamento de horas extras para um enum aninhado privado e passar uma instância desse enum estratégico para o construtor do enum do PayPayDay. O enum do PayrollDay então delega o cálculo do pagamento de horas extras ao strategyenum, eliminando a necessidade de uma declaração de troca ou implementação de método específico constante no PayrollDay. Embora esse padrão seja menos conciso que a instrução switch, ele é mais seguro e flexível:

```
The strategy enum patternenum
PayrollDay {
    SEGUNDA, TERÇA, QUARTA, QUINTA, SEXTA, SÁBADO
    (PayType.WEEKEND), DOMINGO (PayType.WEEKEND);

    payType final privado;

    PayPayDay(PayType payType) { this.payType = payType;
    }PayrollDay() { this(PayType.WEEKDAY); }Padrão

    int pay(int minutosTrabalhados, int payRate) {
        return payType.pay(minutosWorked, payRate);}

A estratégia enum
typeprivate enum PayType {
    DIA ÚTIL {
        int overtimePay(int minsWorked, int payRate) {
            retorno minsTrabalhado <= MINS_PER_SHIFT ? 0 :
            (minsTrabalhado - MINS_PER_SHIFT) * salárioTarifa /
            2;},FIM de SEMANA {

            int overtimePay(int minsWorked, int payRate) {
                retorno minsTrabalhado * payRate / 2;}};

        abstract int OvertimePay(int mins, int payTax);
        Privado Static Final Int MINS_PER_SHIFT = 8 * 60;

        int pay(int minsWorked, int payRate) {
            int basePay = minsWorked * payRate; return basePay +
            OvertimePay(minsWorked, payRate);}}}
```

Se as instruções `switch` em `enums` não são uma boa escolha para implementar comportamento específico de constantes em `enums`, para que servem? `Switches` em `enums` são bons para aumentar tipos de `enum` com comportamento específico constante. Por exemplo, suponha que o `enum` da Operação não esteja sob seu controle e você deseje que ele tivesse um método de instância para retornar o inverso de cada operação. Você poderia simular o efeito com o seguinte método estático:

```
Ative um enum para simular um método ausente estático
público Operação inversa (Operação op) {
    switch(op) {
        caso MAIS:      retorno da Operação.MINUS;
        caso MENOS:     retorne a Operação.PLUS;
        HORÁRIOS DO     retorne à Operação.DIVIDE;
        caso DIVIDE:    retorno Operação.TIMES;

        padrão:        lançar um novo AssertionError("Desconhecido em: " + sobre);
    }}
}
```

Você também deve usar essa técnica em tipos de `enum` que estão sob seu controle se um método simplesmente não pertencer ao tipo de `enum`. O método pode ser necessário para algum uso, mas geralmente não é útil o suficiente para justificar a inclusão no tipo `enum`.

`Enums` são, de modo geral, comparáveis em desempenho às constantes de inteligência. Uma pequena desvantagem de desempenho dos `enums` é que há um custo de espaço e tempo para carregar e inicializar tipos de `enum`, mas é improvável que isso seja perceptível na prática.

Então, quando você deve usar `enums`? Use `enums` sempre que precisar de um conjunto de constantes cujos membros sejam conhecidos na compilação. Claro, isso inclui "tipos naturalmente enumerados", como os planetas, os dias da semana e as peças de xadrez. Mas também inclui outros conjuntos para os quais você conhece todos os valores possíveis em tempo de compilação, como escolhas em um menu, códigos de operação e flags de linha de comando. Não é necessário que o conjunto de constantes em um tipo de `enum` permaneça fixo por todo o tempo. O recurso de `enum` foi especificamente projetado para permitir a evolução binária compatível dos tipos de `enum`.

Em resumo, as vantagens dos tipos de `enum` em relação às constantes de inteligência são convincentes. `Enums` são mais legíveis, mais seguros e mais poderosos. Muitos `enums` requerem construtores ou membros não explícitos, mas outros se beneficiam ao associar dados com cada constante e fornecer métodos cujo comportamento é afetado por esses dados. Menos `enums` se beneficiam ao associar múltiplos comportamentos a um único método. Neste caso relativamente raro, prefiro métodos específicos de constantes a `enums` que ativam seus próprios valores. Considere o padrão de estratégia `enum` se algumas, mas não todas, as constantes de `enum` compartilharem comportamentos comuns.

Item 35: Usar campos de instância em vez de ordinais

Muitos enums estão naturalmente associados a um único valor int. Todos os enums possuem um método ordinal, que retorna a posição numérica de cada constante de enum em seu tipo. Você pode se sentir tentado a derivar um valor int associado a partir do ordinal:

```
Abuso do ordinal para derivar um valor associado - DON'T DO
THISpublic enum Ensemble {
    SÓ,          DUETO,   TRIO,  QUARTETO,  QUINTETO,
    SEXTETO, SEPTETO, OCTETO, NONETO, DECTET;

    public int numberOfMusicians() { return ordinal() + 1; }}
```

Embora esse enum funcione, é um pesadelo de manutenção. Se as constantes forem reordenadas, o método numberOfMusicians quebrará. Se você quiser adicionar uma constante enum sec-ond associada a um valor int que já usou, está sem sorte. Por exemplo, pode ser interessante adicionar uma constante para quarteto duplo, que, como um octeto, consiste em oito músicos, mas não há como fazer isso.

Além disso, você não pode adicionar uma constante para um valor int sem adicionar constantes para todos os valores int intermediários. Por exemplo, suponha que você queira adicionar uma repetição constante de um quarteto triplo, que consiste em doze músicos. Não existe um termo padrão para um conjunto composto por onze músicos, então você é obrigado a adicionar uma constante de adumidade para o valor de int não usado (11). No máximo, isso é feio. Se muitos intvalues não forem usados, é impraticável.

Felizmente, existe uma solução simples para esses problemas. Nunca derive um valor associado a um enum a partir de seu ordinal; Armazene-o em um campo de instância:

```
public enum Ensemble {
    SOLO(1), DUETO(2), TRIO(3), QUARTETO(4), QUINTETO(5),
    SEXTETO(6), SEPTETO(7), OCTETO(8), DOUBLE_QUARTET(8),
    NENHUM(9), DECTET(10), TRIPLE_QUARTET(12);

    número privado de identificação final OfMusicians; Ensemble(int size)
    { this.numberOfMusicians = size; }número de int públicosOfMúsicos() {
    return numberOfMusicians; }}
```

A especificação Enum diz o seguinte sobre ordinal: "A maioria dos programadores não terá uso para este método. Ele foi projetado para uso por estruturas de dados baseadas em Enum de uso geral, como EnumSet e EnumMap." A menos que você esteja escrevendo código com esse caractere, o melhor é evitar completamente o método ordinal.

Item 36: Use EnumSet em vez de campos de bits

Se os elementos de um tipo enumerado são usados principalmente em conjuntos, é tradicional usar o padrão `int` `enum` (Item 34), atribuindo uma potência diferente de 2 a cada constante:

```
Constantes de enumeração de campos de bits -
OBSOLETE! classe pública Text {
    Estática pública STYLE_BOLD final de INT      = 1 << 0; // 1
    Estática pública final STYLE_ITALIC            = 1 << 1; // 2
    Estática pública STYLE_UNDERLINE final de INT  = 1 << 2; // 4
    Int final estático público STYLE_STRIKETHROUGH = 1 << 3; // 8

    O parâmetro é bit a bit OR de zero ou mais STYLE_ constantes public
    void applyStyles(int styles) { ... }}
```

Essa representação permite que você use a operação OR bit a bit para combinar vários con-stants em um conjunto, conhecido como campo de bits:

```
text.applyStyles(STYLE_BOLD | STYLE_ITALIC);
```

A representação do campo de bits também permite realizar operações de conjunto, como união e interseção, de forma eficiente usando aritmética bit a bit. Mas os campos de bits têm todas as desvantagens das constantes de `enum int` e mais. É ainda mais difícil interpretar um campo de bits do que uma simples constante `int` `enum` quando é impresso como um número. Não há uma maneira fácil de iterar sobre todos os elementos representados por um campo de bits. Por fim, você precisa prever o número máximo de bits que vai precisar no momento em que estiver escrevendo a API e escolher um tipo para o campo de bits (normalmente `int` ou `long`) de acordo. Depois de escolher um tipo, não pode exceder sua largura (32 ou 64 bits) sem mudar a API.

Alguns programadores que preferem `enums` às constantes `int` ainda se apegam ao uso de campos de bits quando precisam passar conjuntos de constantes. Não há motivo para isso, porque existe uma alternativa melhor. O pacote `java.util` promove a classe `EnumSet` para representar eficientemente conjuntos de valores extraídos de um tipo `singleenum`. Esta classe implementa a interface `Set`, fornecendo toda a riqueza, segurança de tipos e interoperabilidade que você obtém com qualquer outra implementação de `Set`. Mas, internamente, cada `EnumSet` é representado como um vetor de bits. Se o tipo `enum` subjacente tiver sessenta e quatro ou menos elementos — e a maioria tem — todo o `EnumSet` é representado com um único longo, então seu desempenho é comparável ao de um campo de bits. Operações em massa, como `removeAll` e `retainAll`, são implementadas usando bit-

Aritmética sábia, assim como você faria manualmente para campos de bits. Mas você está protegido da feiura e da propensão a erros do manipular manualmente o bit: a `EnumSet` faz o trabalho difícil por você.

Veja como o exemplo anterior aparece quando modificado para usar conjuntos de andênio enums em vez de campos de bits. É mais curto, claro e seguro:

```
EnumSet - uma substituição moderna para a classe bit
fieldspublic Text {
    public enum Style { NEGRITO, ITÁLICO, SUBLINHADO, RISCADO }
```

```
Qualquer Set poderia ser passado, mas EnumSet é claramente
bestpublic void applyStyles(Set<Style> styles) { ... }}
```

Aqui está o código cliente que passa uma instância `EnumSet` para o método `applyStyles`. A classe `EnumSet` fornece um conjunto rico de fábricas estáticas para fácil criação de conjuntos, uma das quais está ilustrada neste código:

```
text.applyStyles(EnumSet.of(Style.BOLD, Style.ITALIC));
```

Note que o método `applyStyles` assume um `Set<Style>` em vez de um `<Style>EnumSet`. Embora pareça provável que todos os clientes passem um `EnumSet` para o método, geralmente é boa prática aceitar o tipo de interface em vez do tipo de implementação (Item 64). Isso permite a possibilidade de um cliente incomum passar alguma outra implementação de `Set`.

Em resumo, só porque um tipo enumerado será usado em conjuntos, não há razão para representá-lo com campos de bits. A classe `EnumSet` combina a concisão e o desempenho dos campos de bits com todas as muitas vantagens dos tipos de enum descritas no Item 34. A única desvantagem real do `EnumSet` é que, desde o Java 9, não é possível criar um `EnumSet` imutável, mas isso provavelmente será corrigido em uma próxima versão. Enquanto isso, você pode envolver um `EnumSet` com `withCollections.unmodifiableSet`, mas a concisão e o desempenho vão sofrer.

Item 37: Use o EnumMap em vez da indexação ordinal

Ocasionalmente, você pode ver código que usa o método ordinal (Item 35) para indexar em um array ou lista. Por exemplo, considere esta classe simplista destinada a representar planta:

```

classe Plant {
    enum LifeCycle { ANNUAL, PERENNIAL, BIENNIAL }

    nome final da corda; ciclo
    de vida final;

    Plant(String name, LifeCycle lifeCycle) {
        this.name = nome; this.lifeCycle
        = lifeCycle;}

    @Override público String toString() {
        retorne nome;}}

```

Agora suponha que você tenha uma variedade de plantas representando um jardim e queira listar essas plantas organizadas por ciclo de vida (anual, perene ou bienal). Para isso, você constrói três conjuntos, um para cada ciclo de vida, e itera pelo jardim, colocando cada planta no conjunto apropriado. Alguns programadores faziam isso colocando os conjuntos em um array indexado pelo ordinal do ciclo de vida:

```

Usando ordinal() para indexar em um array - NÃO FAÇA ISSO!
<Plant>Definir[] plantasPorCicloDeVida =
    (Set<Plant>[]) novo Set[Planta.ValoresCicloDe Vida().comprimento];
para (int i = 0; i < plantsByLifeCycle.length; i++)
    plantsByLifeCycle[i] = novo HashSet<>();

para (Planta p : jardim)
    plantsByLifeCycle[p.lifeCycle.ordinal()].add(p);

Imprima os resultadospara (int i = 0; i <
    plantsByLifeCycle.length; i++) {
    System.out.printf("%s: %s%n",
        Planta.ValoresCiclode de Vida()[i], plantasPeloCicloVida[i]);
}Essa técnica funciona, mas está cheia de problemas. Como os arrays não são
compatíveis com genéricos (Item 28), o programa requer um cast não verificado
e

```


Não compilará de forma limpa. Como o array não sabe qual seu índice representa resentedo, você precisa rotular a saída manualmente. Mas o problema mais sério dessa técnica é que, quando você acessa um array indexado pelo ordinal de um enum, é sua responsabilidade usar o valor int correto; INTs não oferecem segurança de tipo de ENUMs. Se você usar o valor errado, o programa fará a coisa errada silenciosamente ou — se tiver sorte — lançará um `ArrayIndexOutOfBoundsException`.

Existe uma maneira muito melhor de alcançar o mesmo efeito. O array serve efetivamente como um mapa do enum para um valor, então é melhor usar um Mapa. Mais especificamente, existe uma implementação Map muito rápida projetada para uso com enumkeys, conhecida como `java.util.EnumMap`. Aqui está como o programa fica quando é reescrito para usar `EnumMap`:

```
Usar um EnumMap para associar dados a um
enumMap<Plant.LifeCycle, Set<Plant>> plantsByLifeCycle =
    novo EnumMap<>(Plant.LifeCycle.class); para
    (Plant.LifeCycle lc : Plant.LifeCycle.values())
    plantsByLifeCycle.put(lc, novo HashSet<>()); para
    (Planta p : jardim)
    plantsByLifeCycle.get(p.lifeCycle).add(p);
System.out.println(plantsByLifeCycle);
```

Este programa é mais curto, mais claro, mais seguro e comparável em velocidade à versão original. Não há elenco perigoso; Não há necessidade de rotular a saída manualmente porque as teclas themap são enums que sabem como se traduzir para strings imprimíveis; e não há possibilidade de erro no cálculo dos índices do array. A razão pela qual o `EnumMap` é comparável em velocidade a um array indexado por ordinais é que o `EnumMap` usa esse anarray internamente, mas esconde esse detalhe de implementação do programador, combinando a riqueza e a segurança de tipos de um Map com a velocidade de um array. Note que o construtor `EnumMap` assume o objeto Class do tipo chave: este é o token `abounded type`, que fornece informações genéricas de tipo em tempo de execução (Item 33).

O programa anterior pode ser ainda mais encurtado usando um fluxo (Item 45) para gerenciar o mapa. Aqui está o código mais simples baseado em fluxo que em grande parte duplica o comportamento do exemplo anterior:

```
Abordagem ingênua baseada em stream - improvável que produza um
EnumMap! System.out.println(Arrays.stream(jardim)
    .collect(groupingBy(p -> p.lifeCycle)));
```

O problema desse código é que ele escolhe sua própria implementação de mapa, e na prática não será um `EnumMap`, então não vai igualar o desempenho espacial e temporal da versão com o `EnumMap` explícito. Para corrigir esse problema, use os três-

forma do parâmetro `Collectors.groupingBy`, que permite ao chamador especificar a implementação do mapa usando o parâmetro `mapFactory`:

```
Usando um stream e um EnumMap para associar dados a um
enumSystem.out.println(Arrays.stream(jardim)
    .collect(groupingBy(p -> p.lifeCycle,
        () -> novo EnumMap<>(LifeCycle.class), toSet())));
```

Essa otimização não valeria a pena em um programa de brinquedo como este, mas poderia ser crítica em um programa que fizesse uso intenso do mapa.

O comportamento das versões baseadas em stream difere ligeiramente do da versão `theEnumMap`. A versão `EnumMap` sempre cria um mapa aninhado para cada ciclo de vida da planta, enquanto as versões baseadas em rios só criam um mapa aninhado se o jardim contiver uma ou mais plantas com esse ciclo de vida. Assim, por exemplo, se o jardim contém anuais e perenes, mas não bienais, o tamanho das `plantasByLifeCycle` será três na versão `EnumMap` e duas em ambas as versões baseadas em rios.

Você pode ver um array de arrays indexados (duas vezes!) por ordinais usados para representar mapeamentos a partir de dois valores de enum. Por exemplo, este programa usa tal array tomam de duas fases para uma transição de fase (líquido para sólido é congelante, líquido para gás é ebulição, e assim por diante):

```
Usando ordinal() para indexar array de arrays - NÃO FAÇA
ISSO!public enum Fase {
    SÓLIDO, LÍQUIDO, GASOSO;

    public enum Transition {
        DERRETER, CONGELAR, FERVER, CONDENSAR, SUBLIME, DEPOSITAR;

        Linhas indexadas por from-ordinal, cols por to-ordinal
        private static final Transition[][] TRANSITIONS = {
            { null,    DERRETER,  SUBLIME },
            { CONGELE,  nulo,     Ferver  },
            { DEPÓSITO, CONDENSAR, nulo    }
        };

        Retorna a transição de fase de uma fase para outra.
        Transição estática pública de(Fase de, Fase para) {
            return TRANSITIONS[from.ordinal()][para.ordinal()];}}}

```

Esse programa funciona e pode até parecer elegante, mas as aparências podem enganar. Como no exemplo mais simples do jardim mostrado anteriormente, o compilador não tem como saber a relação entre ordinais e índices de arrays. Se você fizer um

erro na tabela de transição ou esquecer de atualizá-la quando você modifica o tipo enum `Phase` ou `Transition`, seu programa falhará em tempo de execução. A falha pode ser uma `ArrayIndexOutOfBoundsException`, uma `NullPointerException` ou (pior) comportamento errôneo silencioso. E o tamanho da tabela é quadrático em relação ao número de fases, mesmo que o número de entradas não nulas seja menor.

Novamente, você pode fazer muito melhor com o `EnumMap`. Como cada transição de fase é indexada por um par de enums de fase, o ideal é representar a relação como um mapa de um enum (a fase "de") para um mapa do segundo enum (a fase "para") para o resultado (a transição de fase). As duas fases associadas à transição de fase são melhor capturadas associando-as ao `transicionenum` de fase, que pode então ser usado para inicializar o `EnumMap` aninhado:

```
Usando um EnumMap aninhado para associar dados a pares
enumpublic Fase {
    SÓLIDO, LÍQUIDO, GASOSO;

    public enum Transition {
        DERRETER (SÓLIDO, LÍQUIDO), CONGELAR (LÍQUIDO, SÓLIDO),
        FERVER (LÍQUIDO, GÁS), CONDENSAR (GÁS, LÍQUIDO),
        SUBLIME (SÓLIDO, GÁS), DEPÓSITO (GÁS, SÓLIDO);

        fase final privada de; fase
        final privada para;

        Transição(Fase de, Fase para) {
            this.from = from;
            this.to = para;}

        Inicialize o mapa de transição de fase privado estático final
        Mapa<Fase, Mapa<Fase, Transição>>m =
        Stream.of(values().collect(groupingBy(t -> t.from,
            () -> novo EnumMap<>
                (Phase.class),toMap(t -> t.to, t -> t,
                    (x, y) -> y, () -> novo EnumMap<>(Phase.class))));

        transição estática pública de(Fase de, Fase para) {
            return m.get(from).get(to);}}
```

O código para inicializar o mapa de transição de fase é um pouco complicado. O tipo do mapa é `Mapa<Fase, Mapa<Fase, Transição>>`, que significa "mapear de (fonte) fase para mapeamento de (destino) fase para transição." Este mapa de mapas é inicializado usando uma sequência em cascata de dois coletores. Os primeiros grupos de coletores

as transições por fase de origem, e a segunda cria um EnumMap com mapeamentos da fase de destino para a transição. A função de fusão no segundo coletor ((x, y) -> y)) não é utilizada; ela é necessária apenas porque precisamos especificar uma faixa de mapa para obter um EnumMap, e a Collectors fornece fábricas telescópicas. A edição anterior deste livro usou iteração explícita para inicializar o mapa de transição de fase. O código era mais prolixo, mas, talvez, mais fácil de entender.

Agora suponha que você queira adicionar uma nova fase ao sistema: plasma ou gás ionizado. Existem apenas duas transições associadas a essa fase: ionização, que leva um gás a um plasma; e desionização, que transforma um plasma em gás. Para atualizar o programa baseado em arrays, você teria que adicionar uma nova constante ao Phase e duas ao Phase.Transition, e substituir o array original de nove elementos por uma nova versão de dezesseis elementos. Se você adicionar muitos ou poucos elementos ao array ou colocar um elemento fora de ordem, você está sem sorte: o programa irá compilar, mas falhará em tempo de execução. Para atualizar a versão baseada em EnumMap, tudo o que você precisa fazer é adicionar PLASMA à lista de fases, e IONIZE (GAS, PLASMA) e DEIONIZE (PLASMA, GAS) à lista de transições de fase:

```
Adicionando uma nova fase usando a implementação aninhada EnumMap
enum public Phase {
    SÓLIDO, LÍQUIDO, GASOSO, PLASMA;
    public enum Transition {
        DERRETER (SÓLIDO, LÍQUIDO), CONGELAR (LÍQUIDO, SÓLIDO),
        FERVER (LÍQUIDO, GÁS), CONDENSAR (GÁS, LÍQUIDO),
        SUBLIME (SÓLIDO, GÁS), DEPOSITAR (GÁS, SÓLIDO), IONIZAR
        (GÁS, PLASMA), DEIONIZAR (PLASMA, GÁS);... // Restante
        inalterado}}
```

O programa cuida de todo o resto e praticamente não deixa chance de erro. Internamente, o mapa de mapas é implementado com uma variedade de arrays, então você paga pouco em custo de espaço ou tempo pela maior clareza, segurança e facilidade de manutenção.

Para ser breve, os exemplos acima usam nulo para indicar a ausência de uma mudança de estado (onde para e de são idênticos). Isso não é uma boa prática e provavelmente resultará em uma NullPointerException em tempo de execução. Projetar uma solução limpa e elegante para esse problema é surpreendentemente complicado, e os programas resultantes são suficientemente longos para prejudicar o material principal desse item.

Em resumo, raramente é apropriado usar ordinais para indexar em arrays: use EnumMap em vez disso. Se a relação que você está representando for multidimensional, use EnumMap<..., EnumMap<...>>. Este é um caso especial do princípio geral de que programadores de aplicações raramente, se é que alguma vez, devem usar o Enum.ordinal (Item 35).

Item 38: Emular enums extensíveis com interfaces

Em quase todos os aspectos, os tipos de enum são superiores ao padrão de enum seguro para tipos descrito na primeira edição deste livro [Bloch01]. À primeira vista, uma exceção diz respeito à extensibilidade, que era possível sob o padrão original, mas não é suportada pelo construto da linguagem. Em outras palavras, usando o padrão, era possível que um tipo enumerado estendesse outro; Usando o recurso de idioma, não é. Isso não é acidente. Na maior parte dos casos, a extensibilidade dos enums acaba sendo uma má ideia. É confuso que elementos de um tipo de extensão sejam instâncias do tipo base e não o contrário. Não há uma boa maneira de enumerar sobre todos os elementos de um tipo base e suas extensões. Por fim, a extensibilidade complicaria muitos aspectos do projeto e da implementação.

Dito isso, há pelo menos um caso de uso convincente para tipos enumerados extensíveis, que são os códigos de operação, também conhecidos como opcodes. Um opcode é um tipo enumerado cujos elementos representam operações em alguma máquina, como o tipo Operação no Item 34, que representa as funções em uma calculadora simples. Às vezes, é desejável permitir que os usuários de uma API forneçam suas próprias operações, estendendo efetivamente o conjunto de operações fornecidas pela API.

Felizmente, existe uma boa maneira de conseguir esse efeito usando tipos enum. A ideia básica é aproveitar o fato de que os tipos de enum podem implementar interfaces arbitrárias definindo uma interface para o tipo de opcode e um enum que é a implementação padrão da interface. Por exemplo, aqui está uma versão extensível do tipo `theOperation` do Item 34:

```
Emulado enum extensível usando uma interface
interface pública Operação {
    Aplicação dupla(duplo x, duplo y);}

public enum BasicOperation implementa Operation {
    MAIS("+") {
        public double apply(double x, double y) { return x + y;
    }, MINUS("-") {

        public double apply(double x, double y) { return x - y;
    }, TIMES("*.") {

        public double apply(double x, double y) { return x * y;
    }, DIVIDE("/") {

        Public double apply(double x, double y) { return x / y; }};
```

```

símbolo final privado de corda;

BasicOperation(símbolo de string) {
    this.symbol = symbol;}

@Override público String toString() {
    símbolo de retorno;}}

```

Embora o tipo enum (`BasicOperation`) não seja extensível, o tipo de interface (`Operation`) é, e é o tipo de interface usado para representar operações em APIs. Você pode definir outro tipo de enum que implemente essa interface e use instances desse novo tipo no lugar do tipo base. Por exemplo, suponha que você queira definir uma extensão do tipo de operação mostrado anteriormente, consistindo em operações de exponenciação e remanescente. Tudo o que você precisa fazer é escrever um enumtype que implemente a interface de Operação:

```

Emulated extension enumpublic enum ExtendedOperation
implementa Operation {
    EXP("^") {
        Public Double Apply(double x, double y) {
            return Math.pow(x,
                y);}, REMAINDER("%") {

        Public Double Apply(double x, double y) {
            retorne x % y;}};

    símbolo final privado de corda;

    ExtendedOperation(Símbolo de string) {
        this.symbol = symbol;}

    @Override público String toString() {
        símbolo de retorno;}}

```

Agora você pode usar suas novas operações em qualquer lugar, pode usar as operações básicas, desde que as APIs sejam escritas para assumir o tipo de interface (`Operation`), e não a implementação (`BasicOperation`). Note que você não precisa declarar o

O código resultante é um pouco menos complexo, e o método de teste é um pouco mais flexível: ele permite ao chamador combinar operações de múltiplos tipos de implementação. Por outro lado, você dispensa a possibilidade de usar EnumSet (Item 36) e EnumMap (Item 37) nas operações especificadas.

Ambos os programas mostrados anteriormente produzirão essa saída quando executados com os argumentos da linha de comunicação 4 e 2:

```
4.000000^2.000000 = 16.0000000  
4.000000 % 2.000000 = 0.000000
```

Uma desvantagem menor do uso de interfaces para emular enums extensíveis é que implementações não podem ser herdadas de um tipo de enum para outro. Se o código de implementação não depender de nenhum estado, ele pode ser colocado na interface, usando implementações padrão (Item 20). No caso do nosso exemplo de Operação, a lógica para armazenar e recuperar o símbolo associado a uma operação deve ser duplicada em BasicOperation e ExtendedOperation. Neste caso, isso não importa porque muito pouco código é duplicado. Se houvesse uma quantidade maior de funcionalidades compartilhadas, você poderia encapsulá-la em uma classe helper ou em um método helper estático para eliminar a duplicação de código.

O padrão descrito neste item é usado nas bibliotecas Java. Por exemplo, o tipo enum `java.nio.file.LinkOption` implementa as interfaces `CopyOption` e `OpenOption`.

Resumindo, embora você não possa escrever um tipo enum extensível, pode emulá-lo escrevendo uma interface para acompanhar um tipo básico de enum que implemente a interface. Isso permite que os clientes escrevam seus próprios enums (ou outros tipos) que implementam a interface. Instâncias desses tipos podem então ser usadas onde quer que instâncias do tipo básico de enum possam ser usadas, assumindo que as APIs sejam escritas em termos da interface.

Item 39: Prefira anotações a padrões de nomeação

Historicamente, era comum usar padrões de nomenclatura para indicar que alguns elementos de programa exigiam tratamento especial por meio de uma ferramenta ou framework. Por exemplo, antes da versão 4, o framework de teste JUnit exigia que seus usuários designassem metanfetaminêtas testadas começando seus nomes com o teste de caracteres [Beck04]. Essa técnica funciona, mas tem várias grandes desvantagens. Primeiro, erros tipográficos resultam em falhas silenciosas. Por exemplo, suponha que você acidentalmente nomeie um método de `testtsetSafetyOverride` em vez de `testSafetyOverride`. JUnit 3 não explicava, mas também não executava o teste, levando a uma falsa sensação de segurança.

Uma segunda desvantagem dos padrões de nomeação é que não há como garantir que eles sejam usados apenas em elementos apropriados do programa. Por exemplo, suponha que você chamasse uma classe de `TestSafetyMechanisms` na esperança de que o JUnit 3 testasse automaticamente todos os seus métodos, independentemente de seus nomes. Novamente, o JUnit 3 não reclamava, mas também não executava os testes.

Uma terceira desvantagem dos padrões de nomeação é que eles não oferecem uma boa forma de associar valores de parâmetros aos elementos do programa. Por exemplo, suponha que você queira suportar uma categoria de teste que só tem sucesso se ela gerar uma exceção específica. O tipo de exceção é essencialmente um parâmetro do teste. Você poderia codificar o nome do tipo de exceção no nome do método de teste usando algum padrão elaborado e amado, mas isso seria feio e frágil (Item 62). O compilador não teria como saber se a string que deveria nomear uma exceção realmente fez isso. Se a classe nomeada não existisse ou não fosse exceção, você só descobriria quando tentasse rodar o teste.

As anotações [JLS, 9.7] resolvem todos esses problemas muito bem, e a JUnit as adotou a partir da versão 4. Neste item, vamos escrever nosso próprio quadro de testes de brinquedos para mostrar como funcionam as anotações. Suponha que você queira definir um annotationtype para designar testes simples que são executados automaticamente e falham se eles lançarem uma exceção. Veja como um tipo de anotação assim, chamado `Test`, pode ser:

```
Tipo de anotação de marcador
declarationimport java.lang.annotation.*;

/**
 * Indica que o método anotado é um método de teste.* Use
 apenas em métodos estáticos sem
 parâmetros.*/@Retention(RetentionPolicy.RUNTIME)@Target(Eleme
 ntType.METHOD)Public @interface Test {}
```

A declaração para o tipo de anotação de Teste é anotada com anotações Retention e Target. Tais anotações em declarações de tipo anotação são conhecidas como meta-anotações. A meta-anotação `@Retention(RetentionPolicy.RUN-TIME)` indica que as anotações de Teste devem ser mantidas em tempo de execução. Sem ele, as anotações de teste seriam invisíveis para a ferramenta de teste.

The `@Target.get(ElementType.METHOD)` a meta-anotação indica que a `Testannotation` é legal apenas em declarações de métodos: ela não pode ser aplicada a declarações de classe, declarações de campo ou outros elementos do programa.

O comentário antes da declaração de anotação de teste diz: "Use apenas métodos estáticos `onparameterless`." Seria bom se o compilador pudesse impor isso, mas não pode, a menos que você escreva um processador de anotação para isso. Para mais informações sobre esse tema, veja a documentação do `javax.annotation.processing`. Na ausência de tal processador de anotação, se você colocar uma anotação de teste na declaração de um método de instância ou em um método com um ou mais parâmetros, o programa de teste ainda compilará, deixando para a ferramenta de teste lidar com o problema em tempo de execução.

Veja como a anotação de Teste é na prática. Ela é chamada de anotação marcadora porque não possui parâmetros, mas simplesmente "marca" o elemento anotado. Se o programador escrevesse incorretamente `Test` ou aplicasse a anotação `Test` a um elemento `program` que não fosse uma declaração de método, o programa não compilaria:

```
Programa contendo marcadores
anotaçõespublic Sample {
    @Test vazio estático público m1() { } O teste deve passar
    Void estático público m2() { }@Test
    Void estático público m3() {          O teste deve falhar
        lançar um novo RuntimeException("Boom");empty estático público
    m4() { }@Test void público m5() { } // USO INVÁLIDO: método não
    estático void estático público m6() { }@Test void estático
    público m7() { // Teste deve falhar

    lançar um novo RuntimeException("Crash");}Void
    estático público m8() { }}
```

A classe `Sample` possui sete métodos estáticos, quatro dos quais são `astests` anotados. Dois desses, `m3` e `m7`, fazem exceções de lançamento, e dois, `m1` e `m5`, não. Mas um dos métodos anotados que não lança uma exceção, `m5`, é um método de instância, então não é um uso válido da anotação. Em resumo, o `Sample` contém quatro testes: um passará, dois falharão e um é inválido. Os quatro métodos que são anotados com a anotação de teste serão ignorados pela ferramenta de teste.

As anotações de Teste não têm efeito direto sobre a semântica da Classe de Amostra. Eles servem apenas para fornecer informações para uso de programas interessados. De forma mais geral, as anotações não alteram a semântica do código anotado, mas permitem que ele seja tratado de forma especial por ferramentas como este simples executor de testes:

```
Programa para processar anotações de
marcadores importam java.lang.reflect.*;

classe pública RunTests {
    public static void main(String[] args) throws Exception {
        testes de inteligência = 0; Int passado = 0; Class<?>
        testClass = Class.forName(args[0]); para (Método m :
        testClass.getDeclaredMethods()) {

            if (m.isAnnotationPresent(Test.class)) {
                tests++;
                try {
                    m.invoke(nulo); passed++;} catch
                    (InvocationTargetException wrappedExc) {

                        Exc arremessável = wrappedExc.getCause();
                        System.out.println(m + " failed: " + exc);} catch
                        (Exceção exc) {
                            System.out.println("Invalid @Test: " +
                                m);}}System.out.printf("Passado: %d, Reprovado: %d\n",

                                                                    aprovado, testes - aprovado);
            }}
}
```

A ferramenta de execução de testes assume um nome de classe totalmente qualificado na linha de comando e executa todos os métodos anotados por teste da classe de forma reflexiva, chamando `Method.invoke`. O método `isAnnotationPresent` informa à ferramenta quais métodos executar. Se um método de teste lançar uma exceção, a função de reflexão a envolve em uma `InvocationTargetException`. A ferramenta detecta essa exceção e imprime um relatório de falha contendo a exceção original lançada pelo método de teste, que é extraída do `InvocationTargetException` com o método `getCause`.

Se uma tentativa de invocar um método de teste por reflexão lançar qualquer exceção que não seja `InvocationTargetException`, isso indica um uso inválido da anotação de teste que não foi capturado em tempo de compilação. Esses usos incluem a anotação do método de instância, de um método com um ou mais parâmetros, ou de um método inacessível. O segundo bloco de captura no teste executa esses erros de uso do teste

e imprime uma mensagem de erro apropriada. Aqui está o resultado que é impresso se o `ifRunTests` é executado em `Sample`:

```
empty estático público Sample.m3() falhou: RuntimeException:
BoomInvalid @Test: empty público Sample.m5()empty estático público
Sample.m7() falhou: RuntimeException: CrashPassed: 1, Failed: 3
```

Agora vamos adicionar suporte para testes que só têm sucesso se eles lançarem uma exceção específica. Vamos precisar de um novo tipo de anotação para isso:

```
Tipo de anotação com um
parameterimport
java.lang.annotation.*;**
* Indica que o método anotado é um método de teste que* deve
lançar a exceção designada para ter
sucesso.*/@Retention(RetentionPolicy.RUNTIME)@Target(ElementType.
METHOD)public @interface ExceptionTest {
```

```
Class<? estende Throwable> value();}
```

O tipo do parâmetro para essa anotação é `Class<? extends Throwable>`. Esse tipo imprevisível é, admitidamente, um nome complicado. Em inglês, significa "o objeto `Class` for some class that extends `Throwable`" e permite ao usuário da anotação especificar qualquer tipo de exceção (ou erro). Esse uso é um exame de um token do tipo limitado (Item 33). Veja como a anotação é na prática. Note que literais de classe são usados como valores para o parâmetro de anotação:

```
Programa contendo anotações com uma classe
parameterpublic Sample2 {
    @ExceptionTest(ArithmeticException.class)empty
    estático público m1() { // Teste deve passar

    int i = 0; i = i /
    i;}@ExceptionTest(ArithmeticException.class)public static void
    m2() { // Deve falhar (exceção errada)
```

```
int[] a = novo int[0]; int i =
a[1];}@ExceptionTest(ArithmeticException.class)empty estático
público m3() { } // Deve falhar (sem exceção)}
```

Agora vamos modificar a ferramenta de teste para processar a nova anotação. Fazer isso consiste em adicionar o seguinte código ao método principal:

```
if (m.isAnnotationPresent(ExceptionTest.class)) {
    tests++;
    try {
        m.invoke(nulo); System.out.printf("Testes %s falhados: sem
        exceção%n", m);} catch (InvocationTargetException wrappedEx) {

            Exc arremessável = wrappedEx.getCause();
            Class<? estende Throwable> excType =
            m.getAnnotation(ExceptionTest.class).value(); if
            (excType.isInstance(exc)) {
                passed++;}
            else {
                System.out.printf(
                    "Testes %s falharam: esperados %s, obteve
                    %s%n",m, excType.getName(), exc);
            }} catch (Exceção exc) {

                System.out.println("Invalid @Test: " + m);}}
```

Esse código é semelhante ao que usamos para processar anotações de teste, com uma exceção: esse código extrai o valor do parâmetro de anotação e o usa para verificar se a exceção lançada pelo teste é do tipo correto. Não há explicitcasts, e portanto não há perigo de uma `ClassCastException`. O fato de o `testprogram` compilado garantir que seus parâmetros de anotação representam tipos `validexception`, com uma ressalva: se os parâmetros de anotação eram válidos em tempo de compilação, mas o arquivo de classe que representa um tipo de exceção especificado não estiver mais presente em tempo de execução, o executor de teste lançará `TypeNotPresentException`.

Levando nosso exemplo de teste de exceção um passo adiante, é possível imaginar um teste que passa se ele lançar qualquer uma de várias exceções especificadas. O mecanismo de anotação possui uma funcionalidade que facilita o suporte a esse uso. Suponha que mudemos o tipo de parâmetro da anotação `ExceptionTest` para um array de objetos de Classe:

```
Tipo de anotação com um array
parameter@Retention(RetentionPolicy.RUNTIME)@
Target(ElementType.METHOD)public @interface
ExceptionTest {
    Class<? estende Exception>[] value();}
```

A sintaxe dos parâmetros do array em anotações é flexível. Ele é otimizado para arrays de elemento único. Todas as anotações anteriores do `ExceptionTest` ainda são válidas com a nova versão de parâmetros de array do `ExceptionTest` e resultam em arrays de elemento único. Para especificar um array de múltiplos elementos, envolva os elementos com colchetes curvados e separe-os com vírgulas:

```
Código contendo uma anotação com um array
parameter@ExceptionTest({ IndexOutOfBoundsException.class,
                          NullPointerException.class })
público estático void doublelyBad() {
    <String> Lista de listas = nova ArrayList<>();

    A especificação permite que esse método lance ou//
    IndexOutOfBoundsException ou
    NullPointerExceptionlist.addAll(5, null);}
```

É razoavelmente simples modificar a ferramenta de execução de testes para processar a nova versão do `ExceptionTest`. Este código substitui a versão original:

```
if (m.isAnnotationPresent(ExceptionTest.class)) {
    tests++;
    try {
        m.invoke(nulo); System.out.printf("Testes %s falhados: sem
        exceção%n", m);} catch (Throwable wrappedExc) {

            Exc arremessável = wrappedExc.getCause();
            int oldPassed = passado; Class<? estende
            Exceção>[] excTipos =
            m.getAnnotation(ExceptionTest.class).value(); for
            (Class<? estende Exception> excType : excTypes) {
                if (excType.isInstance(exc)) {
                    aprovado++; pausa;}}if
                    (passed == oldPassed)

            System.out.printf("Teste %s falhado: %s %n", m, exc);
        }}
```

A partir do Java 8, há outra forma de fazer anotações multivaloradas. Em vez de declarar um tipo de anotação com um parâmetro de array, você pode anotar a declaração de uma anotação com a `@Repeatable` meta-anotação, para indicar que a anotação pode ser aplicada repetidamente a um único elemento. Essa meta-anotação

toma um único parâmetro, que é o objeto de classe de um tipo de anotação contendo, cujo único parâmetro é um array do tipo de anotação [JLS, 9.6.3]. Veja como as declarações de anotação ficam se adotarmos essa abordagem com nossa `ExceptionTest` annotation. Note que o tipo de anotação contendo deve ser anotado com uma política de retenção e um destino apropriados, ou as declarações não serão compiladas:

```
Anotação repetível
type@Retention(RetentionPolicy.RUNTIME)@Target(
    ElementType.METHOD)@Repeatable(ExceptionTestContainer.class)public @interface
ExceptionTest {
    Class<? estende Exception> value();}
```

```
@Retention(RetentionPolicy.RUNTIME)@Target(
    ElementType.METHOD)public @interface
ExceptionTestContainer {
    ExceptionTest[] value();}
```

Veja como nosso teste `doublyBad` se apresenta com uma anotação repetida no lugar da anotação `anarray`:

```
Código contendo um
annotation@ExceptionTest(IndexOutOfBoundsException.
class)@ExceptionTest(NullPointerException.class)emp
ty estático público doublyBad() { ... }
```

Processar anotações repetíveis exige cuidado. Uma anotação repetida gera uma anotação sintética do tipo de anotação contendo. O método `ThegetAnnotationsByType` ignora esse fato e pode ser usado para acessar tanto anotações repetidas quanto não repetidas de um tipo de anotação repetível. `ButisAnnotationPresent` deixa explícito que anotações repetidas não são do tipo de anotação, mas sim do tipo de anotação que contém. Se um elemento tem anotação repetida de algum tipo e você usa o método `isAnnotationPresent` para verificar se o elemento tem uma anotação desse tipo, você verá que não tem. Usar esse método para verificar a presença de um tipo de anotação fará com que seu programa ignore silenciosamente as anotações repetidas. Da mesma forma, usar esse método para verificar o tipo de anotação contendo fará com que o programa ignore silenciosamente anotações não repetidas. Para detectar anotações repetidas e não repetidas com `isAnnotationPresent`, você deve verificar tanto o `annotationtype` quanto o tipo de anotação que o contém dele. Veja como a parte relevante do nosso

O programa `RunTests` parece quando modificado para usar a versão repetível da anotação `theExceptionTest`:

```
Processando anotações repetíveis
(m.isAnnotationPresent(ExceptionTest.class))
    || m.isAnnotationPresent(ExceptionTestContainer.class))
{tests++; try {

    m.invoke(nulo); System.out.printf("Testes %s falhados: sem
    exceção%n", m);} catch (Throwable wrappedExc) {

        Exc arremessável = wrappedExc.getCause();
        int oldPassed = passado; ExceptionTest[]
        excTests =

            m.getAnnotationsByType(ExceptionTest.class);
        for (ExceptionTest excTest : excTests) {
            if (excTest.value().isInstance(exc)) {
                aprovado++; pausa;}}if
                (passed == oldPassed)

        System.out.printf("Teste %s falhado: %s %n", m, exc);

    }}
```

Anotações repetíveis foram adicionadas para melhorar a legibilidade do código-fonte que aplica logicamente múltiplas instâncias do mesmo tipo de anotação a um determinado elemento do programa. Se você acha que elas melhoram a legibilidade do seu código-fonte, use-as, mas lembre-se de que há mais padrão na declaração e processamento de anotações repetíveis, e que o processamento de anotações repetíveis é propenso a erros.

O framework de testes neste item é apenas um brinquedo, mas demonstra claramente a superioridade das anotações sobre os padrões de nomeação, e só arranha a superfície do que você pode fazer com elas. Se você escreve uma ferramenta que exige que programadores adicionem informações ao código-fonte, defina os tipos de anotação apropriados. Simplesmente não há razão para usar padrões de nomes quando você pode usar anotações em vez disso.

Dito isso, com exceção dos ferramenteiros, a maioria dos programadores não precisará definir tipos de anotação. Mas todos os programadores devem usar os tipos pré-definidos de anotação que o Java fornece (Itens 40, 27). Além disso, considere usar as anotações fornecidas pelo seu IDE ou ferramentas de análise estática. Tais anotações podem melhorar a qualidade das informações diagnósticas fornecidas por essas ferramentas. Note, no entanto, que essas anotações ainda não foram padronizadas, então você pode ter algum trabalho a fazer se trocar de ferramenta ou se surgir um padrão.

Item 40: Use consistentemente a anotação Override

As bibliotecas Java contêm vários tipos de anotação. Para o programador típico, o mais importante deles é `@Override`. Essa anotação pode ser usada apenas em declarações de método, e indica que a declaração de método anotada sobrepõe-se a uma declaração em um supertipo. Se você usar essa anotação consistentemente, ela vai te proteger de uma grande classe de insetos nefastos. Considere este programa, no qual a classe `Bigram` representa um bigrama, ou par ordenado de letras:

```
Você consegue identificar o
bug?
classe pública Bigram {
    Soldado último Char primeiro;
    soldado de último char segundo;

    Bigram público (char primeiro, char segundo) {
        isso.primeiro = primeiro;
        this.second = second; }
    booleano
    público é igual a (Bigram b) {

        retorno b.primeiro == primeiro && b.segundo ==
        segundo; }
    public int hashCode() {

        retorno 31 * primeiro + segundo; }

    public static void main(String[] args) {
        Set<Bigram> s = novo HashSet<>();
        para (int i = 0; i < 10; i++)
            para (char ch = 'a'; ch <= 'z'; ch++)
                s.add (novo Bigram(ch, ch));
        System.out.println(s.size()); } }
```

O programa principal adiciona repetidamente vinte e seis bigramas, cada um consistindo em duas letras minúsculas idênticas, a um conjunto. Depois, ele imprime o tamanho do conjunto. Você pode esperar que o programa imprima 26, já que os conjuntos não podem conter duplicados. Se você tentar rodar o programa, vai perceber que ele não imprime 26, mas 260. O que há de errado com ele?

Claramente, o autor da classe `Bigram` pretendia sobrescrever o `equalsmethod` (Item 10) e até lembrou de sobrescrever `hashCode` em conjunto (Item 11). Infelizmente, nosso programador desajeitado não conseguiu sobrescrever o `equal`, sobrecarregando (Item 52). Para sobrescrever `Object.equals`, você deve definir o método `anequals` cujo parâmetro é do tipo `Object`, mas o parâmetro de `Bigram`

o método `equals` não é do tipo `Object`, então `Bigram` herda o método `equals` de `Object`. Isso equivale a testes de método para identidade de objeto, assim como o operador `==`. Cada uma das dez cópias de cada bigrama é distinta das outras nove, então são consideradas desiguais pelo `Object.equals`, o que explica por que o programa imprime 260. Felizmente, o compilador pode ajudar você a encontrar esse erro, mas só se você ajudar informando que pretende sobrescrever `Object.equals`. Para isso, anote `Bigram` igual a `@Override`, como mostrado aqui:

```
@Override booleano público é igual a (Bigram b) {
    retorno b.primeiro == primeiro && b.segundo ==
    segundo;}

```

Se você inserir essa anotação e tentar recompilar o programa, o compilador gerará uma mensagem de erro como esta:

```
Bigram.java:10: método não substitui nem implementa um método de um
supertipo
    @Override booleano público é igual a (Bigram
    b) {^

```

Você vai perceber imediatamente o que fez de errado, se dar um tapa na testa e substituir a implementação de 'broken equal' por uma correta (Item 10):

```
@Override booleano público é igual a (Object o) {
    se (!( o instância de Bigram))
    retorne falso; Bigrama b = (Bigrama) o; retorno
    b.primeiro == primeiro && b.segundo == segundo;}

```

Portanto, você deve usar a anotação `Override` em toda declaração de método que acredite sobrepor uma declaração de superclasse. Há uma exceção menor a essa regra. Se você está escrevendo uma classe que não é rotulada como abstrata e acredita que ela sobrepe um método abstrato em sua superclasse, não precisa se preocupar em colocar a anotação `Override` nesse método. Em uma classe que não é declarada abstrata, o compilador emitirá uma mensagem de erro se você não sobrescrever um método `abstractsuperclass`. No entanto, talvez queira chamar atenção para todos os métodos da sua turma que sobrepe métodos de superclasse, caso contrário, sintase à vontade para anotar esses métodos também. A maioria dos IDEs pode ser configurada para inserir anotações de `Override` automaticamente quando você opta por sobrepor um método.

A maioria dos IDEs oferece outro motivo para usar a anotação `Override` de forma consistente. Se você ativar a verificação apropriada, o IDE gerará um aviso se você tiver um método que não tem anotação de `Override`, mas que substitui um método de superclasse. Se você usar a anotação `Override` consistentemente, esses avisos vão alertar sobre `overwriting` não intencional. Eles complementam as mensagens de erro do compilador, que alertam sobre falha não intencional em sobrescrever. Entre o IDE e o compilador, você pode ter certeza de que está sobrescrevendo métodos em todos os lugares que quiser e em nenhum outro lugar.

A anotação `Override` pode ser usada em declarações de método que substituem declarações de interfaces, bem como de classes. Com o advento dos `methods padrão`, é uma boa prática usar o `Override` em implementações concretas de métodos de interface para garantir que a assinatura esteja correta. Se você souber que uma interface não possui métodos padrão, pode optar por omitir anotações de `Override` em implementações concretas de métodos de interface para reduzir a bagunça.

Em uma classe abstrata ou interface, no entanto, vale a pena anotar todos os métodos que você acredita sobrepor métodos de superclasse ou superinterface, sejam concretos ou abstratos. Por exemplo, a interface `Set` não adiciona novos métodos à interface da `Coleção`, então deve incluir anotações de `Override` em todas as declarações de seu método para garantir que não adicione acidentalmente novos métodos à interface da `Coleção`.

Em resumo, o compilador pode protegê-lo de muitos erros se você usar a anotação `Override` em toda declaração de método que acreditar sobrepor uma declaração de supertipo, com uma exceção. Em aulas concretas, é necessário não anotar métodos que acredite sobrepor declarações abstratas de métodos (embora isso não seja prejudicial).

Item 41: Usar interfaces de marcadores para definir tipos

Uma interface marcador é uma interface que não contém declarações de método, mas apenas designa (ou "marca") uma classe que implementa a interface como tendo alguma propriedade. Por exemplo, considere a interface `Serializable` (Capítulo 12). Ao implementar essa interface, uma classe indica que suas instâncias podem ser escritas em um `ObjectOutputStream` (ou "serializadas").

Você pode ouvir dizer que as anotações de marcadores (Item 39) tornam as interfaces de marcadores obsoletas. Essa afirmação está incorreta. Interfaces de marcadores têm duas vantagens em relação às anotações de marcadores. Primeiramente, interfaces de marcadores definem um tipo que é implementado por instâncias da classe marcada; Anotações de marcadores não importam. A existência de um tipo de interface de marcador permite que você detecte erros em tempo de compilação que você só poderia detectar se usasse uma anotação de marcadores.

A funcionalidade de serialização do Java (Capítulo 6) usa a interface do marcador `Serializable` para indicar que um tipo é serializável. O método `ObjectOutputStream.writeObject`, que serializa o objeto que lhe é passado, exige que seu argumento seja serializável. Se o argumento desse método fosse do tipo `Serializable`, uma tentativa de serializar um objeto inadequado teria sido detectada em tempo de compilação (por meio de verificação de tipos). A detecção de erros em tempo de compilação é a intenção dos interfaces de marcadores, mas infelizmente, a API `ObjectOutputStream.writeObject` não aproveita a interface `Serializable`: seu argumento é declarado como de tipo `Object`, então tentativas de serializar um objeto não serializável não falham até o tempo de execução.

Outra vantagem das interfaces de marcadores em relação às anotações de marcadores é que elas podem ser direcionadas com mais precisão. Se um tipo de anotação for declarado com `targetElementType.TYPE`, ele pode ser aplicado a qualquer classe ou interface. Suponha que você tenha um marcador aplicável apenas a implementações de uma interface específica. Se você a definir como uma interface de marcador, pode estender a interface única à qual é aplicável, garantindo que todos os tipos marcados também sejam subtipos da interface soleada à qual é aplicável.

Pode-se argumentar que a interface `Set` é exatamente essa interface de marcadores restrita. Ele é aplicável apenas a subtipos de `Coleção`, mas não adiciona métodos além daqueles definidos por `Coleção`. Geralmente, não é considerada uma interface de marcador porque refina os contratos de vários métodos de `Coleção`, incluindo `soma`, `igual` e `hashCode`. Mas é fácil imaginar uma interface marcadora que seja aplicável apenas a subtipos de alguma interface específica e que não refina os contratos de nenhum dos métodos dessa interface. Tal interface de marcador pode descrever algum invariante de todo o objeto ou indicar que as instâncias são elegíveis para processamento

por um método de alguma outra classe (da mesma forma que a interface `Serializable` indica que as instâncias são elegíveis para processamento pelo `ObjectOutputStream`).

A principal vantagem das anotações de marcadores em relação às interfaces de marcadores é que elas fazem parte da maior facilidade de anotação. Portanto, as anotações de marcadores permitem consistência em frameworks baseados em anotação.

Então, quando você deve usar uma anotação de marcador e quando deve usar uma interface de marcador? Claramente, você deve usar uma anotação se o marcador se aplica a qualquer elemento do programa que não seja uma classe ou interface, porque apenas classes e interfaces podem ser feitas para implementar ou estender uma interface. Se o marcador se aplica apenas a classes e interfaces, pergunte a si mesmo: "Será que eu gostaria de escrever um ou mais métodos que aceitem apenas objetos que possuem essa marcação?" Se sim, você deve usar uma interface de marcador em vez de uma anotação. Isso permitirá que você use a interface como tipo de parâmetro para os métodos em questão, o que resultará no benefício da verificação de tipos em tempo de compilação. Se você conseguir se convencer de que nunca vai querer escrever um método que aceite apenas objetos com `themarking`, então provavelmente é melhor usar uma anotação de marcadores. Se, além disso, a marcação faz parte de uma estrutura que faz uso intensivo de anotações, então uma anotação de marcador é a escolha clara.

Em resumo, interfaces de marcadores e anotações de marcadores têm suas utilidades. Se você quiser definir um tipo que não tenha novos métodos associados, a interface `amarker` é o caminho a seguir. Se você quiser marcar elementos do programa além de classes e interfaces ou encaixar o marcador em uma estrutura que já faz uso intenso de tipos de anotação, então uma anotação de marcador é a escolha correta. Se você se pegar escrevendo um tipo de anotação de marcador cujo alvo é `ElementType.TYPE`, reserve um tempo para descobrir se realmente deve ser o tipo de anotação ou se uma interface de marcador seria mais apropriada.

De certa forma, esse item é o inverso do item 22, que diz: "Se você não quiser definir um tipo, não use uma interface." Em uma primeira aproximação, este item diz: "Se você quiser definir um tipo, use uma interface."

CHAPTER 7

Lambdas e Riachos

No Java 8, interfaces funcionais, lambdas e referências de métodos foram adicionadas para facilitar a criação de objetos de função. A API de streams foi adicionada em conjunto com essas mudanças de linguagem para fornecer suporte a bibliotecas para processamento de sequências de elementos de dados. Neste capítulo, discutimos como aproveitar ao máximo essas instalações.

Item 42: Prefira lambdas a classes anônimas

Historicamente, interfaces (ou, raramente, classes abstratas) com um único método abstrato eram usadas como tipos de função. Suas instâncias, conhecidas como objetos função, representações ou ações. Desde que o JDK 1.1 foi lançado em 1997, o principal meio de criar um objeto função era a classe anônima (Item 24). Aqui está um trecho de código para ordenar uma lista de strings em ordem de comprimento, usando uma classe anônima para criar a função de comparação do sorting (que impõe a ordem de ordenação):

```
Instância anônima de classe como objeto função - obsoleta!
Collections.sort(words, new Comparator<String>() {
    public int compare(String s1, String s2) {
        return Integer.compare(s1.length(), s2.length());});
```

Classes anônimas eram adequadas para os clássicos padrões de design orientados a objetos que exigiam objetos de função, notadamente o padrão Estratégia [Gamma95]. A interface `TheComparator` representa uma estratégia abstrata para ordenação; A classe anônimo acima é uma estratégia concreta para ordenar cadeias de caracteres. A verbosidade das classes anônimas, no entanto, tornava a programação funcional em Java uma perspectiva pouco atraente.

No Java 8, a linguagem formalizou a noção de que interfaces com um método `singleabstract` são especiais e merecem tratamento especial. Essas interfaces agora são conhecidas como interfaces funcionais, e a linguagem permite criar instâncias dessas interfaces usando expressões lambda, ou lambdas para abreviar.

Lambdas são semelhantes em função a classes anônimas, mas muito mais concisas. Veja como o trecho de código acima fica com a classe anônima substituída por alambda. O padrão padrão desapareceu, e o comportamento é claramente evidente:

```
Expressão lambda como objeto de função (substitui a classe
anônima)Collections.sort(words,
    (s1, s2) -> Integer.compare(s1.length(), s2.length()));
```

Note que os tipos do lambda (`Comparador<String>`), de seus parâmetros (`s1` e `s2`, ambos `String`) e de seu valor de retorno (`int`) não estão presentes no código. O compilador deduz esses tipos a partir do contexto, usando um processo conhecido como inferência de tipos. Em alguns casos, o compilador não conseguirá determinar os tipos, e você terá que especificá-los. As regras para inferência de tipos são complexas: elas ocupam um capítulo inteiro na JLS [JLS, 18]. Poucos programadores entendem essas regras em detalhes, mas tudo bem. Omita os tipos de todos os parâmetros lambda, a menos que a presença deles deixe seu programa mais claro. Se o compilador gerar um erro dizendo que não consegue inferir o tipo de parâmetro lambda, então especifique-o. Às vezes você pode precisar lançar o valor de retorno ou toda a expressão lambda, mas isso é raro.

Uma ressalva deve ser acrescentada sobre inferência de tipo. O Item 26 diz para não usar tipos brutos, o Item 29 diz para você preferir tipos genéricos, e o Item 30 orienta para preferir métodos genéricos. Esse conselho é duplamente importante quando você usa lambdas, porque o compilador obtém a maior parte das informações de tipo que permitem realizar inferência de tipos a partir de genéricos. Se você não fornecer essa informação, o `thcompiler` não poderá fazer inferência de tipos, e você terá que especificar os tipos manualmente nos seus lambdas, o que aumentará muito a verbosidade deles. Por exemplo, o trecho de código acima não será compilado se as palavras da variável forem declaradas como do tipo bruto `List` em vez do tipo parametrizado `<String>List`.

A propósito, o comparador no snippet pode ser ainda mais sucinto se o método de construção do comparador for usado no lugar de um lambda (Itens 14. 43):

```
Collections.sort(words, comparingInt(String::length));
```

Na verdade, o trecho pode ser ainda mais curto aproveitando o `sortmethod` que foi adicionado à interface `List` no Java 8:

```
words.sort(comparingInt(String::length));
```

A adição de lambdas à linguagem torna prático usar objetos de função onde antes não faria sentido. Por exemplo, considere o tipo de enum da Operação no Item 34. Como cada enum exigia comportamentos diferentes para seu método de aplicação, usamos corpos de classe específicos de constantes e sobrescrevemos o método de aplicação em cada constante de enum. Para refrescar sua memória, aqui está o código:

Tipo enum com corpos de classe e dados específicos constantes (Item 34)

```
public enum Operation {
    MAIS("+") {
        public double apply(double x, double y) { return x + y;
    }, MINUS("-") {

        public double apply(double x, double y) { return x - y;
    }, TIMES("*") {

        public double apply(double x, double y) { return x * y;
    }, DIVIDE("/") {

        Public double apply(double x, double y) { return x / y; }};

    símbolo final privado de corda; Operação(Símbolo de
    string) { this.symbol = symbol; } @Override String
    toString() { símbolo de retorno; }
```

Resumo Público Double Apply(Double X, Double Y);}

O item 34 diz que campos de instância enum são preferíveis a classbodies específicos de constantes. Lambdas facilitam a implementação de comportamento específico constante usando thefirst em vez do segundo. Basta passar um lambda implementando o comportamento de cada enumconstante para seu construtor. O construtor armazena o lambda em um campo de instância, e o método apply encaminha as invocações para o lambda. O código resultante é mais simples e claro do que a versão original:

Enum com campos de objeto de função & comportamento específico constante enum público Operação

```
{
    MAIS("+", (x, y) -> x +
    y), MENOS("-", (x, y) -> x -
    y), TIMES("*", (x, y) -> x *
    y), DIVIDE("/", (x, y) -> x / y);

    símbolo final privado de corda; operação privada final
    DoubleBinaryOperator;

    Operation(String symbol, DoubleBinaryOperator op) {
        this.symbol = symbol;
        this.op = op; }

    @Override String toString() { símbolo de retorno; }

    Public Double Apply(double x, double y) {
        return op.applyAsDouble(x, y); }}
```


Note que estamos usando a interface `DoubleBinaryOperator` para os lambdas que representam o comportamento da constante `enum`. Esta é uma das muitas interfaces pré-funcionais em `java.util.function` (Item 44). Ela representa uma função que toma dois argumentos duplos e retorna um resultado duplo.

Olhando para o `enum` de Operação baseado em lambda, você pode pensar que corpos de método específico de constantes já perderam sua utilidade, mas não é esse o caso. Ao contrário de métodos e classes, as lambdas carecem de nomes e documentação; Se a computação não for autoexplicativa ou ultrapassar algumas linhas, não coloque no `Alambda`. Uma linha é ideal para um lambda, e três linhas é um máximo razoável. Se você violar essa regra, pode causar sérios danos à legibilidade dos seus programas. Se um lambda for longo ou difícil de ler, encontre uma forma de simplificá-lo ou refatore seu programa para eliminá-lo. Além disso, os argumentos passados aos `enumconstructors` são avaliados em um contexto estático. Assim, lambdas no `enum` `constructorscan't access members` de instância do `enum`. Corpos de classe específicos para constantes ainda são o caminho a seguir se um tipo de `enum` tem um comportamento específico de constante difícil de entender, que não pode ser implementado em poucas linhas, ou que exige acesso a campos ou métodos de instância.

Da mesma forma, você pode pensar que classes anônimas estão obsoletas na era dos `flambdas`. Isso é mais próximo da verdade, mas há algumas coisas que você pode fazer com classes anônimas que não pode fazer com lambdas. Lambdas são limitadas a interfaces funcionais. Se você quiser criar uma instância de uma classe abstrata, pode fazer isso com uma classe anônima, mas não com uma lambda. Da mesma forma, você pode usar classes `anonymousclasses` para criar instâncias de interfaces com múltiplos métodos abstratos. Por fim, a `alambda` não pode obter uma referência a si mesma. Em um lambda, essa palavra-chave se refere à instância de anexo, que normalmente é o que você deseja. Em uma classe anônima, essa palavra-chave refere-se à instância da classe anônima. Se você precisa de acesso ao objeto função a partir de dentro do corpo dele, então deve usar uma classe anônima.

Lambdas compartilham com classes anônimas a propriedade de que você não pode serializá-las e desserializá-las de forma confiável entre implementações. Portanto, raramente, se é que alguma vez, você deve serializar um lambda (ou uma instância de classe anônima). Se você tem um objeto de função que deseja tornar serializável, como um `Comparador`, use uma instância de uma classe estática privada aninhada (Item 24).

Em resumo, a partir do Java 8, lambdas são de longe a melhor forma de representar objetos `smallfunction`. Não use classes anônimas para objetos de função, a menos que você precise criar instâncias de tipos que não sejam interfaces funcionais. Além disso, lembre-se que os lambdas facilitam muito a representação de pequenos objetos de função que isso abre a porta para técnicas de programação funcional que antes não eram práticas em Java.

Item 43: Preferência de referências de método para lambdas

A principal vantagem das lambdas em relação às classes anônimas é que elas são mais sucintas. Java oferece uma forma de gerar objetos de função ainda mais sucintos do que as lambdas: referências de métodos. Aqui está um trecho de código de um programa que mantém um mapa de chaves arbitrárias para valores inteiros. Se o valor for interpretado como a contagem do número de instâncias da chave, então o programa é uma implementação multiconjunto. A função do trecho de código é associar o número 1 à chave se ela não estiver no mapa e incrementar o valor associado se a chave já estiver presente:

```
map.merge(chave, 1, (contagem, incr) -contagem > + incr);
```

Note que esse código usa o método de fusão, que foi adicionado ao `MapInterface` no Java 8. Se não houver mapeamento para a chave dada, o método simplesmente insere o valor dado; Se já houver um mapeamento, o `merge` aplica a função dada ao valor atual e ao valor dado e sobrescreve o valor atual com o resultado. Esse código representa um caso de uso típico para o método de `merge`.

O código é agradável, mas ainda há alguns clichês padrão. Os parâmetros `contagem` e `incr` não agregam muito valor e ocupam bastante espaço. Na verdade, tudo o que a lambda diz é que a função retorna a soma de seus dois argumentos. A partir do Java 8, `Integer` (e todos os outros tipos numéricos primitivos em caixa) produz uma soma de método estática que faz exatamente a mesma coisa. Podemos simplesmente fazer referência a esse método e obter o mesmo resultado com menos desordem visual:

```
map.merge(chave, 1, Integer::sum);
```

Quanto mais parâmetros um método tiver, mais padrão você pode eliminar com uma referência de método. Em algumas lambdas, entretanto, os nomes dos parâmetros que você escolhe fornecem documentação útil, tornando a lambda mais legível e sustentável do que uma referência de método, mesmo que a lambda seja mais longa.

Não há nada que você possa fazer com uma referência de método que não possa fazer também com um lambda (com uma exceção obscura — veja JLS, 9.9-2 se estiver curioso). Dito isso, referências a métodos geralmente resultam em códigos mais curtos e claros. Eles também dão uma saída se um lambda ficar muito longo ou complexo: você pode extrair o código do `thelambda` para um novo método e substituir o `lambda` por uma referência a esse método. Você pode dar um bom nome ao método e documentá-lo à vontade.

Se você estiver programando com um IDE, ele oferecerá substituir um `lambda` por uma referência de método sempre que possível. Você normalmente, mas nem sempre, deve aceitar a oferta do IDE. Ocasionalmente, um `lambda` será mais sucinto do que um método de referência. Isso acontece mais frequentemente quando o método está na mesma classe do

Lambda. Por exemplo, considere este trecho, que se presume ocorrer em uma classe chamada `GoshThisClassNameIsHumongous`:

```
service.execute(NossaQueNomeClasseÉEnorme::ação);
```

O equivalente lambda é o seguinte:

```
service.execute(() -> ação());
```

O trecho que usa a referência do método não é nem mais curto nem mais claro do que o tesouro usando a lambda, então prefiro o segundo. De forma semelhante, a `FunctionInterface` fornece um método genérico de fábrica estática para retornar a função identidade, `Function.identity()`. Normalmente é mais curto e mais limpo não usar esse método, mas sim codificar o equivalente lambda inline: `x -> x`.

Muitas referências a métodos se referem a métodos estáticos, mas existem quatro tipos que não o fazem. Dois deles são referências de métodos de instância vinculada e não vinculada. Em `boundReferences`, o objeto receptor é especificado na referência do método. As referências de ligação são semelhantes em natureza a referências estáticas: o objeto função assume os mesmos argumentos do método referenciado. Em referências não vinculadas, o objeto receptor é especificado quando o objeto função é aplicado, por meio de um parâmetro adicional antes dos parâmetros declarados pelo método. Referências não vinculadas são frequentemente usadas como `asmapeamento` e funções de filtro em pipelines de fluxo (Item 45). Por fim, existem dois tipos de referências de construtores, para classes e arrays. Referências de construtor servem como objetos de fábrica. Todos os cinco tipos de referências de métodos estão resumidos na tabela abaixo:

Tipo de referência do método	Exemplo	Equivalente Lambda
Estática	<code>Integer::parseInt</code>	<code>str -> Integer.parseInt(str)</code>
Bound	<code>Instant.now()::isAfter</code>	<code>Instantâneo então = Instante.agora(); t -> então.éDepois(t)</code>
Livre	<code>String::toLowerCase</code>	<code>str -> str.toLowerCase()</code>
Construtor de Classe	<code>TreeMap<K,V>::new</code>	<code>() -> novo TreeMap<K,V></code>
Construtor de Array	<code>int[]::novo</code>	<code>len -> novo int[len]</code>

Em resumo, referências de métodos frequentemente fornecem uma alternativa `tolambda` mais sucinta. Quando as referências de métodos são mais curtas e claras, use-as; Quando não estiverem, fique com lambdas.

Item 44: Favorecer o uso de interfaces funcionais padrão

Agora que o Java tem lambdas, as melhores práticas para escrever APIs mudaram consideravelmente. Por exemplo, o padrão Template Method [Gamma95], no qual uma subclasse sobrepõe um método primitivo para especializar o comportamento de sua superclasse, é muito menos atraente. A alternativa moderna é fornecer uma fábrica estática ou construtor que aceite um objeto função para alcançar o mesmo efeito. Mais geralmente, você vai escrever mais construtores e métodos que tomam objetos função como parâmetros. Escolher o tipo certo de parâmetro funcional exige cuidado.

Considere o `LinkedHashMap`. Você pode usar essa classe como cache sobrescrevendo o método `removeEldestEntry` protegido, que é invocado pelo put toda vez que uma nova chave é adicionada ao mapa. Quando esse método retorna verdadeiro, o mapa remove sua entrada mais sel, que é passada para o método. A seguinte substituição permite que o `themap` cresça para cem entradas e depois exclui a entrada mais antiga a cada vez que uma nova chave é adicionada, mantendo as cem entradas mais recentes:

```
protected boolean removeEldestEntry(Map.Entry<K,V> eldest) {  
    return size() > 100;}  

```

Essa técnica funciona bem, mas você pode se sair muito melhor com lambdas. Se o `LinkedHashMap` fosse escrito hoje, teria uma fábrica estática ou construtor que aceitasse um objeto função. Olhando a declaração para `removeEldestEntry`, você pode pensar que o objeto função deveria pegar um `Map.Entry<K,V>` e retornar um booleano, mas isso não resolveria exatamente: O método `removeEldestEntry` chama `size()` para obter o número de entradas no mapa, o que funciona porque `removeEldestEntry` é um método de instância no mapa. O objeto de função que você passa para o construtor não é um método de instância no mapa e não pode capturá-lo porque o mapa ainda não existe quando sua fábrica ou construtor é invocado. Assim, o mapa deve se passar para o objeto função, que deve receber o mapa na entrada assim como sua entrada mais antiga. Se você declarasse essa interface não funcional, seria algo assim:

```
Interface funcional desnecessária; usar um padrão  
instead.@FunctionalInterface interface EldestEntryRemovalFunction<K,V>{  
    boolean remove(Map<K,V> map, Map.Entry<K,V> mais velho);}  

```

Essa interface funcionaria bem, mas você não deve usá-la, porque não precisa declarar uma nova interface para esse propósito. O `java.util.function` package oferece uma grande coleção de interfaces funcionais padrão para seu uso.

Se uma das interfaces funcionais padrão funcionar, geralmente você deve usá-la em vez de uma interface funcional feita para esse fim. Isso tornará sua API mais fácil de aprender, reduzindo sua área conceitual de superfície, e proporcionará benefícios significativos de interoperabilidade, já que muitas das interfaces funcionais padrão fornecem métodos padrão úteis. A interface de predicados, por exemplo, fornece métodos para combinar predicados. No caso do nosso exemplo do LinkedHashMap, a interface padrão `BiPredicate<Map<K,V>, Map.Entry<K,V>>` deve ser usada em preferência a uma interface personalizada `EldestEntryRemovalFunction`.

Existem quarenta e três interfaces em `java.util.Function`. Não dá para esperar que você lembre de todas, mas se lembrar de seis interfaces básicas, pode derivar o restante quando precisar. As interfaces básicas operam com base em tipos de referência-objeto. As interfaces de Operador representam funções cujos tipos de resultado e argumento são iguais. A interface Predicado representa uma função que pega um argumento e retorna um booleano. A interface de Função representa uma função cujos tipos de argumento e retorno diferem. A interface Fornecedor representa uma função que não aceita argumentos e retorna (ou "fornece") um valor. Por fim, o Consumidor representa uma função que pega um argumento e não retorna nada, essencialmente consumindo seu argumento. As seis interfaces funcionais básicas são resumidas abaixo:

Interface	Assinatura de Função	Exemplo
<code>UnaryOperator<T></code>	<code>T apply(T t)</code>	<code>String::toLowerCase</code>
<code>Operador<T></code> binário	<code>T apply(T t1, T t2)</code>	<code>BigInteger::add</code>
<code>Predicado<T></code>	<code>teste booleano (T t)</code>	<code>Coleção::éVazia</code>
<code>Função<T,R></code>	<code>R apply(T t)</code>	<code>Arrays::asList</code>
<code>Fornecedor<T></code>	<code>T get()</code>	<code>Instantâneo::agora</code>
<code>Consumidor<T></code>	<code>void accept(T t)</code>	<code>System.out::println</code>

Existem também três variantes de cada uma das seis interfaces básicas para operar nos tipos primitivos `int`, `long` e `double`. Seus nomes são derivados das interfaces básicas, prefixando-as com um tipo primitivo. Então, por exemplo, um predicado que recebe um `int` é um `IntPredicado`, e um operador binário que toma dois valores longos e retorna um longo é um `LongBinaryOperator`. Nenhum desses tipos de variantes é parametrizado, exceto as variantes de função, que são parametrizadas por `returnType`. Por exemplo, `LongFunction<int[]>` demora um tempo e retorna um `int[]`.

Existem nove variantes adicionais da interface de função, para uso quando o tipo de resultado é primitivo. Os tipos de fonte e resultado sempre diferem, porque uma função de um tipo para si mesmo é um `UnaryOperator`. Se tanto os tipos de fonte quanto de resultado forem primitivos, prefixe `Function` com `SrcToResult`, por exemplo `LongToIntFunction` (seis variantes). Se a fonte for uma primitiva e o resultado for uma referência de objeto, prefixe `Função` com `<Src>ToObj`, por exemplo `DoubleToObjFunction` (três variantes).

Existem versões de dois argumentos das três interfaces funcionais básicas para as quais faz sentido tê-las: `BiPredicado<T,U>`, `BiFunção<T,U,R>` e `BiConsumidor<T,U>`. Também existem variantes de `BiFunction` que retornam os três tipos primitivos relevantes: `ToIntBiFunction<T,U>`, `ToLongBiFunction<T,U>` e `ToDoubleBiFunction<T,U>`. Existem variantes de dois argumentos do Consumidor que aceitam uma referência de objeto e um tipo primitivo: `<T>ObjDoubleConsumer`, `ObjIntConsumer <T>` e `ObjLongConsumer<T>`. No total, existem nove versões com dois argumentos das interfaces básicas.

Por fim, há a interface `BooleanSupplier`, uma variante do `Supplier` que retorna valores booleanos. Esta é a única menção explícita do tipo booleano em qualquer nome padrão de interface funcional, mas valores de retorno booleanos são suportados via `Predicate` e suas quatro formas variantes. A `BooleanSupplier` interface e as quarenta e duas interfaces descritas nos parágrafos anteriores são responsáveis por todas as quarenta e três interfaces funcionais padrão. Admito que isso é um grande esforço para engolir, e não é nada muito ortogonal. Por outro lado, a maior parte das interfaces funcionais que você vai precisar já foi escrita para você e seus nomes são regulares o suficiente para que você não tenha muita dificuldade em criar uma quando precisar.

A maioria das interfaces funcionais padrão existe apenas para fornecer suporte a tipos primitivos. Não se deixe tentar a usar interfaces funcionais básicas com `boxedprimitives` em vez de interfaces funcionais primitivas. Embora funcione, viola o conselho do Item 61, "prefira tipos primitivos a primitivos em caixa." As consequências de desempenho do uso de primitivas em caixa para operações em massa podem ser fatais.

Agora você sabe que normalmente deve usar interfaces funcionais padrão em vez de escrever suas próprias. Mas quando você deve escrever o seu próprio? Claro que você precisa escrever o seu próprio se nenhum dos padrão fizer o que você precisa, por exemplo, se você exigir um predicado que use três parâmetros, ou um que gere uma exceção marcada. Mas há momentos em que você deveria escrever sua própria interface funcional, mesmo quando uma das padrão é estruturalmente idêntica.

Considere nosso velho amigo `Comparador<T>`, que é estruturalmente idêntico à interface `ToIntBiFunction<T,T>`. Mesmo que a segunda interface existisse quando a primeira foi adicionada às bibliotecas, teria sido errado usá-la. Ali

existem várias razões pelas quais o `Comparator` merece sua própria interface. Primeiro, seu nome fornece uma excelente documentação toda vez que é usado em uma API, e é muito utilizado. Segundo, a interface do `Comparator` possui exigências rigorosas sobre o que constitui uma instância válida, que compõem seu contrato geral. Ao implementar a interface, você está se comprometendo a seguir seu contrato. Terceiro, a interface é fortemente equipada com métodos padrão úteis para transformar e combinar comparadores.

Você deve considerar seriamente escrever uma interface funcional feita para esse fim, preferindo usar uma padrão, se precisar de uma interface funcional que compartilhe uma ou mais das seguintes características com o `Comparator`:

- Será de uso comum e pode se beneficiar de um nome descritivo.
- Tem um contrato forte associado.
- Ele se beneficiaria de métodos padrão personalizados. Se você optar por criar sua própria interface funcional, lembre-se de que ela é uma interface e, portanto, deve ser projetada com muito cuidado (Item 21). Note que a interface `EldestEntryRemovalFunction` (página 199) está rotulada com a anotação `@FunctionalInterface`. Esse tipo de anotação é semelhante ao de inspiração `@Override`. É uma declaração de intenção do programador que serve a três propósitos: informa aos leitores da classe e sua documentação que a interface foi projetada para habilitar lambdas; Isso mantém você honesto porque a interface não se acumula a menos que tenha exatamente um método abstrato; e impede que os mantenedores adicionem acidentalmente métodos abstratos à interface conforme ela evolui. Sempre anote suas interfaces funcionais com a anotação `@FunctionalInterface`. Um ponto final deve ser feito sobre o uso de interfaces funcionais em APIs. Não forneça um método com múltiplas sobrecargas que tomem diferentes interfaces funcionais na mesma posição de argumento se isso puder criar uma possível ambiguidade no cliente. Isso não é apenas um problema teórico. O método de submissão do `ExecutorService` pode aceitar tanto um `Chamável<T>` quanto um `Executável`, e é possível escrever um programa cliente que exija um `cast` para indicar a sobrecarga correta (Item 52). A maneira mais fácil de evitar esse problema é não escrever sobrecargas que tomem diferentes interfaces funcionais na mesma posição de argumento. Este é um caso especial do conselho do Item 52, "use sobrecarregadamente." Resumindo, agora que o Java tem lambdas, é fundamental que você projete suas APIs com lambdas em mente. Aceite tipos de interface funcional na entrada e retorne na saída. Geralmente é melhor usar as interfaces padrão fornecidas `injava.util.function.Function`, mas fique atento aos casos relativamente raros em que seria melhor escrever sua própria interface funcional.

Item 45: Use os fluxos com critério

A API de streams foi adicionada no Java 8 para facilitar a realização de operações em massa, sequencialmente ou em paralelo. Essa API fornece duas abstrações principais: o fluxo, que representa uma sequência finita ou infinita de elementos de dados, e o pipeline de fluxo, que representa um cálculo em múltiplos estágios sobre esses elementos. Os elementos de um riacho podem vir de qualquer lugar. Fontes comuns incluem coleções, arrays, arquivos, correspondedores de padrões de expressões regulares, geradores de números pseudoaleatórios e outros fluxos. Os elementos de dados em um fluxo podem ser referências a objetos ou primitivos. Três tipos primitivos são suportados: `int`, `long` e `double`.

Um pipeline de fluxo consiste em um fluxo fonte seguido por zero ou mais operações intermediárias e uma operação terminal. Cada operação intermediária transforma o fluxo de alguma forma, como mapear cada elemento para uma função desse elemento ou filtrar todos os elementos que não satisfazem alguma condição. Operações intermediárias transformam um fluxo em outro, cujo tipo de elemento pode ser igual ao fluxo de entrada ou diferente dele. A operação terminal realiza um cálculo final sobre o fluxo resultante da última operação intermediária, como armazenar seus elementos em uma coleção, retornar um determinado elemento ou imprimir todos os seus elementos.

Pipelines de fluxo são avaliados de forma preguiçosa: a avaliação só começa quando a operação terminal é invocada, e elementos de dados que não são necessários para completar a operação terminal nunca são computados. Essa avaliação preguiçosa é o que torna possível trabalhar com fluxos infinitos. Note que um pipeline de fluxo sem operação terminal é um no-op silencioso, então não se esqueça de incluir um.

A API dos fluxos é fluente: ela foi projetada para permitir que todas as chamadas que compõem um pipeline sejam encadeadas em uma única expressão. Na verdade, múltiplos pipelines podem ser encadeados em uma única expressão.

Por padrão, pipelines de fluxo rodam sequencialmente. Fazer um pipeline executar de forma paralela é tão simples quanto invocar o método `parallel` em qualquer fluxo na tubulação, mas raramente é apropriado fazer isso (Item 48).

A API de streams é suficientemente versátil para que praticamente qualquer computação possa ser feita usando streams, mas só porque você pode, não significa que deva fazê-lo. Quando usados adequadamente, os fluxos podem tornar os programas mais curtos e claros; Quando usados de forma inadequada, podem dificultar a leitura e manutenção dos programas. Não existem regras rígidas sobre quando usar streams, mas existem heurísticas.

Considere o seguinte programa, que lê as palavras de um arquivo de dicionário e imprime todos os grupos de anagramas cujo tamanho atende a um mínimo especificado pelo usuário. Lembre-se que duas palavras são anagramas se forem das mesmas letras em diferentes

Ordem. O programa lê cada palavra de um arquivo de dicionário especificado pelo usuário e coloca as palavras em um mapa. A chave do mapa é a palavra com suas letras em ordem alfabética, então a chave para "staple" é "aelpst", e a chave para "pétalas" também é "aelpst": as duas palavras são anagramas, e todos os anagramas compartilham a mesma forma alfabetizada (ou alphagram, como às vezes é chamado). O valor do mapa é uma lista contendo todas as palavras que compartilham uma forma alfabetizada. Após o dicionário ser processado, cada lista é um grupo completo de anagramas. O programa então itera através da visualização dos valores() do mapa e imprime cada lista cujo tamanho atende ao limite:

```
Imprime todos os grandes grupos de anagramas em um dicionário
iterativamentepublic classe Anagrams {
    empty static public main(String[] args) throws IOException {
        Dicionário de arquivos = novo Arquivo(args[0]);
        int minGroupSize = Integer.parseInt(args[1]);

        Grupos Map<String, Set<String>> = novo HashMap<>();
        try (Scanner s = new Scanner(dictionary)) {
            enquanto (s.hasNext()) {
                Palavra da cadeia = s.next();
                groups.computeIfAbsent(alfabetizar(palavra),
                    (não utilizado) -> novo TreeSet<>()).add(word);
            }

            para (<String>Grupo de conjuntos : groups.values())
                if (group.size() >= minGroupSize)
                    System.out.println(group.size() + ": " + group);
        }

        private static String alphabetize(Strings s) {
            char[] a = s.toCharArray();
            Arrays.sort(a); return new
            String(a);}
    }
}
```

Um passo desse programa merece destaque. A inserção de cada palavra no themap, que é mostrada em **negrito**, usa o método `computeIfAbsent`, que foi adicionado no Java 8. Esse método procura uma chave no mapa: Se a chave estiver presente, o método simplesmente retorna o valor associado a ela. Se não, o método calcula um valor aplicando o objeto de função dado à chave, associa esse valor à chave e retorna o valor calculado. O método `computeIfAbsent` simplifica a implementação de mapas que associam múltiplos valores a cada chave.

Agora considere o seguinte programa, que resolve o mesmo problema, mas faz uso intenso de streams. Note que todo o programa, com exceção de

O código que abre o arquivo do dicionário está contido em uma única expressão. A única razão pela qual o dicionário é aberto em uma expressão separada é para permitir o uso da instrução `try-with-resources`, que garante que o arquivo do dicionário esteja fechado:

```

Uso excessivo de streams - não faça
isso! classe pública Anagramas {

    empty static public main(String[] args) throws IOException {
        Dicionário de caminhos = Paths.get(args[0]); int
        minGroupSize = Integer.parseInt(args[1]);

        try (Palavras de fluxo<String> = Files.lines(dictionary)) {
            words.collect(
                groupingBy(word -> word.chars().sorted()
                    .collect(StringBuilder::new,
                        (sb, c) -> sb.append((char)
                            c),StringBuilder::append).toString()))
                .valores().stream()
                .filter(group -> group.size() >= minGroupSize).map(group ->
                    group.size() + ": " + group).forEach(System.out::println));}}

```

Se você achar esse código difícil de ler, não se preocupe; Você não está sozinho. É mais curto, mas também menos legível, especialmente para programadores que não são especialistas no uso de streams. O uso excessivo de streams torna os programas difíceis de ler e manter.

Felizmente, existe um meio-termo feliz. O programa a seguir resolve o mesmo problema, usando streams sem excesso. O resultado é um programa que é ao mesmo tempo mais curto e mais claro que o original:

```

O uso elegante de streams aumenta clareza e concisão classe pública
Anagramas {static void public, main(String[] args) throws
IOException {
    Dicionário de caminhos = Paths.get(args[0]); int
    minGroupSize = Integer.parseInt(args[1]);

    try (Palavras de fluxo<String> = Files.lines(dictionary)) {
        words.collect(groupingBy(word -> alfabetize(word)))
        .values().stream().filter(group -> group.size() >=
            minGroupSize).forEach(g -> System.out.println(g.size() + ": " +
                g));}}

```

O método de alfabetização é o mesmo da versão original}

Mesmo que você tenha pouca experiência prévia com streams, este programa não é difícil de entender. Ele abre o arquivo de dicionário em um bloco try-with-resources, obtendo um fluxo composto por todas as linhas do arquivo. A variável do fluxo é chamada `palavras` para sugerir que cada elemento do fluxo é uma palavra. O gasoduto nesse rio não possui operações intermediárias; sua operação terminal coleta todas as palavras em um mapa que as agrupa por sua forma alfabética (Item 46). Este é exatamente o mesmo mapa que foi construído em ambas as versões anteriores do programa. Depois, um novo `Stream<List<String>>` é aberto na visualização `values()` do mapa. Os elementos desse curso são, claro, os grupos de anagramas. O fluxo é filtrado para que todos os grupos cujo tamanho seja menor que `minGroupSize` sejam ignorados e, finalmente, os grupos restantes são impressos pela operação terminal para cada um.

Note que os nomes dos parâmetros `lambda` foram escolhidos cuidadosamente. O parâmetro deveria realmente ser chamado grupo, mas a linha de código resultante seria larga demais para o livro. Na ausência de tipos explícitos, uma nomeação cuidadosa dos `lambda` parâmetros é essencial para a legibilidade dos pipelines de fluxo.

Note também que a alfabetização das palavras é feita em um método alfabetizado separado. Isso melhora a legibilidade ao fornecer um nome para a operação e manter detalhes da implementação fora do programa principal. Usar métodos auxiliares é ainda mais importante para a legibilidade em pipelines de fluxo do que em código iterativo, porque pipelines não têm informações explícitas de tipo e variáveis temporárias nomeadas.

O método de alfabetização poderia ter sido reimplementado para usar fluxos, mas um método alfabetizado baseado em fluxos teria sido menos claro, mais difícil de escrever corretamente e provavelmente mais lento. Essas deficiências resultam da falta de suporte do Java para fluxos de caracteres primitivos (o que não significa que o Java deveria ter suportado fluxos de caracteres; isso seria inviável). Para demonstrar os perigos de processar valores de `char` com fluxos, considere o seguinte código:

```
"Olá, mundo!".chars().forEach(System.out::println);
```

Você pode esperar que imprima `Hello World!`, mas se rodar, vai perceber que imprime `721011081081113211911111410810033`. Isso acontece porque os elementos do fluxo retornam com `"Olá, mundo!"`. `Chars()` não são valores de `char`, mas sim valores `int`, então a sobrecarga de `INT` da impressão é invocada. É admitidamente uma fusão que um método chamado `chars` retorna um fluxo de valores `int`. Você poderia inserir o programa usando um `cast` para forçar a invocação da sobrecarga correta:

```
"Olá, mundo!".chars().forEach(x -> System.out.print((char) x));
```

Mas o ideal é evitar usar streams para processar valores de caractere.

Quando você começa a usar streams, pode sentir vontade de converter todos os seus loops em streams, mas resistir à vontade. Embora seja possível, provavelmente vai prejudicar a capacidade de adaptação e a manutenção da sua base de código. Como regra, mesmo tarefas moderadamente complexas são melhor realizadas usando alguma combinação de fluxos e iteração, como ilustrado pelos programas Anagramas acima. Então refatore o código existente para usar streams e usá-los em novo código apenas onde fizer sentido.

Como mostrado nos programas deste item, pipelines de fluxo expressam com-putação repetida usando objetos de função (tipicamente lambdas ou referências de métodos), enquanto o código iterativo expressa computação repetida usando blocos de código. Existem coisas que você pode fazer a partir de blocos de código que não pode fazer a partir de objetos de função:

- A partir de um bloco de código, você pode ler ou modificar qualquer variável local no escopo; A partir do alambda, você só pode ler variáveis finais ou efetivamente finais [JLS 4.12.4], e não pode modificar nenhuma variável local.
- A partir de um bloco de código, você pode retornar do método enclosure, quebrar ou continuar um loop de enclosure, ou lançar qualquer exceção verificada que esse método seja declarado para lançar; De um lambda, você não pode fazer nenhuma dessas coisas.

Se uma computação for melhor expressa usando essas técnicas, provavelmente ela não é uma boa correspondência para fluxos. Por outro lado, os streams facilitam muito algumas coisas:

- Transformar uniformemente sequências de elementos
- Sequências de filtro de elementos
- Combinar sequências de elementos usando uma única operação (por exemplo, adicioná-los, concatená-los ou calcular seu mínimo)
- Acumular sequências de elementos em uma coleção, talvez agrupando-os por algum atributo comum
- Buscar em uma sequência de elementos um elemento que satisfaça algum critério

Se um cálculo for melhor expresso usando essas técnicas, então ele é um bom candidato para fluxos.

Uma coisa difícil de fazer com fluxos é acessar elementos correspondentes de múltiplas etapas de um pipeline simultaneamente: uma vez que você mapeia um valor para outro valor, o valor original se perde. Uma solução alternativa é mapear cada valor para um objeto de par contendo o valor original e o novo valor, mas isso não é uma solução satisfatória, especialmente se os objetos de par forem necessários para múltiplas etapas de um pipeline. O código resultante é bagunçado e prolixo, o que vai contra um dos principais propósitos dos streams. Quando aplicável, uma solução alternativa melhor é inverter o themapping quando você precisar de acesso ao valor do estágio anterior.

Por exemplo, vamos escrever um programa para imprimir os primeiros vinte primos de Mersenne. Para refrescar sua memória, um número de Mersenne é um número da forma $2^p - 1$. Se p for primo, o número de Mersenne correspondente pode ser primo; se sim, é um Mersenneprime. Como fluxo inicial em nosso pipeline, queremos todos os números primos. Aqui está um método para devolver esse fluxo (infinito). Assumimos que uma importação estática foi usada para fácil acesso aos membros estáticos do `BigInteger`:

```
Stream primos<BigInteger> estáticos() {
    return Stream.iterate(DOIS, BigInteger::nextProbablePrime);}
```

O nome do método (`primos`) é um substantivo plural que descreve os elementos do fluxo de corrente. Essa convenção de nomenclatura é altamente recomendada para todos os métodos que retornam fluxos porque melhora a legibilidade dos pipelines de fluxos. O método usa o `Stream.iterate` de fábrica estático, que utiliza dois parâmetros: o primeiro elemento no fluxo e uma função para gerar o próximo elemento do fluxo a partir do anterior. Aqui está o programa para imprimir os primeiros vinte primos Mersenne:

```
public static void main(String[] args) {
    primos().map(p -> TWO.pow(p.intValueExact()).subtract(ONE))
        .filter(mersenne ->
            mersenne.isProbablePrime(50)).limit(20).forEach(System.out::println);
}
```

Este programa é uma codificação direta da descrição em prosa acima: começa com os primos, calcula os números de Mersenne correspondentes, filtra todos exceto os primos (o número mágico 50 controla o teste probabilístico de primalidade), limita o fluxo resultante a vinte elementos e os imprime.

Agora suponha que queiramos preceder cada primo de Mersenne com seu expoente (p). Esse valor está presente apenas no fluxo inicial, então é inacessível na operação terminal, que imprime os resultados. Felizmente, é fácil calcular a consequência de um número de Mersenne invertendo o mapeamento que ocorreu na operação do primeiro intermediário. O expoente é simplesmente o número de bits na representação binária, então essa operação terminal gera o resultado desejado:

```
.forEach(mp -> System.out.println(mp.bitLength() + ": " + mp));
```

Existem muitas tarefas em que não é óbvio se deve usar a oriteração de streams. Por exemplo, considere a tarefa de inicializar um novo baralho de cartas. Suponha que `Card` seja uma classe de valor imutável que encapsula um `Valor` e um `Naipes`, ambos tipos `enum`. Essa tarefa representa qualquer tarefa que

requer calcular todos os pares de elementos que podem ser escolhidos entre dois conjuntos. Matemáticos chamam isso de produto cartesiano dos dois conjuntos. Aqui está uma implementação iterativa com um loop aninhado for-each que deve parecer muito familiar para você:

```
Iterativo Cartesiano produto computationprivate
static List<Card> newDeck() {
    Resultado da lista<Card> = novo ArrayList<>
    (); para (Naipes : Valores.naipes())
        para (Rank rank : Rank.values())
            result.add (nova carta (naipes, valor));
    retorne resultado;}

```

E aqui está uma implementação baseada em fluxo que utiliza o flatMap de operações intermediárias. Essa operação mapeia cada elemento de um fluxo para um fluxo e então concatena todos esses novos fluxos em um único fluxo (ou os achatando). Note que essa implementação contém um lambda aninhado, mostrado em negrito:

```
Computação de produto cartesiano baseada em
fluxos List<Card> newDeck() {
    return Stream.of(Valores.naipes())
        .flatMap(suit ->
            Stream.of(Rank.values())
                .map(rank -> nova carta (naipes, valor)))
        .collect(toList());
}

```

Qual das duas versões do NewDeck é melhor? No fim das contas, tudo se resume à preferência pessoal e ao ambiente em que você está programando. A primeira versão é mais simples e talvez pareça mais natural. Uma fração maior dos programadores Java será capaz de entender e manter, mas alguns programadores se sentirão mais confortáveis com a segunda versão (baseada em stream). É um pouco mais conciso e não tão difícil de entender se você já entende bem streams e programação funcional. Se você não tem certeza de qual versão prefere, a versão iterativa provavelmente é a escolha mais segura. Se você prefere a versão de stream e acredita que outros programadores que vão trabalhar com o código vão compartilhar sua preferência, então você deveria usá-la.

Em resumo, algumas tarefas são melhor realizadas com streams, e outras com iteração. Muitas tarefas são melhor realizadas combinando as duas abordagens. Não existem regras rígidas para escolher qual abordagem usar para uma tarefa, mas existem algumas heurísticas úteis. Em muitos casos, ficará claro qual abordagem usar; Em alguns casos, não vai. Se você não tem certeza se uma tarefa é melhor atendida por streams ou iterações, tente ambos e veja qual funciona melhor.

Item 46: Prefira funções sem efeitos colaterais em streams

Se você é novo em streams, pode ser difícil pegar o jeito. Simplesmente expressar seu cálculo como um pipeline de fluxo pode ser difícil. Quando você tiver sucesso, seu programa vai funcionar, mas você pode perceber pouco ou nenhum benefício. Streams não é apenas uma API, é um paradigma baseado em programação funcional. Para obter a expressividade, velocidade e, em alguns casos, paralelizabilidade que os fluxos oferecem, é preciso adotar tanto o paradigma quanto a API.

A parte mais importante do paradigma dos fluxos é estruturar sua composição como uma sequência de transformações onde o resultado de cada estágio é o mais próximo possível de uma função pura do resultado da etapa anterior. Uma função pura é cujo resultado depende apenas de sua entrada: não depende de nenhum estado mutável, nem atualiza nenhum estado. Para alcançar isso, quaisquer objetos de função que você passa para operações de fluxo, tanto intermediários quanto terminais, devem estar livres de efeitos colaterais.

Ocasionalmente, você pode ver o código dos fluxos que se parece com este trecho, que constrói uma tabela de frequências das palavras em um arquivo de texto:

```
Usa a API de streams, mas não o paradigma – Não faça isso!
Map<String, Long> freq = novo HashMap<>(); try (<String>Stream
words = new Scanner(file).tokens()) {
    words.forEach(word -> {
        freq.merge(word.toLowerCase(), 1L, Long::sum));});}
```

O que há de errado com esse código? Afinal, ele usa streams, lambdas e methodreferences, e recebe a resposta certa. Simplificando, não é código de streams de verdade; É código iterativo disfarçado de código de streams. Ela não obtém benefícios da API thestreams, e é (um pouco) mais longa, mais difícil de ler e menos sustentável do que o código iterativo correspondente. O problema decorre do fato de que esse código está fazendo todo o seu trabalho em um terminal para cada operação, usando um lambda que muta o estado externo (a tabela de frequência). Uma operação `forEach` que faz algo além de apresentar o resultado do cálculo realizado por um fluxo é um "código de cheiro ruim", assim como um lambda que muta o estado. Então, como esse código deve parecer?

```
Uso correto de fluxos para inicializar um frequency
tableMap<String, Long> frequência; try (<String>Stream words =
new Scanner(file).tokens()) {
    freq = palavras
        .collect(agrupamentoPor(String::paraMinúscula, contando()));
}
```

Este trecho faz o mesmo que o anterior, mas faz uso adequado da API dos streams. É mais curto e claro. Então, por que alguém escreveria de outra forma? Porque usa ferramentas com as quais eles já estão familiarizados. Programadores Java sabem usar loops `for-eachs`, e a operação do terminal `forEach` é semelhante. Mas para cada operação está entre as menos poderosas das operações de terminal e a menos amigável ao fluxo de água. É explicitamente iterativo e, portanto, não é passível de paralização. A operação `forEach` deve ser usada apenas para reportar o resultado do cálculo `astream`, não para realizar o cálculo. Ocasionalmente, faz sentido usar `forEach` para algum outro propósito, como adicionar os resultados do cálculo `astream` a uma coleção pré-existente.

O código aprimorado usa um coletor, que é um conceito novo que você precisa aprender para usar fluxos. A API do `Collectors` é intimidadora: possui trinta e nove métodos, alguns dos quais chegam a ter até cinco parâmetros de tipo. A boa notícia é que você pode obter a maior parte do benefício dessa API sem precisar se aprofundar em toda a sua complexidade. Para começar, você pode ignorar a interface do Coletor e pensar em um coletor como um objeto opaco que encapsula uma estratégia de redução. Nesse contexto, redução significa combinar os elementos de um fluxo em um único objeto. O objeto produzido por um coletor é tipicamente uma coleção (que explica o nome coletor).

Os coletores para reunir os elementos de um fluxo em uma verdadeira Coleção são simples. Existem três desses coletores: `toList()`, `toSet()` e `toCollection(collectionFactory)`. Eles retornam, respectivamente, um conjunto, uma lista e um tipo de coleção especificado pelo programador. Munidos desse conhecimento, podemos escrever um pipeline de fluxo para extrair uma lista dos dez melhores da nossa tabela de frequências.

```
Pipeline para obter uma lista das dez melhores palavras a partir de
uma tabela de<String> frequências
topTen = freq.keySet().stream()
    .sorted(comparing(frequent::get).reversed())
    .limit(10).collect(toList());
```

Note que não qualificamos o método `toList` com sua classe, `Collectors`. É costume e sábio importar estaticamente todos os membros dos Coletores porque isso torna os pipelines de fluxo mais legíveis.

A única parte complicada desse código é o comparador que passamos para `sorted,comparing(freq::get).reversed()`. O método de comparação é um método de construção comparativa (Item 14) que utiliza uma função de extração de chaves. A função recebe uma palavra, e a "extração" é na verdade uma consulta de tabela: o método de referência vinculado `freq::get` pesquisa a palavra na tabela de frequência e retorna o número de vezes que a palavra aparece no arquivo. Finalmente, chamamos `reverso` no

Comparador, então estamos ordenando as palavras de mais frequentes para menos frequentes. Depois, é simples limitar o fluxo a dez palavras e reuni-las em uma lista.

Os trechos de código anteriores usam o método de fluxo do Scanner para obter um fluxo pelo scanner. Esse método foi adicionado no Java 9. Se você estiver usando uma versão anterior, pode traduzir o scanner, que implementa o Iterator, para um stream usando um adaptador semelhante ao do Item 47 (`streamOf(Iterable<E>))`).

E quanto aos outros trinta e seis métodos em Collectors? A maioria deles existe para permitir que você colete riachos em mapas, o que é muito mais complicado do que colocá-los em coleções verdadeiras. Cada elemento do fluxo é associado a uma chave e um valor, e múltiplos elementos do fluxo podem ser associados à mesma chave.

O coletor de mapas mais simples é o `toMap` (`keyMapper`, `valueMapper`), que aceita duas funções, uma das quais mapeia um elemento de fluxo para uma chave, a outra para um valor. Usamos esse coletor em nossa implementação `fromString` no Item 34 para fazer um mapa da forma de string de um enum para o próprio enum:

```
Usando um coletor toMap para fazer um mapa de string para
enum private estático final Map<String, Operation> stringToEnum =
    Stream.of(values()).collect(
        toMap(Object::toString, e -> e));
```

Essa forma simples de `toMap` é perfeita se cada elemento do fluxo mapear para uma chave única. Se múltiplos elementos do fluxo mapearem para a mesma chave, o pipeline irá se contra-minar com um `UnlawfulStateException`.

As formas mais complexas do `toMap`, assim como o método agrupamento, oferecem várias formas de fornecer estratégias para lidar com tais colisões. Uma forma é fornecer ao método `toMap` uma função de fusão além de seus mapeadores de chave e valor. A função de fusão é um Operador Binário<V>, onde V é o tipo de valor do mapa. Quaisquer valores adicionais associados a uma chave são combinados com o valor existente usando a função de fusão, então, por exemplo, se a função de fusão for multiplicação, você acaba com um valor que é o produto de todos os valores associados à chave pelo mapeador de valores.

A forma de três argumentos do `toMap` também é útil para fazer um mapa de uma chave para, um elemento escolhido associado a essa chave. Por exemplo, suponha que temos uma série de álbuns de vários artistas e queremos um mapa do artista para o álbum mais vendido. Esse coletor vai fazer o trabalho.

```
Collector para gerar um mapa da chave ao elemento escolhido para
keyMap<Artist, Album> topHits = albums.collect(
    toMap(Album::artist, a->a, maxBy(comparing(Album::sales))));
```

Note que o comparador usa o método de fábrica estática `maxBy`, que é importado staticamente do `BinaryOperator`. Esse método converte um `Comparador<T>` em um `<T> OperadorBinário` que calcula o máximo implícito pelo comparador especificado. Nesse caso, o comparador é retornado pelo método de construção do comparador comparando, que utiliza a função extratora de chave `Album::sales`. Isso pode parecer um pouco confuso, mas o código é bem lê. Falando de forma geral, diz: "converta o fluxo de álbuns em um mapa, mapeando cada artista para o álbum que tem o melhor álbum por vendas." Isso está surpreendentemente próximo do estado problemático.

Outro uso da forma de três argumentos do `toMap` é produzir um coletor que impõe uma política de última escrita-vitória quando há colisões. Para muitos fluxos, os resultados serão não determinísticos, mas se todos os valores que podem ser associados a a key pelas funções de mapeamento forem idênticos, ou se todos forem aceitáveis, o comportamento do `thiscollector` pode ser exatamente o que você deseja:

```
Collector para impor a policy toMap(keyMapper,  
valueMapper, (v1, v2) -> v2)
```

A terceira e última versão do `toMap` adota um quarto argumento, que é uma fábrica de mapas, para uso quando você deseja especificar uma implementação específica de mapa, como um `EnumMap` ou um `TreeMap`.

Também existem formas variantes das três primeiras versões do `toMap`, chamadas `toConcurrentMap`, que rodam eficientemente em paralelo e produzem instâncias `ConcurrentHash-Map`.

Além do método `toMap`, a API `Collectors` fornece o `groupingByMethod`, que retorna coletores para produzir mapas que agrupam elementos em categorias baseadas em uma função classificadora. A função classificadora pega um elemento e retorna a categoria em que ele se encaixa. Esta categoria serve como a chave do elemento `smap`. A versão mais simples do método `groupingBy` toma apenas um classificador e retorna um mapa cujos valores são listas de todos os elementos de cada categoria. Este é o coletor que usamos no programa Anagrama no Item 45 para gerar um mapa de uma palavra alfabetizada para uma lista das palavras que compartilham a alfabetização:

```
words.collect(groupingBy(word -> alfabetize(word)))
```

Se você quiser que o `groupingBy` retorne um coletor que produza um mapa com valores diferentes das listas, você pode especificar um coletor a jusante além de um classificador. Um coletor a jusante produz um valor a partir de um fluxo contendo todos os

elementos em uma categoria. O uso mais simples desse parâmetro é passar para `Set()`, o que resulta em um mapa cujos valores são conjuntos de elementos em vez de listas.

Alternativamente, você pode passar para `Collection(collectionFactory)`, que permite criar as coleções nas quais cada categoria de elementos é colocada. Isso te dá a flexibilidade de escolher qualquer tipo de coleção que desejar. Outro uso simples da forma de agrupamento de dois argumentos `By` é passar `counting()` como coletor a jusante. Isso resulta em um mapa que associa cada categoria ao número de elementos da categoria, em vez de uma coleção contendo os elementos. Foi isso que você viu no exemplo da tabela de frequência no início deste item:

```
Map<String, Long> freq = palavras
    .collect(agrupamentoPor(String::paraMinúscula, contando()));
```

A terceira versão do `groupingBy` permite que você especifique uma fábrica de mapas além de um coletor a jusante. Note que esse método viola o padrão padrão de lista de argumentos telescopados: o parâmetro `mapFactory` precede, em vez de seguir, o parâmetro `downStream`. Esta versão do `groupingBy` te dá controle sobre o mapa contido, assim como sobre as coleções contidas, então, por exemplo, você pode especificar um coletor que retorna um `TreeMap` cujos valores são `TreeSets`.

O método `groupingByConcurrent` fornece variantes dos três `overload-ings` do `groupingBy`. Essas variantes rodam eficientemente em paralelo e produzem instâncias `ConcurrentHashMap`. Há também um parente raramente usado de `groupingBy` chamado `partitioningBy`. Em vez de um método classificador, ele pega um predicado e retorna um mapa cuja chave é booleana. Existem duas sobrecargas desse método, uma das quais leva um coletor a jusante além de um predicado.

Os coletores retornados pelo método de contagem são destinados apenas ao uso como coletores a jusante. A mesma funcionalidade está disponível diretamente no `Stream`, via método de contagem, então nunca há motivo para dizer coletar (`contando()`). Existem mais quinze métodos de `Colecionadores` com essa propriedade. Eles incluem métodos de `tenina` cujos nomes começam com `soma`, `média` e `resumo` (cuja funcionalidade está disponível nos tipos de fluxo primitivos correspondentes). Eles também incluem todas as sobrecargas do método `reduz`, além dos métodos de `filtragem`, `mapeamento`, `mapeamento plano` e `coleta` e `Depois`. A maioria dos programadores consegue ignorar a maioria desses métodos com segurança. Do ponto de vista do projeto, esses coletores representam uma tentativa de duplicar parcialmente a funcionalidade dos `streams` nos coletores para que coletores posteriores possam atuar como "ministreams".

Existem três métodos de `Coletores` que ainda não mencionamos. Embora estejam em `Colecionadores`, não envolvem coleções. Os dois primeiros são `minBy` e `maxBy`, que pegam um comparador e retornam o elemento mínimo ou máximo em

o fluxo conforme determinado pelo comparador. São generalizações menores dos métodos `themin` e `max` na interface `Stream` e são análogos coletores dos operadores binários retornados pelos métodos de mesmo nome em `BinaryOperator`. Lembre-se que usamos `BinaryOperator.maxBy` em nosso exemplo de álbum best-seller.

O método final dos `Collectors` é o `joining`, que opera apenas em fluxos de instâncias `CharSequence`, como strings. Em sua forma sem parâmetros, ele retorna um coletor que simplesmente concatena os elementos. Sua forma de argumento único toma um parâmetro `singleCharSequence` chamado delimitador e retorna um coletor que une os elementos do fluxo, inserindo o delimitador entre elementos adjacentes. Se você passar uma vírgula como delimitador, o coletor retorna uma cadeia de valores separada por vírgulas (mas atenção, a cadeia será ambígua se algum dos elementos do fluxo contiver vírgulas). A forma de três argumentos recebe um prefixo e um sufixo além do delimitador. O coletor resultante gera cadeias como as que você recebe quando imprime uma coleção, por exemplo `[veio, viu, conquistou]`.

Em resumo, a essência dos pipelines de fluxo de programação são os objetos sem efeitos colaterais. Isso se aplica a todos os muitos objetos função passados para os objetos relacionados `streamsand` (`streamsand related objects`). A operação terminal para `Each` deve ser usada apenas para reportar o resultado de um cálculo realizado por um fluxo, não para realizar o cálculo. Para usar os fluxos corretamente, você precisa conhecer os coletores. As fábricas de colecionadores mais importantes são `toList`, `toSet`, `toMap`, `groupingBy` e `joining`.

Item 47: Preferência Coleta a Fluxo como tipo de devolução

Muitos métodos retornam sequências de elementos. Antes do Java 8, os tipos de retorno óbvios para tais métodos eram as interfaces de coleção `Collection`, `Set` e `List`; `Iterável`; e os tipos de arranjos. Normalmente, era fácil decidir qual desses tipos devolter. O padrão era uma interface de coleta. Se o método existisse apenas para habilitar loops `for-each` ou se a sequência retornada não pudesse ser feita implementar algum método `Collection` (tipicamente, `contains(Object)`), a interface `Iterable` era usada. Se os elementos retornados fossem valores primitivos ou houvesse requisitos rigorosos de desempenho, eram usados arrays. No Java 8, fluxos eram adicionados à plataforma, complicando substancialmente a tarefa de escolher o tipo de retorno apropriado para um método de retorno de sequência.

Você pode ouvir dizer que streams agora são a escolha óbvia para retornar uma sequência de elementos, mas como discutido no Item 45, streams não tornam a iteração obsoleta: escrever um bom código exige combinar fluxos e iterações de forma criteriosa. Se uma API retorna apenas um fluxo e alguns usuários quiserem iterar sobre a sequência retornada com um loop `for-each`, esses usuários ficarão justificadamente irritados. É especialmente frustrante porque a interface `Stream` contém o único método abstrato na interface `Iterable`, e a especificação do `Stream` para esse método é compatível com a do `Iterable`. A única coisa que impede programadores de usarem um `toIterable` de loop `for-each` sobre um fluxo é a falha do `Stream` em estender o `Iterable`.

Infelizmente, não existe uma boa solução alternativa para esse problema. À primeira vista, pode parecer que passar uma referência de método para o método iterador do `Stream` funcionaria. O código resultante talvez seja um pouco barulhento e opaco, mas não irrazoável:

```
Não compila, devido a limitações na inferência de tipos do Java
(ProcessHandle ph : ProcessHandle.allProcesses()::iterator) {
    Processe o processo}
```

Infelizmente, se você tentar compilar esse código, receberá uma mensagem de erro:

```
Test.java:6: erro: referência ao método não esperada aqui
(ProcessHandle ph : ProcessHandle.allProcesses()::iterator) {
    ^
```

Para que o código seja compilado, você precisa lançar a referência do método para um `Iterable` parametrizado de forma apropriada:

```
Solução horrível para iterar sobre um streamfor
(ProcessHandle ph : (Iterável<ProcessHandle>)
    ProcessHandle.allProcesses()::iterator)
```

Esse código cliente funciona, mas é barulhento e opaco demais para usar na prática. Uma solução melhor é usar um método de adaptador. O JDK não oferece esse tipo de método, mas é fácil escrever um, usando a mesma técnica usada em linha nos trechos de texto acima. Note que não é necessário cast no método adaptador porque a inferência de tipos do Java funciona corretamente nesse contexto:

```
Adaptador de      Transmitir<E> para o Iterable<E>
publicamente estático <E> <E> Iterable iterableOf(<E>Stream stream) {
return stream::iterator;}
```

Com esse adaptador, você pode iterar sobre qualquer stream com uma instrução for-each:

```
for (ProcessHandle p : iterableOf(ProcessHandle.allProcesses())) {
  Processe o processo}
```

Note que as versões em fluxo do programa Anagrams no Item 34 usam o método `theFiles.lines` para ler o dicionário, enquanto a versão iterativa usa um scanner. O método `Files.lines` é superior a um scanner, que engole silenciosamente quaisquer exceções encontradas durante a leitura do arquivo. Idealmente, teríamos usado o `Files.lines` também na versão iterativa. Esse é o tipo de compromisso que os programadores fazem se uma API fornecer apenas acesso de fluxo a uma sequência e eles quiserem iterar sobre a sequência com uma instrução for-each.

Por outro lado, um programador que deseja processar uma sequência usando um streampipeline ficará justificado chateado com uma API que fornece apenas um `Iterable`. Novamente, o JDK não fornece um adaptador, mas é fácil escrever um:

```
Adaptador do Iterável<E> para Stream<E>estático público <E> <E>
streamOf(<E>Iterable iterable) {
return StreamSupport.stream(iterable.spliterator(), false);}
```

Se você está escrevendo um método que retorna uma sequência de objetos e sabe que ele só será usado em um pipeline de fluxo, então, claro, deve se sentir à vontade para devolver um fluxo. De forma semelhante, um método que retorne uma sequência que será usada apenas para iteração deve retornar um `Iterável`. Mas se você está escrevendo uma API pública que retorna uma sequência, deve fornecer tanto para usuários que querem escrever streampipelines quanto para aqueles que querem escrever para cada outdoor, a menos que tenha um bom motivo para acreditar que a maioria dos seus usuários vai querer usar o mesmo mecanismo.

A interface `Collection` é um subtipo do `Iterable` e possui um método de fluxo, então oferece tanto iteração quanto acesso em fluxo. Portanto, `Collection` ou um subtipo apropriado geralmente é o melhor tipo de retorno para um método público de retorno de sequência. Os arrays também permitem fácil iteração e acesso a fluxos com os métodos `Arrays.asList` e `Stream.of`. Se a sequência que você está retornando for pequena o suficiente para caber facilmente na memória, provavelmente é melhor retornar uma das implementações padrão de coleções, como `ArrayList` ou `HashSet`. Mas não armazene uma sequência grande na memória só para devolvê-la como uma coleção.

Se a sequência que você está retornando for grande, mas puder ser representada de forma concisa, considere implementar uma coleção de propósito especial. Por exemplo, suponha que você queira retornar o conjunto das partes de um dado conjunto, que consiste em todos os seus subconjuntos. O conjunto das partes $\{a, b, c\}$ é $\{\{\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$. Se `aset` tem `n` elementos, seu conjunto de partes tem 2^n . Portanto, você nem deveria considerar armazenar o conjunto de potências em uma implementação de coleção padrão. No entanto, é fácil implementar uma coleção personalizada para o trabalho com a ajuda do `AbstractList`.

O truque é usar o índice de cada elemento do conjunto de potências como um vetor de bits, onde o n -ésimo bit no índice indica a presença ou ausência do n -ésimo elemento do conjunto de origem. Em essência, existe um mapeamento natural entre os números binários de 0 a $2^n - 1$ e o conjunto das partes de um conjunto de n elementos. Aqui está o código:

```
Retorna o conjunto de potências de um conjunto de entrada como classe powerpublic collectionpublic
personalizada {

    public static <E> final Collection<Set<E>> of(Set<E> s) {
        List<E> src = nova ArrayList<>(s); se (src.size() > 30)

            lançar um novo IllegalArgumentException("Set muito grande " + s); return new
            AbstractList<Set<E>>() {
                @Override public int size() {
                    retorne 1 << src.size(); 2 para o poder srcSize

                @Override boolean público contém(Object o) {
                    return o instância de Set && src.containsAll((Set)o);}

                @Override public<E> Set get(int index) {
                    Set<E> result = novo HashSet<>(); para (int i = 0; índice != 0; i++, índice >= 1)

                        if ((índice & 1) == 1)
                            result.add(src.get(i)); return result;}}}
```

Note que o `PowerSet.of` lança uma exceção se o conjunto de entrada tiver mais de 30 elementos. Isso destaca uma desvantagem de usar `Collection` como tipo de retorno em vez de `Stream` ou `Iterable`: `Collection` possui um método de tamanho com retorno `int`, que limita o comprimento da sequência retornada a `Integer.MAX_VALUE`, ou $2^{31} - 1$. A especificação `Collection` permite que o método `size` retorne $2^{31} - 1$ se a coleção for maior, até infinita, mas isso não é uma solução totalmente satisfatória.

Para escrever uma implementação de `Collection` sobre `AbstractCollection`, você precisa implementar apenas dois métodos além do necessário para `Iterable`: `contains` e `size`. Frequentemente, é fácil escrever implementações eficientes desses métodos. Se não for viável, talvez porque o conteúdo da sequência não está pré-determinado antes da iteração, retorne um fluxo ou iterável, o que parecer mais natural. Se quiser, pode devolver ambos usando dois métodos separados.

Há momentos em que você escolhe o tipo de retorno apenas pela facilidade de implementação. Por exemplo, suponha que você queira escrever um método que retorne todas as sublistas (contíguas) de uma lista de entrada. São necessárias apenas três linhas de código para gerar essas sublistas e colocá-las em uma coleção padrão, mas a memória necessária para armazenar essa coleção é quadrática em relação ao tamanho da lista fonte. Embora isso não seja tão ruim quanto o conjunto de potências, que é exponencial, é claramente inaceitável. Implementar uma coleção personalizada, como fizemos para o conjunto de potências, seria tedioso, ainda mais porque o JDK não tem uma implementação esquelética no `Iterator` para nos ajudar.

No entanto, é simples implementar um fluxo de todas as sublistas de uma lista de entrada, embora isso exija um pequeno insight. Vamos chamar uma sublista que contém o primeiro elemento de uma lista de prefixo da lista. Por exemplo, os prefixos de (a, b, c) são (a), (a, b) e (a, b, c). De forma semelhante, vamos chamar uma sublista que contém o último elemento de sufixo, então os sufixos de (a, b, c) são (a, b, c), (b, c) e (c). A ideia é que as sublistas de uma lista são simplesmente os sufixos dos prefixos (ou, idênticamente, os prefixos dos sufixos) e da lista vazia. Essa observação leva diretamente a uma implementação clara e razoavelmente concisa:

Retorna um fluxo de todas as sublistas de sua classe de entrada

```
list public Sublists {
```

```
    <E> público estático Stream<List<E>> of(<E>List list) {
        return Stream.concat(Stream.of(Collections.emptyList()),
            prefixes(list).flatMap(SubLists::sufixos));}
}
```

```
private <E> static Stream<List<E>> prefixes(<E>List list) {
    return IntStream.rangeClosed(1, list.size())
        .mapToObj(fim -> list.subList(0, fim));}
}
```



```
private <E> static Stream<List<E>> suffixes(List<E> list) {
    return IntStream.range(0, list.size())
        .mapToObj(start -> list.subList(start, list.size()));}
```

Note que o método `Stream.concat` é usado para adicionar a lista vazia ao fluxo retornado. Note também que o método `flatMap` (Item 45) é usado para gerar um único fluxo composto por todos os sufixos de todos os prefixos. Por fim, observe que geramos os prefixos e sufixos mapeando um fluxo de intvalues consecutivos retornados por `IntStream.range` e `IntStream.rangeClosed`. Esse idiomista, de forma geral, é o equivalente em fluxo do ciclo padrão `for` em índices inteiros. Assim, nossa implementação de sublistas é semelhante em espírito ao óbvio ciclo `aninhado` `for`:

```
para (int start = 0; start < src.size(); start++)
    para (int end = start + 1; end <= src.size(); end++)
        System.out.println(src.subList(início, fim));
```

É possível traduzir esse `for-loop` diretamente para um fluxo. O resultado é mais conciso do que nossa implementação anterior, mas talvez um pouco menos legível. É semelhante em espírito ao código de fluxos para o produto cartesiano no Item 45:

```
Retorna um fluxo de todas as sublistas de sua entrada
listpublic <E> static Stream<List<E>> of(<E>List list) {
    return IntStream.range(0, list.size())
        .mapToObj(start ->
            IntStream.rangeClosed(start + 1, list.size())
                .mapToObj(fim -> list.subList(começo, fim))).flatMap(x ->
x);}
```

Assim como o `for-loop` que o precede, esse código não emite a lista vazia. Para corrigir essa deficiência, você pode usar `concat`, como fizemos na versão anterior, ou substituir `1` por `(int) Math.signum(start)` na chamada `rangeClosed`.

Qualquer uma dessas implementações de sublistas de fluxo é aceitável, mas ambas exigirão que alguns usuários empreguem um adaptador `Stream-para-Iterável` ou que usem um fluxo em lugares onde a iteração seria mais natural. O `Stream-to-IterableAdapter` não só atrapalha o código do cliente, como também desacelera o loop em um fator de 2,3 no `mymachine`. Uma implementação de `Collection` feita para esse propósito (não mostrada aqui) é consideravelmente mais prolixa, mas roda cerca de 1,4 vezes mais rápido que nossa implementação baseada em fluxo na minha máquina.

Em resumo, ao escrever um método que retorna uma sequência de elementos, lembre-se de que alguns dos seus usuários podem querer processá-los como um fluxo, enquanto outros podem querer iterar sobre eles. Tente acomodar os dois grupos. Se for possível devolver uma coleção, faça isso. Se você já tem os elementos em uma coleção ou o número de elementos na sequência é pequeno o suficiente para justificar a criação de uma nova, retorne uma coleção padrão como `ArrayList`. Caso contrário, considere implementar uma coleção personalizada como fizemos para o conjunto de poderes. Se não for viável devolver uma coleção, retorne um stream ou iterável, o que parecer mais natural. Se, em uma futura versão em Java, a declaração da interface do `Stream` for modificada para estender o `Iterable`, então você deve se sentir à vontade para devolver os fluxos, pois eles permitirão tanto o processamento quanto a iteração do fluxo.

Item 48: Tenha cautela ao fazer fluxos paralelos

Entre as linguagens mais comuns, o Java sempre esteve na vanguarda da oferta de recursos para facilitar a tarefa de programação concorrente. Quando o Java foi lançado em 1996, ele tinha suporte embutido para threads, com sincronização e esperar/notificar. O Java 5 introduziu a `java.util.concurrent library`, com coleções concorrentes e o framework executor. O Java 7 introduziu o pacote `fork-join`, uma estrutura de alto desempenho para decomposição paralela. O Java 8 introduziu fluxos, que podem ser paralelizados com uma única chamada ao método `paralelo`. Escrever programas concorrentes em Java está ficando mais fácil, mas escrever programas concorrentes corretos e rápidos continua tão difícil quanto sempre foi. Violações de segurança e vitalidade são um fato da vida em programação concorrente, e pipelines de fluxo paralelo não são exceção.

Considere este programa do Item 45:

```
Programa baseado em stream para gerar os primeiros 20 Mersenne
primes public static void main(String[] args) {
    primes().map(p -> Two.pow(p.intValueExact()).subtract(ONE))
        .filter(mersenne ->
            mersenne.isProbablePrime(50)).limit(20).forEach(Syste
            m.out::println);
}

Stream primos<BigInteger> estáticos() {
    return Stream.iterate(DOIS, BigInteger::nextProbablePrime);}
```

Na minha máquina, esse programa imediatamente começa a imprimir primes e toma 12,5 segundos para a conclusão da corrida. Suponha que eu ingenuamente tente acelerar adicionando um `acall` ao `paralelo()` ao pipeline do fluxo. O que você acha que vai acontecer com seu desempenho? Vai ficar alguns por cento mais rápido? Alguns por cento mais devagar? Infelizmente, o que acontece é que ele não imprime nada, mas o uso da CPU dispara para 90% e permanece indefinidamente (uma falha de liveness). O programa poderia acabar eventualmente, mas eu não estava disposto a descobrir; Parei à força depois de meia hora. O que está acontecendo aqui? Simplificando, a biblioteca de fluxos não faz ideia de como `par-allelizar` esse pipeline e as heurísticas falham. Mesmo nas melhores circunstâncias, paralelizar um pipeline dificilmente aumentará seu desempenho se o código-fonte for do `Stream.iterate`, ou se for usado o limite intermediário de operações. Esse duto precisa lidar com essas duas questões. Pior ainda, a estratégia padrão de paralelização lida com a imprevisibilidade do limite assumindo que não há problema em processar alguns elementos extras e descartar quaisquer resultados desnecessários. Neste caso,

leva aproximadamente o dobro do tempo para encontrar cada primo de Mersenne do que para encontrar o anterior. Assim, o custo de calcular um único elemento extra é aproximadamente igual ao custo de calcular todos os elementos anteriores combinados, e esse pipeline aparentemente inofensivo coloca o algoritmo automático de paralelização de joelhos. A moral dessa história é simples: não paralelize os oleodutos de rios indiscriminadamente. As consequências de desempenho podem ser desastrosas.

Como regra, ganhos de desempenho com paralelismo são melhores em streams sobre instâncias `ArrayList`, `HashMap`, `HashSet` e `ConcurrentHashMap`; matrizes; intangíveis; e longas distâncias. O que essas estruturas de dados têm em comum é que todas podem ser divididas de forma precisa e barata em subfaixas de tamanhos desejados, o que facilita a divisão do trabalho entre threads paralelas. A abstração usada pela biblioteca `theStreams` para realizar essa tarefa é o `spliterator`, que é retornado pelo método `theSpliterator` no `Stream` e no `Iterable`.

Outro fator importante que todas essas estruturas de dados têm em comum é que elas fornecem uma localidade de referência de boa a excelente quando processadas sequencialmente: referências sequenciais a elementos são armazenadas juntas na memória. Os objetos referenciados por essas referências podem não estar próximos uns dos outros na memória, o que reduz a localidade da referência. A localidade de referência se mostra criticamente importante para paralelizar operações em massa: sem ela, as threads passam grande parte do tempo ociosas, esperando que os dados sejam transferidos da memória para o cache do processador. As estruturas de dados com a melhor localidade de referência são matrizes primitivas — porque os próprios dados são armazenados contíguos na memória.

A natureza da operação terminal de um pipeline de fluxo também afeta a eficácia da execução paralela. Se uma quantidade significativa de trabalho for realizada na operação terminal em comparação com o trabalho geral do pipeline e essa operação for inerentemente sequencial, então a paralelização do pipeline terá eficácia limitada. As melhores operações terminais para paralelismo são reduções, onde todos os elementos que emergem do pipeline são combinados usando um dos `reduce` methods do `Stream`, ou reduções pré-empacotadas como `min`, `max`, `count` e `sum`. As operações de curto-circuito qualquer `Match`, `AllMatch` e `noneMatch` também são passíveis de paralelismo. As operações realizadas pelo método `collect` do `Stream`, conhecidas como reduções mutáveis, não são boas candidatas para paralelismo porque a sobrecarga de combinar coleções é custosa.

Se você escreve sua própria implementação de `Stream`, `Iterable` ou `Collection` e quer um desempenho paralelo decente, deve sobrescrever o método `spliterator` e testar extensivamente o desempenho paralelo dos fluxos resultantes. Escrever `spliterators` de alta qualidade é difícil e está além do escopo deste livro.

Paralelizar um fluxo pode não apenas levar a um desempenho ruim, incluindo falhas de vivacidade; Pode levar a resultados incorretos e comportamentos imprevisíveis (falhas de segurança). Falhas de segurança podem ocorrer da paralelização de um pipeline que utiliza `esmappers`, filtros e outros objetos funcionais fornecidos pelo programador que não cumprem suas especificações. A especificação `Stream` impõe requisitos rigorosos a esses objetos de função. Por exemplo, as funções acumulador e combinador passadas para a operação de redução do `Stream` devem ser associativas, não interferentes e sem estado. Se você violar esses requisitos (alguns dos quais são discutidos no Item 46), mas executar seu pipeline sequencialmente, provavelmente ele trará resultados corretos; Se você paralisar a área, provavelmente vai falhar, talvez de forma catastrófica.

Nesse sentido, vale notar que, mesmo que o programa `MersennePrimes` paralelizado tivesse rodado até a conclusão, ele não teria impresso os primos na ordem correta (ascendente). Para preservar a ordem exibida pela versão sequencial, você teria que substituir a operação de terminal `forEach` por `forEachOrdered`, que garante atravessar fluxos paralelos na ordem de encontros.

Mesmo assumindo que você está usando um fluxo fonte divisível eficientemente, uma operação de terminal paralelizável ou barata, e objetos de função não interferentes, você não terá uma boa aceleração com a paralelização a menos que o pipeline esteja fazendo trabalho real suficiente para compensar os custos associados ao paralelismo. Como uma estimativa muito aproximada, o número de elementos no fluxo multiplicado pelo número de linhas de código executado por elemento deve ser pelo menos cem mil [Lea14].

É importante lembrar que paralelizar um fluxo é estritamente uma otimização de desempenho. Como ocorre com qualquer otimização, você deve testar o desempenho antes e depois da mudança para garantir que vale a pena (Item 67). Idealmente, você deve realizar o teste em um ambiente de sistema realista. Normalmente, todos os pipelines paralelos de fluxo em um programa rodam em um pool comum de `fork-join`. Um único pipeline com comportamento inadequado pode prejudicar o desempenho de outros em partes não relacionadas do sistema.

Se parece que as chances estão contra você ao paralelizar `stream` pipelines, é porque estão. Um conhecido que mantém um código de vários milhões de linhas e faz uso intensivo de `streams` encontrou apenas alguns poucos lugares onde `streams` paralelos foram eficazes. Isso não significa que você deva evitar paralelizar fluxos. Sob as circunstâncias certas, é possível alcançar um aumento quase linear de aceleração no número de núcleos de processador simplesmente adicionando uma chamada paralela a um pipeline de fluxo. Certos domínios, como aprendizado de máquina e processamento de dados, são particularmente receptivos a esses aumentos de aceleração.

Como um exemplo simples de um pipeline de fluxo onde o paralelismo é eficaz, considere essa função para calcular $\pi(n)$, o número de primos menores ou iguais a n :

```
Pipeline de fluxo com contagem de primos - se beneficia da
paralelização long pi(long n) {
    return LongStream.rangeClosed(2, n)
        .mapToObj(BigInteger::valueOf).filter(
            i -> i.isProbablePrime(50)).count();
}
```

Na minha máquina, leva 31 segundos para calcular $\pi(108)$ usando essa função. Simplesmente adicionar uma chamada `paralela()` reduz o tempo para 9,2 segundos:

```
Pipeline de fluxo de contagem de primos - versão
paralela estática long pi(long n) {
    return LongStream.rangeClosed(2, n)
        .parallel().mapToObj(BigInteger::value
            Of).filter(i ->
            i.isProbablePrime(50)).count();
}
```

Em outras palavras, paralelizar o cálculo acelera por um fator de 3,7 na máquina `myquad-core`. Vale notar que não é assim que você calcularia $\pi(n)$ para valores grandes de n na prática. Existem algoritmos muito mais eficientes, notadamente a fórmula de Lehmer.

Se você for paralelizar um fluxo de números aleatórios, comece com uma instância `SplittableRandom` em vez de um `ThreadLocalRandom` (ou o essencialmente obsoleto `Random`). O `SplittableRandom` foi projetado exatamente para esse uso e tem potencial para aceleração linear. `ThreadLocalRandom` foi projetado para uso por uma única thread e se adaptará para funcionar como uma fonte de fluxo paralelo, mas não será tão rápido quanto o `SplittableRandom`. Sincroniza aleatoriamente em toda operação, então resultará em uma contenção excessiva que mata o paralelismo.

Em resumo, nem tente paralelizar um pipeline de fluxo a menos que você tenha bons motivos para acreditar que ele preservará a correção do cálculo e aumentará sua velocidade. O custo de paralelizar inadequadamente um fluxo pode ser uma falha de programa ou desastre de desempenho. Se você acredita que o paralelismo pode ser justificado, certifique-se de que seu código permaneça correto quando executado em paralelo e faça medições cuidadosas de desempenho em condições realistas. Se seu código permanecer correto e esses experimentos confirmarem sua suspeita de aumento de desempenho, então e somente então paralelize o fluxo no código de produção.

Esta página intencionalmente deixada em branco

CHAPTER 8

Métodos

ESTE capítulo discute vários aspectos do design de métodos: como tratar parâmetros e valores de retorno, como projetar assinaturas de métodos e como documentar métodos. Grande parte do material deste capítulo se aplica tanto a construtores quanto a métodos. Assim como o Capítulo 4, este capítulo foca em usabilidade, robustez e flexibilidade.

Item 49: Verifique parâmetros quanto à validade

A maioria dos métodos e construtores tem algumas restrições sobre quais valores podem ser transferidos em seus parâmetros. Por exemplo, não é incomum que os valores de índice sejam não negativos e as referências de objetos não nulas. Você deve documentar claramente todas essas restrições e aplicá-las com verificações no início do processo de processo. Este é um caso especial do princípio geral de que você deve tentar detectar erros o mais rápido possível após eles ocorrerem. Não fazer isso torna menos provável que um erro seja detectado e dificulta determinar a origem do erro depois de detectado.

Se um valor de parâmetro inválido for passado para um método e o método verificar seus parâmetros antes da execução, ele falhará rápida e limpamente com uma exceção apropriada. Se o método não verificar seus parâmetros, várias coisas podem acontecer. O método pode falhar com uma exceção confusa no meio do processamento. Pior ainda, o método pode retornar normalmente, mas calcular silenciosamente o resultado errado. O pior de tudo é que o método pode retornar normalmente, mas deixar algum objeto em estado comprometido, causando um erro em algum ponto não relacionado do código, em um momento indeterminado no futuro. Em outras palavras, a falha em validar parâmetros pode resultar em violação da atomicidade da falha (Item 76).

Para métodos públicos e protegidos, use a tag Javadoc `@throws` para documentar a exceção que será lançada se uma restrição nos valores dos parâmetros for violada

(Item 74). Normalmente, a exceção resultante será `ExceçãoArgumentoIlegal`, `ExceçãoIndeLimites`, ou `ExceçãoPontoNula` (Item 72). Depois de documentar as restrições nos parâmetros de um método e documentar as exceções que serão lançadas caso essas restrições sejam violadas, é fácil fazer cumprir as restrições. Aqui está um exemplo típico:

```
/**
 * Retorna um BigInteger cujo valor é (este mod m).      Esse método
 * difere do método do resto porque sempre retorna um BigInteger não
 * negativo.** @param o módulo, que deve ser positivo* @return esse
 * mod m* @throws ExceçãoAritmética se m for menor ou igual a
 * 0*/BigInteger público mod(BigInteger m) {

    se (m.signum() <= 0)
    lançar um novo ArithmeticException("Modulus <= 0: " + m);... //
    Faça o cálculo}
```

Note que o comentário doc não diz "mod throws NullPointerException if m is null", embora o método faça exatamente isso, como um subproduto de `invokingm.signum()`. Essa exceção está documentada no comentário documentário em nível de classe para a classe `BigInteger` que encerra. O comentário em nível de turma se aplica a todos os parâmetros em todos os métodos públicos da turma. Essa é uma boa forma de evitar a confusão de documentar cada `NullPointerException` em cada método individualmente. Pode ser combinado com o uso de `@Nullable` ou uma anotação semelhante para indicar que um determinado parâmetro pode ser nulo, mas essa prática não é padrão, e múltiplas notações são usadas para esse propósito.

O método `Objects.requireNonNull`, adicionado no Java 7, é flexível e conveniente, então não há mais motivo para fazer verificações de null manualmente. Você pode especificar sua própria mensagem de detalhamento de exceção, se quiser. O método retorna sua entrada, então você pode realizar uma verificação de nulo ao mesmo tempo em que usa um valor:

```
Uso inline da facilidade de verificação nula do Java
this.this.strategy = Objects.requireNonNull(strategy, "strategy");
```

Você também pode ignorar o valor de retorno e usar `Objects.requireNonNull` como uma verificação de null independente onde isso atender às suas necessidades.

No Java 9, uma funcionalidade de verificação de alcance foi adicionada ao `java.util.Objects`. Essa funcionalidade consiste em três métodos: `checkFromIndexSize`, `checkFromToIndex` e `checkIndex`. Essa funcionalidade não é tão flexível quanto o método de verificação nula. Ele não permite que você especifique sua própria mensagem de detalhamento de exceção, e é projetado exclusivamente para uso em índices de lista e array. Ele não lida com intervalos fechados (que contêm ambos os extremos). Mas se ele faz o que você precisa, é uma conveniência útil.

Para um método não exportado, você, como autor do pacote, controla as circunstâncias em que o método é chamado, então pode e deve garantir que apenas valores válidos de parâmetros sejam passados. Portanto, métodos não públicos podem verificar seus parâmetros usando asserções, conforme mostrado abaixo:

```
Função auxiliar privada para um sorting recursivo estático void  
sort(long a[], int offset, int length) {  
    assert a != null; afirmar deslocamento >= 0 & deslocamento  
    <= a.comprimento; assert comprimento >= 0 && comprimento  
    <= a.comprimento - deslocamento;... // Faça o cálculo}
```

Em essência, essas afirmações são alegações de que a condição alegada será verdadeira, independentemente de como o pacote anexo seja usado por seus clientes. Diferente das verificações de normalvalidade, as asserções lançam `AssertionError` se falharem. E diferente das verificações de normalvalidade, elas não têm efeito e basicamente nenhum custo, a menos que você as ative, o que você faz passando a flag `-ea` (ou `-enableassertions`) para o comando `java`. Para mais informações sobre asserções, veja o tutorial [Asserts].

É particularmente importante verificar a validade de parâmetros que não são usados por um método, mas armazenados para uso posterior. Por exemplo, considere o método `factory` estático na página 101, que pega um `array int` e retorna uma visualização de `Lista` do array. Se um cliente passasse `null`, o método lançaria um `NullPointerException` porque o método tem uma verificação explícita (a chamada para `Objects.requireNonNull`). Se a verificação tivesse sido omitida, o método retornaria uma referência a uma instância recém-criada da `Lista` que lançaria um `NullPointerException` assim que um cliente tentasse usá-la. Nesse momento, a origem da instância `List` poderia ser difícil de determinar, o que poderia complicar muito a tarefa de depuração.

Construtores representam um caso especial do princípio de que você deve verificar a validade dos parâmetros que devem ser armazenados para uso posterior. É fundamental verificar a validade dos parâmetros do construtor para evitar a construção de um objeto que viole seus invariantes de classe.

Existem exceções à regra de que você deve verificar explicitamente os parâmetros de um método antes de realizar seu cálculo. Uma exceção importante é o caso

em que a verificação de validade seria cara ou impraticável e a verificação seria realizada implicitamente no processo de cálculo. Por exemplo, considere um método que ordena uma lista de objetos, como `Collections.sort(List)`. Todos os objetos da lista devem ser mutuamente comparáveis. No processo de ordenar a lista, cada objeto da lista será comparado com outro objeto da lista. Se os objetos não forem mutuamente comparáveis, uma dessas comparações gerará uma `ClassCastException`, que é exatamente o que o método `sort` deve fazer. Portanto, não faria sentido verificar antecipadamente se os elementos da lista eram mutuamente comparáveis. Note, no entanto, que a confiança indiscriminada em verificações implícitas de validade pode resultar na perda de atomicidade da falha (Item 76).

Ocasionalmente, um cálculo realiza implicitamente uma verificação de validade obrigatória, mas lança a exceção errada se a verificação falhar. Em outras palavras, a exceção que o cálculo naturalmente lançaria como resultado de um valor inválido do parâmetro não corresponde à exceção que o método está documentado para lançar. Nessas circunstâncias, você deve usar o idioma de tradução de exceções, descrito no Item 73, para traduzir a exceção natural para a correta.

Não infira a partir desse item que restrições arbitrárias nos parâmetros são algo bom. Pelo contrário, você deve projetar métodos para serem o mais gerais possível. Quanto menos restrições você impor aos parâmetros, melhor, assumindo que o método consiga fazer algo razoável com todos os valores de parâmetros que aceita. Frequentemente, porém, algumas restrições são intrínsecas à abstração que está sendo implementada.

Para resumir, cada vez que você escreve um método ou construtor, deve pensar em quais restrições existem em seus parâmetros. Você deve documentar essas restrições e aplicá-las com verificações explícitas no início do corpo do método. É importante criar o hábito de fazer isso. O trabalho modesto que isso implica será pago com juros na primeira vez que uma verificação de validade falhar.

Item 50: Faça cópias defensivas quando necessário

Uma coisa que torna o Java um prazer de usar é que ele é uma linguagem segura. Isso significa que, na ausência de métodos nativos, ele é imune a overruns de buffer, arrayovers, wild pointers e outros erros de corrupção de memória que afetam linguagens não seguras como C e C++. Em uma linguagem segura, é possível escrever classes e saber com certeza que seus invariantes serão válidos, não importa o que aconteça em qualquer outra parte do sistema. Isso não é possível em linguagens que tratam toda a memória como um grande array.

Mesmo em uma linguagem segura, você não fica isolado das outras turmas sem algum esforço da sua parte. Você deve programar defensivamente, partindo do pressuposto de que os clientes da sua classe farão o possível para destruir seus invariantes. Isso se torna cada vez mais verdadeiro à medida que as pessoas se esforçam mais para quebrar a segurança dos sistemas, mas, mais comumente, sua turma terá que lidar com comportamentos inesperados resultantes dos erros honestos de programadores bem-intencionados. De qualquer forma, vale a pena dedicar tempo para escrever aulas robustas diante de clientes malcomportados.

Embora seja impossível para outra classe modificar o estado interno de um objeto sem alguma assistência do objeto, é surpreendentemente fácil fornecer tal assistência sem querer fazê-lo. Por exemplo, considere a seguinte classe, que pretende representar um período de tempo imutável:

```

Período de tempo "imutável" quebrado
classe pública final classe Ponto {
    private final de encontro;
    privado final fim do encontro;

    /**
     * @param início do período
     * @param encerrar o fim do período; não deve preceder o início
     * @throws IllegalArgumentException se o início for após o fim*
     * @throws NullPointerException se o início ou o fim for
     null*/public Period(Data de início, Data de término) {

        if (start.compareTo(end) > 0)
            Lançar um novo IllegalArgumentException(
                início + " após " + fim);
        this.start = start;
        this.end= fim;}

    public Data início() {
        retorno inicial;}

```

```

        public Data final() {
            retorno fim ;}

        ...   Restante omitido
    }

```

À primeira vista, essa classe pode parecer imutável e impor o invariante de que o início de um período não segue seu fim. No entanto, é fácil violar esse invariante explorando o fato de que *Data* é mutável:

```

Ataque os componentes internos de um período
instance Data start = nova Data(); Fim da data =
nova Data(); Período p = novo Período (início,
fim); end.setYear(78);// Modifica os internos de
p!

```

A partir do Java 8, a maneira óbvia de resolver esse problema é usar *Instant* (ou *LocalDateTime*, ou *ZonedDateTime*) no lugar de uma *Data*, porque *Instant* (e as classes *otherjava.time*) são imutáveis (Item 17). A *data* está obsoleta e não deve ser usado nenhum lugar em um novo código. Dito isso, o problema ainda existe: há momentos em que você terá que usar tipos de valor mutáveis em suas APIs e representações internas, e as técnicas discutidas neste item são adequadas para esses momentos.

Para proteger os componentes internos de uma instância de período contra esse tipo de ataque, é essencial fazer uma cópia defensiva de cada parâmetro mutável para o construtor e usar as cópias como componentes da instância de período em vez dos originais:

```

Construtor reparado - faz cópias defensivas dos parâmetros public
Period(Data de início, Data de término) {
    this.start = nova Data(start.getTime());
    this.end= nova Data(fim.obterTempo());

    if (this.start.compareTo(this.end) > 0)
        lançar um novo
        IllegalArgumentException(this.start + " depois de
        " + this.end);}

```

Com o novo construtor instalado, o ataque anterior não terá efeito na instância do *Período*. Note que cópias defensivas são feitas antes de verificar a validade dos parâmetros (Item 49), e a verificação de validade é realizada nas cópias em vez dos originais. Embora isso possa parecer antinatural, é necessário. Ela protege a classe contra alterações nos parâmetros de outra

thread durante a janela de vulnerabilidade entre o momento em que os parâmetros são verificados e o momento em que são copiados. Na comunidade de segurança informática, isso é conhecido como ataque de tempo de verificação/horário de uso ou TOCTOU [Viega01].

Note também que não usamos o método clone do Date para fazer as cópias defensivas. Como Date não é final, o método clone não garante que retorne um objeto cuja classe seja java.util.Date: ele pode retornar uma instância de uma subclasse untrustedsubclass especificamente projetada para malícias. Tal subclasse poderia, por exemplo, registrar uma referência a cada instância em uma lista estática privada no momento de sua criação e permitir que o atacante acessasse essa lista. Isso daria ao atacante total liberdade sobre todas as instâncias. Para evitar esse tipo de ataque, não use o método theclone para criar uma cópia defensiva de um parâmetro cujo tipo é subclassificado por partes não confiáveis.

Embora o construtor substituto defenda com sucesso contra o ataque anterior, ainda é possível mutar uma instância de Período, pois seus acessores oferecem acesso aos seus internos mutáveis:

```
Segundo ataque ao interior de um Período instance Data
start = nova Data(); Fim da data = nova Data(); Período p
= novo Período(início, fim); p.end().setYear(78);//
Modifica os internos de p!
```

Para se defender do segundo ataque, basta modificar os acessores para devolver cópias defensivas de campos internos mutáveis:

```
Acessórios reparados - criar cópias defensivas dos campos internos
público Data início() {
return new Data(start.getTime());}

public Data final() {
return new Date(end.getTime());}
```

Com o novo construtor e os novos acessórios instalados, Period é verdadeiramente imutável. Não importa o quão malicioso ou incompetente seja o programador, não há como violar o invariante de que o início de um período não segue seu fim (sem recorrer a meios extralinguísticos como métodos nativos e reflexão). Isso é verdade porque não há como qualquer classe além do próprio Período obter acesso a qualquer um dos campos mutáveis em uma instância de Período. Esses campos estão verdadeiramente encapsulados dentro do objeto.

Nos acessórios, ao contrário do construtor, seria permitido usar o método do clone para fazer as cópias defensivas. Isso porque sabemos que a classe dos objetos `Data` internos do `Period` é `java.util.Date`, e não alguma subclasse não confiável. Dito isso, geralmente é melhor usar uma fábrica de construtores estática para copiar uma instância, pelos motivos explicados no Item 13.

A cópia defensiva dos parâmetros não é apenas para classes imutáveis. Sempre que você escrever um método ou construtor que armazene uma referência a um objeto fornecido pelo cliente em uma estrutura de dados interna, pense se o objeto fornecido pelo cliente é potencialmente mutável. Se for, pense se sua classe poderia tolerar uma mudança no objeto depois que ele foi inserido na estrutura de dados. Se a resposta for não, você deve copiar defensivamente o objeto e inserir a cópia na estrutura de dados no lugar do original. Por exemplo, se você está considerando usar uma referência de objeto fornecido pelo cliente como elemento em uma instância interna de `Set` ou como chave em uma instância `internalMap`, deve estar ciente de que os invariantes do conjunto ou mapa seriam corrompidos se o objeto fosse modificado após ser inserido.

O mesmo vale para a cópia defensiva dos componentes internos antes de devolvê-los aos clientes. Seja sua classe imutável ou não, você deve pensar duas vezes antes de retornar uma referência a um componente interno que seja mutável. É provável que você deva devolver uma cópia defensiva. Lembre-se de que `longarrays` não nulos são sempre mutáveis. Portanto, você deve sempre fazer uma cópia defensiva de um array interno antes de retorná-lo a um cliente. Alternativamente, você poderia devolver uma visão imutável do array. Ambas as técnicas são mostradas no Item 15.

Pode-se argumentar que a verdadeira lição de tudo isso é que você deve, sempre que possível, usar objetos imutáveis como componentes dos seus objetos, para que não precise se preocupar com cópias defensivas (Item 17). No caso do nosso exemplo de `Periodo`, `useInstant` (ou `LocalDateTime` ou `ZonedDateTime`), a menos que você esteja usando uma versão anterior ao Java 8. Se você estiver usando uma versão anterior, uma opção é armazenar `theprimitive long returned por Date.getTime()` no lugar de uma referência de `Data`.

Pode haver uma penalidade de desempenho associada à cópia defensiva e nem sempre é justificada. Se uma classe confia em seu chamador para não modificar um `internalcomponent`, talvez porque a classe e seu cliente fazem parte do mesmo pacote, então pode ser apropriado dispensar a cópia defensiva. Nessas circunstâncias, a documentação da classe deve deixar claro que o chamador não deve modificar os parâmetros ou valores de retorno afetados.

Mesmo entre os limites dos pacotes, nem sempre é apropriado fazer uma cópia defensiva de um parâmetro mutável antes de integrá-lo a um objeto. Existem alguns métodos e construtores cuja invocação indica uma transferência explícita do objeto referenciado por um parâmetro. Ao invocar tal método, o cliente

promete que não modificará mais o objeto diretamente. Um método ou construtor que espera assumir a propriedade de um objeto mutável fornecido pelo cliente deve deixar isso claro em sua documentação.

Classes contendo métodos ou construtores cuja invocação indica uma transferência de controle não podem se defender contra clientes maliciosos. Tais classes são aceitáveis apenas quando há confiança mútua entre uma classe e seu cliente, quando danos aos invariantes da classe prejudicariam ninguém além do cliente. Um exemplo da última situação é o padrão de classe wrapper (Item 18). Dependendo da natureza da classe de wrapper, o cliente pode destruir os invariantes da classe acessando diretamente um objeto após ele ter sido embrulhado, mas isso normalmente prejudicaria apenas o cliente.

Em resumo, se uma classe possui componentes mutáveis que obtém ou retorna aos seus clientes, a classe deve copiar defensivamente esses componentes. Se o custo da cópia for proibitivo e a classe confiar em seus clientes que não modificarão os componentes de forma inadequada, então a cópia defensiva pode ser substituída por documentação que descreve a responsabilidade do cliente de não modificar os componentes afetados.

Item 51: Assinaturas cuidadosamente feitas pelo método de design

Este item é um conjunto de dicas de design de APIs que não merecem itens próprios. Juntos, eles vão ajudar a tornar sua API mais fácil de aprender e usar, além de ser menos propensa a erros.

Escolha cuidadosamente os nomes dos métodos. Os nomes devem sempre obedecer às convenções de nomeação padrão (Item 68). Seu objetivo principal deve ser escolher nomes que sejam compreensíveis e consistentes com outros nomes do mesmo pacote. Seu objetivo secundário deve ser escolher nomes consistentes com o consenso mais amplo, onde ele existe. Evite nomes longos de métodos. Em caso de dúvida, procure orientações nas APIs de bibliotecas do Java. Embora haja muitas inconsistências — inevitáveis, dado o tamanho e o escopo dessas bibliotecas — também há um bom consenso.

Não exagere em oferecer métodos de conveniência. Todo método deve "fazer sua parte". Muitos métodos tornam uma aula difícil de aprender, usar, documentar, testar e manter. Isso é ainda mais verdadeiro para interfaces, onde muitos métodos complicam a vida tanto para implementadores quanto para usuários. Para cada ação suportada pela sua classe ou interface, forneça um método totalmente funcional. Considere fornecer uma "abreviação" apenas se for usada com frequência. Em caso de dúvida, deixe de lado.

Evite listas longas de parâmetros. Mire em quatro parâmetros ou menos. A maioria dos programadores não consegue lembrar listas de parâmetros mais longas. Se muitos dos seus métodos ultrapassarem esse limite, sua API não será utilizável sem referência constante à sua documentação. IDEs modernos ajudam, mas você ainda é muito melhor com listas de parâmetros curtos. Sequências longas de parâmetros idênticos são especialmente prejudiciais. Além de os usuários não conseguirem lembrar a ordem dos parâmetros, quando transpõem parâmetros acidentalmente, seus programas ainda compilam e rodam. Eles simplesmente não fazem o que seus autores pretendiam.

Existem três técnicas para encurtar listas de parâmetros excessivamente longas. Uma é dividir o método em múltiplos métodos, cada um dos quais requer apenas um subconjunto dos parâmetros. Se feito de forma descuidada, isso pode levar a muitos métodos, mas também pode ajudar a reduzir a quantidade de métodos ao aumentar a ortogonalidade. Por exemplo, considere a interface `java.util.List`. Ele não fornece métodos para encontrar o primeiro ou último índice de um elemento em uma sublista, ambos exigiriam três parâmetros. Em vez disso, fornece o método `subList`, que assume dois parâmetros e retorna uma visão de uma sublista. Esse método pode ser combinado com o método `indexOf` ou `lastIndexOf`, cada um dos quais possui um único parâmetro, para obter a funcionalidade desejada. Além disso, o método `subList` pode ser combinado com qualquer método que opere em uma instância de `List` para realizar cálculos arbitrários em sublistas. A API resultante possui uma relação potência-peso muito alta.

Uma segunda técnica para encurtar listas longas de parâmetros é criar classes auxiliares para armazenar grupos de parâmetros. Normalmente, essas classes auxiliares são classes de membro estático (Item 24). Essa técnica é recomendada se uma sequência de parâmetros em anel que ocorre frequentemente representa alguma entidade distinta. Por exemplo, suponha que você esteja escrevendo uma classe representando um jogo de cartas, e você se veja constantemente passando uma sequência de dois parâmetros que representam o valor e o naipe de uma carta. Sua API, assim como os internos da sua classe, provavelmente se beneficiariam se você adicionasse uma classe auxiliar para representar uma carta e substituísse toda ocorrência da sequência de parâmetros por um único parâmetro da classe auxiliar.

Uma terceira técnica que combina aspectos das duas primeiras é adaptar o Builderpattern (Item 2) da construção do objeto para a invocação do método. Se você tem um método com muitos parâmetros, especialmente se alguns deles forem opcionais, pode ser benéfico definir um objeto que represente todos os parâmetros e permitir que o cliente faça múltiplas chamadas "setter" nesse objeto, cada um dos quais define um único parâmetro ou um pequeno grupo relacionado. Uma vez definidos os parâmetros desejados, o cliente invoca o método "executar" do objeto, que realiza as verificações finais de validade dos parâmetros e realiza o cálculo real.

Para tipos de parâmetros, prefira interfaces em vez de classes (Item 64). Se houver uma interface apropriada para definir um parâmetro, use-a em favor de uma classe que implemente a interface. Por exemplo, não há motivo para escrever um método que receba `HashMap` na entrada — use `Map` em vez disso. Isso permite que você passe um `HashMap`, um `TreeMap`, um `ConcurrentHashMap`, um `SubMap` de um `TreeMap` ou qualquer implementação de `Map` ainda por escrever. Ao usar uma classe em vez de uma interface, você restringe seu cliente a uma implementação específica e força uma operação de cópia desnecessária e potencialmente cara caso os dados de entrada existam em outra forma.

Prefira tipos de `enum` de dois elementos a parâmetros booleanos, a menos que o significado do booleano seja claro pelo nome do método. Enums facilitam sua leitura e escrita no seu código. Além disso, facilitam adicionar mais opções depois. Por exemplo, você pode ter um termômetro com uma fábrica estática que aceita esse `enum`:

```
public enum TemperatureScale { FAHRENHEIT, CELSIUS }
```

Além de fazer muito mais sentido o

`Thermometer.newInstance(TemperatureScale.CELSIUS)` faz muito mais sentido do que o `Thermometer.newInstance(true)`, como você pode adicionar `KELVIN` ao `TemperatureScale` em uma versão futura sem precisar adicionar uma nova fábrica estática ao `Thermometer`. Além disso, você pode refatorar dependências da escala de temperatura em métodos sobre as constantes de `enum` (Item 34). Por exemplo, cada constante de escala poderia ter um método que pegasse um valor duplo e o convertesse em Celsius.

Item 52: Use a sobrecarga com critério

O programa a seguir é uma tentativa bem-intencionada de classificar coleções de acordo com se são conjuntos, listas ou algum outro tipo de coleção:

```

Quebrado! - O que este programa imprime?
classe pública CollectionClassifier {
    public static String classify(Set<?> s) {
        retorne "Set";}

    Public static String classify(List<?> lst) {
        return "Lista";}

    public static String classify(Collection<?> c) {
        voltar "Coleção Desconhecida";}

    public static void main(String[] args) {
        Coleção<?>[] coleções = {
            novo <String>HashSet(), novo
            <BigInteger>ArrayList(), novo HashMap<String,
            String>().values());

        para (Coleção<?> c : coleções)
            System.out.println(classificar(c));
    }
}

```

Você poderia esperar que este programa imprimissem Set, seguido de List e UnknownCollection, mas não imprime. Ele imprime a Coleção Desconhecida três vezes. Por que isso acontece? Porque o método classify está sobrecarregado, e a escolha de qual sobrecarga invocar é feita em tempo de compilação. Para as três iterações do ciclo, o tipo de tempo de compilação do parâmetro é o mesmo: Collection<?>. O tipo de tempo de Theruntime é diferente a cada iteração, mas isso não afeta a escolha de sobrecarga. Como o tipo de tempo de compilação do parâmetro é Collection<?>, a única sobrecarga aplicável é a terceira, classificar(Collection<?>), e essa sobrecarga é invocada em cada iteração do ciclo.

O comportamento desse programa é contraintuitivo porque a seleção entre métodos sobrecarregados é estática, enquanto a seleção entre métodos sobrepostos é dinâmica. A versão correta de um método sobrescrito é escolhida em tempo de execução,

baseado no tipo de tempo de execução do objeto no qual o método é invocado. Como lembrete, um método é sobrescrito quando uma subclasse contém uma declaração de método com a mesma assinatura que uma declaração de método em um ancestral. Se um método de instância for sobrescrito em uma subclasse e esse método for invocado em uma instância da subclasse, o método de substituição da subclasse é executado, independentemente do tipo em tempo de compilação da instância da subclasse. Para concretizar isso, considere o seguinte programa:

```

classe Wine {
    Nome da cadeia() { return "wine"; }}

classe SparklingWine estende Wine {
    @Override String name() { return "sparkling wine"; }}

classe Champagne estende SparklingWine {
    @Override String name() { return "champagne"; }}

classe pública Sobreescrevendo {
    public static void main(String[] args) {
        Lista<Wine> vinhoLista = Lista.de(
            novo Vinho(), novo EspumanteVinho (), novo Champanhe());

        para (Vinho vinho: vinhoLista)
            System.out.println(vinho.name());
    }}

```

O método do nome é declarado na classe Vinho e é substituído nas subclasses Vinho Espumante e Champanhe. Como era de se esperar, este programa imprime vinho, espumante e champanhe, mesmo que o tipo de compilação da instância seja Vinho em cada iteração do loop. O tipo em tempo de compilação de um objeto não tem efeito sobre qual método é executado quando um método sobrescrito é invocado; O método "mais específico" de sobreposição sempre é executado. Compare isso com sobrecarga, onde o tipo de runtime de um objeto não tem efeito sobre qual overload é executado; A seleção é feita em tempo de compilação, baseada inteiramente nos tipos de tempo de compilação dos parâmetros.

No exemplo do CollectionClassifier, a intenção do programa era discernir o tipo do parâmetro despachando automaticamente para a sobrecarga do método apropriado com base no tipo de runtime do parâmetro, assim como o `namemethod` fez no exemplo do Wine. A sobrecarga de métodos simplesmente não proporciona isso

funcionalidade. Assumindo que um método estático seja necessário, a melhor forma de corrigir o programa `CollectionClassifier` é substituir todas as três sobrecargas do classificador por um único método que faça testes explícitos de instância:

```
public static String classify(Collection<?> c) {  
    retorne c instância de Set ? "Set":  
        c instância de Lista ? "Lista": "Coleção Desconhecida";  
}
```

Como a sobreposição é a norma e a sobrecarga é a exceção, a sobreposição define as expectativas das pessoas quanto ao comportamento da invocação do método. Como demonstrado pelo exemplo do `CollectionClassifier`, a sobrecarga pode facilmente confundir essas expectativas. É má prática escrever código cujo comportamento provavelmente confunda os programadores. Isso é especialmente verdadeiro para APIs. Se o usuário típico de uma API não souber qual dos vários métodos sobrecarregados será invocado para um determinado conjunto de parâmetros, o uso da API provavelmente resultará em erros. Esses erros provavelmente se manifestarão como comportamento errático em tempo de execução, e muitos programadores terão dificuldade para diagnosticá-los. Portanto, você deve evitar usos confusos de sobrecarga.

Exatamente o que constitui um uso confuso de sobrecarga é motivo de debate. Uma política segura e conservadora nunca é exportar duas sobrecargas com o mesmo número de parâmetros. Se um método usa varargs, uma política conservadora é não sobrecarregá-lo de forma alguma, exceto conforme descrito no Item 53. Se você seguir essas restrições, os programadores nunca terão dúvidas sobre qual sobrecarga se aplica a qualquer conjunto real de parâmetros. Essas restrições não são tão onerosas porque você sempre pode dar nomes diferentes aos métodos em vez de sobrecarregá-los.

Por exemplo, considere a classe `ObjectOutputStream`. Ele possui uma variante do método `write` para cada tipo primitivo e para vários tipos de referência. Em vez de sobrecarregar o método de escrita, essas variantes têm nomes diferentes, como `writeBoolean(boolean)`, `writeInt(int)` e `writeLong(long)`. Um benefício adicional desse padrão de nomenclatura, quando comparado à sobrecarga, é que é possível fornecer métodos de leitura com nomes correspondentes, por exemplo, `readBoolean()`, `readInt()` e `readLong()`. A classe `ObjectInputStream`, de fato, fornece esses métodos de leitura.

Para construtores, você não tem a opção de usar nomes diferentes: múltiplos construtores para uma classe estão sempre sobrecarregados. Em muitos casos, você tem a opção de exportar fábricas estáticas em vez de construtores (Item 1). Além disso, com construtores você não precisa se preocupar com interações entre sobrecarga e sobreposição, porque construtores não podem ser sobrescritos. Provavelmente você terá

É motivo para exportar múltiplos construtores com o mesmo número de parâmetros, então vale a pena saber como fazer isso com segurança.

Exportar múltiplas sobrecargas com o mesmo número de parâmetros dificilmente confundirá os programadores se estiver sempre claro qual sobrecarga se aplicará a qualquer conjunto de parâmetros reais. Isso ocorre quando pelo menos um parâmetro formal correspondente em cada par de sobrecargas tem um tipo "radicalmente diferente" nas duas sobrecargas. Dois tipos são radicalmente diferentes se for claramente impossível conjurar qualquer expressão não nula para ambos os tipos. Nessas circunstâncias, qual sobrecarga se aplica a um determinado conjunto de parâmetros reais é determinada pelos tipos de tempo de execução dos parâmetros e não pode ser afetada por seus tipos de compilação, então uma grande fonte de confusão desaparece. Por exemplo, o `ArrayList` tem um construtor que recebe um `int` e um segundo construtor que recebe uma `Collection`. É difícil imaginar qualquer confusão sobre qual desses dois construtores será invocado em qualquer circunstância.

Antes do Java 5, todos os tipos primitivos eram radicalmente diferentes de todos os tipos de referência, mas isso não é verdade na presença do autoboxing, e isso causou sérios problemas. Considere o seguinte programa:

```
classe pública SetList {
    public static void main(String[] args) {
        Set<Integer> set = novo TreeSet<>(); <Integer>
        lista de listas = nova ArrayList<>();

        para (int i = -3; i < 3; i++) {
            set.add(i); list.add(i);}para
            (int i = 0; i < 3; i++) {

set.remove(i);
list.remove(i);}System.out.println(set + " " +
lista);}}
```

Primeiro, o programa soma os inteiros de -3 a 2 , inclusive, a um conjunto ordenado e alist. Depois, faz três chamadas idênticas para remover do conjunto e da lista. Se você é como a maioria das pessoas, esperaria que o programa removesse os valores não negativos (0 , 1 e 2) do conjunto e da lista e imprimisse $[-3, -2, -1]$ $[-3, -2, -1]$. Na verdade, o programa remove os valores não negativos do conjunto e os valores ímpares da lista e imprime $[-3, -2, -1]$ $[-2, 0, 2]$. É pouca dizer que esse comportamento é confuso.

Aqui está o que está acontecendo: A chamada para `set.remove(i)` seleciona o `sobrecarregamentoremove(E)`, onde `E` é o tipo de elemento do conjunto (Inteiro), e `autoboxa ifrom int` para Inteiro. Esse é o comportamento que você esperaria, então o programa acaba removendo os valores positivos do conjunto. A chamada para `list.remove(i)`, por outro lado, seleciona a sobrecarga `remove(int i)`, que remove o elemento na posição especificada na lista. Se você começar com a lista `[-3, -2, -1, 0, 1, 2]` e remover o elemento zero, depois o primeiro, e depois o segundo, você fica com `[-2, 0, 2]`, e o mistério é resolvido. Para resolver o problema, faça o `cast.remove'sargument` para Inteiro, forçando a seleção correta da sobrecarga. Alternativamente, você pode invocar `Integer.valueOf` em `i` e passar o resultado para `list.remove`. De qualquer forma, o programa imprime `[-3, -2, -1] [-3, -2, -1]`, como esperado:

```
para (int i = 0; i < 3; i++) {
    set.remove(i);
    list.remove((Inteiro) i);    ou remove(Integer.valueOf(i))
}
```

O comportamento confuso demonstrado pelo exemplo anterior surgiu porque `a<E> interface List` tem duas sobrecargas do método `remove:remove(E)` e `remove(int)`. Antes do Java 5, quando a interface `List` era "genérica", ela tinha um método `remove(Object)` no lugar de `remove(E)`, e os tipos de parâmetros correspondentes, `Object` e `int`, eram radicalmente diferentes. Mas, na presença dos genéricos e do autoboxing, os dois tipos de parâmetros não são mais radicalmente diferentes. Em outras palavras, adicionar genéricos e autoboxing ao idioma danificou a interface da Lista. Felizmente, poucas, se é que nenhuma outra, APIs nas bibliotecas Java foram igualmente danificadas, mas essa história deixa claro que autoboxing e genéricos aumentaram a importância da cautela ao sobrecarregar.

A adição de lambdas e referências de métodos no Java 8 aumentou ainda mais o potencial de confusão em sobrecarga. Por exemplo, considere estes dois trechos:

```
new Thread(System.out::p rintln).start();

ExecutorService exec = Executors.newCachedThreadPool();
exec.submit(System.out::p rintln);
```

Enquanto a invocação do construtor `Thread` e a invocação do método `submit` são semelhantes, a primeira compila enquanto a segunda não. Os argumentos são idênticos (`System.out::p rintln`), e tanto o construtor quanto o método têm uma sobrecarga que toma um `Runnable`. O que está acontecendo aqui? A resposta surpreendente é que o método `submit` tem uma sobrecarga que recebe um `<T>Callable`, enquanto o construtor `theThread` não tem. Você pode pensar que isso não deveria fazer nenhum

diferença porque todas as sobrecargas do `println` retornam o `void`, então a `methodreference` não poderia ser um `Callable`. Isso faz todo sentido, mas não é assim que o algoritmo de resolução de sobrecarga funciona. Talvez igualmente surpreendente seja que a invocação do método `submit` seria legal se o método `println` não estivesse também sobrecarregado. É a combinação da sobrecarga do método referenciado (`println`) e do método invocado (`submit`) que impede que o algoritmo de resolução de sobrecarga se comporte como você esperaria.

Tecnicamente falando, o problema é que `System.out::println` é uma referência de método inexato [JLS, 15.13.1] e que "certas expressões de argumento que contêm expressões `lambda` implicitamente tipadas ou referências de método imprecisas são ignoradas pelos testes de aplicabilidade, porque seu significado não pode ser determinado até que um tipo alvo seja selecionado [JLS, 15.12.2]." Não se preocupe se não entender esta passagem; É direcionado a escritores de compiladores. O ponto chave é que sobrecarregar métodos ou construtores com diferentes interfaces funcionais na mesma posição de argumento causa confusão. Portanto, não sobrecarregue métodos para tomar diferentes interfaces funcionais na mesma posição argumental. Na linguagem deste item, diferentes interfaces funcionais não são radicalmente diferentes. O compilador Java vai te alertar sobre esse tipo de sobrecarga problemática se você passar no interruptor de linha de comando `-Xlint:overloads`.

Tipos de `array` e tipos de classe além do `Object` são radicalmente diferentes. Além disso, tipos de `array` e tipos de interface além de `Serializable` e `Clonable` são radicalmente diferentes. Diz-se que duas classes distintas não são relacionadas se nenhuma das classes for descendente da outra [JLS, 5.5]. Por exemplo, `Fio` e `Arremessável` não têm relação. É impossível que qualquer objeto seja uma instância de duas classes não relacionadas, então classes não relacionadas também são radicalmente diferentes.

Existem outros pares de tipos que não podem ser convertidos em nenhuma das direções [JLS, 5.1.12], mas uma vez que você vai além dos casos simples descritos acima, torna-se muito difícil para a maioria dos programadores discernir qual, se houver, sobrecarga se aplica ao conjunto de parâmetros reais. As regras que determinam qual sobrecarga é selecionada são extremamente complexas e ficam mais complexas a cada lançamento. Poucos programadores entendem todas as sutilezas deles. Pode haver momentos em que você sinta necessidade de violar as diretrizes deste item, especialmente ao evoluir as classes existentes. Por exemplo, considere o `String`, que tem um método `contentEquals(StringBuffer)` desde o Java 4. No Java 5, o `CharSequence` foi adicionado para fornecer uma interface comum para `StringBuffer`, `StringBuilder`, `String`, `CharBuffer` e outros tipos semelhantes. Ao mesmo tempo em que o `CharSequence` foi adicionado, o `String` foi equipado com um método de sobrecarga do `thecontentEquals` que aceita um `CharSequence`.

Embora a sobrecarga resultante claramente viole as diretrizes desse item, ela não causa dano porque ambos os métodos sobrecarregados fazem exatamente a mesma coisa quando são invocados na mesma referência de objeto. O programador pode não saber qual sobrecarga será invocada, mas não tem importância desde que se comportem de forma idêntica. A maneira padrão de garantir esse comportamento é ter a sobrecarga mais específica para a mais geral:

```
Garantindo que 2 métodos tenham comportamento idêntico ao encaminhar  
contentEquals(StringBuffer sb) {  
    return contentEquals((CharSequence) sb);}
```

Embora as bibliotecas Java em grande parte sigam o espírito do conselho deste item, há várias classes que o violam. Por exemplo, o `String` exporta dois métodos estáticos de fábrica sobrecarregados, `valueOf(char[])` e `valueOf(Object)`, que fazem coisas completamente diferentes quando recebem a mesma referência de objeto. Não há justificativa real para isso, e deve ser considerado uma anomalia com a potencial de confusão real.

Para resumir, só porque você pode sobrecarregar métodos não significa que deva fazer isso. Geralmente é melhor evitar sobrecarregar métodos com múltiplas assinaturas que possuem o mesmo número de parâmetros. Em alguns casos, especialmente quando construtores estão envolvidos, pode ser impossível seguir esse conselho. Nesses casos, você deve pelo menos evitar situações em que o mesmo conjunto de parâmetros pode ser passado para diferentes sobrecargas pela adição de casts. Se isso não puder ser evitado, por exemplo, porque você está adaptando uma classe existente para implementar uma nova interface, deve garantir que todas as sobrecargas se comportem de forma idêntica quando recebem os mesmos parâmetros. Se você não fizer isso, os programadores terão dificuldade em fazer uso eficaz do método ou construtor sobrecarregado, e não entenderão por que não funciona.

Item 53: Use varargs com critério

Métodos Varargs, formalmente conhecidos como métodos de aridade variável [JLS, 8.4.1], aceitam zero ou mais argumentos de um tipo especificado. A funcionalidade varargs funciona primeiro criando um array cujo tamanho é o número de argumentos passados no local da chamada, depois colocando os valores dos argumentos no array e, por fim, passando o array para o método (themethod).

Por exemplo, aqui está um método varargs que toma uma sequência de argumentos int e retorna sua soma. Como era de se esperar, o valor de soma (1, 2, 3) é 6, e o valor de soma () é 0:

```
Uso simples do varargsstatic
int sum(int... args) {
    soma int = 0; para
    (int arg : args)
    pois += arg; soma de
    retorno;}
```

Às vezes, é apropriado escrever um método que exija um ou mais argumentos de algum tipo, em vez de zero ou mais. Por exemplo, suponha que você queira escrever uma função que calcula o mínimo de seus argumentos. Essa função não está bem definida se o cliente não passar argumentos. Você pode verificar o comprimento do array em tempo de execução:

```
A MANEIRA ERRADA de usar varargs para passar um ou mais
argumentos!static int min(int... args) {
    if (args.length == 0)
        lançar um novo ArgumentoExceção ("Poucos argumentos"); int min =
        args[0]; para (int i = 1; i < args.length; i++)

        if (args[i] < min)
            min = args[i];
    retorne min;}
```

Essa solução tem vários problemas. O mais grave é que, se o cliente invocar esse método sem argumentos, ele falha em tempo de execução em vez de compilação. Outro problema é que é feio. Você precisa incluir uma verificação explícita de validade no args, e não pode usar um loop for-each a menos que inicialize o mínimo toInteger.MAX_VALUE, o que também é feio.

Felizmente, existe uma maneira muito melhor de alcançar o efeito desejado. Declare o método para dois parâmetros, um parâmetro normal do tipo especificado e

um parâmetro `varargs` desse tipo. Esta solução corrige todas as deficiências da anterior:

```
A maneira correta de usar varargs para passar um ou mais
argumentsstatic int min(int firstArg, int... remainingArgs) {
    int min = firstArg; para (int
        arg : remainingArgs)
        if (arg < min)
            min = arg;
    retorne min;}
```

Como você pode ver neste exemplo, `varargs` são eficazes em circunstâncias em que você quer um método com um número variável de argumentos. `Varargs` foram projetados para `printf`, que foi adicionado à plataforma ao mesmo tempo que `varargs`, e para a instalação central de reflexão (Item 65), que foi adaptada. Tanto a reflexão do `printf` quanto a se beneficiou enormemente dos `varargs`.

Tenha cuidado ao usar `varargs` em situações críticas de desempenho. Cada invocação de um método `varargs` causa uma alocação e inicialização de arrays. Se você determinou empiricamente que não pode arcar com esse custo, mas precisa da flexibilidade dos `varargs`, existe um padrão que permite que você tenha o melhor dos dois mundos. Suponha que você tenha determinado que 95% das chamadas para um método têm três ou menos parâmetros. Depois, declare cinco sobrecargas do método, uma com zero até três parâmetros ordinários, e um único método `varargs` para uso quando o número de argumentos exceder três:

```
público vazio foo() { }público vazio foo(int a1) { }público
vazio foo(int a1, int a2) { }público vazio foo(int a1, int
a2, int a3) { }público vazio foo(int a1, int a2, int a3,
int... descanso) { }
```

Agora você sabe que pagará o custo da criação do array apenas nos 5% de todas as invocações onde o número de parâmetros excede três. Como a maioria das otimizações de desempenho, essa técnica geralmente não é adequada, mas quando é, é uma salvação.

As fábricas estáticas para `EnumSet` usam essa técnica para reduzir ao mínimo o custo de criação de conjuntos de enum. Isso era apropriado porque era crítico que conjuntos de enum forneçam uma substituição competitiva em desempenho para campos de bits (Item 36).

Em resumo, `varargs` são inestimáveis quando você precisa definir métodos com um número variável de argumentos. Anteceda o parâmetro `varargs` com quaisquer parâmetros necessários e esteja ciente das consequências de desempenho do uso de `varargs`.

Item 54: Devolver coleções ou arrays vazios, não nulos

Não é incomum ver métodos que se parecem com este:

```
Retorna nulo para indicar uma coleção vazia. Não faça isso! lista
final privada<Cheese> queijosInStock = ...;

/**
 * @return uma lista contendo todos os queijos da loja,*ou null se
 * nenhum queijo estiver disponível para compra.*/public
List<Cheese> getCheeses() {

    devolver queijosInStock.isEmpty() ? null
        : nova ArrayList<>(queijosEmEstoque);
}
```

Não há motivo para fazer um caso especial na situação em que não há queijos disponíveis para compra. Fazer isso requer código extra no cliente para lidar com o valor possivelmente nullreturn, por exemplo:

```
Lista<Cheese> de queijos = shop.getCheeses(); if (queijos !=
null && queijos contém(Cheese.STILTON))
    system.out.println ("Muito bom, exatamente o que precisa.");
```

Esse tipo de circunloquio é necessário em quase todo uso de um método que retorna nulo no lugar de uma coleção ou array vazio. Ele é propenso a erros, porque o programador que escreve o cliente pode esquecer de escrever o código do caso especial para lidar com um retorno nulo. Tal erro pode passar despercebido por anos porque tais métodos geralmente retornam um ou mais objetos. Além disso, retornar null no lugar de um contenedor vazio complica a implementação do método que retorna o container.

Às vezes argumenta-se que um valor de retorno nulo é preferível a uma coleção vazia ou array porque evita o custo de alocar o contêiner vazio. Esse argumento falha por dois motivos. Primeiro, não é recomendável se preocupar com o desempenho nesse nível, a menos que as medições tenham demonstrado que a alocação em questão é um fator real para os problemas de desempenho (Item 67). Segundo, é possível retornar coleções e arrays vazios sem alocá-los. Aqui está o código típico para devolver uma coleção possivelmente vazia. Normalmente, isso é tudo que você precisa:

```
A maneira correta de retornar uma possivelmente vazia
collectionpublic<Cheese> List getCheeses() {
    return new ArrayList<>(cheesesInStock);}
```

No improvável caso de você ter evidências sugerindo que alocar colções vazias está prejudicando o desempenho, você pode evitar as alocações retornando o

a mesma coleção imutável vazia repetidamente, assim como objetos imutáveis podem ser compartilhados livremente (Item 17). Aqui está o código para fazer isso, usando o método `Collections.emptyList`. Se você estivesse devolvendo um conjunto, usaria `Collections.emptySet`; se você estivesse devolvendo um mapa, usaria `Collections.emptyMap`. Mas lembre-se, isso é uma otimização, e raramente é necessária. Se você acha que precisa, meça o desempenho antes e depois para garantir que está realmente ajudando:

```
Otimização - evita alocar coleções públicas vazias<Cheese>
List getCheeses() {
    devolver queijosInStock.isEmpty() ? Collections.emptyList()
        : nova ArrayList<>(queijosEmEstoque);
}
```

A situação para arrays é idêntica à das coleções. Nunca retorne null em vez de um array de comprimento zero. Normalmente, você deve simplesmente retornar um array com o comprimento correto, que pode ser zero. Note que estamos passando um array de comprimento zero no método `toArray` para indicar o tipo de retorno desejado, que é `Cheese[]`:

```
A maneira correta de devolver um array público
possivelmente vazio Cheese[] getCheeses() {
    return cheesesInStock.toArray(new Cheese[0]);}
```

Se você acredita que alocar arrays de comprimento zero prejudica o desempenho, pode retornar repetidamente o mesmo array de comprimento zero porque todos os arrays de comprimento zero são imutáveis:

```
Otimização - evita alocar arrays vazios, estático privado final
Cheese[] EMPTY_CHEESE_ARRAY = novo Cheese[0];

Queijo público[] getCheeses() {
    return cheesesInStock.toArray(EMPTY_CHEESE_ARRAY);}
```

Na versão otimizada, passamos o mesmo array vazio para cada chamada `toArray`, e esse array será retornado do `getCheeses` sempre que `cheesesInStock` estiver vazio. Não pré-aloque o array passado para o `toArray` na esperança de melhorar o desempenho. Estudos mostraram que é contraproducente [Shipilëv16]:

```
Não faça isso - pré-realocar o array prejudica o desempenho!return
cheesesInStock.toArray(new Cheese[cheesesInStock.size()]);
```

Em resumo, nunca retorne null no lugar de um array ou coleção vazia. Isso torna sua API mais difícil de usar e mais propensa a erros, e não tem vantagens de desempenho.

Item 55: Devolver opcionais com critério

Antes do Java 8, havia duas abordagens ao escrever um método que não conseguia devolver um valor em certas circunstâncias. Ou você poderia lançar uma exceção, ou poderia retornar null (assumindo que o tipo de retorno fosse um tipo de referência de objeto). Nenhuma dessas abordagens é perfeita. Exceções devem ser reservadas para condições excepcionais (Item 69), e lançar uma exceção é caro porque toda a trilha da pilha é capturada quando uma exceção é criada. Devolver o null não tem essas limitações, mas tem as suas próprias. Se um método retorna nulo, os clientes devem conter código de casos especiais para lidar com a possibilidade de retorno nulo, a menos que o programador possa provar que um retorno nulo é impossível. Se um cliente não verificar um retorno nulo e armazenar um valor de retorno nulo em alguma estrutura de dados, uma exceção NullPointerException pode resultar em algum momento arbitrário — no futuro, em algum ponto do código que não tem relação com o problema.

No Java 8, existe uma terceira abordagem para escrever métodos que podem não ser capazes de devolver um valor. A <T> classe Opcional representa um contêiner imutável que pode conter uma única referência T não nula ou nada. Um opcional que não contém nada é dito vazio. Diz-se que um valor está presente em um opcional que não está vazio. Um opcional é essencialmente uma coleção imutável que pode conter no máximo um elemento. Opcional<T> não implementa a Coleta<T>, mas pode ser em princípio.

Um método que conceitualmente retorna um T, mas pode não conseguir fazê-lo em certas circunstâncias, pode, em vez disso, ser declarado como retornando um Opcional<T>. Isso permite que o método retorne um resultado vazio para indicar que não poderia retornar um resultado válido. Um método que retorna opcionalmente é mais flexível e fácil de usar do que um que lança uma exceção, e é menos propenso a erros do que um que retorna nulo.

No Item 30, mostramos esse método para calcular o valor máximo em uma coleta, de acordo com a ordem natural dos elementos.

```
Retorna o valor máximo na coleção - lança exceção se empty
public static <E estende Comparable<E>> E max(Collection<E> c) {
    if (c.isEmpty())
        lançar um novo IllegalArgumentException("Coleção vazia");

    E resultado = nulo;
    para (E e : c)
        if (resultado == null || e.compareTo(resultado) > 0)
            resultado = Objects.requireNonNull(e);

    retorne resultado;}
```

Esse método gera uma `UnlawfulArgumentException` se a coleção dada for vazia. Mencionamos no item 30 que uma alternativa melhor seria `returnOptional<E>`. Veja como o método fica quando é modificado para isso:

```
Retorna valor máximo na coleção como um<E>Opcional público
estático <E estende Comparável<E>>
    <E> Opcional max(Collection<E> c) {if
        (c.isEmpty())
            return Optional.vty();

        E resultado = nulo;
        para (E e : c)
            if (resultado == null || e.compareTo(resultado) > 0)
                resultado = Objects.requireNonNull(e);

        return Optional.of(result);}
```

Como você pode ver, é fácil devolver um opcional. Tudo o que você precisa fazer é criar o opcional com a fábrica estática apropriada. Neste programa, usamos dois: `Optional.empty()` retorna um opcional vazio, e `Optional.of(valor)` retorna um opcional contendo o valor não nulo dado. É um erro de programação para passar o null para o `Optional.of(valor)`. Se você fizer isso, o método responde lançando uma `NullPointerException`. O método `Optional.ofNullable(value)` aceita um valor possivelmente nulo e retorna um opcional vazio se o valor nulo for passado. Nunca retorne um valor nulo de um método com retorno opcional: isso anula todo o propósito da instalação.

Muitas operações de terminal em fluxos retornam opcionais. Se reescrevermos o `maxmethod` para usar um fluxo, a operação `max` do `Stream` faz o trabalho de gerar um opcional para nós (embora tenhamos que passar um comparador explícito):

```
Retorna o valor máximo na coleção como Opcional<E> - usa estática
do streampublic <E estende o Comparável<E>>
Opcional<E> max(Collection<E> c) {return
    c.stream().max(Comparator.naturalOrder());}
```

Então, como você escolhe devolver um opcional em vez de devolver um null ou lançar uma exceção? Opcionais são semelhantes em espírito às exceções verificadas (Item 71), pois forçam o usuário de uma API a enfrentar o fato de que talvez não haja valor retornado. Lançar uma exceção desverificada ou retornar um null permite que o usuário ignore essa eventualidade, com consequências potencialmente graves. No entanto, lançar uma exceção verificada requer código padrão adicional no cliente.

Se um método retorna um opcional, o cliente pode escolher qual ação tomar caso o método não possa devolver um valor. Você pode especificar um valor padrão:

```
Usando um opcional para fornecer um valor padrão escolhidoString
lastWordInLexicon = max(words).orElse("Sem palavras...");
```

Ou você pode lançar qualquer exceção que for apropriada. Note que passamos por uma fábrica de exceções em vez de uma exceção real. Isso evita o custo de criar a exceção, a menos que ela realmente seja descartada:

```
Usando um opcional para jogar uma exceção escolhida Toy myToy =
max(toys).orElseThrow(TemperTantrumException::new);
```

Se você conseguir provar que um opcional é não vazio, pode obter o valor de `theOptional` sem especificar uma ação a ser tomada se o opcional estiver vazio, mas se você estiver errado, seu código gerará uma `NoSuchElementException`:

```
Usando opcional quando você sabe que há um valor de
retornoElement lastNobleGas = max(Elements.NOBLE_GASES).get();
```

Ocasionalmente, você pode se deparar com uma situação em que é caro obter o valor padrão, e você quer evitar esse custo, a menos que seja necessário. Para essas situações, o Opcional fornece um método que pega um `<T>` Fornecedor e o invoca apenas quando necessário. Esse método é chamado de `orElseGet`, mas talvez devesse ter sido chamado de `orElseCompute` porque está intimamente relacionado aos três métodos `Map` cujos nomes começam com `compute`. Existem vários métodos Opcionais para lidar com casos de uso mais especializados: `filter`, `map`, `flatMap` e `ifPresent`. In Java 9, mais dois desses métodos foram adicionados: `ou` e `ifPresentOrElse`. Se os métodos básicos descritos acima não forem adequados para o seu caso, veja a documentação desses métodos mais avançados e veja se eles funcionam.

Caso nenhum desses métodos atenda às suas necessidades, o `Optional` oferece o método `theisPresent()`, que pode ser visto como uma válvula de segurança. Ele retorna `true` se o opcional contiver um valor, `false` se estiver vazio. Você pode usar esse método para realizar qualquer processamento que quiser em um resultado opcional, mas certifique-se de usá-lo de forma inteligente. Muitos usos do `isPresent` podem ser substituídos de forma lucrativa por um dos métodos mencionados acima. O código resultante normalmente será mais curto, claro e idiomático.

Por exemplo, considere este trecho de código, que imprime o ID do processo do pai de um processo, ou N/A se o processo não tiver pai. O snippet usa a classe `ProcessHandle`, introduzida no Java 9:

```
<ProcessHandle> Opcional paiProcesso = ph.parent();
System.out.println("Parent PID: " + (parentProcess.isPresent() ?
    String.valueOf(parentProcess.get().pid()) : "N/A"));
```

O trecho de código acima pode ser substituído por este, que usa a função `mapfunction` do `Optional`:

```
System.out.println ("PID pai: " +
    ph.parent().map(h -> String.valueOf(h.pid()).orElse("N/A"));
```

Ao programar com streams, não é incomum encontrar `aStream<Opcional<T>>` e exigir um `Stream<T>` contendo todos os elementos dos opcionais não vazios para prosseguir. Se você está usando Java 8, veja como preencher essa lacuna:

```
streamOfOptionals
    .filter(Opcional::isPresent).map
    .map(Opcional::get)
```

No Java 9, o `Optional` era equipado com um método `stream()`. Esse método é um adaptador que transforma um `Opcional` em um Fluxo contendo um elemento se um estiver presente no opcional, ou nenhum se ele estiver vazio. Em conjunto com o método `StreamsflatMap` (Item 45), este método fornece uma substituição concisa para o trecho do código acima:

```
streamOfOptionals.
    .flatMap(Opcional::stream)
```

Nem todos os tipos de devolução se beneficiam do tratamento opcional. Tipos de container, incluindo coleções, mapas, streams, arrays e opcionais, não devem estar envolvidos em opcionais. Em vez de devolver uma `Lista<Opcional<T>>` vazia, você deve simplesmente devolver uma `Lista<T>` vazia (Item 54). Devolver o contêiner vazio eliminará a necessidade de o código do cliente processar um opcional. A classe `ProcessHandle` possui o método `arguments`, que retorna `Optional<String[]>`, mas esse método deve ser considerado uma anomalia que não deve ser emulada.

Então, quando você deve declarar um método para devolver `Opcional<T>` em vez de `T`? Como regra, você deve declarar um método para devolver `Opcional<T>` se ele não conseguir devolver um resultado, e os clientes terão que realizar um processamento especial caso nenhum resultado seja devolvido. Dito isso, devolver um `Opcional<T>` não é sem custo.

Um Opcional é um objeto que precisa ser alocado e inicializado, e ler o valor do opcional requer uma indireção extra. Isso torna o opcional apropriado para uso em algumas situações críticas de desempenho. Se um método específico se enquadrar nessa categoria só pode ser determinado por medição cuidadosa (Item 67).

Devolver um opcional que contenha um tipo primitivo em caixa é proibitivamente caro comparado a devolver um tipo primitivo, porque o opcional tem dois níveis de boxe em vez de zero. Portanto, os projetistas da biblioteca consideraram adequado fornecer análogos de `Optional<T>` para os tipos primitivos `int`, `long` e `double`. Esses tipos opcionais são `OptionalInt`, `OptionalLong` e `OptionalDouble`. Eles contêm a maioria, mas não todos, os métodos do `Opcional<T>`. Portanto, você nunca deve devolver um opcional de um tipo primitivo em caixa, com possível exceção dos "tipos primitivos menores", `Booleano`, `Byte`, `Character`, `Short` e `Float`.

Até agora, discutimos a devolução dos opcionais e o processamento deles após o retorno. Não discutimos outros usos possíveis, e isso porque a maioria dos outros usos dos opcionais é duvidosa. Por exemplo, você nunca deve usar valores opcionais como mapa. Se você fizer isso, você tem duas formas de expressar a ausência lógica de uma chave no mapa: ou a chave pode estar ausente do mapa, ou pode estar presente e mapear para um opcional vazio. Isso representa complexidade desnecessária com grande potencial para confusão e erros. De forma mais geral, quase nunca é apropriado usar um opcional como chave, valor ou elemento em uma coleção ou array.

Isso deixa uma grande pergunta sem resposta. É apropriado armazenar um opcional em um campo de instância? Frequentemente é um "cheiro ruim": sugere que talvez você devesse ter uma subclasse contendo os campos opcionais. Mas às vezes pode ser justificado. Considere o caso da nossa aula `NutritionFacts` no item 2. A instância `ANutritionFacts` contém muitos campos que não são necessários. Você não pode ter uma subclasse para cada combinação possível desses campos. Além disso, os campos têm tipos primitivos, o que torna estranho expressar a ausência diretamente. A `bestAPI` para `NutritionFacts` retornaria um opcional do `getter` para cada campo opcional, então faz sentido simplesmente armazenar esses opcionais como campos no objeto.

Resumindo, se você se pegar escrevendo um método que nem sempre pode devolver um valor e acha importante que os usuários do método considerem essa possibilidade toda vez que o ligam, então provavelmente deveria devolver um opcional. No entanto, você deve estar ciente de que existem consequências reais de desempenho associadas ao devolver opcionais; para métodos críticos de desempenho, pode ser melhor retornar um `null` ou lançar uma exceção. Por fim, raramente você deve usar um opcional em outra função que não seja como valor de retorno.

Item 56: Escrever comentários de documentação para todos os elementos da API expostos

Se uma API for utilizável, ela deve ser documentada. Tradicionalmente, a documentação da API era gerada manualmente, e mantê-la sincronizada com o código era uma tarefa árdua. O ambiente de programação Java facilita essa tarefa com a ferramenta Javadoc. Javadoc gera documentação de API automaticamente a partir do código-fonte com comentários de documentação especialmente formatados, mais conhecidos como comentários doc.

Embora as convenções de comentários de documentos não façam parte oficial da linguagem, elas constituem uma API de fato que todo programador Java deveria conhecer. Essas convenções são descritas na página web de Como Escrever Comentários de Documentação [Javadoc-guide]. Embora esta página não tenha sido atualizada desde o lançamento do Java 4, ainda é um recurso inestimável. Uma tag doc importante foi adicionada no Java 9, `{@index}`; onein Java 8, `{@implSpec}`; e dois em Java 5, `{@literal}` e `{@code}`. Essas tags estão ausentes da página mencionada, mas são discutidas neste item.

Para documentar sua API corretamente, você deve preceder cada declaração de classe, interface, construtor, método e campo exportada com um comentário de documentação. Se a aclass for serializável, você também deve documentar sua forma serializada (Item 87). Na ausência de um comentário documental, o melhor que o Javadoc pode fazer é reproduzir a declaração como a única documentação para o elemento da API afetado. É frustrante e propenso a erros usar uma API com comentários de documentação faltando. Classes públicas não devem usar construtores padrão porque não há como fornecer com-ments de documentação para eles. Para escrever código manutentable, você também deve escrever comentários de doc para a maioria das classes, interfaces, construtores, métodos e campos não exportados, embora esses comentários não precisem ser tão completos quanto os dos elementos de API exportados.

O comentário do método para um método deve descrever de forma sucinta o contrato entre o método e seu cliente. Com exceção dos métodos em classes destinadas à herança (Item 19), o contrato deve dizer o que o método faz, e não como ele cumpre seu papel. O comentário do doc deve enumerar todas as pré-condições do themethod, que são as coisas que precisam ser verdadeiras para que um cliente possa invocá-las, e suas postconditions, que são as coisas que serão verdadeiras após a invocação ser concluída com sucesso. Tipicamente, pré-condições são descritas implicitamente pelas tags `@throws` para exceções não verificadas; cada exceção não verificada corresponde a uma violação de pré-condição. Além disso, pré-condições podem ser especificadas junto com os parâmetros afetados em suas tags `@param`.

Além das pré-condições e pós-condições, os métodos devem documentar quaisquer efeitos colaterais. Um efeito colateral é uma mudança observável no estado do sistema que não é obviamente necessária para alcançar a condição pós-condição. Por exemplo, se um método inicia uma thread em segundo plano, a documentação deve anotá-lo.

Para descrever completamente o contrato de um método, o comentário doc deve ter `an@param` tag para cada parâmetro, uma tag `@return` a menos que o método tenha um tipo `void` return, e uma tag `@throws` para cada exceção lançada pelo método, seja marcada ou não marcada (Item 74). Se o texto na tag `@return` for idêntico à descrição do método, pode ser permitido omitir essa informação, dependendo dos padrões de codificação que você está seguindo.

Por convenção, o texto após uma tag `@param` ou tag `@return` deve ser uma frase anúncia que descreve o valor representado pelo parâmetro ou valor de retorno. Raramente, expressões aritméticas são usadas no lugar de sintagmas nominais; veja `BigInteger` para exemplos. O texto após uma tag `@throws` deve consistir na palavra "if", seguida de uma cláusula descrevendo as condições sob as quais a exceção é rejeitada. Por convenção, a frase ou cláusula após uma etiqueta `@param`, `@return` or `@throws` não é terminada por um ponto. Todas essas convenções são ilustradas pelo seguinte comentário no documento:

```
/**
 * Retorna o elemento na posição especificada nesta lista.** <p>Esse
 método <i>não</i> garante que execute em tempo constante. Em algumas
 implementações, pode rodar em tempo proporcional* à posição do
 elemento.** @param index índice do elemento a retornar; deve ser* não-
 negativo e menor que o tamanho desta lista* @return o elemento na
 posição especificada desta lista* @throws IndexOutOfBoundsException
 se o índice estiver fora do intervalo*({@code índice < 0 || índice >=
 this.size()})*/E get(int index);
```

Note o uso de tags HTML neste comentário do documento (`<p>` `<i>` `</i>` `<e>`). O `Javadoc` traduz comentários doc para HTML, e elementos HTML arbitrários em doccomments acabam no documento HTML resultante. Ocasionalmente, programadores chegam ao ponto de incorporar tabelas HTML em seus comentários de documentação, embora isso seja raro.

Também note o uso da tag `Javadoc` `{@code}` ao redor do fragmento de código em `the@throws` cláusula. Essa tag serve a dois propósitos: ela faz com que o fragmento de código seja renderizado em fonte de código e suprime o processamento de marcação HTML e tags `Javadoc` aninhadas no fragmento de código. Essa última propriedade é o que nos permite usar o sinal (`<`) no fragmento de código, mesmo sendo um HTML metacharacter. To incluir um exemplo de código multilinha em um comentário de documento, usar uma tag `Javadoc` `{@code}` encapsulada dentro de uma `<pre>` tag HTML. Em outras palavras, preceda o exemplo de código com os caracteres `<pre>{@code}` e siga com `</pre>`. Isso preserva a linha

quebra o código e elimina a necessidade de escapar dos metacaracteres HTML, mas não do sinal at (@), que deve ser escapado se o exemplo de código usar anotações.

Por fim, repare no uso das palavras "esta lista" no comentário do documento. Por convenção, a palavra "isto" refere-se ao objeto sobre o qual um método é invocado quando é usado no comentário doc para um método de instância.

Como mencionado no Item 15, ao projetar uma classe para herança, você deve documentar seus padrões de autouso, para que os programadores conheçam a semântica dos métodos overriding. Esses padrões de autouso devem ser documentados usando o `@implSpec` tag, adicionado no Java 8. Lembre-se de que comentários comuns de documentos descrevem o contrato entre um método e seu cliente; `@implSpec` comentários, por outro lado, descrevem o contrato entre um método e sua subclasse, permitindo que subclasses dependam do comportamento de implementação se herdarem o método ou o chamarem via super. Veja como fica na prática:

```
/**
 * Retorna verdadeiro se esta coleção estiver vazia.**
 * @implSpec Esta implementação retorna {@code this.size() ==
 * 0}.** @return verdadeiro se esta coleção estiver
 * vazia*/public boolean isEmpty() { ... }
```

A partir do Java 9, a utilidade Javadoc ainda ignora a tag `@implSpec`, a menos que você passe a tag switch de linha de comando "implSpec:a:Implementation Requirements:". Espero que isso seja corrigido em um lançamento futuro.

Não se esqueça de que você deve tomar uma ação especial para gerar documentação que contenha metacaracteres HTML, como o signo menor que (<), o signo maior que (>) e o e e A melhor maneira de colocar esses caracteres na documentação é cercá-los com a tag `{@literal}`, que suprime o processamento de marcação HTML e tags Javadoc aninhadas. É como a tag `{@code}`, exceto que não renderiza o texto em fonte de código. Por exemplo, este fragmento do Javadoc:

```
* Uma série geométrica converge se {@literal |r| < 1}.
```

gera a documentação: "Uma série geométrica converge se $|r| < 1$." A tag `{@literal}` poderia ter sido colocada apenas em torno do sinal de menos que em vez de toda a desigualdade com a mesma documentação resultante, mas o comentário do documento teria sido menos legível no código-fonte. Isso ilustra o princípio geral de que os comentários do doc devem ser legíveis tanto no código-fonte quanto na documentação gerada. Se você não conseguir fazer ambos, a legibilidade da documentação gerada supera a do código-fonte.

A primeira "frase" de cada comentário documental (conforme definido abaixo) torna-se a descrição resumida do elemento ao qual o comentário se refere. Por exemplo, a descrição resumida no comentário do documento na página 255 é "Retorna o elementat na posição especificada nesta lista." A descrição resumida deve se sustentar sozinha para descrever a funcionalidade do elemento que ela resume. Para evitar confusão, nenhum membro ou construtor em uma classe ou interface deve ter a mesma descrição resumida. Preste atenção especial às sobrecargas, para as quais muitas vezes é natural usar a mesma primeira frase (mas inaceitável em comentários de documentos).

Tenha cuidado se a descrição resumida pretendida contiver um ponto, pois o período pode terminar prematuramente a descrição. Por exemplo, um comentário de um documento que começa com a frase "Um diploma universitário, como B.S., M.S. ou Ph.D." resultará na descrição resumida "Um diploma universitário, como B.S., M.S." O problema é que a descrição resumida termina no primeiro período seguido por um terminador de espaço, tabulação ou linha (ou na primeira tag de bloco) [Javadoc-ref]. Aqui, o segundo ponto na abreviação "M.S." é seguido por um espaço. A melhor solução é cercar o ponto ofensivo e qualquer texto associado com uma tag `{@literal}`, para que o ponto não seja mais seguido por um espaço no código-fonte:

```
/**  
 * Um diploma universitário, como B.S., {@literal M.S.} ou  
 Ph.D.*/diploma público { ... }
```

É um pouco enganoso dizer que a descrição resumida é a primeira frase de um comentário documental. A convenção dita que raramente deve ser uma frase completa. Para métodos e construtores, a descrição resumida deve ser uma frase verbal (incluindo qualquer objeto) que descreve a ação realizada pelo método. Por exemplo:

- `ArrayList(int initialCapacity)`—Constrói uma lista vazia com a capacidade inicial especificada.
- `Collection.size()`—Retorna o número de elementos nesta coleção.

Como mostrado nestes exemplos, use o tempo declarativo da terceira pessoa ("retorna o número") em vez do imperativo de segunda pessoa ("retorne o número").

Para classes, interfaces e campos, a descrição resumida deve ser uma frase nominal que descreve o objeto representado por uma instância da classe ou interface ou pelo próprio campo. Por exemplo:

- `Instantâneo` — Um ponto instantâneo na linha do tempo.
- `Math.PI` — O valor duplo que é mais próximo do que qualquer outro de pi, a razão entre a circunferência de um círculo e seu diâmetro.

No Java 9, um índice do lado do cliente era adicionado ao HTML gerado pelo Javadoc. Esse índice, que facilita a navegação por grandes conjuntos de documentação da API, assume a forma de uma caixa de busca no canto superior direito da página. Quando você digita na caixa, aparece um menu suspenso com páginas correspondentes. Elementos da API, como classes, métodos e campos, são indexados automaticamente. Ocasionalmente, você pode querer indexar termos adicionais que sejam importantes para sua API. A tag `{@index}` foi adicionada para esse propósito. Indexar um termo que aparece em um comentário de documento é tão simples quanto envolvê-lo nesta tag, como mostrado neste fragmento:

```
* Este método está em conformidade com o padrão {@index IEEE 754}.
```

Genéricos, enums e anotações exigem cuidado especial nos comentários dos documentos. Ao documentar um tipo ou método genérico, certifique-se de documentar todos os parâmetros do tipo:

```
/**
 * Um objeto que mapeia chaves para valores. Um mapa não pode conter
 * chaves duplicadas; cada chave pode mapear no máximo um
 * valor.** (Restante omitido)** @param <K> o tipo de chaves
 mantidas por este mapa* @param <V> o tipo de valores
 mapeados*/interface pública Mapa<K, V> { ... }
```

Ao documentar um tipo de enum, certifique-se de documentar as constantes, assim como o tipo e quaisquer métodos públicos. Note que você pode colocar um documentário inteiro em uma linha se for curto:

```
/**
 * Uma seção de instrumentos de uma orquestra
 sinfônica.*/public enum OrchestraSection {

    /** Madeiras, como flauta, clarinete e oboé. */MADEIRA,

    /** Instrumentos de metal, como trompa francesa e trompete.
     */BRASS,

    /** Instrumentos de percussão, como tímpanos e pratos.
     */PERCUSSÃO,

    /** Instrumentos de corda, como violino e violoncelo.
     */STRING;}
```

Ao documentar um tipo de anotação, certifique-se de documentar quaisquer memorizações, assim como o próprio tipo. Documente os membros com frases nominais, como se fossem corpos. Para a descrição resumida do tipo, use uma frase verbal que explique o que significa quando um elemento do programa possui uma anotação desse tipo:

```
/**
 * Indica que o método anotado é um método de teste que* deve
 * lançar a exceção designada para
 * passar.*/@Retention(RetentionPolicy.RUNTIME)@Target(ElementType.M
 * ETHOD)public @interface ExceptionTest {

    /**
     * A exceção que o método de teste anotado deve lançar* para passar.
     * (O teste pode lançar qualquer* subtipo do tipo descrito por este
     * objeto de classe.)*@Class<? estende o valor Throwable> ();}
```

Comentários em nível de pacote devem ser colocados em um arquivo chamado `package-info.java`. Além desses comentários, `package-info.java` deve conter uma declaração de pacote e pode conter anotações sobre essa declaração. Da mesma forma, se você optar por usar o sistema de módulos (Item 15), comentários em nível de módulo devem ser colocados no arquivo `module-info.java`.

Dois aspectos das APIs que muitas vezes são negligenciados na documentação são a segurança de threads e a serializabilidade. Se uma classe ou método estático é seguro para threads ou não, você deve documentar seu nível de segurança de thread, conforme descrito no Item 82. Se uma classe for serializável, você deve documentar sua forma serializada, conforme descrito no Item 87.

Javadoc tem a capacidade de "herdar" comentários de métodos. Se um elemento API não possui um comentário doc, o Javadoc busca o doccomment, mais específico aplicável, dando preferência às interfaces em vez das superclasses. Os detalhes do algoritmo de busca podem ser encontrados no Guia de Referência do Javadoc [Javadoc-ref]. Você também pode herdar partes dos comentários do documento de supertipos usando a tag `{@inheritDoc}`. Isso significa, entre outras coisas, que as classes podem reutilizar comentários do doc a partir das interfaces que implementam, em vez de copiar esses comentários. Essa funcionalidade tem potencial para reduzir o peso de manter múltiplos conjuntos de comentários quase idênticos no documento, mas é difícil de usar e tem algumas limitações. Os detalhes estão além do escopo deste livro.

Uma ressalva deve ser acrescentada sobre comentários sobre documentação. Embora seja necessário fornecer comentários documentais para todos os elementos da API exportados, nem sempre é suficiente. Para APIs complexas compostas por múltiplas classes inter-relacionadas, muitas vezes é necessário complementar os comentários da documentação com um documento externo descrevendo a arquitetura geral da API. Se tal documento existir, os comentários relevantes sobre a documentação da classe ou pacote devem incluir um link para ele.

O Javadoc verifica automaticamente a adesão a muitas das recomendações deste produto. No Java 7, o interruptor de linha de comando `-Xdoclint` era necessário para obter este comportamento. No Java 8 e 9, a verificação está ativada por padrão. Plugins IDE como o checkstyle vão além na verificação da adesão a essas recomendações[Burn01]. Você também pode reduzir a probabilidade de erros nos comentários do documento ao passar os arquivos HTML gerados pelo Javadoc por um verificador de validade HTML. Isso detectará muitos usos incorretos das tags HTML. Vários desses verificadores estão disponíveis para download, e você pode validar HTML na web usando o serviço de validação W3Cmarkup [W3C-validator]. Ao validar HTML gerado, lembre-se de que, a partir do Java 9, o Javadoc é capaz de gerar HTML5 assim como HTML 4.01, embora ainda gere o HTML 4.01 por padrão. Use o switch de linha de comando `-html5` se quiser que o Javadoc gere HTML5.

As convenções descritas neste item cobrem o básico. Embora tenha quinze anos na época em que escrevo isto, o guia definitivo para escrever comentários documentais ainda é Como Escrever Comentários de Documentação [guia Javadoc].

Se você seguir as diretrizes deste item, a documentação gerada deve fornecer uma descrição clara da sua API. A única forma de ter certeza, no entanto, é ler as páginas da web geradas pelo utilitário Javadoc. Vale a pena fazer isso para cada API que será usada por outros. Assim como testar um programa quase inevitavelmente resulta em algumas alterações no código, ler a documentação geralmente resulta em pelo menos algumas pequenas alterações nos comentários do documento.

Para resumir, comentários de documentação são a melhor e mais eficaz forma de documentar sua API. Seu uso deve ser considerado obrigatório para todos os elementos exportados da API. Adote um estilo consistente que siga as convenções padrão. Lembre-se de que HTML arbitrário é permitido em comentários de documentação e que metacaracteres HTML devem ser evitados.

CHAPTER 9

Programação Geral

ESTE capítulo é dedicado aos detalhes práticos da língua. Ele discute o local variáveis, estruturas de controle, bibliotecas, tipos de dados e duas facilidades extralinguísticas: métodos de reflexão e métodos nativos. Por fim, discute otimização e convenções de nomes.

Item 57: Minimizar o escopo das variáveis locais

Este item é semelhante ao Item 15, "Minimizar a acessibilidade de aulas e membros." Ao minimizar o escopo das variáveis locais, você aumenta a capacidade de leitura e manutenção do seu código e reduz a probabilidade de erro.

Linguagens de programação mais antigas, como C, exigiam que variáveis locais fossem declaradas no início de um bloco, e alguns programadores continuam fazendo isso por hábito. É um hábito que vale a pena quebrar. Como lembrete gentil, o Java permite declarar variáveis em qualquer lugar onde uma instrução seja legal (assim como o C, desde o C99).

A técnica mais poderosa para minimizar o escopo de uma variável local é declará-la onde ela foi usada pela primeira vez. Se uma variável é declarada antes de ser usada, é apenas bagunça — mais uma coisa para distrair o leitor que está tentando entender o que o programa faz. Quando a variável é usada, o leitor pode não lembrar o tipo ou o valor inicial da variável.

Declarar uma variável local prematuramente pode fazer com que seu escopo não apenas comece cedo demais, mas também termine tarde demais. O escopo de uma variável local se estende do ponto onde ela é declarada até o final do bloco que encerra. Se uma variável for declarada fora do bloco em que é usada, ela permanece visível após o programa sair desse bloco. Se uma variável for usada acidentalmente antes ou depois de sua região de uso pretendida, as consequências podem ser desastrosas.

Quase toda declaração de variável local deve conter um inicializador. Se você ainda não tem informações suficientes para inicializar uma variável de forma sensata, deveria

Adie a declaração até que você faça isso. Uma exceção a essa regra diz respeito às declarações try-catch. Se uma variável for inicializada em uma expressão cuja avaliação pode lançar uma exceção verificada, a variável deve ser inicializada dentro de um bloco try (a menos que o método de anexo possa propagar a exceção). Se o valor deve ser usado fora do bloco try, então ele deve ser declarado antes do bloco try, onde ainda não pode ser "inicializado sensatamente". Para um exemplo, veja a página 283.

Os ciclos apresentam uma oportunidade especial para minimizar o escopo das variáveis. O loop thefor, tanto em sua forma tradicional quanto for-eada, permite que você declare variáveis de loop, limitando seu escopo à região exata onde são necessárias. (Esta região consiste no corpo do loop e no código entre parênteses entre a palavra-chave thefor e o corpo.) Portanto, prefira para loops para loops while, assumindo que o conteúdo da variável loop não seja necessário após o término do loop.

Por exemplo, aqui está o idioma preferido para iterar sobre uma coleção (Item 58):

```
Idioma preferido para iterar sobre uma coleção ou arrayfor
(Elemento e : c) {
... // Faça Algo com e}
```

Se você precisar de acesso ao iterador, talvez para chamar seu método remove, o preferidídmio usa um loop tradicional for no lugar do loop for-each:

```
Idioma para iterar quando você precisa do iteratorfor
(<Element>Iterator i = c.iterator(); i.hasNext(); ) {
Elemento e = i.next();... // Faça algo
com e e i}
```

Para entender por que esses loops for são preferíveis a um loop while, considere o seguinte fragmento de código, que contém dois loops while e um bug:

```
<Element> Iterador i = c.iterator();
while (i.hasNext()) {
doAlgo(i.next());}...
```

```
Iterador<Element> i2 = c2.iterator(); while
(i.hasNext()) {// BUG!
doAlgoMais (i2.next());}
```

O segundo loop contém um erro de copiar e colar: ele inicializa uma nova variável de loop, i2, mas usa a antiga, i, que, infelizmente, ainda está no escopo. O resultado

O código compila sem erro e roda sem lançar uma exceção, mas faz a coisa errada. Em vez de iterar sobre `c2`, o segundo ciclo termina imediatamente, dando a falsa impressão de que `c2` está vazio. Como o programa é errado, o erro pode permanecer indetectado por muito tempo.

Se um erro semelhante de copiar e colar fosse cometido em conjunto com qualquer um dos loops `thefor` (`for-each` ou tradicional), o código resultante nem sequer compilaria. A variável `elemento` (ou iterador) do primeiro ciclo não estaria no escopo do segundo ciclo. Veja como fica com o tradicional `loop for`:

```
para (<Element>Iterador i = c.iterador(); i.hasNext(); ) {
    Elemento e = i.next();... // Faça algo
    com e e i}...
```

```
Erro em tempo de compilação - não é possível encontrar o símbolo
ifor (Iterador<Element> i2 = c2.iterador(); i.hasNext(); ) {
    Elemento e2 = i2.next();... // Faça algo
    com e2 e i2}
```

Além disso, se você usar um `loop for`, é muito menos provável que cometa o erro de copiar e colar, porque não há incentivo para usar nomes diferentes de variáveis nos dois loops. Os loops são completamente independentes, então não há problema em reutilizar o nome da variável `elemento` (ou iterador). Na verdade, isso costuma ser estiloso.

O `loop for` tem mais uma vantagem sobre o `loop while`: é mais curto, o que melhora a legibilidade.

Aqui está outro idioma de `loop` que minimiza o escopo das variáveis locais:

```
para (int i = 0, n = caraComputação(); i < n; i++) {
    ... // Faça algo com i;}
```

O importante a notar sobre esse idioma é que ele possui duas variáveis de `loop`, `i`, e `n`, ambas com exatamente o escopo certo. A segunda variável, `n`, é usada para armazenar o limite da primeira, evitando assim o custo de uma computação redundante em cada iteração. Como regra, você deve usar esse idioma se o teste de `loop` envolver uma invocação de método que garanta o mesmo resultado em cada iteração.

Uma técnica final para minimizar o escopo das variáveis locais é manter os métodos pequenos e focados. Se você combinar duas atividades no mesmo método, as variáveis locais relevantes para uma atividade podem estar no escopo do código que realiza a outra. Para evitar que isso aconteça, basta separar o método em dois: um para cada atividade.

Item 58: Preferência de loops for-each aos tradicionais loops for

Como discutido no Item 45, algumas tarefas são melhor realizadas com fluxos, outras com iteração. Aqui está um loop tradicional for para iterar sobre uma coleção:

```
Não é a melhor forma de iterar sobre uma coleção!for
(<Element>Iterador i = c.iterator(); i.hasNext(); ) {
    Elemento e = i.next();... //
    Faça algo com e}
```

E aqui está um loop tradicional FOR para iterar sobre um array:

```
Não é a melhor forma de iterar sobre um array!for
(int i = 0; i < a.length; i++) {
    ... // Faça algo com a[i]}
```

Esses idiomas são melhores que loops while (Item 57), mas não são perfeitos. Theiterator e as variáveis de índice são apenas clutter—tudo que você precisa são os elementos. Além disso, representam oportunidades de erro. O iterador ocorre três vezes em cada loop e a variável índice quatro, o que te dá muitas chances de usar a variável errada. Se você fizer isso, não há garantia de que o compilador vai detectar o problema. Por fim, os dois loops são bastante diferentes, chamando atenção desnecessária para o tipo do recipiente e adicionando um (pequeno) transtorno para mudar esse tipo.

O loop for-each (oficialmente conhecido como "instrução for aprimorada") resolve todos esses problemas. Isso elimina a bagunça e a possibilidade de erro ao esconder o iterador ou a variável de índice. O idioma resultante se aplica igualmente a coletores e arrays, facilitando o processo de trocar o tipo de implementação de um para o outro:

```
O idioma preferido para iterar sobre coleções e arrays (Elemento e :
elements) {
    ... // Faça algo com e}
```

Quando você vir o colôn (:), leia como "em". Assim, o laço acima é "para cada elemento e em elementos." Não há penalidade de desempenho ao usar for-each loops, mesmo para arrays: o código que eles geram é essencialmente idêntico ao código que você escreveria manualmente.

As vantagens do loop for-each em relação ao loop for tradicional são ainda maiores quando se trata de iterações aninhadas. Aqui está um erro comum que as pessoas cometem ao fazer iterações aninhadas:

```
Você consegue identificar o naipe do inseto { PAUS, OUROS, COPAS,
ESPADAS } rank { ÁS, DOIS, TRÊS, QUATRO, CINCO, SEIS, SETE, OITO,
```

```
NOVE, DEZ, VALETE, DAMA, REI }
... Coleção estática<Suit> = Arrays.asList(Suit.values()); Ranks
de coleção<Rank> estática = Arrays.asList(Rank.values());
```

```
Baralho de lista<Card> = novo ArrayList<>(); para (<Suit>Iterador
i = suits.iterator(); i.hasNext(); )para (Iterador<Rank> j =
ranks.iterator(); j.hasNext(); )
    deck.add(nova Carta(i.next(), j.next()));
```

Não se sinta mal se você não percebeu o bug. Muitos programadores especialistas cometeram esse erro em algum momento. O problema é que o próximo método é chamado muitas vezes no iterador para a coleção externa (suits). Deve ser chamado do laço externo para ser chamado uma vez por naipe, mas em vez disso é chamado do laço interno, então é chamado uma vez por carta. Depois que você acaba os naipes, o theloop lança uma `NãoNãoNãoNãoNãoElementoExceção`.

Se você tiver muita azar e o tamanho da coleção externa for múltiplo do tamanho da coleção interna — talvez porque são a mesma coleção — o loop terminará normalmente, mas não fará o que você quer. Por exemplo, considere esta tentativa mal concebida de imprimir todos os possíveis lançamentos de um par de dados:

```
Mesmo bug, sintoma diferente!enum Face { UM, DOIS,
TRÊS, QUATRO, CINCO, SEIS }... Fontes de coleção<Face>
= EnumSet.allOf(Face.class);
```

```
para (<Face>Iterador i = faces.iterator(); i.hasNext(); )para
(Iterador<Face> j = faces.iterator(); j.hasNext(); )
    System.out.println(i.next() + " " + j.next());
```

O programa não faz exceção, mas imprime apenas os seis "dúpos" (de "ONE ONE" a "SIX SIX"), em vez das trinta e seis combinações esperadas.

Para corrigir os bugs nesses exemplos, você deve adicionar uma variável no escopo do laço externo para conter o elemento externo:

```
Fixo, mas feio - você pode fazer melhor!for (<Suit>Iterador i =
suits.iterator(); i.hasNext(); ) {
    Naipe = i.next(); para (Iterador<Rank> j = ranks.iterator();
j.hasNext(); )
        deck.add(nova carta (naipe, j.next()));
}
```

Se, em vez disso, você usar um loop aninhado para cada um, o problema simplesmente desaparece. O código resultante é tão sucinto quanto você poderia desejar:

```
Idioma preferido para iteração aninhada em coleções e arrays (Naipes:
naipes)
    para (Patente de patente: patentes)
        deck.add (nova carta (naipes, valor));
```

Infelizmente, existem três situações comuns em que você não pode usar para cada uma:

- **Filtragem destrutiva** — Se você precisa atravessar uma coleção removendo elementos selecionados, então precisa usar um iterador explícito para que possa chamar o método `itsremove`. Você frequentemente pode evitar a travessia explícita usando o método `CollectionsremoveIf`, adicionado no Java 8.
- **Transformação** — Se você precisa percorrer uma lista ou array e substituir alguns ou todos os valores de seus elementos, então é necessário o iterador de listas ou índice de array para substituir o valor de um elemento.
- **Iteração paralela** — Se você precisar atravessar múltiplas coleções em paralelo, então precisa de controle explícito sobre o iterador ou variável índice para que todos os iterators ou variáveis de índice possam ser avançados em passo conjunto (como demonstrado de forma não intencional nos exemplos de cartas e dados bugados acima).

Se você se encontrar em alguma dessas situações, use um loop `for` comum e fique atento às armadilhas mencionadas neste item.

O loop `for-each` não só permite iterar sobre coleções e arrays, como também permite iterar sobre qualquer objeto que implemente a interface `Iterable`, que consiste em um único método. Veja como a interface está:

```
interface pública Iterable<E> {
    Retorna um iterador sobre os elementos deste<E>
    iteradorIterador();}
```

É um pouco complicado implementar o `Iterable` se você precisa escrever sua própria implementação do zero, mas se você está escrevendo um tipo que representa um grupo de elementos, deve considerar fortemente que ele implemente o `Iterable`, mesmo que você escolha não implementar o `Collection`. Isso permitirá que seus usuários compartilhem seu tipo usando o loop `for-each`, e eles ficarão eternamente gratos.

Em resumo, o loop `for-each` oferece vantagens convincentes sobre o loop `for` tradicional em clareza, flexibilidade e prevenção de bugs, sem penalidade de desempenho. Use loops `for-each` em vez de loops `for` sempre que possível.

Item 59: Conhecer e usar as bibliotecas

Suponha que você queira gerar inteiros aleatórios entre zero e algum limite superior. Diante dessa tarefa comum, muitos programadores escreveriam um `littlemethod` que se parecesse com isto:

```
Comum, mas profundamente falho!static
Random rnd = novo Random();

static int random(int n) {
    return Math.abs(rnd.nextInt()) % n;}
```

Esse método pode parecer bom, mas tem três falhas. A primeira é que, se `n` for uma pequena potência de dois, a sequência de números aleatórios se repetirá após um período relativamente curto. A segunda falha é que, se `n` não for uma potência de dois, alguns números serão, em média, retornados com mais frequência do que outros. Se `n` for grande, esse efeito pode ser bastante pronunciado. Isso é demonstrado de forma contundente pelo programa a seguir, que gera um milhão de números aleatórios em um intervalo cuidadosamente escolhido e depois imprime quantos desses números caíram na metade inferior do intervalo:

```
public static void main(String[] args) {
    int n = 2 * (Integer.MAX_VALUE / 3);
    int low = 0; para (int i = 0; i <
        1000000; i++)
        se (random(n) < n/2)
            low++;
    System.out.println(low);}
```

Se o método aleatório funcionasse corretamente, o programa imprimiria um número próximo de meio milhão, mas se você executar, verá que ele imprime um número próximo de 666.666. Dois terços dos números gerados pelo método aleatório estão na metade inferior do seu alcance!

A terceira falha do método aleatório é que ele pode, em raras ocasiões, falhar de forma catastrófica, retornando um número fora do intervalo especificado. Isso ocorre porque o método tenta mapear o valor retornado por `rnd.nextInt()` para um não-negativo int chamando `Math.abs`. Se `nextInt()` devolver `Integer.MIN_VALUE`, `Math.abs` também devolverá `Integer.MIN_VALUE`, e o operador restante (%) devolverá um número negativo, assumindo que `n` não seja uma potência de dois. Isso quase certamente fará com que seu programa fracasse, e a falha pode ser difícil de reproduzir.

Para escrever uma versão do método aleatório que corrija essas falhas, você teria que saber bastante sobre geradores de números pseudoaleatórios, teoria dos números,

e a aritmética do complemento dois. Felizmente, você não precisa fazer isso — já foi feito para você. Chama-se `Random.nextInt(int)`. Você não precisa se preocupar com os detalhes de como ele faz seu trabalho (embora possa estudar a documentação ou o código-fonte se tiver curiosidade). Um engenheiro sênior com experiência em algoritmos passou bastante tempo projetando, implementando e testando esse método e depois o mostrou para vários especialistas na área para garantir que estava correto. Depois, a biblioteca foi testada em beta, lançada e usada extensivamente por milhões de programadores por quase duas décadas. Ainda não foram encontradas falhas no método, mas se uma falha fosse descoberta, ela seria corrigida na próxima versão. Ao usar uma biblioteca padrão, você aproveita o conhecimento dos especialistas que a escreveram e a experiência daqueles que a usaram antes de você.

A partir do Java 7, você não deve mais usar o `Random`. Para a maioria dos usos, o gerador de números aleatórios preferido agora é o `ThreadLocalRandom`. Ele produz números aleatórios de maior qualidade e é muito rápido. No meu computador, ele é 3,6 vezes mais rápido que o `Random`. Para pools de joins de fork e fluxos paralelos, use o `SplittableRandom`.

Uma segunda vantagem de usar as bibliotecas é que você não precisa perder tempo escrevendo soluções ad hoc para problemas que estão apenas marginalmente relacionados ao seu trabalho. Se você é como a maioria dos programadores, prefere gastar seu tempo trabalhando na sua aplicação do que no encanamento subjacente.

Uma terceira vantagem de usar bibliotecas padrão é que seu desempenho tende a melhorar com o tempo, sem esforço da sua parte. Como muitas pessoas as utilizam e porque são usadas em benchmarks padrão da indústria, as organizações que fornecem essas bibliotecas têm um forte incentivo para fazê-las rodar mais rápido. Muitas das bibliotecas da plataforma Java foram reescritas ao longo dos anos, às vezes repetidamente, resultando em melhorias dramáticas de desempenho.

Uma quarta vantagem de usar bibliotecas é que elas tendem a ganhar funcionalidade com o tempo. Se uma biblioteca estiver faltando algo, a comunidade de desenvolvedores fará isso público, e a funcionalidade ausente pode ser adicionada em uma versão subsequente.

Uma vantagem final de usar as bibliotecas padrão é que você coloca seu código no mainstream. Esse código é mais facilmente legível, mantido e reutilizável pela multidão de desenvolvedores.

Dadas todas essas vantagens, parece lógico usar as facilidades da biblioteca em vez de implementações ad hoc, mas muitos programadores não o fazem. Por quê? Talvez eles não saibam que as instalações da biblioteca existem. Diversos recursos são adicionados às bibliotecas em cada grande lançamento, e vale a pena acompanhar essas novidades. Cada vez que há uma grande versão da plataforma Java, uma página web é publicada descrevendo seus novos recursos. Estas páginas valem muito a pena ser lidas [Java8-feat, Java9-feat]. Para reforçar esse ponto, suponha que você queira escrever um

programa para imprimir o conteúdo de uma URL especificada na linha de comando (que é aproximadamente o que o comando `curl` do Linux faz). Antes do Java 9, esse código era meio trabalhoso, mas no Java 9 o método `transferTo` foi adicionado ao `InputStream`. Aqui está um programa completo para realizar essa tarefa usando este novo método:

```
Imprimir o conteúdo de uma URL com transferTo, adicionado no Java
9public static void main(String[] args) lança IOException {
    try (InputStream in = new URL(args[0]).openStream()) {
        in.transferTo(System.out);}}
```

As bibliotecas são grandes demais para estudar toda a documentação [Java9-api], mas todo programador deve estar familiarizado com o básico de `java.lang`, `java.util`, `andjava.io` e seus subpacotes. O conhecimento de outras bibliotecas pode ser adquirido conforme necessário. Está além do escopo deste item resumir as conexões nas bibliotecas, que cresceram imensamente ao longo dos anos.

Várias bibliotecas merecem menção especial. O framework de coleções e a biblioteca `thestreams` (Itens 45–48) devem fazer parte do kit básico de ferramentas de todo programador, assim como partes das utilidades de concorrência em `java.util.concurrent`. Este pacote contém tanto utilitários de alto nível para simplificar a tarefa de programação `multithreaded` quanto primitivas de baixo nível para permitir que especialistas escrevam suas próprias abstrações concorrentes de nível superior. As partes de alto nível do `java.util.concurrent` são discutidas nos Itens 80 e 81.

Ocasionalmente, uma biblioteca pode não atender às suas necessidades. Quanto mais especiais forem suas necessidades, maior a chance disso acontecer. Embora seu primeiro impulso deva ser usar as bibliotecas, se você olhou o que elas oferecem em alguma área e isso não atende às suas necessidades, então use uma implementação alternativa. Sempre haverá lacunas na funcionalidade fornecida por qualquer conjunto finito de bibliotecas. Se você não consegue encontrar o que precisa nas bibliotecas da plataforma Java, sua próxima escolha deve ser procurar bibliotecas de terceiros de alta qualidade, como a excelente biblioteca de código aberto `Guava` [Guava] do Google. Se você não conseguir encontrar a funcionalidade necessária em nenhuma biblioteca apropriada, talvez não tenha escolha a não ser implementá-la você mesmo.

Para resumir, não reinvente a roda. Se você precisa fazer algo que parece ser razoavelmente comum, pode já existir uma instalação nas bibliotecas que faz o que você quer. Se houver, use-a; Se você não sabe, confere. De modo geral, o código da biblioteca provavelmente será melhor do que o código que você mesmo escreveria e provavelmente vai melhorar com o tempo. Isso não reflete suas habilidades como programador. Economias de escala determinam que o código das bibliotecas receba muito mais atenção do que a maioria dos desenvolvedores poderia se dar ao luxo de dedicar à mesma funcionalidade.

Item 60: Evite flutuar e dobrar se forem necessárias respostas exatas

O tipo flutuador e duplo são projetados principalmente para cálculos científicos e de engenharia. Eles realizam aritmética binária de ponto flutuante, cuidadosamente projetada para fornecer aproximações precisas rapidamente em uma ampla gama de magnitudes. No entanto, eles não fornecem resultados exatos e não devem ser usados quando resultados exatos são necessários. Os tipos float e double são particularmente inadequados para cálculos monetários porque é impossível representar 0,1 (ou qualquer outra potência negativa de dez) exatamente como um float ou duplo.

Por exemplo, suponha que você tenha \$1,03 no bolso e gaste 42 centavos. Quanto dinheiro ainda te resta? Aqui está um fragmento ingênuo de programa que tenta responder a essa pergunta:

```
System.out.println(1.03 - 0.42);
```

Infelizmente, imprime 0,6100000000000001. Este não é um caso isolado. Suponha que você tenha um dólar no bolso e compre nove máquinas de arruela com preço de dez centavos cada. Quanto de mudança você recebe?

```
System.out.println(1,00 - 9 * 0,10);
```

De acordo com este fragmento do programa, você recebe \$0,099999999999999998. Você pode pensar que o problema poderia ser resolvido apenas arredondando os resultados antes da impressão, mas infelizmente isso nem sempre funciona. Por exemplo, suponha que você tenha um dólar no bolso e veja uma prateleira com uma fileira de doces deliciosos com preços de 10¢, 20¢, 30¢, e assim por diante, até um dólar. Você compra um de cada doce, começando pelo que custa 10 centavos, até não poder comprar o próximo doce na prateleira. Quantos doces você compra e quanto troco você recebe? Aqui está um programa ingênuo projetado para resolver esse problema:

```
Broken - usa ponto flutuante para cálculo monetário!public
static void main(String[] args) {
    fundos duplos = 1,00; int itensComprado = 0; para (preço dobro
        = 0,10; fundos >= preço; preço += 0,10) {

    fundos -= preço;
    itensBought++;}System.out.println(itensComprados + "itens
    comprados."); System.out.println("Change: $" + fundos);}
```

Se você rodar o programa, vai perceber que pode pagar três doces, e ainda tem \$0,3999999999999999 de sobra. Essa é a resposta errada! A maneira correta de resolver esse problema é usar `BigDecimal`, `int` ou `long` para cálculos monetários.

Aqui está uma transformação direta do programa anterior para usar o tipo `BigDecimal` no lugar de `double`. Note que o construtor de cadeias do `BigDecimal` é usado em vez de seu construtor duplo. Isso é necessário para evitar introduzir valores imprecisos no cálculo [Bloch05, Quebra-cabeça 2]:

```
public static void main(String[] args) {
    final BigDecimal TEN_CENTS = novo BigDecimal("10");

    int itensComprado = 0; Fundos BigDecimal =
        novo BigDecimal("1.00"); para (preço
        BigDecimal = TEN_CENTS;
    funds.compareTo(preço) >= 0; preço =
    preço.adicionar(TEN_CENTS)) {fundos =
    fundos.subtrair(preço);
    itemsBought++;}System.out.println(itensComprados + "itens
    comprados."); System.out.println("Dinheiro sobrando: $" +
    fundos);}
```

Se você rodar o programa revisado, verá que pode comprar quatro doces, com \$0,00 sobrando. Essa é a resposta correta.

No entanto, existem duas desvantagens em usar o `BigDecimal`: ele é muito menos prático do que usar um tipo aritmético primitivo e é muito mais lento. A desvantagem da segunda opção é irrelevante se você estiver resolvendo um problema curto, mas a primeira pode te irritar.

Uma alternativa ao uso do `BigDecimal` é usar `int` ou `long`, dependendo dos valores envolvidos, e acompanhar o ponto decimal por conta própria. Neste exemplo, a abordagem óbvia é fazer todo o cálculo em centavos em vez de dólares. Aqui está uma transformação direta que adota essa abordagem:

```
public static void main(String[] args) {
    int itensComprado = 0; INT fundos = 100; para (int
    price = 10; fundos >= preço; price += 10) {

    fundos -= preço; itemsBought++;}System.out.println(itensComprados +
    "itens comprados."); System.out.println ("Dinheiro sobrando: " +
    fundos + "centavos");}
```

Resumindo, não use float ou double para cálculos que exigem uma resposta exata. Use o BigDecimal se quiser que o sistema acompanhe o ponto decimal e não se importar com o incômodo e o custo de não usar um tipo primitivo. Usar o BigDecimal tem a vantagem adicional de te dar controle total de arredondamento, permitindo escolher entre oito modos de arredondamento sempre que uma operação que envolva arredondamento for realizada. Isso é útil se você estiver realizando cálculos de negócio com um comportamento de arredondamento legalmente obrigatório. Se o desempenho é essencial, você não se importa de acompanhar a vírgula decimal você mesmo, e as quantidades não forem muito grandes, use int ou long. Se as quantidades não excederem nove dígitos decimais, você pode usar int; Se não ultrapassarem dezoito dígitos, você pode usar o long. Se as quantidades puderem exceder dezoito dígitos, use o BigDecimal.

Item 61: Prefira tipos primitivos a primitivos em caixa

Java possui um sistema de tipos em duas partes, consistindo em primitivas, como `int`, `double`, e `boolean`, e tipos de referência, como `String` e `List`. Todo tipo primitivo possui um tipo de referência correspondente, chamado primitivo em caixa. Os primitivos em caixa correspondentes a `int`, `double` e `boolean` são `Integer`, `Double` e `Boolean`.

Como mencionado no Item 6, `autoboxing` e `auto-unboxing` confundem, mas não apagam a distinção entre os tipos primitivo e boxeados. Existem diferenças reais entre os dois, e é importante que você mantenha consciência de qual está usando e escolha cuidadosamente entre eles.

Existem três grandes diferenças entre primitivos e primitivos em caixa. Primeiro, primitivas têm apenas seus valores, enquanto primitivas em caixa possuem identidades distintas de seus valores. Em outras palavras, duas instâncias primitivas em caixa podem ter o mesmo valor e identidades diferentes. Segundo, tipos primitivos possuem apenas valores totalmente funcionais, enquanto cada tipo primitivo `boxed` tem um valor não funcional, que é nulo, além de todos os valores funcionais do tipo primitivo correspondente. Por fim, primitivos são mais eficientes em tempo e espaço do que primitivos em caixa. Essas três diferenças podem te causar sérios problemas se você não tomar cuidado.

Considere o comparador a seguir, que foi projetado para representar a ordem numérica crescente sobre valores inteiros. (Lembre-se de que o método de comparação de um comparador retorna um número negativo, zero ou positivo, dependendo se seu primeiro argumento é menor, igual ou maior que o segundo.) Você não precisaria escrever esse comparador na prática porque ele implementa a ordem natural `onInteger`, mas é um exemplo interessante:

```
Comparador quebrado - você consegue identificar a
falha? Comparador<Integer> natural Ordem =
(I, J) -> (I < J) ? -1 : (i == j ? 0 : 1);
```

Esse comparador parece que deve funcionar e vai passar em muitos testes. Por exemplo, pode ser usado com o `Collections.sort` para ordenar corretamente uma lista de um milhão de elementos, independentemente de a lista contendo ou não elementos duplicados. Mas o comparador é profundamente falho. Para se convencer disso, basta imprimir o valor do `naturalOrder.compare` (novo `Inteiro(42)`, novo `Inteiro(42)`). Ambas as instâncias inteiras representam o mesmo valor (42), então o valor dessa expressão deveria ser 0, mas é 1, o que indica que o primeiro valor inteiro é maior que o segundo!

Então, qual é o problema? O primeiro teste no `naturalOrder` funciona bem. Avaliar a expressão `i < j` faz com que as instâncias inteiras referidas por `i` e `j` sejam automaticamente desbloqueadas; ou seja, extrai seus valores primitivos. A avaliação prossegue para

Verifique se o primeiro dos valores de `int` resultantes é menor que o segundo. Mas suponha que não seja. Depois, o próximo teste avalia a expressão `i == j`, que realiza uma comparação de identidade nas duas referências de objetos. Se `i` e `j` se referem a instâncias inteiras distintas que representam o mesmo valor `int`, essa comparação retornará `false`, e o comparador retornará incorretamente `1`, indicando que o primeiro valor inteiro é maior que o segundo. Aplicar o operador `==` a `boxedprimitives` quase sempre está errado.

Na prática, se você precisa de um comparador para descrever a ordem natural de um tipo, basta chamar `Comparator.naturalOrder()`, e se você mesmo escrever um comparador, deve usar os métodos de construção do comparador ou os métodos estáticos `com-para` em tipos primitivos (Item 14). Dito isso, você poderia resolver o problema no comparador quebrado adicionando duas variáveis locais para armazenar os valores primitivos correspondentes aos parâmetros inteiros em caixa, e realizando todas as comparações nessas variáveis. Isso evita a comparação incorreta de identidade:

```
Comparator<Integer> naturalOrder = (iBoxed, jBoxed) -> {
    int i = iBoxed, j = jBoxed; Auto-unboxing return i <
    j ? -1 : (i == j ? 0 : 1);};
```

Em seguida, considere este pequeno programa encantador:

```
classe pública Inacreditável {
    inteiro estático i;

    public static void main(String[] args) {
        if (i == 42)
            System.out.println ("Inacreditável");
    }}

```

Não, não publica `Inacreditável` — mas o que faz é quase tão estranho quanto. Ele lança `NullPointerException` ao avaliar a expressão `i == 42`. O problema é que `i` é um inteiro, não um `int`, e como todos os campos de referência de objetos não constantes, seu valor inicial é nulo. Quando o programa avalia a expressão `i == 42`, ele está comparando um inteiro com um `int`. Em quase todos os casos, quando você mistura primitivas e primitivas em caixas em uma operação, a primitiva encaixada é automaticamente desencaixada. Se uma referência a `null object` for auto-descaixada, você recebe um `NullPointerException`. Como este programa demonstra, isso pode acontecer quase em qualquer lugar. Resolver o problema é tão simples quanto declarar `i` como um `int` em vez de um inteiro.

Por fim, considere o programa da página 24 do Item 6:

```
Programa horrivelmente lento! Você consegue identificar a criação
do objeto? public static void main(String[] args) {
    Soma longa = 0L; for (long i = 0; i <
        Integer.MAX_VALUE; i++) {
    soma +=
    i;} System.out.println(sum);}
```

Esse programa é muito mais lento do que deveria porque declara acidentalmente uma variável local (soma) como sendo do tipo primitivo em caixa Long, em vez do tipo primitivo longo. O programa compila sem erro ou aviso, e a variável é repetidamente encaixada e desencaixada, causando a degradação de desempenho observada.

Em todos os três programas discutidos neste item, o problema era o mesmo: o programador ignorava a distinção entre primitivos e primitivos em caixa e sofria as consequências. Nos dois primeiros programas, as consequências foram fracasso total; no terceiro, graves problemas de desempenho.

Então, quando você deve usar primitivas em caixa? Eles têm vários usos legítimos. A primeira é como elementos, chaves e valores em coleções. Você não pode colocar primitivos em coleções, então é obrigado a usar primitivos em caixa. Este é um caso especial de amor geral. Você deve usar primitivas em caixa como parâmetros de tipo e métodos inparametrizados (Capítulo 5), porque a linguagem não permite o uso de primitivas. Por exemplo, você não pode declarar uma variável como oftype ThreadLocal<int>, então deve usar<Integer> ThreadLocal em vez disso. Por fim, você deve usar primitivas em caixa ao fazer invocações de métodos reflexivos (Item 65).

Em resumo, use primitivas em vez de primitivas em caixa sempre que tiver escolha. Tipos primitivos são mais simples e rápidos. Se você precisa usar boxedprimitives, tome cuidado! O autoboxing reduz a verbosidade, mas não o perigo, de usar primitivos em caixa. Quando seu programa compara duas primitivas em caixa com o operador ==, ele faz uma comparação de identidade, o que quase certamente não é o que você quer. Quando seu programa faz cálculos mistos envolvendo primitivas em box e unboxed, ele faz unboxing, e quando seu programa faz unboxing, pode lançar uma NullPointerException. Por fim, quando seu programa coloca valores primitivos, isso pode resultar em criações de objetos caras e desnecessárias.

Item 62: Evite strings onde outros tipos são mais apropriados

As strings são feitas para representar texto, e fazem um ótimo trabalho nisso. Como as cordas são tão comuns e tão bem apoiadas pela linguagem, há uma tendência natural a usar cordas para fins diferentes daqueles para os quais foram projetadas. Este item discute algumas coisas que você não deve fazer com cordas.

As strings são substitutos ruins para outros tipos de valor. Quando um dado chega a um programa a partir de um arquivo, da rede ou da entrada do teclado, geralmente é em forma de string. Há uma tendência natural de deixar assim, mas essa dez é justificada apenas se os dados realmente forem de natureza textual. Se for numérico, deve ser traduzido para o tipo numérico apropriado, como `int`, `float` ou `BigInteger`. Se for a resposta para uma pergunta de sim ou não, deve ser traduzida para um tipo de `enum` apropriado ou um `booleano`. Mais geralmente, se houver um tipo de valor apropriado, seja primitivo ou referência a objetos, você deve usá-lo; Se não houver, você deveria escrever um. Embora esse conselho possa parecer óbvio, ele frequentemente é violado.

Cordas são substitutos ruins para os tipos de `enum`. Como discutido no Item 34, `enums` produzem constantes de tipo enumeradas muito melhores do que cadeias.

Strings são substitutos ruins para tipos agregados. Se uma entidade tem múltiplos componentes, geralmente não é uma boa ideia representá-la como uma única cadeia. Por exemplo, aqui está uma linha de código que vem de um sistema real — nomes de identificadores foram trocados para proteger os culpados:

```
Uso inadequado de string como tipo agregadoString
compoundKey = className + "#" + i.next();
```

Essa abordagem tem muitas desvantagens. Se o caractere usado para separar campos ocorrer em um dos campos, pode resultar em caos. Para acessar campos individuais, você precisa analisar a string, que é lenta, tediosa e propensa a erros. Você não pode fornecer métodos iguais, `toString` ou `compareTo`, mas é forçado a aceitar o comportamento que o `String` proporciona. Uma abordagem melhor é simplesmente escrever uma classe para representar o agregado, frequentemente uma classe privada estática (Item 24).

Cordas são substitutos ruins para capacidades. Ocasionalmente, strings são usadas para conceder acesso a alguma funcionalidade. Por exemplo, considere o projeto da facilidade de variável local de um `thread`. Tal funcionalidade fornece variáveis para as quais cada `thread` tem seu próprio valor. As bibliotecas Java têm facilidade de variável local de `thread` desde a versão 1.2, mas antes disso, os programadores precisavam criar suas próprias. Quando confrontados com a tarefa de projetar tal instalação há muitos anos, vários

Pessoas criaram independentemente o mesmo design, no qual chaves de string fornecidas pelo cliente são usadas para identificar cada variável local de thread:

```
Broken - uso inadequado da string como capability!public
class ThreadLocal {
    private ThreadLocal() { } // Noninstantiable

    Define o valor da thread atual para a variável nomeada.public
    static void set(String key, valor do objeto);

    Retorna o valor da thread atual para a variável nomeada.public static
    Object get(String key);}
```

O problema dessa abordagem é que as chaves de string representam um namespace sharedglobal para variáveis locais de thread. Para que a abordagem funcione, as chaves de string fornecidas pelo cliente precisam ser únicas: se dois clientes decidem independentemente usar o mesmo nome para sua variável local de thread, eles compartilham involuntariamente uma única variável, o que geralmente fará com que ambos os clientes falhem. Além disso, a segurança é ruim. Um cliente malicioso pode usar intencionalmente a mesma chave string que outro cliente para obter acesso ilícito aos dados do outro cliente.

Essa API pode ser corrigida substituindo a string por uma chave não falsificável (às vezes chamada de capacidade):

```
classe pública ThreadLocal {
    private ThreadLocal() { }    Não instanciado

    chave da classe pública estática { (Capacidade)
    Key() { }}

    Gera uma chave estática única e não
    falsificável KeyKey() {
    return new Key();}

    conjunto estático público (chave chave, valor do
    objeto); objeto estático público get(Key key);}
```

Embora isso resolva ambos os problemas da API baseada em strings, você pode fazer muito melhor. Você realmente não precisa mais dos métodos estáticos. Eles podem, em vez disso, se tornar métodos de instância na chave, momento em que a chave deixa de ser uma chave para uma variável local de thread: ela é uma variável local de thread. Neste ponto, o nível superior

a classe não está mais fazendo nada por você, então é melhor se livrar dela e nomear a classe aninhada para `ThreadLocal`:

```
classe final pública ThreadLocal {  
    ThreadLocal(); público conjunto  
    público void (valor do objeto);  
    Objeto público get();}
```

Essa API não é typesafe, porque você precisa lançar o valor de `Object` para seu type atual ao recuperá-lo de uma variável `thread-local`. É impossível tornar a API original baseada em strings typesafe e difícil tornar a key-based API typesafe, mas é simples tornar essa API typesafe tornando o `ThreadLocal` uma classe parametrizada (Item 29):

```
classe final pública ThreadLocal<T> {  
    ThreadLocal(); público conjunto  
    público void (valor T); public  
    T get();}
```

Esta é, de forma geral, a API que `java.lang.ThreadLocal` oferece. Além de resolver os problemas da API baseada em strings, ela é mais rápida e elegante do que qualquer uma das APIs baseadas em chaves.

Resumindo, evite a tendência natural de representar objetos como cadeias quando existem ou podem ser escritos tipos de dados melhores. Usadas de forma inadequada, as cordas são mais pesadas, menos flexíveis, mais lentas e mais propensas a erros do que outros tipos. Tipos para os quais as cadeias são comumente mal utilizadas incluem tipos primitivos, enums e tipos agregados.

Item 63: Cuidado com o desempenho da concatenação de strings

O operador de concatenação de cadeias (+) é uma maneira conveniente de combinar algumas cadeias em uma. É aceitável para gerar uma única linha de saída ou construir a representação de strings de um objeto pequeno de tamanho fixo, mas não escala. Usar repetidamente o operador de concatenação de strings para concatenar *n* strings requer um tempo qua-drástico em *n*. Isso é uma consequência infeliz do fato de que as cordas são imutáveis (Item 17).

Quando duas cadeias são concatenadas, o conteúdo de ambas é copiado.

Por exemplo, considere este método, que constrói a representação em cadeia de uma declaração de faturamento concatenando repetidamente uma linha para cada item:

```
Use inadequado da concatenação de strings - Tem desempenho ruim!
public String Statement() {
    Resultado da sequência = "";
    para (int i = 0; i < numItems(); i++)
        resultado += linhaParaItem(i); Concatenação de cadeias
    retorne resultado;}

```

O método tem um desempenho péssimo se o número de itens for grande. Para alcançar desempenho aceitável, use um `StringBuilder` no lugar de um `String` para armazenar a instrução sob construção:

```
Instrução pública String() {
    StringBuilder b = novo StringBuilder(numItems() * LINE_WIDTH);
    para (int i = 0; i < numItems(); i++)
        b.append(linhaParaItem(i));
    return b.toString();}

```

Muito trabalho foi feito para tornar a concatenação de strings mais rápida desde o Java 6, mas a diferença no desempenho dos dois métodos ainda é dramática: `getNumItems` retorna 100 e `lineForItem` retorna uma string de 80 caracteres, o segundo método roda 6,5 vezes mais rápido que o primeiro na minha máquina. Como o primeiro método é quadrático no número de itens e o segundo é linear, a diferença de desempenho aumenta muito à medida que o número de itens cresce. Note que o segundo método pré-aloca um `StringBuilder` grande o suficiente para conter o resultado completo, eliminando a necessidade de crescimento automático. Mesmo que esteja desafinado para usar um `StringBuilder` de tamanho padrão, ainda é 5,5 vezes mais rápido que o primeiro método.

A moral é simples: Não use o operador de concatenação de strings para combinar mais de algumas strings, a menos que o desempenho seja irrelevante. Use `StringBuilder`, em vez disso, o método `append`. Alternativamente, use um array de caracteres ou processe as strings uma de cada vez em vez de combiná-las.

Item 64: Referir-se a objetos por suas interfaces

O item 51 diz que você deve usar interfaces em vez de classes como tipos de parâmetros. Mais geralmente, você deve preferir o uso de interfaces em vez de classes para referenciar objetos. Se existirem tipos de interface apropriados, então parâmetros, valores de retorno, variáveis e campos devem ser todos declarados usando tipos de interface. A única vez que você realmente precisa se referir à classe de um objeto é quando está criando com um construtor. Para tornar isso concreto, considere o caso do `LinkedHashSet`, que é uma implementação da interface de `Set`. Crie o hábito de digitar isto:

```
Good - usa interface como typeSet<Son>
sonSet = new LinkedHashSet<>();
```

Não é isso:

```
Ruim - usa classe como tipo! LinkedHashSet<Son> sonSet
= novo LinkedHashSet<>();
```

Se você criar o hábito de usar interfaces como tipos, seu programa ficará muito mais flexível. Se você decidir que quer mudar de implementação, tudo o que precisa fazer é mudar o nome da classe no construtor (ou usar uma fábrica estática diferente). Por exemplo, a primeira declaração poderia ser alterada para ser:

```
Set<Son> sonSet = novo HashSet<>();
```

e todo o código ao redor continuaria funcionando. O código ao redor não estava ciente do tipo de implementação antigo, então estaria alheio à mudança.

Há uma ressalva: se a implementação original oferecia alguma funcionalidade especial não exigida pelo contrato geral da interface e o código dependia dessa funcionalidade, então é fundamental que a nova implementação ofereça a mesma funcionalidade. Por exemplo, se o código em torno da firstdeclaration dependesse da política de ordenação do `LinkedHashSet`, então seria incorreto substituir `HashSet` por `LinkedHashSet` na declaração, pois o `HashSet` não garante a ordem de iteração.

Então, por que você iria querer mudar um tipo de implementação? Porque a segunda implementação oferece melhor desempenho que a original, ou porque oferece funcionalidades desejáveis que a implementação original não possui. Por exemplo, suponha que um campo contenha uma instância `HashMap`. Mudar para um `EnumMap` vai proporcionar melhor desempenho e ordem de iteração consistentes com a ordem natural das chaves, mas você só pode usar um `EnumMap` se o tipo de chave for do tipo `enum`.

Mudar o `HashMap` para um `LinkedHashMap` fornecerá uma ordem de iteração previsível com desempenho comparável ao do `HashMap`, sem exigir nenhuma necessidade especial sobre o tipo de chave.

Você pode achar que é aceitável declarar uma variável usando seu tipo de implementação, porque você pode mudar o tipo de declaração e o tipo de implementação ao mesmo tempo, mas não há garantia de que essa mudança resultará em um programa que compila. Se o código cliente usou métodos no tipo de implementação original que também não estão presentes em sua substituição ou se o código cliente passou a instância para um método que requer o tipo de implementação original, então o código não será mais compilado após essa alteração. Declarar a variável com o `interface type` te mantém honesto.

É totalmente apropriado referir-se a um objeto por uma classe em vez de uma interface se não existir uma interface apropriada. Por exemplo, considere classes de valor, como `String` e `BigInteger`. Classes de valor raramente são escritas pensando em múltiplas implementações. Elas costumam ser finais e raramente possuem interfaces correspondentes. É perfeitamente apropriado usar uma classe de valor como parâmetro, variável, campo ou tipo de retorno.

Um segundo caso em que não existe um tipo de interface apropriado é o de objetos pertencentes a um framework cujos tipos fundamentais são classes em vez de interfaces. Se um objeto pertence a esse tipo de estrutura baseada em classes, é preferível referir-se a ele pela classe base relevante, que muitas vezes é abstrata, em vez de sua classe de implementação. Muitas classes `java.io`, como `OutputStream`, se enquadram nessa categoria.

Um caso final em que não existe um tipo de interface apropriado é o de classes que implementam uma interface, mas também fornecem métodos extras que não estão na interface — por exemplo, o `PriorityQueue` possui um método comparador que não está presente na interface da `Queue`. Tal classe deve ser usada para se referir apenas às suas instâncias se o programa depender dos métodos extras, e isso deve ser muito raro.

Esses três casos não pretendem ser exaustivos, mas apenas transmitir o tipo de situações em que é apropriado referir-se a um objeto por sua classe. Na prática, deveria ser evidente se um determinado objeto possui uma interface apropriada. Se isso acontecer, seu programa será mais flexível e estiloso se você usar a interface para se referir ao objeto. Se não houver uma interface apropriada, basta usar a classe menos específica na hierarquia de classes que forneça a funcionalidade necessária.

Item 65: Prefira interfaces à reflexão

A ferramenta central de reflexão, `java.lang.reflect`, oferece acesso programático a classes arbitrárias. Dado um objeto `Class`, você pode obter instâncias `Constructor`, `Method` e `Field` representando os construtores, métodos e campos da classe representada pela instância de `Class`. Esses objetos fornecem acesso programático aos nomes dos membros, tipos de campo, assinaturas de métodos da classe e assim por diante.

Além disso, instâncias de `Constructor`, `Método` e `Campo` permitem manipular seus equivalentes subjacentes de forma reflexiva: você pode construir instâncias, invocar métodos e acessar campos da classe subjacente invocando métodos nas instâncias `Constructor`, `Method` e `Field`. Por exemplo, `Method.invoke` permite que você invoque qualquer método em qualquer objeto de qualquer classe (sujeito às restrições usuais de segurança). A reflexão permite que uma classe use outra, mesmo que a segunda classe não existisse quando a primeira foi compilada. Esse poder, no entanto, tem um preço:

- Você perde todos os benefícios da verificação de tipos em tempo de compilação, incluindo a verificação de exceções. Se um programa tentar invocar um método inexistente ou inacessível de forma reflexiva, ele falhará em tempo de execução, a menos que você tenha tomado precauções especiais.
- O código necessário para realizar o acesso reflexivo é desajeitado e prolixo. É difícil de escrever e difícil de ler.
- O desempenho sofre. A invocação do método reflexivo é muito mais lenta do que a invocação do método normal. É difícil dizer exatamente o quanto mais lento, pois há muitos fatores em jogo. Na minha máquina, invocar um método sem parâmetros de entrada e retorno de `int` era onze vezes mais lento quando feito de forma reflexiva.

Existem algumas aplicações sofisticadas que exigem reflexão. Exemplos incluem ferramentas de análise de código e frameworks de injeção de dependências. Mesmo essas ferramentas têm se afastado da reflexão ultimamente, à medida que suas desvantagens ficam mais claras. Se você tem dúvidas sobre se sua candidatura precisa de reflexão, provavelmente não precisa.

Você pode obter muitos dos benefícios da reflexão incorrendo em poucos de seus custos ao usá-la apenas de forma muito limitada. Para muitos programas que precisam usar uma classe indisponível em tempo de compilação, existe em tempo de compilação uma interface ou superclasse apropriada para se referir à classe (Item 64). Se for esse o caso, você pode criar instâncias reflexivamente e acessá-las normalmente via interface ou superclasse deles.

Por exemplo, aqui está um programa que cria uma `<String>` instância `Set` cuja classe `se` é especificada pelo primeiro argumento da linha de comando. O programa insere o

Argumentos restantes na linha de comando no conjunto e imprime o conjunto. Independentemente do primeiro argumento, o programa imprime os argumentos restantes com duplicatas eliminadas. A ordem em que esses argumentos são impressos, no entanto, depende da classe especificada no primeiro argumento. Se você especificar `java.util.HashSet`, eles são impressos em ordem aparentemente aleatória; se você especificar `java.util.TreeSet`, eles são impressos em ordem alfabética porque os elementos em um `TreeSet` estão organizados:

```

Instanciação reflexiva com interface acessível
static void main(String[] args) {
    Traduza o nome da classe para um Class
    ObjectClass<? estende Set<String>> cl = null; try {

        cl = (Classe<? estende Set<String>>) Elenco descontrolado!
        Classe.paraNome(args[0]);
    } captura (ClassNotFoundException e) {
        fatalError("Classe não encontrada.");}Obtenha o
        constructorConstructor<? estende Set<String>> cons
        = null; try {

            contras = cl.getDeclaredConstructor();}
            catch (NoSuchMethodException e) {
                fatalError("No parameterless
                constructor");}Instanciar o conjuntoSet<String> s =
                nulo; try {

                    s = cons.newInstance();} catch
                    (IllegalAccessException e) {
                        fatalError("Constructor not accessed");} catch
                    (InstantiationException e) {
                        fatalError("Classe não instanciada.");} catch
                    (InvocationTargetException e) {
                        fatalError("Constructor threw " + e.getCause());} catch
                    (ClassCastException e) {
                        fatalError("Class não implementa Set");}Exerce o
                    sets.addAll(Arrays.asList(args).subList(1, args.length));
                    System.out.println(s);}private static void fatalError(String
                    msg) {

                        System.err.println(msg);
                        System.exit(1);}

```


Embora esse programa seja apenas um brinquedo, a técnica que ele demonstra é bastante poderosa. O programa brinquedo poderia facilmente ser transformado em um testador genérico de Set que validasse a implementação especificada de Set manipulando agressivamente uma ou mais instâncias e verificando que elas obedecem ao contrato de Set. Da mesma forma, poderia se transformar em uma ferramenta genérica de análise de desempenho de conjuntos. Na verdade, essa técnica é suficientemente poderosa para implementar um framework completo de provedor de serviços (Item 1). Normalmente, essa técnica é tudo o que você precisa em termos de reflexão.

Este exemplo demonstra duas desvantagens da reflexão. Primeiro, o exemplo pode gerar seis exceções diferentes em tempo de execução, todas elas teriam sido erros em tempo de compilação se a instanciação reflexiva não fosse usada. (Por diversão, você pode fazer o programa gerar cada uma das seis exceções passando os argumentos apropriados na linha de comando.) A segunda desvantagem é que são necessárias vinte e cinco linhas de código tedioso para gerar uma instância da classe a partir do nome, enquanto a invocação `aconstructor` caberia perfeitamente em uma única linha. O comprimento do programa poderia ser reduzido capturando o `ReflectiveOperationException`, uma superclasse das várias exceções reflexivas introduzida no Java 7. Ambas as desvantagens são restritas à parte do programa que instancia o objeto. Uma vez instanciado, o conjunto é indistinguível de qualquer outra instância de Set. Em um programa real, a grande maior parte do código não é afetada por esse uso limitado de reflexão.

Se você compilar esse programa, receberá um aviso de elenco desmarcado. Este aviso é legítimo, pois o cast para `Class<? estende Set<String>>` terá sucesso mesmo que a classe nomeada não seja uma implementação de Set, caso em que o programa lançará uma `ClassCastException` ao instanciar a classe. Para saber sobre suprimir o aviso, leia o Item 27.

Um uso legítimo, embora raro, da reflexão é gerenciar as dependências de uma classe em relação a outras classes, métodos ou campos que podem estar ausentes em tempo de execução. Isso pode ser útil se você estiver escrevendo um pacote que deve rodar contra múltiplas versões de outro pacote. A técnica é compilar seu pacote com o ambiente mínimo necessário para suportá-lo, tipicamente a versão mais antiga, e acessar quaisquer classes ou métodos mais recentes de forma reflexiva. Para que isso funcione, você precisa tomar as medidas apropriadas se uma classe ou método mais novo que você está tentando acessar não existir em tempo de execução. A ação adequada pode consistir em usar algum meio alternativo para alcançar o mesmo objetivo ou operar com menor funcionalidade.

Em resumo, a reflexão é uma funcionalidade poderosa necessária para certas tarefas sofisticadas de programação de sistemas, mas possui muitas desvantagens. Se você está escrevendo um programa que precisa trabalhar com classes desconhecidas em tempo de compilação, deveria, se possível, usar reflexão apenas para instanciar objetos e acessar os objetos usando alguma interface ou superclasse conhecida em tempo de compilação.

Item 66: Use métodos nativos com critério

A Java Native Interface (JNI) permite que programas Java chamem métodos nativos, que são métodos escritos em linguagens nativas de programação como C ou C++. Historicamente, os métodos nativos tiveram três usos principais. Eles oferecem acesso a instalações específicas da plataforma, como registros de registros. Eles fornecem acesso a bibliotecas existentes de código nativo, incluindo bibliotecas legadas que dão acesso a dados legados. Por fim, métodos nativos são usados para escrever partes críticas de desempenho de aplicações em línguas nativas, visando melhorar o desempenho.

É legítimo usar métodos nativos para acessar instalações específicas da plataforma, mas raramente é necessário: à medida que a plataforma Java amadureceu, passou a fornecer acesso a muitos recursos que antes eram encontrados apenas em plataformas hospedeiras. Por exemplo, a API de processos, adicionada no Java 9, fornece acesso aos processos do sistema operacional. Também é legítimo usar `nativeMethods` para usar bibliotecas nativas quando não existem bibliotecas equivalentes disponíveis em Java.

Raramente é recomendável usar métodos nativos para melhorar o desempenho. Lançamentos iniciais (antes do Java 3), isso era frequentemente necessário, mas as JVMs ficaram muito mais rápidas desde então. Para a maioria das tarefas, agora é possível obter desempenho comparável em Java. Por exemplo, quando `java.math` foi adicionado na versão 1.1, o `BigInteger` dependia de uma biblioteca aritmética multiprecisão então rápida escrita em C. No Java 3, o `BigInteger` foi reimplementado em Java e cuidadosamente ajustado para rodar mais rápido que a implementação nativa original.

Um triste epílogo a essa história é que o `BigInteger` mudou pouco desde então, com exceção da multiplicação mais rápida para números grandes no Java 8. Naquela época, o trabalho continuou em ritmo acelerado em bibliotecas nativas, notadamente a GNU Multiple Precision Arithmetic Library (GMP). Programadores Java que precisam de multiprecisão aritmética verdadeiramente de alto desempenho agora estão justificados em usar GMP por métodos nativos [Blum14].

O uso de métodos nativos tem sérias desvantajas. Como as línguas nativas não são seguras (Item 50), aplicações que usam métodos nativos não estão mais imunes a erros de corrupção de memória. Como linguagens nativas dependem mais da plataforma do que o Java, programas que usam métodos nativos são menos portáteis. Eles também são mais difíceis de debugar. Se você não tomar cuidado, métodos nativos podem diminuir o desempenho porque o coletor de lixo não pode automatizar, nem sequer acompanhar, o uso de memória nativa (Item 8), e há um custo associado ao entrar e sair do código nativo. Por fim, métodos nativos exigem um "código de colagem" difícil de ler e tedioso de escrever.

Resumindo, pense duas vezes antes de usar métodos nativos. É raro que você precise usá-los para melhorar o desempenho. Se você precisar usar métodos nativos para acessar recursos de baixo nível ou bibliotecas nativas, use o mínimo de código nativo possível e teste tudo de forma detalhada. Um único bug no código nativo pode corromper toda a sua aplicação.

Item 67: Otimizar com critério

Existem três aforismos relacionados à otimização que todos deveriam conhecer:

Mais pecados computacionais são cometidos em nome da eficiência (sem necessariamente alcançá-la) do que por qualquer outro motivo único — incluindo estupidez cega.

—William A. Wulf [Wulf72]

Devemos esquecer pequenas eficiências, digamos cerca de 97% das vezes: a otimização pré-artificial é a raiz de todo o mal.

—Donald E. Knuth [Knuth74]

Seguimos duas regras no que diz respeito à otimização:

Regra 1. Não faça isso. Regra 2 (apenas para especialistas). Não faça isso ainda—ou seja, não até ter um

Solução perfeitamente clara e não otimizada.

—M. A. Jackson [Jackson75]

Todos esses aforismos antecedem a linguagem de programação Java em duas décadas. Eles dizem uma verdade profunda sobre otimização: é fácil causar mais mal do que bem, especialmente se você otimizar prematuramente. Nesse processo, você pode produzir um software que não é nem rápido nem correto e que não pode ser facilmente consertado.

Não sacrifique princípios arquitetônicos sólidos pelo desempenho. Procure escrever bons programas em vez de rápidos. Se um bom programa não for rápido o suficiente, sua arquitetura permitirá que ele seja otimizado. Bons programas incorporam o princípio do ocultamento de informação: sempre que possível, localizam decisões de projeto dentro de componentes individuais, de modo que decisões individuais possam ser alteradas sem afetar o seu próprio sistema (Item 15).

Isso não significa que você pode ignorar preocupações de desempenho até que seu programa esteja completo. Problemas de implementação podem ser corrigidos por otimização posterior, mas falhas arquitetônicas persistentes que limitam o desempenho podem ser impossíveis de corrigir sem reescrever o sistema. Mudar um aspecto fundamental do seu design depois do fato pode resultar em um sistema mal estruturado, difícil de manter e evoluir. Portanto, você deve considerar o desempenho durante o processo de design.

Procure evitar decisões de design que limitem o desempenho. Os componentes de um projeto que são mais difíceis de mudar depois do fato são aqueles que especificam interações entre componentes e com o mundo exterior. Entre esses principais componentes de design estão APIs, protocolos em nível de fio e formatos de dados persistentes. Esses componentes de projeto não são apenas difíceis ou impossíveis de serem alterados posteriormente, mas todos eles podem impor limitações significativas ao desempenho que um sistema pode alcançar.

Considere as consequências de desempenho das suas decisões de design de API. Tornar um tipo público mutável pode exigir muita cópia defensiva desnecessária (Item 50). De forma semelhante, usar herança em uma classe pública onde a composição teria sido apropriada vincula a classe para sempre à sua superclasse, o que pode impor limites artificiais ao desempenho da subclasse (Item 18). Como exemplo final, usar um tipo de implementação em vez de uma interface em uma API conecta você a uma implementação específica, mesmo que implementações mais rápidas possam ser escritas no futuro (Item 64).

Os efeitos do design de API no desempenho são muito reais. Considere o `getSize` método na classe `java.awt.Component`. A decisão de que esse método crítico de desempenho deveria retornar uma instância de `Dimensão`, juntamente com a decisão de que as instâncias de `Dimensão` são mutáveis, força qualquer implementação desse método a alocar uma nova instância de `Dimensão` em cada invocação. Embora alocar objetos pequenos seja barato em uma VM moderna, alocar milhões de objetos desnecessariamente pode causar um dano real ao desempenho.

Existiam várias alternativas de design de APIs. Idealmente, a `Dimensão` deveria ser imutável (Item 17); alternativamente, `getSize` poderia ter sido substituído por `twoMethods` retornando os componentes primitivos individuais de um objeto `Dimension`. Na verdade, dois desses métodos foram adicionados ao `Component` no Java 2 por razões de desempenho. No entanto, o código cliente pré-existente ainda usa o método `getSize` e ainda sofre as consequências de desempenho das decisões originais de design da API.

Felizmente, geralmente é verdade que um bom design de API é consistente com bom desempenho. É uma péssima ideia distorcer uma API para alcançar um bom desempenho. O problema de desempenho que fez você distorcer a API pode desaparecer em uma futura versão da plataforma ou de outro software subjacente, mas a API distorcida e as dores de cabeça de suporte que ela acompanham vão ficar com você para sempre.

Depois de projetar cuidadosamente seu programa e produzir uma implementação clara, concisa e bem estruturada, pode ser hora de considerar a otimização, assumindo que você ainda não esteja satisfeito com o desempenho do programa.

Lembre-se de que as duas regras de otimização de Jackson eram "Não faça isso" e "(apenas para especialistas). Não faça isso ainda." Ele poderia ter acrescentado mais uma: medir desempenho antes e depois de cada tentativa de otimização. Você pode se surpreender com o que encontrar. Frequentemente, as tentativas de otimização não têm efeito mensurável na performance; Às vezes, eles pioram a situação. O principal motivo é que é difícil adivinhar onde seu programa está gastando seu tempo. A parte do programa que você acha lenta pode não ser a culpada, nesse caso você estaria perdendo seu tempo tentando otimizá-la. A sabedoria comum diz que os programas passam 90% do tempo em 10% do código.

Ferramentas de perfilamento podem ajudar você a decidir onde focar seus esforços de otimização. Essas ferramentas fornecem informações em tempo de execução, como aproximadamente quanto tempo cada método está consumindo e quantas vezes ele é invocado. Além de focar seus esforços de ajuste, isso pode alertá-lo sobre a necessidade de mudanças algorítmicas. Se um algoritmo quadrático (ou pior) estiver escondido dentro do seu programa, nenhum ajuste vai resolver o problema. Você deve substituir o algoritmo por um que seja mais eficiente. Quanto mais código estiver no sistema, mais importante é usar um perfilador. É como procurar uma agulha em um palheiro: quanto maior o palheiro, mais útil é ter um detector de metais. Outra ferramenta que merece menção especial é o *jmh*, que não é um perfilador, mas sim uma framework de microbenchmarking que oferece uma visibilidade paralela do desempenho detalhado do código Java [JMH].

A necessidade de medir os efeitos da tentativa de otimização é ainda maior em Java do que em linguagens mais tradicionais como C e C++, porque o Java tem um modelo de desempenho mais fraco: o custo relativo das várias operações primitivas é pouco bem definido. A "lacuna de abstração" entre o que o programador escreve e o que a CPU executa é maior, o que torna ainda mais difícil prever de forma confiável as consequências de desempenho das otimizações. Existem muitos mitos de performance circulando que acabam sendo meias-verdades ou mentiras descaradas.

O modelo de desempenho do Java não é apenas indefinido, como varia de implementação para implementação, de lançamento para lançamento e de processador para processador. Se você vai rodar seu programa em múltiplas implementações ou em múltiplas plataformas de hardware, é importante que você meça os efeitos da sua otimização em cada uma. Ocasionalmente, você pode ser forçado a fazer concessões entre desempenho em diferentes implementações ou plataformas de hardware.

Nas quase duas décadas desde que este item foi escrito, cada componente da pilha de software Java cresceu em complexidade, desde processadores até VMs e bibliotecas, e a variedade de hardware em que o Java roda cresceu imensamente. Tudo isso combinado tornou o desempenho dos programas Java ainda menos previsível agora do que era em 2001, com um aumento correspondente na necessidade de medi-lo.

Para resumir, não busque escrever programas rápidos — busque escrever bons programas; A velocidade vai seguir. Mas pense no desempenho enquanto projeta sistemas, especialmente ao projetar APIs, protocolos em nível de conexão e formatos de dados persistentes. Quando você terminar de construir o sistema, meça seu desempenho. Se for rápido o suficiente, você acabou. Se não, localize a origem do problema com a ajuda de um perfilador e comece a otimizar as partes relevantes do sistema. O primeiro passo é examinar sua escolha de algoritmos: nenhuma quantidade de otimização de baixo nível pode compensar uma escolha ruim de algoritmo. Repita esse processo conforme necessário, medindo o desempenho após cada mudança, até ficar satisfeito.

Item 68: Seguir as convenções de nomenclatura geralmente aceitas

A plataforma Java possui um conjunto bem estabelecido de convenções de nomenclatura, muitas das quais estão contidas na Especificação da Linguagem Java [JLS, 6.1]. Falando de forma geral, as convenções de nomeação se dividem em duas categorias: tipográficas e gramaticais.

Existem apenas algumas poucas convenções tipográficas de nomenclatura, coveringpackages, classes, interfaces, métodos, campos e variáveis de tipo. Você raramente deve violá-los e nunca sem um motivo muito bom. Se uma API violar essas convenções, pode ser difícil de usar. Se uma implementação violar esses problemas, pode ser difícil manter. Em ambos os casos, violações têm o potencial de confundir e irritar outros programadores que trabalham com o código e podem causar suposições equivocadas que levam a erros. As convenções são resumidas neste item.

Os nomes dos pacotes e módulos devem ser hierárquicos, com os componentes separados por períodos. Os componentes devem consistir em caracteres alfabéticos minúsculos e, raramente, dígitos. O nome de qualquer pacote que será usado fora da sua organização deve começar com o domínio da Internet da sua organização com os componentes invertidos, por exemplo, edu.cmu, com.google, org.eff. As bibliotecas padrão e pacotes opcionais, cujos nomes começam com java e javax, são exceções a essa regra. Os usuários não devem criar pacotes ou módulos cujos nomes comecem com java ou javax. Regras detalhadas para converter nomes de domínio da Internet em prefixos de nomes de pacotes podem ser encontradas no JLS [JLS, 6.1].

O restante do nome de um pacote deve consistir em um ou mais componentes que descrevam o pacote. Os componentes devem ser curtos, geralmente oito ou menos caracteres. Abreviações significativas são incentivadas, por exemplo, util em vez de utilities. Siglas são aceitáveis, por exemplo, awt. Os componentes geralmente devem consistir em uma única palavra ou abreviação.

Muitos pacotes possuem nomes com apenas um componente além do nome de domínio da Internet. Componentes adicionais são apropriados para grandes instalações cujo tamanho exige que sejam divididas em uma hierarquia informal. Por exemplo, o pacote thejavax.util possui uma hierarquia rica de pacotes com nomes como asjava.util.concurrent.atomic. Tais pacotes são conhecidos como subpacotes, embora quase não haja suporte linguístico para hierarquias de pacotes.

Nomes de classes e interfaces, incluindo nomes de enum e tipos de anotação, devem consistir em uma ou mais palavras, com a primeira letra de cada palavra capitalizada, por exemplo, List ou FutureTask. Abreviações devem ser evitadas, exceto por siglas e certas abreviações comuns como max e min. Há alguma discordância sobre se as siglas devem ser maiúsculas ou apenas a primeira letra maiúscula. Embora alguns programadores ainda usem maiúsculas, um argumento forte

pode ser feito a favor de capitalizar apenas a primeira letra: mesmo que múltiplas siglas ocorram uma atrás da outra, você ainda pode identificar onde uma palavra começa e a próxima termina. Qual nome de classe você prefere ver, `HTTPURL` ou `HttpUrl`?

Os nomes de métodos e campos seguem as mesmas convenções tipográficas dos nomes de classes e interfaces, exceto que a primeira letra de um nome de método ou campo deve ser minúscula, por exemplo, `remove` ou `ensureCapacity`. Se um acrônimo aparece como a primeira palavra de um método ou nome de campo, deve ser minúsculo.

A única exceção à regra anterior diz respeito aos "campos constantes", cujos nomes devem consistir em uma ou mais palavras maiúsculas separadas pelo sublinhado, por exemplo, `VALUES` ou `NEGATIVE_INFINITY`. Um corpo constante é um campo estático cujo valor é imutável. Se um campo final estático tem um tipo primitivo ou tipo de referência imutável (Item 17), então ele é um corpo constante. Por exemplo, constantes de enum são corpos constantes. Se um campo final estático tem um tipo de referência mutável, ele ainda pode ser um campo constante se o objeto referenciado for imutável. Note que campos constantes constituem o único uso recomendado de sublinhados.

Nomes locais de variáveis possuem convenções tipográficas semelhantes aos nomes `member`, exceto que abreviações são permitidas, assim como caracteres individuais — e sequências curtas de caracteres cujo significado depende do contexto em que ocorrem, por exemplo, `i`, `denom`, `houseNum`. Parâmetros de entrada são um tipo especial de variável local. Eles devem ser nomeados com muito mais cuidado do que as localvariáveis comuns, pois seus nomes são parte integrante da documentação do método.

Os nomes dos parâmetros de tipo geralmente consistem em uma única letra. Mais comumente, é um destes cinco: `T` para um tipo arbitrário, `E` para o tipo de elemento de uma coleção, `K` e `V` para os tipos de chave e valor de um mapa, e `X` para uma exceção. O tipo de retorno de uma função geralmente é `R`. Uma sequência de tipos arbitrários pode ser `T`, `U`, `V` ou `T1`, `T2`, `T3`.

Para referência rápida, a tabela a seguir mostra exemplos de convenções tipográficas.

Tipo de Identificação Exemplos

Pacote ou módulo `org.junit.jupiter.api`, `com.google.common.collect`

Classe ou Interface `Stream`, `FutureTask`, `LinkedHashMap`, `HttpClient`

Método ou Campo `remove`, `groupingBy`, `getCrc`

Campo Constante `MIN_VALUE`, `NEGATIVE_INFINITY`

Variável Local `i`, `denom`, `casaNum`

Parâmetro de Tipo `T`, `E`, `K`, `V`, `X`, `R`, `U`, `V`, `T1`, `T2`

Convenções gramaticais de nomenclatura são mais flexíveis e mais controversas do que convenções tipográficas. Não existem convenções gramaticais de nomeação para pacotes. Classes instanciadas, incluindo tipos `enum`, geralmente são nomeadas com um único substantivo ou frase nominal, como `Thread`, `PriorityQueue`, ou `ChessPiece`. Classes utilitárias não instanciáveis (Item 4) são frequentemente nomeadas com substantivos plurais, como `Collectors` ou `Collections`. Interfaces são chamadas de *likeclasses*, por exemplo, `Collection` ou `Comparator`, ou com um adjetivo que termina in *able* ou *ible*, por exemplo, `Runnable`, `Iterable` ou `Accessible`. Como os tipos de anotação têm tantos usos, nenhuma parte do discurso predomina. Substantivos, verbos, preposições e adjetivos são comuns, por exemplo, `BindingAnnotation`, `Inject`, `ImplementedBy` ou `Singleton`.

Métodos que realizam alguma ação geralmente são nomeados com um verbo ou sintagma verbal (incluindo objeto), por exemplo, `append` ou `drawImage`. Métodos que retornam um valor booleano geralmente têm nomes que começam com a palavra *is* ou, menos com *monly*, têm, seguidos por um substantivo, frase nominal ou qualquer palavra ou frase que funcione como adjetivo, por exemplo, `isDigit`, `isProvablePrime`, `isEmpty`, `isEnabled` ou `hasSiblings`.

Métodos que retornam uma função ou atributo não booleano do objeto em que são invocados geralmente são nomeados com um substantivo, um sintagma nominal ou uma frase verbal que começa com o verbo *get*, por exemplo, `size`, `hashCode` ou `getTime`. Existe um contingente vocal que afirma que apenas a terceira forma (começando por *get*) é aceitável, mas há pouca base para essa afirmação. As duas primeiras formas normalmente levam a um código mais legível, por exemplo:

```
se (car.speed() > 2 * SPEED_LIMIT)
    generateAudibleAlert ("Cuidado com a polícia!");
```

A forma que começa por *get* tem suas raízes na praticamente obsoleta Java Beans specification, que formou a base de uma arquitetura inicial de componentes reutilizáveis. Existem ferramentas modernas que continuam a depender da convenção de nomes Beans, e você pode usá-la em qualquer código que vá ser usado em conjunto com essas ferramentas. Há também um forte precedente para seguir essa convenção de nomenclatura se uma classe contiver tanto um setter quanto um getter para o mesmo atributo. Nesse caso, os dois métodos são tipicamente chamados `getAttribute` e `setAttribute`. Alguns nomes de métodos merecem menção especial. Métodos de instância que convertem o tipo de um objeto, retornando um objeto independente de outro tipo, são frequentemente chamados de *toType*, por exemplo, `toString` ou `toArray`. Métodos que retornam uma visualização (Item 6) cujo tipo difere do objeto receptor são frequentemente chamados

asType, por exemplo, asList. Métodos que retornam uma primitiva com o mesmo valor do objeto no qual são invocados são frequentemente chamados de typeValue, por exemplo, intValue. Nomes comuns para fábricas estáticas incluem from, of, valueOf, instance, getInstance, newInstance, getType e newType (Item 1, página 9).

As convenções gramaticais para nomes de campos são menos estabelecidas e menos importantes do que as para nomes de classes, interfaces e métodos, porque APIs bem projetadas contêm poucos ou nenhum campo exposto. Campos do tipo booleano são frequentemente nomeados como métodos de acesso booleano com a inicial omitida, por exemplo, inicializado, composto. Campos de outros tipos geralmente são nomeados com substantivos ou sintagmas nominais, como altura, dígitos ou corpoEstilo. Convenções gramaticais para variáveis locais são semelhantes às dos corpos, mas ainda mais fracas.

Para resumir, internalize as convenções padrão de nomes e aprenda a usá-las como algo natural. As convenções tipográficas são diretas e em grande parte inequívocas; As convenções gramaticais são mais complexas e flexíveis. Para citar The Java Language Specification [JLS, 6.1], "Essas convenções não devem ser seguidas esservilmente se o uso convencional há muito tempo dita o contrário." Use o bom senso.

CHAPTER 10

Exceções

QUANDO usadas da melhor forma, exceções podem melhorar a legibilidade de um programa, confiabilidade e manutenção. Quando usados de forma inadequada, podem ter o efeito oposto. Este capítulo fornece diretrizes para usar exceções de forma eficaz.

Item 69: Usar exceções apenas para condições excepcionais

Um dia, se você tiver azar, pode acabar encontrando um pedaço de código que se parece com algo assim:

```
Abuso horrível de exceções. Nunca faça isso! tente {  
  
    int i = 0;  
    while(verdadeiro)  
        alcance[i++].subir();  
} catch (ArrayIndexOutOfBoundsException e) {}
```

O que esse código faz? Não é nada óbvio pela inspeção, e isso já é motivo suficiente para não usá-lo (Item 67). Acaba sendo um idiomismo terrivelmente mal concebido para fazer looping entre os elementos de um arranjo. O loop infinito termina ao lançar, capturar e ignorar um `ArrayIndexOutOfBoundsException` quando tenta acessar o primeiro elemento do array fora dos limites do array. Ele deveria ser equivalente ao idioma padrão para looping por um array, que é instantaneamente reconhecível para qualquer programador Java:

```
para (Montanha m : cordilheira)  
    m.climb();
```

Então, por que alguém usaria o loop baseado em exceções em vez do que testava e é verdadeiro? É uma tentativa equivocada de melhorar o desempenho com base no defeito

raciocínio de que, como a VM verifica os limites de todos os acessos ao array, o teste de terminação do `normalloop` — oculto pelo compilador, mas ainda presente no `for-eachloop` — é redundante e deve ser evitado. Há três coisas erradas nesse raciocínio:

- Como as exceções são projetadas para circunstâncias excepcionais, há pouco incentivo para os implementadores da JVM torná-las tão rápidas quanto testes explícitos.
- Colocar código dentro de um bloco `try-catch` inibe certas otimizações que implementações de JVMs poderiam realizar.
- O idioma padrão para looping por um array não necessariamente resulta em verificações inredundantes. Muitas implementações de JVM otimizam esses recursos.

Na verdade, o idioma baseado em exceções é muito mais lento que o padrão. No `mymachine`, o idioma baseado em exceções é cerca de duas vezes mais lento que o padrão de `forarrays` de cem elementos.

Além de o loop baseado em exceções ofuscar o propósito do código e reduzir seu desempenho, também não é garantido que funcione. Se houver um bug no loop, o uso de exceções para controle de fluxo pode mascarar o bug, complicando muito o processo de depuração. Suponha que o cálculo no corpo do loop invoque um método que realiza um acesso fora dos limites a algum arranjo não relacionado. Se um idioma loop razoável fosse usado, o bug geraria uma `uncaughtexception`, resultando na terminação imediata da thread com um rastreo de pilha completa. Se um loop baseado em exceções `themisguided` fosse usado, a exceção relacionada ao bug seria capturada e interpretada erroneamente como uma terminação normal de loop.

A moral dessa história é simples: exceções são, como o nome indica, usadas apenas para condições excepcionais; elas nunca devem ser usadas para fluxo de controle comum. De forma mais geral, use uma preferência padrão e facilmente reconhecível em relação a técnicas excessivamente inteligentes que pretendem oferecer melhor desempenho. Mesmo que a vantagem de desempenho seja real, ela pode não permanecer diante de implementações de plataforma que melhoram constantemente. Os bugs sutis e dores de cabeça de manutenção que vêm de técnicas excessivamente inteligentes, no entanto, certamente permanecerão.

Esse princípio também tem implicações para o design de APIs. Uma API bem projetada não deve forçar seus clientes a usarem exceções para o fluxo de controle comum. Uma classe com um método "dependente do estado" que só pode ser invocado sob certas condições imprevisíveis deve, em geral, ter um método separado de "teste de estado" que indique se é apropriado invocar o método dependente do estado. Por exemplo, a interface `Iterator` tem o método dependente do estado em seguida e o

método correspondente de teste de estado tem `Next`. Isso permite o idioma padrão que foritera sobre uma coleção com um loop for tradicional (assim como o loop `for-each`, onde o método `hasNext` é usado internamente):

```
para (<Foo>Iterador i = collection.iterator(); i.hasNext(); ) {
    Foo foo = i.next();...}
```

Se o `Iterator` não tivesse o método `hasNext`, os clientes seriam obrigados a fazer isso:

```
Não use esse código horrível para iteração sobre uma coleção!try {
    <Foo> Iterator i = collection.iterator();
    while(true) {
        Foo foo = i.next();...} catch
    (NoSuchElementException e) {}
```

Isso deve parecer muito familiar depois do exemplo da iteração de array que iniciou esse item. Além de ser prolixo e enganoso, o loop baseado em exceções provavelmente terá um desempenho ruim e pode mascarar bugs em partes não relacionadas do sistema.

Uma alternativa a fornecer um método separado de teste de estado é que o método dependente do estado retorne um opcional vazio (Item 55) ou um valor distinto—como `null`, caso não possa realizar o cálculo desejado.

Aqui estão algumas orientações para ajudar você a escolher entre um método de teste de estado e um valor de retorno opcional ou distinto. Se um objeto for acessado simultaneamente sem sincronização externa ou estiver sujeito a transições de estado induzidas externamente, você deve usar um valor de retorno opcional ou distinto, pois o estado do objeto pode mudar no intervalo entre a invocação de um método de teste de estado e seu método dependente do estado. Preocupações de desempenho podem ditar que um valor de retorno opcional ou distinto seja usado se um método de teste de estado separado duplique o trabalho do método dependente do estado. Tudo o mais igual, um método de teste de estado é levemente preferível a um valor de retorno distinto. Oferece uma legibilidade um pouco melhor, e o uso incorreto pode ser mais fácil de detectar: se você esquecer de chamar um método de teste de estado, o método dependente do estado lançará uma exceção, tornando o bug óbvio; Se você esquecer de verificar um valor distinto, o bug pode ser sutil. Isso não é um problema para valores de retorno opcionais.

Em resumo, exceções são projetadas para condições excepcionais. Não os use para fluxo de controle comum, e não escreva APIs que obriguem outros a fazer isso.

Item 70: Usar exceções marcadas para condições recuperáveis e Exceções em tempo de execução para erros de programação

O Java oferece três tipos de *throwables*: exceções verificadas, exceções em tempo de execução e erros. Há alguma confusão entre os programadores sobre quando é apropriado usar cada tipo de *throwable*. Embora a decisão nem sempre seja clara, existem algumas regras gerais que fornecem orientações fortes.

A regra fundamental para decidir se usar uma exceção verificada ou não verificada é esta: use exceções verificadas para condições das quais o chamador pode razoavelmente se esperar se recuperar. Ao lançar uma exceção verificada, você força o chamador a lidar com a exceção em uma cláusula *catch* ou a propagá-la para fora. Cada exceção verificada que um método é declarado para lançar é, portanto, uma indicação potente para o usuário da API de que a condição associada é um possível resultado da invocação do método.

Ao confrontar o usuário com uma exceção marcada, o designer da API apresenta um mandato para se recuperar da condição. O usuário pode ignorar o mandato ignorando a exceção, mas isso geralmente é uma má ideia (Item 77).

Existem dois tipos de *throwables* não verificados: exceções em tempo de execução e erros. Eles são idênticos em seu comportamento: ambos são objetos arremessáveis que não precisam, e geralmente não deveriam, ser capturados. Se um programa lança uma exceção ou erro não verificado, geralmente a recuperação é impossível e a execução continuada faria mais mal do que bem. Se um programa não detectar tal lança-objeto, ele fará com que a *thread* atual pare com uma mensagem de erro apropriada.

Use exceções em tempo de execução para indicar erros de programação. A grande maioria das exceções em tempo de execução indica violações de pré-condição. Uma violação de pré-condição é simplesmente a falha do cliente de uma API em aderir ao contrato estabelecido pela especificação da API. Por exemplo, o contrato de acesso ao array especifica que o índice do array deve estar entre zero e o comprimento do array menos um, inclusive. *ArrayIndexOutOfBoundsException* indica que essa pré-condição foi violada.

Um problema com esse conselho é que nem sempre fica claro se você está lidando com uma condição recuperável ou com um erro de programação. Por exemplo, considere o caso de esgotamento de recursos, que pode ser causado por um erro de programação, como a alocação de um array excessivamente grande, ou por uma real escassez de recursos. Se o esgotamento de recursos for causado por uma escassez temporária ou por uma demanda temporariamente elevada, a condição pode ser recuperável. É uma questão de julgamento por parte do projetista da API se uma dada instância de esgotamento de recursos provavelmente permitirá a recuperação. Se você acredita que uma condição pode permitir a recuperação, use uma exceção marcada; se não, use um *runtime*

exceção. Se não estiver claro se a recuperação é possível, provavelmente é melhor usar uma exceção não marcada, pelos motivos discutidos no Item 71.

Embora a Especificação da Linguagem Java não exija isso, há uma forte convenção de que erros são reservados para uso pela JVM para indicar deficiências de recursos, falhas invariantes ou outras condições que tornam impossível continuar a execução. Dada a aceitação quase universal dessa convenção, é melhor não implementar novas subclasses de `Erro`. Portanto, todos os `uncheckedthrowables` que você implementar devem subclassar `RuntimeException` (direta ou indiretamente). Você não só não deveria definir subclasses de `Erro`, mas, com exceção do `ErrorAssertionError`, também não deveria lançá-las.

É possível definir um projetável que não seja uma subclasse de `Exception`, `RuntimeException` ou `Error`. O JLS não aborda tais `throwables` diretamente, mas especifica implicitamente que eles se comportam como exceções verificadas comuns (que são subclasses de `Exception`, mas não de `RuntimeException`). Então, quando você deve usar uma fera assim? Em uma palavra, nunca. Eles não têm benefícios em relação aos `checkedexceptions` comuns e serviriam apenas para confundir o usuário da sua API.

Designers de APIs frequentemente esquecem que exceções são objetos completos nos quais métodos arbitrários podem ser definidos. O uso principal desses métodos é fornecer código que captura a exceção com informações adicionais sobre a condição que causou o lançamento da exceção. Na ausência desses métodos, programadores já foram conhecidos por analisar a representação da `string` de uma exceção para extrair informações adicionais. Isso é uma prática extremamente ruim (Item 12). Classes `throwable` raramente especificam os detalhes de suas representações de `string`, representações `sostring` podem variar de implementação para implementação e de `release` para `release`. Portanto, código que analisa a representação de `string` de uma exceção provavelmente será não portátil e frágil.

Como exceções verificadas geralmente indicam condições recuperáveis, é especialmente importante que forneçam métodos que forneçam informações para ajudar o chamador a se recuperar da condição excepcional. Por exemplo, suponha que uma `checkedexception` seja lançada quando uma tentativa de compra com um cartão-presente falha devido a fundos insuficientes. A exceção deve fornecer um método de acesso para consultar o valor do déficit. Isso permitirá que o usuário repasse o valor para o cliente. Veja o Item 75 para mais informações sobre esse tema.

Para resumir, inclua exceções verificadas para condições recuperáveis e exceções não verificadas para erros de programação. Em caso de dúvida, jogue `uncheckedexceptions`. Não defina `throwables` que não sejam exceções verificadas nem exceções em tempo de execução. Apresente métodos para suas exceções marcadas para ajudar na recuperação.

Item 71: Evitar o uso desnecessário de exceções verificadas

Muitos programadores Java não gostam de exceções verificadas, mas usadas corretamente, podem melhorar APIs e programas. Ao contrário dos códigos de retorno e das exceções não verificadas, eles forçam os programadores a lidar com problemas, aumentando a confiabilidade. Dito isso, o uso excessivo de exceções verificadas em APIs pode torná-las muito menos agradáveis de usar. Se um método lança exceções verificadas, o código que o invoca deve lidar com elas em um ou mais blocos de captura, ou declarar que ele as lança e permitir que se propaguem para fora. De qualquer forma, isso coloca um ônus sobre o usuário da API. O ônus aumentou no Java 8, pois exceções verificadas de métodos não podem ser usados diretamente em fluxos (Itens 45–48).

Esse ônus pode ser justificado se a condição excepcional não puder ser evitada pelo uso adequado da API e o programador que a utiliza pode tomar alguma ação útil ao se deparar com a exceção. A menos que ambas as condições sejam atendidas, uma exceção não verificada é apropriada. Como teste decisivo, pergunte a si mesmo como o programador vai lidar com a exceção. Isso é o melhor que pode ser feito?

```
} catch (TheCheckedException e) {  
    lançar um novo AssertionError(); Não pode  
    acontecer!}
```

Ou isso?

```
} catch (TheCheckedException e) {  
    e.printStackTrace();// Ah, bem, perdemos.  
    System.exit(1);}
```

Se o programador não conseguir fazer melhor, uma exceção não marcada é necessária. O ônus adicional sobre o programador causado por uma exceção verificada é substancialmente maior se for a única exceção verificada lançada por um método. Se houver outros, o método já deve aparecer em um bloco de tentativa, e essa exceção requer, no máximo, outro bloco de captura. Se um método lança uma única `checkedexception`, essa exceção é a única razão pela qual o método deve aparecer em um bloco `try` e não pode ser usado diretamente em streams. Nessas circunstâncias, vale a pena se perguntar se existe uma forma de evitar a exceção verificada.

A maneira mais fácil de eliminar uma exceção marcada é retornar um opcional do tipo de resultado desejado (Item 55). Em vez de lançar uma exceção verificada, o método simplesmente retorna um opcional vazio. A desvantagem dessa técnica é que o método não pode retornar nenhuma informação adicional detalhando sua incapacidade de realizar o cálculo desejado. Exceções, por outro lado, possuem tipos descritivos e podem exportar métodos para fornecer informações adicionais (Item 70).

Você também pode transformar uma exceção verificada em uma exceção não verificada quebrando o método que lança a exceção em dois métodos, sendo o primeiro deles retorna um booleano indicando se a exceção será lançada. Esse APIrefactoring transforma a sequência de chamada a partir disso:

```
Invocação com exceção verificada {  
  
    obj.action(args);} catch  
    (TheCheckedException e) {  
    ... // Lidar com condição excepcional}
```

A isso:

```
Invocação com método de teste de estado e unchecked exceptionif  
(obj.actionPermitted(args)) {  
    obj.action(args);}  
else {  
    ... // Lidar com condição excepcional}
```

Essa refatoração nem sempre é apropriada, mas onde está, pode tornar uma API mais agradável de usar. Embora a segunda sequência de chamadas não seja mais bonita que a primeira, a API refatorada é mais flexível. Se o programador sabe que a chamada terá sucesso, ou está satisfeito em deixar a thread terminar caso ela falhe, a refatoração também permite esta sequência trivial de chamadas:

```
obj.action (args);
```

Se você suspeita que a sequência trivial de chamadas será a norma, então a refatoração API pode ser apropriada. A API resultante é essencialmente a API de método de teste de estado no Item 69 e as mesmas ressalvas se aplicam: se um objeto for acessado simultaneamente sem sincronização externa ou estiver sujeito a transições de estado induzidas externamente, essa refatoração é inadequada porque o estado do objeto pode mudar entre as chamadas para `actionAllowed` e `action`. Se um método `separateActionAllowed` duplicar o trabalho do método de ação, `therefactoring` pode ser descartado por razões de desempenho.

Em resumo, quando usados com moderação, exceções verificadas podem aumentar a confiabilidade dos programas; quando usados em excesso, tornam APIs difíceis de usar. Se os chamadores não conseguirem recuperar de falhas, jogue exceções não verificadas. Se a recuperação for possível e você quiser forçar os chamadores a lidar com condições excepcionais, primeiro considere devolver uma opção opcional. Somente se isso fornecer informações insuficientes em caso de falha você deve lançar uma exceção marcada.

Item 72: Favorecer o uso de exceções padrão

Uma característica que diferencia programadores especialistas dos menos experientes é que os especialistas buscam e geralmente alcançam um alto grau de reutilização do código. Exceções não são exceção à regra de que a reutilização do código é algo bom. As bibliotecas Java fornecem um conjunto de exceções que cobre a maioria das necessidades de arrejção de exceções da maioria das APIs.

Reutilizar exceções padrão traz vários benefícios. A principal delas é que ela torna sua API mais fácil de aprender e usar, pois corresponde às convenções estabelecidas com as quais os programadores já estão familiarizados. Um segundo lugar próximo é que programas que usam sua API são mais fáceis de ler porque não ficam sobrecarregados com exceções desconhecidas. Por último, menos classes de exceção significam menor área de memória e menos tempo gasto carregando classes.

O tipo de exceção mais comumente reutilizado é `IllegalArgumentException` (Item 49). Geralmente, essa é a exceção ao `throw` quando o chamador passa um argumento cujo valor é inadequado. Por exemplo, isso seria a exceção do `throwto` se o chamador passasse um número negativo em um parâmetro que represente o número de vezes que uma ação deveria ser repetida.

Outra exceção comumente reutilizada é a `IllegalStateException`. Isso geralmente é a exceção para o `throw` se a invocação for ilegal devido ao estado do objeto receptor. Por exemplo, isso seria a exceção a ser lançado se o `thecaller` tentasse usar algum objeto antes que ele tivesse sido devidamente inicializado.

Pode-se argumentar que toda invocação de método equivocada se resume a um argumento ou estado ilegal, mas outras exceções são normalmente usadas para certos tipos de argumentos e estados ilegais. Se um chamador passa `null` em algum parâmetro para o qual valores nulos são proibidos, a convenção determina que `NullPointerException` seja lançado em vez de `IllegalArgumentException`. De forma semelhante, se um chamador passa um valor fora de alcance em um parâmetro representando um índice em uma sequência, `IndexOutOfBoundsException` deve ser lançado em vez de `IllegalArgumentException`.

Outra exceção reutilizável é a `ConcurrentModificationException`. Ele deve ser lançado se um objeto projetado para uso por uma única thread (ou com sincronização externa) detectar que está sendo modificado simultaneamente. Essa exceção é, no máximo, uma pista, pois é impossível detectar modificações simultâneas de forma confiável.

Uma última exceção padrão a notar é `UnsupportedOperationException`. Esta é a exceção para `throw` se um objeto não suportar uma operação tentada. O uso deles é raro porque a maioria dos objetos suporta todos os seus métodos. Essa exceção é usada por classes que não implementam uma ou mais operações opcionais definidas por uma interface que implementam. Por exemplo, uma implementação de `Lista` apenas para anexar lançaria essa exceção se alguém tentasse excluir um elemento da lista.

Não reutilize `Exception`, `RuntimeException`, `Throwable` ou `Error` diretamente. Trate essas aulas como se fossem abstratas. Você não pode testar essas exceções de forma confiável porque elas são superclasses de outras exceções que um método pode lançar.

Esta tabela resume as exceções mais comumente reutilizadas:

Exceção	Ocasião de Uso
<code>ExceçãoArgumentoIllegal</code>	Valor de parâmetro não nulo é inadequado
<code>Exceção IllegalStateT</code>	O estado do objeto é inadequado para <code>methodinvocation</code>
<code>NullPointerException</code>	O valor do parâmetro é nulo onde proibido
<code>ExceçãoIndexOutOfBounds</code>	O valor do parâmetro de índice está fora da faixa
<code>ExceçãoModificaçãoConcorrenteT</code>	A modificação simultânea de um objeto foi detectada quando é proibida
<code>OperaçãoExceçãoSem Suporte</code>	<code>Object</code> não suporta método

Embora essas sejam de longe as exceções mais reutilizadas, outras podem ser desenterradas quando as circunstâncias exigem. Por exemplo, seria apropriado reusar `ArithmeticException` e `NumberFormatException` se você estivesse implementando objetos aritméticos como números complexos ou racionais. Se uma exceção atender às suas necessidades, use em frente, mas somente se as condições sob as quais você a colocaria forem consistentes com a documentação da exceção: a reutilização deve ser baseada em semântica documentada, não apenas no nome. Além disso, sintá-se à vontade para subclassificar uma exceção padrão se quiser adicionar mais detalhes (Item 75), mas lembre-se de que exceções são serializáveis (Capítulo 12). Só isso já é motivo para não escrever sua própria aula de exceção sem um bom motivo.

Escolher qual exceção reutilizar pode ser complicado porque as "ocasiões para uso" na tabela acima não parecem ser mutuamente exclusivas. Considere o caso de um objeto representando um baralho de cartas, e suponha que existisse um método para distribuir tudo do baralho que tomasse como argumento o tamanho da mão. Se o caller passasse um valor maior que o número de cartas restantes no baralho, isso poderia ser considerado como um `IllegalArgumentException` (o valor do parâmetro `HandSize` é alto demais) ou um `IllegalStateException` (o baralho contém poucas cartas). Nessas circunstâncias, a regra é lançar `IllegalStateException` se nenhum `argumentvalues` teria funcionado, caso contrário jogar `IllegalArgumentException`.

Item 73: Incluir exceções apropriadas à abstração

É desconcertante quando um método lança uma exceção que não tem aparente ligação com a tarefa que realiza. Isso frequentemente acontece quando um método propaga uma exceção lançada por uma abstração de nível inferior. Além de ser desconcertante, isso também polui a API da camada superior com detalhes de implementação. Se a implementação da camada superior mudar em uma versão posterior, as exceções que ela lança também mudarão, podendo quebrar programas clientes existentes.

Para evitar esse problema, camadas superiores devem capturar exceções de nível inferior e, em seu lugar, lançar exceções que possam ser explicadas em termos da abstração de nível superior. Esse idioma é conhecido como tradução de exceção:

```
Exceptional Translationtry
{
    ... // Use abstração de nível inferior para fazer nossa
    vontade} captura (LowerLevelException e) {
    lançar um novo HigherLevelException(...);}
```

Aqui é um Exemplo de Exceção Tradução Levado de o Classe `AbstractSequentialList`, que é uma implementação esquelética (Item 20) da interface `List`. Neste exemplo, a tradução de exceção é exigida pela especificação do método `get na<E>` interface `List`:

```
/**
 * Retorna o elemento na posição especificada nesta lista.* @throws
 * IndexOutOfBoundsException se o índice estiver fora do intervalo*
 * ({@code índice < 0 || índice >= size()}).*public E get(int index) {

    ListIterator<E> i = listIterator(index);
    try {
        return i.next();} catch
        (NoSuchElementException e) {
        lançar novo IndexOutOfBoundsException("Index: " + index);}}
```

Uma forma especial de tradução de exceções chamada cadeia de exceções é chamada em casos em que a exceção de nível inferior pode ser útil para alguém que está depurando o problema que causou a exceção de nível superior. A exceção de nível inferior (o

causa) é passada para a exceção de nível superior; que fornece um método de acesso (método `getCause` do `Throwable`) para recuperar a exceção de nível inferior:

```
Encadeamento de
exceções {
... // Use abstração de nível inferior para cumprir nosso
pedido} catch (LowerLevelException causa) {
lançar um novo HigherLevelException(causa);}
```

O construtor da exceção de nível superior passa a causa para um construtor de superclasse consciente encadeando, de modo que ela é passada para um dos construtores conscientes de encadeamento do `Throwable`, como `Throwable(Throwable)`:

```
Exceção com constructorclass consciente de
encadeamento HigherLevelException estende Exception {
    HigherLevelException(Causa arremessável) {
        super(causa);}}
```

A maioria das exceções padrão possui construtores conscientes de encadeamento. Para exceções que não fazem, você pode definir a causa usando o método `initCause` do `Throwable`. O encadeamento de exceções não só permite acessar a causa programaticamente (com `getCause`), como também integra o rastreamento da pilha da causa ao da exceção de nível superior.

Embora a tradução de exceção seja superior à propagação automática de exceções de camadas inferiores, ela não deve ser usada em excesso. Sempre que possível, a melhor maneira de lidar com exceções das camadas inferiores é evitá-las, garantindo que métodos de nível inferior tenham sucesso. Às vezes, você pode fazer isso verificando a validade dos parâmetros do método de nível superior antes de passá-los para camadas inferiores.

Se for impossível evitar exceções de camadas inferiores, a melhor opção é que a camada superior trabalhe silenciosamente em torno dessas exceções, isolando o chamador do método de nível superior de problemas de nível inferior. Nessas circunstâncias, pode ser apropriado registrar a exceção usando alguma funcionalidade de registro apropriada, como `java.util.logging`. Isso permite que os programadores investiguem o problema, enquanto isolam o código cliente e os usuários dele.

Em resumo, se não for viável prevenir ou lidar com exceções de camadas inferiores, use a tradução de exceções, a menos que o método de nível inferior garanta que todas as suas exceções sejam adequadas para o nível superior. O encadeamento oferece o melhor dos dois mundos: permite que você faça uma exceção de nível superior apropriada, enquanto captura a causa subjacente da análise de falha (Item 75).

Item 74: Documente todas as exceções lançadas por cada método

Uma descrição das exceções lançadas por um método é uma parte importante da documentação necessária para usar o método corretamente. Portanto, é fundamental que você dedique tempo para documentar cuidadosamente todas as exceções lançadas por cada método (Item 56).

Sempre declare exceções verificadas individualmente e documente com precisão as condições sob as quais cada uma é lançada usando a tag de `@throws` Javadoc. Não use o atalho de declarar que um método lança alguma superclasse de múltiplas classes de exceção que ele pode lançar. Como exemplo extremo, não declare que um método público lança Exceção ou, pior, joga Throwable. Além de negar qualquer orientação ao usuário do método sobre as exceções que ele pode lançar, tal declaração dificulta muito o uso do método porque efetivamente obscurece qualquer outra exceção que possa ser lançada no mesmo contexto. Uma exceção a esse conselho é o método principal, que pode ser declarado com segurança para lançar Exception porque é chamado apenas pela VM.

Embora a linguagem não exija que os programadores declarem as exceções não marcadas que um método pode lançar, é prudente documentá-las tão cuidadosamente quanto as exceções verificadas. Exceções não marcadas geralmente representam erros de programação (Item 70), e familiarizar os programadores com todos os erros que podem cometer ajuda a evitar esses erros. Uma lista bem documentada das exceções não verificadas que um método pode lançar descreve efetivamente as pré-condições para sua execução bem-sucedida. É essencial que a documentação de cada publicmethod descreva suas pré-condições (Item 56), e documentar exceções `unchecked` é a melhor forma de satisfazer esse requisito.

É particularmente importante que os métodos nas interfaces documentem as exceções não verificadas que podem lançar. Essa documentação faz parte do contrato geral da interface e possibilita comportamentos comuns entre múltiplas implementações da interface.

Use a tag Javadoc `@throws` para documentar cada exceção que um método pode lançar, mas não use a palavra-chave `throws` em exceções não verificadas. É importante que os programadores que usam sua API estejam cientes de quais exceções estão verificadas e quais não, pois as responsabilidades dos programadores diferem nesses dois casos. A documentação gerada pelo `@throwstag` Javadoc sem uma cláusula `throws` correspondente na declaração do método fornece uma forte pista visual ao programador de que uma exceção está desmarcada.

Deve-se notar que documentar todas as exceções não verificadas que cada método pode lançar é um ideal, nem sempre alcançável no mundo real. Quando uma classe passa por revisão, não é violação da compatibilidade de origem ou binário se um método exportado for modificado para lançar exceções adicionais não verificadas. Suponha que uma classe invoque um método de outra classe escrita de forma independente. Os autores da primeira classe podem documentar cuidadosamente todas as exceções não verificadas que cada método lança, mas se a segunda classe for revisada para lançar exceções não verificadas adicionais, é bastante provável que a primeira classe (que não passou por revisão) propague as novas exceções não verificadas mesmo que não as documente.

Se uma exceção for lançada por vários métodos em uma turma pelo mesmo motivo, você pode documentar a exceção no comentário de documentação da turma, em vez de documentá-la individualmente para cada método. Um exemplo comum é `NullPointerException`. É aceitável que um comentário de documentação da classe diga: "Todos os métodos desta classe lançam uma `NullPointerException` se uma `objectreference` nula for passada em qualquer parâmetro", ou palavras do tipo.

Em resumo, documente todas as exceções que puderem ser lançadas por cada método que você escrever. Isso vale tanto para exceções não verificadas quanto para métodos abstratos e concretos. Esta documentação deve ser `of@throws` tags nos comentários do documento. Declare cada exceção verificada individualmente na cláusula `throws` do `amethod`, mas não declare exceções não verificadas. Se você não documentar as exceções que seus métodos podem causar, será difícil ou impossível para outros fazerem uso eficaz das suas classes e interfaces.

Item 75: Incluir informações de captura de falhas em mensagens detalhadas

Quando um programa falha devido a uma exceção não capturada, o sistema automaticamente imprime o rastreo de pilha da exceção. O traço de pilha contém a representação de strings da exceção, resultado da invocação do método `toString`. Isso normalmente consiste no nome da classe da exceção seguido de sua mensagem detalhada. Frequentemente, essa é a única informação que programadores ou engenheiros de confiabilidade de site terão ao investigar uma falha de software. Se a falha não for facilmente reproduzível, pode ser difícil ou impossível obter mais informações. Portanto, é de extrema importância que o método `toString` da exceção retorne o máximo de informações possível sobre a causa da falha. Em outras palavras, a mensagem detalhada de uma exceção deve capturar a falha para análises subsequentes.

Para capturar uma falha, a mensagem de detalhe de uma exceção deve conter os valores de todos os parâmetros e campos que contribuíram para a exceção. Por exemplo, a mensagem de detalhe de uma `Exceção IndexOutOfBoundsException` deve conter o limite inferior, o limite superior e o valor do índice que não ficou entre os limites. Essa informação diz muito sobre a falha. Qualquer um ou todos os três valores podem estar errados. O índice pode ser um a menos que o limite inferior ou igual ao limite superior (um "erro de poste de cerca"), ou pode ser um valor curing, muito baixo ou alto. O limite inferior pode ser maior que o limite superior (uma falha interna linvariante grave). Cada uma dessas situações aponta para um problema diferente, e isso ajuda muito no diagnóstico se você souber que tipo de erro está procurando.

Uma ressalva diz respeito a informações sensíveis à segurança. Como rastreamentos de pilha podem ser vistos por muitas pessoas no processo de diagnosticar e corrigir problemas de software, não incluem senhas, chaves de criptografia e mensagens detalhadas do tipo.

Embora seja fundamental incluir todos os dados pertinentes na mensagem detalhada de uma exceção, geralmente não é importante incluir muita prosa. A pilha trace é destinada a ser analisada em conjunto com a documentação e, se necessário, o código-fonte. Geralmente contém o número exato de arquivo e linha de onde a exceção foi lançada, assim como os arquivos e números de linha de todas as outras invocações de método na pilha. Longas descrições em prosa da falha são supérfluas; As informações podem ser obtidas lendo a documentação e o código-fonte.

A mensagem detalhada de uma exceção não deve ser confundida com uma mensagem de erro em nível de usuário, que deve ser inteligível para os usuários finais. Ao contrário de uma mensagem de erro em nível de usuário, a mensagem de detalhe é principalmente para benefício de programadores ou engenheiros de confiabilidade de sites, ao analisar uma falha. Portanto, o conteúdo informativo é muito mais importante do que a legibilidade. Mensagens de erro em nível de usuário geralmente são localizadas, enquanto mensagens de detalhamento de exceção raramente são.

Uma forma de garantir que exceções contenham informações adequadas de captura de falhas em suas mensagens de detalhe é exigir essa informação em seus construtores em vez de uma mensagem de detalhe de string. A mensagem detalhada pode então ser gerada automaticamente para incluir as informações. Por exemplo, em vez de um construtor `String`, `IndexOutOfBoundsException` poderia ter tido um construtor que se parece com isto:

```
/**
 * Constrói um IndexOutOfBoundsException. ** @param lowerBound o menor
 * valor legal do índice* @param upperBound o maior valor legal do
 * índice mais um* @param indexo o valor real do índice*/public
IndexOutOfBoundsException(int lowerBound, int upperBound,

                                int index) {
    Gerar uma mensagem detalhada que capture o
    failuresuper(String.format(
        "Limite inferior: %d, Limite superior: %d, Índice:
        %d", Limite inferior, Limite superior, índice));

    Salve informações de falha para o acesso programático
    this.lowerBound = lowerBound; this.upperBound = upperBound;
    this.index = index;}
```

A partir do Java 9, o `IndexOutOfBoundsException` finalmente adquiriu um construtor que recebe um parâmetro de índice com valor `int`, mas infelizmente omite os parâmetros `lowerBound` e `upperBound`. De modo mais geral, as bibliotecas Java não fazem uso intenso desse idioma, mas é altamente recomendado. Isso facilita para o programador que lança uma exceção capturar a falha. Na verdade, dificulta para o programador não capturar a falha! Na prática, o idioma centraliza o código para gerar uma mensagem de detalhe de alta qualidade na classe de exceção, em vez de exigir que cada usuário da classe gere a mensagem de detalhe de forma redundante.

Como sugerido no Item 70, pode ser apropriado que uma exceção forneça métodos acessórios para suas informações de captura de falhas (`lowerBound`, `upperBound` e `index` no exemplo acima). É mais importante fornecer tais métodos acessórios em exceções verificadas do que não verificadas, pois a informação de captura de falha pode ser útil para a recuperação da falha. É raro (embora nada inconcebível) que um programador queira acesso programático aos detalhes de uma exceção não verificada. Mesmo para exceções não verificadas, no entanto, parece aconselhável fornecer esses acessórios por princípio geral (Item 12, página 57).

Item 76: Buscar a atomicidade do fracasso

Após um objeto lançar uma exceção, geralmente é desejável que o objeto ainda esteja em um estado bem definido e utilizável, mesmo que a falha tenha ocorrido durante a execução de uma operação. Isso é especialmente verdadeiro para exceções verificadas, das quais espera-se que o chamador se recupere. De modo geral, uma invocação de método falhada deve deixar o objeto no estado em que estava antes da invocação. Um método com essa propriedade é dito ser atômico de falha.

Existem várias maneiras de alcançar esse efeito. O mais simples é projetar objetos imutáveis (Item 17). Se um objeto é imutável, a atomicidade da falha é livre. Se uma operação falhar, pode impedir a criação de um novo objeto, mas nunca deixará um objeto existente em estado inconsistente, porque o estado de cada objeto é consistente quando é criado e não pode ser modificado posteriormente.

Para métodos que operam em objetos mutáveis, a forma mais comum de alcançar a atomicidade da falha é verificar parâmetros quanto à validade antes de realizar a operação (Item 49). Isso faz com que a maioria das exceções seja descartada antes do início da modificação de objetos. Por exemplo, considere o método `Stack.pop` no Item 7:

```
Objeto público pop() {  
    if (tamanho == 0)  
        lançar um novo EmptyStackException(); Resultado do objeto =  
        elementos[--tamanho]; elementos[tamanho] = nulo; Eliminar o  
        resultado obsoleto de referência;}
```

Se a verificação inicial de tamanho fosse eliminada, o método ainda lançaria uma exceção ao tentar tirar um elemento de uma pilha vazia. No entanto, isso deixaria o campo `size` em um estado inconsistente (negativo), fazendo com que qualquer invocação de `futureMethod` no objeto falhasse. Além disso, a `ArrayIndexOutOfBoundsException` lançada pelo método `pop` seria inadequada para a abstração (Item 73).

Uma abordagem intimamente relacionada para alcançar a atomicidade da falha é ordenar a composição de modo que qualquer parte que possa falhar ocorra antes de qualquer parte que modifique o objeto. Essa abordagem é uma extensão natural da anterior, quando argumentos não podem ser verificados sem realizar uma parte do cálculo. Por exemplo, considere o caso do `TreeMap`, cujos elementos são ordenados de acordo com alguma ordenação. Para adicionar um elemento a um `TreeMap`, o elemento deve ser de um tipo que possa ser comparado usando a ordem do `TreeMap`. Tentando adicionar um incorretamente

o elemento tipado naturalmente falhará com uma `ClassCastException` como resultado da busca pelo elemento na árvore, antes que a árvore tenha sido modificada de qualquer forma.

Uma terceira abordagem para alcançar a atomicidade da falha é realizar a operação sobre uma cópia temporária do objeto e substituir o conteúdo do objeto pela cópia temporária após a conclusão da operação. Essa abordagem ocorre naturalmente quando o cálculo pode ser realizado mais rapidamente uma vez que os dados tenham sido armazenados em uma estrutura de dados temporária. Por exemplo, algumas funções de ordenação copiam sua lista de entrada para um array antes da ordenação para reduzir o custo de acessar elementos no loop interno da ordenação. Isso é feito para desempenho, mas como benefício adicional, garante que a lista de entrada permaneça intacta caso a ordenação falhe.

Uma última e muito menos comum abordagem para alcançar a atomicidade da falha é escrever código de recuperação que intercepte uma falha que ocorre no meio de uma operação e faça o objeto reverter seu estado para o ponto anterior ao início da operação. Essa abordagem é usada principalmente para estruturas de dados duráveis (baseadas em disco).

Embora a atomicidade da falha seja geralmente desejável, nem sempre é alcançável. Por exemplo, se dois threads tentarem modificar o mesmo objeto simultaneamente sem sincronização adequada, o objeto pode ficar em um estado inconsistente. Portanto, seria errado assumir que um objeto ainda era utilizável após capturar uma `ExceçãoModificação Concorrente`. Erros são irrecuperáveis, então você nem precisa tentar preservar a atomicidade da falha ao lançar o `AssertionError`.

Mesmo quando a atomicidade da falha é possível, nem sempre é desejável. Para algumas operações, isso aumentaria significativamente o custo ou a complexidade. Dito isso, muitas vezes é gratuito e fácil alcançar a atomicidade de falha quando você percebe o problema.

Em resumo, como regra, qualquer exceção gerada que faça parte da especificação de um método deve deixar o objeto no mesmo estado em que estava antes da invocação do método. Quando essa regra for violada, a documentação da API deve indicar claramente em que estado o objeto será deixado. Infelizmente, muita documentação de API existente não corresponde a esse ideal.

Item 77: Não ignore exceções

Embora esse conselho possa parecer óbvio, ele é violado com frequência suficiente para merecer ser repetido. Quando os desenvolvedores de uma API declaram um método para lançar uma exceção, eles estão tentando te dizer algo. Não ignore! É fácil ignorar exceções ao cercar uma invocação de método com uma instrução `try` cujo bloco `catch` está vazio:

```
Bloco vazio de captura ignora exceção - Altamente suspeito!
try {
...} catch (SomeException e)
{}
```

Um bloco de captura vazio anula o propósito das exceções, que é forçar você a lidar com condições excepcionais. Ignorar uma exceção é análogo a ignorar um alarme de incêndio — e desligá-lo para que ninguém mais tenha a chance de ver se há um incêndio real. Você pode se safar, ou os resultados podem ser desastrosos. Sempre que você vê um bloco de captura vazio, os alarmes devem disparar na sua cabeça.

Existem situações em que é apropriado ignorar uma exceção. Para exame-ple, pode ser apropriado ao fechar um `FileInputStream`. Você não alterou o estado do arquivo, então não há necessidade de realizar nenhuma ação de recuperação, e você já leu as informações necessárias do arquivo, então não há motivo para abortar a operação em andamento. Pode ser sensato registrar a exceção, para que você possa investigar o caso essas exceções ocorram com frequência. Se você optar por ignorar uma exceção, o bloco de captura deve conter um comentário explicando por que é apropriado fazê-lo, e a variável deve ser nomeada ignorada:

```
Future<Integer> f = exec.submit(planarMap::chromaticNumber); int
numColors = 4; Padrão; Garantido suficiente para qualquer mapa {

numColors = f.get(1L, TimeUnit.SECONDS);} captura
(TimeoutException | ExecutionException ignorada) {
Use o padrão: coloração mínima é desejável, não obrigatória}
```

As orientações deste item se aplicam igualmente a exceções marcadas e não marcadas. Se uma exceção representa uma condição excepcional previsível ou um erro de programação, ignorá-la com um bloco de captura vazio resultará em um programa que continua silenciosamente diante do erro. O programa pode então falhar em um momento arbitrário no futuro, em um ponto do código que não tenha relação aparente com a origem do problema. Lidar corretamente com uma exceção pode evitar a falha completamente. Apenas deixar uma exceção se propagar para fora pode pelo menos fazer o programa falhar rapidamente, preservando informações para ajudar a depurar a falha.

CHAPTER 11

Concorrência

O THREADS permite que múltiplas atividades ocorram simultaneamente. Pró-concorrente

gramar é mais difícil do que programação single-thread, porque mais coisas podem dar errado, e falhas podem ser difíceis de reproduzir. Você não pode evitar a concorrência. Isso é inerente à plataforma e uma necessidade para obter bom desempenho dos processadores multicore, que agora são onipresentes. Este capítulo contém conselhos para ajudar você a escrever programas concorrentes claros, corretos e bem documentados.

Item 78: Sincronizar o acesso a dados mutáveis compartilhados

A palavra-chave sincronizada garante que apenas uma única thread possa executar um bloco de método ao mesmo tempo. Muitos programadores pensam na sincronização apenas como um meio de exclusão mútua, para evitar que um objeto seja visto em estado inconsistente por um thread enquanto está sendo modificado por outro. Nessa visão, um objeto é criado em um estado consistente (Item 17) e travado pelos métodos que o acessam. Esses métodos observam o estado e, opcionalmente, causam uma transição de estado, transformando o objeto de um estado consistente para outro. O uso adequado da sincronização garante que nenhum método jamais observará o objeto em um estado inconsistente.

Essa visão está correta, mas é só metade da história. Sem sincronização, as mudanças do onethread podem não ser visíveis para outras threads. A *synchronization* não só impede que threads observem um objeto em estado inconsistente, como também garante que cada thread que entra em um método ou bloco sincronizado veja os efeitos de todas as modificações anteriores que foram protegidas pelo mesmo lock.

A especificação da linguagem garante que ler ou escrever uma variável é atômico, a menos que a variável seja do tipo longo ou duplo [JLS, 17.4, 17.7]. Em outras palavras, ler uma variável diferente de uma longa ou dupla garante que retornará um valor que foi armazenado nessa variável por alguma thread, mesmo que múltiplas threads modifiquem a variável simultaneamente e sem sincronização.

Você pode ouvir dizer que, para melhorar o desempenho, é melhor dispensar a sincronização ao ler ou gravar dados atômicos. Esse conselho está perigosamente errado. Embora a especificação da linguagem garanta que uma thread não verá um valor arbitrário ao ler um campo, ela não garante que um valor escrito por uma thread será visível para outra. A sincronização é necessária para comunicação confiável entre threads, bem como para exclusão mútua. Isso se deve a uma parte da especificação da linguagem conhecida como modelo de memória, que especifica quando e como as mudanças feitas por uma thread se tornam visíveis para outras [JLS, 17.4; Goetz06, 16].

As consequências de não sincronizar o acesso a dados mutáveis compartilhados podem ser graves, mesmo que os dados sejam legíveis e graváveis atômicamente. Considere a tarefa de interromper um fio de discussão entre outros. As bibliotecas fornecem o método `Thread.stop`, mas esse método foi descontinuado há muito tempo porque é inerentemente inseguro — seu uso pode resultar em corrupção de dados. Não use o `Thread.stop`. Uma forma recomendada de parar uma thread de outra é fazer com que a primeira thread questione um campo booleano que inicialmente é falso, mas pode ser definido como `true` pela segunda thread para indicar que a primeira thread deve se autoparar. Como ler e escrever um campo booleano é atômico, alguns programadores dispensam a sincronização ao acessar o campo:

```
Quebrado! - Por quanto tempo você espera que esse programa dure
a rodar? classe pública StopThread {
    estática booleana privada stopSolicitado;

    Public static void main(String[] args)
        throws InterruptedException {Thread
        backgroundThread = new Thread(() -> {
            int i = 0; enquanto
                (!stopRequested)
                    i++;
        });
        backgroundThread.start();

    TimeUnit.SECONDS.sleep(1);
    stopRequested = true;}}
```

Você pode esperar que esse programa rode por cerca de um segundo, após o qual a thread principal define o `stopRequested` para `true`, fazendo com que o loop da thread em segundo plano se termine. No meu computador, porém, o programa nunca termina: o thread de fundo fica em loop para sempre!

O problema é que, na ausência de sincronização, não há garantia de quando, se é que acontecer, a thread em segundo plano verá a alteração no valor do `stopRequested` feita pela thread principal. Na ausência de sincronização, é perfeitamente aceitável que a máquina virtual transforme este código:

```
    enquanto (!stopRequested)
        i++;
```

neste código:

```
    if (!stopRequested)
        enquanto (verdadeiro)
            i++;
```

Essa otimização é conhecida como *hoisting*, e é exatamente o que a VM OpenJDKServer faz. O resultado é uma falha de vividade: o programa não consegue produzir prog-ress. Uma forma de resolver o problema é sincronizar o acesso ao campo `stopRequested`. Este programa termina em cerca de um segundo, como esperado:

```
Terminação cooperativa de thread corretamente sincronizada
classe pública StopThread {
    estática booleana privada stopSolicitado;

    private static synchronized void requestStop() {
        stopRequested = true;}

    private static synchronized boolean stopRequested() {
        retornoParaSolicitado;}

    Public static void main(String[] args)
        throws InterruptedException {Thread
        backgroundThread = new Thread(() -> {
            int i = 0; enquanto
            (!stopRequested())
                i++;
        });
        backgroundThread.start();

    TimeUnit.SECONDS.sleep(1);
    requestStop();}}
```

Note que tanto o método de escrita (`requestStop`) quanto o método de leitura (`stopRequested`) são sincronizados. Não basta sincronizar apenas o `writemethod`! A sincronização não tem garantia de funcionar a menos que ambas as operações de leitura e escrita estejam sincronizadas. Ocasionalmente, um programa que sincroniza apenas gravações (ou leituras) pode parecer funcionar em algumas máquinas, mas nesse caso, as aparências enganam.

As ações dos métodos sincronizados no `StopThread` seriam atômicas mesmo sem sincronização. Em outras palavras, a sincronização nesses métodos é usada apenas para seus efeitos de comunicação, não para exclusão mútua. Embora o custo de sincronização em cada iteração do loop seja pequeno, existe uma alternativa correta que é menos prolixa e cujo desempenho provavelmente será melhor. O travamento na segunda versão do `StopThread` pode ser omitido se `stopRequested` for declarado volátil. Embora o modificador volátil não execute exclusão mútua, ele garante que qualquer thread que leia o campo verá o valor mais recentemente escrito:

```
Terminação cooperativa de thread com uma classe StopThread {

    estática privada volátil booleana stopRequested;

    Public static void main(String[] args)
        throws InterruptedException {Thread
        backgroundThread = new Thread(() -> {
            int i = 0; enquanto
            (!stopRequested)
                i++;
        });
        backgroundThread.start();

    TimeUnit.SECONDS.sleep(1);
    stopRequested = true;}}
```

Você precisa ter cuidado ao usar volátil. Considere o método a seguir, que supostamente gera números de série:

```
Quebrado - requer sincronização!private static
volatile int nextSerialNumber = 0;

public static int generateSerialNumber() {
    return nextSerialNumber++;}
```

A intenção do método é garantir que cada invocação retorne um valor único (desde que não haja mais que 232 invocações). Os con-sentes de estado do método de um único corpo atômicamente acessível, `nextSerialNumber`, e todos os valores possíveis desse corpo são legais. Portanto, não é necessária sincronização para proteger seus invariantes. Ainda assim, o método não funciona direito sem sincronização.

O problema é que o operador de incremento (`++`) não é atômico. Ele executa duas operações no campo `nextSerialNumber`: primeiro lê o valor e depois reescreve um novo valor, igual ao valor antigo mais um. Se uma segunda thread lê o campo entre o momento em que uma thread lê o valor antigo e reescreve um novo, a segunda thread verá o mesmo valor da primeira e retornará o mesmo número de série. Isso é uma falha de segurança: o programa calcula os resultados errados.

Uma forma de corrigir `generateSerialNumber` é adicionar o modificador `sincronizado` à sua declaração. Isso garante que múltiplas invocações não serão intercaladas e que cada invocação do método verá os efeitos de todas as invocações anteriores. Depois de fazer isso, você pode e deve remover o modificador volátil do `nextSerialNumber`. Para proteger o método à prova de projeções, use `long` em vez de `int`, ou `throw an exception` se `nextSerialNumber` estiver prestes a finalizar.

Melhor ainda, siga o conselho do Item 59 e use a classe `AtomicLong`, que faz parte do `java.util.concurrent.atomic`. Este pacote fornece primitivas sem bloqueios, programação segura para threads em variáveis simples. Enquanto o volátil fornece apenas os efeitos de comunicação da sincronização, este pacote também proporciona atomicidade. Isso é exatamente o que queremos para o `generateSerialNumber`, e é provável que ele supere a versão sincronizada:

```
Sincronização sem trava com java.util.concurrent.atomicprivate static
final AtomicLong nextSerialNum = new AtomicLong();

public static long generateSerialNumber() {
    return nextSerialNum.getAndIncrement();}
```

A melhor forma de evitar os problemas discutidos neste item é não compartilhar dados mutáveis. Ou compartilhe dados imutáveis (Item 17) ou não compartilhe nada. Em outras palavras, confinar dados mutáveis a um único thread. Se você adotar essa política, é importante documentá-la para que ela seja mantida à medida que seu programa evolui. Também é importante ter um entendimento profundo dos frameworks e bibliotecas que você está usando, pois eles podem introduzir threads que você não conhece.

É aceitável que uma thread modifique um objeto de dados por um tempo e depois o compartilhe com outras threads, sincronizando apenas o ato de compartilhar a objetreferência. Outras threads podem então ler o objeto sem mais sincronização,

desde que não seja modificado novamente. Diz-se que tais objetos são efetivamente imutáveis[Goetz06, 3.5.4]. Transferir tal referência de objeto de um thread para outro é chamado de publicação segura [Goetz06, 3.5.3]. Existem muitas maneiras seguras de publicar referências a objetos: você pode armazená-lo em um campo estático como parte da inicialização da classe; Você pode armazená-lo em um campo volátil, um campo final ou um campo que é acessado com travamento normal; ou você pode colocá-lo em uma coleção simultânea (Item 81).

Em resumo, quando múltiplas threads compartilham dados mutáveis, cada thread que lê ou escreve os dados deve realizar a sincronização. Na ausência de sincronização, não há garantia de que as mudanças de uma thread serão visíveis para outra. As penalidades por não sincronizar dados mutáveis compartilhados são falhas de vividade e segurança. Essas falhas estão entre as mais difíceis de depurar. Elas podem ser intermitentes e dependentes do tempo, e o comportamento dos programas pode variar radicalmente de uma VM para outra. Se você precisar apenas de comunicação entre threads, e não exclusão mútua, o modificador volátil é uma forma aceitável de sincronização, mas pode ser complicado de usar corretamente.

Item 79: Evite sincronização excessiva

O item 78 alerta para os perigos da sincronização insuficiente. Este item diz respeito ao problema oposto. Dependendo da situação, a sincronização excessiva pode causar redução de desempenho, deadlocks ou até mesmo comportamento não determinístico.

Para evitar falhas de vividade e segurança, nunca ceda o controle ao cliente dentro de um método ou bloco sincronizado. Em outras palavras, dentro de uma região sincronizada, não invoque um método projetado para ser sobreposto, nem um fornecido por um cliente na forma de um objeto função (Item 24). Do ponto de vista da classe com a região sincronizada, tais métodos são estranhos. A turma não tem conhecimento do que o método faz e não tem controle sobre isso. Dependendo do que um método alienígena faz, chamá-lo de uma região sincronizada pode causar exceções, bloqueios ou corrupção de dados.

Para tornar isso concreto, considere a seguinte classe, que implementa um conjunto observável de invólucro. Ele permite que os clientes assinem notificações quando elementos são adicionados ao conjunto. Este é o padrão Observer [Gamma95]. Para ser breve, a classe não fornece notificações quando elementos são removidos de theset, mas seria simples fornecê-los. Esta classe é implementada sobre o ForwardingSet reutilizável do Item 18 (página 90):

```
Broken - invoca o método alienígena da classe synchronized
block!public ObservableSet<E> estende o<E> ForwardingSet {
    public ObservableSet<E> set) { super(set); }

    observadores finais privados da Lista<SetObserver<E>>
        = nova ArrayList<>();

    public void addObserver(<E>SetObserver observer) {
        sincronizado(observadores) {
            observers.add(observer);}}

    public boolean removeObserver(<E>SetObserver observer) {
        sincronizado(observadores) {
            return observers.remove(observer);}}

    private void notifyElementAdded(E element) {
        sincronizado(observadores) {
            para (<E>Observador SetObserver : observadores)
                observer.added(this, element);
        }}
}
```

```

@Override publicamente booleano add(E elemento) {
    booleano adicionado =
        super.add(elemento); if (adicionado)
        notifyElementAdded(elemento); retorno
        adicionado;}

@Override booleano público addAll(Collection<? estende E> c) {
    resultado booleano = falso;
    para (elemento E : c)
        resultado |= add(elemento); Chamadas notifyElementAdded
    retorne resultado;}}

```

Observadores subscrevem notificações invocando o método `addObserver` e cancelam a inscrição invocando o método `removeObserver`. Em ambos os casos, uma instância dessa interface de retorno de chamada é passada para o método.

```

@FunctionalInterface interface pública SetObserver<E> {
    Invocado quando um elemento é adicionado ao setvoid observável
    adicionado (conjunto<E> ObservableSet, elemento E);}

```

Essa interface é estruturalmente idêntica ao `BiConsumidor<<E>ObservableSet,E>`. Optamos por definir uma interface funcional personalizada porque a interface e os nomes dos métodos tornam o código mais legível e porque a interface poderia evoluir para incorporar múltiplos callbacks. Dito isso, também poderia ser apresentado um argumento razoável para o uso do `BiConsumidor` (Item 44).

Em uma inspeção superficial, o `ObservableSet` parece funcionar bem. Por exemplo, o programa a seguir imprime os números de 0 a 99:

```

public static void main(String[] args) {
    <Integer> Conjunto observávelSet =
        novo ObservableSet<>(novo HashSet<>());

    set.addObserver((s, e) -> System.out.println(e));

    para (int i = 0; i < 100; i++)
        set.add(i);
}

```

Agora vamos tentar algo um pouco mais sofisticado. Suponha que substituamos o `addObservercall` por um que passa um observador que imprime o valor inteiro que foi adicionado ao conjunto e se remove se o valor for 23:

```
set.addObserver(new SetObserver<>() {
    public void added(ObservableSet<Integer> s, Integer e) {
        System.out.println(e); if (e == 23)

            s.removeObserver(this);

    });
});
```

Note que essa chamada utiliza uma instância de classe anônima no lugar do lambda usado na chamada anterior. Isso ocorre porque o objeto função precisa passar a si mesmo `tos.removeObserver`, e as lambdas não podem se acessar (Item 42).

Você poderia esperar que o programa imprimissem os números de 0 a 23, após o que o observador cancelaria a inscrição e o programa terminaria silenciosamente. Na verdade, ele imprime esses números e depois lança uma `AtualExceçãoModificaçãoConcorrente`. O problema é que `notifyElementAdded` está no processo de iterar sobre a lista de observadores quando invoca o método adicionado do observador. O método adicionado chama o método `removeObserver` do conjunto observável, que por sua vez chama o `methodobservers.remove`. Agora estamos encrocados. Estamos tentando remover um elemento de uma lista enquanto estamos iterando sobre ele, o que é ilegal. A iteração no método `thenotifyElementAdded` está em um bloco sincronizado para evitar a modificação concorrente, mas isso não impede que o próprio thread iterador volte a chamar para o conjunto observável e modifique sua lista de observadores.

Agora vamos tentar algo estranho: vamos escrever um observador que tenta cancelar a inscrição, mas em vez de chamar `removeObserver` diretamente, ele recorre aos serviços de outro thread para fazer o trabalho. Este observador utiliza um serviço executor (Item 80):

```
Observador que usa um thread em segundo plano needlessly.addObserver(new
SetObserver<>() {
    public void added(ObservableSet<Integer> s, Integer e) {
        System.out.println(e); if
        (e == 23) {ExecutorService
        exec =
            Executors.newSingleThreadExecutor(); try {

                exec.submit(() -> s.removeObserver(this)).get();} captura
                (ExecutionException | InterruptedException ex) {
                    lançar um novo
                    AssertionError(ex);} finalmente {

                exec.shutdown();}}}});
```

A propósito, note que este programa captura dois tipos diferentes de exceção na cláusula `oncatch`. Essa funcionalidade, informalmente conhecida como `multi-catch`, foi adicionada em Java 7. It pode aumentar muito a clareza e reduzir o tamanho de programas que se comportam da mesma forma em resposta a múltiplos tipos de exceção.

Quando executamos esse programa, não temos exceção; Temos um impasse. A `thread` em segundo plano chama `s.removeObserver`, que tenta travar observadores, mas não consegue adquirir o bloqueio, porque a `thread` principal já possui o bloqueio. Enquanto isso, o `thread` principal está esperando o `thread` em segundo plano terminar de remover o observador, o que explica o `deadlock`.

Esse exemplo é forçado porque não há razão para o observador usar um `thread` em fundo para se desinscrever, mas o problema é real. Invocar alienmétodos de regiões sincronizadas causou muitos bloqueios em sistemas reais, como kits de ferramentas de interface gráfica.

Em ambos os exemplos anteriores (a exceção e o impasse) tivemos sorte. O recurso protegido pela região sincronizada (observadores) estava em estado consistente quando o método alienígena (adicionado) foi invocado. Suponha que você invocasse um método alienígena de uma região sincronizada enquanto a invari-ant protegida pela região sincronizada estivesse temporariamente inválida. Como os locks na linguagem de programação Java são reentrantes, essas chamadas não travam. Como no primeiro exemplo, que resultou em uma exceção, a `thread` que chama já mantém o bloqueio, então a `thread` terá sucesso quando tentar readquirir o lock, mesmo que outra operação conceitualmente não relacionada esteja em andamento nos dados guardados pelo lock. As consequências de tal falha podem ser catastróficas. Em essência, a eclusa falhou em cumprir seu papel. Bloqueios reentrantes simplificam a construção de programas orientados a objetos multithreaded, mas podem transformar falhas de liveness em falhas de segurança.

Felizmente, geralmente não é tão difícil resolver esse tipo de problema movendo invocações `alienmethod` para fora dos blocos sincronizados. Para o método `notifyElementAdded`, isso envolve tirar um "instantâneo" da lista de observadores que pode então ser percorrido com segurança sem bloqueio. Com essa mudança, ambos os exemplos anteriores rodam sem exceção ou `deadlock`:

```
Método alienígena movido para fora do bloco sincronizado - abrir
chamadasprivate void notifyElementAdded(E element) {
    List<SetObserver<E>> snapshot = null;
    sincronizado(observadores) {
        snapshot = nova ArrayList<>(observers);}para
        (<E>SetObserver observer : snapshot)

        observer.added(this, element);
    }
}
```

Na verdade, existe uma maneira melhor de mover as invocações do método alienígena para fora do bloco sincronizado. As bibliotecas fornecem uma coleção simultânea (Item 81) conhecida como `CopyOnWriteArrayList`, feita sob medida para esse propósito. Esta `ListImplementation` é uma variante do `ArrayList` na qual todas as operações de modificação são implementadas fazendo uma cópia nova de todo o array subjacente. Como o array interno nunca é modificado, a iteração não requer travamento e é muito rápida. Para a maioria dos usos, o desempenho do `CopyOnWriteArrayList` seria péssimo, mas é perfeito para listas de observadores, que raramente são modificadas e frequentemente exploradas.

Os métodos `add` e `addAll` do `ObservableSet` não precisam ser alterados se `thelist` for modificado para usar `CopyOnWriteArrayList`. Aqui está como o restante da turma está. Note que não há nenhuma sincronização explícita:

```
Conjunto observável seguro por threads com
CopyOnWriteArrayListprivate final List<SetObserver<E>> observadores =
    novo CopyOnWriteArrayList<>();

public void addObserver(<E>SetObserver observer) {
    observers.add(observer);}

public boolean removeObserver(<E>SetObserver observer) {
    return observers.remove(observer);}

private void notifyElementAdded(E element) {
    para (<E>Observador SetObserver : observadores)
        observer.added(this, element);
}
```

Um método alienígena invocado fora de uma região sincronizada é conhecido como chamada aberta [Goetz06, 10.1.4]. Além de evitar falhas, as chamadas abertas podem aumentar muito a concorrência. Um método alienígena pode durar um período arbitrariamente longo. Se o método alienígena fosse invocado de uma região sincronizada, outros threads seriam desnecessariamente negados ao recurso protegido.

Como regra, você deve fazer o mínimo de trabalho possível dentro de regiões sincronizadas. Obtenha o cadeado, examine os dados compartilhados, transforme-o conforme necessário e solte o cadeado. Se precisar realizar alguma atividade demorada, encontre uma forma de movê-la para fora da região sincronizada sem violar as diretrizes do Item 78.

A primeira parte deste ponto era sobre correção. Agora vamos dar uma breve olhada no desempenho. Embora o custo da sincronização tenha despencado desde os primórdios do Java, é mais importante do que nunca não sincronizar demais. Em um mundo multicore, o custo real da sincronização excessiva não é o tempo gasto da CPU obtendo locks; é a contenção: as oportunidades perdidas para o paralelismo e os atrasos

imposta pela necessidade de garantir que cada núcleo tenha uma visão consistente da memória. Outro custo oculto da supersincronização é que ela pode limitar a capacidade da VM de otimizar a execução do código.

Se você está escrevendo uma classe mutável, tem duas opções: pode omitir a `allsynchronization` e permitir que o cliente sincronize externamente se o uso concorrente for desejável, ou pode sincronizar internamente, tornando a classe segura para threads (Item 82). Você só deve escolher a segunda opção se conseguir concorrência significativamente maior com sincronização interna do que teria fazendo o cliente bloquear todo o objeto externamente. As coleções em `java.util` (com exceção dos obsoletos `Vector` e `Hashtable`) adotam a primeira abordagem, enquanto as `injava.util.concurrent` adotam a segunda (Item 81).

Nos primeiros dias do Java, muitas aulas violavam essas diretrizes. Por exemplo, instâncias `StringBuffer` quase sempre são usadas por um único thread, mas elas realizam sincronização interna. É por essa razão que o `StringBuffer` foi substituído pelo `StringBuilder`, que é apenas um `StringBuffer` não sincronizado. Da mesma forma, é uma grande parte da razão pela qual o gerador de números pseudoaleatórios seguro para threads em `java.util.Random` foi substituído pela implementação não sincronizada em `java.util.concurrent.ThreadLocalRandom`. Em caso de dúvida, não sincronize sua aula, mas anote que ela não é segura para threads.

Se você sincronizar sua classe internamente, pode usar várias técnicas para alcançar alta concorrência, como divisão de bloqueios, *striping* de bloqueio e controle de concorrência sem bloqueio. Essas técnicas estão além do escopo deste livro, mas são discutidas em outros lugares [Goetz06, Herlihy08].

Se um método modificar um campo estático e houver qualquer possibilidade de que o método seja chamado de múltiplas threads, você deve sincronizar o acesso ao campo internamente (a menos que a classe tolere comportamento não determinístico). Não é possível para um cliente multithread realizar sincronização externa em tal método, pois clientes não relacionados podem invocar o método sem sincronização. O campo é essencialmente uma variável global, mesmo que seja privado, porque pode ser lido e modificado por clientes não relacionados. O campo `nextSerialNumber` usado pelo método `generateSerialNumber` no Item 78 exemplifica essa situação.

Em resumo, para evitar impasse e corrupção de dados, nunca chame um método alienígena de dentro de uma região sincronizada. Mais geralmente, mantenha ao mínimo a quantidade de trabalho que você faz dentro de regiões sincronizadas. Quando você estiver projetando uma classe mutável, pense se ela deveria fazer sua própria sincronização. Na era multicore, é mais importante do que nunca não sincronizar demais. Sincronize sua aula internamente somente se houver um bom motivo para isso, e documente sua decisão claramente (Item 82).

Item 80: Preferir executores, tarefas e fluxos a threads

A primeira edição deste livro continha código para uma fila de trabalho simples [Bloch01,Item 49]. Essa classe permitia que os clientes enfileirassem trabalho para processamento assíncrono por um thread em segundo plano. Quando a fila de trabalho não era mais necessária, o cliente podia invocar um método para pedir que o thread em segundo plano se encerrasse de forma graciosa após concluir qualquer trabalho já na fila. A implementação era pouco mais que um brinquedo, mas mesmo assim, exigia uma página inteira de código sutil e delicado, do tipo que pode causar falhas de segurança e vividade se você não acertar. Felizmente, não há mais motivo para escrever esse tipo de código.

Quando a segunda edição deste livro foi lançada, `java.util.concurrent` já havia sido adicionado ao Java. Este pacote contém um `Executor Framework`, que é uma ferramenta flexível de execução de tarefas baseada em interfaces. Criar uma fila de trabalho que seja melhor em todos os aspectos do que a da primeira edição deste livro requer apenas uma linha de código:

```
ExecutorService exec = Executors.newSingleThreadExecutor();
```

Veja como enviar um executável para execução:

```
exec.execute(executável);
```

E aqui está como dizer ao executor para terminar de forma elegante (se você não fizer isso, é provável que sua VM não saia):

```
exec.shutdown();
```

Você pode fazer muito mais coisas com um serviço de executor. Por exemplo, você pode esperar que uma tarefa específica seja concluída (com o método `get`, como mostrado no Item 79, página 319), pode esperar que qualquer ou toda uma coleção de tarefas seja concluída (usando os métodos `invokeAny` ou `invokeAll`), pode esperar o serviço executor terminar (usando o método `awaitTermination`), pode recuperar os resultados das tarefas um a um conforme elas são concluídas (usando um `ExecutorCompletionService`), você pode agendar tarefas para serem executadas em um horário específico ou periodicamente (usando um `ScheduledThreadPoolExecutor`), e assim por diante.

Se você quiser que mais de uma thread processe solicitações da fila, basta chamar uma fábrica estática diferente que crie um tipo diferente de serviço executor chamado pool de threads. Você pode criar um pool de threads com um número fixo ou variável de threads. A classe `java.util.concurrent.executors` contém fábricas estáticas que fornecem a maioria dos executores que você vai precisar. Se, no entanto, você quiser um pouco-

algo fora do comum, você pode usar diretamente a classe `ThreadPoolExecutor`. Essa classe permite configurar quase todos os aspectos da operação de um pool de threads. Escolher o serviço executor para uma aplicação específica pode ser complicado. Para um programa pequeno, ou um servidor pouco carregado, `Executors.newCachedThreadPool` geralmente é uma boa escolha porque não exige configuração e geralmente "faz a coisa certa". Mas um pool de threads em cache não é uma boa escolha para um servidor de produção muito carregado! Em um pool de threads em cache, as tarefas enviadas não são enfileiradas, mas imediatamente encaminhadas para execução a uma thread. Se não houver threads disponíveis, uma nova é criada. Se um servidor estiver tão carregado que todas as suas CPUs sejam totalmente utilizadas e mais tarefas chegarem, mais threads serão criados, o que só piorará a situação. Portanto, em um servidor de produção muito carregado, é muito melhor usar o `Executors.newFixedThreadPool`, que oferece um pool com um número fixo de threads, ou usar diretamente a classe `ThreadPoolExecutor` para controle máximo.

Você não deve apenas evitar escrever suas próprias filas de trabalho, mas também deve geralmente evitar trabalhar diretamente com threads. Quando você trabalha diretamente com threads, uma Thread serve tanto como unidade de trabalho quanto como mecanismo para executá-la. No framework executor, a unidade de trabalho e o mecanismo de execução são separados. A abstração chave é a unidade de trabalho, que é a tarefa. Existem dois tipos de tarefas: `Runnable` e seu primo próximo, `Callable` (que é como `Runnable`, exceto que retorna um valor e pode lançar exceções arbitrárias). O mecanismo geral para executar tarefas é o serviço executor. Se você pensar em termos de tarefas e deixar um serviço executor executá-las para você, ganha flexibilidade para selecionar uma política de execução adequada para atender às suas necessidades e para mudar a política caso suas necessidades mudem. Em essência, o Executor Framework executa o que o Collections Framework fez para agregação.

No Java 7, o Executor Framework foi estendido para suportar tarefas fork-join, que são executadas por um tipo especial de serviço executor conhecido como fork-join pool. A tarefa `Afork-join`, representada por uma instância `ForkJoinTask`, pode ser dividida em subtarefas menores, e as threads que compõem um `ForkJoinPool` não apenas processam essas tarefas, mas "roubem" tarefas umas das outras para garantir que todas as threads permaneçam ocupadas, resultando em maior utilização da CPU, maior taxa de transferência e menor latência. Escrever e usar tarefas de encaixe no diapasão é complicado. Fluxos paralelos (Item 48) são escritos sobre pools de fork join e permitem que você aproveite seus benefícios de desempenho com pouco esforço, assumindo que sejam adequados para a tarefa em questão.

Um tratamento completo do Executor Framework está além do escopo deste livro, mas o leitor interessado é direcionado para *Java Concurrency in Practice*[Goetz06].

Item 81: Prefira que as utilidades de concorrência aguardem e notifiquem

A primeira edição deste livro dedicou um item ao uso correto de esperar e notificar [Bloch01, Item 50]. Seu conselho ainda é válido e é resumido ao final deste item, mas esse conselho é muito menos importante do que era antes. Isso porque há muito menos motivos para usar esperar e notificar. Desde o Java 5, a plataforma oferece utilitários de concorrência de nível mais alto que fazem o tipo de coisa que antes você precisava programar manualmente para esperar e notificar. Dada a dificuldade de usar `wait` e `notify` correct, você deveria usar as utilidades de concorrência de nível mais alto.

As utilidades de nível mais alto em `java.util.concurrent` se dividem em três categorias: o `Executor Framework`, que foi brevemente abordado no Item 80; `concurrentcollections`; e sincronizadores. Coleções e sincronizadores simultâneos são brevemente abordados neste item.

As coleções concorrentes são implementações concorrentes de alto desempenho de interfaces padrão de coleção, como `Lista`, `Fila` e `Mapa`. Para fornecer alta concorrência, essas implementações gerenciam sua própria sincronização internamente (Item 79). Portanto, é impossível excluir a atividade concorrente da coleta concorrente; Travar só vai desacelerar o programa.

Como você não pode excluir atividade concorrente em coleções concorrentes, também não pode compor invocações de métodos atômicamente sobre elas. Portanto, interfaces de coleta concorrente foram equipadas com operações de modificação dependentes do estado, que combinam várias primitivas em uma única operação atômica. Essas operações se mostraram suficientemente úteis em coleções concorrentes para serem adicionadas às interfaces correspondentes de coleta no Java 8, usando métodos padrão (Item 21).

Por exemplo, o método `putIfAbsent(key, value)` do `Map` insere um mapeamento para uma chave se nenhuma estivesse presente e retorna o valor anterior associado à chave, ou `null` se não houvesse nenhuma. Isso facilita a implementação de mapas de identificação canônica com segurança em threads. Esse método simula o comportamento do `String.intern`:

```
Mapa de canonização concorrente sobre o ConcurrentHashMap - não ótimo
mapa estático final do ConcurrentHashMap<String, String> =
    novo ConcurrentHashMap<>();

estática pública String intern(Strings s) {
    String previousValue = map.putIfAbsent(s, s); return
    previousValue == null ? s : previousValue;}

```

Na verdade, você pode fazer ainda melhor. O `ConcurrentHashMap` é otimizado para operações de recuperação, como `get`. Portanto, vale a pena invocar `get` inicialmente e `callingputIfAbsent` apenas se `get` indicar que é necessário:

```

Mapa de canonização concorrente no topo do ConcurrentMap -
faster!public static String intern(Strings s) {
    String result = map.get(s);
    if (resultado == null) {
        resultado = map.putIfAbsent(s,
            s); if (resultado == null)
            resultado = s;
    }return result;}

```

Além de oferecer excelente concorrência, o `ConcurrentHashMap` é muito rápido. No `mymachine`, o método `intern` acima é mais de seis vezes mais rápido que o `String.intern` (mas lembre-se de que o `String.intern` precisa empregar alguma estratégia para evitar vazamentos de memória em uma aplicação de longa duração). Coleções simultâneas tornam as coleções sincronizadas praticamente obsoletas. Por exemplo, use `ConcurrentHashMap` em preferência a `Collections.synchronizedMap`. Simplesmente substituir mapas sincronizados por mapas concorrentes pode aumentar dramaticamente o desempenho de aplicações concorrentes.

Algumas das interfaces de coleta foram estendidas com operações de bloqueio, que aguardam (ou bloqueiam) até que possam ser realizadas com sucesso. Por exemplo, `BlockingQueue` estende `Queue` e adiciona vários métodos, incluindo `take`, que remove e retorna o elemento cabeça da fila, esperando se a fila estiver vazia. Isso permite que filas de bloqueio sejam usadas para filas de trabalho (também conhecidas como filas produtor-consumidor), para as quais uma ou mais threads produtoras enfileiram itens de trabalho e a partir das quais uma ou mais threads de consumo fazem a enqueue e processam itens conforme ficam disponíveis. Como era de se esperar, a maioria das implementações do `ExecutorService`, incluindo o `ThreadPoolExecutor`, usa um `BlockingQueue` (Item 80).

Sincronizadores são objetos que permitem que threads esperem uns pelos outros, permitindo que eles coordenem suas atividades. Os sincronizadores mais usados são `CountDownLatch` e `Semaphore`. Menos comumente usados são `CyclicBarrier` e `Exchanger`. O sincronizador mais poderoso é o `Phaser`.

Travas de contagem regressiva são barreiras de uso único que permitem que uma ou mais roscas aguardem que uma ou mais outras fíos façam algo. O único construtor para `CountDownLatch` recebe um `int`, que é o número de vezes que o método `countDown` deve ser invocado no latch antes que todos os threads em espera possam prosseguir.

É surpreendentemente fácil construir coisas úteis sobre essa primitiva simples. Por exemplo, suponha que você queira construir uma estrutura simples para cronometrar a execução concorrente de uma ação. Esse framework consiste em um único método que utiliza um executor para executar a ação, um nível de concorrência representando o número de ações a serem executadas simultaneamente, e um executável representando a ação. Todos

As threads de trabalho se preparam para executar a ação antes que a thread temporizadora inicie o relógio. Quando a última thread de trabalho está pronta para executar a ação, a thread temporizadora "dispara a arma de partida", permitindo que as threads de trabalho realizem a ação. Assim que a última thread worker termina de executar a ação, o timer thread para o relógio. Implementar essa lógica diretamente sobre esperar e notificar seria confuso, para dizer o mínimo, mas é surpreendentemente simples sobre o `CountDownLatch`:

```
Framework simples para temporizar execução concorrente estática
de longo tempo (executor executor, int concurrency,
    Ação executável) lança InterruptedException {
    CountDownLatch pronto = novo CountDownLatch (concorrência);
    Início do CountDownLatch = novo CountDownLatch(1);
    CountDownLatch concluído= novo CountDownLatch (concorrência);

    para (int i = 0; i < concorrência; i++) {
        executor.execute() -> {
            ready.countDown(); Avise ao temporizador que estamos prontos,
            tente {
                start.await(); Espere até que os pares estejam
                readyaction.run();} catch (InterruptedException e)
                {
                    Thread.currentThread().interrupt();} finalmente {

                        feito.contagem      Avise ao temporizador que terminamos
                        regressiva();

                    }});}

    pronto.await();      Espere todos os trabalhadores estarem prontos
    início longo Nanos = System.nanoTime(); começar.countDown();
    E eles estão prontos!feito.await();// Espere todos os
    trabalhadores terminarem de voltar System.nanoTime() -
    startNanos;}
```

Note que o método usa três travas de contagem regressiva. A primeira, pronta, é usada pelas threads de trabalhadores para avisar a thread temporizadora quando ela está pronta. A rosca de trabalhador então espera o segundo trinco, que é o início. Quando a última thread worker invoca `ready.countDown`, a thread temporizadora registra o horário de início e invoca `start.countDown`, permitindo que todas as threads worker prossigam. Então o timerthread espera no terceiro latch, pronto, até que o último dos threads de trabalho termine de executar a ação e chame `done.countDown`. Assim que isso acontece, o fio do tempo desperta e registra o tempo final.

Mais alguns detalhes merecem ser mencionados. O executor passado para o método de tempo deve permitir a criação de pelo menos tantas threads quanto o nível de concorrência dado, ou o teste nunca será concluído. Isso é conhecido como *impassé* por falta de threads[Goetz06, 8.1.1]. Se um thread worker captura uma `InterruptedException`, ele reafirma a interrupção usando o idioma `Thread.currentThread().interrupt()` e retorna do método de execução. Isso permite que o executor lide com as interrupções que achar adequadas. Note que o `System.nanoTime` é usado para cronometrar a atividade. Para intervalocrometria, sempre use `System.nanoTime` em vez de

`System.currentTimeMillis`. `System.nanoTime` é tanto mais preciso quanto mais preciso e não é afetado por ajustes no relógio em tempo real do sistema. Por fim, note que o código neste exemplo não vai fornecer tempos precisos a menos que a ação faça bastante trabalho, digamos um segundo ou mais. O *microbenchmarking* preciso é notoriamente difícil e é melhor feito com o auxílio de um framework especializado como o `jmh` [JMH].

Esse item só arranha a superfície do que você pode fazer com os utilitários de concorrência. Por exemplo, as três travas de contagem regressiva no exemplo anterior poderiam ser substituídas por uma única instância `CyclicBarrier` ou `Phaser`. O código resultante seria um pouco mais conciso, mas talvez mais difícil de entender.

Embora você deva sempre usar as utilidades de concorrência em vez de esperar e notificar, talvez precise manter código legado que usa `wait` e notificar. O método de espera é usado para fazer uma thread esperar por alguma condição. Ele deve ser invocado dentro de uma região sincronizada que trava o objeto no qual é invocado. Aqui está o idioma padrão para usar o método de espera:

```
O idioma padrão para usar o método wait
synchronized (obj) {
    enquanto (<condição não se confirma>)
    obj.wait(); (Solta a trava e recupera ao acordar)... // Realize ação
    apropriada à condição}
```

Sempre use o idioma do loop de espera para invocar o método de espera; Nunca a invoque fora de um ciclo. O loop serve para testar a condição antes e depois da espera.

Testar a condição antes de esperar e pular a espera se a condição já estiver em condições são necessários para garantir a vivacidade. Se a condição já for válida e o método `notify` (ou `notifyAll`) já foi invocado antes que uma thread espere, não há garantia de que a thread acordará da espera.

Testar a condição após esperar e esperar novamente caso a condição não se mantenha é necessário para garantir a segurança. Se a thread prosseguir com a ação quando a condição não for válida, ela pode destruir o invariante guardado pelo bloqueio. Existem várias razões pelas quais um tópico pode despertar quando a condição não se mantém:

- Outra thread poderia ter obtido o lock e alterado o estado guardado entre o momento em que uma thread invocou o notify e a thread em espera acordou.
- Outro tópico poderia ter invocado notificação acidentalmente ou maliciosamente quando a condição não se cumpriu. As classes se expõem a esse tipo de travessura esperando por objetos acessíveis ao público. Qualquer espera em um método sincronizado de um objeto acessível publicamente é suscetível a esse problema.
- A thread notificadora pode ser excessivamente "generosa" ao acordar threads em espera. Por exemplo, a thread notificadora pode invocar notifyAll mesmo que apenas algumas das threads em espera tenham sua condição satisfeita.
- O fio de espera podia (raramente) acordar na ausência de notificação. Isso é conhecido como despertar espúrio [POSIX, 11.4.3.6.1; Java9-api].

Uma questão relacionada é se deve usar notify ou notifyAll para acordar threads em espera. (Recall that notify desperta uma única thread aguardando, assumindo que tal thread exista, e notifyAll ativa todas as threads aguardando.) Às vezes dizem que você deve sempre usar o notifyAll. Esse é um conselho razoável e conservador. Sempre vai dar resultados corretos porque garante que você vai despertar os fios que precisam ser despertados. Você pode ativar outros tópicos também, mas isso não afetará a correção do seu programa. Esses tópicos verificam a condição pela qual estão esperando e, ao encontrá-la falsa, continuam esperando.

Como otimização, você pode optar por invocar notify em vez de notifyAllif todas as threads que poderiam estar no wait-set estão aguardando a mesma condição e apenas uma thread por vez pode se beneficiar da condição se tornar verdadeira.

Mesmo que essas pré-condições sejam satisfeitas, pode haver motivo para usar notifyAllin no lugar de notify. Assim como colocar a invocação de espera em um loop protege contra notificações acidentais ou maliciosas em um objeto acessível publicamente, usar notifyAll no lugar de notify protege contra esperas acidentais ou maliciosas por um tópico não relacionado. Tais esperas poderiam, de outra forma, "engolir" uma notificação crítica, deixando seu destinatário pretendido esperando indefinidamente.

Em resumo, usar wait e notify diretamente é como programar em "assembly language concur-rency", em comparação com a linguagem de nível mais alto fornecida por java.util.concurrent. Raramente, se é que há alguma razão, para usar wait and notify em um novo código. Se você mantém código que usa wait e notify, certifique-se de que ele sempre invoque wait dentro de um loop while usando o idioma padrão. O método notifyAll deve geralmente ser usado em vez de notificar. Se o notify for utilizado, é preciso muito cuidado para garantir a vivacidade.

Item 82: Segurança da linha de fiação do documento

Como uma classe se comporta quando seus métodos são usados simultaneamente é uma parte importante de seu contrato com seus clientes. Se você não documentar esse aspecto do comportamento de uma classe, seus usuários serão forçados a fazer suposições. Se essas suposições estiverem erradas, o programa resultante pode realizar sincronização insuficiente (Item 78) ou sincronização excessiva (Item 79). Em ambos os casos, podem ocorrer erros graves.

Você pode ouvir dizer que é possível saber se um método é seguro para threads procurando o modificador sincronizado na documentação dele. Isso está errado em vários aspectos. Em operação normal, o Javadoc não inclui o modificador sincronizado em sua saída, e com razão. A presença do modificador sincronizado na declaração de um método é um detalhe de implementação, não parte de sua API. Isso não indica de forma confiável que um método seja seguro para rosca.

Além disso, a afirmação de que a presença do modificador sincronizado é suficiente para a segurança de threads em documentos incorpora o equívoco de que a segurança de threads é uma propriedade de tudo ou nada. Na verdade, existem vários níveis de segurança com rosca. Para permitir o uso concorrente seguro, uma classe deve documentar claramente qual nível de segurança de threads suporta. A lista a seguir resume os níveis de segurança da rosca. Não é exaustiva, mas cobre os casos comuns:

- **Imutável** — Instâncias dessa classe parecem constantes. Não é necessária sincronização externa. Exemplos incluem `String`, `Long` e `BigInteger` (item 17).
- **Incondicionalmente seguro para threads** — Instâncias desta classe são mutáveis, mas a classe possui sincronização interna suficiente para que suas instâncias possam ser usadas simultaneamente sem a necessidade de qualquer sincronização externa. Exemplos incluem `AtomicLong` e `ConcurrentHashMap`.
- **Condicionalmente seguro contra threads** — Assim como incondicionalmente seguro contra threads, exceto que alguns métodos exigem sincronização externa para uso concorrente seguro. Exemplos incluem as coleções retornadas pelos `Collections.synchronizedWrappers`, cujos iteradores requerem sincronização externa.
- **Não seguro para threads** — Instâncias dessa classe são mutáveis. Para usá-los simultaneamente, os clientes devem cercar cada invocação de método (ou sequência de invocação) com sincronização externa escolhida pelos clientes. Exemplos incluem implementações de coleções de uso geral, como `ArrayList` e `HashMap`.
- **Hostil a threads** — Esta classe é insegura para uso concorrente, mesmo que toda invocação de método esteja cercada por sincronização externa. A hostilidade de threads geralmente resulta da modificação de dados estáticos sem sincronização. Ninguém escreve um

classe hostil à thread de propósito; tais classes normalmente resultam da falha em considerar a concorrência. Quando uma classe ou método é considerado hostil à thread, normalmente é corrigido ou obsoleto. O método `generateSerialNumber` no Item 78 seria hostil à thread na ausência de sincronização interna, conforme discutido na página 322.

Essas categorias (exceto hostis a threads) correspondem aproximadamente às anotações de segurança de threads no *Java Concurrency in Practice*, que são `Immutable`, `ThreadSafe` e `NotThreadSafe` [Goetz06, Apêndice A]. As categorias incondicionalmente e condicionalmente seguras contra thread na taxonomia acima estão ambas cobertas pela anotação `ThreadSafe`.

Documentar uma classe condicionalmente segura para threads exige cuidado. Você deve indicar quais sequências de invocação requerem sincronização externa e quais locks (ou, em casos raros, locks) devem ser adquiridos para executar essas sequências. Normalmente é o bloqueio da própria instância, mas há exceções. Por exemplo, a documentação do `Collections.synchronizedMap` diz o seguinte:

É imprescindível que o usuário sincronize manualmente no mapa retornado ao iterar sobre qualquer uma de suas visualizações de coleção:

```
Map<K, V> m = Collections.synchronizedMap(novo HashMap<>());
Defina<K> s = m.keySet();// Não precisa estar no bloco sincronizado
...
sincronizado(m) {    Sincronizando em m, não em s!
    para (tecla K : s)
        key.f();
}
```

A falha em seguir esse conselho pode resultar em comportamento não determinístico.

A descrição da segurança de thread de uma classe geralmente pertence ao `doccomment`, mas métodos com propriedades especiais de segurança de thread devem descrever essas propriedades em seus próprios comentários de documentação. Não é necessário documentar a imutabilidade dos tipos de `enum`. A menos que seja óbvio pelo tipo de retorno, as fábricas de estática devem documentar a segurança da thread do objeto retornado, como demonstrado pelo `Collections.synchronizedMap` (acima).

Quando uma classe se compromete a usar um lock acessível publicamente, ela permite que os clientes executem uma sequência de invocações de métodos atômica, mas essa flexibilidade tem um preço. É incompatível com o controle interno de concorrência de alto desempenho, do tipo usado por coleções concorrentes como o `ConcurrentHashMap`. Além disso, um cliente pode realizar um ataque de negação de serviço mantendo o bloqueio publicamente acessível por um período prolongado. Isso pode ser feito acidentalmente ou intencionalmente.

Para evitar esse ataque de negação de serviço, você pode usar um objeto de bloqueio privado em vez de usar métodos sincronizados (que implicam um bloqueio acessível publicamente):

Idioma de objeto de bloqueio privado - impede ataque de negação de serviço, bloqueio final de objeto privado = novo Objeto();

```
empty público foo() {  
    sincronizado(lock) {  
        ...}}}
```

Como o objeto de bloqueio privado é inacessível fora da classe, é impossível para os clientes interferirem na sincronização do objeto. Na prática, estamos aplicando o conselho do Item 15 ao encapsular o objeto de bloqueio no objeto que ele sincroniza.

Note que o campo de bloqueio é declarado final. Isso impede que você altere indiretamente seu conteúdo, o que pode resultar em acesso não sincronizado catastrófico (Item 78). Estamos aplicando o conselho do Item 17, minimizando a multibildade do campo de bloqueio. Campos de bloqueio devem sempre ser declarados finais. Isso é verdade tanto que você use um lock de monitor comum (como mostrado acima) quanto um lock do pacote `java.util.concurrent.locks`.

O idioma de objeto de lock privado só pode ser usado em `thread-safe` classes incondicionalmente. Classes condicionalmente `thread-safe` não podem usar esse idioma porque devem documentar qual bloqueio seus clientes devem adquirir ao executar certas sequências de invocação de métodos.

O idioma de objeto de bloqueio privado é particularmente adequado para classes projetadas para herança (Item 19). Se tal classe usasse suas instâncias para travamento, a subclasse poderia facilmente e involuntariamente interferir na operação da classe base, ou vice-versa. Ao usar a mesma trava para propósitos diferentes, a subclasse e a classe base podem acabar "pisando nos pés uns dos outros." Isso não é apenas um problema atóxico; isso aconteceu com a classe `Thread` [Bloch05, Quebra-cabeça 77].

Para resumir, cada classe deve documentar claramente suas propriedades de segurança de threads com uma descrição em prosa cuidadosamente redigida ou uma anotação de segurança de threads. O modificador `sincronizado` não tem papel nesta documentação. Classes condicionalmente seguras para threads devem documentar quais sequências de invocação de métodos requerem sincronização externa e quais locks devem ser adquiridos ao executar essas sequências. Se você escreve uma classe incondicionalmente `thread-safe`, considere usar um objeto de bloqueio privado no lugar dos métodos sincronizados. Isso protege contra interferência de sincronização por clientes e subclasses e oferece mais flexibilidade para adotar uma abordagem sofisticada de controle de concorrência em uma versão posterior.

Item 83: Use a inicialização preguiçosa com critério

Inicialização preguiçosa é o ato de atrasar a inicialização de um campo até que seu valor seja necessário. Se o valor nunca for necessário, o campo nunca é inicializado. Essa técnica é aplicável tanto a campos estáticos quanto a campos de instância. Embora a inicialização preguiçosa seja otimização primária, ela também pode ser usada para quebrar circularidades prejudiciais na inicialização de classes e instâncias [Bloch05, Quebra-cabeça 51].

Como acontece com a maioria das otimizações, o melhor conselho para inicialização preguiçosa é "não faça isso a menos que seja necessário" (Item 67). Inicialização preguiçosa é uma faca de dois gumes. Isso diminui o custo de inicializar uma classe ou criar uma instância, à custa de aumentar o custo de acesso ao campo inicializado de forma preguiçosa. Dependendo de qual fração desses campos eventualmente precisa de inicialização, quão caro é inicializá-los e com que frequência cada um é acessado após a inicialização, a inicialização preguiçosa pode (como muitas "otimizações") na verdade prejudicar o desempenho.

Dito isso, a inicialização preguiçosa tem suas utilidades. Se um campo for acessado apenas em uma fração das instâncias de uma classe e for custoso inicializar o campo, então a inicialização preguiçosa pode valer a pena. A única forma de saber com certeza é medir o desempenho da classe com e sem inicialização preguiçosa.

Na presença de múltiplas threads, inicialização preguiçosa é complicada. Se dois ou mais threads compartilham um campo inicializado de forma preguiçosa, é fundamental que algum tipo de sincronização seja empregada, ou podem surgir bugs graves (Item 78). Todas as técnicas de inicialização discutidas neste item são seguras para threads.

Na maioria das circunstâncias, a inicialização normal é preferível à inicialização preguiçosa. Aqui está uma declaração típica para um campo de instância normalmente inicializado. Note o uso do modificador final (Item 17):

```
Inicialização normal de uma instância fieldType  
final FieldType = computeFieldValue();
```

Se você usar inicialização preguiçosa para quebrar uma circularidade de inicialização, use um acessor sincronizado porque é a alternativa mais simples e clara:

```
Inicialização preguiçosa do campo da instância - campo FieldType  
sincronizado do acessorprivate;
```

```
private synchronized FieldType getField() {  
    if (campo == null)  
        campo = computarValorCampo(); return  
    field;} 
```

Ambos esses idiomas (inicialização normal e inicialização preguiçosa com acessor sincronizado) permanecem inalterados quando aplicados a campos estáticos, exceto que você adiciona o modificador estático às declarações do campo e do acessor.

Se você precisar usar inicialização preguiçosa para desempenho em um campo estático, use o idioma da classe holder da inicialização preguiçosa. Esse idioma explora a garantia de que uma classe não será inicializada até ser usada [JLS, 12.4.1]. É assim que fica:

```
Idioma de classe holder de inicialização preguiçosa para campos estáticos classe
estática privada FieldHolder {
    campo final estático = computeFieldValue();

    private static FieldType getField() { return FieldHolder.field; }
```

Quando `getField` é invocado pela primeira vez, ele lê `FieldHolder.field` pela primeira vez, causando a inicialização da classe `FieldHolder`. A beleza desse idioma é que o método `getField` não é sincronizado e executa apenas o acesso a `field`, então uma inicialização preguiçosa praticamente não acrescenta nada ao custo do acesso. VM atípica sincroniza o acesso ao campo apenas para inicializar a classe. Uma vez inicializada a classis, a VM atualiza o código para que o acesso subsequente ao campo não envolva testes ou sincronização.

Se você precisar usar inicialização preguiçosa para desempenho em um campo de instância, use o idioma de verificação dupla. Esse idioma evita o custo de travamento ao acessar o campo após a inicialização (Item 79). A ideia por trás do ditado é verificar o valor do campo duas vezes (daí o nome *double-check*): uma vez sem travar e, se o campo parecer não inicializado, uma segunda vez com bloqueio. Somente se a segunda verificação indicar que o campo está não inicializado é que a chamada inicializa o campo. Como não há travamento após a inicialização do campo, é fundamental que o campo seja declarado volátil (Item 78). Aqui está o ditado:

```
Verificar duas vezes o idioma para inicialização preguiçosa de campos privados volátil
de CampoCampo;

private FieldType getField() {
    FieldType resultado = campo; if (resultado == nulo) { // Primeira verificação (sem
        sincronizado(isto) { // travamento)
            if (campo == null) Segunda checagem (com trava)
                campo = resultado = computeFieldValue();
        }}return result;}
```

Esse código pode parecer um pouco complicado. Em particular, a necessidade da localvariável (resultado) pode não ser clara. O que essa variável faz é garantir que o campo seja lido apenas uma vez no caso comum em que já está inicializado. Embora não seja estritamente necessário, isso pode melhorar o desempenho e é mais elegante pelos *standards* aplicados à programação concorrente de baixo nível. Na minha máquina, o método acima é cerca de 1,4 vezes mais rápido que a versão óbvia sem variável local.

Embora você possa aplicar o idioma de verificação dupla também a campos estáticos, não há motivo para isso: o idioma preguiçoso da classe holder de inicialização é uma escolha melhor.

Duas variantes do idioma *double check* merecem ser mencionadas. Ocasionalmente, você pode precisar inicializar preguiçosamente um campo de instância que tolere inicializações repetidas. Se você se encontrar nessa situação, pode usar uma variante do idioma de verificação dupla que dispensa a segunda verificação. Não surpreendentemente, é conhecido como o idioma do cheque único. Aqui está como ela se apresenta. Note que o campo ainda é declarado volátil:

Idioma de verificação única - pode causar inicialização repetida do campo `FieldType` volátil privado;

```
private FieldType getField() {
    FieldType resultado = campo;
    if (resultado == null)
        campo = resultado = computeFieldValue(); retorne
        resultado;}
```

Todas as técnicas de inicialização discutidas neste item se aplicam tanto a campos primitivos quanto a campos de referência de objetos. Quando o *double-check* ou *single-checkidiom* é aplicado a um campo primitivo numérico, o valor do campo é verificado contra 0 (o valor padrão para variáveis primitivas numéricas) em vez de null.

Se você não se importar se cada thread recalcula o valor de um campo, e o tipo do campo é um primitivo diferente de longo ou duplo, então pode optar por remover o modificador volátil da declaração de campo no *single-checkidiom*. Essa variante é conhecida como o estilo atrevido de cheque único. Ele acelera o acesso ao campo em algumas arquiteturas, ao custo de inicializações adicionais (até um thread que acessa o campo). Definitivamente é uma técnica exótica, não para uso cotidiano.

Resumindo, você deve inicializar a maioria dos campos normalmente, não de forma preguiçosa. Se você precisa inicializar um campo preguiçosamente para alcançar seus objetivos de desempenho ou para quebrar a circularidade de inicialização prejudicial, então use a técnica adequada de inicialização preguiçosa. Para campos de exemplo, é o idioma de verificação dupla; para campos estáticos, idioma da classe holder de inicialização preguiçosa. Por exemplo, campos que toleram inicialização repetida, você também pode considerar o idioma de verificação única.

Item 84: Não dependa do agendador de threads

Quando muitas threads são executáveis, o agendador de threads determina quais podem rodar e por quanto tempo. Qualquer sistema operacional razoável tentará fazer essa determinação de forma justa, mas a política pode variar. Portanto, programas bem escritos não devem depender dos detalhes dessa política. Qualquer programa que dependa do agendador de threads para correção ou desempenho provavelmente será não portátil.

A melhor maneira de escrever um programa robusto, responsivo e portátil é garantir que o número médio de threads executáveis não seja significativamente maior que o número de processadores. Isso deixa o agendador de threads com pouca escolha: ele simplesmente executa as threads executáveis até que elas não possam mais ser executáveis. O comportamento do programa não varia muito, mesmo sob políticas radicalmente diferentes de agendamento de threads. Note que o número de threads executáveis não é o mesmo que o total de threads, que pode ser muito maior. Threads que estão esperando não são executáveis.

A principal técnica para manter o número de threads executáveis baixo é fazer cada thread fazer um trabalho útil e depois esperar por mais. Threads não devem rodar se não estiverem fazendo um trabalho útil. Em termos do Executor Framework (Item 80), isso significa dimensionar os pools de threads adequadamente [Goetz06, 8.2] e manter as tarefas curtas, mas não muito curtas, ou despachar sobrecarga prejudicará o desempenho.

Threads não devem ficar ocupados, verificando repetidamente um objeto compartilhado esperando que seu estado mude. Além de tornar o programa vulnerável às variações do agendador de threads, a espera ocupada aumenta muito a carga no processador, reduzindo a quantidade de trabalho útil que outros podem realizar. Como um exemplo extremo do que não fazer, considere esta reimplementação perversa do `CountDownLatch`:

```
Implementação horrível do CountDownLatch - ocupada-espera
incessantemente! classe pública SlowCountDownLatch {
    contagem de inteligência privada;

    public SlowCountDownLatch(int count) {
        se (conte < 0)
            lançar um novo IllegalArgumentException(count + " < 0");
        this.count = count;
    }

    Await() {
        enquanto (verdadeiro) {
            sincronizado(isto) {
                if (contagem == 0)
                    retorno;
            }
        }
    }
}
```

```
    contagem regressiva sincronizada do vazio público() {  
        se (conte != 0)  
            conte--;  
    }}
```

Na minha máquina, o `SlowCountDownLatch` é cerca de dez vezes mais lento que o `CountDownLatch` do Java quando 1.000 threads esperam por um latch. Embora este exemplo possa parecer um pouco improvável, não é incomum ver sistemas com um ou mais threads que são desnecessariamente executáveis. Desempenho e portabilidade provavelmente vão sofrer.

Quando você se depara com um programa que mal funciona porque algumas threads não têm tempo suficiente de CPU em relação a outras, resista à tentação de "consertar" o programa fazendo chamadas para o `Thread.yield`. Você pode conseguir fazer o programa funcionar de certa forma, mas não será portátil. As mesmas `yield` invocações que melhoram o desempenho em uma implementação de JVM podem piorar em uma segunda e não ter efeito em uma terceira. `Thread.yield` não possui `testable semantics`. Uma ação melhor é reestruturar a aplicação para reduzir o número de threads executáveis simultaneamente.

Uma técnica relacionada, à qual ressalvas semelhantes se aplicam, é ajustar os laços *priori* de fio. Prioridades de threads estão entre os recursos menos portáteis do Java. Não é irracional ajustar a capacidade de resposta de uma aplicação ajustando algumas prioridades de threads, mas raramente é necessário e não é portátil. É irrazoável tentar resolver um problema sério de vivacidade ajustando as prioridades das threads. O problema provavelmente vai voltar até que você encontre e conserte a causa subjacente.

Em resumo, não dependa do agendador de threads para a correção do seu programa. O programa resultante não será nem robusto nem portátil. Como corolário, não confie em prioridades de `Thread.yield` ou threads. Essas facilidades são apenas pistas para o agendador. Prioridades de threads podem ser usadas com moderação para melhorar a qualidade do serviço de um programa já em funcionamento, mas nunca devem ser usadas para "consertar" um programa que mal funciona.

Esta página intencionalmente deixada em branco

CHAPTER 12

Serialização

ESTE capítulo trata da serialização de objetos, que é o framework do Java para codificando objetos como fluxos de bytes (serializando) e reconstruindo objetos a partir de suas codificações (desserializando). Uma vez que um objeto foi serializado, sua codificação pode ser enviada de uma VM para outra ou armazenada em disco para posterior desserialização. Este capítulo foca nos perigos da serialização e em como minimizá-los.

Item 85: Prefiro alternativas à serialização em Java

Quando a serialização foi adicionada ao Java em 1997, era conhecida por ser um tanto arriscada. A abordagem havia sido tentada em uma linguagem de pesquisa (Modula-3), mas nunca em linguagem de produção. Embora a promessa de objetos distribuídos com pouco esforço por parte do programador fosse atraente, o preço era construtores invisíveis e linhas borradas entre API e implementação, com potencial para problemas de correção, desempenho, segurança e manutenção. Os defensores acreditavam que os benefícios superavam os riscos, mas a história mostrou o contrário.

As questões de segurança descritas em edições anteriores deste livro acabaram sendo tão sérias quanto alguns temiam. As vulnerabilidades discutidas no início dos anos 2000 foram transformadas em explorações sérias na década seguinte, incluindo um ataque de ransomware à San Francisco Metropolitan Transit AgencyMunicipal Railway (SFMTA Muni), que desligou todo o sistema de cobrança de tarifas por dois dias em novembro de 2016 [Gallagher16].

Um problema fundamental da serialização é que sua superfície de ataque é grande demais para ser protegida e está crescendo constantemente: grafos de objetos são desserializados invocando o método `readObject` em um `ObjectInputStream`. Esse método é essencialmente um construtor mágico que pode ser feito para instanciar objetos de quase qualquer tipo no caminho da classe, desde que o tipo implemente a interface `Serializable`. No processo de desserializar um fluxo de bytes, esse método pode executar código de qualquer um desses tipos, então o código para todos esses tipos faz parte da superfície de ataque.

A superfície de ataque inclui classes nas bibliotecas da plataforma Java, em bibliotecas de terceiros como as Coleções Apache Commons e na própria aplicação. Mesmo que você siga todas as melhores práticas relevantes e tenha sucesso em escrever serializáveis classes que são invulneráveis a ataques, sua aplicação ainda pode ser vulnerável. Citando Robert Seacord, gerente técnico do Centro de Coordenação CERT:

A desserialização Java é um perigo claro e presente, pois é amplamente usada tanto diretamente por aplicações quanto indiretamente por subsistemas Java como RMI (Remote Method Invocation), JMX (Java Management Extension) e JMS (Java Messaging System). A desserialização de fluxos não confiáveis pode resultar em execução remota de código (RCE), negação de serviço (DoS) e uma série de outras explorações. As aplicações podem ser vulneráveis a esses ataques se não fizeram nada errado. [Seacord17] Atacantes e pesquisadores de segurança estudam os tipos serializáveis nas bibliotecas Java e em bibliotecas de terceiros comumente usadas, buscando métodos invocados durante a desserialização que realizem atividades potencialmente perigosas. Esses métodos são conhecidos como gadgets. Múltiplos gadgets podem ser usados em conjunto para formar uma cadeia de gadgets. De tempos em tempos, descobre-se uma cadeia de gadgets suficientemente poderosa para permitir que um atacante execute código nativo arbitrário no hardware subjacente, tendo apenas a oportunidade de submeter um fluxo de bytes cuidadosamente elaborado para desserialização. Foi exatamente isso que aconteceu no ataque à SFMTA Muni. Esse ataque não foi isolado. Houve outros, e haverá mais.

Sem usar nenhum dispositivo, você pode facilmente montar um ataque de negação de serviço causando a desserialização de um fluxo curto que leva muito tempo para desserializar. Esses fluxos são conhecidos como bombas de desserialização [Svoboda16]. Aqui está um exemplo de Wouter Coekaerts que usa apenas conjuntos de hash e uma string [Coekaerts15]:

```
Bomba de desserialização - desserializar esse fluxo leva uma
eternidade byte[] bomba() {
    Set<Object> root = novo HashSet<>();
    Defina<Object> s1 = raiz; Set<Object>
    s2 = novo HashSet<>(); para (int i =
    0; i < 100; i++) {
        Set<Object> t1 = novo HashSet<>(); Defina<Object> t2 = novo
        HashSet<>(); t1.add("foo"); Torne t1 igual a t2s1.add(t1);
        s1.ADD(t2); s2.add(t1); s2.add(t2); s1 = t1; s2 = t2;}retorne
        serialize(raiz); Método omitido por brevidade}
```

O grafo de objetos consiste em 201 instâncias `HashSet`, cada uma contendo 3 ou menos referências a objetos. O fluxo inteiro tem 5.744 bytes de comprimento, mas o sol se apagaria muito antes de você conseguir desserializá-lo. O problema é que desserializar uma instância `HashSet` requer o cálculo dos códigos hash de seus elementos. Os 2 elementos do conjunto raiz do hash são eles mesmos conjuntos de hash contendo 2 elementos do conjunto de hash, cada um contendo 2 elementos do conjunto de hash, e assim por diante, com 100 níveis de profundidade. Portanto, a desserialização do conjunto faz com que o método `hashCode` seja invocado mais de 2100 vezes. Além do fato de que a desserialização está demorando uma eternidade, o desserializador não indica que algo esteja errado. Poucos objetos são produzidos, e a profundidade da pilha é limitada.

Então, o que você pode fazer para se defender desses problemas? Você se expõe a ataques sempre que desserializa um fluxo de bytes em que não confia. A melhor forma de evitar exploits de serialização é nunca desserializar nada. Nas palavras do computador chamado Joshua no filme *WarGames* de 1983, "o único movimento vencedor é não jogar." Não há motivo para usar serialização em Java em nenhum novo sistema que você escreva. Existem outros mecanismos para traduzir entre objetos e sequências de bytes que evitam muitos dos perigos da serialização em Java, ao mesmo tempo em que oferecem inúmeras vantagens, como suporte multiplataforma, alto desempenho, um grande ecossistema de ferramentas e uma ampla comunidade de especialização. Neste livro, referimo-nos a esses mecanismos como representações de dados estruturados multiplataforma. Enquanto outros às vezes os chamam de sistemas de serialização, este livro evita esse uso para evitar confusão com a serialização em Java.

O que essas representações têm em comum é que são muito mais simples do que a serialização em Java. Eles não suportam serialização automática e desserialização de grafos de objeto arbitrários. Em vez disso, eles suportam objetos de dados simples e estruturados, consistindo em uma coleção de pares atributo-valor. Apenas alguns tipos primitivos e `arraydata` são suportados. Essa abstração simples se mostra suficiente para construir sistemas distribuídos extremamente poderosos e simples o bastante para evitar os sérios problemas que têm atormentado a serialização Java desde seu início.

As principais representações de dados estruturados multiplataforma são JSON [`JSON`] e Protocol Buffers, também conhecidos como `protobuf` [`Protobuf`]. O JSON foi desenvolvido por Douglas Crockford para comunicação navegador-servidor, e buffers de protocolo foram projetados pelo Google para armazenar e trocar dados estruturados entre seus servidores. Embora essas representações às vezes sejam chamadas de neutras em linguagem, JSON foi originalmente desenvolvido para JavaScript e `protobuf` para C++; ambas as repetições mantêm vestígios de suas origens.

As diferenças mais significativas entre JSON e `protobuf` são que JSON é baseado em texto e legível por humanos, enquanto `protobuf` é binário e substancialmente mais

eficiente; e que JSON é exclusivamente uma representação de dados, enquanto protobuf oferece esquemas (tipos) para documentar e impor o uso apropriado. Embora o protobuf seja mais eficiente que o JSON, o JSON é extremamente eficiente para uma representação baseada em texto. E embora protobuf seja uma representação binária, ele oferece uma representação alternativa do texto para uso onde a legibilidade humana é desejada (pbtxt).

Se você não consegue evitar totalmente a serialização em Java, talvez porque está trabalhando no contexto de um sistema legado que exige isso, sua próxima melhor alternativa é nunca desserializar dados não confiáveis. Em particular, você nunca deve aceitar tráfego RMI de fontes não confiáveis. As diretrizes oficiais de codificação segura para Java dizem: "Desserialização de dados não confiáveis é inerentemente perigosa e deve ser evitada." Esta frase está em letra grande, negrito, itálico e vermelho, e é o único texto em todo o documento que recebe esse tratamento [Java-secure].

Se você não consegue evitar a serialização e não tem absoluta certeza da segurança dos dados que está desserializando, use o filtro de desserialização de objetos adicionado em Java 9 e reportado para versões anteriores (`java.io.ObjectInputFilter`). Essa facilidade permite que você especifique um filtro aplicado a fluxos de dados antes de eles serem re-serIALIZADOS. Ele opera na granularidade da classe, permitindo que você aceite ou rejeite certas classes. Aceitar classes por padrão e rejeitar uma lista de perigosos potencialmente perigosos é conhecido como lista negra; Rejeitar classes por padrão e aceitar uma lista daquelas que são consideradas seguras é conhecido como whitelisting. Prefira a lista branca a lista negra, pois a lista negra só te protege contra ameaças conhecidas. Uma ferramenta chamada Serial Whitelist Application Trainer (SWAT) pode ser usada para preparar automaticamente uma lista branca para sua aplicação [Schneider16]. A função de filtragem também vai te proteger contra o uso excessivo de memória e grafos de objetos excessivamente profundos, mas não vai te proteger contra bombas de serialização como a mostrada acima.

Infelizmente, a serialização ainda é onnipresente no ecossistema Java. Se você mantém um sistema baseado em serialização em Java, considere seriamente migrar para uma representação de dados estruturados multiplataforma, mesmo que isso possa ser um trabalho demorado. Realisticamente, você ainda pode se ver tendo que escrever ou manter uma classe serializável. É necessário muito cuidado para escrever uma classe serializável que seja correta, segura e eficiente. O restante deste capítulo oferece conselhos sobre quando e como fazer isso.

Em resumo, a serialização é perigosa e deve ser evitada. Se você está projetando um sistema do zero, use uma representação de dados estruturados multiplataforma, como JSON ou protobuf. Não desserialize dados não confiáveis. Se for necessário, use filtragem de desserialização de objetos, mas esteja ciente de que não é garantido para impedir todos os ataques. Evite escrever classes serializáveis. Se for preciso, tenha muito cuidado.

Item 86: Implemente o Serializable com muita cautela

Permitir que as instâncias de uma classe sejam serializadas pode ser tão simples quanto adicionar o `implements Serializable` à sua declaração. Como isso é tão fácil de fazer, havia um equívoco comum de que a serialização exige pouco esforço por parte do programador. A verdade é muito mais complexa. Embora o custo imediato para tornar uma classe serializável possa ser negligenciável, os custos de longo prazo costumam ser substanciais.

Um grande custo de implementar o `Serializable` é que ele diminui a flexibilidade para alterar a implementação de uma classe depois de lançada. Quando uma classe implementa o `Serializable`, sua codificação em fluxo de bytes (ou forma serializada) passa a fazer parte da API exportada. Depois de distribuir amplamente uma classe, geralmente é obrigado a suportar a forma serializada para sempre, assim como é obrigado a suportar todas as outras partes da API exportada. Se você não se esforçar para projetar um formulário serializado personalizado, mas apenas aceitar o padrão, o formulário serializado ficará para sempre atrelado à representação interna original da classe. Em outras palavras, se você aceita a forma serializada padrão, os campos privado e do pacote-instância privada da classe passam a fazer parte da API exportada, e a prática de minimizar o acesso a campos (Item 15) perde sua eficácia como ferramenta para ocultação de informações.

Se você aceitar a forma serializada padrão e depois alterar a representação interna de uma classe, ocorrerá uma alteração incompatível na forma serializada. Clientes que tentam serializar uma instância usando uma versão antiga da classe e desserializá-la usando a nova (ou vice-versa) terão falhas no programa. É possível alterar a representação interna mantendo a forma serializada original (usando `ObjectOutputStream.putFields` e `ObjectInputStream.readFields`), mas isso pode ser difícil e deixar falhas visíveis no código-fonte. Se você optar por tornar uma classe serializável, deve cuidadosamente projetar uma forma serializada de alta qualidade com a qual esteja disposto a conviver a longo prazo (Itens 87, 90). Fazer isso vai aumentar o custo inicial do desenvolvimento, mas vale a pena o esforço. Mesmo uma forma serializada bem projetada impõe restrições à evolução de uma classe; uma forma serializada mal projetada pode ser devastadora.

Um exemplo simples das restrições à evolução impostas pela serializabilidade diz respeito a identificadores únicos de fluxo, mais comumente conhecidos como `versionUIDs` seriais. Cada classe serializável tem um número de identificação único associado a ela. Se você não especificar esse número declarando um campo longo final estático chamado `serialVersionUID`, o sistema o gera automaticamente em tempo de execução aplicando uma função hash criptográfica (SHA-1) à estrutura da classe. Esse valor é afetado pelos nomes da classe, pelas interfaces que ela implementa e pela maioria de seus membros, incluindo membros sintéticos gerados pelo compilador. Se você mudar

qualquer uma dessas coisas, por exemplo, ao adicionar um método de conveniência, o UID da versão gerada em série muda. Se você não declarar um UID de versão serial, a compatibilidade será quebrada, resultando em uma `InvalidClassException` em tempo de execução.

Um segundo custo de implementar o `Serializable` é que ele aumenta a probabilidade de bugs e falhas de segurança (Item 85). Normalmente, objetos são criados com construtores; A serialização é um mecanismo extralinguístico para criar objetos. Quer você aceite o comportamento padrão ou o sobrescreva, a desserialização é um "hiddenconstructor" com todos os mesmos problemas que outros construtores. Como não há um construtor explícito associado à desserialização, é fácil esquecer que você deve garantir que ele garanta todos os invariantes estabelecidos pelos construtores e que não permita que um atacante tenha acesso aos internos do objeto sob construção. Confiar no mecanismo padrão de desserialização pode facilmente deixar objetos expostos a corrupção invariante e acesso ilegal (Item 88).

Um terceiro custo de implementar o `Serializable` é que ele aumenta a carga de testes associada ao lançamento de uma nova versão de uma classe. Quando uma `serializableclass` é revisada, é importante verificar se é possível serializar uma instância na nova versão e desserializá-la em versões antigas, e vice-versa. A quantidade de testes exigidos é, portanto, proporcional ao produto do número de classes serializáveis e do número de lançamentos, que pode ser grande. Você deve garantir tanto que o processo de serialização-desserialização seja bem-sucedido quanto que resulte em uma réplica fiel do objeto original. A necessidade de testes é reduzida se uma `serializedform` personalizada for cuidadosamente projetada quando a classe é escrita pela primeira vez (Itens 87, 90).

Implementar o `Serializable` não é uma decisão a ser tomada levemente. É essencial se uma classe quiser participar de um framework que depende da serialização Java para transmissão ou persistência de objetos. Além disso, facilita muito o uso de uma classe como componente em outra classe que deve implementar o `Serializable`. No entanto, existem muitos custos associados à implementação do `Serializable`. Cada vez que você projetar uma aula, pese os custos em relação aos benefícios.

Historicamente, classes de valor eram como `BigInteger` e `Instant` implementaram `Serializable`, e classes de coleção também. Classes que representam entidades ativas, como pools de threads, raramente devem implementar o `Serializable`.

Classes projetadas para herança (Item 19) raramente devem implementar `Serializable`, e interfaces raramente devem estendê-lo. Violar essa regra impõe um ônus substancial a qualquer pessoa que estenda a aula ou implemente a interface. Há momentos em que é apropriado violar a regra. Por exemplo, se uma classe ou interface existe principalmente para participar de um framework que exige que todos os participantes implementem o `Serializable`, então pode fazer sentido que a classe ou interface implemente ou estenda o `Serializable`.

Classes projetadas para herança que implementam `Serializable` incluem: `Throwable` e `Component`. O `Throwable` implementa o `Serializable` para que o RMI possa enviar exceções do servidor para o cliente. Implementa componentes SoGUIs serializáveis podem ser enviadas, salvas e restauradas, mas mesmo na época áurea do Swing e AWT, essa funcionalidade era pouco usada na prática.

Se você implementar uma classe com campos de instância que seja tanto serializável quanto extensível, há vários riscos a serem conscientes. Se houver invariantes nos valores do campo da instância, é fundamental evitar que subclasses sobreponham o método `finalize`, o que a classe pode fazer sobrescrevendo `finalize` e declarando-o. Caso contrário, a classe ficará suscetível a ataques de finalizador (Item 8). Finalmente, se a classe tiver invariantes que seriam violados se seus campos de instância fossem inicializados para seus valores padrão (zero para tipos integrais, false para booleanos e null para tipos de referência de objetos), você deve adicionar este método `readObjectNoData`:

```
readObjectNoData para classes extensíveis extensíveis e serializáveis com  
estado, void privado, readObjectNoData() lança InvalidObjectException {  
    lançar um novo InvalidObjectException("Dados de fluxo  
    necessários");}
```

Esse método foi adicionado no Java 4 para cobrir um caso de canto envolvendo a adição de uma superclasse serializável a uma classe serializável existente [Serialization, 3.5].

Há uma ressalva sobre a decisão de não implementar o `Serializable`. Se uma classe projetada para herança não for serializável, pode exigir esforço extra para escrever uma subclasse serializável. A desserialização normal de tal classe exige que a superclasse tenha um construtor sem parâmetros acessível [Serialização, 1.10]. Se você não fornecer tal construtor, as subclasses são forçadas a usar o padrão `serializationproxy` (Item 90).

Classes internas (Item 24) não devem implementar o `Serializable`. Eles utilizam campos sintéticos gerados por compiladores para armazenar referências a instâncias envolventes e para armazenar valores de variáveis locais de escopos envolventes. Como esses campos correspondem à definição de classe não é especificado, assim como os nomes das classes anônimas e locais. Portanto, a forma serializada padrão de uma classe interna é indefinida. Uma classe membro estática pode, no entanto, implementar o `Serializable`.

Resumindo, a facilidade de implementar o `Serializable` é duvidosa. A menos que a classe seja usada apenas em um ambiente protegido onde versões nunca precisarão interoperar e servidores nunca serão expostos a dados não confiáveis, implementar o `Serializable` é um compromisso sério que deve ser feito com muito cuidado. É necessário ter cautela extra se uma classe permitir herança.

Item 87: Considere usar um formulário serializado personalizado

Quando você está escrevendo uma aula sob pressão de tempo, geralmente é apropriado concentrar seus esforços em projetar a melhor API. Às vezes, isso significa lançar uma implementação "descartável" que você sabe que vai substituir em uma versão futura. Normalmente isso não é um problema, mas se a classe implementar o `Serializable` e usar a forma serializada padrão, você nunca conseguirá sair completamente da implementação descartável. Ele ditará a forma serializada para sempre. Isso não é apenas um problema teórico. Isso aconteceu com várias classes nas Javalibraries, incluindo o `BigInteger`.

Não aceite a forma serializada padrão sem antes considerar se ela é apropriada. Aceitar a forma serializada padrão deve ser uma decisão consciente de que essa codificação é razoável do ponto de vista da flexibilidade, desempenho e correção. De modo geral, você deve aceitar a forma serializada padrão apenas se ela for em grande parte idêntica à codificação que escolheria se estivesse projetando uma forma serializada personalizada.

A forma serializada padrão de um objeto é uma codificação razoavelmente eficiente da representação física do grafo de objeto enraizado no objeto. Em outras palavras, ele descreve os dados contidos no objeto e em cada objeto que pode ser alcançado a partir desse objeto. Também descreve a topologia pela qual todos esses objetos estão interligados. A forma serializada ideal de um objeto contém apenas os dados lógicos representados pelo objeto. É independente da representação física.

A forma serializada padrão provavelmente será apropriada se a representação física de um objeto for idêntica ao seu conteúdo lógico. Por exemplo, a forma serializada padrão seria razoável para a seguinte classe, que representa de forma simplista o nome de uma pessoa:

```
Bom candidato para a classe formalpublic
serializada padrão Name implementa Serializable {
    /**
     * Sobrenome. Deve ser não-nulo.*
     * @serial*/final privado
     ApellidoNome;

    /**
     * Primeiro nome. Deve ser não-
     nulo.* @serial*/final privado
     Primeiro Nome;
```

```
/**  
 * Nome do meio, ou nulo se não houver  
 nenhum.* @serial*/final privado String Nome  
 do Meio;  
  
... // Restante omitido}
```

Logicamente falando, um nome consiste em três sequências que representam um sobrenome, um primeiro nome e um nome do meio. Os campos de instância em `Name` refletem exatamente esse conteúdo lógico.

Mesmo que você decida que a forma serializada padrão é apropriada, muitas vezes é necessário fornecer um método `readObject` para garantir invariantes e segurança. No caso de `Nome`, o método `readObject` deve garantir que os campos `NomeÚltimo` e `Primeiro Nome` não sejam nulos. Essa questão é discutida extensivamente nos Itens 88 e 90.

Note que há comentários documentacionais nos campos `Sobrenome`, `Primeiro Nome` e `Nome do Meio`, mesmo que sejam privados. Isso porque esses campos privados definem uma API pública, que é a forma serializada da classe, e essa publicAPI deve ser documentada. A presença da tag `@serial` indica ao Javadoc que coloque essa documentação em uma página especial que documenta formulários serializados.

Perto do extremo oposto do espectro em relação ao `Nome`, considere a seguinte classe, que representa uma lista de strings (ignorando por enquanto que provavelmente seria melhor usar uma das implementações padrão da `Lista`):

```
Candidato terrível para a classe final formal serializada  
padrão StringList implementa Serializable {  
    private int size = 0; cabeça  
    de entrada privada = nulo;  
  
    classe estática privada Entry implementa Serializable {  
        Dados em cadeia;  
        Entrada próxima;  
        Entry previous;}  
  
... // Restante omitido}
```

Logicamente falando, essa classe representa uma sequência de cadeias. Fisicamente, ela representa a sequência como uma lista duplamente ligada. Se você aceitar a forma serializada padrão, a forma serializada irá espelhar cuidadosamente cada entrada na lista encadeada e todos os links entre as entradas, em ambas as direções.

Usar a forma serializada padrão quando a representação física de um objeto difere substancialmente de seu conteúdo lógico de dados tem quatro desvantagens:

- Ela vincula permanentemente a API exportada à representação interna atual. No exemplo acima, a classe privada `StringList.Entry` passa a fazer parte da API pública. Se a representação for alterada em uma versão futura, a classe `theStringList` ainda precisará aceitar a representação da lista encadeada na entrada e gerá-la na saída. A classe nunca vai se livrar de todo o código que lida com entradas de lista enlaçada, mesmo que não as use mais.
- Pode consumir espaço excessivo. No exemplo acima, a forma serializada representa desnecessariamente cada entrada na lista enlaçada e todos os links. Essas entradas e links são meros detalhes de implementação, não dignos de inclusão na forma serializada. Como a forma serializada é excessivamente grande, gravá-la em disco ou enviá-la pela rede será excessivamente lenta.
- Pode consumir tempo excessivo. A lógica de serialização não tem conhecimento da topologia do grafo objeto, então deve passar por um atravessamento caro do grafo. No exemplo acima, seria suficiente simplesmente seguir as próximas referências.
- Pode causar transbordamentos de pilha. O procedimento padrão de serialização realiza a travessia recursiva do grafo de objetos, o que pode causar excessos de pilha mesmo para grafos de objeto de tamanho moderado. Serializar uma instância de `StringList` com 1.000–1.800 elementos gera um `StackOverflowError` na minha máquina. Surpreendentemente, o tamanho mínimo da lista para o qual a serialização causa um excesso de pilha varia de execução para sequência (na minha máquina). O tamanho mínimo da lista que apresenta esse problema pode depender da implementação da plataforma e das flags da linha de comando; Algumas implementações podem não ter esse problema algum.

Uma forma serializada razoável para o `StringList` é simplesmente o número de strings na lista, seguido pelas próprias strings. Isso constitui os dados lógicos representados por uma `StringList`, despojados dos detalhes de sua representação física. Aqui está uma versão revisada do `StringList` com `writeObject` e `readObjectMethods` que implementam essa forma serializada. Como lembrete, o `transientmodifier` indica que um campo de instância deve ser omitido da forma serializada padrão de uma classe:

```
StringList com uma classe final formalpublic serializada
razoavelmente personalizada StringList implementa Serializable {

    Tamanho de INT de transiente privado = 0;
    cabeça de entrada transitória privada = nula;

    Não é mais Serializable!private
    static class Entrada {
        Dados em cadeia;
        Entradapróxima;
        Entryprevious;}

    Adiciona a string especificada ao listpublic
    final void add(String s) { ... }

    /**
     * Serialize esta instância {@code StringList}.** @serialData O
     tamanho da lista (o número de strings* que ela contém) é emitido
     ({@code int}), seguido por todos os seus elementos (cada um uma
     {@code String}), na sequência adequada*./private void
     writeObject(ObjectOutputStream s)

        lança IOException
        {s.defaultWriteObject();
        s.writeInt(tamanho);

        Escreva todos os elementos na ordem correta.for
        (Entrada e = cabeça; e != nulo; e = e.next)
            s.writeObject(e.data);
        }

        private void readObject(ObjectInputStream s)throws
        IOException, ClassNotFoundException {
            s.defaultReadObject(); int
            numElements = s.readInt();

            Leia todos os elementos e insira-os na listfor (int
            i = 0; i < numElements; i++)
                add((String) s.readObject());
        }

    ... // Restante omitido}
```

A primeira coisa que `writeObject` faz é invocar o `defaultWriteObject`, e a primeira coisa que `readObject` faz é invocar o `defaultReadObject`, mesmo que todos os campos do `StringList` sejam transitórios. Você pode ouvir dizer que, se todos os campos de instância de uma classe forem transitórios, você pode dispensar a invocação de `defaultWriteObject` e `defaultReadObject`, mas a especificação de serialização exige que você os invoque de qualquer forma. A presença dessas chamadas possibilita adicionar campos de instância não transiente em uma versão posterior, preservando a compatibilidade para trás e para trás. Se uma instância for serializada em uma versão posterior e desserializada em uma versão anterior, os campos adicionados serão ignorados. Se o `readObjectMethod` da versão anterior não tivesse conseguido invocar o `defaultReadObject`, a desserialização falharia com `StreamCorruptedException`.

Note que há um comentário de documentação sobre o método `writeObject`, mesmo sendo privado. Isso é análogo ao comentário de documentação sobre os campos privados na classe `Nome`. Esse método privado define uma API pública, que é a forma serializada, e essa API pública deve ser documentada. Assim como o `@serialtag` para campos, a tag `@serialData` para métodos orienta a utilitária Javadoc a colocar essa documentação na página de formulários serializados.

Para dar uma noção de escala à discussão anterior sobre performance, se o comprimento médio da string for de dez caracteres, a forma serializada da versão revisada do `StringList` ocupa cerca de metade do espaço da forma serializada do original. Na minha máquina, serializar a versão revisada do `StringList` é mais do que duas vezes mais rápido do que serializar a versão original, com um comprimento de lista de dez. Por fim, não há problema de excesso de pilha na forma revisada e, portanto, não existe limite prático para o tamanho do `StringList` que possa ser serializado.

Embora a forma serializada padrão seja ruim para o `StringList`, existem classes para as quais seria muito pior. Para o `StringList`, a `serializedForm` padrão é inflexível e tem um desempenho ruim, mas é correta no sentido de que serializar e desserializar uma instância do `StringList` gera uma cópia fiel do objeto original com todos os seus invariantes intactos. Isso não ocorre para qualquer objeto cujos seínvariantes estejam ligados a detalhes específicos da implementação.

Por exemplo, considere o caso de uma tabela hash. A representação física é uma sequência de baldes de hash contendo entradas-chave-valor. O balde em que uma entrada reside é uma função do código hash de sua chave, que, em geral, não é garantido que seja o mesmo de implementação para implementação. Na verdade, nem sequer é garantido que seja igual de uma corrida para outra. Portanto, aceitar o formulário serializado padrão para uma tabela hash constituiria um bug sério. Serializar e desserializar a tabela hash podia gerar um objeto cujos invariantes eram seriamente corrompidos.

Quer você aceite ou não a forma serializada padrão, todo campo de instância que não seja rotulado como transitório será serializado quando o método `defaultWriteObject` for invocado. Portanto, todo campo de instância que pode ser declarado transitório deve ser. Isso inclui campos derivados, cujos valores podem ser calculados a partir de campos de dados primários, como um valor de hash em cache. Também inclui campos cujos valores estão ligados a uma execução específica da JVM, como um campo longo representando um ponteiro para uma estrutura de dados nativa. Antes de decidir tornar um campo não transitório, convença-se de que seu valor faz parte do estado lógico do objeto. Se você usar uma forma serializada personalizada, a maioria ou todos os campos de instância devem ser rotulados como transientes, como no exemplo do `StringList` acima.

Se você estiver usando a forma serializada padrão e rotulou um ou mais campos como transitórios, lembre-se de que esses campos serão inicializados para seus valores padrão quando uma instância for desserializada: `null` para campos de referência de objetos, zero para campos primitivos numéricos e `false` para campos booleanos [JLS, 4.12.5]. Se esses valores forem inaceitáveis para quaisquer campos transitórios, você deve fornecer um método `readObject` que invoca o método `defaultReadObject` e então restaura os campos transitórios para valores aceitáveis (Item 88). Alternativamente, esses campos podem ser inicializados preguiçosamente na primeira vez que são usados (Item 83).

Independentemente de você usar ou não a forma serializada padrão, deve impor qualquer sincronização à serialização de objetos que impeça a qualquer outro método que leia todo o estado do objeto. Então, por exemplo, se você tem um objeto seguro para threads (Item 82) que alcança sua segurança de thread sincronizando cada método e você optar por usar a forma serializada padrão, use o seguinte método `write-Object`:

```
writeObject para classe sincronizada com o writeObject sincronizado padrão de  
formprivate, sincronizado de void writeObject(ObjectOutputStream s)  
throws IOException  
{s.defaultWriteObject();}
```

Se você colocar a sincronização no método `writeObject`, deve garantir que ele cumpra as mesmas restrições de ordenação de bloqueio que outras atividades, ou corre o risco de um deadlock de ordenação de recursos [Goetz06, 10.1.5].

Independentemente da forma serializada que você escolher, declare um UID de versão serializada explícita em cada classe serializável que você escrever. Isso elimina o UID de versão seriada como uma possível fonte de incompatibilidade (Item 86). Há também um pequeno benefício de desempenho. Se não for fornecido um UID de versão serial, é realizada uma computação cara para gerar um em tempo de execução.

Declarar uma versão serial UID é simples. Basta adicionar esta frase à sua aula:

```
private static final long serialVersionUID = randomLongValue;
```

Se você escrever uma nova classe, não importa qual valor você escolha para `randomLongValue`. Você pode gerar o valor rodando o utilitário `serialver` na classe, mas também é tranquilo escolher um número do nada. Não é obrigatório que UIDs de versão serial sejam únicos. Se você modificar uma classe existente que não possui UID de versão serial, e quiser que a nova versão aceite instâncias serializadas existentes, deve usar o valor que foi gerado automaticamente para a versão antiga. Você pode obter esse número rodando a utilidade `serialver` na versão antiga da classe — aquela para a qual existem instâncias serializadas.

Se você quiser criar uma nova versão de uma classe que seja incompatível com versões existentes, basta mudar o valor na declaração do UID da versão serial. Isso fará com que tentativas de desserializar instâncias serializadas de versões anteriores criem uma `InvalidClassException`. Não mude o UID da versão serial a menos que você queira quebrar a compatibilidade com todas as instâncias serializadas existentes de uma classe.

Para resumir, se você decidiu que uma classe deve ser serializável (Item 86), pense bem sobre qual deve ser a forma serializada. Use a forma serializada padrão apenas se for uma descrição razoável do estado lógico do objeto; caso contrário, projete uma forma serializada personalizada que descreva adequadamente o objeto. Você deve dedicar tanto tempo ao design da forma serializada de uma classe quanto ao design de um método exportado (Item 51). Assim como você não pode eliminar métodos exportados de versões futuras, também não pode eliminar campos da forma serializada; eles devem ser preservados para sempre para garantir a compatibilidade de serialização. Escolher a forma serializada errada pode ter um impacto permanente e negativo na complexidade e desempenho de uma classe.

Item 88: Escrever métodos readObject defensivamente

O item 50 contém uma classe de intervalo de datas imutável com campos de data privados mutáveis. A classe faz grandes esforços para preservar seus invariantes e imutabilidade, copiando de forma definitiva objetos Date em seu construtor e acessórios. Aqui está a aula:

```

Classe imutável que usa copyingpublic defensive
final class Period {
    private final de encontro;
    privado final fim do encontro;

    /**
     * @param início do período
     * @param encerrar o fim do período; não deve preceder o início
     * @throws IllegalArgumentException se o início for após o fim*
     * @throws NullPointerException se o início ou o fim for
     * null*/public Period(Data de início, Data de término) {

        this.start = nova Data(start.getTime());
        this.end= nova Data(fim.obterTempo()); if
        (this.start.compareTo(this.end) > 0)
            Lançar um novo IllegalArgumentException(
                início + " após " + fim);
    }

    public Data início () { return new Data(start.getTime()); }
    public Data fim () { return new Data(end.getTime()); }
    public String toString() { return start + " - " + end; }

    ... // Restante omitido}

```

Suponha que você decida que quer que essa classe seja serializável. Como a representação física de um objeto de Período reflete exatamente seu conteúdo lógico de dados, não é irrazoável usar a forma serializada padrão (Item 87). Portanto, pode parecer que tudo o que você precisa fazer para tornar a classe serializável é adicionar as palavras implementas Serializable à declaração de classe. Se você fizesse isso, porém, a classe não garantiria mais seus invariantes críticos.

O problema é que o método readObject é efetivamente outro construtor público, e exige todo o mesmo cuidado que qualquer outro construtor. Assim como um construtor deve verificar seus argumentos quanto à validade (Item 49) e fazer cópias defensivas dos parâmetros quando apropriado (Item 50), um método readObject também deve fazer. Se um método readObject falhar em fazer qualquer uma dessas coisas, é relativamente simples para um atacante violar os invariantes da classe.

Falando de forma geral, `readObject` é um construtor que recebe um fluxo de bytes como seu único parâmetro. No uso normal, o fluxo de bytes é gerado pela serialização de uma instância normalmente construída. O problema surge quando o `readObject` é apresentado com um fluxo de bytes que é artificialmente construído para gerar um objeto que viola os invariantes de sua classe. Tal fluxo de bytes pode ser usado para criar um objeto impossível, que não poderia ter sido criado usando um construtor normal.

Suponha que simplesmente adicionamos implements `Serializable` à declaração de classe para `Período`. Esse programa feio então geraria uma instância de período cujo fim precede seu início. Os casts em valores de byte cujo bit de ordem alta é set, são consequência da falta de literais de byte em Java combinada com a decisão infeliz de tornar o tipo de byte assinado:

```

classe pública BogusPeriod {
    O fluxo de bytes não poderia ter vindo de uma instância real de
    Período; byte final estático privado[] serializedForm = {
        (byte)0xac, (byte)0xed, 0x00, 0x05, 0x73, 0x72, 0x00, 0x06, 0x50,
        0x65, 0x72, 0x69, 0x6f, 0x64, 0x40, 0x7e, (byte)0xf8, 0x2b, 0x4f,
        0x46, (byte)0xc0, (byte)0xf4, 0x02, 0x00, 0x02, 0x4c, 0x00, 0x03,
        0x65, 0x6e, 0x64, 0x74, 0x00, 0x10, 0x4c, 0x6a, 0x61, 0x76, 0x61,
        0x2f, 0x75, 0x74, 0x69, 0x6c, 0x2f, 0x44, 0x61, 0x74, 0x65, 0x3b,
        0x4c, 0x00, 0x05, 0x73, 0x74, 0x61, 0x72, 0x74, 0x71, 0x00, 0x7e,
        0x00, 0x01, 0x78, 0x70, 0x73, 0x72, 0x00, 0x0e, 0x6a, 0x61, 0x76,
        0x61, 0x2e, 0x75, 0x74, 0x69, 0x6c, 0x2e, 0x44, 0x61, 0x74, 0x65,
        0x68, 0x6a, (byte)0x81, 0x01, 0x4b, 0x59, 0x74, 0x19, 0x03, 0x00,
        0x00, 0x78, 0x70, 0x77, 0x08, 0x00, 0x00, 0x00, 0x66,
        (byte)0xdf, 0x6e, 0x1e, 0x00, 0x78, 0x73, 0x71, 0x00, 0x7e, 0x00,
        0x03, 0x77, 0x08, 0x00, 0x00, 0x00, (byte)0xd5, 0x17, 0x69,
        0x22, 0x00, 0x78};

    public static void main(String[] args) {
        Período p = (Período) desserializar(serializadaForma);
        System.out.println(p);}

    Retorna o objeto com o objeto serializado Objeto formestático
    especificado deserialize(byte[] sf) {
        try {
            return new ObjectInputStream(
                novo ByteArrayInputStream(sf)).readObject();} catch
                (IOException | ClassNotFoundException e) {
            lançar um novo IllegalArgumentException(e);}}

```

O literal do array de bytes usado para inicializar `serializedForm` foi gerado serializando uma instância normal de `Period` e editando manualmente o fluxo de bytes resultante. Os detalhes do fluxo não são importantes para o exemplo, mas se você estiver curioso, o formato `byte-stream` de `tharilização` é descrito na `Java Object Serialization Specification` [6]. Se você rodar este programa, ele imprime `Sex Jan 0112:00:00 PST 1999 - Dom Jan 01 12:00:00 PST 1984`. Simplesmente declarar `Periodserializable` nos permitiu criar um objeto que viola seus invariantes de classe.

Para resolver esse problema, forneça um método `readObject` para `Period` que chame o `defaultReadObject` e depois verifique a validade do objeto deserializado. Se a verificação de validade falhar, o método `readObject` lança `InvalidObjectException`, impedindo a desserialização de ser concluída:

```
método readObject com verificação de validade -
insuficiente!private void readObject(ObjectInputStream s)
    lança IOException, ClassNotFoundException
    {s.defaultReadObject();

    Verifique se nossos invariantes estão
    satisfeitos (start.compareTo(end) > 0)

    lançar um novo InvalidObjectException (start +" após "+ fim);
}
```

Embora isso impeça que um atacante crie uma instância de `Período` inválida, ainda existe um problema mais sutil à espreita. É possível criar uma `Periodinstance` mutável fabricando um fluxo de bytes que começa com uma instância válida de `Period` e depois adiciona referências extras aos campos de `Data` privados internos da `Periodinstance`. O atacante lê a instância `Period` do `ObjectInputStream` e então lê as "referências a objetos não autorizados" que foram adicionadas ao fluxo. Essas referências dão ao atacante acesso aos objetos referenciados pelos campos `privateDate` dentro do objeto `Período`. Ao mutar essas instâncias `Data`, o atacante pode mutar a instância de `Período`. A seguinte classe demonstra este ataque:

```
classe pública MutablePeriod {
    Um período de instância
    pública final período período;

    campo de início do período, ao qual não deveríamos ter
    acessoPublic Data final de início;

    campo de fim de período, para o qual não deveríamos ter
    acesso público final Data fim;
```



```

MutablePeriod publico() {
    try {
        ByteArrayOutputStream bos =
            novo ByteArrayOutputStream();
        ObjectOutputStream fora =
            novo ObjectOutputStream(bos);

        Serializar uma instância válida de
        Período.writeObject(new Period(new Date(), new Date()));

        /*
         * Adicione "referências de objeto anteriores" rogue
         para campos internos de data em Período. Para detalhes,
         veja "Especificação de Serialização de Objetos Java*,"
         Seção 6.4.*/byte[] ref = { 0x71, 0, 0x7e, 0, 5 }; Ref
         #5bos.write(ref); 0 fieldref[4] = 4; // Ref #
         4bos.write(ref); 0 campo final

        Desserializar Período e Data "roubada"
        referênciasObjectInputStream em = novo ObjectInputStream(
        novo ByteArrayInputStream(bos.toByteArray())); período =
        (Período) em.readObject(); start= (Data)em.readObject();
        fim= (Data)em.ObjetoLer();} catch (IOException |
        ClassNotFoundException e) {

        throw new AssertionError(e);}}

```

Para ver o ataque em ação, execute o seguinte programa:

```

public static void main(String[] args) {
    PeríodoMutableMp = novo PeríodoMutable();
    Período p = mp.período; Data pEnd =
    mp.end;

    Vamos voltar no
    tempopEnd.setYear(78);
    System.out.println(p);

    Traga de volta os anos
    60!pEnd.setYear(69);
    System.out.println(p);}

```

Na minha região, executar este programa gera a seguinte saída:

```
Qua 22 Nov 00:21:29 PST 2017 - Qua Nov 22 00:21:29 PST 1978 Qua
Nov 22 00:21:29 PST 2017 - Sáb 22 Nov 00:21:29 PST 1969
```

Embora a instância de Período seja criada com seus invariantes intactos, é possível modificar seus componentes internos à vontade. Uma vez em posse de uma instância de Período mutável, um atacante pode causar grande dano ao passar a instância para uma classe que depende da imutabilidade de Período para sua segurança. Isso não é tão improvável: existem classes que dependem da imutabilidade de String para sua segurança.

A origem do problema é que o método `readObject` do `Period` não faz cópias defensivas suficientes. Quando um objeto é desserializado, é fundamental copiar defensivamente qualquer campo contendo uma referência a um objeto que um cliente não deve possuir. Portanto, toda classe imutável serializável contendo componentes `privatetable` deve copiar defensivamente esses componentes em seu `readObject` method. O seguinte método `readObject` é suficiente para garantir os invariantes de `Period` e para manter sua imutabilidade:

```
método readObject com cópia defensiva e verificação de
validadeprivate void readObject(ObjectInputStream s)
    lança IOException, ClassNotFoundException
    {s.defaultReadObject();

    Copie defensivamente nossos componentes
    mutáveis = nova Data(start.getTime()); fim=
    nova Data(fim.getTime());

    Verifique se nossos invariantes estão
    satisfeitos (start.compareTo(end) > 0)
        lançar um novo InvalidObjectException (start +" após "+" fim);
}
```

Note que a cópia defensiva é feita antes da verificação de validade e que não usamos o método `clone` do `Date` para executar a cópia defensiva. Ambos esses detalhes são necessários para proteger o `Period` contra ataques (Item 50). Note também que a cópia defensiva não é possível para campos finais. Para usar o método `readObject`, devemos tornar os campos inicial e final não finais. Isso é lamentável, mas é o menor dos dois males. Com o novo método `readObject` implementado e o mod-ifier final removido dos campos de início e fim, a classe `MutablePeriod` se torna ineficaz. O programa de ataque acima agora gera esta saída:

```
Qua 22 Nov 00:23:41 PST 2017 - Qua Nov 22 00:23:41 PST 2017 Qua
Nov 22 00:23:41 PST 2017 - Qua Nov 22 00:23:41 PST 2017
```

Aqui está um teste simples para decidir se o `readObjectMethod` padrão é aceitável para uma classe: você se sentiria confortável em adicionar um `constructor` público que tomasse como parâmetros os valores de cada campo não transiente no objeto e armazenasse os valores nos campos sem nenhuma validação? Se não, você deve fornecer um método `readObject`, que deve realizar toda a verificação de validade e cópia defensiva que seria exigida de um construtor. Alternativamente, você pode usar o padrão proxy de serialização (Item 90). Esse padrão é altamente recomendado porque reduz grande parte do esforço da desserialização segura.

Há outra semelhança entre métodos `readObject` e construtores que se aplica a classes serializáveis não finais. Como um construtor, um `readObjectMethod` não deve invocar um método sobrescritivo, nem direta nem indiretamente (Item 19). Se essa regra for violada e o método em questão for sobreposto, o método `theoverriding` será executado antes que o estado da subclasse seja desserializado. A falha do programa provavelmente resultará [Bloch05, Quebra-cabeça 91].

Para resumir, sempre que você escrever um método `readObject`, adote a mentalidade de que você está escrevendo um construtor público que deve produzir uma instância válida, independentemente do fluxo de bytes que ele recebe. Não presuma que o fluxo de bytes representa uma instância serializada real. Embora os exemplos neste item digam respeito a uma classe que usa a forma serializada padrão, todas as questões levantadas se aplicam igualmente a classes com formulários serializados personalizados. Aqui, em resumo, estão as diretrizes para escrever um método `readObject`:

- Para classes com campos de referência de objeto que devem permanecer privados, copie defensivamente cada objeto em tal campo. Componentes mutáveis de classes imutáveis se enquadram nessa categoria.
- Verificar quaisquer invariantes e lançar um `InvalidObjectException` caso uma verificação falhe. As verificações devem seguir qualquer cópia defensiva.
- Se um grafo de objeto inteiro precisar ser validado após ser desserializado, use a interface `ObjectInputValidation` (não discutida neste livro).
- Não invoque nenhum método sobrescrito na classe, direta ou indiretamente.

Item 89: Por exemplo, controle, prefira tipos de enum para readResolve

O item 3 descreve o padrão Singleton e apresenta o seguinte exemplo de uma classe de uma tonelada. Esta classe restringe o acesso ao seu construtor para garantir que apenas uma única instância seja criada:

```
classe pública Elvis {
    instância pública estática final Elvis INSTANCE =
        novo Elvis(); soldado Elvis() { ... }

    vazio público deixaOPrédio() { ... }}
```

Como mencionado no Item 3, essa classe deixaria de ser singleton se os `wordsimplements Serializable` fossem adicionados à sua declaração. Não importa se a classe usa a forma serializada padrão ou uma forma serializada personalizada (Item 87), nem se a classe fornece um `readObjectmethod` explícito (Item 88). Qualquer método `readObject`, seja explícito ou padrão, retorna uma instância recém-criada, que não será a mesma instância criada no momento da inicialização da classe.

O recurso `readResolve` permite substituir outra instância pela criada pelo `readObject` [Serialization, 3.7]. Se a classe de um objeto que está sendo deserializado define um método `readResolve` com a declaração adequada, esse método é invocado no objeto recém-criado após ele ser desserializado. A referência do objeto retornada por esse método é então reintroduzida no lugar do objeto recém-criado. Na maioria dos usos dessa funcionalidade, nenhuma referência ao objeto recém-criado é mantida, tornando-se imediatamente elegível para coleta de lixo.

Se a classe `Elvis` for feita para implementar `Serializable`, o seguinte método `read-Resolve` é suficiente para garantir a propriedade singleton:

```
readResolve para controle de instância - você pode fazer
melhor!private Object readResolve() {
    Devolva o único Elvis verdadeiro e deixe o coletor de lixo// cuidar
    do imitador do Elvis.return INSTANCE;}
```

Esse método ignora o objeto desserializado, retornando o `Elvisinstance` distinto que foi criado quando a classe foi inicializada. Portanto, a forma serializada de uma instância de `Elvis` não precisa conter dados reais; Todos os campos de instância devem ser declarados transitórios. Na verdade, se você depender do `readResolve`, por exemplo

controle, todos os campos de instância com tipos de referência de objeto devem ser declarados transientes. Caso contrário, é possível que um atacante determinado assegure a referência ao objeto desserializado antes que seu método `readResolve` seja executado, usando uma técnica um pouco semelhante ao ataque `MutablePeriod` no Item 88.

O ataque é um pouco complicado, mas a ideia subjacente é simples. Se o singleton contém um campo de referência de objeto não transitório, o conteúdo desse campo será desserializado antes que o método `readResolve` do singleton seja executado. Isso permite que um fluxo cuidadosamente elaborado "roube" uma referência ao singleton originalmente desserializado no momento em que o conteúdo do campo de referência do objeto é desserializado.

Veja como funciona com mais detalhes. Primeiro, escreva uma classe "stealer" que tenha tanto o método `readResolve` quanto um campo de instância que se refira ao singleton serializado no qual o stealer "se esconde". No fluxo de serialização, substitua o campo não transitório do singleton por uma instância do stealer. Agora você tem uma circularidade: o singleton contém o ladrão, e o ladrão se refere ao singleton.

Como o singleton contém o stealer, o método `readResolve` do stealer é executado primeiro quando o singleton é desserializado. Como resultado, quando o método `stealer.readResolve` é executado, seu campo de instância ainda se refere ao singleton parcialmente desserializado (e ainda não resolvido).

O método `readResolve` do stealer copia a referência do campo de instância para um campo estático, de modo que a referência possa ser acessada após a execução do `readResolve` method. O método então retorna um valor do tipo correto para o campo em que está escondido. Se isso não acontecesse, a VM lançava uma `ClassCastException` quando o sistema de serialização tentava armazenar a referência stealer nesse campo.

Para tornar isso concreto, considere o seguinte singleton quebrado:

```
Singleton quebrado - possui campo de referência de objeto não
transiente! classe pública Elvis implementa Serializable {
    instância pública estática final Elvis INSTANCE =
        novo Elvis(); privado Elvis() { }

    privado Corda[] favoritoMúsicas =
    { "Hound Dog", "Hotel do Coração Partido"
    }; p vazio Ubbique impressa Favoritos() {
        System.out.println(Arrays.toString(favoritaMúsicas)); }

    Objeto privado readResolve() {
        return INSTANCE; } }
```

Aqui está uma classe "ladrão", construída conforme a descrição acima:

```

classe pública ElvisStealer implementa Serializable {
    imitador estático de Elvis;
    carga útil Elvis privada;

    Objeto privado readResolve() {
        Salve uma referência ao "não resolvido" Elvis
        instanceimpersonator = payload;

    Return objeto do tipo correto para favoriteSongs fieldreturn new
    String[] { "A Fool Such as I" };}private static final long
    serialEditionUID = 0;}
```

Por fim, aqui está um programa feio que desserializa um fluxo artesanal para produzir dois exemplos distintos do singleton falho. O método de desserializar é omitido neste programa porque é idêntico ao da página 354:

```

classe pública ElvisImpersonator {
    O fluxo de bytes não poderia ter vindo de uma instância real do
    Elvis!private static final byte[] serializedForm = {
        (byte)0xac, (byte)0xed, 0x00, 0x05, 0x73, 0x72, 0x00, 0x05,0x45,
        0x6c, 0x76, 0x69, 0x73, (byte)0x84, (byte)0xe6,(byte)0x93, 0x33,
        (byte)0xc3, (byte)0xf4, (byte)0x8b.0x32, 0x02, 0x00, 0x01, 0x4c,
        0x00, 0x0d, 0x66, 0x61, 0x76,0x6f, 0x72, 0x69, 0x74, 0x65, 0x53,
        0x6f, 0x6e, 0x67, 0x73,0x74, 0x00, 0x12, 0x4c, 0x6a, 0x61, 0x76,
        0x61, 0x2f, 0x6c,0x61, 0x6e, 0x67, 0x2f, 0x4f, 0x62, 0x6a, 0x65,
        0x63, 0x74,' 0x3b, 0x78, 0x70, 0x73, 0x72, 0x00, 0x0c, 0x45, 0x6c,
        0x76,0x69, 0x73, 0x53, 0x74, 0x65, 0x61, 0x6c, 0x65, 0x72,
        0x00,0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x02, 0x00,
        0x01,0x4c, 0x00, 0x07, 0x70, 0x61, 0x79, 0x6c, 0x6f, 0x61,
        0x64,0x74, 0x00, 0x07, 0x4c, 0x45, 0x6c, 0x76, 0x69, 0x73,
        0x3b,0x78, 0x70, 0x71, 0x00, 0x7e, 0x00, 0x02};

    public static void main(String[] args) {
        Inicializa ElvisStealer.imitador e retorna// o
        verdadeiro Elvis (que é Elvis.INSTANCE)Elvis elvis =
        (Elvis) desserializa(serializadaForma); Elvis imitador =
        ElvisStealer.imitador;

        elvis.printFavoritos();
        impersonator.printFavoritos();}}
```

Executar este programa produz o seguinte resultado, provando conclusivamente que é possível criar duas instâncias distintas de Elvis (com gostos musicais diferentes):

```
[Cão de Caça, Hotel Coração
Partido][Um Tolo Como Eu]
```

Você poderia resolver o problema declarando o campo `favoriteSongs` como transitório, mas é melhor resolver fazendo do Elvis um tipo enum de elemento único (Item 3). Como demonstrado pelo ataque `ElvisStealer`, usar um `readResolveMethod` para impedir que uma instância "temporária" desserializada seja acessada por um atacante é frágil e exige grande cuidado.

Se você escrever sua classe controlada por instância serializável como um enum, o Java garante que não pode haver instâncias além das constantes declaradas, a menos que um atacante abuse de um método privilegiado como `AccessibleObject.setAccessible`. Qualquer atacante que possa fazer isso já tem privilégios suficientes para executar código arbitrário nativo, e tudo está em aberto. Veja como nosso exemplo do Elvis se apresenta como um enum:

```
Enum singleton - a abordagem preferida public
enum Elvis {
    EXEMPLO; privado Corda[]
    favoritoMúsicas =
        { "Hound Dog", "Hotel do Coração Partido"
        }; p vazioUbligue impressaFavoritos() {
    System.out.println(Arrays.toString(favoriteSongs)); }
```

O uso do `readResolve` para controle de instâncias não é obsoleto. Se você precisar escrever uma classe controlada por instância serializável cujas instâncias não são conhecidas em tempo de compilação, não será possível representar a classe como um tipo enum.

A acessibilidade do `readResolve` é significativa. Se você colocar um `readResolveMethod` em uma última aula, ela deve ser privada. Se você colocar um método `readResolve` em uma aula não final, deve considerar cuidadosamente sua acessibilidade. Se for privado, não se aplicará a nenhuma subclasse. Se for privada por pacote, ela se aplicará apenas a subclasses do mesmo pacote. Se for protegido ou público, se aplicará a todas as subclasses que não o sobreponham. Se um método `readResolve` for protegido ou se uma subclasse pública e uma subclasse não o sobrepor, a desserialização de uma instância de subclasse produzirá uma instância de superclasse, o que provavelmente causará uma `ClassCastException`.

Para resumir, use tipos de enum para impor invariantes de controle de instância sempre que possível. Se isso não for possível e você precisar que uma classe seja tanto serializável quanto controlada por instâncias, deve fornecer um método `readResolve` e garantir que todos os campos de instância da classe sejam primitivos ou transitórios.

Item 90: Considere proxies de serialização em vez de serializados Instâncias

Como mencionado nos Itens 85 e 86 e discutido ao longo deste capítulo, a decisão de implementar o `Serializable` aumenta a probabilidade de bugs e problemas de segurança, pois permite a criação de instâncias usando um mecanismo extralinguístico no lugar de construtores comuns. Existe, no entanto, uma técnica que reduz muito esses riscos. Essa técnica é conhecida como padrão proxy de serialização.

O padrão proxy de serialização é razoavelmente simples. Primeiro, projete uma classe estática privada aninhada que represente de forma concisa o estado lógico de uma instância da classe envolvente. Essa classe aninhada é conhecida como proxy de serialização da classe enclos. Deve ter um único construtor, cujo tipo de parâmetro é a classe enclos. Esse construtor apenas copia os dados de seu argumento: ele não precisa fazer nenhuma verificação de consistência ou cópia defensiva. Por design, a forma serializada padrão do proxy de serialização é a forma serializada perfeita da classe theenclos. Tanto a classe envolvente quanto seu proxy de serialização devem ser declarados para implementar o `Serializable`.

Por exemplo, considere a classe `Período` imutável escrita no Item 50 e tornada serializável no Item 88. Aqui está um proxy de serialização para essa classe. Ponto tão simples que seu proxy de serialização tem exatamente os mesmos campos que a classe:

```
Proxy de serialização para Period class
private static class SerializationProxy
implements Serializable {
    private final de encontro; privado final fim do encontro;

    SerializationProx(Period p) {
        this.start = p.start;
        this.end = p.end;}

    private static final long serialVersionUID =
        234098243823485285L; Qualquer número serve (Item 87)
}
```

Em seguida, adicione o seguinte método `writeReplace` à classe de anexo. Esse método pode ser copiado literalmente para qualquer classe com um proxy de serialização:

```
Método writeReplace para o proxy de serialização,
pattern private, Object writeReplace() {
    return new SerializationProoxy(this);}
```


A presença desse método na classe de encapsulamento faz com que o sistema de serialização emita uma instância de Proxy de Serialização em vez de uma instância da classe de enclosing. Em outras palavras, o método `writeReplace` traduz uma instância da classe `theEnclosing` para seu proxy de serialização antes da serialização.

Com esse método `writeReplace` implementado, o sistema de serialização nunca gerará uma instância serializada da classe que encerra, mas um atacante pode fabricar uma na tentativa de violar os invariantes da classe. Para garantir que o ataque *suchan* falharia, basta adicionar este método `readObject` à classe que envolve:

```
Método readObject para o proxy de serialização, pattern, void
privado, readObject(ObjectInputStream)
throws InvalidObjectException {throw new
InvalidObjectException("Proxy required");}
```

Por fim, forneça um método `readResolve` na classe `SerializationProxy` que retorne uma instância logicamente equivalente da classe que encerra. A presença desse método faz com que o sistema de serialização traduza o proxy de serialização de volta para uma instância da classe anexa ao desserializar.

Esse método `readResolve` cria uma instância da classe que encerra usando apenas sua API pública, e aí reside a beleza do padrão. Ela elimina em grande parte o caráter extralinguístico da serialização, pois a instância desserializada é criada usando os mesmos construtores, fábricas estáticas e métodos que qualquer outra instância. Isso te libera de ter que garantir separadamente que as instâncias desserializadas obedecem aos invariantes da classe. Se as fábricas ou construtores estáticos da classe estabelecerem esses invariantes e seus métodos de instância os mantiverem, você garantiu que os invariantes também serão mantidos por serialização.

Aqui está o método `readResolve` para `Period.SerializationProxy` acima:

```
método readResolve para
Period.SerializationProxyprivate Objeto readResolve() {
    retorno novo Período (início,      Usa construtor público
}    fim);
```

Assim como a abordagem de cópia defensiva (página 357), a abordagem de proxy de serialização impede o ataque de fluxo de bytes falso (página 354) e o ataque de roubo interno de campo (página 356) imediatamente. Diferentemente das duas abordagens anteriores, esta permite que os campos de `Período` sejam finais, o que é necessário para que a classe `Período` seja verdadeiramente imutável (Item 17). E, ao contrário das duas abordagens anteriores, esta não envolve muita reflexão. Você não precisa

Descubra quais campos podem ser comprometidos por ataques de serialização maliciosos, nem é necessário realizar explicitamente verificação de validade como parte da desserialização.

Existe outra forma pela qual o padrão proxy de serialização é mais poderoso do que a cópia defensiva no `readObject`. O padrão proxy de serialização permite que a instância desserializada tenha uma classe diferente da instância originalmente serializada. Você pode não achar que isso seria útil na prática, mas é.

Considere o caso do `EnumSet` (Item 36). Esta classe não possui construtores públicos, apenas fábricas estáticas. Do ponto de vista do cliente, eles retornam instâncias `EnumSet`, mas na implementação atual do OpenJDK, retornam uma de duas subclasses, dependendo do tamanho do tipo enum subjacente. Se o tipo enum subjacente tiver sessenta e quatro ou menos elementos, as fábricas estáticas retornam um `RegularEnumSet`; caso contrário, eles retornam um `JumboEnumSet`.

Agora considere o que acontece se você serializar um conjunto de enum cujo tipo de enum tem sessenta elementos, depois adicionar mais cinco elementos ao tipo de enum e, em seguida, desserializar o conjunto de enum. Era uma instância `RegularEnumSet` quando foi serializada, mas é melhor que seja uma instância `JumboEnumSet` assim que for desserializada. Na verdade, é exatamente isso que acontece, porque o `EnumSet` usa o padrão proxy de serialização. Caso esteja curioso, aqui está o proxy de serialização do `EnumSet`. É realmente assim tão simples:

```
A classe estática privada de serialização do EnumSet,
SerializationProxy <E, estende o Enum<E>>

    implementa Serializable { // O tipo de
        elemento deste enum set. private final<E>
        Class elementType;

    Os elementos contidos neste conjunto
    enum. private final Enum<?>[] elementos;

    SerializationProxy(<E>EnumSet set) {
        elementType = set.elementType; elements =
        set.toArray(new Enum<?>[0]);}

    Objeto privado readResolve() {
        Resultado EnumSet<E> = EnumSet.noneOf(elementType);
        para (Enum<?> e : elementos)
            resultado.add((E)e); retorne
            resultado;}

    private static final long serialVersionUID =
        362491234563181265L;
}
```

O padrão proxy de serialização tem duas limitações. Não é compatível com classes que são extensíveis por seus usuários (Item 19). Além disso, não é compatível com algumas classes cujos grafos objeto contêm circularidades: se você tentar invocar um método em tal objeto a partir do `readResolveMethod` do proxy de serialização, você obterá uma `ClassCastException` porque ainda não tem o objeto, apenas seu proxy de serialização.

Por fim, a potência e segurança adicionais do padrão proxy de serialização não são gratuitas. Na minha máquina, é 14% mais caro serializar e desserializar instâncias de `Period` com proxies de serialização do que com cópia defensiva.

Em resumo, considere o padrão proxy de serialização sempre que você se vir precisando escrever um método `readObject` ou `writeObject` em uma classe que não é extensível por seus clientes. Esse padrão talvez seja a maneira mais fácil de serializar robustamente objetos com invariantes não triviais.

Itens correspondentes à Segunda Edição

Segunda Edição Número do Item	Terceira Edição Número do Item, Título
1	1, Considere métodos de fábrica estática em vez de construtores
2	2, Considere um construtor diante de muitos parâmetros do construtor
3	3, Fazer cumprir a propriedade singleton com um construtor privado ou tipo anenum
4	4, Impor a não instanciabilidade com um construtor privado
5	6, Evite criar objetos desnecessários
6	7, Eliminar referências obsoletas a objetos
7	8, Evite finalizadores e limpadores
8	10, Obedecer ao contrato geral quando prevalecer é igual
9	11, Sempre sobrescrever hashCode quando você sobrescreve é igual a
10	12, Sempre sobrescreva paraString
11	13, Substitua o clone com critério
12	14, Considere implementar o Comparável
13	15, Minimizar a acessibilidade de turmas e membros
14	16, Em classes públicas, use métodos de acessor, não campos públicos
15	17, Minimizar a mutabilidade
16	18, Composição a favor da herança
17	19, Projetar e documentar para herança ou então proibi-la

Segunda Edição Número do Item	Terceira Edição Número do Item, Título
18	20, Prefira interfaces para classes abstratas
19	22, Use interfaces apenas para definir tipos
20	23, Prefira hierarquias de classes a classes marcadas
21	42, Prefere lambdas a classes anônimas
22	24, Favoreça classes membros estáticas em vez de não estáticas
23	26, Não use tipos brutos
24	27, Eliminar avisos descontrolados
25	28, Prefiro listas a arrays
26	29, Preferir tipos genéricos
27	30, Favor métodos genéricos
28	31, Use curingas limitados para aumentar a flexibilidade da API
29	33, Considere contêineres heterogêneos typesafe
30	34, Use enums em vez de constantes de inteligência
31	35, Use campos de instância em vez de ordinais
32	36, Use EnumSet em vez de campos de bits
33	37, Use EnumMap em vez de indexação ordinal
34	38, Emular enums extensíveis com interfaces
35	39, Prefira anotações a padrões de nomeação
36	40, Use consistentemente a anotação Override
37	41, Usar interfaces de marcadores para definir tipos
38	49, Verifique parâmetros quanto à validade
39	50, Faça cópias defensivas quando necessário
40	51, Assinaturas de métodos de projeto cuidadosamente
41	52, Use a sobrecarga com critério

Segunda Edição Número do Item	Terceira EdiçãoNúmero do Item, Título
42	53, Use varargs com cuidado
43	54, Devolver coleções ou arrays vazios, não nulos
44	56, Escrever comentários de documentação para todos os elementos da API expostos
45	57, Minimizar o escopo das variáveis locais
46	58, Prefere loops for-each para loops tradicionais for
47	59, Conheça e use as bibliotecas
48	60, Evite flutuar e dobre se respostas exatas forem necessárias
49	61, Prefere tipos primitivos a primitivos em caixa
50	62, Evite cordas onde outros tipos são mais apropriados
51	63, Cuidado com o desempenho da concatenação de strings
52	64, Referir-se a objetos por suas interfaces
53	65, Prefira interfaces à reflexão
54	66, Use métodos nativos com discernimento
55	67, Otimize com Cuidado
56	68, Seguir as convenções de nomeação geralmente aceitas
57	69, Usar exceções apenas para condições excepcionais
58	70, Use exceções verificadas para condições recuperáveis e runtimeexceptions para erros de programação
59	71, Evitar o uso desnecessário de exceções marcadas
60	72, Favoreço o uso de exceções padrão
61	73, Jogar exceções apropriadas à abstração
62	74, Documente todas as exceções lançadas por cada método
63	75, Incluir informações de captura de falha em mensagens detalhadas
64	76, Busque a atomicidade do fracasso
65	77, Não ignore exceções

Segunda Edição Número do Item	Terceira EdiçãoNúmero do Item, Título
66	78, Sincronizar o acesso a dados mutáveis compartilhados
67	79, Evite sincronização excessiva
68	80, Prefere executores, tarefas e fluxos a threads
69	81, 81, Prefiro que as concessionárias concorrentes aguardem e notifiquem
70	82, Segurança de fios de documentos
71	83, Use a inicialização preguiçosa com critério
72	84, Não dependa do agendador de threads
73	(Aposentado)
74	85, Prefiro alternativas à serialização Java 86, 86, Implemente o Serializável com grande cautela
75	85, Prefira alternativas à serialização Java 87, Considere usar uma forma serializada personalizada
76	85, Prefira alternativas à serialização Java 88, Escreva métodos readObject defensivamente
77	85, Prefere alternativas à serialização Java 89, Para controle de instância, prefere tipos de enum para readResolve
78	85, Prefira alternativas à serialização em Java 90, Considere proxies de serialização em vez de instâncias serializadas

Referências

- [Afirma] *Programação com Asserções*. 2002. Sun Microsystems.<http://docs.oracle.com/javase/8/docs/technotes/guides/language/assert.html>
- [Beck04] Beck, Kent. 2004. Guia de Bolso de Junit. Sebastopol, CA: O'ReillyMedia, Inc. ISBN: 0596007434.
- [Bloch01] Bloch, Joshua. 2001. Guia da Linguagem de Programação Java Eficaz. Boston: Addison-Wesley. ISBN: 0201310058.
- [Bloch05] Bloch, Joshua e Neal Gafter. 2005. Java Puzzlers: Armadilhas, Armadilhas e Casos de Esquina. Boston: Addison-Wesley. ISBN: 032133678X.
- [Blum14] Blum, Scott. 2014. "RSA mais rápido em Java com GMP." The SquareCorner (blog). 14 de fevereiro de 2014. <https://medium.com/square-corner-blog/faster-rsa-em-java-com-gmp-8b13c51c6ec4>
- [Bracha04] Bracha, Gilad. 2004. "Lição: Genéricos" suplemento online para o TheJava Tutorial: Um Curso Curto sobre o Básico, 6ª ed. Upper SaddleRiver, NJ: Addison-Wesley, 2014. <https://docs.oracle.com/javase/tutorial/extra/generics/>
- [Burn01] Queima, Oliver. 2001–2017. Checkstyle.<http://checkstyle.sourceforge.net>

- [Coekaerts15] Coekaerts, Wouter (@WouterCoekaerts). 2015. "DoS no estilo bilhões de risadas para serialização Java [https://gist.github.com/cookie/a27cc406fc9f3dc7a70d ...](https://gist.github.com/cookie/a27cc406fc9f3dc7a70d...) WONTFIX," Twitter, 9 de novembro de 2015, 9h46 <https://twitter.com/woutercoekaerts/status/663774695381078016>
- [CompSci17] Memorando de Cientistas da Computação como Amici Curiae para o Tribunal de Apelações do Circuito Federal dos Estados Unidos, Caso nº 17-1118, OracleAmerica, Inc. v. Google, Inc. em Apoio ao Réu-Recorrido. (2017)
- [Adaga] *Adaga. 2013. Square, Inc. <http://square.github.io/dagger/>*
- [Gallagher16] Gallagher, Sean. 2016. "Hacker do sistema muni atingiu outros ao escanear vulnerabilidade Java de um ano atrás." *Ars Technica*, 29 de novembro de 2016. <https://arstechnica.com/information-technology/2016/11/san-francisco-transit-ransomware-attacker-provavelmente-usado-ano-java-exploit/>
- [Gamma95] Gamma, Erich, Richard Helm, Ralph Johnson e John Vlissides. 1995. *Padrões de Design: Elementos de Software Orientado a Objetos Reutilizável*. Reading, MA: Addison-Wesley. ISBN: 0201633612.
- [Goetz06] Goetz, Brian. 2006. *Concorrência Java na Prática*. Com TimPeierls, Joshua Bloch, Joseph Bowbeer, David Holmes e DougLea. Boston: Addison-Wesley. ISBN: 0321349601.
- [Gosling97] Gosling, James. 1997. "A Sensação de Java." *Computer* 30 nº 6 (junho de 1997): 53-57. <http://dx.doi.org/10.1109/2.587548>
- [Goiaba] *Goiaba. 2017. Google Inc. <https://github.com/google/guava>*
- [Guice] *Guice. 2006. Google Inc. <https://github.com/google/guice>*
- [Herlihy12] Herlihy, Maurice e Nir Shavit. 2012. *A Arte da Programação Multiprocessador, Reimpressão Revisada*. Waltham, MA: Morgan Kaufmann Editora. ISBN: 0123973376.

- [Jackson75] Jackson, M. A. 1975. *Princípios do Design de Programas*. Londres: Academic Press. ISBN: 0123790506.
- [Seguro para Java] *Diretrizes de Programação Segura para Java SE*. 2017. *Oráculo*. <http://www.oracle.com/technetwork/java/seccodeguide-139067.html>
- [Java8-feat] *Novidades no JDK 8*. 2014. *Oracle*. <http://www.oracle.com/technetwork/java/javase/8-whats-new-2157071.html>
- [Java9-feat] *Plataforma Java, Edição Padrão O que há de novo no Oracle JDK 9*. 2017. *Oracle*. <https://docs.oracle.com/javase/9/whatsnew/toc.htm>
- [Java9-api] *Plataforma Java, Edição Padrão e Especificação da API Java Development Kit Version 9*. 2017. *Oracle*. <https://docs.oracle.com/javase/9/docs/api/overview-summary.html>
- [Javadoc-guia] Como Escrever Comentários de Documentação para a Ferramenta Javadoc. 2000–2004. Sun Microsystems. <http://www.oracle.com/technetwork/java/javase/documentation/index-137868.html>
- [Javadoc-ref] *Guia de Referência Javadoc*. 2014-2017. *Oracle*. <https://docs.oracle.com/javase/9/javadoc/javadoc.htm>
- [JLS] Gosling, James, Bill Joy, Guy Steele e Gilad Bracha. 2014. *A Especificação da Linguagem Java, Java SE 8 Edição*. Boston: Addison-Wesley. ISBN: 013390069X.
- [JMH] *Ferramentas de Código: JMH*. 2014. *Oracle*. <http://openjdk.java.net/projects/code-tools/jmh/>
- [JSON] *Apresentando JSON*. 2013. *Ecma Internacional*. <https://www.json.org>
- [Kahan91] Kahan, William e J. W. Thomas. 1991. *Ampliando uma Linguagem de Programação com Aritmética Complexa*. UCB/CSD-91-667, Universidade da Califórnia, Berkeley.

- [Knuth74] Knuth, Donald. 1974. Programação Estruturada com *goto* Statements. Em *Computação Pesquisas* 6: 261–301.
- [Lea14] Lea, Doug. 2014. Quando usar fluxos paralelos. <http://gee.cs.oswego.edu/dl/html/StreamParallelGuidance.html>
- [Lieberman86] Lieberman, Henry. 1986. Usando Objetos Prototípicos para Implementar Comportamento compartilhado em sistemas orientados a objetos. Em *Anais da Primeira Conferência da ACM sobre Sistemas, Linguagens e Aplicações de Programação Orientada a Objetos*, páginas 214–223, Portland, setembro de 1986. ACM Press.
- [Liskov87] Liskov, nascido em 1988. Abstração e Hierarquia de Dados. No *Adendo a os Anais da OOPSLA '87 e dos Avisos SIGPLAN, Vol. 23, N° 5: 17–34, maio de 1988*.
- [Naftalin07] Naftalin, Maurice e Philip Wadler. 2007. *Genéricos e Coleções de Java*. Sebastopol, CA: O'Reilly Media, Inc. ISBN: 0596527756.
- [Parnas72] Parnas, D. L. 1972. Sobre os critérios a serem usados na decomposição de sistemas em módulos. Em *Comunicações do ACM* 15: 1053–1058.
- [POSIX] 9945-1:1996 (ISO/IEC) [IEEE/ANSI Std. 1003.1 Edição 1995] *Tecnologia da Informação—Interface Portátil do Sistema Operacional (POSIX)—Parte 1: Aplicação do Sistema: Interface de Programa (API) CLanguage* (ANSI), IEEE Standards Press, ISBN: 1559375736.
- [Protobuf] *Buffers de protocolo. 2017. Google Inc.* <https://developers.google.com/protocol-buffers>
- [Schneider16] Schneider, Christian. 2016. SWAT (Serial Whitelist Application Trainer). <https://github.com/cschneider4711/SWAT/>

- [Seacord17] Seacord, Robert. 2017. Combatendo Vulnerabilidades de Desserialização em Java com Fluxos de Entrada de Objetos Antecipados (LAOIS). São Francisco: Whitepaper do NCC Group. https://www.nccgroup.trust/globalassets/our-research/us/whitepapers/2017/june/ncc_group_combating_java_deserialization_vulnerabilities_with_look-ahead_object_input_streams1.pdf
- [Serialização] Especificação de Serialização de Objetos em Java. Março de 2005. Sun Microsystems. <http://docs.oracle.com/javase/9/docs/specs/serialization/index.html>
- [Sestoft16] Sestoft, Peter. 2016. Java Precisely, 3ª ed. Cambridge, MA: TheMIT Press. ISBN: 0262529076.
- [Shipilëv16] Aleksey Shipilëv. 2016. Matrizes da Sabedoria dos Antigos. <https://shipilev.net/blog/2016/arrays-wisdom-ancients/>
- [Smith62] Smith, Robert. 1962. Algoritmo 116 Complexo Division. In Comunicações do ACM 5, nº 8 (agosto de 1962): 435.
- [Snyder86] Snyder, Alan. 1986. "Encapsulamento e Herança em Linguagens de Programação Orientadas a Objetos." Em Conferências de Sistemas, Linguagens e Aplicações de Programação Orientada a Objetos, 38–45. Nova York, NY: ACM Press.
- [Spring] *Estrutura de Primavera. Pivotal Software, Inc. 2017.* <https://projects.spring.io/spring-framework/>
- [Stroustrup] Stroustrup, Bjarne. [c. 2000]. "Java é a linguagem que você teria projetado se não precisasse ser compatível com C?" Perguntas frequentes do BjarneStroustrup. Atualizado em 1º de agosto de 2017. http://www.stroustrup.com/bs_faq.html#Java

- [Stroustrup95] Stroustrup, Bjarne. 1995. "Por que o C++ não é apenas orientado a objetos linguagem de programação." Em Adendo aos anais da 10ª conferência anual sobre sistemas, linguagens e aplicações de programação orientada a objetos, editada por Steven Craig Bilow e Patricia S. Bilow Nova York, NY: ACM.<http://dx.doi.org/10.1145/260094.260207>
- [Svoboda16] Svoboda, David. 2016. Explorando a Serialização em Java para Diversão e Lucro. Instituto de Engenharia de Software, Universidade Carnegie Mellon.<https://resources.sei.cmu.edu/library/asset-view.cfm?assetid=484347>
- [Thomas94] Thomas, Jim e Jerome T. Coonen. 1994. "Questões Relacionadas a Tipos Imaginários para C e C++." No The Journal of C Language Translation 5, nº 3 (março de 1994): 134–138.
- [ThreadStop] *Por que Thread.stop, Thread.suspend, Thread.resume e Runtime.runFinalizersOnExit estão obsoletos?* 1999. Sun Microsystems. <https://docs.oracle.com/javase/8/docs/technotes/guides/concurrency/threadPrimitiveDeprecation.html>
- [Viega01] Viega, John e Gary McGraw. 2001. Construindo Software Seguro: Como Evitar Problemas de Segurança da Maneira Correta. Boston: Addison-Wesley. ISBN: 020172152X.
- [W3C-validador] Serviço de Validação de Marcação do W3C. 2007. World Wide Web Consórcio. <http://validator.w3.org/>
- [Wulf72] Wulf, W. Um caso contra o GOTO. 1972. Nos Anais da 25ª Conferência Nacional da ACM 2: 791–797. Nova York, NY: ACM Press.

Símbolos

_ em letras numéricas, 108...
Argumento de aridade variável
Veja varargs < > *delimitadores de parâmetros de tipo*, 117, 123 <?> *tipo de wildcard ilimitado*, 120

A

Classes abstratas
Projetando para herança, 97 vs.
interfaces, 99–103 não
instanciabilidade e, 19 subclasse,
e equivale a, 45 mecanismos de
controle de acesso, 74 níveis de
acesso, 73–77
nível de módulo, 76–77 de
readResolve, 362 regras
práticas para, 74–75 de classes
membros estáticas, 112
modificadores de acesso, 74
métodos de acesso, 78
cópias defensivas e, 80, 233 para
dados de captura de falhas, 297,
307 imutabilidade e, 80
convensões de nomenclatura para,
291–292 vs. campos públicos, 78–
79 para paraDados de string, 57
parâmetros reais de tipo,
117 Padrão adaptador, 23, 101, 113
tipos agregados vs. strings,
276 métodos alienígenas,
317 anotações, 157, 180–192
definindo, 180 detectando repetidos e
não repetidos, 186 documentando, 259

Interfaces funcionais e, 202 vs.
Padrões de nomeação, 180–187
sintético, 186 *Veja também*
marcadores anotações classes
anônimas, 112, 114, 193

Em adaptadores, 101
preferem lambdas a, 193–
196 serialização e,
196 antipadões, 2

padrão enum de campo de bits, 169 tipos de
curinga limitados como tipos de retorno,
142 quebrando convensões de nomeação, 82
esperas ocupadas, 336 constante como
hashCode, interface 51 constante, construtor
107–108 copy do objeto imutável,
comparadores baseados em diferença 83, 71
blocos de captura vazios, 310 iguais com
recursos não confiáveis, 45 exceções para
controle de fluxo, 293 concatenação
excessiva de strings, 279 ponto flutuante
para cálculos monetários, 270 hardwiring
recursos, 20 hashCode com campos
significativos faltando, 54 classes imutáveis
não efetivamente final, 85 subclasses
inadequadas, padrão enum 92 int, padrões de
nomenclatura 157, retornos nulos 180 para
coleções, arrays, 247–248 abuso ordinal,
168, 171 sobrescrevendo iguais mas não
hashCode, 50 classes internas serializáveis,
345 assinas diferem por ordem de tipo de
parâmetro, padrão enum 6 String, uso
excessivo de 158 strings, 276 classes
etiquetadas, subclasses e iguais a
componentes de valor 109–111, 44

Design de APIs

níveis de acesso e, 74 tipos de curinga limitados e, 139–145
 callbacks, padrão de interface 28
 constante, e, 107
 exceptions e, 294, 296–297
 ocultação de informações e, 286
 inheritance e, 93–98
 interfaces como tipos de parâmetros, classes 170
 member e, 114
 performance e, 286–287
 serialização e, 343–345
 singletons, 18
 elementos API, 4

documenting, 254–260
 API, toString retornando valores como de fato, 57
 arrays

clone e, 65
 tipagem covariante, 126
 cópia defensiva de, 76, 234
 vs. nulo como valor de retorno, 247–248
 para implementar genéricos, 131–133
 vs. listas, 126–129
 mutabilidade e, 234, 248
 reificadas, 126
 questões de segurança, 76
 assertions, 229
 atomicidade

de variáveis, sincronização 311
 e, 312–314
 autoboxing, 24, 273–275

desempenho e, 275
 Interface AutoFecheable, 31–32, 35

B

Objetos de apoio, classes 23
 base, classe 281
 BigDecimal

compararA inconsistente com iguais, 68
 para cálculos monetários, 270
 de desempenho e, campos de 271
 bits vs. conjuntos enum, 169–170
 operações de bloqueio, ataques de fluxo de bytes 326
 bogus, tipos 354
 booleanos vs. enum, 237

parâmetros de tipo limitado, 134

Para tokens do tipo limitado, 154
 vs. Tipos de coringa limitados, 144
 tokens de tipo limitado, tipos de coringa 154, 172, 178, 183
 tipos de wildcard limitados, 136, 140

para flexibilidade de API, 139–145
 vs. parâmetros de tipo limitado, 144
 para tokens de tipo limitado, 154
 vs. objetos de classe, injeção de dependência 178
 e, mnemônico 21
 PECS para, tipos de retorno 141
 como devolução, 142
 vs. Tipos de coringa ilimitados, primitivas 121
 em caixas

= operador e, 274
 usos apropriados de, 275
 genéricos e, 134
 preferem tipos primitivos a, 24, 273–275
 Padrão Bridge, padrão 8
 Builder, 10–16

adaptado para invocação de método, 237
 busy waits, 336

C

Cache

evitando vazamentos de memória de, 28
 de objetos caros, 22–23
 de códigos hash, 53
 imutáveis de objetos e, 82, 85
 frameworks de callback, classes de wrapper e, 91
 callbacks, evitando vazamentos de memória de, 28
 formas canônicas, 47
 capacidades vs. strings, 276–277
 casts

Dinâmico, 153, 155
 invisível (veja casts gerados pelo compilador) desmarcados, avisos de, valores 127, 129, 137
 char e streams, 206
 exceções verificadas

Evitando o uso excessivo de, 298–299
 declarando, 304
 falhas atomicidade e, 308
 propósito de, 296
 refatorando para não controlado, 299
 vs. sem controle, 296–297

circularidades
 em limpadores, ataques de
 33inicialização, 333, 366serialização
 e, classe 360Class, como chave
 parametrizada, hierarquias de classe
 151class, 110
 Padrão construtor e, explosões
 combinatórias 14 em, literais de
 classe 100
 Como valores de parâmetros de
 anotação, 183as genéricos, 151raw
 tipos em, frameworks baseados em
 classes 121, 281classes, 73–114

níveis de acesso, 74Anônimo (ver
 Classes Anônimas)Base,
 281Composição, 87–92Desenhando
 para Herança, 93–98Documentando

para herança, segurança de threads 93–94,
 330–332genérica, 117helper, para encurtar
 listas de parâmetros, 237hierarquia de
 (veja hierarquias de classes)herança de
 implementação imutável (veja objetos
 imutáveis), 87–92 instâncias de, 3 níveis
 de segurança de threads, 330members, 3
 minimizando acessibilidade de, 73–
 77mutável e segurança de threads, 322
 convenções de nomenclatura para,
 companheiros aninhados (veja classes
 aninhadas) não instanciados,
 encaminhamento 7reutilizável, 89–
 91singletons (veja singletons) descrições
 resumidas de, 257SuppressWarnings
 anotação e, 124 tag, vs. hierarquias de
 classes, 109–111 instanciado não
 intencionalmente, 19não relacionado,
 243utility (veja classes utilitária) wrapper
 (veja classes de wrapper) Veja também
 nomes individuais de classes funções
 classificador, 213cleaners, 29–33

Método do clone, 58–65
 e, 65 como construtor, 61, 96 vs.
 Construtor de cópia ou conversão, 65
 cópias defensivas e, 76, 233campos
 finais e, 61 contrato geral, 58–59
 objetos imutáveis e, 83 classes não
 finais e, 233 métodos não finais e,
 64métodos sobrescritos e, 96-
 segurança e, 64Interface clonável, 58–
 65

Alternativas a, 65Behavior of,
 58Designing for Inheritance e, 64,
 96Implementing, 64Collections

Concurrent, 321, 325–326 vs. valor de
 retorno nulo, 247optionals e, 252
 como tipos de retorno, vs. fluxos,
 216–221collectors, 211–215

downstream, 213Collectors API,
 organização de, 211–215 explosões
 combinatórias, 100 classes companheiras

mutável, 84não instanciável,
 7Interface comparável, 66–72

como consumidores em PECS, 143
 limites de tipos recursivos e, 137 Veja
 também comparar Métodos de construção
 de metodocomparadores, 70,
 194comparadores, 69

como consumidores no método
 PECS, 143 compareTo, 66–68

*Veja também Compatibilidade
 de interface comparável*

para trás, 350binary, 107,
 305forward, 350migration,
 119source, 305unchecked
 exceptions e, 305compiler
 warnings, tipos de, 123

Casts gerados por compiladores, 117,
119, 127components, 2composition, 8,
89

é igual a e, 44 contra.

herança, 87–92 peso
conceitual,

7concorrência, 311

documentando o comportamento do método
para, 330–332melhorando via sincronização
interna, 322Concurrency utilities, 323–329

vs. esperar e notificar, 325–329 coleções
concorrentes, 321, 325–326 classes
condicionalmente seguras para threads,

Documentação, requisitos
de consistência 331

igual a 38, 45hashCode,
50consistente com igual a
68

recursos não confiáveis e, 45 campos
constantes, convenções de nomenclatura
para, 290 interfaces constantes, 107 classes
de utilidade constantes, 108 constantes, 76

em classes anônimas, 114 convenções de
nomenclatura para, 290 comportamentos
específicos de constantes, 162–166

lambdas para, 195 corpos de classe específicos
para constantes, 162 implementações de
métodos específicos para constantes

*Veja construtores de comportamentos
específicos de constantes, 4*

chamando métodos sobrescritos em,
95checking parâmetros of, 353clone
como a, 61copy e conversion,
65default, 19 cópia defensiva dos
parâmetros, 232desserialização
como, 344estabelecendo invariantes,
82, 86não instanciabilidade e,
19private (veja construtores
privados)readObject como a,
353reflection e, 282substituindo por
fábricas estáticas, 5–9
sobrecarregando com segurança,
240–241

para singletons, 17–18 descrições
resumidas de, 257SuppressWarnings
anotação e, 124 parâmetros de verificação
de validade de, 229contenção,
sincronização e, 321contratos

clone, 58compareTo, 66documentation as,
304equals, 38–46hashCode, 50toString, 55
objetos corrompidos, 12, 30, 227, 309 travas
regressivas, 326 arrays covariantes, 126 tipos
de retorno covariantes, 16, 60criação de
objetos, 5–33 representações de dados
estruturados multiplataforma,

341 formulários serializados
personalizados, 346–352

D

Consistência de dados
mantendo em caso de falha, 308–
309sincronização para, 311–
316corrupção de dados, 285, 312Data,
substituições para, 232deadlocks

Ordenação de recursos, 320,
351thread famvation, padrão
328Decorator, acesso 91
padrão

*Veja pacotes de acesso privado e
construtores leveldefault, implementações
19default, métodos 104default em
interfaces, formulários serializados 99,
104–105default, 346–352*

Desvantagens de, 348
cópias defensivas, 231–235
de matrizes, 234builders e,
14clone e,
233desserialização e, 353,
357documenting, 234
objetos imutáveis e, 83

de corpos internos mutáveis, 233
 de parâmetros mutáveis, 232–233
 vs. reutilização de objetos, 25
 desempenho e, 234readObject e,
 353, 357transferências de
 controle e, 235 verificação de
 validade e, 232delegação, 91
 ataques de negação de serviço,
 331 injeção de dependência, 20–
 21 campos derivados, 47,
 52desserialização

Como construtor, 344
 singletons e, 18 bombas de
 desserialização,
 340desserialização filtrando,
 342destruindo objetos, 26–
 33 mensagens detalhadas,
 operador 306diamond,
 123documentando

Tipos de anotação, 259compareTo, 67
 thread safety condicional, 331enum
 types, 258, 331exceptions, 227, 304–
 305exposed API elements, 254–
 260generics, 258hashCode, 54for
 inheritance, 93–94methods, 254–255
 exemplos de código multilinha, 255
 object state após exceções,
 309parameters, 227return value do
 toString, 56-self-use of overridable
 methods, 93, 98-selfuse patterns, 256
 campos serializados, 347
 implementações esqueletais, 103
 fábricas estáticas, 9SuppressWarnings
 anotação, 125thread safety, 330–
 332writeObject para serialização,
 350Veja também Javadoc

duplo
 para aritmética binária de ponto flutuante, 270
 quando evitar, 270–272

Duplo Verificação Idioma, 334–
 335 Coletores a Jusante, 213
 Fundições Dinâmicas, 153, 155

E
 objetos efetivamente imutáveis, arrays vazios
 316, vs. valor de retorno nulo, 247–
 248encapsulamento, 73, 286
 quebrado por herança, 87, 94
 quebrado por serialização,
 343of, 78 instâncias
 envoltantes, 112 tipos de
 enum, 157–192

Adicionando dados e comportamentos
 a, 159–161 vs. Campos de bits, 169–
 170vs. Boolean, 237 mecanismo de
 serialização incorporado, 362
 constantes específicas para, 162–166
 documentando, 258, 331extensibilidade
 e, 176–179imutabilidade de, 160
 controlado por instância, 158vs.
 constantes int, 157–167preferi
 readResolve, 359–362remove elements
 de, 161singletons de, 17–18strategy
 enum pattern, 166vs. Constantes de
 string, 158, 276 instruções switch e,
 167 como classes de nível superior ou
 membro, 161paraString e, 160type
 safety from, 158when to use, 167 tipos
 enumerados

*Veja tipos de enumEnumMap
 vs. ordinais, 171–175EnumSet
 vs. campos de bits, método
 169–170equals, 37*

sobrecarga acidental de, 49, 188
 formas canônicas e,
 47compareTo e, 68composição e,
 44 contrato geral para, 38–
 46hashCode e, 48, 50–54 como
 escrever, 46Sobrescrição
 anotação e, 188

Método de iguais (continua)
 retorno valores de, e compareTo, 68
 subclassing e, 42, 45
 recursos não confiáveis e, 45
 when to override, classes de equivalência 37–38,
 relações de equivalência 39,
 38
 erasure, 119, 126
 erros

criação de arrays genéricos, 126–127,
 133
 propósito de, 297
 runtime, métodos padrão e, 105
 resultados exatos, tipos para obtenção, cadeia de
 270
 exceptions, idioma de tradução de exceções 302–303, 230,
 302
 exceptions, 293–310
 métodos acessórios para, 297, 307
 verificados vs. não marcados, 296–
 297
 Escolhendo entre, 301
 frequentemente reutilizado, 301
 controle fluxo e, 294
 mensagens detalhadas para, 306–307
 documentando, 227, 304–305
 dados de falha-captura, 307
 para parâmetros de método inválidos, 228
 ignorando, 310
 logging de, 303
 multi-catch facility, 320
 vs. Opcionais ou valores especiais de retorno, 295
 Prefere Standard Existente, 300–301
 Preventing, 303
 Vs. métodos de teste de estado, 294
 supressão de, 36
 uncaught e finalizadores, 30
 using apropriadamente, 293–295
 Veja também nomes individuais de exceção
 Framework executor, serviço executor 323, 323–324
 argumentos de tipo explícitos, 142
 declarações de exportação, APIs 76
 exportadas

*Veja design de API;
 Classes API extending*

*Veja herança;
 subclasse Extensíveis de Interfaces,
 4 extensíveis enums, 176–179*

Mecanismos extralinguísticos
 Clonagem, 58, 65
 Métodos nativos, 285
 Reflection, 282
 serialização, 344,
 363
 Veja também construtores ocultos

F

Padrão do Método de Fábrica, 5, 21
 de falha atômica, 230, 308–309
 campos
 níveis de acesso de, 73–77
 invariantes de classe e, 75
 constante, convenções de nomenclatura para, 290
 derivado, 47, 52
 exposto, vs. métodos de acessório, 78–79
 finais (ver campos finais)
 técnicas de inicialização para, 335
 mutáveis, cópias defensivas de, 233
 convenções de nomenclatura para, 290,
 292
 final estático público, para singletons, 17
 reflection e, 282
 descrições resumidas de, 257
 tags, segurança de 109
 thread e, 75
 campos finais

para definir constantes, 290
 incompatível com clonagem, 61
 incompatível com serialização, ataques 357
 finalizers e prevenção, 30–31
 finalizers, 29–33

alternativa a, 31

Flutuador

para aritmética binária de ponto flutuante, 270
 when to avoid, 270–272
 APIs fluentes, 14, 203
 Flyweight pattern, 66
 footprint

*Veja consumo de espaço
 para ciclos*

Idioma de variáveis duplas, 263
 prefere loops for-each para, 264–266
 vs. loops while, loops 262
 para-cada

limitações de, 266
 preferem em vez de loops for, 264–266

Tarefas e pools de fork-join,
parâmetros de tipo 324 formais,
métodos 117forwarding, 89,
102frameworks

callback, baseado em
classe 91, executor
281executor, baseado em
interface 323, tipo não
hierárquico, provedor de
serviços 99, objetos de
função 8, 114
vs. blocos de código,
207interfaces funcionais, 193
sobrecarga de métodos e,
243Organização do Padrão, 200–
201Usando o Padrão, 199–
202Programação Funcional, 82

G

Gadgets, 340 Coleta de Lixo, 27,
29–30, 113 Contratos Gerais

*Veja contratos genéricos erros de criação
de arrays, 126–127, 133 classes e
interfaces genéricas, 117 métodos
genéricos, 135–138*

vs. tipos de currying ilimitados, 121
fábricas genéricas singleton, 18,
136 parâmetros genéricos de tipo

*Veja parâmetros de tipo tipos
genéricos, 14, 117, 130–134*

Documentação, 258imutabilidade
e, 136genéricos arrays de
parâmetros Varargs

Poliuição de pilha de, 147–148
substituindo por listas, 149
inseguro como armazenamento,
146 inseguro para exposição,
147–148genéricos, 117–155

primitivas em caixa, 134 casts gerados
por compiladores e, 117erasure e,
126implementando sobre arrays, 131–
133incompatibilidade com tipos
primitivos, 134 tipos invariantes,
126varargs e, 127, 146–150

Generificando o código existente,
130Princípio Get and Put, 141

H

Método hashCode
iguais e, 48, 50–54 contrato
geral para, 50como escrever,
51 objetos imutáveis e, 53
poluição de pilha, 133, 146–
148 perfiladores de heap, 28
classes auxiliares, 112

para encurtar listas de parâmetros, 237hidden
constructors, 61, 96, 339, 344, 353

*Veja também mecanismos extralinguísticos
padrão construtor hierárquico, 16*

I

Objetos imutáveis
formas canônicas e, 47clone e, 59
injeção de dependência e, 21 arrays
vazios e, 248enum types e,
160EnumSets e, 170 falhas de
atomicidade e, 308 abordagem
funcional e, 82hashCode e,
53JavaBeans e, 12 classes
companheiras mutáveis para, 84
reutilização de objetos e, 22 regras
para, 80serialização e, 85, 353–358
métodos de fábrica estáticos e,
84subclassing e, segurança de
97thread e, 82 programação
imperativa, 82 detalhes de
implementação

documentando para herança,
94exposto, 92implementação
herança, 87implementando
interfaces, 4

métodos padrão e, 105
inconsistente com iguais, 67–68

Recursos pouco confiáveis e,
45Informação Oculta

Veja encapsulamento

herança, 3
 vs. composição, 87–92
 constructors e, 95
 designing para, 93–98 documentando para, 93–94
 encapsulação e, 87
 fragilidade de, 89
 hooks para facilitar, 94
 implementação vs. interface, 3, 87 de comentários do documento de método, 259
 múltiplos, simulados, 102–autouso de métodos sobrescritos e, 98
 usos de, 92
 Veja também subclassing
 initialization

circularidades, 333, 366
 cópia defensiva e, 80
 of campos sobre desserialização, 351
 incompleto, 96
 preguiçoso (veja inicialização preguiçosa) de variáveis locais, 261
 normal vs. preguiçoso, 333
 na criação de objetos, 86
 classes internas, 112

Serializável e, 345 para estender implementações esqueléticas, campos de 102 instâncias

níveis de acesso, 75
 inicializando, 333
 inicialização preguiçosa de, 334
 vs. ordinais, 168
 classes controladas por instância, 6.158

singletons, métodos de fábrica estáticos 17–18 e, classes de utilidade, 19
 Veja também tipos enum
 Instanceof Operator, Tipos de Parâmetros, Constantes 121
 Int vs. tipos Enum, 157–167
 Int, para cálculos monetários, Frameworks baseados em 270
 interface, 6, Herança 99–103
 interface, 87
 interfaces, 73–114

vs. classes abstratas, níveis de acesso 99–103, 74
 acessibilidade dos membros estáticos, 7
 métodos padrão ativados, 99, 104–105

para definir tipos, 107–108, 191–192
 Design de, 104–106
 emulação de enums extensíveis com, 176–179
 possibilidade de melhorias de funcionalidade, 100
 genérico, 117
 marcadores (veja interfaces de marcadores)
 mixin, 58, 99
 convenções de nomenclatura para, 289–291
 para frameworks de tipos não hierárquicos, 99
 classes companheiras não instanciáveis e, 7
 as tipos de parâmetros não instanciados, 170, 237
 preferem a reflexão, 282
 propósito de, 58, 107–108
 para referência a objetos, 280–281
 instanciação refletiva de, 283–284
 serialização e, 344
 implementações esqueléticas e, 100–103
 métodos estáticos e, 7
 descrições resumidas de, 257
 Veja também nomes de interfaces individuais
 ataques internos de roubo de campo, 360–362
 tipos invariantes, 126, 139
 invariantes

clone e, 61
 concorrência e, 328
 constructors e, 82, 86
 corruption de, 92
 enum tipos e, 362
 mantendo, 229, 234, 308
 de objetos e membros, 75, 78

J

JavaBeans
 imutabilidade e, convenções de nomeação de 12 métodos, 291
 pattern, 11–12
 Javadoc, 254

documentos de arquitetura e, 260
 comentários em nível de classe, 228, 331
 índices do lado do cliente, 258
 comentários de herança, 259
 formatação, 255–256
 comentários em nível de módulo e pacotes, 259
 descrições resumidas, 257

K

Funções do extrator de chaves, 70

L

lambdas, 70, 193–225
 limpadores e, 33 para comportamentos
 específicos constantes, 195 prefere
 referências ao método, 197–198 prefere
 classes anônimas, 193–196 serialização e,
 196 inicialização preguiçosa, 23, 53, 85, 333–
 335 idioma preguiçoso de titular de
 inicialização, 334–335 inicialização
 preguiçosa com um sistema sincronizado
 idioma acessório, abstrações 333–
 334leaky, 146libraries, 267–269Princípio
 de substituição de Liskov, 43, 75listeners,
 evitando vazamentos de memória de,
 281istas

vs. arrays, 126–129 para arrays genéricos
 de parâmetros Varargs, 149
 comparabilidade mútua em, 138
 vivacidade

garantindo 317–322, 328
 falhas de, 224, 313
 classes locais, 112, 114
 variáveis locais

Minimizando o escopo de, 261–
 263 convenções de nomenclatura
 para, 290, 292localidade de
 referência, 223locks

campos contendo 332finalizadores
 ou limpadores e, 30privado,
 332reentrante, 320 igualdade lógica,
 37–38longo, para cálculos
 monetários, 270loops

aninhados, 264–266Veja também loops; Loops for-
 each

M

Mapas

Classes membros e, 113
 aninhadas, 173–175 vs.
 Streams, comportamento de,
 173 marcadores, 181

vs. interfaces de marcadores, interfaces
 de 191 marcadores, 191–192

Turmas de membros, 112–114
*Veja também membros membros
 estáticos das classes, 3*
 minimizando acessibilidade de 73–
 77 de memória
*Veja vazamentos de memória
 por consumo de espaço, 26–27*
 classes membros não estáticas e,
 113-autogerenciamento de
 memória, modelo 28memory,
 80merge functions, 212meta-
 anotações, 181method chaining,
 14method overloading, 238–244

acidentais, de iguais, 49efeitos do autoboxing
 e genéricos, 241–242interfaces funcionais e,
 202, 243parâmetros e, 240 seleção estática
 entre métodos, 238 sobreposição de métodos,
 49, 238–239

níveis de acesso e, 75clone, 58 seleção
 dinâmica entre métodos, 238equals, 37–
 49hashCode, 50–54selfuse e,
 98toString, 55–57unintentional,
 190methods references, 18

tipos de, 198 preferem
 lambdas, 197–198métodos,
 3, 227–260

níveis de acesso de, 74Acessor (veja
 métodos de acessor) alien,
 317Comum a todos os objetos, 37–
 72Especifico constante, para tipos
 enum, 162Documenting, 254–255

exceções lançadas por, 304–
 305overridable, 93 descrições
 resumidas de, 257thread safety of,
 330–332failure atomicity e, 308–
 309forwarding (ver métodos de
 encaminhamento)genérico, 121,
 135–138invocation, reflection e,
 282legal for SafeVarargs, 149

Métodos (continuação)

Minimizando acessibilidade de, 73
 convenções de nomenclatura para, 9, 290–291
 nativo, 31, 285
 não final e clone, 64
 sobrecarga (veja sobrecarga do método)
 overwriting (ver método overriding) listas de parâmetros para, 236
 private, para capturar tipos de wildcard, 145
 shortening parâmetros listas de, 236
 assinaturas de, 3, 236–237
 tamanho de, 263
 state-testing, vs. valor especial de retorno, 295
 estática (veja métodos de fábrica estática)
 SuppressWarnings anotação e, 124
 parâmetros de verificação de validade, 227–230
 varargs, 245–246
 Veja também nomes individuais dos métodos: interfaces
 mixin, 58, 99
 mixin mixing primitivos, primitivas em caixa, 24, 274
 módulos, 76–77
 cálculos monetários, tipos para, 270–271
 referência Monty Python, sutil, facilidade
 247–multi-catch, 320
 multiple inheritance, simulado, 102
 mutabilidade

Padrão JavaBeans e, 11
 minimizando, 80–86
 classes companheiras mutáveis, 84
 reduções mutáveis, 223
 mutadores, 78
 comparabilidade mútua, 138
 exclusão mútua, 311

N

parâmetros opcionais nomeados, 14
 convenções de nomenclatura, 236, 289–292
 de parâmetros genéricos de tipo, 131
 gramáticas, 291–292
 de classes de implementação esquelética, 101
 de métodos de fábrica estática, 9
 fluxos e, 208
 de parâmetros de tipo, 135
 padrões de nomeação vs. anotações, 180–187
 métodos nativos, 31, 285
 pares nativos, 31
 ordenação natural, 66

Classes aninhadas, 112
 níveis de acesso, 74

Acessibilidade decrescente com, padrão proxy de serialização de 74
 polegadas, 363
 tipos de, 112
 interfaces aninhadas, níveis de acesso de, 74
 mapas aninhados, 173–175
 frameworks de tipos não hierárquicos, 99
 classes não instanciáveis, 19
 classes companheiras não instanciáveis, 7
 não-nulidade de iguais, 38, 45
 tipos não reificáveis, 127, 131, 146
 classes membros não estáticas, 112–114
 notify vs. notifyAll, 328–329
 verificação nula, 228
 anulando referências a objetos obsoletos, 27
 NullPointerException, igual a contrato

e, 46

O

pools de objetos, campos de referência de 24
 objetos, igual a e, 47
 objetos, 3

evitando o acesso refletivo, 282–284
 classes base e, 281
 criando e destruindo, 5–33
 criação e desempenho, 6, 22–23
 desserializando filtragem de, 342
 efetivamente imutável, 316
 eliminando referências obsoletas a, 26–28, 60, 308
 despesas de criação, 24
 favorece referência a por interfaces, 280–281
 funções, 114
 imutáveis (veja objetos imutáveis) em estados inconsistentes, 11–12, 96, 309

(veja também objetos corrompidos)
 métodos comuns a todos, 37–72
 anulando referências obsoletas a, 27
 processo, 114
 reutilização, 22–25
 publicação segura de, 316
 representações em string de, 55–57
 quando consultar por classe, padrão 281
 Observer, 317
 referências a objetos obsoletos, 26–28, 60, 308
 chamadas abertas, 321

Otimizações, 286–288
 códigos hash em cache, inicialização
 preguiçosa 533, 333–335 notify em
 vez de notifyAll, reutilização de
 objetos 329, ordem de comparações
 22–25 em iguais, 47 paralelizando
 fluxos, inicialização estática 224,
 23 StringBuffer e, 279 using == em
 iguais, 46 optionals, 249

Exceções e, 295, 298
 Valores de retorno, 249–
 253 ordinais

vs. mapas enum, 171–
 175 vs. Campos de Instância,
 168 Sobre carregando

*Veja sobrecarga do
 método Anotações sobrepostas, 49,
 188–190 Sobreposição*

Veja sobreposição de método

P

Nível de acesso privado-pacote, 4, 74, 84
 pacotes, convenções de nomenclatura para,
 289–290 paralelizando fluxos, 222–225
 listas de parâmetros

de construtores, 14
 shortening, 236–237 varargs
 e, 245–246 tipos
 parametrizados, 117–122

reficável, 127 construtores
 sem parâmetros, 19
 parâmetros

cópias defensivas do Mutable,
 verificação de validade do tipo 232 (veja
 parâmetros do tipo), 227–230, 353–355
 Mnemônico PECS, desempenho 141,
 286–288

autoboxing e, 24, 201,
 275 BigDecimal e, 271 builder
 padrão, 16 limpadores, 29–30
 cópia defensiva e, 234 de enums,
 167, 170 de igual, 46–47 de
 sincronização excessiva, 321

finalizadores, 29–30 para cada loops e,
 264 de hashCode, 50, 53 classes
 imutáveis e, 83–85 bibliotecas e, 268
 medições, 287 vazamentos de memória e,
 27 métodos nativos e, 285 criação de
 objetos e, 6, 22–23 de reflexão, 282
 paralelizando fluxos e, 222–225 de
 serialização, arquitetura de software 348–
 350 e, 286–287 testes de estado vs. valor
 de retorno especial, 295 fábricas estáticas
 e, 6 de concatenação de strings, 279
 para string e, 57 varargs e, 246 classes
 wrapper e, 91 *Veja também*
 otimizações modelo de desempenho, 288
 portabilidade

limpadores e, 29
 finalizadores e, 29
 métodos nativos e, 285
 prioridades de thread e,
 337 thread scheduler e,
 336 predicados, 104
 campos primitivos
 compareTo e, 69 é
 igual a e, 47 tipos
 primitivos, 273
 incompatibilidade com tipos genéricos,
 134 optionals e, 253 prefere a primitivas em
 caixa, 24, 273–275 *Veja também* tipos
 primitivos individuais nível de acesso
 privado, 74 construtores privados, 84

para não instanciabilidade, 19
 para singletons, idioma de
 objetos de bloqueio privado 17–
 18, 332 objetos de bloqueio
 privado, programação procedural
 332, objetos de processo 82, 114
 filas produtor-consumidor, 326
 princípios de programação, 2
 promptness of finalization, 29

Nível de acesso protegido, 74–75 nível de acesso público, 74 campos públicos vs. métodos de acessório, 78–79 fechaduras de acesso público, 331

Q

qualificado este construto, 112

R

idioma de verificação única e agressiva, verificação de 335 intervalos, tipos 229raw, método 117–122readObject, 353–358

Cópias defensivas e, 80como escrever, 358incompatível com controlado por instâncias objetos, métodos sobreescrevíveis 359 e, 96, método 358readResolve

níveis de acesso, 97 escolhendo níveis de acesso de, 362 preferência tipos ENUM para, 359–362Usando para classes controladas por instâncias, 359recipes

Adição de comportamentos ao enum individual constantes, 162adicionando dados aos tipos de enum, padrão 160builder, 12verificando campos significativos em equals, 47clone, 64compareTo, 68eliminando autouso, 98 igual, 46gerando uma classe, 130–133hashCode, 51implementando genéricos em cima de arrays, 131–133 cadeia de métodos, 14 classes não instanciáveis, 19readObject, proxies de 358serialização, singletons 363–364 singletons serializados, 18singletons como enums de elemento único, 18singletons com construtores privados, 17 implementações esqueléticas, classes 102 marcadas para classe Hierarquias, 110–111Veja também regras

limites de tipo recursivo, 137 parâmetros de tipo recursivo, estratégia de 14redução, 211 reduções, 223travas reentrantes, 320tipos de referência, 3, 273reflexão, 282–284

ObjectoAcessível.setAcessível ataques, 17 clone e, 58 desvantagens de, 282 instância de interface refletiva, 283usos para, 282, 284 requisitos de reflexividade

compareTo, 68 iguais, tipos 38–39reificados, 126 fábricas de recursos, 21 impasses de ordenação de recursos, 351 recursos

travado, e finalizadores, 30releasing, 31restricted marker interfaces, 191return classes, variados

padrão proxy de serialização e, métodos de fábrica 365static e, instruções 7–8return, SuppressWarnings

Anotação e, 124Retorno Tipos

Tipos de coringa limitados como, 142collections vs. streams, métodos estáticos de fábrica 216–221 e, 6 classes de encaminhamento reutilizáveis, regras 89–91

acessibilidade, 74–75adequação de exceções verificadas, 298Escolha de tipos de curinga limitados, 141Escolha de tipos de exceção, 296–297Diminuição de perigos de serialização, 341–342 para objetos imutáveis, 80 mapeamento de domínios para nomes de pacotes, 289 marcadores de interfaces vs. anotações, 192otimização, 286 para desempenho de fluxos paralelos, 223substituindo parâmetros de tipo por curingas,

para anotações do SafeVarargs,
 acessibilidade estática de membros 149,
 112 comentários de documentos de
 escrita, exceções de 254–260 runtime

Veja exceções não marcadas

S

Acessos a arrays seguros,
 76Línguas SAFE,
 231Publicação Segura,
 316Falhas de Segurança, 315
 Fluxos paralelos e, 224wait e,
 328SafeVarargs anotações, 147

Usos legais de,
 149escopo
 variáveis locais, 261–
 263SuppressWarnings anotações, 124of
 variáveis, referências obsoletas e,
 27security, cópia defensiva para, 25, 231
 questões de segurança

Arrays acessíveis de comprimento não zero, 76
 ObjectoAcessível.setAcessível
 ataques, ataques de negação de serviço,
 bombas de desserialização 331, ataques
 ElvisStealer 339–340, ataques 362
 finalizer, 30–31gadgets, ataques de roubo
 interno em campo, ataques ransomware
 355–357, 339reflection, execução remota
 de código, ataques de referência de objeto
 340 rogue, 355–357serialização, ataques
 339, 344, 353, 353, ataques 360stealer,
 362strings como chaves para conceder
 acesso a dados, subclassing 277 e, 89-time-
 of-check/time-of-use (TOCTOU)

ataques, problema
 233SELF, 91-
 autouso

documentando, para herança,
 93eliminando, para herança, 98
 versões seriadas UIDs, 343, 351–352

Serializável, 343–
 345serialização, 339–366
 classes anônimas e, 196 custos de, 343
 diminuindo os perigos de, 341–342
 projetando para herança e, 96–97
 documentando para, 347, 350 efeito em APIs
 exportadas, 343 classes de retorno flexíveis
 para, 365imutabilidade e, 85, 353 ataques
 internos de roubo de campo e, 360–
 362lambdas e, 196 filtragem de
 desserialização de objetos, 342 prefere
 alternativas a, 339–342 singletons e, 18
 sincronização para, 351 campos transientes
 para, 348 verificação de validade, 357quando
 usar, Padrão de proxy de 345serialização,
 formulários serializados 363–3666, como
 parte de APIs exportadas, instâncias 343
 serializadas vs. proxy de serialização

Pattern, 363–3666 frameworks de
 provedores de serviços, 8
 operações de curto-circuito, 223
 assinaturas de métodos, 3, 236–
 237 função signum, 67
 implementações simples, 103
 herança múltipla simulada, 102
 idiom self-type simulado, idioma
 14-check-check, 335singletons,
 17–18

vs. injeção de dependências,
 implementações esqueléticas 20,
 arquivos fonte 100–101, consumo
 de espaço 115–116

tipos ENUM, 175 objetos imutáveis e,
 83 vazamentos de memória e, 27
 classes membros não estáticas e,
 113spliterator, 223 despertares
 espúrios, 329 operações de
 modificação dependentes de estado,
 325 métodos de teste de estado, 294–
 295, 299

Métodos de fábrica estática, 5
 vantagens de, 5–8 classes anônimas dentro, 114 em documentação de API, 8 vs. clonagem, 65 cópias e fábricas de conversão, 65 flexibilidade em classes retornadas, 7–8 para singletons genéricos, 18, 136 objetos imutáveis e, 22, 82, 84 classes controladas por instância e, 6 limitações de, 8–9 convenções de nomenclatura para, 9, 292 substituindo construtores por, 5–9, 22, 240 retorno tipos de, 6–8 para frameworks de provedores de serviço, 8 para singletons, 17 subclasses e, 8 campos estáticos

para definição de constantes, 290
 inicialização preguiçosa de, 334
 sincronização do mutável, facilidade de importação 322
 estática, 108 importações estáticas, 70 classes membros estáticas, 112
 Limpadores e, 33 usos comuns de, 112–113 para tipos de enum, 161 vs. não estático, 112, 114 para representar agregados, 276 para encurtar listas de parâmetros, 237 membros estáticos, acessibilidade em interfaces, 7 pools de armazenamento, padrão enum 28 estratégia, padrão 166 estratégia, pipelines 193 stream, 203

Efeitos colaterais gratuitos, identificadores únicos de fluxo 210–215

Veja versão serial

UIDs streams, 193, 203–225

valores de carcaça e, 206 coletores para, 211–215 para programação funcional, 210–215 vs. Mapas, comportamento de, 173 paralelização, 222–225 preservando a ordem do paralelo, 224 as tipos de retorno, vs. coleções, 216–221

Especificando colecionadores para, 173, 214 forças de, 207 vs. threads, 323–324 using, 203–209 *Veja também* coletores Constantes de string vs. tipos de enum, representações de 158 strings, 55–57, 306 strings

Concatenando, 279 as substituições para outros tipos, 276–278 subclassificando, 3, 87

classes abstratas, e equivale a 45 níveis de acesso e, 75 adequação de, 92 Clonável e, 96 compareTo e, 68 igual e, 40, 42 finalizar ataca e, 31 fragilidade, 89 corrupção invariante e, 92 níveis de acesso ao método e, 75 proibindo, 8, 18–19, 85, 97 serialização e, 344 implementações esqueléticas, métodos de fábrica 102 estáticas e, 8 as teste de projeto para herança, 95 *Veja também* relações de subtipo de herança, 134, 140 descrições resumidas em Javadoc, 257 supertipos relacionamentos, 141 SuppressWarnings anotação, instruções 123–125 switch e tipos de enum, requisitos de simetria 164, 167

compareTo, 68

igual, 38–

39 sincronização

de dados atômicos, 312–314 excessivos, 317–322

e desempenho, 321 ramificações de, 317 interna, 322 para exclusão mútua, 311 serialização e, 351 para dados mutáveis compartilhados, 311–316 técnicas para, 314–316 para comunicação por threads, 312–314

Regiões sincronizadas
 métodos alienígenas e, 317–
 321 trabalho minimizando
 em, 321 sincronizadores, 326
 anotações sintéticas, 186

T

Campos de tags, finalização de atraso 109,
 29tarefas, 324vs. threads, 323–324Padrões
 de construtores telescopados, 10–11Padrão
 de método template, 101, 199this, nos
 comentários do documento, 256this, em
 classes lambdas vs. anônimas, pools de
 threads 196, 323

Dimensionamento de,
 prioridades de 336 roscas,
 segurança de 337 roscas
 documentando, 322, 330–332 objetos
 imutáveis e, 82níveis de, 330–
 331mutabilidade e, 75, 322
 escalonadores de threads,
 imtravamentos de 336–337 thread em
 inanição, método 328Thread.yield,
 evitando, 337threads, esperando
 ocupado, 336throwables, tipos de,
 ataques 296time-of-check/time-of-
 use, ataques 233TOCTOU, método
 233toString, 55–57

tipos de enum e,
 160contrato geral para,
 55quando substituir, 57
 campos transitórios, 348–
 351 com readResolve, 360

Quando usar, requisitos
 de 351transitividade
 compareTo, 68
 igual, 38, 40–45
 tentativa-finalmente

Prefiro tentar com recursos a, 34
 tentar-com-recursos
 prefiro tentar-finalmente, limites
 34–36tipo, recursivo, inferência
 de 137tipo, 70, 123, 142, 194

listas de parâmetros de tipo, 135
 parâmetros de tipo, 117, 135
 limitado, 134, 154 convenções de
 nomenclatura para, 135, 290
 recursivamente encadernado, 14,
 137 vs. Coringas, safety tipo 144

Moldes dinâmicos e, 154 de
 tipos de enum, 158 tipos de
 poluição de pilha e, 146 tipos
 parametrizados e, 119 tipos
 brutos e, 119 tokens de tipo,
 151tipos

Conversão de, 65, 291generic, 117, 130–
 134interfaces para definição, 107–108,
 191–192não-recopiável, 127,
 131parametrizado, 117–122primitivo,
 273raw, 117reference, 273Veja também
 tipos de curinga limitados; ilimitado

tipos curingaTipoPadrão Heterogêneo
 Seguro de Container,
 151–155incompatibilidade com tipos não reutilizáveis,
 154

U

Parâmetros de tipo ilimitados
 vs. tipos de wildcard limitados, 144
 tipos de wildcard ilimitados, 120
 vs. tipos de coringa limitados, 121
 aninhados, 152 vs. Tipos brutos, 121
 reificáveis, 127vs. parâmetros de tipo
 ilimitados, 144 exceções não
 verificadas

vs. exceções verificadas, 296–
 297compatibilidade e, 305excluindo das
 declarações de métodos, 304propósito de,
 296 avisos não verificados, 123–125

de moldes, 127, 129, 137 sublinhados,
 em literais numéricos, 108 retenções
 de objetos não intencionais

Veja vazamentos de memória

classes instanciadas
involuntariamente, 19
users de APIs, 4
classes utilitárias, 19
vs. interfaces constantes, 108
vs. Injeção de dependência, 20

V

Verificação de validade
construtores e, 14
parâmetros construtores,
229
cópias defensivas e, 232
de objetos desserializados,
355
implícitos, 230
parâmetros, 227–230
atomicidade de falha e, parâmetros
308
readObject, classes de valor
353–355, 38
toString e, 56
tipos de valor vs. strings,
276
varargs, 245–246
builders e, 16
with genéricos, 146–150
genéricos, e avisos de compilador,
127
de performance, métodos de
aridade de 246
variáveis, classes de
retorno de 245
variáveis

padrão proxy de serialização e,
métodos de fábrica 365
estáticos e, 7–8
variáveis

Operações atômicas em, 311
local (veja variáveis locais)

Convenções de nomenclatura para,
290
escopo de, e referências obsoletas,
27
para evitar subclasses, 44, 68
para manter invariantes, 234
convenções de nomenclatura para, 291
reutilização de objetos e, modificador
23
volátil, 314–315

W

Idiom de loop de espera,
328–329
avisos, desmarcados
Veja alertas desmarcados
referências fracas, loops
28
while vs. loops for, tipos de
curingas 262

capturando, 145
vs. parâmetros de tipo, 144
Veja também tipos de curinga limitados;
ilimitado

Tipos de coringa
Janela de Vulnerabilidade, Filas
233
Work, idioma de classe
326
wrapper, classes
100
Wrapper, 89–91

Cópia defensiva e, 235
incompatível com frameworks de retorno de chamada, 91
vs. subclasse, método 97
writeRe, níveis de acesso de, 97

Z

Matrizes de comprimento zero, imutabilidade de, 248